ASH

The Ashley Project
ASH Programming Language

Programming Guide and Reference Manual

Language, compiler (ashc), ASHIR, shasm assembler and the nlib library

October 2026
ASH-0001-EN

Copyright © 2026 Mallionic Inc. All rights reserved.
Document Number: ASH-0001-EN

This page intentionally left blank.

Copyright © 2026 Mallionic Inc. All rights reserved.

Notices and Disclaimers

This document is part of **The Ashley Project**. It describes the ASH programming language, the `ashc` compiler (version 1.0.0), the ASHIR intermediate representation, the `shasm` assembler (version 1.2.0) and the `nlib` system library that ships embedded in the compiler.

The Ashley Project is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 3, or (at your option) any later version. The Ashley Project is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

Every program listing in this manual that is presented as an *example* was compiled with `ashc` 1.0.0 while this document was being produced. Program output, exit statuses, ASHIR listings and disassembly listings shown next to an example are the real results of that build; they were not typed by hand. Results may differ with other compiler versions, other C libraries, other linkers or other operating systems.

The information in this manual is subject to change without notice. Code generation details (register choices, instruction order, stack layout, helper routine names) are described to help you understand and debug your programs; they are not a stable interface unless a section says so explicitly.

Software and workloads used in examples may have been optimized for the x86-64 Linux environment in which this document was built. Where a feature depends on the target (16-bit flat binaries, 32-bit ELF objects, Windows COFF objects, programs without the C library) the dependency is stated in the text.

x86, x86-64, Intel 64 and IA-32 are names used to identify processor architectures; Linux is a registered trademark of Linus Torvalds; Windows is a trademark of Microsoft Corporation; Java and GraalVM are trademarks of Oracle and/or its affiliates; other names and brands may be claimed as the property of others.

Copyright © 2026 Mallionic Inc. All rights reserved.

Revision History

Revision	Date	Description
1.0	October 2026	Initial release. Covers ashc 1.0.0, shasm 1.2.0 and the nlib libraries args, delay, math, null, stdbool, stdio, stdlib, string, syscalls and types.

Contents

1 Introduction to ASH 28
1.1 What ASH Is 28
1.2 A First Look 29
1.3 Why Use ASH 30
1.3.1 Close to the machine without leaving the language 30
1.3.2 One toolchain for every x86 mode 30
1.3.3 Readable output 30
1.3.4 Safety by default, control when needed 31
1.3.5 Works with the code you already have 31
1.4 Why the Compiler Is Written in Java 31
1.4.1 Portability of the compiler itself 31
1.4.2 A native executable without a runtime dependency 31
1.4.3 Correctness and safety of the tool 31
1.4.4 A rich standard library 31
1.4.5 Fast iteration 32
1.5 What ASH Can Do 32
1.6 General Purpose and Design Goals 33
1.7 Who Should Read This Manual 33
1.8 How This Manual Is Organized 34
1.9 Notational Conventions 34
1.9.1 Typography 34
1.9.2 Code listings 34
1.9.3 Numbers and sizes 35
1.9.4 Syntax notation 35
1.10 Related Software and Documents 35

2 Programming Basics 36
2.1 What a Computer Does 36
2.2 Bits, Bytes and Numbers 36
2.2.1 Bits and bytes 36
2.2.2 Binary and hexadecimal 37
2.2.3 Negative numbers 38
2.2.4 Text is numbers too 39
2.3 From Source Code to a Running Program 41
2.4 The Parts of an ASH Program 42

Copyright © 2026 Mallionic Inc. All rights reserved.

2.5 Words You Will Meet . . . 44
2.6 Two Kinds of Mistakes . . . 44
2.7 Exercises . . . 45
2.7.1 Exercise 1: multiplication table . . . 45
2.7.2 Exercise 2: even or odd . . . 46
2.7.3 Exercise 3: largest value . . . 46
2.7.4 Exercise 4: counting letters . . . 47
2.7.5 Exercise 5: temperature conversion . . . 47

3 Getting Started . . . 48
3.1 Requirements . . . 48
3.1.1 Running the compiler . . . 48
3.1.2 Linking and running programs . . . 48
3.1.3 Building the compiler from source . . . 49
3.2 Building and Installing . . . 49
3.3 Verifying the Installation . . . 50
3.4 Your First Program . . . 50
3.4.1 Step 1: compile to an object file . . . 51
3.4.2 Step 2: link and run . . . 51
3.4.3 Looking inside: ASHIR and machine code . . . 51
3.5 Entry Points and Linking Recipes . . . 52
3.6 A Program That Uses Its Exit Status . . . 53
3.7 A Program Without the C Library . . . 54
3.8 A 32-bit Program . . . 56
3.9 A 16-bit Flat Binary . . . 57
3.10 A Windows Program . . . 58
3.11 Files Created by the Compiler . . . 60
3.12 Editor Support . . . 60
3.13 Where to Go Next . . . 60

4 A Tour of ASH . . . 61
4.1 Values and Variables . . . 61
4.2 Making Decisions . . . 63
4.3 Repeating Things . . . 64
4.4 Functions . . . 65
4.5 Arrays . . . 66
4.6 Text . . . 66
4.7 Grouping Data with Structures . . . 67

Copyright © 2026 Mallionic Inc. All rights reserved.

4.8 Organizing Code with Classes . . . 69
4.9 Using Libraries . . . 70
4.10 Going Low Level . . . 70
4.11 Where to Go from Here . . . 72

5 The ashc Command Line . . . 73
5.1 Synopsis . . . 73
5.2 Output Modes . . . 74
5.3 Options Reference . . . 75
5.3.1 Output location: -o, --outdir . . . 75
5.3.2 --stdout . . . 75
5.3.3 --bits *n* . . . 76
5.3.4 --no-libc . . . 76
5.3.5 -e, --epilogue . . . 76
5.3.6 -s, --silent and --msg . . . 77
5.3.7 --tokens . . . 78
5.3.8 --debug . . . 78
5.3.9 --trace . . . 78
5.3.10 --list-nlib . . . 78
5.3.11 -v, --version; --license; -h, --help . . . 79
5.4 Output File Names . . . 79
5.5 Diagnostics . . . 79
5.6 Exit Status of ashc . . . 80
5.7 Using ashc from Build Systems . . . 81

6 Architecture of the Compiler . . . 82
6.1 Overview . . . 82
6.2 The Front End . . . 84
6.2.1 Extraction of assembly blocks . . . 84
6.2.2 Compile-time directives . . . 84
6.2.3 The lexer (Untokenizer) . . . 85
6.2.4 Token-level passes . . . 85
6.2.5 The parser . . . 85
6.2.6 AST passes . . . 85
6.2.7 ASHIR generation . . . 86
6.3 The Back End . . . 86
6.3.1 ASHIR parsing . . . 86
6.3.2 Code generation . . . 86
6.3.3 Assembly through shasm . . . 87

Copyright © 2026 Mallionic Inc. All rights reserved.

6.3.4 Object writers . . . 87
6.4 Observing the Stages . . . 88
6.5 Design Notes . . . 90
6.5.1 Why a textual intermediate representation . . . 90
6.5.2 Why many small passes . . . 90
6.5.3 Determinism . . . 90

7 Lexical Structure . . . 91
7.1 Source Files . . . 91
7.2 White Space and Line Structure . . . 91
7.2.1 Semicolons . . . 91
7.3 Comments . . . 92
7.4 Identifiers . . . 92
7.5 Keywords . . . 93
7.6 Literals . . . 94
7.6.1 Integer literals . . . 94
7.6.2 Floating-point literals . . . 95
7.6.3 Character literals . . . 95
7.6.4 String literals . . . 96
7.6.5 Wide string literals . . . 96
7.7 Operators and Punctuation . . . 97
7.8 Token-Level Transformations . . . 97
7.8.1 Multi-word C type names . . . 97
7.8.2 Compound assignment . . . 98
7.8.3 The ashley intrinsics . . . 99

8 Types . . . 100
8.1 Overview . . . 100
8.2 Integer Types . . . 100
8.2.1 Integers in ASHIR . . . 101
8.2.2 Widening: sign and zero extension . . . 102
8.2.3 Narrowing: truncation on store . . . 104
8.2.4 The type of a literal . . . 105
8.2.5 Operations that depend on signedness . . . 105
8.3 Integer Width and the Target . . . 106
8.4 Boolean . . . 106
8.5 Characters . . . 107
8.6 Strings . . . 107
8.7 Pointers . . . 108

Copyright © 2026 Mallionic Inc. All rights reserved.

8.7.1 Function pointers . . . 109
8.8 Floating-Point Types . . . 110
8.9 Void . . . 110
8.10 Casts . . . 110
8.11 Type Aliases . . . 111
8.12 Sizes and Alignment . . . 112

9 Variables and Storage . . . 113
9.1 Local Variables . . . 113
9.1.1 Declaration . . . 113
9.1.2 Default initialization . . . 114
9.1.3 Scope and lifetime . . . 114
9.1.4 Assignment . . . 115
9.2 Where Locals Live . . . 115
9.3 Volatile Variables . . . 116
9.4 Global Variables . . . 117
9.4.1 How globals are accessed . . . 119
9.4.2 The address of a global . . . 119
9.4.3 Global arrays . . . 120
9.4.4 Runnable arrays . . . 121
9.4.5 Placing a global in a section . . . 121
9.5 Sharing Variables with C . . . 121

10 Expressions and Operators . . . 123
10.1 Kinds of Expressions . . . 123
10.2 Precedence and Associativity . . . 123
10.3 Evaluation and Temporaries . . . 124
10.4 Arithmetic Operators . . . 125
10.5 Power and Repeated Squaring . . . 127
10.6 Shift Operators . . . 128
10.7 Bitwise Operators . . . 129
10.8 Unary Operators . . . 129
10.9 Comparison Operators . . . 130
10.10 Function Calls . . . 131
10.11 Constant Folding . . . 131
10.12 Summary of Operators and Their Translation . . . 131

11 Statements and Control Flow . . . 132

Copyright © 2026 Mallionic Inc. All rights reserved.

11.1 Kinds of Statements . . . 132
11.2 Blocks . . . 132
11.3 Conditions . . . 133
11.3.1 How conditions are compiled . . . 133
11.4 The if Statement . . . 134
11.5 Loops . . . 134
11.5.1 while . . . 134
11.5.2 for . . . 136
11.5.3 repeat . . . 136
11.5.4 loop . . . 138
11.6 break and continue . . . 138
11.7 The switch Statement . . . 138
11.7.1 How switch is compiled . . . 139
11.8 Leaving a Function . . . 140
11.8.1 return . . . 140
11.8.2 terminate . . . 140
11.9 What ASH Does Not Have . . . 141

12 Functions . . . 142
12.1 Declaring a Function . . . 142
12.2 Parameters . . . 143
12.2.1 Assigning to a parameter . . . 143
12.2.2 Passing by reference . . . 144
12.3 Return Values . . . 145
12.4 Calling Conventions . . . 145
12.4.1 More than six arguments . . . 146
12.4.2 Stack frames . . . 147
12.5 Recursion . . . 148
12.6 Variadic Functions . . . 148
12.6.1 How variadic functions are implemented . . . 149
12.7 Function Pointers and Callbacks . . . 150
12.8 External Functions . . . 152
12.8.1 Implicit external functions . . . 153
12.9 The Entry Point . . . 153
12.10 Naked Functions . . . 154
12.11 Methods . . . 154

13 Arrays, Strings and Matrices . . . 155

Copyright © 2026 Mallionic Inc. All rights reserved.

13.1 Declaring Arrays . . . 155
13.2 Memory Layout . . . 156
13.3 Indexing . . . 156
13.3.1 Bounds checking . . . 158
13.4 The Length Operator # . . . 158
13.5 Arrays as Parameters . . . 159
13.6 Two-Dimensional Arrays . . . 159
13.7 Matrices . . . 160
13.8 Text in Arrays and Buffers . . . 161
13.8.1 char arrays . . . 162
13.9 Global and Runnable Arrays . . . 162
13.10 Cost Model . . . 162

14 Structures and Pointers . . . 163
14.1 Declaring Structures . . . 163
14.2 Layout . . . 163
14.3 Structure Variables . . . 164
14.4 Field Access . . . 165
14.5 Structures and Functions . . . 166
14.6 Sharing Structures with C . . . 166
14.7 Structures for Memory-Mapped Data . . . 167
14.8 Limitations . . . 167

15 Classes and Visibility . . . 168
15.1 Declaring a Class . . . 168
15.2 Calling Methods . . . 169
15.3 Symbols Produced by a Class . . . 169
15.4 Private Methods . . . 169
15.5 Visibility of Functions Across Files . . . 170

16 Cracks and Pending Return Values . . . 171
16.1 Cracks . . . 171
16.1.1 Declaring a crack . . . 171
16.1.2 How cracks are compiled . . . 172
16.1.3 When to use cracks . . . 173
16.2 Pending Return Values . . . 173
16.2.1 How pending values are compiled . . . 174

Copyright © 2026 Mallionic Inc. All rights reserved.

17 Annotations (Attributes) . . . 176

17.1 Annotations in Five Minutes . . . 176
17.2 The Three Kinds of Annotations . . . 177
17.3 Summary Table . . . 178
17.4 @Entry: Where the Program Starts . . . 178
17.5 @bits: Choosing the Processor Mode . . . 180
17.6 @originoffs: Where the Program Will Be Loaded . . . 183
17.7 @beautycode: Clean Code or Full Frames . . . 184
17.8 @bounds: Array Checking On or Off . . . 185
17.9 @strict: Extra Safety Checks . . . 186
17.10 @nolibc: Programs Without the C Library . . . 187
17.11 @naked: Functions Without Compiler Code . . . 188
17.12 @msabi and @efiapi: The Microsoft Calling Convention . . . 189
17.13 @section and @attribute: Choosing Where Code and Data Go . . . 190
17.14 @ovr and @override: Which Definition Wins . . . 191
17.15 @emit: Raw Bytes as a Statement . . . 192
17.16 Your Own Annotations . . . 193
17.17 Placement Rules . . . 195

18 Compile-Time Directives and Macros . . . 197

18.1 Directive Syntax . . . 197
18.2 Predefined Symbols . . . 198
18.3 Directive Expressions . . . 199
18.4 The define Macro . . . 201
18.5 Block Macros . . . 202
18.6 typedef . . . 202
18.7 redefine . . . 202

19 Modules, Imports and Override Rights . . . 204

19.1 The import Statement . . . 204
19.2 Where Files Are Found . . . 204
19.2.1 System libraries: import . . . 204
19.2.2 Project files: import "file.ash" . . . 204
19.3 What an Import Brings In . . . 205
19.3.1 Tree shaking . . . 205
19.4 Override Rights . . . 206
19.4.1 Your function beats the library . . . 207
19.4.2 Choosing between imported files . . . 207

Copyright © 2026 Mallionic Inc. All rights reserved.

19.4.3 Raising a right from the main file . . . 208
19.5 Modules and C Objects . . . 208

20 Inline Assembly and Machine Code . . . 210
20.1 asm Blocks . . . 210
20.1.1 Using ASH variables . . . 210
20.1.2 Labels and control flow . . . 212
20.1.3 Registers: what an asm block may change . . . 213
20.2 The asm("...") Form . . . 213
20.3 emit: Raw Bytes . . . 214
20.4 Runnable Arrays . . . 215
20.5 Naked Functions . . . 216
20.6 Assembly in 32-bit and 16-bit Programs . . . 217
20.7 How an asm Block Reaches the Object File . . . 218

21 Strict Mode . . . 220
21.1 Enabling Strict Mode . . . 220
21.2 The Rules . . . 220
21.3 Examples . . . 221
21.4 When to Use Strict Mode . . . 223

22 Targets, Runtimes and Interoperability . . . 224
22.1 Overview . . . 224
22.2 Linux x86-64 . . . 224
22.2.1 Linking with the C library . . . 224
22.2.2 Without the C library . . . 225
22.3 Linux i386 . . . 226
22.4 Linux i386 Executables (--exe) . . . 227
22.5 Flat Binaries . . . 228
22.6 Windows x64 . . . 229
22.7 UEFI Applications . . . 231
22.8 Corresponding C and ASH Types . . . 232
22.9 Calling Directions . . . 232

23 ASHIR: The Intermediate Representation . . . 233
23.1 Overview . . . 233
23.2 Lexical Rules . . . 234
23.3 Types . . . 235

Copyright © 2026 Mallionic Inc. All rights reserved.

23.4 Functions and Externs . . . 235
23.5 Operations . . . 235
23.5.1 Destination form . . . 235
23.5.2 Statement form . . . 237
23.6 Global Data Directives . . . 237
23.7 How ASH Constructs Appear in ASHIR . . . 239
23.8 Runtime Helper Functions . . . 240
23.9 Working with ASHIR Files . . . 247

24 Code Generation for x86 . . . 248
24.1 A Direct Translator . . . 248
24.2 Register Usage on x86-64 . . . 249
24.3 Prologue and Epilogue . . . 249
24.4 Constants and Moves . . . 251
24.5 Arithmetic, Division and Shifts . . . 251
24.6 Conditions and Loops . . . 253
24.7 Calls . . . 254
24.8 Memory Access . . . 255
24.9 The 32-bit Generator . . . 258
24.10 The 16-bit Generator . . . 259
24.11 Reading Generated Code . . . 260

25 An x86 Primer for ASH Programmers . . . 261
25.1 Operating Modes . . . 261
25.2 General-Purpose Registers . . . 261
25.2.1 Other registers . . . 262
25.3 Flags . . . 262
25.4 Memory Operands . . . 263
25.5 The Stack . . . 264
25.6 Instruction Encoding . . . 264
25.6.1 Reading a ModRM byte . . . 265
25.7 Instructions Used by ASH . . . 265
25.8 Calling Conventions at a Glance . . . 266

26 The shasm Assembler . . . 267
26.1 Running shasm . . . 267
26.2 Source Syntax . . . 268
26.3 Numbers, Characters and Strings . . . 269

Copyright © 2026 Mallionic Inc. All rights reserved.

26.4 Expressions . . . 269
26.5 Data Directives . . . 270
26.6 Assembler Directives . . . 271
26.7 The Preprocessor . . . 271
26.7.1 Metadata directives . . . 273
26.8 Output Formats . . . 274
26.9 A Linux Program in Assembly . . . 274
26.10 A Boot Sector . . . 275
26.11 Listings . . . 277
26.12 Diagnostics . . . 277
26.13 shasm Inside ashc . . . 277

27 The nlib System Library . . . 278
27.1 Overview . . . 278
27.1.1 Conventions . . . 278
27.2 : Strings and Memory . . . 279
27.2.1 memset . . . 279
27.2.2 memcpy . . . 280
27.2.3 memmove . . . 282
27.2.4 memcmp . . . 285
27.2.5 memchr . . . 287
27.2.6 strlen . . . 289
27.2.7 strnlen . . . 290
27.2.8 strcpy . . . 292
27.2.9 strncpy . . . 294
27.2.10 strcat . . . 296
27.2.11 strncat . . . 299
27.2.12 strcmp . . . 301
27.2.13 strncmp . . . 304
27.2.14 strcasecmp . . . 306
27.2.15 strchr . . . 308
27.2.16 strrchr . . . 311
27.2.17 strstr . . . 313
27.2.18 strspn . . . 315
27.2.19 strcspn . . . 317
27.2.20 isdigit . . . 319
27.2.21 isupper . . . 321
27.2.22 islower . . . 322
27.2.23 isalpha . . . 323

Copyright © 2026 Mallionic Inc. All rights reserved.

27.2.24 isalnum . . . 325
27.2.25 isspace . . . 326
27.2.26 isxdigit . . . 327
27.2.27 toupper . . . 329
27.2.28 tolower . . . 331
27.2.29 str_to_int . . . 332
27.2.30 atoi . . . 336
27.2.31 uint_to_str . . . 337
27.2.32 int_to_str . . . 340
27.2.33 str_reverse . . . 342
27.2.34 str_upper . . . 345
27.2.35 str_lower . . . 347
27.2.36 str_equals . . . 349
27.2.37 str_starts . . . 351
27.2.38 str_ends . . . 352
27.3 : Integer and Fixed-Point Mathematics . . . 354
27.3.1 The Q16.16 format . . . 354
27.3.2 abs . . . 355
27.3.3 min . . . 356
27.3.4 max . . . 357
27.3.5 clamp . . . 358
27.3.6 sign . . . 360
27.3.7 gcd . . . 361
27.3.8 lcm . . . 363
27.3.9 ipow . . . 365
27.3.10 isqrt . . . 366
27.3.11 ilog2 . . . 368
27.3.12 ilog10 . . . 370
27.3.13 factorial . . . 371
27.3.14 is_prime . . . 373
27.3.15 mod_pow . . . 375
27.3.16 div_floor . . . 377
27.3.17 mod_floor . . . 379
27.3.18 is_pow2 . . . 381
27.3.19 next_pow2 . . . 382
27.3.20 popcount . . . 383
27.3.21 align_up . . . 385
27.3.22 align_down . . . 386
27.3.23 fx . . . 387

Copyright © 2026 Mallionic Inc. All rights reserved.

27.3.24 fx_int . . . 388
27.3.25 fx_round . . . 389
27.3.26 fx_frac . . . 391
27.3.27 fx_abs . . . 392
27.3.28 fx_mul . . . 393
27.3.29 fx_div . . . 394
27.3.30 fx_sqrt . . . 396
27.3.31 fx_deg . . . 397
27.3.32 fx_sin . . . 398
27.3.33 fx_cos . . . 401
27.3.34 fx_tan . . . 402
27.3.35 fx_print_buf . . . 404
27.4 : Linux System Calls without libc . . . 408
27.4.1 syscall6 . . . 408
27.4.2 sys_read . . . 410
27.4.3 sys_write . . . 412
27.4.4 sys_open . . . 413
27.4.5 sys_close . . . 415
27.4.6 sys_lseek . . . 416
27.4.7 sys_mmap . . . 418
27.4.8 sys_munmap . . . 419
27.4.9 sys_brk . . . 421
27.4.10 sys_getpid . . . 422
27.4.11 sys_kill . . . 424
27.4.12 sys_mkdir . . . 425
27.4.13 sys_unlink . . . 427
27.4.14 sys_nanosleep . . . 428
27.4.15 sys_exit . . . 431
27.4.16 print . . . 432
27.4.17 println . . . 434
27.4.18 print_int . . . 435
27.5 : Command Line Arguments . . . 438
27.5.1 arg_at . . . 439
27.5.2 arg_equals . . . 440
27.5.3 arg_int . . . 440
27.5.4 arg_has . . . 440
27.6 : Pauses . . . 441
27.6.1 delay . . . 441
27.6.2 delayus . . . 442

Copyright © 2026 Mallionic Inc. All rights reserved.

27.7 : C Standard Input and Output . . . 443
27.8 : C Standard Library Wrappers . . . 449
27.9 : Floating-Point Special Values . . . 451
27.9.1 inf . . . 451
27.9.2 ninf . . . 451
27.9.3 pos_inf . . . 452
27.9.4 neg_inf . . . 452
27.9.5 float_nan . . . 453
27.9.6 float_inf . . . 453
27.9.7 float_ninf . . . 454
27.9.8 double_nan . . . 454
27.9.9 double_inf . . . 455
27.9.10 double_ninf . . . 456
27.9.11 nan . . . 456
27.10 and : Constants . . . 457

28 Diagnostics Reference . . . 459
28.1 Reading a Diagnostic . . . 459
28.2 Lexical Errors . . . 459
28.2.1 Unexpected character . . . 459
28.2.2 Unterminated string . . . 459
28.3 Syntax Errors . . . 460
28.3.1 Unexpected statement . . . 460
28.3.2 Missing brace . . . 460
28.3.3 Expected func, class, extern or import . . . 460
28.3.4 Annotation errors . . . 461
28.4 Semantic Errors . . . 461
28.4.1 Undeclared variable . . . 462
28.4.2 Global without visibility . . . 462
28.4.3 Assignment to a constant global . . . 462
28.4.4 Literal out of range . . . 463
28.4.5 Private method . . . 463
28.4.6 Too many variadic arguments . . . 463
28.4.7 Comparison used as a value . . . 464
28.4.8 Constant expression too large . . . 464
28.4.9 Division by a constant zero . . . 464
28.4.10 Matrix rows of different length . . . 465
28.5 Module Errors . . . 465
28.5.1 Unknown library . . . 465

Copyright © 2026 Mallionic Inc. All rights reserved.

28.5.2 Ambiguous definitions . . . 465
28.6 Directive Errors . . . 466
28.7 Strict Mode Errors . . . 466
28.8 Command Line Errors . . . 466
28.9 Errors Reported by the Linker . . . 466
28.10 Errors at Run Time . . . 467

29 Tooling . . . 468
29.1 The VS Code Extension . . . 468
29.1.1 Installation . . . 468
29.1.2 The linter . . . 468
29.1.3 Snippets . . . 469
29.1.4 Changing colors . . . 470
29.2 Build Scripts . . . 471
29.3 The Test Suite . . . 471
29.4 Building This Manual . . . 471

30 Cookbook: Complete Programs . . . 472
30.1 Prime Numbers with the Sieve of Eratosthenes . . . 472
30.2 Sorting: Insertion Sort and qsort . . . 473
30.3 A Linked List with Structures . . . 474
30.4 A Stack Calculator in Reverse Polish Notation . . . 475
30.5 Counting Lines, Words and Characters . . . 476
30.6 Number Base Conversion . . . 476
30.7 Hashing with FNV-1a . . . 477
30.8 Matrix Multiplication . . . 478
30.9 ASCII Mandelbrot Set in Fixed Point . . . 478
30.10 Conway's Game of Life . . . 480
30.11 A Hex Dump Tool . . . 481
30.12 Writing a WAV File: a Bytebeat Generator . . . 482
30.13 cat Without the C Library . . . 484
30.14 A Command Line Tool with Options . . . 484
30.15 Generating Machine Code at Run Time . . . 485
30.16 Callbacks and Function Tables . . . 486
30.17 Statistics with Variadic Functions . . . 487
30.18 A Real-Mode Routine . . . 488
30.19 A Windows Console Program . . . 489

Copyright © 2026 Mallionic Inc. All rights reserved.

31 Bare-Metal Programming: The VGA Text Screen . . . 490
31.1 The Memory of a PC . . . 490
31.2 How the Text Screen Works . . . 490
31.3 The Skeleton of a Kernel . . . 491
31.4 Hello, Screen . . . 493
31.5 All the Colors . . . 494
31.6 A Small VGA Driver . . . 496
31.7 Going Further . . . 501

32 Common Mistakes, Techniques and Questions . . . 502
32.1 Common Mistakes . . . 502
32.1.1 Using = instead of == in a condition . . . 502
32.1.2 Forgetting the braces of if and loops . . . 503
32.1.3 Counting array positions from 1 . . . 503
32.1.4 Expecting decimals from integer division . . . 504
32.1.5 Choosing a type that is too small . . . 504
32.1.6 Using %d for very large numbers . . . 505
32.1.7 Forgetting return in a function with a result . . . 505
32.1.8 Changing the letters of a string literal . . . 506
32.1.9 Joining strings with + . . . 507
32.1.10 Storing a comparison in a variable . . . 508
32.1.11 Declaring a global without general or ungeneral . . . 508
32.1.12 Mixing printf with direct writes . . . 509
32.1.13 Adding to a pointer to move one element . . . 509
32.1.14 More mistakes in brief . . . 510
32.2 Techniques to Master ASH . . . 510
32.2.1 Look at what the compiler did . . . 510
32.2.2 Use the exit status for quick tests . . . 511
32.2.3 Write your own checks . . . 511
32.2.4 Trace functions with an annotation . . . 512
32.2.5 Let the compiler help you . . . 513
32.2.6 Give numbers a name . . . 513
32.2.7 Measure how long code takes . . . 514
32.2.8 Bit tricks . . . 514
32.2.9 Debug with gdb . . . 515
32.2.10 Write code for several targets . . . 515
32.3 Frequently Asked Questions . . . 516
32.3.1 Is ASH a replacement for C? . . . 516
32.3.2 Do ASH programs need Java to run? . . . 516

Copyright © 2026 Mallionic Inc. All rights reserved.

32.3.3 Which name should the entry function have? . . . 516
32.3.4 Why does my program print nothing and return a strange number? . . . 516
32.3.5 How do I read a number typed by the user? . . . 516
32.3.6 How do I generate random numbers? . . . 516
32.3.7 How do I allocate memory whose size is known only at run time? . . . 517
32.3.8 How do I return several values from a function? . . . 517
32.3.9 Can I write a whole program in assembly? . . . 518
32.3.10 Why are local arrays not freed? . . . 518
32.3.11 How big are ASH programs? . . . 518
32.3.12 Can ASH build an operating system or a boot loader? . . . 518
32.3.13 Where can I see more examples? . . . 518

33 Language Dictionary . . . 519

33.1 Symbols and Operators . . . 519
33.1.1 # . . . 519
33.1.2 & . . . 519
33.1.3 * . . . 520
33.1.4 ** . . . 520
33.1.5 *** . . . 521
33.1.6 ++ and -- . . . 521
33.1.7 += -= *= /= %= . . . 521
33.1.8 -> and . . . 522
33.1.9 522
33.1.10 << and >> . . . 523
33.1.11 == != < <= > >= . . . 523
33.1.12 && || ! . . . 524
33.1.13 @ . . . 524
33.2 Words, from A to Z . . . 525
33.2.1 asm . . . 525
33.2.2 bool, true, false . . . 525
33.2.3 break . . . 525
33.2.4 case and default . . . 526
33.2.5 char . . . 526
33.2.6 class . . . 527
33.2.7 continue . . . 527
33.2.8 define . . . 528
33.2.9 elif and else . . . 528
33.2.10 emit . . . 529
33.2.11 extern . . . 529

Copyright © 2026 Mallionic Inc. All rights reserved.

33.2.12 for . . . 530
33.2.13 func . . . 530
33.2.14 general and ungeneral . . . 530
33.2.15 if . . . 531
33.2.16 import . . . 531
33.2.17 int8 ... uint64 . . . 532
33.2.18 len, length, rows, cols, count . . . 532
33.2.19 loop . . . 533
33.2.20 macro . . . 533
33.2.21 matrix . . . 534
33.2.22 pub and priv . . . 534
33.2.23 redefine . . . 534
33.2.24 repeat . . . 535
33.2.25 return . . . 535
33.2.26 runnable . . . 536
33.2.27 static . . . 536
33.2.28 string . . . 536
33.2.29 struct and typedef . . . 537
33.2.30 switch . . . 537
33.2.31 terminate . . . 538
33.2.32 unreturnable . . . 538
33.2.33 "use strict" . . . 539
33.2.34 va_count and va_arg . . . 539
33.2.35 void . . . 539
33.2.36 volatile . . . 540
33.2.37 while . . . 540
33.2.38 %if, %define and other directives . . . 541

Appendix A x86 Instruction Reference . . . 542

A.1 How to Read the Tables . . . 542
A.2 General-purpose Instructions . . . 542
A.2.1 MOV — Move . . . 543
A.2.2 MOVZX — Move with Zero-Extend . . . 544
A.2.3 MOVSX — Move with Sign-Extend . . . 544
A.2.4 MOVSXD — Move Doubleword with Sign-Extend . . . 545
A.2.5 XCHG — Exchange . . . 545
A.2.6 XADD — Exchange and Add . . . 545
A.2.7 CMPXCHG — Compare and Exchange . . . 545
A.2.8 CMPXCHG8B — Compare and Exchange 8 Bytes . . . 546

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.9 BSWAP — Byte Swap . . . 546
A.2.10 MOVBE — Move Data After Swapping Bytes . . . 546
A.2.11 PUSH — Push onto the Stack . . . 547
A.2.12 POP — Pop from the Stack . . . 548
A.2.13 PUSHF — Push Flags . . . 548
A.2.14 POPF — Pop Flags . . . 548
A.2.15 LEA — Load Effective Address . . . 548
A.2.16 CBW — Convert Byte to Word . . . 549
A.2.17 CWD — Convert Word to Doubleword . . . 549
A.2.18 CQO — Convert Quadword to Octoword . . . 549
A.2.19 CDQ — Convert Doubleword to Quadword . . . 549
A.2.20 CDQE — Convert Doubleword to Quadword in RAX . . . 549
A.2.21 CWDE — Convert Word to Doubleword in EAX . . . 550
A.2.22 XLATB — Table Look-up Translation . . . 550
A.2.23 ADD — Add . . . 551
A.2.24 ADC — Add with Carry . . . 552
A.2.25 SUB — Subtract . . . 553
A.2.26 SBB — Subtract with Borrow . . . 554
A.2.27 INC — Increment by 1 . . . 555
A.2.28 DEC — Decrement by 1 . . . 555
A.2.29 NEG — Two's Complement Negation . . . 555
A.2.30 CMP — Compare Two Operands . . . 556
A.2.31 MUL — Unsigned Multiply . . . 557
A.2.32 IMUL — Signed Multiply . . . 557
A.2.33 DIV — Unsigned Divide . . . 558
A.2.34 IDIV — Signed Divide . . . 558
A.2.35 AND — Logical AND . . . 559
A.2.36 OR — Logical Inclusive OR . . . 560
A.2.37 XOR — Logical Exclusive OR . . . 561
A.2.38 NOT — One's Complement Negation . . . 562
A.2.39 TEST — Logical Compare . . . 562
A.2.40 SHL — Shift Left . . . 563
A.2.41 SAL — Shift Arithmetic Left . . . 564
A.2.42 SHR — Shift Logical Right . . . 565
A.2.43 SAR — Shift Arithmetic Right . . . 566
A.2.44 ROL — Rotate Left . . . 567
A.2.45 ROR — Rotate Right . . . 568
A.2.46 RCL — Rotate Through Carry Left . . . 569
A.2.47 RCR — Rotate Through Carry Right . . . 570

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.48 SHLD — Double Precision Shift Left . . . 570
A.2.49 SHRD — Double Precision Shift Right . . . 570
A.2.50 BT — Bit Test . . . 571
A.2.51 BTS — Bit Test and Set . . . 571
A.2.52 BTR — Bit Test and Reset . . . 571
A.2.53 BTC — Bit Test and Complement . . . 571
A.2.54 BSF — Bit Scan Forward . . . 571
A.2.55 BSR — Bit Scan Reverse . . . 572
A.2.56 POPCNT — Population Count . . . 572
A.2.57 LZCNT — Count Leading Zero Bits . . . 572
A.2.58 TZCNT — Count Trailing Zero Bits . . . 572
A.2.59 CRC32 — Accumulate CRC32 Value . . . 573
A.2.60 JMP — Jump . . . 573
A.2.61 Jcc — Jump if Condition Is Met . . . 573
A.2.62 JCXZ — Jump if CX Is Zero . . . 574
A.2.63 LOOP — Loop According to ECX Counter . . . 574
A.2.64 CALL — Call Procedure . . . 574
A.2.65 RET — Return from Procedure . . . 574
A.2.66 ENTER — Make Stack Frame . . . 575
A.2.67 LEAVE — High Level Procedure Exit . . . 575
A.2.68 INT — Call to Interrupt Procedure . . . 575
A.2.69 INT3 — Breakpoint . . . 575
A.2.70 INTO — Interrupt on Overflow . . . 575
A.2.71 IRET — Interrupt Return . . . 576
A.2.72 SETcc — Set Byte on Condition . . . 576
A.2.73 CMOVcc — Conditional Move . . . 576
A.2.74 CLC — Clear Carry Flag . . . 576
A.2.75 STC — Set Carry Flag . . . 576
A.2.76 CMC — Complement Carry Flag . . . 577
A.2.77 CLD — Clear Direction Flag . . . 577
A.2.78 STD — Set Direction Flag . . . 577
A.2.79 CLI — Clear Interrupt Flag . . . 577
A.2.80 STI — Set Interrupt Flag . . . 577
A.2.81 LAHF — Load Status Flags into AH . . . 578
A.2.82 SAHF — Store AH into Flags . . . 578
A.2.83 NOP — No Operation . . . 578
A.2.84 PAUSE — Spin Loop Hint . . . 578
A.2.85 HLT — Halt . . . 578
A.2.86 UD2 — Undefined Instruction . . . 578

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.87 CPUID — CPU Identification . . . 579
A.2.88 RDTSC — Read Time-Stamp Counter . . . 579
A.2.89 RDTSCP — Read Time-Stamp Counter and Processor ID . . . 579
A.2.90 RDRAND — Read Random Number . . . 579
A.2.91 RDSEED — Read Random Seed . . . 579
A.2.92 MFENCE — Memory Fence . . . 580
A.2.93 LFENCE — Load Fence . . . 580
A.2.94 SFENCE — Store Fence . . . 580
A.2.95 CLFLUSH — Flush Cache Line . . . 580
A.2.96 PREFETCHT0 — Prefetch Data into Caches . . . 580
A.2.97 MOVSB — Move String Byte . . . 581
A.2.98 STOSB — Store String Byte . . . 581
A.2.99 LODSB — Load String Byte . . . 581
A.2.100 SCASB — Scan String Byte . . . 581
A.2.101 CMPSB — Compare String Bytes . . . 581
A.2.102 IN — Input from Port . . . 582
A.2.103 OUT — Output to Port . . . 582
A.2.104 INSB — Input String from Port . . . 582
A.2.105 OUTSB — Output String to Port . . . 582
A.3 System Instructions . . . 583
A.3.1 SYSCALL — Fast System Call . . . 583
A.3.2 SYSRET — Return from Fast System Call . . . 583
A.3.3 SYSENTER — Fast System Call (32-bit) . . . 583
A.3.4 SYSEXIT — Fast Return from SYSENTER . . . 583
A.3.5 LGDT — Load Global Descriptor Table Register . . . 583
A.3.6 LIDT — Load Interrupt Descriptor Table Register . . . 584
A.3.7 SGDT — Store Global Descriptor Table Register . . . 584
A.3.8 SIDT — Store Interrupt Descriptor Table Register . . . 584
A.3.9 LLDT — Load Local Descriptor Table Register . . . 584
A.3.10 LTR — Load Task Register . . . 584
A.3.11 INVLPG — Invalidate TLB Entry . . . 585
A.3.12 WBINVD — Write Back and Invalidate Cache . . . 585
A.3.13 RDMSR — Read from Model Specific Register . . . 585
A.3.14 WRMSR — Write to Model Specific Register . . . 585
A.3.15 RDPMC — Read Performance-Monitoring Counter . . . 585
A.3.16 CLTS — Clear Task-Switched Flag in CR0 . . . 586
A.3.17 SWAPGS — Swap GS Base Register . . . 586
A.3.18 LMSW — Load Machine Status Word . . . 586
A.3.19 SMSW — Store Machine Status Word . . . 586

Copyright © 2026 Mallionic Inc. All rights reserved.

A.3.20 ARPL — Adjust RPL Field of Segment Selector . . . 586
A.3.21 LAR — Load Access Rights Byte . . . 586
A.3.22 LSL — Load Segment Limit . . . 587
A.3.23 VERR — Verify a Segment for Reading . . . 587
A.3.24 VERW — Verify a Segment for Writing . . . 587
A.4 x87 floating point Instructions . . . 587
A.4.1 FLD — Load Floating-Point Value . . . 587
A.4.2 FST — Store Floating-Point Value . . . 588
A.4.3 FSTP — Store Floating-Point Value and Pop . . . 588
A.4.4 FILD — Load Integer . . . 588
A.4.5 FISTP — Store Integer and Pop . . . 588
A.4.6 FADD — Add . . . 589
A.4.7 FSUB — Subtract . . . 589
A.4.8 FMUL — Multiply . . . 589
A.4.9 FDIV — Divide . . . 590
A.4.10 FCHS — Change Sign . . . 590
A.4.11 FABS — Absolute Value . . . 590
A.4.12 FSQRT — Square Root . . . 590
A.4.13 FSIN — Sine . . . 590
A.4.14 FCOS — Cosine . . . 591
A.4.15 FXCH — Exchange Register Contents . . . 591
A.4.16 FCOM — Compare Floating-Point Values . . . 591
A.4.17 FCOMI — Compare and Set EFLAGS . . . 591
A.4.18 FINIT — Initialize FPU . . . 591
A.4.19 FLDZ — Load +0.0 . . . 592
A.4.20 FLD1 — Load +1.0 . . . 592
A.4.21 FLDPI — Load Pi . . . 592
A.4.22 FWAIT — Wait . . . 592
A.5 SSE and SSE2 Instructions . . . 592
A.5.1 MOVAPS — Move Aligned Packed Single . . . 593
A.5.2 MOVUPS — Move Unaligned Packed Single . . . 593
A.5.3 MOVDQA — Move Aligned Double Quadword . . . 593
A.5.4 MOVDQU — Move Unaligned Double Quadword . . . 593
A.5.5 MOVSS — Move Scalar Single . . . 594
A.5.6 MOVSD — Move Scalar Double . . . 594
A.5.7 MOVD — Move Doubleword . . . 594
A.5.8 MOVQ — Move Quadword . . . 595
A.5.9 ADDSS — Add Scalar Single . . . 595
A.5.10 ADDSD — Add Scalar Double . . . 595

Copyright © 2026 Mallionic Inc. All rights reserved.

A.5.11 SUBSD — Subtract Scalar Double . . . 596
A.5.12 MULSD — Multiply Scalar Double . . . 596
A.5.13 DIVSD — Divide Scalar Double . . . 596
A.5.14 SQRTSD — Square Root Scalar Double . . . 596
A.5.15 ADDPS — Add Packed Single . . . 596
A.5.16 MULPS — Multiply Packed Single . . . 597
A.5.17 CVTSI2SD — Convert Integer to Scalar Double . . . 597
A.5.18 CVTTSD2SI — Convert with Truncation Scalar Double to Integer . . . 597
A.5.19 CVTSD2SS — Convert Scalar Double to Scalar Single . . . 597
A.5.20 CVTSS2SD — Convert Scalar Single to Scalar Double . . . 597
A.5.21 COMISD — Compare Scalar Ordered Double . . . 598
A.5.22 UCOMISD — Unordered Compare Scalar Double . . . 598
A.5.23 ANDPS — Bitwise AND of Packed Single . . . 598
A.5.24 ORPS — Bitwise OR of Packed Single . . . 598
A.5.25 XORPS — Bitwise XOR of Packed Single . . . 598
A.5.26 PXOR — Packed Logical XOR . . . 599
A.5.27 PAND — Packed Logical AND . . . 599
A.5.28 POR — Packed Logical OR . . . 599
A.5.29 PADDB — Packed Add Bytes . . . 599
A.5.30 PADDD — Packed Add Doublewords . . . 600
A.5.31 PADDQ — Packed Add Quadwords . . . 600
A.5.32 PSUBD — Packed Subtract Doublewords . . . 600
A.5.33 PCMPEQB — Packed Compare for Equal Bytes . . . 600
A.5.34 PMOVMSKB — Move Byte Mask . . . 601
A.5.35 PSHUFD — Shuffle Packed Doublewords . . . 601
A.5.36 SHUFPS — Shuffle Packed Single . . . 601
A.5.37 UNPCKLPS — Unpack and Interleave Low Packed Single . . . 601
A.5.38 LDMXCSR — Load MXCSR Register . . . 601
A.5.39 STMXCSR — Store MXCSR Register . . . 602
A.5.40 EMMS — Empty MMX State . . . 602

Appendix B Register Reference . . . 603

Appendix C Grammar Summary . . . 610

C.1 Program Structure . . . 610
C.2 Functions and Classes . . . 610
C.3 Types . . . 611
C.4 Statements . . . 611
C.5 Expressions and Conditions . . . 612

Copyright © 2026 Mallionic Inc. All rights reserved.

C.6 Lexical Elements . . . 612

Appendix D Index of the Example Programs . . . 613

Appendix E Known Limitations of ashc 1.0 . . . 616

E.1 Language . . . 616
E.2 Code Generation . . . 617
E.3 Libraries and Tools . . . 618

Appendix F Glossary . . . 619

Appendix G ASH for C Programmers . . . 622

G.1 Syntax Differences . . . 623
G.2 What ASH Adds . . . 623
G.3 What ASH Does Not Have (in 1.0) . . . 624
G.4 Semantic Differences That Matter . . . 624
G.5 Porting a C Function . . . 624

Appendix H Quick Reference . . . 626

H.1 Program Skeletons . . . 626
H.2 Declarations . . . 627
H.3 Statements . . . 628
H.4 Operators . . . 628
H.5 Annotations . . . 629
H.6 Command Line . . . 629
H.7 Library Functions . . . 630

Index . . . 631

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 1

INTRODUCTION TO ASH

This manual is the programming guide and reference for **ASH**, the systems programming language of **The Ashley Project**. It describes the language itself, the `ashc` compiler that translates it, the ASHIR intermediate representation produced by the compiler, the `shasm` assembler that encodes inline assembly and stand-alone assembly files, and the `nlib` system library embedded in the compiler.

The manual is written for programmers who want to understand not only *what* an ASH program does but also *how* it becomes machine code. For that reason almost every chapter shows, next to the source code, the ASHIR listing and the x86 disassembly that the compiler really produced.

1.1 WHAT ASH IS

ASH is a compiled, statically typed, imperative language for the x86 family of processors. Its syntax is close to C: functions, blocks delimited by braces, `if`, `while`, `for`, `switch`, pointers, arrays and structures behave the way a C programmer expects. On top of that familiar base ASH adds a number of features aimed at people who write low-level software and want to stay close to the hardware:

- **Assembly is part of the language.** An `asm { }` block can name ASH variables directly (`mov rax, total`), the GCC-style string form `asm volatile("...")` is accepted, raw machine bytes can be placed in the instruction stream with `emit(...)`, and a function can be declared `@naked` so that the compiler generates nothing but the bytes you write.
- **One source, several machines.** The same compiler produces 16-bit flat binaries (boot sectors, DOS-style `.COM` images, firmware blobs), 32-bit i386 objects and executables, 64-bit ELF objects for Linux and 64-bit COFF objects for Windows. The target is chosen with `@bits(...)` in the source or with command line options.
- **Libraries are optional.** A program can use the C library (`printf`, `malloc`, `qsort`...), or be compiled with `--no-libc`, in which case the compiler supplies a minimal runtime built on Linux system calls. The `nlib` library offers string, memory, math (including fixed-point trigonometry without an FPU) and system-call helpers that work in both modes and on bare metal.
- **Interoperability with C is direct.** ASH functions are ordinary symbols that follow the platform calling convention; C code can call them, and ASH can call any C function declared with `extern`. Global variables declared `general` are visible to C under their own names.
- **The compiler is transparent.** Every program can be dumped as ASHIR, a readable three-address representation, before it is turned into machine code. You can always see what the compiler did with your code.

ASH is not a managed language: there is no garbage collector, no hidden runtime type information and no exceptions. Memory that is allocated is either released by the program or returned to the operating system when the process ends. What ASH does add, by default, is safety where it costs little: array accesses are bounds-checked (see Section 17.7), arrays carry their length, and an optional strict mode (Chapter 21) rejects narrowing assignments and out-of-range literals.

Copyright © 2026 Mallionic Inc. All rights reserved.

1.2 A FIRST LOOK

The following program prints a line through the C library and returns 0 to the operating system. It is complete: there is no header to include, no `main` prototype to repeat and no build script to write.

Example 1-1. A complete ASH program

ASH

```
@Entry
pub func _start(): int64 {
    printf("Hello from ASH!\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Hello from ASH!
```

The `@Entry` annotation marks the function where execution begins. `pub` makes the function visible outside the file. The function returns an `int64`, the return type being written after the parameter list. Everything else is what a C programmer would write.

The next listing shows the same idea at the other end of the spectrum: a function whose body is a block of x86-64 assembly that reads and writes ASH variables by name. The compiler decides where `x` lives (a register or a stack slot) and substitutes the right operand into each instruction.

Example 1-2. Assembly that names ASH variables

ASH

```
@Entry
pub func _start(): int64 {
    int64 x = 40;
    asm {
        add x, 2
    }
    return x;           // exit status 42
}
```

OUTPUT — EXIT STATUS 42

```
(no output)
```

ASHIR

```
[entry]
func qword _start() {
.type x qword
    %x mov[40]
    %_ asm[x0a2020202020202020616464204076617 23a782c20320a202020200a]
    ret qword x
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <_start>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 83 e4 f0             and    rsp,0xfffffffffffffff0
   8:   48 81 ec 20 00 00 00    sub    rsp,0x20
   f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  13:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
17:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
1b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
1f:   b8 28 00 00 00          mov    eax,0x28
24:   49 89 c4                mov    r12,rax
27:   49 83 c4 02             add    r12,0x2
2b:   4c 89 e0                mov    rax,r12
2e:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
32:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
36:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
3a:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
3e:   c9                      leave
3f:   c3                      ret
```

The ASHIR listing shows the `asm` block kept as an opaque hexadecimal payload (`asm[x...]`); the disassembly shows the instruction `add r12, 2` that `shasm` produced after the compiler replaced the name `x` with the register that holds it. Because the variable is used inside an `asm` block, the compiler keeps it in a callee-saved register (`r12`) and the function saves and restores `r12`–`r15` around its body; Chapter 20 explains this in detail.

1.3 WHY USE ASH

Many languages can produce native code for x86 processors. ASH was created for a specific set of needs that existing tools serve only partially.

1.3.1 Close to the machine without leaving the language

In C, inline assembly is a compiler extension with a complex constraint language, and its syntax differs between compilers. In ASH the assembly block is a first-class statement: you write Intel-syntax instructions, you use variable names as operands, and the compiler takes care of the rest. When even that is too much abstraction, `@naked` functions, `emit(...)` and `runnable` byte arrays give you complete control of every byte.

1.3.2 One toolchain for every x86 mode

Writing a boot sector, a protected-mode loader, a 64-bit kernel and a user-space tool usually means juggling an assembler, a cross compiler, a linker script and several sets of flags. `ashc` handles 16, 32 and 64-bit code from the same source language and ships with its own assembler (`shasm`), so a project that spans several processor modes can be written in one language and built with one tool.

1.3.3 Readable output

ASH keeps the gap between source and binary small and visible. The ASHIR representation is plain text and close to the source; the code generator maps each ASHIR operation to a short, predictable instruction sequence. This makes ASH a good language for learning how compilers and processors work, for teaching, and for situations where you must be able to explain every instruction in a binary (boot code, firmware, exploit mitigation research, reverse engineering exercises).

Copyright © 2026 Mallionic Inc. All rights reserved.

1.3.4 Safety by default, control when needed

Arrays know their length and are bounds-checked unless you turn the checks off with `@beautycode(false)` or `@bounds(false)`. Parameters cannot be modified by accident (the compiler makes a private copy when you assign to one). Global variables are constants unless declared `static`. Strict mode adds compile-time checks on literal ranges, return statements and narrowing conversions. Each of these defaults can be relaxed when performance or code size matter more than checks.

1.3.5 Works with the code you already have

ASH objects link with `gcc`, `ld`, `link.exe` and MinGW like any other object file. You can add an ASH module to an existing C project, call C libraries such as ALSA or the Windows API from ASH, or export ASH functions and variables to C.

1.4 WHY THE COMPILER IS WRITTEN IN JAVA

`ashc` and `shasm` are written in Java (about 74,000 lines across the front end, the code generators, the object file writers and the assembler) and are distributed as a self-contained native executable produced with GraalVM `native-image`. The choice of Java was deliberate; the main reasons are summarized here.

1.4.1 Portability of the compiler itself

The compiler must run on the same machines that its users develop on: Linux and Windows on x86-64 today, and other hosts in the future. Java code compiles once and runs unchanged on every supported host. The repository contains both `ashc` (Linux) and `ashc.exe` (Windows) built from the same sources, without conditional compilation and without a porting layer.

1.4.2 A native executable without a runtime dependency

A traditional objection to Java tools is start-up time and the need for an installed JVM. GraalVM `native-image` removes both: the build script (`build.sh native`) compiles the whole compiler ahead of time into a single executable that starts in milliseconds and needs no Java installation. The libraries of `nlib` and the instruction tables of `shasm` are embedded in the executable as data (`tools/EmbedNlib.java` generates `Builder/ash/NlibEmbedded.java`), so the compiler is one file that can be copied anywhere. For development, the same sources also run directly on a JVM (`java arch/Main.java ...` or the jar in `out/`).

1.4.3 Correctness and safety of the tool

A compiler manipulates large, irregular data structures: token lists, syntax trees, symbol tables, relocation lists. In Java an out-of-bounds index or a null reference in the compiler produces a clear exception with a stack trace (shown with `--trace`) instead of silent memory corruption. Bugs in the compiler are therefore reported as *internal errors* (in red) rather than producing a subtly broken binary.

1.4.4 A rich standard library

Several parts of the compiler rely directly on the Java platform: arbitrary-precision arithmetic (`java.math.BigInteger`) for constant folding of `**`, `***` and `%define` expressions without overflow, regular expressions for the ASHIR post-processing passes, `java.nio` for file handling across operating systems, and Unicode-aware string handling for escape decoding and wide (`L"..."`) strings.

Copyright © 2026 Mallionic Inc. All rights reserved.

1.4.5 Fast iteration

Java's tooling (IDEs, debuggers, profilers) and fast incremental compilation make it practical to evolve the language quickly: new statements, annotations and targets are added as small, independent passes over the syntax tree (see Chapter 6).

> **NOTE**
> The language of the implementation has no influence on the code you write or on the code ASH generates. ASH programs do not run on a JVM and do not depend on Java in any way: they become ordinary x86 machine code.

1.5 WHAT ASH CAN DO

The following list summarizes the capabilities of ASH 1.0. Each item is described in detail later in this manual.

Table 1-1. Capabilities of ASH 1.0

Area	Capability
Targets	64-bit ELF relocatable objects (Linux, System V ABI); 64-bit COFF objects (Windows x64 ABI); 32-bit ELF and COFF objects (i386); 32-bit Linux executables (`--exe`); 16-bit and 32-bit flat binaries (`--compile`).
Types	Signed and unsigned integers of 8, 16, 32 and 64 bits under many names (`int64`, `uint8`, `unsigned long long`, `size_t`, ...), `bool`, `char`, `string`, raw pointers, arrays, two-dimensional arrays and matrices, C-compatible structures, function pointers.
Control flow	`if` / `elif` / `else`, `while`, `for`, `repeat (n)`, `loop`, `switch` with fall-through, `break`, `continue`, `return`, `terminate` (leave the function from any depth) and `unreturnable` (pending return values).
Functions	Typed parameters, any number of arguments (beyond the register arguments the stack is used), variadic functions with `va_count()` / `va_arg(i)`, function pointers and callbacks, classes with `pub` / `priv` methods, cracks (named configuration entry points inside a function).
Low level	`asm { }` blocks, GCC-style `asm volatile("..." : outputs : inputs)`, `emit` of raw bytes and words, `@naked` functions, executable and writable `runnable` arrays, custom ELF sections with `@section`, Microsoft x64 calling convention with `@msabi` / `@efiapi`.
Compile time	`%if` / `%ifdef` / `%define` conditional compilation with arithmetic, `define` constants, `typedef`, `redefine` aliases, user-defined annotations whose bodies are inserted into functions, override rights to choose between competing definitions.
Libraries	`nlib`: strings and memory, integer and Q16.16 fixed-point math, Linux system calls without libc, command line arguments, delays, C `stdio` and `stdlib` declarations. Any C library through `extern`.
Tooling	ASHIR dump, token dump, per-stage timings, colored diagnostics with tips, the stand-alone `shasm` assembler with listings, a VS Code extension with highlighting, snippets and a live linter.

Copyright © 2026 Mallionic Inc. All rights reserved.

1.6 GENERAL PURPOSE AND DESIGN GOALS

ASH is a general-purpose systems language. It is suitable for command line tools, libraries called from C, audio and graphics experiments, operating system and boot loader development, embedded firmware images and teaching. The design is guided by a small set of goals:

1. **Predictability.** A construct should translate to code whose shape the programmer can anticipate. There is no implicit heap allocation except for arrays and strings, no hidden copy constructors and no implicit virtual dispatch.
2. **Transparency.** Every intermediate result of the compiler can be inspected: tokens (`--tokens`), ASHIR (the default output), the final machine code (with any disassembler) and timings (`--debug`).
3. **Familiarity.** Programmers who know C should be able to read ASH immediately. Where ASH differs (return type after the parameters, `elif`, `repeat`, `loop`, `**`), the difference adds something rather than renaming something.
4. **Reach.** The same language should be usable from the first instruction executed by a processor after reset to a user-space program that calls a C library.
5. **Safety by default.** Checks that catch common mistakes (array bounds, constant globals, read-only parameters) are on unless explicitly turned off.

1.7 WHO SHOULD READ THIS MANUAL

The manual assumes that you can program in at least one imperative language. Knowledge of C helps but is not required; every construct is explained from first principles. Knowledge of x86 assembly is not required for the language chapters; Chapter 25 introduces the processor concepts used in the rest of the book, and Chapter 26 teaches the assembler syntax.

- **Beginners** should start with Chapter 2, then follow the tour in Chapter 4, keep the dictionary of Chapter 33 at hand, and read Chapter 32 for the most common mistakes.
- **Application programmers** should read Chapters 3 to 16 (starting with the tour in Chapter 4) and use the library reference in Chapter 27 and the cookbook in Chapter 30.
- **Systems programmers** (boot loaders, kernels, firmware) should add Chapter 17, Chapter 20, Chapter 22 and Chapter 26.
- **Compiler and language enthusiasts** will find the internals in Chapter 6, Chapter 23 and Chapter 24.

Copyright © 2026 Mallionic Inc. All rights reserved.

1.8 HOW THIS MANUAL IS ORGANIZED

Table 1-2. Organization of this manual

Part	Contents
Chapters 1–2	Introduction to ASH, and programming basics for complete beginners (numbers, memory, compiling, the parts of a program, exercises).
Chapters 3–5	Installation and first programs, a guided tour of the language with every line explained, and the complete command line reference of `ashc`.
Chapter 6	Architecture of the compiler: every stage from source text to object file.
Chapters 7–16	The language: lexical structure, types, variables, expressions, control flow, functions, arrays and strings, structures and pointers, classes, cracks.
Chapters 17–21	Annotations, compile-time directives and macros, modules and imports, inline assembly and machine code, strict mode.
Chapters 22–24	Targets and calling conventions, the ASHIR intermediate representation, code generation for x86.
Chapters 25–26	An x86 primer for ASH programmers and the complete guide to the `shasm` assembler.
Chapter 27	Reference of every library and function of `nlib`, with an example and the machine code of each function.
Chapters 28–30	Diagnostics, tooling (VS Code, build scripts, emulators) and a cookbook of complete programs.
Chapter 31	Common mistakes with their corrections, techniques to master ASH, and frequently asked questions.
Chapter 32	The language dictionary: every keyword, operator and built-in, with an example.
Appendices	x86 instruction reference generated from the `shasm` instruction database, register reference, grammar summary, example index, limitations, glossary, ASH for C programmers, quick reference.

1.9 NOTATIONAL CONVENTIONS

1.9.1 Typography

- `Monospaced text` denotes source code, file names, command line options, ASHIR operations and machine instructions.
- *Italic text* introduces a new term or names a placeholder that you replace with your own text, as in `import <`*name*`>;`.
- **Bold text** marks important rules and warnings.

1.9.2 Code listings

Listings use colored boxes. A label in the upper right corner tells what the box contains:

Label	Meaning
ASH	ASH source code. Every listing labelled ASH was compiled by `ashc` during the production of this manual.
OUTPUT	What the program printed when it was run, followed by its exit status.

Copyright © 2026 Mallionic Inc. All rights reserved.

Label	Meaning
ASHIR	The intermediate representation produced by `ashc file.ash` (runtime helper routines are omitted unless they are the subject of the example).
X86-64 DISASSEMBLY	The machine code of the object file as shown by `objdump -d -M intel`: address, encoded bytes, instruction.
SHASM	Assembly source for the `shasm` assembler.
SHELL	Commands typed at a shell prompt.
COMPILER DIAGNOSTICS	Errors or warnings printed by `ashc`.

1.9.3 Numbers and sizes

Hexadecimal numbers are written with the prefix `0x` (as in ASH source) or, in tables of instruction encodings, as bare pairs of hexadecimal digits (`48 89 E5`). Binary numbers use the prefix `0b`. A *byte* is 8 bits, a *word* 16 bits, a *doubleword* (`dword`) 32 bits and a *quadword* (`qword`) 64 bits, following the x86 tradition that ASHIR also uses.

1.9.4 Syntax notation

Grammar fragments use the following notation: `[ x ]` means *x* is optional, `{ x }` means zero or more repetitions of *x*, `a | b` means either *a* or *b*, and *italic* words are non-terminals. Literal symbols are shown in monospace.

1.10 RELATED SOFTWARE AND DOCUMENTS

- *System V Application Binary Interface, AMD64 Architecture Processor Supplement* — the calling convention used for 64-bit Linux targets.
- *Microsoft x64 calling convention* — used for `--win` objects and `@msabi` / `@efiapi` functions.
- *Intel® 64 and IA-32 Architectures Software Developer's Manual* — the authoritative description of the instructions that `shasm` encodes.
- *Executable and Linkable Format (ELF)* and *Microsoft PE/COFF Specification* — the object formats written by `ashc`.
- The source tree of The Ashley Project: `Builder/` (front end), `arch/` (back ends and object writers), `shasm/` (assembler), `nlib/` (libraries), `examples/`, `tests/` and `vscode-ash/`.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 2

PROGRAMMING BASICS

This chapter is for readers who have never programmed, or who want a refresher of the basic ideas. It explains what a computer does with a program, how numbers and text are stored, what a compiler is, and the words used in the rest of this manual. Every idea is shown with a small ASH program. If you already program in another language, you can skim this chapter and continue with Chapter 3.

2.1 WHAT A COMPUTER DOES

A computer has two main parts that matter to a programmer:

- The **processor** (CPU) executes *instructions*: very small steps such as "add these two numbers", "copy this value", "jump to that instruction if the result was zero". A modern processor executes billions of them per second.
- The **memory** (RAM) stores numbers. It is a long row of cells called *bytes*; each byte has a number called its **address**, starting from 0. Both the instructions of a program and its data live in memory.

The processor also has a few very fast storage places of its own, called **registers** (`rax`, `rbx`, `rcx` ... on x86 processors). Calculations happen in registers; values travel between memory and registers.

A **program** is a list of instructions in memory. Writing those instructions directly as numbers is possible but very hard. A **programming language** such as ASH lets you describe what you want in words and symbols, and a **compiler** translates that description into processor instructions.

2.2 BITS, BYTES AND NUMBERS

2.2.1 Bits and bytes

A **bit** is the smallest piece of information: 0 or 1. Eight bits form a **byte**, which can hold 2^8 = 256 different values, from 0 to 255. Larger numbers use several bytes: 2 bytes (16 bits), 4 bytes (32 bits) or 8 bytes (64 bits). The more bits, the larger the numbers that fit:

Table 2-1. How many values fit in N bits

Bits	Bytes	Unsigned range	Signed range
8	1	0 to 255	-128 to 127
16	2	0 to 65 535	-32 768 to 32 767
32	4	0 to 4 294 967 295	about -2.1 to 2.1 billion
64	8	0 to about 1.8 * 10^19	about -9.2 to 9.2 * 10^18

In ASH you choose the size of every number you store, with types such as `uint8` (an unsigned number of 8 bits) or `int64` (a signed number of 64 bits). Chapter 8 lists them all.

Copyright © 2026 Mallionic Inc. All rights reserved.

2.2.2 Binary and hexadecimal

Computers count in **binary** (base 2): the number 13 is `1101` in binary, because 13 = 8 + 4 + 1. Binary numbers are long, so programmers often use **hexadecimal** (base 16), which uses the digits `0`-`9` and the letters `A`-`F` (10 to 15). One hexadecimal digit is exactly four bits, so a byte is always two hexadecimal digits: 255 is `FF`, 16 is `10`, 13 is `0D`. In ASH a hexadecimal number is written with the prefix `0x`: `0xFF`.

Example 2-1. The same numbers in three bases

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[65] text;
    int64[] numbers = {13, 100, 255, 4096};
    for (int64 i = 0; i < #numbers; i++) {
        int64 n = numbers[i];
        printf("decimal %5d = hexadecimal %4s", n, uint_to_str(n, &text, 16));
        printf(" = binary %s\n", uint_to_str(n, &text, 2));
    }
    printf("0xFF is %d and 0x10 is %d\n", 0xFF, 0x10);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
decimal    13 = hexadecimal    d = binary 1101
decimal   100 = hexadecimal   64 = binary 1100100
decimal   255 = hexadecimal   ff = binary 11111111
decimal  4096 = hexadecimal 1000 = binary 1000000000000
0xFF is 255 and 0x10 is 16
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 2-2. Powers of two

n	2^n (decimal)	2^n (hexadecimal)	Where it appears
0	1	0x1	
1	2	0x2	
2	4	0x4	
3	8	0x8	
4	16	0x10	one hexadecimal digit
5	32	0x20	
6	64	0x40	
7	128	0x80	
8	256	0x100	values of one byte (0..255)
9	512	0x200	
10	1024	0x400	one kibibyte (1 KiB = 1024 bytes)
11	2048	0x800	
12	4096	0x1000	one memory page (4096 bytes)
13	8192	0x2000	
14	16384	0x4000	
15	32768	0x8000	
16	65536	0x10000	values of a 16-bit integer
20	1048576	0x100000	one mebibyte (1 MiB)
24	16777216	0x1000000	
30	1073741824	0x40000000	one gibibyte (1 GiB)
31	2147483648	0x80000000	limit of int32
32	4294967296	0x100000000	values of a 32-bit integer
40	1099511627776	0x10000000000	
48	281474976710656	0x1000000000000	
63	9223372036854775808	0x8000000000000000	limit of int64
64	18446744073709551616	0x10000000000000000	values of a 64-bit integer

2.2.3 Negative numbers

Signed types store negative numbers in **two's complement**: -1 is "all bits set", -2 is all bits except the lowest, and so on. You rarely need to think about this, but it explains why an unsigned byte with all bits set reads as 255 while a signed byte with the same bits reads as -1:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 2-2. One bit pattern, two meanings

ASH

```
@Entry
pub func main(): int64 {
    uint8 u = 0xFF;          // the bits 11111111
    int8 s = 0;
    s = s - 1;               // also the bits 11111111
    int64 wu = u;
    int64 ws = s;
    printf("as unsigned: %d   as signed: %d\n", wu, ws);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
as unsigned: 255   as signed: -1
```

2.2.4 Text is numbers too

Each character of text is stored as a number, its **character code**. ASH, like most languages, uses the ASCII codes for the basic characters: `A` is 65, `a` is 97, the digit `0` is 48, a space is 32 and the end of a line is 10. A text (a *string*) is a row of such codes in memory, ended by a byte with the value 0.

Example 2-3. Characters are numbers

ASH

```
@Entry
pub func main(): int64 {
    string word = "Hi!";
    printf("'%c' has code %d\n", word[0], word[0]);
    printf("'%c' has code %d\n", word[1], word[1]);
    printf("'%c' has code %d\n", word[2], word[2]);
    printf("the byte after the text is %d\n", word[3]);
    printf("'A' + 1 is '%c'\n", 'A' + 1);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
'H' has code 72
'i' has code 105
'!' has code 33
the byte after the text is 0
'A' + 1 is 'B'
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 2-3. ASCII character codes

Dec	Hex	Bin	Char	Dec	Hex	Bin	Char
0	00	0000000	NUL	64	40	1000000	@
1	01	0000001	control	65	41	1000001	A
2	02	0000010	control	66	42	1000010	B
3	03	0000011	control	67	43	1000011	C
4	04	0000100	control	68	44	1000100	D
5	05	0000101	control	69	45	1000101	E
6	06	0000110	control	70	46	1000110	F
7	07	0000111	BEL (\a)	71	47	1000111	G
8	08	0001000	BS (\b)	72	48	1001000	H
9	09	0001001	TAB (\t)	73	49	1001001	I
10	0A	0001010	LF (\n)	74	4A	1001010	J
11	0B	0001011	VT (\v)	75	4B	1001011	K
12	0C	0001100	FF (\f)	76	4C	1001100	L
13	0D	0001101	CR (\r)	77	4D	1001101	M
14	0E	0001110	control	78	4E	1001110	N
15	0F	0001111	control	79	4F	1001111	O
16	10	0010000	control	80	50	1010000	P
17	11	0010001	control	81	51	1010001	Q
18	12	0010010	control	82	52	1010010	R
19	13	0010011	control	83	53	1010011	S
20	14	0010100	control	84	54	1010100	T
21	15	0010101	control	85	55	1010101	U
22	16	0010110	control	86	56	1010110	V
23	17	0010111	control	87	57	1010111	W
24	18	0011000	control	88	58	1011000	X
25	19	0011001	control	89	59	1011001	Y
26	1A	0011010	control	90	5A	1011010	Z
27	1B	0011011	ESC	91	5B	1011011	[
28	1C	0011100	control	92	5C	1011100	\
29	1D	0011101	control	93	5D	1011101	]
30	1E	0011110	control	94	5E	1011110	^
31	1F	0011111	control	95	5F	1011111	_
32	20	0100000	space	96	60	1100000	backquote
33	21	0100001	!	97	61	1100001	a
34	22	0100010	"	98	62	1100010	b

Copyright © 2026 Mallionic Inc. All rights reserved.

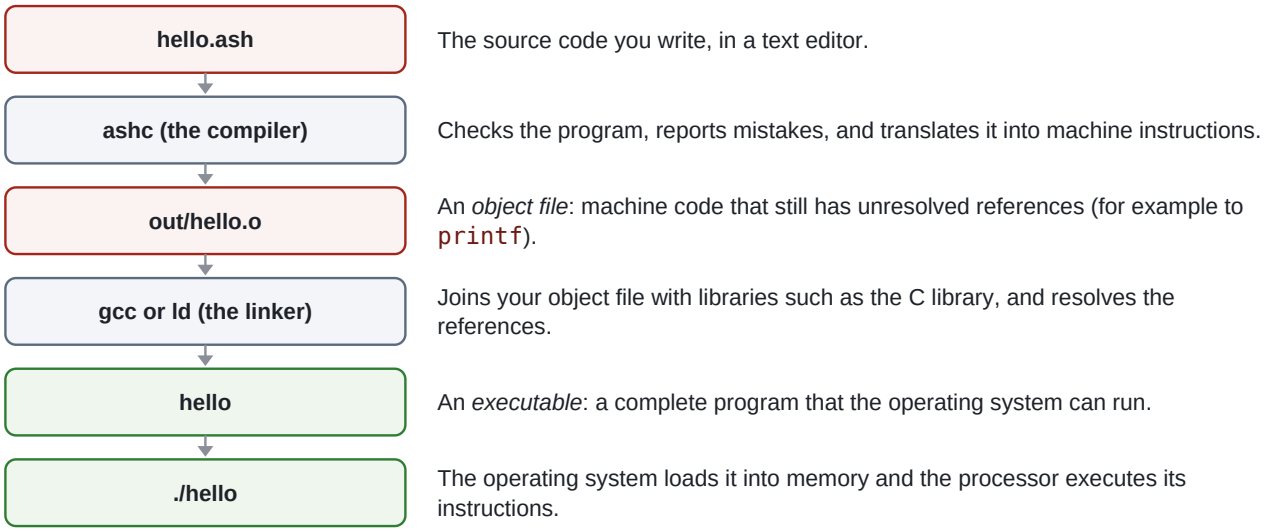
Dec	Hex	Bin	Char	Dec	Hex	Bin	Char
35	23	0100011	#	99	63	1100011	c
36	24	0100100	$	100	64	1100100	d
37	25	0100101	%	101	65	1100101	e
38	26	0100110	&	102	66	1100110	f
39	27	0100111	'	103	67	1100111	g
40	28	0101000	(	104	68	1101000	h
41	29	0101001	)	105	69	1101001	i
42	2A	0101010	*	106	6A	1101010	j
43	2B	0101011	+	107	6B	1101011	k
44	2C	0101100	,	108	6C	1101100	l
45	2D	0101101	-	109	6D	1101101	m
46	2E	0101110	.	110	6E	1101110	n
47	2F	0101111	/	111	6F	1101111	o
48	30	0110000	0	112	70	1110000	p
49	31	0110001	1	113	71	1110001	q
50	32	0110010	2	114	72	1110010	r
51	33	0110011	3	115	73	1110011	s
52	34	0110100	4	116	74	1110100	t
53	35	0110101	5	117	75	1110101	u
54	36	0110110	6	118	76	1110110	v
55	37	0110111	7	119	77	1110111	w
56	38	0111000	8	120	78	1111000	x
57	39	0111001	9	121	79	1111001	y
58	3A	0111010	:	122	7A	1111010	z
59	3B	0111011	;	123	7B	1111011	{
60	3C	0111100	<	124	7C	1111100	vertical bar
61	3D	0111101	=	125	7D	1111101	}
62	3E	0111110	>	126	7E	1111110	~
63	3F	0111111	?	127	7F	1111111	DEL

2.3 FROM SOURCE CODE TO A RUNNING PROGRAM

The text you write is called **source code**. It is stored in a file whose name ends in `.ash`. Several tools turn it into a program:

Copyright © 2026 Mallionic Inc. All rights reserved.

Figure 2-1. From source code to a running program



If the compiler finds a mistake, it prints an **error message** and produces no object file. Reading error messages is a normal part of programming; Chapter 28 explains the common ones.

2.4 THE PARTS OF AN ASH PROGRAM

Here is a small but complete program. The table after it explains every line.

Example 2-4. A complete program, explained

ASH

```ash
// A program that adds the numbers from 1 to 10
func add_up_to(int64 limit): int64 {
    int64 total = 0;
    for (int64 n = 1; n <= limit; n++) {
        total = total + n;
    }
    return total;
}

@Entry
pub func main(): int64 {
    int64 result = add_up_to(10);
    printf("1 + 2 + ... + 10 = %d\n", result);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 + 2 + ... + 10 = 55
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 2-4. Line by line

Line	Code	Explanation
1	`// A program that ...`	A **comment**: text for people. The compiler ignores everything after `//` on the line.
2	`func add_up_to(int64 limit): int64 {`	Starts a **function** named `add_up_to`. It receives one value, called `limit`, of type `int64`, and gives back (*returns*) an `int64`. The `{` opens the function's **body**.
3	`int64 total = 0;`	Creates a **variable** named `total` that holds an `int64`, starting at 0. The `;` ends the **statement**.
4	`for (int64 n = 1; n <= limit; n++) {`	A **loop**: create `n` with the value 1; while `n <= limit`, run the body; after each run, add 1 to `n` (`n++`).
5	`total = total + n;`	Takes the current `total`, adds `n`, and stores the result back in `total`.
6	`}`	Closes the loop body.
7	`return total;`	Ends the function and gives `total` back to whoever called it.
8	`}`	Closes the function.
9	(empty)	Blank lines only make the code easier to read.
10	`@Entry`	An **annotation**: the next function is where the program starts.
11	`pub func main(): int64 {`	The function `main`. `pub` makes it visible to the operating system's start-up code. It receives nothing (empty parentheses) and returns an `int64`.
12	`int64 result = add_up_to(10);`	**Calls** `add_up_to` with the value 10 and stores what it returns in `result`.
13	`printf("... %d\n", result);`	Prints a text. `%d` is replaced by the value of `result`, and `\n` ends the line.
14	`return 0;`	Returns 0 from `main`, which tells the operating system that the program succeeded.
15	`}`	Closes `main`.

Copyright © 2026 Mallionic Inc. All rights reserved.

2.5 WORDS YOU WILL MEET

Table 2-5. Basic vocabulary

Word	Meaning
statement	One instruction of the program, usually ended by `;`: a declaration, an assignment, a call, an `if`, a loop...
expression	Something that produces a value: `3`, `x`, `x + 1`, `square(4)`.
variable	A named place in memory that holds a value of a given type. Its value can change.
type	What kind of value a variable holds and how many bytes it uses: `int64`, `uint8`, `string`...
literal	A value written directly in the code: `42`, `0xFF`, `'A'`, `"text"`.
operator	A symbol that combines values: `+`, `-`, `*`, `/`, `==`, `<`...
function	A named group of statements that receives values (*parameters*), does some work and may return a value.
call	Running a function: `square(4)`.
argument	A value given to a function in a call (`4` in `square(4)`).
parameter	The name, inside a function, that receives an argument (`x` in `func square(int64 x)`).
return value	The value a function gives back with `return`.
block	Statements between `{` and `}`.
condition	A question that is true or false, used by `if` and loops: `x > 3`.
loop	A statement that repeats a block: `while`, `for`, `repeat`, `loop`.
array	A row of values of the same type, accessed by position: `v[0]`, `v[1]`...
string	A text.
pointer	A variable that holds a memory address.
library	A collection of ready-made functions that a program can use, brought in with `import`.
compiler	The program that translates source code into machine code (`ashc`).

2.6 TWO KINDS OF MISTAKES

Compile-time errors are found by the compiler before the program runs: a misspelled name, a missing `}`, a number too big for its type. The compiler tells you the problem, often with the line, and no program is produced.

Example 2-5. A compile-time error: a misspelled variable

ASH

```
@Entry
pub func main(): int64 {
    int64 total = 5;
    printf("%d\n", totl);
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

COMPILER DIAGNOSTICS

```
error: unknown symbol 'totl'
tip: check the spelling, or declare it before use
1 error generated.
```

Run-time errors happen while the program runs: dividing by zero, using a position outside an array. ASH stops the program in a controlled way when an array position is wrong:

Example 2-6. A run-time error: a position outside an array

ASH

```
@Entry
pub func main(): int64 {
    int64[] v = {10, 20, 30};
    printf("v[2] = %d\n", v[2]);     // positions 0, 1 and 2 exist
    fflush(0);
    printf("v[3] = %d\n", v[3]);     // there is no position 3: the program stops
    return 0;
}
```

OUTPUT — EXIT STATUS 134

```
v[2] = 30
```

The program stopped with exit status 134 at the wrong position, before the second line could be printed.

2.7 EXERCISES

Try to write each program yourself before reading the solution. Every solution was compiled and run.

2.7.1 Exercise 1: multiplication table

Print the multiplication table of 7, from 7 x 1 to 7 x 10.

Example 2-7. Solution 1

ASH

```
@Entry
pub func main(): int64 {
    for (int64 i = 1; i <= 10; i++) {
        printf("7 x %2d = %2d\n", i, 7 * i);
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
7 x  1 =  7
7 x  2 = 14
7 x  3 = 21
7 x  4 = 28
7 x  5 = 35
7 x  6 = 42
7 x  7 = 49
7 x  8 = 56
7 x  9 = 63
7 x 10 = 70
```

Copyright © 2026 Mallionic Inc. All rights reserved.

2.7.2 Exercise 2: even or odd

Write a function `is_even` that returns 1 for even numbers and 0 for odd numbers, and print the result for 0 to 5.

Example 2-8. Solution 2

ASH

```
func is_even(int64 n): int64 {
    if (n % 2 == 0) {
        return 1;
    }
    return 0;
}

@Entry
pub func main(): int64 {
    for (int64 n = 0; n <= 5; n++) {
        if (is_even(n) == 1) {
            printf("%d is even\n", n);
        } else {
            printf("%d is odd\n", n);
        }
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0 is even
1 is odd
2 is even
3 is odd
4 is even
5 is odd
```

2.7.3 Exercise 3: largest value

Find the largest value of the array `{4, 19, 7, 23, 11}`.

Example 2-9. Solution 3

ASH

```
@Entry
pub func main(): int64 {
    int64[] values = {4, 19, 7, 23, 11};
    int64 largest = values[0];
    for (int64 i = 1; i < #values; i++) {
        if (values[i] > largest) {
            largest = values[i];
        }
    }
    printf("the largest value is %d\n", largest);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
the largest value is 23
```

Copyright © 2026 Mallionic Inc. All rights reserved.

2.7.4 Exercise 4: counting letters

Count how many times the letter a appears in the text "banana and papaya".

Example 2-10. Solution 4

ASH

```
@Entry
pub func main(): int64 {
    string text = "banana and papaya";
    int64 count = 0;
    int64 i = 0;
    while (text[i] != 0) {          // a text ends with the code 0
        if (text[i] == 'a') {
            count++;
        }
        i++;
    }
    printf("'a' appears %d times\n", count);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
'a' appears 7 times
```

2.7.5 Exercise 5: temperature conversion

Print a table that converts Celsius degrees from -10 to 40, in steps of 10, to Fahrenheit (F = C * 9 / 5 + 32).

Example 2-11. Solution 5

ASH

```
func to_fahrenheit(int64 c): int64 {
    return c * 9 / 5 + 32;
}

@Entry
pub func main(): int64 {
    for (int64 c = -10; c <= 40; c = c + 10) {
        printf("%4d C = %4d F\n", c, to_fahrenheit(c));
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
 -10 C =   14 F
   0 C =   32 F
  10 C =   50 F
  20 C =   68 F
  30 C =   86 F
  40 C =  104 F
```

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 3

GETTING STARTED

This chapter explains how to build and install the ASH toolchain, how to compile, link and run your first programs on each supported target, and how to set up the editor support. At the end of the chapter you will be able to turn any `.ash` file into a running program.

3.1 REQUIREMENTS

3.1.1 Running the compiler

The compiler is distributed as a single native executable:

Table 3-1. Compiler executables

File	Description
`ashc`	Native executable for Linux x86-64, built with GraalVM `native-image`. No Java installation is needed to run it.
`ashc.exe`	Native executable for Windows x64, built from the same sources.
`out/ashc-1.0.0.jar`	The compiler as a Java archive. Runs on any host with a Java 25 (or newer) runtime: `java -jar ashc-1.0.0.jar file.ash ...`

3.1.2 Linking and running programs

`ashc` produces object files, flat binaries and (for 32-bit Linux) stand-alone executables. To turn an object file into a program you also need a linker:

Table 3-2. Linkers used with ashc

Target	Tools
Linux x86-64 (`--elf`)	`gcc` (recommended when you use the C library) or GNU `ld` from binutils (programs built with `--no-libc`).
Linux i386 (`@bits(32)` + `--elf`)	`gcc -m32` with the 32-bit C library installed (`gcc-multilib` on Debian and Ubuntu), or `ld -m elf_i386`.
Windows x64 (`--win`)	Microsoft `link.exe`, or MinGW-w64 `x86_64-w64-mingw32-gcc` / `ld` on Linux.
Flat binaries (`--compile`)	No linker. The output is loaded directly: by a boot loader, by BIOS (boot sectors), by an emulator or by your own loader.
Linux i386 executable (`--exe`)	No linker. `ashc` writes the ELF header itself.

The `objdump` program from binutils is useful to inspect the generated machine code; all disassembly listings in this manual were produced with it.

Copyright © 2026 Mallionic Inc. All rights reserved.

3.1.3 Building the compiler from source

Building `ashc` requires a GraalVM JDK with `native-image` (GraalVM for JDK 25 was used for this release), `javac`, `jar` and the C toolchain that `native-image` uses (on Debian-based systems: `build-essential` and `zlib1g-dev`). The build script can check and install missing dependencies for you.

3.2 BUILDING AND INSTALLING

The repository contains `build.sh` (Linux and macOS shells) and `build.bat` (Windows). Both drive the same steps. Run the script without arguments, or with `help`, to see the list of commands:

Table 3-3. build.sh commands

Command	Action
`all`	`clean`, `compile`, `jar`, `native`, `test` and `sdk` in one go.
`clean`	Remove the `out/` directory.
`compile`	Embed `nlib` into the compiler (runs `tools/EmbedNlib.java`, which regenerates `Builder/ash/NlibEmbedded.java`) and compile every Java source into `out/classes`.
`jar`	Build `out/ashc-<version>.jar`.
`native`	Build the native executable `out/ashc` with GraalVM `native-image`. The build log is written to `out/native-image.log`.
`agent`	Run the compiler under the GraalVM tracing agent to regenerate the native-image reflection configuration.
`test`	Run a smoke test of the native binary with `test.ash`.
`sdk`	Assemble the SDK in `out/dist/ashc-<version>/` and a compressed archive `out/ashc-<version>-<platform>.tar.gz` with its SHA-256 checksum.
`install`	Install the SDK under the prefix (default `~/.local`): `bin/ashc`, `lib/ashc-<version>.jar`, `share/doc/ashc`, `share/examples`.
`deps`	Check the required tools and optionally install the missing ones.
`version`	Print the version that will be built.

Table 3-4. build.sh options and environment variables

Option / variable	Meaning
`--prefix DIR` / `PREFIX`	Installation prefix used by `install` (default `$HOME/.local`).
`--quick` / `QUICK=1`	Faster native compilation (`native-image -Ob`); the binary is slightly slower.
`--fast-init` / `FAST_INIT=1`	Initialize selected classes at image build time (faster start-up).
`-v`, `--verbose` / `VERBOSE=1`	List every Java source file while compiling.
`VERSION`	Version string to build (otherwise read from the file `VERSION`, or `1.0.0`).
`MAIN_CLASS`	Main class of the jar (the driver `arch.Main` by default).

A complete build and installation looks like this:

Copyright © 2026 Mallionic Inc. All rights reserved.

```shell
./build.sh deps          # check javac, jar, native-image, zlib headers ...
./build.sh all           # clean + compile + jar + native + test + sdk
./build.sh install       # copy bin/ashc into ~/.local/bin
export PATH="$HOME/.local/bin:$PATH"
```

NOTE

The libraries of `nlib` are embedded in the compiler at build time. If you edit a file in `nlib/`, rebuild the compiler, or keep an `nlib/` directory next to your project: a library found on disk always takes precedence over the embedded copy (see Section 19.2).

3.3 VERIFYING THE INSTALLATION

Ask the compiler for its version and for the list of embedded libraries:

```shell-session
$ ashc --version
ashc 1.0.0
This is The Ashley Project!
Licensed under GNU GPL v3.0
Copyright (C) 2026 Mallionic Inc.
$ ashc --list-nlib
nlib embebida en el compilador:
  <args>   (args.ash)
  <delay>   (delay.ash)
  <math>   (math.ash)
  <null>   (null.ash)
  <stdbool>   (stdbool.ash)
  <stdio>   (stdio.ash)
  <stdlib>   (stdlib.ash)
  <string>   (string.ash)
  <syscalls>   (syscalls.ash)
  <types>   (types.ash)
```

`ashc --help` prints a summary of every option; Chapter 5 describes them in full.

3.4 YOUR FIRST PROGRAM

Create a file named `hello.ash` with the following content:

```hello.ash
@Entry
pub func main(): int64 {
    printf("Hello, world!\n");
    return 0;
}
```

There are four things to notice:

1 `@Entry` marks the entry point of the program. Exactly one function per program carries it.

2 `pub` makes `main` a public symbol of the object file, so the C start-up code can find it.

3 The return type comes after the parameter list: `(): int64`.

Copyright © 2026 Mallionic Inc. All rights reserved.

4 `printf` is not declared anywhere. When a program calls a function that is neither an ASH function nor declared with `extern`, ASH assumes it is a C function with a variable argument list and emits a call to the external symbol. The C library linked by `gcc` provides it.

3.4.1 Step 1: compile to an object file

SHELL SESSION

```
$ ashc hello.ash --elf
[BUILD SUCCESS] File has been compiled: out/hello.o
$ ls out
hello.o
```

With `--elf` the compiler writes a 64-bit ELF relocatable object into the directory `out/` next to the source file. The name of the object is the name of the source with the extension `.o`.

3.4.2 Step 2: link and run

SHELL SESSION

```
$ ashc hello.ash --elf -s
$ gcc out/hello.o -o hello
$ ./hello
Hello, world!
$ echo "exit status: $?"
exit status: 0
```

(The option `-s` makes the compiler silent on success.) `gcc` links the object with the C library and the C start-up files. The start-up code calls `main`, and the value that `main` returns becomes the exit status of the process.

3.4.3 Looking inside: ASHIR and machine code

Without an output option the compiler stops after the front end and writes the ASHIR intermediate representation into `hello.ashir`:

SHELL SESSION

```
$ ashc hello.ash
[BUILD SUCCESS] File has been compiled: hello.ashir
$ cat hello.ashir
[entry]
func qword main() {
    %__tmp0 addr[__str0]
    call[printf, __tmp0]
    ret qword 0
}

.string __str0 "Hello, world!\n"
```

The listing shows the function `main` marked as the entry (`[entry]`), the string literal stored as `__str0`, a call to the external `printf` and the return of the constant 0. Chapter 23 documents this format completely. The machine code can be seen with `objdump`:

Copyright © 2026 Mallionic Inc. All rights reserved.

SHELL SESSION

```
$ ashc hello.ash --elf -s
$ objdump -d -M intel --no-show-raw-insn out/hello.o

out/hello.o:     file format elf64-x86-64


Disassembly of section .text:

0000000000000000 <main>:
   0:   push   rbp
   1:   mov    rbp,rsp
   4:   sub    rsp,0x20
   b:   mov    QWORD PTR [rbp-0x8],r12
   f:   mov    QWORD PTR [rbp-0x10],r13
  13:   mov    QWORD PTR [rbp-0x18],r14
  17:   mov    QWORD PTR [rbp-0x20],r15
  1b:   mov    eax,0x0
  20:   mov    r12,rax
  23:   lea    r12,[rip+0x26]        # 50 <__str0>
  2a:   mov    rdi,r12
  2d:   mov    eax,0x0
  32:   call   37 <main+0x37>
  37:   xor    rax,rax
  3a:   mov    r12,QWORD PTR [rbp-0x8]
  3e:   mov    r13,QWORD PTR [rbp-0x10]
  42:   mov    r14,QWORD PTR [rbp-0x18]
  46:   mov    r15,QWORD PTR [rbp-0x20]
  4a:   mov    rsp,rbp
  4d:   pop    rbp
  4e:   ret
    ...

0000000000000050 <__str0>:
  50:   rex.W
  51:   gs ins BYTE PTR es:[rdi],dx
  53:   ins    BYTE PTR es:[rdi],dx
  54:   outs   dx,DWORD PTR ds:[rsi]
  55:   sub    al,0x20
  57:   ja     c8 <__str0+0x78>
  59:   jb     c7 <__str0+0x77>
  5b:   and    DWORD PTR fs:[rdx],ecx
    ...
```

3.5 ENTRY POINTS AND LINKING RECIPES

The name of the `@Entry` function decides how the program is linked. Two names are common:

- `main` — the C start-up code (`crt1.o`) runs first, initializes the C library and calls `main`. The return value becomes the exit status. **Use `main` whenever the program uses the C library.**
- `_start` — the function is the first instruction of the process. There is no caller to return to, so the program must finish by calling `exit(...)` (or `sys_exit(...)` from `<syscalls>`). This is the natural choice for `--no-libc` programs, for boot code and for kernels.

WARNING

A `_start` function that simply returns crashes: the stack of a new process does not contain a return address. When the entry function is called `_start`, always end the program with `exit(code);`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Many examples in the `examples/` directory use `_start` with the C library. They are linked by renaming the symbol, which turns `_start` into `main` inside the object file:

```shell
ashc prog.ash --elf
objcopy --redefine-sym _start=main out/prog.o out/prog_main.o
gcc -no-pie out/prog_main.o -o prog
```

The examples of this manual that are shown with their output were linked exactly this way when they define `_start`, and with a plain `gcc` command when they define `main`.

Table 3-5. Linking recipes by target

Program kind	Commands
64-bit, C library, entry `main`	`ashc p.ash --elf` then `gcc out/p.o -o p`
64-bit, C library, entry `_start`	`objcopy --redefine-sym _start=main out/p.o m.o` then `gcc -no-pie m.o -o p`
64-bit, no C library	`ashc p.ash --elf --no-libc` then `ld -e _start out/p.o -o p`
32-bit, C library	`@bits(32)` in the source, `ashc p.ash --elf`, then `gcc -m32 -no-pie -z noexecstack out/p.o -o p`
32-bit, no C library	`ashc p.ash --elf --no-libc` then `ld -m elf_i386 -e <entry> ... out/p.o -o p`
32-bit stand-alone executable	`@bits(32)` and `@originoffs(0x08048000)`, then `ashc p.ash --exe`
16-bit or 32-bit flat binary	`@bits(16)` (or 32), then `ashc p.ash --compile`; load the `.bin` file yourself
Windows x64	`ashc p.ash --win`, then `link /subsystem:console /entry:main p.obj kernel32.lib` or `x86_64-w64-mingw32-gcc p.obj -o p.exe -nostartfiles -Wl,-e,main -lkernel32`

3.6 A PROGRAM THAT USES ITS EXIT STATUS

The exit status is the simplest way for a program to report a result without printing anything. Shells keep it in `$?`. The value is truncated to 8 bits by the operating system, so only 0 to 255 are meaningful.

Example 3-1. Returning a computed value

```ash
func square(int64 x): int64 {
    return x * x;
}

@Entry
pub func main(): int64 {
    return square(6) + square(2);     // 36 + 4 = 40
}
```

OUTPUT — EXIT STATUS 40

```
(no output)
```

Copyright © 2026 Mallionic Inc. All rights reserved.

3.7 A PROGRAM WITHOUT THE C LIBRARY

With `--no-libc` the compiler never emits a call into the C library. Arrays and strings still work: the compiler supplies its own `calloc`, `free` and `exit` built on the Linux `mmap` and `exit_group` system calls. The result links with `ld` alone and does not depend on any shared library.

Example 3-2. A statically linked program without libc

NOLIBC.ASH

```
import <syscalls>;

@Entry
pub func _start(): int64 {
    println("No C library here.");
    int64[] squares = {1, 4, 9, 16};
    sys_exit(squares[2] + squares[3]);      // 25
    return 0;
}
```

OUTPUT — EXIT STATUS 25

```
No C library here.
```

ASHIR

```
func qword calloc(qword n, qword size) {
    .type r qword
    %r mov[0]
    %_ asm[x6d6f76207273692c20407661723a6e0a696d756c207273692c20407661723a73697a650a786f7220
↪ 6564692c206564690a6d6f76206564782c20330a6d6f7620723130642c20307832320a6d6f762072382c202d31
↪ 0a786f72207239642c207239640a6d6f76206561782c20390a73797363616c6c0a6d6f7620407661723a722c20
↪ 7261780a]
    ret qword r
}

func qword free(qword p) {
    ret qword 0
}

func qword exit(qword code) {
    %_ asm[x6d6f76207264692c20407661723a636f64650a6d6f76206561782c203233310a73797363616c6c0a
↪ ]
    ret qword 0
}

func qword syscall6(qword n, qword a1, qword a2, qword a3, qword a4, qword a5, qword a6) {
.type r qword
    %r mov[0]
    %_ asm[x6d6f76207261782c20407661723a6e0a6d6f76207264692c20407661723a61310a6d6f7620727369
↪ 2c20407661723a61320a6d6f76207264782c20407661723a61330a6d6f76207231302c20407661723a61340a6d
↪ 6f762072382c20407661723a61350a6d6f762072392c20407661723a61360a73797363616c6c0a6d6f76204076
↪ 61723a722c20726178]
    ret qword r
}

func qword sys_write(qword fd, qword buf, qword n) {
    %__tmp25 mov[1]
    %__tmp26 mov[fd]
    %__tmp27 mov[buf]
    %__tmp28 mov[n]
    %__tmp29 mov[0]
    %__tmp30 mov[0]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
    %__tmp31 mov[0]
    %__ret call[syscall6, __tmp25, __tmp26, __tmp27, __tmp28, __tmp29, __tmp30, __tmp31]
    ret qword __ret
}

func void sys_exit(qword code) {
    %__tmp117 mov[60]
    %__tmp118 mov[code]
    %__tmp119 mov[0]
    %__tmp120 mov[0]
    %__tmp121 mov[0]
    %__tmp122 mov[0]
    %__tmp123 mov[0]
    call[syscall6, __tmp117, __tmp118, __tmp119, __tmp120, __tmp121, __tmp122, __tmp123]
    ret void
}

func qword __sys_len(qword s) {
.type n qword
    %n mov[0]
    %_ label[__L0]
.type __tmp125 qword
    %__tmp125 mov[n]
    %__tmp125 add[s]
.type __tmp126 byte
    %__tmp126 load[__tmp125]
    %__tmp124 mov[__tmp126]
    %_ brf[__tmp124, ne, 0, __L1]
    inc n
    %_ jmp[__L0]
    %_ label[__L1]
    ret qword n
}

func qword print(qword s) {
    %__tmp127 mov[1]
    %__tmp128 mov[s]
    %__tmp130 mov[s]
    %__tmp129 call[__sys_len, __tmp130]
    %__ret call[sys_write, __tmp127, __tmp128, __tmp129]
    ret qword __ret
}

func qword println(qword s) {
.type r qword
    %__tmp131 mov[s]
    %r call[print, __tmp131]
    %__tmp132 mov[1]
    %__tmp133 addr[__str1]
    %__tmp134 mov[1]
    call[sys_write, __tmp132, __tmp133, __tmp134]
    ret qword r
}

.string __str0 "No C library here."
.string __str1 "\n"
```

Copyright © 2026 Mallionic Inc. All rights reserved.

SHELL SESSION

```
$ ashc nolibc.ash --elf --no-libc -s
$ ld -e _start out/nolibc.o -o nolibc
$ ./nolibc; echo "exit status: $?"
No C library here.
exit status: 25
$ file nolibc
nolibc: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, not stripped
```

3.8 A 32-BIT PROGRAM

`@bits(32)` switches the whole file to the i386 instruction set. Integers declared as `int64` are still accepted, but the natural register size is now 32 bits, pointers are 4 bytes wide and arguments are passed on the stack (the `cdecl` convention).

Example 3-3. A 32-bit program linked with the 32-bit C library

ASH

```
@bits(32)

func add(int32 a, int32 b): int32 {
    return a + b;
}

@Entry
pub func main(): int32 {
    printf("32-bit says %d\n", add(30, 12));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
32-bit says 42
```

X86 DISASSEMBLY (32-BIT)

```
00000000 <main>:
   0:   55                      push   ebp
   1:   89 e5                   mov    ebp,esp
   3:   81 ec 10 00 00 00       sub    esp,0x10
   9:   b8 80 00 00 00          mov    eax,0x80
   e:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  11:   b8 1e 00 00 00          mov    eax,0x1e
  16:   89 45 f8                mov    DWORD PTR [ebp-0x8],eax
  19:   b8 0c 00 00 00          mov    eax,0xc
  1e:   89 45 f4                mov    DWORD PTR [ebp-0xc],eax
  21:   8b 45 f4                mov    eax,DWORD PTR [ebp-0xc]
  24:   50                      push   eax
  25:   8b 45 f8                mov    eax,DWORD PTR [ebp-0x8]
  28:   50                      push   eax
  29:   e8 2b 00 00 00          call   59 <add>
  2e:   81 c4 08 00 00 00       add    esp,0x8
  34:   89 45 f0                mov    DWORD PTR [ebp-0x10],eax
  37:   8b 45 f0                mov    eax,DWORD PTR [ebp-0x10]
  3a:   50                      push   eax
  3b:   8b 45 fc                mov    eax,DWORD PTR [ebp-0x4]
  3e:   50                      push   eax
  3f:   e8 fc ff ff ff          call   40 <main+0x40>
  44:   81 c4 08 00 00 00       add    esp,0x8
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86 DISASSEMBLY (32-BIT) (CONTINUED)

```
  4a:   b8 00 00 00 00          mov    eax,0x0
  4f:   89 ec                   mov    esp,ebp
  51:   5d                      pop    ebp
  52:   c3                      ret
  53:   31 c0                   xor    eax,eax
  55:   89 ec                   mov    esp,ebp
  57:   5d                      pop    ebp
  58:   c3                      ret

00000059 <add>:
  59:   55                      push   ebp
  5a:   89 e5                   mov    ebp,esp
  5c:   81 ec 10 00 00 00       sub    esp,0x10
  62:   8b 45 08                mov    eax,DWORD PTR [ebp+0x8]
  65:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  68:   8b 45 fc                mov    eax,DWORD PTR [ebp-0x4]
  6b:   8b 4d 0c                mov    ecx,DWORD PTR [ebp+0xc]
  6e:   01 c8                   add    eax,ecx
  70:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  73:   8b 45 fc                mov    eax,DWORD PTR [ebp-0x4]
  76:   89 ec                   mov    esp,ebp
  78:   5d                      pop    ebp
  79:   c3                      ret
  7a:   31 c0                   xor    eax,eax
  7c:   89 ec                   mov    esp,ebp
  7e:   5d                      pop    ebp
  7f:   c3                      ret
```

3.9 A 16-BIT FLAT BINARY

With `@bits(16)` and `--compile` the compiler emits real-mode 8086 code with no header at all. `@originoffs(0x100)` tells the compiler the address at which the image will be loaded (here, the address used by DOS `.COM` programs). The test suite of the project runs such images in a small emulator that executes the code and prints the value of `AX` when the entry function returns.

Example 3-4. A 16-bit flat binary executed in the emulator

ASH

```
@bits(16)
@originoffs(0x100)

func triple(int16 x): int16 {
    return x * 3;
}

@Entry
pub func _start(): int16 {
    return triple(14);                    // AX = 42
}
```

OUTPUT — EXIT STATUS 0

```
ax = 42  bp = 0x0
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86 DISASSEMBLY (16-BIT)

```
 100:   55                      push   bp
 101:   89 e5                   mov    bp,sp
 103:   81 ec 10 00             sub    sp,0x10
 107:   b8 0e 00                mov    ax,0xe
 10a:   89 46 fe                mov    WORD PTR [bp-0x2],ax
 10d:   8b 46 fe                mov    ax,WORD PTR [bp-0x2]
 110:   50                      push   ax
 111:   e8 14 00                call   0x128
 114:   81 c4 02 00             add    sp,0x2
 118:   89 46 fc                mov    WORD PTR [bp-0x4],ax
 11b:   8b 46 fc                mov    ax,WORD PTR [bp-0x4]
 11e:   89 ec                   mov    sp,bp
 120:   5d                      pop    bp
 121:   c3                      ret
 122:   31 c0                   xor    ax,ax
 124:   89 ec                   mov    sp,bp
 126:   5d                      pop    bp
 127:   c3                      ret
 128:   55                      push   bp
 129:   89 e5                   mov    bp,sp
 12b:   81 ec 10 00             sub    sp,0x10
 12f:   8b 46 04                mov    ax,WORD PTR [bp+0x4]
 132:   89 46 fe                mov    WORD PTR [bp-0x2],ax
 135:   8b 46 fe                mov    ax,WORD PTR [bp-0x2]
```

3.10 A WINDOWS PROGRAM

With `--win` the compiler writes a COFF object for 64-bit Windows and uses the Microsoft x64 calling convention for every external call. The following program writes to the console through `kernel32.dll` and ends with `ExitProcess`. It was linked with MinGW-w64 and executed under Wine while this manual was built.

Example 3-5. Writing to the Windows console

ASH

```
extern uint64 GetStdHandle(int32 nStdHandle);
extern int32  WriteFile(uint64 h, uint64 buf, uint32 n, uint64 written, uint64 overlapped);
extern void   ExitProcess(uint32 code);

@Entry
pub func main(): void {
    uint64 h = GetStdHandle(-11);           // STD_OUTPUT_HANDLE
    uint32 written = 0;
    WriteFile(h, "Hello, Windows!\n", 16, &written, 0);
    ExitProcess(0);
}
```

OUTPUT — EXIT STATUS 0

```
Hello, Windows!
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <main>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 50 00 00 00    sub    rsp,0x50
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
1b:   b8 00 00 00 00          mov    eax,0x0
20:   49 89 c4                mov    r12,rax
23:   4c 89 e0                mov    rax,r12
26:   48 83 e8 0b             sub    rax,0xb
2a:   49 89 c4                mov    r12,rax
2d:   b8 00 00 00 00          mov    eax,0x0
32:   49 89 c5                mov    r13,rax
35:   48 83 ec 20             sub    rsp,0x20
39:   44 89 e1                mov    ecx,r12d
3c:   e8 00 00 00 00          call   41 <main+0x41>
41:   48 83 c4 20             add    rsp,0x20
45:   49 89 c5                mov    r13,rax
48:   b8 00 00 00 00          mov    eax,0x0
4d:   41 89 c6                mov    r14d,eax
50:   4c 89 e8                mov    rax,r13
53:   49 89 c7                mov    r15,rax
56:   b8 00 00 00 00          mov    eax,0x0
5b:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
5f:   48 8d 05 7e 00 00 00    lea    rax,[rip+0x7e]        # e4 <__str0>
66:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
6a:   b8 10 00 00 00          mov    eax,0x10
6f:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
73:   b8 00 00 00 00          mov    eax,0x0
78:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
7c:   44 89 75 c0             mov    DWORD PTR [rbp-0x40],r14d
80:   48 8d 45 c0             lea    rax,[rbp-0x40]
84:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
88:   b8 00 00 00 00          mov    eax,0x0
8d:   49 89 c6                mov    r14,rax
90:   48 83 ec 08             sub    rsp,0x8
94:   4c 89 f0                mov    rax,r14
97:   50                      push   rax
98:   48 83 ec 20             sub    rsp,0x20
9c:   4c 89 f9                mov    rcx,r15
9f:   48 8b 55 d8             mov    rdx,QWORD PTR [rbp-0x28]
a3:   44 8b 45 d0             mov    r8d,DWORD PTR [rbp-0x30]
a7:   4c 8b 4d c8             mov    r9,QWORD PTR [rbp-0x38]
ab:   e8 00 00 00 00          call   b0 <main+0xb0>
b0:   48 83 c4 30             add    rsp,0x30
b4:   b8 00 00 00 00          mov    eax,0x0
b9:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
bd:   48 83 ec 20             sub    rsp,0x20
c1:   8b 4d b8                mov    ecx,DWORD PTR [rbp-0x48]
c4:   e8 00 00 00 00          call   c9 <main+0xc9>
c9:   48 83 c4 20             add    rsp,0x20
cd:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
d1:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
d5:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
d9:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
dd:   48 89 ec                mov    rsp,rbp
e0:   5d                      pop    rbp
e1:   c3                      ret
      ...
```

Copyright © 2026 Mallionic Inc. All rights reserved.

3.11 FILES CREATED BY THE COMPILER

Table 3-6. Files written by ashc

Path	Content
`<name>.ashir`	ASHIR text, written next to the source when no output mode is given.
`out/<name>.o`	ELF object (`--elf`).
`out/<name>.obj`	COFF object (`--win`).
`out/<name>.bin`	Flat binary (`--compile`).
`out/<name>`	32-bit Linux executable (`--exe`).
`logs/build.log`	Detailed internal log of the last build (buffer writes, symbols, relocations, a hexadecimal dump of the output). Useful when reporting a compiler bug.

The output location can be changed with `-o file`, `-o directory/` or `--outdir directory` (Section 5.3.1).

3.12 EDITOR SUPPORT

The directory `vscode-ash/` contains an extension for Visual Studio Code, VSCodium and VS Code Insiders. It provides syntax highlighting for `.ash` and `.ashir` files, two color themes (*ASH Dark* and *ASH Light*), file icons, snippets, and a linter that reports errors while you type. Install it with:

```shell
cd vscode-ash
./install_extension.sh            # Linux / macOS   (--remove uninstalls it)
install_extension.bat             # Windows
```

Restart the editor and select a theme with **Ctrl+K Ctrl+T**. Section 29.1 describes the linter, the snippets and the settings.

3.13 WHERE TO GO NEXT

- Chapter 5 lists every command line option.
- Chapter 7 starts the description of the language.
- The `examples/` directory contains fifty small programs, one per feature; Appendix D indexes them.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 4

A TOUR OF ASH

This chapter is a guided tour of the language for readers who learn best by example. It introduces one idea at a time, each with a complete program that you can type, compile and run. The later chapters describe every feature exhaustively; here the aim is to become productive quickly. All programs in this chapter are compiled and linked as shown in Section 3.4: `ashc prog.ash --elf` followed by `gcc out/prog.o -o prog`.

4.1 VALUES AND VARIABLES

A variable is declared with its type, its name and, usually, an initial value. `int64` is the natural integer type on 64-bit machines; `printf` prints values with the familiar C format codes.

Example 4-1. Variables and arithmetic

ASH

```
@Entry
pub func main(): int64 {
    int64 apples = 12;
    int64 friends = 5;
    int64 each = apples / friends;
    int64 left = apples % friends;
    printf("%d apples, %d friends: %d each, %d left over\n", apples, friends, each, left);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
12 apples, 5 friends: 2 each, 2 left over
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 4-1. Line by line: variables and arithmetic

Code	What it does
`@Entry`	Marks `main` as the function where the program starts.
`pub func main(): int64 {`	Begins the function `main`; it returns a 64-bit integer to the operating system.
`int64 apples = 12;`	Creates a variable called `apples`, of type `int64` (a whole number), holding 12.
`int64 friends = 5;`	A second variable holding 5.
`int64 each = apples / friends;`	Divides 12 by 5. Whole-number division drops the fraction, so `each` is 2.
`int64 left = apples % friends;`	`%` gives the remainder of the division: 12 = 5 * 2 + 2, so `left` is 2.
`printf("%d apples, ...", apples, ...);`	Prints the text; each `%d` is replaced by the next value in the list, in order. `\n` ends the line.
`return 0;`	Tells the operating system that everything went well.
`}`	Ends the function.

Types can be narrower than 64 bits and can be unsigned. A value that does not fit wraps around:

Example 4-2. Narrow and unsigned types

ASH

```
@Entry
pub func main(): int64 {
    uint8 level = 250;
    level = level + 10;          // 260 does not fit in 8 bits: 4
    int16 temperature = -40;
    uint32 big = 4000000000;
    printf("level=%d temperature=%d big=%u\n", level, temperature, big);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
level=4 temperature=-40 big=4000000000
```

Table 4-2. Line by line: narrow and unsigned types

Code	What it does
`uint8 level = 250;`	`uint8` is an unsigned number of 8 bits: it can hold 0 to 255.
`level = level + 10;`	250 + 10 = 260 does not fit in 8 bits; only the low 8 bits are kept, which is 4.
`int16 temperature = -40;`	`int16` is a signed 16-bit number (-32768 to 32767), so negative values are fine.
`uint32 big = 4000000000;`	`uint32` holds 0 to about 4.29 billion.
`printf("... big=%u\n", ...)`	`%u` prints an unsigned value.

Copyright © 2026 Mallionic Inc. All rights reserved.

4.2 MAKING DECISIONS

`if`, `elif` and `else` choose between blocks. Braces are always required. Conditions can combine comparisons with `&&` (and), `||` (or) and `!` (not).

Example 4-3. if, elif and else

ASH

```
func describe(int64 age): void {
    if (age < 0) {
        printf("%d: invalid\n", age);
    } elif (age < 13) {
        printf("%d: child\n", age);
    } elif (age < 20) {
        printf("%d: teenager\n", age);
    } else {
        printf("%d: adult\n", age);
    }
}

@Entry
pub func main(): int64 {
    describe(-1);
    describe(8);
    describe(15);
    describe(42);
    int64 year = 2024;
    if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
        printf("%d is a leap year\n", year);
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-1: invalid
8: child
15: teenager
42: adult
2024 is a leap year
```

Table 4-3. Line by line: if, elif and else

Code	What it does
`func describe(int64 age): void {`	A function that receives one number and returns nothing (`void`).
`if (age < 0) { ... }`	Runs the block only when `age` is less than 0.
`} elif (age < 13) {`	Otherwise, if `age` is less than 13, runs this block.
`} else {`	When none of the conditions above was true, runs this block.
`describe(-1);` … `describe(42);`	Calls the function four times with different values.
`year % 4 == 0 && year % 100 != 0`	"divisible by 4 **and** not divisible by 100".
`... || year % 400 == 0`	"**or** divisible by 400". The parentheses group the first part.

Copyright © 2026 Mallionic Inc. All rights reserved.

4.3 REPEATING THINGS

ASH has the `while` and `for` loops of C, plus two of its own: `repeat (n)` runs a block a fixed number of times, and `loop` runs forever until `break`.

Example 4-4. Four kinds of loops

ASH

```
@Entry
pub func main(): int64 {
    int64 n = 1;
    while (n < 100) {
        n = n * 2;
    }
    printf("first power of two >= 100: %d\n", n);

    int64 sum = 0;
    for (int64 i = 1; i <= 100; i++) {
        sum += i;
    }
    printf("1 + 2 + ... + 100 = %d\n", sum);

    repeat (3) {
        printf("hip ");
    }
    printf("hooray!\n");

    int64 guess = 0;
    loop {
        guess++;
        if (guess * guess > 50) { break; }
    }
    printf("smallest number whose square exceeds 50: %d\n", guess);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
first power of two >= 100: 128
1 + 2 + ... + 100 = 5050
hip hip hip hooray!
smallest number whose square exceeds 50: 8
```

Table 4-4. Line by line: loops

Code	What it does
`while (n < 100) { n = n * 2; }`	Doubles `n` as long as it is below 100: 1, 2, 4, ..., 128.
`for (int64 i = 1; i <= 100; i++) {`	Starts `i` at 1; repeats while `i <= 100`; adds 1 to `i` after each repetition.
`sum += i;`	Short for `sum = sum + i`.
`repeat (3) { printf("hip "); }`	Runs the block exactly three times.
`loop { ... }`	Repeats forever...
`if (guess * guess > 50) { break; }`	...until `break` leaves the loop.

Copyright © 2026 Mallionic Inc. All rights reserved.

4.4 FUNCTIONS

A function lists its parameters with their types, and gives its result type after a colon. Functions can be used before they are defined, and they can call themselves.

Example 4-5. Writing functions

ASH

```
func square(int64 x): int64 {
    return x * x;
}

func power(int64 base, int64 exp): int64 {
    int64 result = 1;
    repeat (exp) {
        result = result * base;
    }
    return result;
}

func fib(int64 n): int64 {
    if (n < 2) { return n; }
    return fib(n - 1) + fib(n - 2);
}

@Entry
pub func main(): int64 {
    printf("square(9)=%d power(2, 10)=%d fib(25)=%d\n", square(9), power(2, 10), fib(25));
    printf("and ASH has a power operator too: 2 ** 10 = %d\n", 2 ** 10);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
square(9)=81 power(2, 10)=1024 fib(25)=75025
and ASH has a power operator too: 2 ** 10 = 1024
```

Table 4-5. Line by line: functions

Code	What it does
`func square(int64 x): int64 {`	A function named `square` with one parameter `x`; it returns an `int64`.
`return x * x;`	Gives back `x` times `x`.
`func power(int64 base, int64 exp): int64 {`	A function with two parameters.
`repeat (exp) { result = result * base; }`	Multiplies `result` by `base`, `exp` times.
`if (n < 2) { return n; }`	In `fib`, the simplest cases: fib(0) = 0, fib(1) = 1.
`return fib(n - 1) + fib(n - 2);`	Otherwise the function calls itself twice (*recursion*).
`2 ** 10`	ASH's power operator, computed by the compiler.

Copyright © 2026 Mallionic Inc. All rights reserved.

4.5 ARRAYS

An array holds several values of one type. Arrays know their length, which the # operator returns, and every index is checked: an index out of range stops the program instead of corrupting memory.

Example 4-6. Working with an array

ASH

```
@Entry
pub func main(): int64 {
    int64[] scores = {72, 91, 58, 85, 99, 64};
    int64 best = scores[0];
    int64 total = 0;
    for (int64 i = 0; i < #scores; i++) {
        total = total + scores[i];
        if (scores[i] > best) { best = scores[i]; }
    }
    printf("%d scores, best %d, average %d\n", #scores, best, total / #scores);

    int64[10] squares;                    // ten zeros
    for (int64 i = 0; i < #squares; i++) {
        squares[i] = i * i;
    }
    printf("squares[7] = %d\n", squares[7]);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
6 scores, best 99, average 78
squares[7] = 49
```

Table 4-6. Line by line: arrays

Code	What it does
`int64[] scores = {72, 91, 58, 85, 99, 64};`	An array of six numbers. Positions go from 0 to 5.
`int64 best = scores[0];`	`scores[0]` is the first element, 72.
`for (int64 i = 0; i < #scores; i++) {`	`#scores` is the number of elements (6), so `i` goes 0, 1, ..., 5.
`total = total + scores[i];`	Adds the element at position `i`.
`if (scores[i] > best) { best = scores[i]; }`	Remembers the largest value seen so far.
`int64[10] squares;`	An array of ten numbers, all starting at 0.
`squares[i] = i * i;`	Stores a value at position `i`.

4.6 TEXT

String literals are written in double quotes. A `string` variable holds the address of a text, which can be passed to `printf` with `%s`. To build or change text at run time, use a byte buffer and the functions of `<string>`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 4-7. Strings and text buffers

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    string name = "Ada";
    string[] colors = {"red", "green", "blue"};
    printf("Hello, %s! You like %s.\n", name, colors[1]);

    uint8[64] message;
    strcpy(&message, "The Ashley ");
    strcat(&message, "Project");
    printf("%s has %d characters\n", &message, strlen(&message));
    str_upper(&message);
    printf("%s\n", &message);
    printf("'Ashley' starts at index %d\n", strstr("The Ashley Project", "Ashley"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Hello, Ada! You like green.
The Ashley Project has 18 characters
THE ASHLEY PROJECT
'Ashley' starts at index 4
```

Table 4-7. Line by line: text

Code	What it does
`import <string>;`	Brings in the text functions of the `<string>` library.
`string name = "Ada";`	A text variable.
`string[] colors = {"red", "green", "blue"};`	An array of texts; `colors[1]` is "green".
`uint8[64] message;`	A buffer of 64 bytes where text can be built and changed.
`strcpy(&message, "The Ashley ");`	Copies a text into the buffer. `&message` is the buffer's address.
`strcat(&message, "Project");`	Appends a text at the end.
`strlen(&message)`	Counts the characters.
`str_upper(&message);`	Turns the letters into capitals, inside the buffer.
`strstr("The Ashley Project", "Ashley")`	Finds where one text appears in another: position 4.

4.7 GROUPING DATA WITH STRUCTURES

A structure groups named fields. Fields are accessed with `.`, or with `->` through a pointer; a function that receives a pointer can change the caller's structure.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 4-8. A structure and a function that changes it

ASH

```
typedef struct {
    int64 x;
    int64 y;
    int64 dx;
    int64 dy;
} Ball;

func step(Ball* b): void {
    b->x = b->x + b->dx;
    b->y = b->y + b->dy;
    if (b->x < 0 || b->x > 20) { b->dx = 0 - b->dx; }
    if (b->y < 0 || b->y > 10) { b->dy = 0 - b->dy; }
}

@Entry
pub func main(): int64 {
    Ball ball;
    ball.x = 18;
    ball.y = 2;
    ball.dx = 3;
    ball.dy = 4;
    repeat (4) {
        step(&ball);
        printf("ball at (%d, %d)\n", ball.x, ball.y);
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
ball at (21, 6)
ball at (18, 10)
ball at (15, 14)
ball at (12, 10)
```

Table 4-8. Line by line: structures

Code	What it does
`typedef struct { ... } Ball;`	Defines a new type, `Ball`, made of four `int64` fields.
`func step(Ball* b): void {`	Receives a pointer to a `Ball`: the address of a ball, so the function can change it.
`b->x = b->x + b->dx;`	`->` reaches a field through the pointer: moves the ball horizontally.
`b->dx = 0 - b->dx;`	Reverses the direction when the ball reaches a wall.
`Ball ball;`	Creates a `Ball` variable (all fields start at 0).
`ball.x = 18;`	`.` reaches a field of the variable itself.
`step(&ball);`	Passes the address of `ball`, so `step` changes this ball.

Copyright © 2026 Mallionic Inc. All rights reserved.

4.8 ORGANIZING CODE WITH CLASSES

A class groups related functions under one name and can keep some of them private.

Example 4-9. A class as a namespace

ASH

```
class Temperature {
    pub func to_fahrenheit(int64 c): int64 {
        return Temperature.scale(c) + 32;
    }
    pub func to_kelvin(int64 c): int64 {
        return c + 273;
    }
    priv func scale(int64 c): int64 {
        return c * 9 / 5;
    }
}

@Entry
pub func main(): int64 {
    int64[] celsius = {-40, 0, 37, 100};
    for (int64 i = 0; i < #celsius; i++) {
        int64 c = celsius[i];
        printf("%4d C = %4d F = %4d K\n", c, Temperature.to_fahrenheit(c), Temperature.to_ke
↪ lvin(c));
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
 -40 C =  -40 F =  233 K
   0 C =   32 F =  273 K
  37 C =   98 F =  310 K
 100 C =  212 F =  373 K
```

Table 4-9. Line by line: classes

Code	What it does
`class Temperature { ... }`	Groups three functions under the name `Temperature`.
`pub func to_fahrenheit(int64 c): int64 {`	A public method: it can be called from anywhere.
`Temperature.scale(c)`	Calls another method of the same class.
`priv func scale(int64 c): int64 {`	A private method: only methods of `Temperature` may call it.
`Temperature.to_kelvin(c)`	Calling a method from outside the class: class name, dot, method name.
`printf("%4d C = ...")`	`%4d` prints the number using at least 4 characters, which aligns the columns.

Copyright © 2026 Mallionic Inc. All rights reserved.

4.9 USING LIBRARIES

`import` brings in a library. The libraries of `nlib` cover strings, mathematics, system calls and command line arguments; any C function can be called as well.

Example 4-10. Arguments, math and C functions together

ASH

```
import <args>;
import <math>;

@Entry
pub func main(int64 argc, int64 argv): int64 {
    if (argc < 3) {
        printf("usage: program NAME NUMBER\n");
        return 1;
    }
    int64 n = arg_int(argv, 2);
    printf("Hello %s!\n", arg_at(argv, 1));
    printf("%d: square root %d, prime %d, factorial digits %d\n",
        n, isqrt(n), is_prime(n), ilog10(factorial(15)) + 1);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Hello Grace!
36: square root 6, prime 0, factorial digits 13
```

Table 4-10. Line by line: libraries and arguments

Code	What it does
`import <args>; import <math>;`	Bring in two libraries.
`pub func main(int64 argc, int64 argv): int64 {`	`main` receives the number of command line words (`argc`) and where they are (`argv`).
`if (argc < 3) { ... return 1; }`	Checks that two words were given after the program name.
`arg_int(argv, 2)`	Converts the second word ("36") to a number.
`arg_at(argv, 1)`	The first word, as text ("Grace").
`isqrt(n)`, `is_prime(n)`, `factorial(15)`, `ilog10(...)`	Functions from `<math>`: square root, primality, factorial, number of digits minus one.

The example above was run with the arguments `Grace 36`.

4.10 GOING LOW LEVEL

When you need full control, you can write processor instructions inside an ASH function and use ASH variables in them by name.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 4-11. An asm block in an ASH function

ASH

```
func rotate(uint64 v, int64 n): uint64 {
    uint64 r = 0;
    asm {
        mov rax, v
        mov rcx, n
        rol rax, cl
        mov r, rax
    }
    return r;
}

@Entry
pub func main(): int64 {
    printf("%llx\n", rotate(0x8000000000000001, 4));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
18
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <rotate>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  1f:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
  23:   b8 00 00 00 00          mov    eax,0x0
  28:   49 89 c4                mov    r12,rax
  2b:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  2f:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
  33:   48 d3 c0                rol    rax,cl
  36:   49 89 c4                mov    r12,rax
  39:   4c 89 e0                mov    rax,r12
  3c:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  40:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  44:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  48:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  4c:   48 89 ec                mov    rsp,rbp
  4f:   5d                      pop    rbp
  50:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 4-11. Line by line: assembly

Code	What it does
`uint64 r = 0;`	A variable that will receive the result.
`asm {`	Starts a block of processor instructions.
`mov rax, v`	Copies the value of the ASH variable `v` into the register `rax`.
`mov rcx, n`	Copies `n` into `rcx`; its low byte is `cl`.
`rol rax, cl`	Rotates the bits of `rax` to the left by `cl` positions.
`mov r, rax`	Stores the result in the ASH variable `r`.
`}`	Ends the assembly block.

The disassembly shows the four instructions of the block, with `v`, `n` and `r` replaced by the places where the compiler keeps them.

4.11 WHERE TO GO FROM HERE

You have now seen the essential features of ASH. The rest of the manual covers each of them in depth:

- the exact rules for literals, types and conversions (Chapter 7, Chapter 8);
- every statement and operator (Chapter 10, Chapter 11);
- functions, callbacks and variadic functions (Chapter 12);
- arrays, structures, classes and cracks (Chapter 13 to Chapter 16);
- annotations, directives, modules and inline assembly (Chapter 17 to Chapter 20);
- every library function, with examples (Chapter 27);
- complete programs (Chapter 30).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 5

THE ASHC COMMAND LINE

This chapter is the reference of the `ashc` command. It describes the general syntax, every output mode, every option, how output file names are chosen and how diagnostics are printed. Each option is illustrated with a session recorded while this manual was built.

5.1 SYNOPSIS

```
ashc <input> [output] [options]
```

input is the source file. Its extension selects the starting point of the pipeline:

Table 5-1. Input files accepted by ashc

Extension	Treatment
`.ash`	ASH source. The full pipeline runs: directives, lexer, macros, parser, rewriting passes, ASHIR generation and (if an output mode asks for it) code generation.
`.ashir`	ASHIR text. The front end is skipped; the file is parsed as ASHIR and compiled to the requested output. This lets you edit the intermediate representation by hand, or generate it with another tool.

An optional second positional argument is the output file (equivalent to `-o`). Options may appear before or after the file names. Options are case-sensitive.

Here is the help text printed by `ashc --help`:

SHELL SESSION

```
$ ashc --help
ashc - the ASH compiler

USAGE
  ashc <file.ash> [options]

OUTPUT MODES
  (default)            ASH -> ASHIR text
  --compile            ASH -> ASHIR -> flat binary
  --elf                ASH -> ELF64 relocatable object (Linux)
  --win                ASH -> COFF AMD64 relocatable object (Windows)
  --bits <16|32|64>    target width (also @bits(N)); 32 + --elf/--win gives
                       an i386 ELF32 / COFF object, 16 gives a flat binary
  --exe                32-bit Linux executable (ELF32) from a flat image
  --no-libc            never call libc: built-in heap/exit, no externs
  -e, --epilogue       emit function prologue/epilogue (stack frame) always;
                       default is clean C-like output (frame only if needed)

OPTIONS
  -o <file|dir/>       write the output to <file> (or into <dir/>), any mode
  --outdir <dir>       output directory instead of <input dir>/out
```

Copyright © 2026 Mallionic Inc. All rights reserved.

SHELL SESSION (CONTINUED)

```
  --list-nlib          list the libraries embedded in the compiler (import <name>)
  -f, --format <fmt>   elf | coff | bin | exe | ashir  (same as --elf, --win, ...)
  --stdout             print the result instead of writing a file
  --tokens             print the token stream and stop
  --debug              print per-stage timings
  -s, --silent         print nothing on success
  --msg <format>       success message; {file} {size} {mode} are replaced
  --trace              show Java stack traces for internal errors
  -v, --version        print the compiler version
  -h, --help           show this help

DIAGNOSTICS
  warning   (yellow)   suspicious code, compilation continues
  tip       (green)    a suggestion to fix the problem above it
  error     (purple)   a problem in your program or command line
  internal  (red)      a bug in ashc itself; please report it

EXAMPLES
  ashc hello.ash --elf -o hello.o
  ashc hello.ash --win -o hello.obj
```

5.2 OUTPUT MODES

Exactly one output mode is active for a run. Without a mode option the compiler writes ASHIR text.

Table 5-2. Output modes

Option	Output	Description
(none)	`<name>.ashir`	ASH is translated to ASHIR text, written next to the source. No machine code is generated.
`--compile`	`out/<name>.bin`	ASHIR is compiled to a flat binary: raw machine code followed by data, no header, no relocations. Addresses are absolute, computed from `@originoffs`. Used for 16-bit code, boot sectors and images loaded by your own loader.
`--elf`	`out/<name>.o`	ELF relocatable object. 64-bit (`ELF64`, x86-64) by default; 32-bit (`ELF32`, i386) when the program uses `@bits(32)` or `--bits 32`.
`--win`	`out/<name>.obj`	COFF relocatable object for Windows. AMD64 by default, i386 with `@bits(32)`. Calls to `extern` functions use the Microsoft x64 convention (`RCX`, `RDX`, `R8`, `R9`, 32 bytes of shadow space).
`--exe`	`out/<name>`	A complete 32-bit Linux executable: an ELF32 header and one loadable segment that wraps a flat image. The image must be linked at the address given by `@originoffs` (normally `0x08048000`).
`-f`, `--format` *fmt*	| Selects the mode by name: `elf`; `coff` (also `win`, `obj`); `bin` (also `flat`, `raw`); `exe` (also `elf32`); `ashir` (also `ir`).	

`--elf` and `--win` are mutually exclusive. `--bits` alone also turns code generation on (a flat binary) unless `--elf` or `--win` is given.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 5-1. The same program in several output modes

SHELL SESSION

```
$ ashc hello.ash -s
$ ashc hello.ash --elf -s
$ ashc hello.ash --win -s
$ ls -l hello.ashir out/
-rw-rw-r-- 1 user user  130 Oct 24 22:19 hello.ashir

out/:
total 8
-rw-rw-r-- 1 user user 1640 Oct 24 22:19 hello.o
-rw-rw-r-- 1 user user  708 Oct 24 22:19 hello.obj
```

NOTE

A flat binary has no symbol table, so it cannot refer to functions of another library. The call to `printf` in `hello.ash` therefore makes `ashc hello.ash --compile` stop with an error (a call to the external function `printf` needs an object file format); `--elf` and `--win` succeed because the object file records an *external* reference that the linker resolves later. Flat images use only their own code, the built-in helpers, and the operating system or firmware they run on.

5.3 OPTIONS REFERENCE

5.3.1 Output location: -o, --outdir

`-o` *path* sets the output. If *path* ends with `/` (or `\` on Windows) or names an existing directory, the output file keeps its default name and is written into that directory. Otherwise *path* is the output file name itself. `--outdir` *dir* replaces the default `out/` directory while keeping the default file name.

SHELL SESSION

```
$ ashc hello.ash --elf -o build/ -s
$ ashc hello.ash --elf -o hello64.o -s
$ ashc hello.ash --elf --outdir objects -s
$ ls build objects hello64.o
hello64.o

build:
hello.o

objects:
hello.o
```

When neither option is given, binary outputs go into a directory named `out` next to the *source file* (not the current directory), which is created if necessary.

5.3.2 --stdout

Prints the result on standard output instead of writing a file. It is mainly useful with ASHIR output, to pipe the intermediate representation into another tool:

Copyright © 2026 Mallionic Inc. All rights reserved.

SQUARE.ASH

```
func square(int64 x): int64 {
    return x * x;
}

@Entry
pub func main(): int64 {
    return square(3) * square(2);
}
```

SHELL SESSION

```
$ ashc square.ash --stdout -s | grep "func "
func qword square(qword x) {
func qword main() {
$ ashc square.ash --stdout -s | grep -c "mul\["
2
```

5.3.3 --bits *n*

Forces the target width to 16, 32 or 64 bits, overriding `@bits(...)` in the source. `--bits 32 --elf` produces an i386 ELF object, `--bits 32 --win` an i386 COFF object, and `--bits 16` (or 32) alone a flat binary. Inside the source the width is visible to the preprocessor as `ASH_BITS` (Section 18.2).

5.3.4 --no-libc

Promises that the program will not be linked with the C library. The compiler then:

- replaces the external `calloc`, `free` and `exit` used by arrays, strings and `exit(...)` with built-in implementations based on the Linux system calls `mmap` (9) and `exit_group` (231);
- defines the preprocessor flag `ASH_NO_LIBC` to 1 (it is 0 otherwise);
- still lets you declare and call `extern` functions, which you must then provide yourself (for example in another object file).

The same effect can be requested from the source with `@nolibc` (Section 17.10). In 32-bit flat mode there is never a C library: only the built-in `calloc`, `free` and `exit` are available.

5.3.5 -e, --epilogue

Always emits a full stack frame (`push rbp` / `mov rbp, rsp` ... `leave`) in every function. By default (`@beautycode(true)`), leaf functions that need no stack space are emitted without a frame. A frame in every function makes stack traces in debuggers more reliable. The option wins over `@beautycode` in the source.

Example 5-2. Default output: a leaf function without a frame

ASH

```
func add(int64 a, int64 b): int64 {
    return a + b;
}

@Entry
pub func main(): int64 {
    return add(40, 2);
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <add>:
   0:   48 89 f8                mov    rax,rdi
   3:   49 89 c2                mov    r10,rax
   6:   4c 89 d0                mov    rax,r10
   9:   48 89 f1                mov    rcx,rsi
   c:   48 01 c8                add    rax,rcx
   f:   49 89 c2                mov    r10,rax
  12:   4c 89 d0                mov    rax,r10
  15:   c3                      ret

0000000000000016 <main>:
  16:   55                      push   rbp
  17:   48 89 e5                mov    rbp,rsp
  1a:   48 81 ec 20 00 00 00    sub    rsp,0x20
  21:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  25:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  29:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  2d:   b8 28 00 00 00          mov    eax,0x28
  32:   49 89 c4                mov    r12,rax
  35:   b8 02 00 00 00          mov    eax,0x2
  3a:   49 89 c5                mov    r13,rax
  3d:   b8 00 00 00 00          mov    eax,0x0
  42:   49 89 c6                mov    r14,rax
  45:   4c 89 e7                mov    rdi,r12
  48:   4c 89 ee                mov    rsi,r13
  4b:   e8 b0 ff ff ff          call   0 <add>
  50:   49 89 c6                mov    r14,rax
  53:   4c 89 f0                mov    rax,r14
  56:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5a:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  5e:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  62:   c9                      leave
  63:   c3                      ret
```

Example 5-3. The same function compiled with -e

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <add>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   48 89 f8                mov    rax,rdi
   e:   49 89 c2                mov    r10,rax
  11:   4c 89 d0                mov    rax,r10
  14:   48 89 f1                mov    rcx,rsi
  17:   48 01 c8                add    rax,rcx
  1a:   49 89 c2                mov    r10,rax
  1d:   4c 89 d0                mov    rax,r10
  20:   c9                      leave
  21:   c3                      ret
```

5.3.6 -s, --silent and --msg

By default a successful build prints one line beginning with `[BUILD SUCCESS]`. `-s` suppresses it (errors and warnings are still printed). `--msg` *format* replaces it with your own text, where `{file}`, `{size}` and `{mode}` are substituted:

Copyright © 2026 Mallionic Inc. All rights reserved.

SHELL SESSION

```
$ ashc hello.ash --elf
[BUILD SUCCESS] File has been compiled: out/hello.o
$ ashc hello.ash --elf --msg "ok: {file} is a {mode} of {size} bytes"
[BUILD SUCCESS] ok: out/hello.o is a x86-64 ELF object of 1640 bytes
$ ashc hello.ash --elf -s
```

5.3.7 --tokens

Prints the token stream produced by the lexer and stops. Each line shows the index of the token, its type, its text and its position (`line:column`). Use it to understand how the lexer splits an unusual construct.

TINY.ASH

```
@Entry
pub func main(): int64 {
    return 0x2A;   // 42
}
```

SHELL SESSION

```
$ ashc tiny.ash --tokens
    0  AT            @                   1:1
    1  IDENT         Entry               1:2
    2  IDENT         pub                 2:1
    3  IDENT         func                2:5
    4  IDENT         main                2:10
    5  LPAREN        (                   2:14
    6  RPAREN        )                   2:15
    7  COLON         :                   2:16
    8  IDENT         int64               2:18
    9  LBRACE        {                   2:24
   10  IDENT         return              3:5
   11  NUMBER        0x2A                3:12
   12  SEMICOLON     ;                   3:16
   13  RBRACE        }                   4:1
   14  EOF                               4:2
[BUILD SUCCESS] File has been compiled: tiny.ashir
```

5.3.8 --debug

Prints the time spent in each stage of the compiler (reading, lexing, ASHIR generation, ASHIR parsing, code generation, object writing) and internal tracing information about symbols and relocations. The output is meant for compiler developers and changes between versions.

5.3.9 --trace

When the compiler itself fails (an *internal compiler error*, printed in red), it normally prints a one-line description. `--trace` adds the Java stack trace, which is the most useful information to attach to a bug report.

5.3.10 --list-nlib

Lists the libraries embedded in the compiler that can be imported with `import <name>;`.

Copyright © 2026 Mallionic Inc. All rights reserved.

5.3.11 -v, --version; --license; -h, --help

Print the version and copyright notice, the license summary, and the help text respectively, then exit.

SHELL SESSION

```
$ ashc --license
The Ashley Project - ashc
Copyright (C) 2026 Mallionic Inc.

This program is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
See the GNU General Public License for more details.

Full license: https://www.gnu.org/licenses/gpl-3.0.html
```

5.4 OUTPUT FILE NAMES

Table 5-3. How the output path is chosen

Situation	Result
ASHIR mode, no `-o`	`<source directory>/<name>.ashir`
Binary mode, no `-o`, no `--outdir`	`<source directory>/out/<name>.<ext>` where *ext* is `o`, `obj`, `bin` or empty (`--exe`).
`--outdir D`	`D/<name>.<ext>`
`-o D/` or `-o D` with `D` an existing directory	`D/<name>.<ext>`
`-o F`	`F` exactly (no extension is added).
`--stdout`	Nothing is written; the result goes to standard output.

5.5 DIAGNOSTICS

The compiler classifies every message it prints. On a color terminal each class has its own color:

Table 5-4. Classes of diagnostics

Class	Color	Meaning
`warning`	yellow	Suspicious code. Compilation continues and an output file is produced.
`tip`	green	A suggestion that follows an error or warning and tells you how to fix it.
`error`	purple	A problem in your program or in the command line. No output file is produced.
`internal compiler error`	red	A bug in `ashc` itself. Please report it with the source file and the output of `--trace`.

Errors detected while parsing show the position (`<input>:line:column`), the offending line and a marker under the problem. Errors detected by later stages (for example an undeclared variable, found while ASHIR

Copyright © 2026 Mallionic Inc. All rights reserved.

is being compiled) name the symbol but not the line. At the end the compiler prints a summary with the number of errors and warnings.

> **NOTE**
> A few messages produced by older stages of the compiler are still written in Spanish; `ashc` translates the most common ones automatically. Where this manual shows such a message, the English translation is shown and the box is labelled accordingly.

Example 5-4. An undeclared variable

ASH

```
@Entry
pub func main(): int64 {
    int64 total = 0;
    totl = 5;
    return total;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: undeclared variable: totl
1 error generated.
```

Example 5-5. An error followed by a tip

ASH

```
uint64 counter = 0

@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS

```
<input>:1:1: error: variable 'counter' declared outside a function: at top level only genera
↪ l, ungeneral, extern, import, define, func, class and annotations are allowed
hint: a variable declared outside a function must say who can see it. Write 'general uint64
↪ counter = ...;' (public: visible to C and to other files under the name 'counter') or 'ung
↪ eneral uint64 counter = ...;' (private to this script). Add 'static' when the value must c
↪ hange at run time ('general static uint64 counter = ...;'); without 'static' the variable
↪ is a constant.
    1 | uint64
      | ^^^^^^
1 error generated.
```

Chapter 28 lists the most common diagnostics with their causes and fixes.

5.6 EXIT STATUS OF ASHC

Status	Meaning
0	Success (or `--help`, `--version`, `--license`, `--list-nlib`).
1	One or more errors (including internal errors and command line errors). No output file was written.

Copyright © 2026 Mallionic Inc. All rights reserved.

SHELL SESSION

```
$ ashc hello.ash --elf -s; echo "status $?"
status 0
$ ashc missing.ash --elf -s; echo "status $?"
error: The file doesn't exit or is not a file: missing.ash
1 error generated.
status 1
```

5.7 USING ASHC FROM BUILD SYSTEMS

Because `ashc` reads one source file and writes one output, it fits naturally in `make` rules. Imports are resolved by the compiler itself, so a program made of several `.ash` files is compiled by naming only the main file.

```
# GNU make
ASHC    ?= ashc
CC      ?= gcc

app: out/main.o
    $(CC) out/main.o -o app

out/main.o: main.ash lib/*.ash
    $(ASHC) main.ash --elf -s -o out/main.o

clean:
    rm -rf out app
```

The VS Code extension runs `ashc file --elf` (or `--compile` for `@bits(16)` files) with a temporary output path every time a file is saved and reports the compiler's errors in the editor (Section 29.1).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 6

ARCHITECTURE OF THE COMPILER

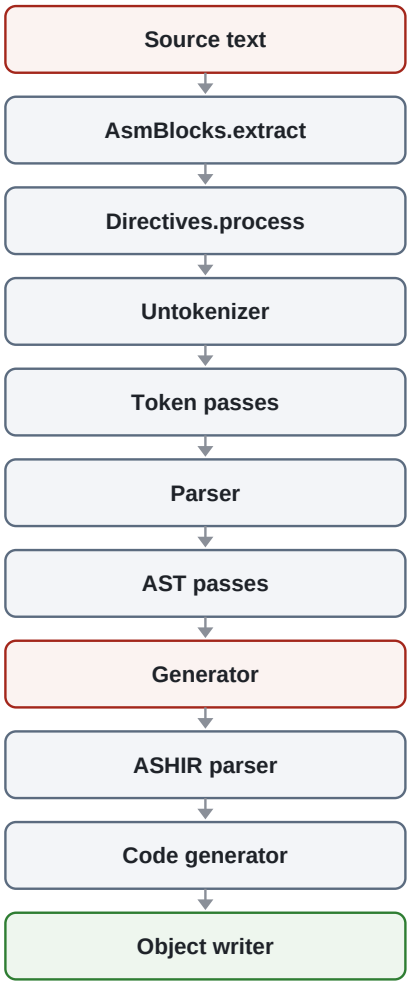
This chapter follows a program through `ashc`, from the bytes of the source file to the bytes of the object file. Knowing the stages helps to read diagnostics (each stage reports different kinds of errors), to understand why some constructs are rewritten into others, and to locate the source code of the compiler when you want to extend it.

6.1 OVERVIEW

`ashc` is organized as a classic two-part compiler. The **front end** (package `Builder` and `Builder.ash`) reads ASH and produces ASHIR, a textual intermediate representation. The **back end** (package `arch`) reads ASHIR and produces machine code wrapped in an output format. The two halves communicate only through ASHIR text, which is why the front end can be stopped after ASHIR (the default output mode) and why the back end accepts `.ashir` files directly.

Copyright © 2026 Mallionic Inc. All rights reserved.

Figure 6-1. The ashc pipeline



The `.ash` file, read as UTF-8.

Removes every `asm { }` and `asm("...")` statement and keeps its text aside; leaves `__asm(N);` in its place.

Evaluates `%if`, `%define`, `%error` ... lines; removed lines become blank so line numbers stay correct.

The lexer: characters to tokens (identifiers, numbers, strings, operators).

Macros (`define`), Redefines, Structs.expand, Typedefs and user annotations.

Builds the AST: functions, classes, globals, externs, imports, statements, expressions.

Imports merged, helpers added, Visibility, Builtins, ParamCopies, Structs, Cracks, Globals.

AST to ASHIR text; adds runtime helpers on demand; tree-shakes unused imported code.

`arch.builtin.Tokens` and `Parsing`: ASHIR text back to a list of operations.

`Backend` (x86-64) or `Backend32` (i386 / 8086): operations to machine code; `shasm` assembles `asm` payloads.

`JavaElfOBJ`, `JavaWinOBJ`, `Obj32` or a flat image (`Builder`), plus the `--exe` wrapper.

The source code of the compiler is laid out as follows:

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 6-1. Source tree of ashc

Directory	Contents
`arch/Main.java`	The driver: command line parsing, choice of output mode, orchestration of front end and back end, writing of the output file.
`Builder/Untokenizer.java`	The lexer of ASH.
`Builder/Parseconv.java`	Parser entry point and the list of AST passes; ASHIR post-processing (tree shaking, `--no-libc` runtime).
`Builder/ash/`	One class per language feature: `Exprs` (expressions), `Conditional` (control flow), `Funcs` (functions and statements), `Vars`, `Structs`, `Globals`, `Cracks`, `Macros`, `Directives`, `Imports`, `Redefines`, `Annotations`, `AsmBlocks`, `Inject` (emit), `StringConv`, `WideStrings`, `Visibility`, `Generator` (AST to ASHIR) ...
`arch/builtin/`	ASHIR lexer and parser (`Tokens`, `Parsing`), the x86-64 code generator (`Backend`, `Vars*`, `backd/*`), the `repeat` macro.
`arch/Backend32.java`	Code generator for 32-bit and 16-bit targets.
`arch/X86.java`, `arch/ALU.java`, `arch/Jumpz.java`, `arch/FloatingPoint.java`, `arch/Registers/`	Instruction encoders used by the back ends.
`arch/formatter/`	Object writers: `JavaElfOBJ` (ELF64), `JavaWinOBJ` (COFF AMD64). `arch/Obj32.java` writes ELF32 and COFF i386.
`arch/AsmBridge.java`	Glue between the code generator and the embedded `shasm` assembler.
`shasm/`	The assembler: preprocessor, expression evaluator, instruction database, encoders, ELF and COFF writers.
`nlib/`	Sources of the system libraries, embedded into the compiler by `tools/EmbedNlib.java`.

6.2 THE FRONT END

6.2.1 Extraction of assembly blocks

The very first pass works on raw characters. Assembly does not follow the lexical rules of ASH (a `;` starts a comment in assembly, `[rax+8]` is not an index expression), so `AsmBlocks.extract` finds every `asm { ... }`, `volatile asm { ... }` and string-form `asm(...)` statement, stores the assembly text in a table and replaces the statement with a placeholder call `__asm(N);`. Newlines are preserved so that every following line keeps its line number. String and comment contents are skipped while scanning, so the word `asm` inside a string is not mistaken for a statement.

6.2.2 Compile-time directives

`Directives.process` handles the lines that begin with `%` (Chapter 18). It evaluates `%if` conditions with the predefined target symbols (`ASH_BITS`, `ASH_FORMAT`, `ASH_OS` ...), keeps or removes the guarded lines, records `%define` symbols and replaces their uses, and stops the build on `%error`. Removed lines are replaced by empty lines. Because this happens before the lexer, a directive can remove any text, even text that would not be valid ASH.

Copyright © 2026 Mallionic Inc. All rights reserved.

6.2.3 The lexer (Untokenizer)

`Untokenizer` turns the text into a list of tokens. Each token has a type (`IDENT`, `NUMBER`, `FLOAT`, `STRING`, `CHAR`, an operator type such as `SHL` or `POW`, punctuation, `EOF`), its text and its line and column. Keywords are not separate token types: they are identifiers that the parser recognizes in context, which is why ASH lets you name a variable `terminate` or `ptr` (Section 7.5). The `--tokens` option prints this list.

6.2.4 Token-level passes

Four passes rewrite the token list before parsing:

Table 6-2. Token-level passes

Pass	Effect
`Macros`	Collects `define NAME value;` and replaces later uses of `NAME` by the value tokens. Macros never reach the parser.
`Redefines`	Handles `redefine` aliases (`redefine uint64 printf(...) uint64 out(...)`) by renaming calls, and turns `redefine global` definitions into exported functions with a high override right.
`Structs.expand`	Removes `struct` and `typedef struct` definitions from the stream after recording their fields, sizes and offsets.
`Typedefs.expand`	Replaces type aliases (`typedef uint32 u32;`) everywhere, and applies user-defined annotations (`@Trace(...) { ... }`) by inserting their bodies at the start of the annotated functions.

6.2.5 The parser

The parser is a hand-written recursive-descent parser spread over several classes. `Parseconv.parseProgram` loops over top-level items: `"use strict"`, override assignments (`name.override = N;`), global variables, `import`, `extern`, annotations, classes and functions. `Funcs` parses function headers and statements, `Conditional` parses control flow, `Vars` parses declarations and assignments and `Exprs` parses expressions with the precedence described in Section 10.2. The result is an abstract syntax tree (`Builder/ash/Ast.java`).

Some constructs are *lowered* to simpler ones while parsing. A `switch` becomes a `while` loop with `if` statements and a match flag (Section 11.7.1); `~x` becomes `x ^ -1`; unary minus becomes `0 - x`; `va_arg(i)` becomes a call to a selector helper; `Class.method(...)` becomes a call to `Class_method(...)`.

6.2.6 AST passes

After parsing, `Parseconv.convert` runs the following passes, in this order:

1 **Imports** — every imported file is parsed in turn and its functions, globals, externs, structs and typedefs are merged into the program. When two definitions share a name, override rights decide which one survives (Section 19.4).

2 **Helpers** — if the program uses variadic functions, a non-constant `**` or a non-constant `***`, the corresponding helper functions (`__va_get`, `__ash_pow`, `__ash_sqpow`) are added.

3 **Visibility** — rejects calls to `priv` class methods from outside the class and calls to `priv` functions of other files.

Copyright © 2026 Mallionic Inc. All rights reserved.

4 **Builtins** — rewrites `len(a)`, `length(a)`, `rows(m)`, `cols(m)` and `count(m)` into length queries, unless you defined functions with those names.

5 **ParamCopies** — a parameter that is assigned inside its function is renamed `__arg_<name>` and a local copy named like the parameter is declared at the start of the function.

6 **Structs** — rewrites `p.field`, `p->field` and chains such as `a->b->c` into typed loads and stores at byte offsets.

7 **Cracks** — moves each crack into its own function `__crack_<function>_<crack>`, rewrites `f.crack = values;` into calls, and implements `unreturnable` with a hidden global `__ret_<function>`.

8 **Globals** — rewrites every access to a global variable `g` into a typed memory access `*(T*)&g`, checks that constants (globals without `static`) are never assigned, and expands `g++`.

6.2.7 ASHIR generation

`Builder/ash/Generator.java` walks the final AST and prints ASHIR. Every expression is broken down into operations on named temporaries (`__tmp1`, `__tmp2`, ...), every control structure into labels and conditional branches. Strings become `.string` data, globals become `.gvar` / `.gdata` entries.

The generator also adds the *runtime helpers* when the program needs them. A program that uses arrays, strings or shifts by a variable amount receives the runtime block (`__ash_alloc` and `__ash_free` for heap blocks, `__ash_bounds` for bounds checking, `__ash_strlen`, `__ash_memcpy`, `__ash_str_new`, `__ash_str_set`, `__ash_matrix`, `__ash_shl`, `__ash_shr`); a program that uses none of these features receives nothing. Variadic functions add `__va_get`, a non-constant `**` adds `__ash_pow` and a non-constant `***` adds `__ash_sqpow`. Section 23.8 describes each helper.

Finally two text-level passes run on the ASHIR:

- **Tree shaking** — a function that came from an imported file is kept only if a function of the main file reaches it, directly or indirectly. Functions of the main file are always kept, because they may be called from C. This is why `import <string>;` costs nothing when you use only `strlen`.
- **--no-libc runtime** — when compiling for 64 bits without the C library, the external declarations of `calloc`, `free` and `exit` are replaced by ASHIR functions that use `mmap` and `exit_group` system calls through inline assembly.

6.3 THE BACK END

6.3.1 ASHIR parsing

The back end does not share data structures with the front end; it starts again from text. `arch.builtin.Tokens` splits the ASHIR into tokens and `Parsing` builds a list of functions, externs, global data and operations. This design keeps the two halves independent and makes the ASHIR format a real interface, which you can generate or edit yourself.

6.3.2 Code generation

The 64-bit code generator (`arch.builtin.Backend` and its helpers in `arch/builtin/backd/`) translates operations one by one into x86-64 instructions:

Copyright © 2026 Mallionic Inc. All rights reserved.

- Local variables and parameters of integer type are kept in the registers `r8` to `r15` while they fit; temporary values use `rax`, `rcx` and `rdx`.
- A first pass over each function measures which of the callee-saved registers `r12`–`r15` it uses, so that the prologue saves only those (compact epilogue).
- Leaf functions that need no stack space are emitted without a frame unless `-e` or `@beautycode(false)` is in effect.
- Calls follow the System V AMD64 convention (`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, then the stack), or the Microsoft x64 convention for `--win` external calls and `@msabi` functions.

The 32-bit and 16-bit generator (`arch/Backend32.java`) uses a simpler model: every variable has a slot in the stack frame addressed through `ebp` (`bp` in 16-bit code), arguments are pushed on the stack (`cdecl`), and the result is returned in `eax` / `ax`. Chapter 24 describes both generators in detail.

6.3.3 Assembly through shasm

When the code generator reaches an `asm` operation, it decodes the hexadecimal payload back into assembly text, replaces each variable reference (written `@var:name` in the payload) with the operand that holds the variable at that point (`r12`, `qword [rbp-0x18]`, ...), and hands the text to the embedded `shasm` assembler through `arch.AsmBridge`. The assembler runs in flat mode with the current origin and bit width and returns the encoded bytes, which are copied into the instruction stream. Because the same assembler is available as a stand-alone tool, anything that assembles with `shasm` can be pasted into an `asm` block (Chapter 26).

6.3.4 Object writers

The code generator writes into a byte buffer and records three kinds of facts: symbol definitions (functions, strings, globals), references to symbols that are not yet known or are external, and section placement requests (`@section`). The object writer then lays out sections and turns references into relocations:

Table 6-3. Object writers

Writer	Output
`JavaElfOBJ`	ELF64 relocatable object for x86-64. Code, string literals and global variables share one section named `.ashtext` (flags *alloc*, *write*, *execute*); functions and data placed with `@section` get their own sections. Empty `.rodata` and `.data` sections are present for compatibility, `.note.GNU-stack` marks the stack as non-executable, and `.comment` records the producer and the path of the source file. Public functions and `general` globals are global symbols; references between functions and to external functions use `R_X86_64_PLT32` and `R_X86_64_PC32` relocations.
`JavaWinOBJ`	COFF object for AMD64 with `IMAGE_REL_AMD64_REL32` relocations for calls to external functions such as those of `kernel32.dll`.
`Obj32`	ELF32 or COFF objects for i386.
`Builder` (flat)	A raw image: code followed by data, addresses resolved against the origin given with `@originoffs`.
`--exe` wrapper	Prepends an ELF32 header and a single read-write-execute program header to a flat 32-bit image whose entry is the first byte of the code. One extra mebibyte of zero-filled memory is reserved after the image for the built-in heap.

Copyright © 2026 Mallionic Inc. All rights reserved.

6.4 OBSERVING THE STAGES

Each stage can be observed from the command line:

Table 6-4. How to look at each stage

Stage	How to observe it
Directives	Put `%warning text` or `%tip text` lines in the source; they are printed when reached.
Lexer	`ashc file.ash --tokens`
AST passes and Generator	`ashc file.ash` (writes `file.ashir`) or `ashc file.ash --stdout`
Code generator	`objdump -d -M intel out/file.o`, or `objdump -D -b binary -m i386 out/file.bin` for flat images
Object writer	`readelf -a out/file.o`, `objdump -h -r -t out/file.o`
Timing of every stage	`ashc file.ash --elf --debug`

The following example takes one small function through the main stages. First the source:

STAGES.ASH

```
func scale(int64 v): int64 {
    return v * 3 + 1;
}

@Entry
pub func main(): int64 {
    return scale(4);           // 13
}
```

The tokens of the first line, as seen by the parser:

SHELL SESSION

```
$ ashc stages.ash --tokens -s | head -14
    0  IDENT         func                  1:1
    1  IDENT         scale                 1:6
    2  LPAREN        (                     1:11
    3  IDENT         int64                 1:12
    4  IDENT         v                     1:18
    5  RPAREN        )                     1:19
    6  COLON         :                     1:20
    7  IDENT         int64                 1:22
    8  LBRACE        {                     1:28
    9  IDENT         return                2:5
   10  IDENT         v                     2:12
   11  STAR          *                     2:14
   12  NUMBER        3                     2:16
   13  PLUS          +                     2:18
```

The ASHIR produced by the front end:

SHELL SESSION

```
$ ashc stages.ash --stdout -s
func qword scale(qword v) {
.type __tmp0 qword
    %__tmp0 mov[v]
    %__tmp0 imul[3]
    %__ret mov[__tmp0]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

SHELL SESSION (CONTINUED)

```
    %__ret add[1]
    ret qword __ret
}

[entry]
func qword main() {
    %__tmp1 mov[4]
    %__ret call[scale, __tmp1]
    ret qword __ret
}
```

The machine code produced by the back end:

SHELL SESSION

```
$ ashc stages.ash --elf -s && objdump -d -M intel out/stages.o | tail -n +7
0000000000000000 <scale>:
   0:   48 89 f8                mov    rax,rdi
   3:   49 89 c2                mov    r10,rax
   6:   4c 89 d0                mov    rax,r10
   9:   48 6b c0 03             imul   rax,rax,0x3
   d:   49 89 c2                mov    r10,rax
  10:   4c 89 d0                mov    rax,r10
  13:   49 89 c3                mov    r11,rax
  16:   4c 89 d8                mov    rax,r11
  19:   48 83 c0 01             add    rax,0x1
  1d:   49 89 c3                mov    r11,rax
  20:   4c 89 d8                mov    rax,r11
  23:   c3                      ret

0000000000000024 <main>:
  24:   55                      push   rbp
  25:   48 89 e5                mov    rbp,rsp
  28:   48 81 ec 20 00 00 00    sub    rsp,0x20
  2f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  33:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  37:   b8 04 00 00 00          mov    eax,0x4
  3c:   49 89 c4                mov    r12,rax
  3f:   b8 00 00 00 00          mov    eax,0x0
  44:   49 89 c5                mov    r13,rax
  47:   4c 89 e7                mov    rdi,r12
  4a:   e8 b1 ff ff ff          call   0 <scale>
  4f:   49 89 c5                mov    r13,rax
  52:   4c 89 e8                mov    rax,r13
  55:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  59:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  5d:   c9                      leave
  5e:   c3                      ret
```

The relocations and the symbols written by the object writer:

SHELL SESSION

```
$ ashc stages.ash --elf -s && objdump -r out/stages.o | tail -n +4
$ nm out/stages.o
0000000000000024 T main
0000000000000000 T scale
```

Copyright © 2026 Mallionic Inc. All rights reserved.

6.5 DESIGN NOTES

6.5.1 Why a textual intermediate representation

Using text between the halves of the compiler has a small cost (the back end parses ASHIR again) and large benefits: every program can be inspected at the boundary, the back end can be tested with hand-written ASHIR, other front ends (for example a generator written in Python) can target ASHIR, and a bug can be located on one side of the boundary simply by reading the ASHIR.

6.5.2 Why many small passes

Each language feature is implemented as an isolated class and, where possible, as a rewrite into features that already exist. `switch` is a loop with `if` statements, `repeat (n)` is a `for` loop with a hidden counter, a crack is an ordinary function, a global access is a pointer dereference. New features can therefore be added without touching the back end, and the back end stays small enough to be verified by reading its output.

6.5.3 Determinism

For a given source file, compiler version and build host, `ashc` always produces the same bytes: temporary names and labels are numbered in program order and no time stamps are written. Compiling the same file twice on the same machine yields identical objects, which makes byte-exact comparisons (like the ones used throughout this manual) possible.

> **NOTE**
> ELF objects written by `ashc` 1.0 also contain a few local *file* symbols that describe the build host (operating system name, kernel version, architecture, user name and home directory), and the `.comment` section records the absolute path of the source file. They do not affect linking, but they make objects built by different users differ, and they reveal information about the build machine. Remove them before distributing an object with `objcopy --strip-unneeded --remove-section=.comment file.o` (the global symbols needed for linking are kept).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 7

LEXICAL STRUCTURE

This chapter describes how ASH source text is divided into tokens: the character set, white space, comments, identifiers, keywords, literals, operators and punctuation. It also describes the transformations that happen at the token level, before parsing, such as the folding of multi-word C type names and the expansion of compound assignments.

7.1 SOURCE FILES

An ASH program is stored in one or more text files with the extension `.ash`. Files are read as UTF-8. Identifiers, keywords and operators use ASCII characters only; other characters may appear inside comments, string literals and character literals.

A file does not need any particular header. The first non-blank item of a file may be a `"use strict";` declaration (Chapter 21), compile-time directives (Chapter 18), imports, externs, globals, annotations, classes or functions, in any order.

7.2 WHITE SPACE AND LINE STRUCTURE

Spaces, tabs, carriage returns and line feeds separate tokens and are otherwise ignored. Line ends have no syntactic meaning, with three exceptions:

- a compile-time directive (`%if`, `%define` ...) occupies exactly one line;
- a `define` without a terminating `;` ends at the end of its line;
- rows of a `matrix` literal are delimited by lines (Section 13.7).

The line and column of every token are recorded and used in diagnostics.

7.2.1 Semicolons

The semicolon terminates statements. The parser is tolerant: in most positions a missing semicolon at the end of a line is accepted, because each statement form is recognized by its first tokens. The following program compiles and runs:

Example 7-1. Statements without semicolons

```ash
@Entry
pub func main(): int64 {
    int64 a = 5
    int64 b = 6
    printf("%d\n", a + b)
    return 0
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
11
```

Global variable declarations are customarily written without a semicolon (`general static uint64 total = 0`). Inside functions this manual always writes the semicolon, which avoids surprises and keeps the code readable for C programmers.

7.3 COMMENTS

ASH has the two comment forms of C:

- `//` starts a comment that extends to the end of the line.
- `/*` starts a comment that extends to the next `*/`. Block comments do **not** nest: the first `*/` ends the comment.

A comment is equivalent to white space. Comments are recognized everywhere except inside string and character literals. Documentation comments written as `/** ... */` are ordinary block comments; the VS Code extension highlights them differently.

ASH

```
/* A block comment
   spanning several lines. */
func area(int64 w, int64 h): int64 {     // a line comment
    return w * h;                          /* also fine here */
}
```

NOTE

Inside `asm { }` blocks the assembly comment character `;` also starts a comment, because the block is handed to the assembler unchanged (Chapter 20).

7.4 IDENTIFIERS

An identifier is a sequence of letters (`A`–`Z`, `a`–`z`), digits (`0`–`9`) and underscores (`_`) that does not begin with a digit. Identifiers are case-sensitive: `total`, `Total` and `TOTAL` are three different names. There is no length limit.

Names that begin with two underscores (`__tmp1`, `__ash_alloc`, `__g_counter`, `__arg_n`, `__crack_f_x`, `__ret_f`, `__va0`) are used by the compiler for generated temporaries, helpers and symbols. Do not declare names of this form in your programs.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 7-1. Reserved name prefixes used by the compiler

Prefix	Used for
`__tmp`*N*	Temporaries created for sub-expressions.
`__L`*N*	Labels created for control flow.
`__str`*N*, `__wstr`*N*	String literals and wide string literals.
`__g_`*name*	Symbols of `ungeneral` global variables.
`__ash_`	Runtime helper functions (`__ash_alloc`, `__ash_bounds` ...).
`__arg_`*name*	Original parameter when a parameter is modified (Section 12.2).
`__crack_f_`*name*	Functions generated for cracks.
`__ret_`*f*	Pending return value of `unreturnable`.
`__vac`, `__va0`–`__va7`	Hidden parameters of variadic functions.
`__rp`*N*, `__sw`*N*	Hidden counters of `repeat` and subjects of `switch`.

7.5 KEYWORDS

ASH keywords are *contextual*: the lexer produces identifiers, and the parser recognizes a keyword only where a keyword can appear. In practice this means that a few keywords can also be used as variable names when there is no ambiguity. For example, a variable may be called `terminate` or `ptr`:

Example 7-2. Contextual keywords used as variable names

ASH

```
@Entry
pub func main(): int64 {
    int64 terminate = 1;       // a variable, not the statement
    int64 ptr = 2;             // 'ptr' is also a type name
    terminate = terminate + ptr;
    printf("%d\n", terminate);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3
```

Even so, using keywords as names makes programs harder to read. The following words should be treated as reserved:

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 7-2. Keywords of ASH

Group	Keywords
Declarations	`func` `class` `struct` `typedef` `define` `redefine` `global` `extern` `import` `matrix` `runnable`
Visibility and storage	`pub` `priv` `general` `ungeneral` `static` `const` `volatile`
Control flow	`if` `elif` `else` `switch` `case` `default` `while` `for` `repeat` `loop` `break` `continue` `return` `terminate` `unreturnable` `unreturn`
Low level	`asm` `__asm__` `emit` `emit8` `emit16` `emit32` `emit64` `ashley`
Types	see Table 8-2: `int8` ... `uint64`, `byte` `word` `dword` `qword`, `char` `bool` `string` `ptr` `void` `float` `double`, the C names `int` `short` `long` `unsigned` `signed` `size_t` ...
Constants	`true` `false` (predefined, see Section 8.4), `NULL` `null` `nil` (from `<null>` and `<stdlib>`)
Built-in queries	`va_count` `va_arg` `len` `length` `rows` `cols` `count`

7.6 LITERALS

7.6.1 Integer literals

Table 7-3. Integer literal forms

Form	Example	Notes
Decimal	`0`, `42`, `18446744073709551615`	A sequence of digits. Values up to 2^64-1 are accepted; values above 2^63-1 are stored as their 64-bit unsigned bit pattern.
Hexadecimal	`0xFF`, `0X7FFF`, `0xcafe`	`0x` or `0X` followed by hexadecimal digits (case-insensitive). No digit separators.
Binary	`0b1011`	`0b` or `0B` followed by binary digits. See the caution below.
Character	`'A'`, `'\n'`	The code of a single character (see Section 7.6.3).

An integer literal may be followed by the C suffixes `u`, `U`, `l`, `L` in any combination of up to three letters (`U`, `L`, `UL`, `LL`, `ULL`, `llu` ...). The suffix is accepted for compatibility with C code and is ignored: the type of a literal is determined by the context in which it is used (Section 8.2.4).

Example 7-3. Integer literals

ASH

```
@Entry
pub func main(): int64 {
    int64 dec = 255;
    int64 hex = 0xFF;
    int64 chr = 'A';
    uint64 big = 18446744073709551615ULL;
    int64 neg = -1234567;
    printf("%d %d %d %llu %lld\n", dec, hex, chr, big, neg);
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
255 255 65 18446744073709551615 -1234567
```

A minus sign is not part of a literal: `-5` is the unary minus operator applied to the literal `5`, which the parser represents as `0 - 5` (Section 10.8).

WARNING

In `ashc` 1.0 a negative constant whose magnitude does not fit in 31 bits (for example `-5000000000` or `0 - 2147483648`) is computed incorrectly inside function bodies, because the subtraction is encoded with a 32-bit immediate. Write such constants in hexadecimal two's complement (`0xFFFFFFFED5FA0E00` is -5000000000), or subtract a variable (`int64 k = 5000000000; int64 n = 0 - k;`). Both forms work correctly.

WARNING

In `ashc` 1.0 binary literals are recognized by the lexer, by compile-time directives and by the initializers of global variables, but a binary literal used inside a function body is not understood by the back end and the build fails with a parse error in the ASHIR stage. Use hexadecimal literals inside functions until this is fixed.

Digit separators (`1_000`) and octal prefixes (`0o17`, C-style `017`) are not supported. A leading `0` does not make a number octal: `017` is seventeen.

7.6.2 Floating-point literals

A literal with a decimal point and digits on both sides (`1.5`, `3.14159`) is a floating-point literal. Floating-point support in ASH 1.0 is limited; see Section 8.8 before using `float` and `double` values.

7.6.3 Character literals

A character literal is a single character, or an escape sequence, enclosed in single quotes: `'A'`, `'0'`, `'\n'`, `'\x1B'`. Its value is the code of the character (`'A'` is 65). Character literals are integers and can be used wherever an integer is expected.

Table 7-4. Escape sequences

Escape	Meaning
`\n` `\r` `\t`	New line (10), carriage return (13), horizontal tab (9).
`\0`	NUL (0).
`\a` `\b` `\f` `\v`	Bell (7), backspace (8), form feed (12), vertical tab (11).
`\\` `\'` `\"`	Backslash, single quote, double quote.
`\x`*hh*	The byte with hexadecimal value *hh* (two digits).
`\`*ooo*	The byte with octal value *ooo* (one to three octal digits after the backslash), as in `\033`.
`\u`*hhhh*, `\U`*hhhhhhhh*	The Unicode code point *h...*, encoded in UTF-8 in ordinary strings and as one UCS-2 unit in wide strings.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 7-4. Escape sequences in strings

ASH

```
@Entry
pub func main(): int64 {
    printf("tab[\t] hex[\x41] octal[\101] unicode[A] quote[\"] backslash[\\]\n");
    printf("%d %d %d\n", '\n', '\x41', '\0');
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
tab[    ] hex[A] octal[A] unicode[A] quote["] backslash[\]
10 65 0
```

7.6.4 String literals

A string literal is a sequence of characters enclosed in double quotes. Escape sequences are decoded as in Table 7-4. A string literal cannot span lines; to write text over several lines use a *triple-quoted* string, which starts with `"""` and ends with the next `"""`. Inside a triple-quoted string, line breaks are part of the text and single `"` characters need no escape.

Adjacent string literals are concatenated at compile time, as in C: `"Hello " "world"` is the same literal as `"Hello world"`. This works for ordinary and triple-quoted strings and is the way to write a long string over several lines of source.

Every string literal is stored once in the read-only data of the program, followed by a terminating NUL byte, so it can be passed directly to C functions.

Example 7-5. Concatenated and triple-quoted string literals

ASH

```
@Entry
pub func main(): int64 {
    string greeting = "Hello, " "concatenated " "world";
    string poem = """Roses are red,
"quotes" need no escape,
and lines keep their breaks.""";
    printf("%s\n%s\n", greeting, poem);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Hello, concatenated world
Roses are red,
"quotes" need no escape,
and lines keep their breaks.
```

7.6.5 Wide string literals

A string literal immediately preceded by the letter `L`, with no space in between, is a *wide string*: each character is stored as a 16-bit little-endian code unit and the string ends with a 16-bit zero. Wide strings are what UEFI firmware and many Windows APIs expect (`CHAR16*`, `WCHAR*`). Adjacent wide literals are concatenated. The value of a wide string literal is the address of its first code unit.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 7-6. A wide string is an array of 16-bit units

ASH

```
@Entry
pub func main(): int64 {
    uint16* w = L"ASH";
    printf("%d %d %d %d\n", w[0], w[1], w[2], w[3]);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
65 83 72 0
```

7.7 OPERATORS AND PUNCTUATION

The following character sequences are operators or punctuators. The lexer always takes the longest sequence that forms a token, so `a***b` is `a *** b` and `x<<=2` is `x <<= 2`.

Table 7-5. Operators and punctuators

Tokens	Use
`+ - * / %`	Arithmetic (Section 10.4).
`** ***`	Power and repeated squaring (Section 10.5).
`<< >>`	Shifts.
`& | ^ ~`	Bitwise operators; `&` is also the address-of operator.
`== != < <= > >=`	Comparisons.
`&& || !`	Logical operators (in conditions).
`=`	Assignment.
`+= -= *= /= %= <<= >>= &= |= ^=`	Compound assignment (Section 7.8.2).
`++ --`	Increment and decrement.
`#`	Length of an array or string (Section 13.4).
`. ->`	Member access, method call, crack call.
`...`	Variadic parameter list.
`@`	Annotation.
`( ) [ ] { } , ; :`	Grouping, indexing, blocks, separators, return type.

7.8 TOKEN-LEVEL TRANSFORMATIONS

A few constructs are rewritten by the lexer before the parser sees them. They are described here because they explain some of the rules of later chapters.

7.8.1 Multi-word C type names

Sequences of the C type words `unsigned`, `signed`, `long`, `short`, `int` and `char` are folded into a single ASH type name:

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 7-6. Folding of C type names

C spelling	ASH type
`unsigned char` / `signed char`	`uint8` / `int8`
`short`, `short int`, `signed short`	`int16`
`unsigned short`, `unsigned short int`	`uint16`
`int`, `signed`, `signed int`	`int32`
`unsigned`, `unsigned int`	`uint32`
`long`, `long int`, `long long`, `long long int`	`int64`
`unsigned long`, `unsigned long long`	`uint64`

The sequence must start with `unsigned`, `signed`, `long` or `short` to be folded; a lone `int` or `char` is already an ASH type name.

7.8.2 Compound assignment

`x op= e` is rewritten into `x = x op (e)` for every operator `+ - * / % << >> & | ^`. The right-hand side is parenthesized, so `x *= a + b` multiplies `x` by the sum. Member increments `p.n++` and `q->n--` are rewritten into `p.n = p.n + 1` and `q->n = q->n - 1`.

Example 7-7. Compound assignment is a rewrite

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 10;
    x *= 2 + 1;         // x = x * (2 + 1)  -> 30
    x -= 4;             // 26
    x <<= 1;            // 52
    return x;
}
```

OUTPUT — EXIT STATUS 52

```
(no output)
```

ASHIR

```
[entry]
func qword main() {
.type x qword
    %x mov[10]
    %x mov[x]
.type __tmp0 qword
    %__tmp0 mov[2]
    %__tmp0 add[1]
    %x imul[__tmp0]
    %x mov[x]
    %x sub[4]
    %x mov[x]
    %x shl[1]
    ret qword x
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

7.8.3 The ashley intrinsics

The identifier `ashley` followed by a dot introduces an intrinsic of the compiler: `ashley.InjectByte(...)`, `ashley.InjectBytes(...)`, `ashley.InjectWord(...)`, `ashley.InjectDword(...)` and `ashley.InjectQword(...)`. They are the long names of `emit`, `emit16`, `emit32` and `emit64` (Section 20.3).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 8

TYPES

ASH is statically typed: every variable, parameter, return value and global has a type fixed at compile time. Types decide how much memory a value occupies, which instructions are used to load, store, extend and compare it, and how it is passed to other functions. This chapter describes every type of ASH 1.0 and shows how each one is represented in ASHIR and in machine code.

8.1 OVERVIEW

Table 8-1. Categories of types

Category	Types
Integers	Signed and unsigned integers of 8, 16, 32 and 64 bits, under ASH names (`int32`, `uint8` ...), x86 names (`byte`, `dword` ...), C names (`int`, `unsigned long` ...) and `<stdint.h>` names (`uint32_t` ...).
Boolean	`bool`, an 8-bit integer with the constants `true` (1) and `false` (0).
Character	`char`, an 8-bit integer holding a character code.
Strings	`string`, a pointer to a NUL-terminated sequence of bytes; `char[]`, an array of characters.
Pointers	`T*` for any type `T`, `void*`, and the untyped `ptr`.
Arrays	`T[N]`, `T[]`, `T[R][C]` and `matrix`: heap blocks that carry their length (Chapter 13).
Structures	`struct` and `typedef struct` types with C layout (Chapter 14).
Floating point	`float` / `float32` and `double` / `float64`: reserved, with limited support in 1.0 (Section 8.8).
Void	`void`: the return type of functions that return nothing.

8.2 INTEGER TYPES

Every integer type of ASH is one of eight *machine types*: signed or unsigned, of 8, 16, 32 or 64 bits. Many names are accepted for each one so that code ported from C, from x86 assembly or from other languages can keep its spelling.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 8-2. Integer types and their aliases

Machine type	Bits	Signed	Names accepted
`int8`	8	yes	`int8`, `int8_t`, `schar`, `signed char`
`uint8`	8	no	`uint8`, `uint8_t`, `byte`, `uchar`, `unsigned char`, `char`, `bool`
`int16`	16	yes	`int16`, `int16_t`, `short`, `short int`, `signed short`
`uint16`	16	no	`uint16`, `uint16_t`, `word`, `ushort`, `unsigned short`
`int32`	32	yes	`int32`, `int32_t`, `int`, `signed`, `signed int`
`uint32`	32	no	`uint32`, `uint32_t`, `dword`, `uint`, `unsigned`, `unsigned int`
`int64`	64	yes	`int64`, `int64_t`, `long`, `long long`, `llong`, `intptr_t`, `ssize_t`, `ptrdiff_t`
`uint64`	64	no	`uint64`, `uint64_t`, `qword`, `ulong`, `ullong`, `unsigned long`, `unsigned long long`, `size_t`, `uintptr_t`

The range of each type is the usual two's complement range:

Table 8-3. Ranges of the integer types

Type	Minimum	Maximum
`int8`	-128	127
`uint8`	0	255
`int16`	-32 768	32 767
`uint16`	0	65 535
`int32`	-2 147 483 648	2 147 483 647
`uint32`	0	4 294 967 295
`int64`	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
`uint64`	0	18 446 744 073 709 551 615

8.2.1 Integers in ASHIR

In ASHIR every integer variable is declared with `.type name class`, where *class* is `byte`, `word`, `dword` or `qword` for unsigned and plain values, and `int8`, `int16` or `int32` for the narrow signed types. Only the narrow signed types keep their sign in ASHIR, because they are the only ones whose extension to a wider register depends on the sign. A signed 64-bit value needs no extension, so `int64` is simply `qword`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 8-1. How integer types appear in ASHIR

ASH

```
@Entry
pub func main(): int64 {
    int8   a = -1;
    uint8  b = 200;
    int16  c = -300;
    uint16 d = 60000;
    int32  e = -70000;
    uint32 f = 4000000000;
    int64  g = -5;
    uint64 h = 5;
    return 0;
}
```

ASHIR

```
[entry]
func qword main() {
.type a int8
.type b byte
.type c int16
.type d word
.type e int32
.type f dword
.type g qword
.type h uint64
    %__tmp0 mov[0]
    %__tmp0 sub[1]
    %a mov[__tmp0]
    %b mov[200]
    %__tmp1 mov[0]
    %__tmp1 sub[300]
    %c mov[__tmp1]
    %d mov[60000]
    %__tmp2 mov[0]
    %__tmp2 sub[70000]
    %e mov[__tmp2]
    %f mov[4000000000]
    %g mov[0]
    %g sub[5]
    %h mov[5]
    ret qword 0
}
```

8.2.2 Widening: sign and zero extension

When a narrow value is used in a wider context (assigned to a wider variable, used in arithmetic, passed as an argument), it is *extended* to the wider size. Unsigned types are **zero-extended** (the new high bits are 0) and signed types are **sign-extended** (the new high bits copy the sign bit), exactly as in C. On x86-64 the compiler uses `movzx` for zero extension, `movsx` for 8-bit and 16-bit signed values and `movsxd` for 32-bit signed values.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 8-2. Sign extension versus zero extension

ASH

```
@Entry
pub func main(): int64 {
    int8   s = -2;
    uint8  u = 254;          // same bit pattern as s: 0xFE
    int64  ws = s;           // sign-extended: -2
    int64  wu = u;           // zero-extended: 254
    int32  m = -7;
    int64  wm = m;           // movsxd: -7
    printf("%d %d %d\n", ws, wu, wm);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-2 254 -7
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <main>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 60 00 00 00    sub    rsp,0x60
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 00 00 00 00          mov    eax,0x0
  20:   49 89 c4                mov    r12,rax
  23:   4c 89 e0                mov    rax,r12
  26:   48 83 e8 02             sub    rax,0x2
  2a:   49 89 c4                mov    r12,rax
  2d:   4c 89 e0                mov    rax,r12
  30:   41 88 c5                mov    r13b,al
  33:   b0 fe                   mov    al,0xfe
  35:   41 88 c6                mov    r14b,al
  38:   44 88 e8                mov    al,r13b
  3b:   0f b6 c0                movzx  eax,al
  3e:   48 0f be c0             movsx  rax,al
  42:   49 89 c7                mov    r15,rax
  45:   44 88 f0                mov    al,r14b
  48:   0f b6 c0                movzx  eax,al
  4b:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
  4f:   b8 00 00 00 00          mov    eax,0x0
  54:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  58:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
  5c:   48 83 e8 07             sub    rax,0x7
  60:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  64:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
  68:   89 45 c8                mov    DWORD PTR [rbp-0x38],eax
  6b:   8b 45 c8                mov    eax,DWORD PTR [rbp-0x38]
  6e:   48 63 c0                movsxd rax,eax
  71:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  75:   b8 00 00 00 00          mov    eax,0x0
  7a:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  7e:   48 8d 05 53 00 00 00    lea    rax,[rip+0x53]        # d8 <__str0>
  85:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  89:   4c 89 f8                mov    rax,r15
  8c:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
  90:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  94:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  98:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
  9c:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
  a0:   48 8b 7d b8             mov    rdi,QWORD PTR [rbp-0x48]
  a4:   48 8b 75 b0             mov    rsi,QWORD PTR [rbp-0x50]
  a8:   48 8b 55 a8             mov    rdx,QWORD PTR [rbp-0x58]
  ac:   48 8b 4d a0             mov    rcx,QWORD PTR [rbp-0x60]
  b0:   b8 00 00 00 00          mov    eax,0x0
  b5:   e8 00 00 00 00          call   ba <main+0xba>
  ba:   48 31 c0                xor    rax,rax
  bd:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  c1:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  c5:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  c9:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  cd:   48 89 ec                mov    rsp,rbp
  d0:   5d                      pop    rbp
  d1:   c3                      ret
  d2:   00 00                   add    BYTE PTR [rax],al
  d4:   00 00                   add    BYTE PTR [rax],al
        ...
```

In the disassembly, look for `movsx` and `movsxd` where the signed variables are widened; unsigned variables are kept zero-extended in their registers, so a plain `mov` suffices for them.

8.2.3 Narrowing: truncation on store

When a value is stored into a narrower variable, only its low bits are kept. Arithmetic on integers wraps around modulo 2^N, where N is the width of the variable that receives the result; there is no overflow trap.

Example 8-3. Wrap-around and truncation

ASH

```
@Entry
pub func main(): int64 {
    uint8  a = 250;
    a = a + 10;                  // 260 mod 256 = 4
    int8   b = 127;
    b = b + 1;                   // wraps to -128
    uint16 c = 65535;
    c = c + 1;                   // 0
    int16  d = -32768;
    d = d - 1;                   // 32767
    uint32 e = 4294967295;
    e = e + 1;                   // 0
    int64  big = 300;
    uint8  low = big;            // low 8 bits of 300 = 44
    printf("%d %d %d %d %u %d\n", a, b, c, d, e, low);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
4 -128 0 32767 0 44
```

Copyright © 2026 Mallionic Inc. All rights reserved.

8.2.4 The type of a literal

An integer literal has no fixed type. Its value is checked against the type of the variable that receives it: a literal that cannot be represented is a compile-time error, even without strict mode. Both signed and unsigned interpretations are accepted for a given width, so `uint8 x = -1` is rejected only in strict mode, while `uint8 x = 300` is always rejected.

Example 8-4. A literal that does not fit

ASH

```
@Entry
pub func main(): int64 {
    uint8 x = 300;
    return x;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: the value 300 does not fit in 8 bits (allowed range: -128..255)
1 error generated.
```

8.2.5 Operations that depend on signedness

Addition, subtraction, multiplication (low half), bitwise operations and left shifts produce the same bits for signed and unsigned operands. Four operations do depend on the sign, and ASH chooses the instruction from the types of the operands:

Table 8-4. Signed and unsigned instructions

Operation	Signed operands	Unsigned operands
Division `/` and `%`	`idiv` (truncates toward zero)	`div`
Right shift `>>`	`sar` (copies the sign bit)	`shr` (shifts in zeros)
Comparisons `< <= > >=`	`jl jle jg jge`	`jb jbe ja jae`
Widening	`movsx` / `movsxd`	`movzx` / `mov`

Example 8-5. Signed and unsigned division and shifts

ASH

```
@Entry
pub func main(): int64 {
    int64  sd = -7 / 2;                      // -3 (truncation toward zero)
    int64  sm = -7 % 2;                      // -1 (sign of the dividend)
    uint64 ud = 0xFFFFFFFFFFFFFFFF / 2;      // unsigned: 9223372036854775807
    int64  sr = -16 >> 2;                    // arithmetic shift: -4
    uint64 ur = 0x8000000000000000 >> 4;     // logical shift: 0x0800000000000000
    printf("%d %d %llu %d %llx\n", sd, sm, ud, sr, ur);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-3 -1 9223372036854775807 -4 800000000000000
```

Copyright © 2026 Mallionic Inc. All rights reserved.

8.3 INTEGER WIDTH AND THE TARGET

The widths in Table 8-2 are those of the 64-bit target. On narrower targets the *machine word* limits the width of every integer:

Table 8-5. Effective integer widths per target

Target	Effective widths
64-bit (default)	8, 16, 32 and 64 bits as declared.
32-bit (`@bits(32)`)	8, 16 and 32 bits as declared; `int64`, `uint64` and their aliases are held in 32 bits.
16-bit (`@bits(16)`)	8 and 16 bits as declared; every wider type is held in 16 bits.

The following 16-bit program, executed in the emulator, shows that `int32` and `int64` behave as 16-bit values on that target: `70000` becomes `70000 mod 65536 = 4464`, and `65535 + 1` becomes 0.

Example 8-6. Integer width on the 16-bit target

ASH

```
@bits(16)
@originoffs(0x100)

@Entry
pub func _start(): int32 {
    int32 x = 70000;          // stored in 16 bits: 4464
    int64 y = 65535;
    y = y + 1;                // 0
    return x + y;             // AX = 4464
}
```

OUTPUT — EXIT STATUS 0

```
ax = 4464  bp = 0x0
```

TIP

Code meant to be portable across targets should use the narrowest type that is wide enough for its values (`int16` for values that always fit in 16 bits, and so on) and test `ASH_BITS` with `%if` where a wider type is essential (Chapter 18).

8.4 BOOLEAN

`bool` is an unsigned 8-bit integer. The words `true` and `false` are predefined constants with the values 1 and 0. They are predefined with override right -1 (Section 19.4), which means that a program or library may define its own `true` and `false` and its definitions win. The library `<stdbool>` also defines `TRUE` and `FALSE`.

Conditions in `if`, `while` and `for` do not require a `bool`: any integer expression can be compared explicitly (`if (flags != 0)`). A `bool` variable holds whatever value is stored into it; ASH does not normalize non-zero values to 1.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 8-7. Booleans

ASH

```
"use strict";

func is_even(int64 n): bool {
    if ((n & 1) == 0) { return true; }
    return false;
}

@Entry
pub func main(): int64 {
    bool a = true;
    bool b = false;
    printf("a=%d b=%d is_even(4)=%d is_even(7)=%d\n", a, b, is_even(4), is_even(7));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
a=1 b=0 is_even(4)=1 is_even(7)=0
```

8.5 CHARACTERS

`char` is an 8-bit unsigned integer that by convention holds a character code. A character literal such as `'A'` is an integer and can be assigned to any integer type. `char[]` declares an array of characters, which is the mutable counterpart of `string` (Section 13.8.1).

Example 8-8. Characters are small integers

ASH

```
@Entry
pub func main(): int64 {
    char c = 'A';
    c = c + 2;                          // 'C'
    int64 code = c;
    char high = 200;                    // char is unsigned: 200, not -56
    int64 h = high;
    printf("%c %d %d\n", c, code, h);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
C 67 200
```

8.6 STRINGS

A value of type `string` is the address of a sequence of bytes that ends with a NUL (0) byte, which is exactly what C calls `char*` (or `const char*`). String literals are stored in the read-only data of the program; assigning a literal to a `string` variable copies only the address.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 8-6. Operations on string values

Operation	Meaning
`string s = "text";`	`s` holds the address of the literal.
`s = "other";`	`s` now holds the address of another literal.
`s[i]`	Reads the byte at index `i` (no bounds check: a `string` has no length header).
`printf("%s", s)`	Passes the address to C.
`f(s)`	Passes the address to an ASH function with a `string`, `uint8*` or `ptr` parameter.

Example 8-9. String values are addresses

ASH

```
@Entry
pub func main(): int64 {
    string s = "Hello";
    printf("%s, first=%c second=%c\n", s, s[0], s[1]);
    s = "World";
    printf("%s\n", s);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Hello, first=H second=e
World
```

WARNING

A string literal lives in read-only memory: writing through a `string` that points to a literal (`s[0] = 'J'`) crashes the program with a segmentation fault. Modify text in a `char[]` or `uint8[N]` buffer instead. The `+` operator does not concatenate strings at run time; use adjacent literals for constant text and `strcpy` / `strcat` from `<string>` for run-time text (Section 27.2). The length operator `#` is meaningful on arrays, not on `string` values; use `strlen`.

8.7 POINTERS

A pointer type is written by appending `*` to a type: `int64*`, `uint8*`, `Persona*`, `void*`. The untyped `ptr` is equivalent to an address-sized unsigned integer. Pointers are 64 bits wide on the 64-bit target, 32 bits on the 32-bit target and 16 bits on the 16-bit target.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 8-7. Pointer operations

Operation	Meaning
`&x`	Address of a variable, array element block, global or function.
`*p`	The value at address `p`, read with the width of the pointed type.
`*p = v;`	Store `v` at address `p` with the width of the pointed type.
`p[i]`	The element `i` of type `T` at address `p + i * sizeof(T)`.
`p + n`, `p - n`	Address arithmetic **in bytes**: `p + 8` is the next `int64` after `p`.
`q - p`	Difference of two addresses in bytes.
`(T*)p`	Reinterpret an address as a pointer to `T`.

IMPORTANT

Unlike C, adding an integer to a pointer does **not** scale by the size of the pointed type. `p + 1` is the address of the next *byte*. Indexing with `p[i]` does scale. Prefer indexing; when you need explicit address arithmetic, multiply by the element size yourself.

Example 8-10. Pointer arithmetic is in bytes, indexing is in elements

ASH

```
@Entry
pub func main(): int64 {
    int64[4] v;
    v[0] = 10; v[1] = 20; v[2] = 30; v[3] = 40;
    int64* p = &v;
    int64* q = p + 8;            // 8 bytes later: element 1
    int64 dist = q - p;          // 8
    int64 a = *q;                // 20
    int64 b = p[2];              // 30: indexing scales by 8
    int64 c = *(p + 24);         // 40
    uint8* bytes = &v;
    int64 d = bytes[16];         // low byte of v[2]: 30
    printf("%d %d %d %d %d\n", dist, a, b, c, d);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
8 20 30 40 30
```

8.7.1 Function pointers

The address of a function, written `&f` or simply `f` where a value is expected, is a pointer that can be passed to C (for callbacks such as `qsort` or `atexit`), stored in a variable of type `int64`, `uint64`, `ptr` or `void*`, and called through a member chain such as `table->handler(args)`. In ASHIR the address of a function is taken with the `faddr` operation. Section 12.7 describes function pointers in detail.

Copyright © 2026 Mallionic Inc. All rights reserved.

8.8 FLOATING-POINT TYPES

The names `float` / `float32` and `double` / `float64` are reserved for IEEE-754 single and double precision values, and the ASHIR format has the classes `float32` and `float64`. In ASH 1.0 their support is limited:

- A 64-bit hexadecimal literal assigned to a `double` is stored as an IEEE-754 *bit pattern*: `double one_and_half = 0x3FF8000000000000;` stores 1.5. The `<types>` library uses this to provide `inf()`, `nan()` and related constants.
- Decimal literals such as `1.5` are rejected inside functions ("the literal is floating point"), and arithmetic between `double` values is not supported by the 1.0 back end.
- `printf("%f", ...)` cannot be used with ASH `double` values, because variadic floating-point arguments are not passed in `xmm` registers.

For numeric work without an FPU, `<math>` provides Q16.16 fixed-point arithmetic, including square root, sine, cosine and tangent (Section 27.3).

8.9 VOID

`void` is the return type of a function that returns no value. In such functions `return;` (or reaching the closing brace) leaves the function. `void*` is a pointer to memory of unspecified type, compatible with every other pointer.

8.10 CASTS

A cast is written `(T)expr` where `T` is a type name, optionally followed by `*`. Casts are used:

- to reinterpret an address: `*(int64*)a` reads an `int64` from the address held in `a` (common in `qsort` comparison functions);
- to make a narrowing assignment explicit, which strict mode requires (Chapter 21): `uint32 low = (uint32)(big & 0xFFFFFFFF);`
- to request sign extension of a value: `(int32)0xFFFFFFFF` is -1 when widened.

> **IMPORTANT**
> In ASH 1.0 a cast to a narrower integer type does not truncate the value inside an expression: `(uint8)300` is still 300 when it is used as a 64-bit value. Truncation happens when the value is *stored* into a narrower variable, or explicitly with a mask: `x & 0xFF`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 8-11. Casts

ASH

```
func compare(int64 a, int64 b): int64 {
    int64 x = *(int64*)a;          // reinterpret the address as int64*
    int64 y = *(int64*)b;
    return x - y;
}

@Entry
pub func main(): int64 {
    int64 a = 5;
    int64 b = 9;
    int64 r = compare(&a, &b);
    int64 s = (int32)0xFFFFFFFF;   // sign-extended: -1
    int64 big = 300;
    int64 masked = big & 0xFF;     // explicit truncation: 44
    printf("%d %d %d\n", r, s, masked);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-4 -1 44
```

8.11 TYPE ALIASES

`typedef` *type name*`;` introduces a new name for an existing type. Aliases are replaced at the token level before parsing, so they are completely equivalent to the original type. Aliases of aliases are resolved, and aliases declared in an imported file are visible in the file that imports it.

Example 8-12. Type aliases

ASH

```
typedef uint32 u32;
typedef int64 word_t;
typedef uint8* bytes;

func sum(word_t a, word_t b): word_t {
    return a + b;
}

@Entry
pub func main(): int64 {
    u32 small = 7;
    word_t total = sum(small, 35);
    bytes text = "typedef";
    printf("%d %c\n", total, text[0]);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
42 t
```

Copyright © 2026 Mallionic Inc. All rights reserved.

8.12 SIZES AND ALIGNMENT

Table 8-8. Size and natural alignment on the 64-bit target

Type	Size	Alignment	Notes
`int8`, `uint8`, `bool`, `char`	1	1 |	
`int16`, `uint16`	2	2 |	
`int32`, `uint32`	4	4 |	
`int64`, `uint64`	8	8 |	
Pointers, `string`, `ptr`	8	8	4 and 2 bytes on the 32-bit and 16-bit targets.
Arrays (variable)	8	8	The variable holds a pointer to a heap block (Section 13.2).
`struct`	sum of fields + padding	largest field	C layout (Section 14.2).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 9

VARIABLES AND STORAGE

A variable is a named piece of storage with a type. ASH has three kinds of variables: *local variables*, declared inside a function; *parameters*, declared in a function header (Chapter 12); and *global variables*, declared at the top level of a file. This chapter describes how each kind is declared, initialized and accessed, and where the compiler stores it.

9.1 LOCAL VARIABLES

9.1.1 Declaration

A local variable is declared with a type followed by a name and an optional initializer:

```
type name [= expression] ;
```

The type may be any type of Chapter 8, including pointer types (`uint8* p`), array types (`int64[] v`, `int64[8] buf`, `int64 buf[8]`) and structure types (`Persona p`). Only one variable is declared per statement.

Example 9-1. Declaring and initializing local variables

ASH

```ash
@Entry
pub func main(): int64 {
    int64 count = 10;
    uint8 small = 200;
    int64 total = count * 2 + small;
    string name = "ASH";
    int64 zero;                 // no initializer: starts at 0
    printf("%s %d %d\n", name, total, zero);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
ASH 220 0
```

ASHIR

```ashir
[entry]
func qword main() {
.type count qword
.type small byte
.type total qword
.type name qword
.type zero qword
    %count mov[10]
    %small mov[200]
.type __tmp0 qword
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
    %__tmp0 mov[count]
    %__tmp0 imul[2]
    %total mov[__tmp0]
    %total add[small]
    %name addr[__str0]
    %zero mov[0]
    %__tmp1 addr[__str1]
    %__tmp2 mov[name]
    %__tmp3 mov[total]
    %__tmp4 mov[zero]
    call[printf, __tmp1, __tmp2, __tmp3, __tmp4]
    ret qword 0
}

.string __str0 "ASH"
.string __str1 "%s %d %d\n"
```

9.1.2 Default initialization

A local variable declared without an initializer starts with the value 0. Arrays declared with a size are allocated zero-filled. This is a guarantee of ASH (unlike C, where an uninitialized local has an indeterminate value): the compiler emits an explicit `mov[0]` for scalars, and arrays are obtained from a zeroing allocator.

9.1.3 Scope and lifetime

A local variable can be used from the point of its declaration to the end of the function, **including after the end of the block in which it was declared**: in ASH 1.0 the scope of every local is the whole remaining body of the function, not the enclosing block. The following program is valid and prints 5:

Example 9-2. Locals are visible until the end of the function

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 1;
    if (x == 1) {
        int64 y = 5;
    }
    printf("%d\n", y);          // y is still visible here
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
5
```

A loop variable declared in the header of a `for` statement follows the same rule. Two `for` loops in the same function may both declare `int64 i`; the second declaration simply re-initializes the same variable.

WARNING

Declaring the same name twice in one function with the same type re-initializes the variable instead of creating a second one. Declaring it again with a different type is an error at code generation time. Write distinct names for distinct variables, and rely on block structure only for readability.

Copyright © 2026 Mallionic Inc. All rights reserved.

9.1.4 Assignment

Assignment is a statement, not an expression: `x = e;` stores the value of `e` in `x`. It cannot appear inside another expression, and `a = b = c` is not valid. The compound forms `+=`, `-=`, `*=`, `/=`, `%=`, `<<=`, `>>=`, `&=`, `|=` and `^=` are rewritten as described in Section 7.8.2. The increment and decrement statements `x++`, `++x`, `x--` and `--x` add or subtract 1.

Assignment targets can be:

Table 9-1. Assignment targets

Target	Example
Local variable or parameter	`total = total + 1;`
Global variable declared `static`	`counter = counter + 1;`
Array element	`v[i] = 0;` `m[i][j] = 1;`
Memory through a pointer	`*p = 42;` `*(int64*)addr = 0;`
Structure field	`p.age = 14;` `ptr->age = 15;` `a->b->c = 0;`
Crack invocation	`f.config = 1, "text";` (Chapter 16)

9.2 WHERE LOCALS LIVE

The x86-64 code generator keeps scalar locals and parameters in registers whenever possible, and uses the stack frame for the rest. The choice depends on whether the function calls other functions:

Table 9-2. Storage of locals on the 64-bit target

Function kind	Storage
Leaf function (calls nothing)	The caller-saved registers `r8`–`r11` first; no registers need to be saved, and the function often needs no frame at all.
Function that makes calls	The callee-saved registers `r12`–`r15` first (they survive calls), then 8-byte slots in the stack frame addressed through `rbp`. The prologue saves exactly the callee-saved registers the function uses.
Variables used in `asm` blocks	A callee-saved register or a stack slot, so that the operand substituted into the assembly is stable (Section 20.1.1).
Temporaries	`rax`, `rcx` and `rdx` while an expression is computed; named temporaries (`__tmp`*N*) get registers or slots like ordinary locals.

The following function has more locals than there are callee-saved registers. The first four live in `r12`–`r15`, and the others in stack slots below the saved registers:

Example 9-3. Registers first, then stack slots

ASH

```
@Entry
pub func main(): int64 {
    int64 a = 1; int64 b = 2; int64 c = 3; int64 d = 4;
    int64 e = 5; int64 f = 6; int64 g = 7;
    printf("%d\n", a + b + c + d + e + f + g);
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
28
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <main>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 70 00 00 00    sub    rsp,0x70
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 01 00 00 00          mov    eax,0x1
  20:   49 89 c4                mov    r12,rax
  23:   b8 02 00 00 00          mov    eax,0x2
  28:   49 89 c5                mov    r13,rax
  2b:   b8 03 00 00 00          mov    eax,0x3
  30:   49 89 c6                mov    r14,rax
  33:   b8 04 00 00 00          mov    eax,0x4
  38:   49 89 c7                mov    r15,rax
  3b:   b8 05 00 00 00          mov    eax,0x5
  40:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
  44:   b8 06 00 00 00          mov    eax,0x6
  49:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  4d:   b8 07 00 00 00          mov    eax,0x7
  52:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  56:   b8 00 00 00 00          mov    eax,0x0
  5b:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  5f:   48 8d 05 ba 00 00 00    lea    rax,[rip+0xba]        # 120 <__str0>
  66:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  6a:   4c 89 e0                mov    rax,r12
  6d:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  71:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
  75:   4c 89 e9                mov    rcx,r13
  78:   48 01 c8                add    rax,rcx
  7b:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  7f:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
  83:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
  87:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
  8b:   4c 89 f1                mov    rcx,r14
  8e:   48 01 c8                add    rax,rcx
  91:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
  95:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
  99:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
... (37 more lines not shown)
```

On the 32-bit and 16-bit targets every local has a slot in the stack frame, addressed as `[ebp-n]` or `[bp-n]` (Section 24.9).

9.3 VOLATILE VARIABLES

The keyword `volatile` before a declaration asks the compiler to keep the variable even if it appears unused. Without it, a local `string` variable whose value is never read is removed together with its literal. `volatile` is useful when a variable is only read by assembly code or by a debugger.

Copyright © 2026 Mallionic Inc. All rights reserved.

```ash
volatile string marker = "BUILD-2026-10";
volatile int64 debug_level = 3;
```

`volatile` also applies to assembly blocks (`volatile asm { ... }`, `asm volatile("...")`), where it marks the block as having side effects (Chapter 20).

9.4 GLOBAL VARIABLES

A variable declared outside every function and class is global. Its declaration must say who can see it:

```
(general | ungeneral) [static] type name [= constant-expression]
```

Table 9-3. Global variable qualifiers

Qualifier	Meaning
`general`	The variable is a public symbol with its own name: C code and other object files can refer to it (`extern unsigned long counter;` in C).
`ungeneral`	The variable is private to the program: its symbol is renamed `__g_<name>` and is local to the object file.
`static`	The variable can be modified at run time. **Without `static` a global is a constant**: assigning to it is a compile-time error.
`const`	Accepted for readability; a global is constant unless it is `static`.

The semicolon after a global declaration is optional and usually omitted. The initializer must be a constant expression: literals combined with the operators `+ - * / % << >> & | ^ ~` and parentheses, evaluated at compile time with arbitrary precision and then checked against the range of the type. A global without initializer is 0.

Example 9-4. Global variables

```ash
general static int64 counter = 3 * 4 + (1 << 4)     // 28
ungeneral static uint8 flags
general uint16 MAGIC = 0xBEEF                        // constant
ungeneral int32 LIMIT = -100                         // constant, private

func bump(): void {
    counter++;
    flags = flags | 2;
}

@Entry
pub func main(): int64 {
    bump();
    bump();
    printf("%d %d %x %d\n", counter, flags, MAGIC, LIMIT);
    return 0;
}
```

```
30 2 beef -100
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR

```
func void bump() {
    %__tmp0 addr[counter]
.type __tmp1 qword
.type __tmp2 qword
    %__tmp3 addr[counter]
.type __tmp4 qword
    %__tmp4 load[__tmp3]
    %__tmp2 mov[__tmp4]
    %__tmp1 mov[__tmp2]
    %__tmp1 add[1]
    store[__tmp0, __tmp1]
    %__tmp5 addr[__g_flags]
.type __tmp6 byte
.type __tmp7 byte
    %__tmp8 addr[__g_flags]
.type __tmp9 byte
    %__tmp9 load[__tmp8]
    %__tmp7 mov[__tmp9]
    %__tmp6 mov[__tmp7]
    %__tmp6 or[2]
    store[__tmp5, __tmp6]
    ret void
}

[entry]
func qword main() {
    call[bump]
    call[bump]
    %__tmp10 addr[__str0]
    %__tmp12 addr[counter]
.type __tmp13 qword
    %__tmp13 load[__tmp12]
    %__tmp11 mov[__tmp13]
    %__tmp15 addr[__g_flags]
.type __tmp16 byte
    %__tmp16 load[__tmp15]
    %__tmp14 mov[__tmp16]
    %__tmp18 addr[MAGIC]
.type __tmp19 word
    %__tmp19 load[__tmp18]
    %__tmp17 mov[__tmp19]
    %__tmp21 addr[__g_LIMIT]
.type __tmp22 dword
    %__tmp22 load[__tmp21]
    %__tmp20 mov[__tmp22]
    call[printf, __tmp10, __tmp11, __tmp14, __tmp17, __tmp20]
    ret qword 0
}

.gvar counter 8 28
.gvar __g_flags 1 0
.gvar MAGIC 2 48879
.gvar __g_LIMIT 4 -100
.string __str0 "%d %d %x %d\n"
```

The ASHIR shows each global as a `.gvar` entry with its symbol, size in bytes and initial value; the private ones carry the `__g_` prefix.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 9-5. Assigning to a constant global

ASH

```
general uint32 LIMIT = 1000

@Entry
pub func main(): int64 {
    LIMIT = 5;
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: the global 'LIMIT' is not static (it is a constant): declare it 'static uint32 LIMIT'
↪  to be able to change it
1 error generated.
```

Example 9-6. A global must say who can see it

ASH

```
uint64 total = 0

@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS

```
<input>:1:1: error: variable 'total' declared outside a function: at top level only general,
↪  ungeneral, extern, import, define, func, class and annotations are allowed
hint: a variable declared outside a function must say who can see it. Write 'general uint64
↪ total = ...;' (public: visible to C and to other files under the name 'total') or 'ungener
↪ al uint64 total = ...;' (private to this script). Add 'static' when the value must change
↪ at run time ('general static uint64 total = ...;'); without 'static' the variable is a con
↪ stant.
    1 | uint64
      | ^^^^^^
1 error generated.
```

9.4.1 How globals are accessed

The front end rewrites every use of a global `g` of type `T` into a typed memory access through its address: reading `g` becomes `*(T*)&g` and `g = v` becomes `*(T*)&g = (T)v`. `g++` and `g--` become `g = g + 1` and `g = g - 1`. In machine code a global is therefore loaded and stored with RIP-relative addressing (`[rip+disp32]`) on x86-64, or with an absolute address on 32-bit and flat targets.

9.4.2 The address of a global

`&g` is the address of the global's storage. It can be passed to C, stored in a pointer and used to read and write the global indirectly:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 9-7. Taking the address of a global

ASH

```
general static uint64 total = 100

@Entry
pub func main(): int64 {
    uint64* p = &total;
    *p = *p + 1;
    printf("total=%llu\n", total);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
total=101
```

9.4.3 Global arrays

A global may be an array with an initializer list, or a sized array that starts zero-filled:

```
(general | ungeneral) [static] type[] name = { constant, ... }
(general | ungeneral) [static] type[N] name
(general | ungeneral) char[] name = """text"""
```

Global arrays have the same layout as local arrays (Section 13.2): the symbol points at the first element and a header holding the length precedes it, so `#name`, loops and bounds checks work exactly as for locals. Element values of a global array can be modified even if the array itself is not `static`.

Example 9-8. Global arrays

ASH

```
general uint32[] table = {10, 20, 30, 40}
ungeneral static uint8[8] buffer
general char[] poem = """line one
line two"""

@Entry
pub func main(): int64 {
    int64 total = 0;
    for (int64 i = 0; i < #table; i++) {
        total = total + table[i];
    }
    buffer[3] = 7;
    printf("total=%d len=%d buffer[3]=%d\n%s\n", total, #table, buffer[3], poem);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
total=100 len=4 buffer[3]=7
line one
line two
```

Copyright © 2026 Mallionic Inc. All rights reserved.

9.4.4 Runnable arrays

`runnable` declares a global array whose contents are *machine code*: the array lives in memory that is both writable and executable, its elements can be modified at run time like any other array, and the array can be called like a function. Arguments are passed in the registers of the calling convention (`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` on x86-64) and the result is taken from `rax`.

Example 9-9. A runnable array patched at run time

ASH

```
runnable byte[] give = {0xB8, 0, 0, 0, 0, 0xC3};      // mov eax, imm32 ; ret
runnable byte[] add2 = {0x48, 0x8D, 0x04, 0x37, 0xC3}; // lea rax, [rdi+rsi] ; ret

@Entry
pub func main(): int64 {
    give[1] = 42;                 // patch the immediate operand
    int64 a = give();
    int64 b = add2(5, 6);
    int64 n = 1234;
    give[1] = n & 255;            // patch it again from a variable
    give[2] = n >> 8;
    int64 c = give();
    printf("give=%d add2=%d patched=%d\n", a, b, c);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
give=42 add2=11 patched=1234
```

Runnable arrays are the simplest way to experiment with self-modifying code, just-in-time code generation and instruction encodings. Section 20.4 shows more uses.

9.4.5 Placing a global in a section

`@section(".name", "flags")` before a global puts its data in its own ELF section, which a linker script can then place at a fixed address (a multiboot header, a table in ROM, a boot signature). See Section 17.13.

9.5 SHARING VARIABLES WITH C

A `general` global is an ordinary data symbol. C code declares it with `extern` and the matching C type, and reads and writes it directly:

SHARED.ASH

```
general static uint64 counter = 100
general uint32 VERSION = 3

pub func bump(): void {
    counter = counter + 1;
}
```

MAIN.C

```
#include <stdio.h>
extern unsigned long counter;
extern unsigned int VERSION;
void bump(void);
```

Copyright © 2026 Mallionic Inc. All rights reserved.

MAIN.C (CONTINUED)

```c
int main(void) {
    bump();
    printf("counter=%lu VERSION=%u\n", counter, VERSION);
    counter = 500;
    bump();
    printf("after C wrote it: %lu\n", counter);
    return 0;
}
```

SHELL SESSION

```
$ ashc shared.ash --elf -s
$ gcc -no-pie -Wl,--no-warn-rwx-segments main.c out/shared.o -o demo
$ ./demo
counter=101 VERSION=3
after C wrote it: 501
```

NOTE

Because code and writable data share the `.ashtext` section, the GNU linker warns that the program has a segment with read, write and execute permissions. The warning is harmless for ASH programs and can be silenced with `-Wl,--no-warn-rwx-segments`.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 10

EXPRESSIONS AND OPERATORS

An expression computes a value from operands and operators. This chapter defines the operators of ASH, their precedence and associativity, the exact meaning of each one, and the ASHIR operations and machine instructions they become.

10.1 KINDS OF EXPRESSIONS

Table 10-1. Primary expressions

Form	Example
Literal	`42`, `0xFF`, `'A'`, `"text"`, `L"wide"`
Variable or parameter	`total`, `argc`
Global variable	`counter` (rewritten to a typed memory access)
Parenthesized expression	`(a + b)`
Function call	`square(3)`, `printf("%d\n", x)`, `Math.twice(4)`
Call through a pointer chain	`table->handler(1, 2)`, `gST->ConOut->OutputString(gST->ConOut, msg)`
Index	`v[i]`, `m[i][j]`, `s[0]`, `p[k]`
Member	`p.age`, `ptr->age`, `a->b->c`
Length	`#v`
Array literal	`{1, 2, 3}` (in initializers)
Cast	`(int32)x`, `(int64*)addr`

ASH distinguishes *value expressions*, which compute an integer or an address, from *conditions*, which appear only in the headers of `if`, `elif`, `while` and `for` (Section 11.3). Comparisons and the logical operators `&&`, `||` and `!` belong to conditions: in ASH 1.0 they cannot be used as values.

10.2 PRECEDENCE AND ASSOCIATIVITY

The following table lists the operators of value expressions from the highest precedence (binding most tightly) to the lowest. Operators in the same row have the same precedence.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 10-2. Operator precedence of value expressions

Level	Operators	Associativity	Description
1	`() [] . ->` postfix `++ --`	left to right	Call, index, member, post-increment
2	prefix `++ --`, unary `+ - ~ & * #`, casts	right to left	Unary operators
3	`** ***`	**right to left**	Power, repeated squaring
4	`* / %`	left to right	Multiplicative
5	`+ -`	left to right	Additive
6	`<< >>`	left to right	Shifts
7	`< <= > >= == !=`	left to right	Comparisons (conditions only)
8	`&`	left to right	Bitwise AND
9	`^`	left to right	Bitwise exclusive OR
10	`|`	left to right	Bitwise inclusive OR

In conditions, `!` applies to a parenthesized condition, `&&` binds more tightly than `||`, and both bind more loosely than any operator of the table.

The relative order of shifts, comparisons and bitwise operators is the same as in C. In particular `a & b == c` means `a & (b == c)` and `1 | 2 ^ 3 & 7` means `1 | (2 ^ (3 & 7))`. Use parentheses whenever you combine bitwise operators with comparisons.

Example 10-1. Precedence in practice

ASH

```
@Entry
pub func main(): int64 {
    int64 a = 2 + 3 * 4 ** 2;      // 2 + (3 * (4 ** 2)) = 50
    int64 b = 1 << 2 + 1;          // 1 << (2 + 1) = 8
    int64 c = 10 - 2 - 3;          // (10 - 2) - 3 = 5
    int64 d = 100 / 10 / 5;        // (100 / 10) / 5 = 2
    int64 e = 2 ** 3 ** 2;         // 2 ** (3 ** 2) = 512
    int64 f = 1 | 2 ^ 3 & 7;       // 1 | (2 ^ (3 & 7)) = 1
    printf("%d %d %d %d %d %d\n", a, b, c, d, e, f);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
50 8 5 2 512 1
```

10.3 EVALUATION AND TEMPORARIES

The front end evaluates an expression by breaking it into operations on *temporaries*. Every operator becomes one ASHIR operation of the form `%dest op[operand]`, which reads `dest`, combines it with `operand` and writes the result back into `dest`. A sub-expression that is not a simple variable or literal gets its own temporary `__tmp`*N*. Operands are evaluated from left to right.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 10-2. An expression broken into ASHIR operations

ASH

```
@Entry
pub func main(): int64 {
    int64 a = 5;
    int64 b = 7;
    int64 r = (a + b) * (a - b) + 3;
    return r;
}
```

ASHIR

```
[entry]
func qword main() {
.type a qword
.type b qword
.type r qword
    %a mov[5]
    %b mov[7]
.type __tmp0 qword
.type __tmp1 qword
    %__tmp1 mov[a]
    %__tmp1 add[b]
    %__tmp0 mov[__tmp1]
.type __tmp2 qword
    %__tmp2 mov[a]
    %__tmp2 sub[b]
    %__tmp0 imul[__tmp2]
    %r mov[__tmp0]
    %r add[3]
    ret qword r
}
```

10.4 ARITHMETIC OPERATORS

Table 10-3. Arithmetic operators

Operator	Meaning	ASHIR	x86-64
`a + b`	Sum, modulo 2^N	`add`	`add`
`a - b`	Difference, modulo 2^N	`sub`	`sub`
`a * b`	Product, low N bits	`imul` / `mul`	`imul`
`a / b`	Quotient, truncated toward zero	`div`	`cqo` + `idiv` (signed), `xor edx,edx` + `div` (unsigned)
`a % b`	Remainder, sign of the dividend	`mod`	`idiv` / `div`, result in `rdx`
`-a`	Negation	`0 - a`	`sub` from zero
`+a`	No effect		

Division and remainder obey the identity `(a / b) * b + a % b == a`, as in C. Division by a constant zero is a compile-time error; division by a variable that is zero at run time raises the processor's divide error, which Linux reports as `SIGFPE`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 10-3. Division and remainder

ASH

```
@Entry
pub func main(): int64 {
    printf("%d %d\n", 17 / 5, 17 % 5);       // 3 2
    printf("%d %d\n", -17 / 5, -17 % 5);     // -3 -2
    printf("%d %d\n", 17 / -5, 17 % -5);     // -3 2
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3 2
-3 -2
-3 2
```

Example 10-4. Machine code for division

ASH

```
func divide(int64 a, int64 b): int64 {
    return a / b;
}

@Entry
pub func main(): int64 {
    return divide(100, 7);
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000018 <main>:
  18:   55                      push   rbp
  19:   48 89 e5                mov    rbp,rsp
  1c:   48 81 ec 20 00 00 00    sub    rsp,0x20
  23:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  27:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  2b:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  2f:   b8 64 00 00 00          mov    eax,0x64
  34:   49 89 c4                mov    r12,rax
  37:   b8 07 00 00 00          mov    eax,0x7
  3c:   49 89 c5                mov    r13,rax
  3f:   b8 00 00 00 00          mov    eax,0x0
  44:   49 89 c6                mov    r14,rax
  47:   4c 89 e7                mov    rdi,r12
  4a:   4c 89 ee                mov    rsi,r13
  4d:   e8 ae ff ff ff          call   0 <divide>
  52:   49 89 c6                mov    r14,rax
  55:   4c 89 f0                mov    rax,r14
  58:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5c:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  60:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  64:   c9                      leave
  65:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

10.5 POWER AND REPEATED SQUARING

ASH has two exponentiation operators. Both work on integers and are right-associative.

- `a ** b` is *a* raised to the power *b* (b >= 0).
- `a *** n` squares *a* repeatedly *n* times, which equals *a* raised to the power 2^*n*: `2 *** 3` is `((2^2)^2)^2 = 256`.

When both operands are constants the result is computed at compile time with arbitrary precision. A result that does not fit in 64 bits, a negative constant exponent, or an exponent larger than 4096 is a compile-time error. When an operand is not constant, the compiler adds a helper function (`__ash_pow`, computing by repeated squaring, or `__ash_sqpow`) and calls it.

Example 10-5. Constant and run-time powers

ASH

```
@Entry
pub func main(): int64 {
    int64 base = 3;
    int64 e = 4;
    int64 n = 2;
    printf("%d %d %d %d\n", 2 ** 10, base ** e, 2 *** 3, base *** n);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1024 81 256 81
```

ASHIR

```
[entry]
func qword main() {
.type base qword
.type e qword
.type n qword
    %base mov[3]
    %e mov[4]
    %n mov[2]
    %__tmp0 addr[__str0]
    %__tmp1 mov[1024]
    %__tmp3 mov[base]
    %__tmp4 mov[e]
    %__tmp2 call[__ash_pow, __tmp3, __tmp4]
    %__tmp5 mov[256]
    %__tmp7 mov[base]
    %__tmp8 mov[n]
    %__tmp6 call[__ash_sqpow, __tmp7, __tmp8]
    call[printf, __tmp0, __tmp1, __tmp2, __tmp5, __tmp6]
    ret qword 0
}
```

The ASHIR shows that `2 ** 10` and `2 *** 3` became the constants 1024 and 256, while `base ** e` and `base *** n` became calls to the helpers.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 10-6. A constant power that does not fit in 64 bits

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 2 ** 70;
    return x;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
<input>:3:22: error: the result of '**' does not fit in 64 bits: 118059162071741130342 4
    3 | ;
      |                      ^
1 error generated.
```

10.6 SHIFT OPERATORS

`a << n` shifts *a* left by *n* bits, filling with zeros. `a >> n` shifts right: with a **constant** count the compiler emits `sar` for signed operands (the sign bit is copied) and `shr` for unsigned operands (zeros are shifted in). Only the low 6 bits of the count are used on x86-64 (5 bits for 32-bit values), as with the hardware instructions.

With a **variable** count the shift is performed by the runtime helpers `__ash_shl` and `__ash_shr`.

Example 10-7. Shifts by constants and by variables

ASH

```
@Entry
pub func main(): int64 {
    int64 n = 5;
    int64 m = -64;
    uint64 u = 0xF000000000000000;
    printf("%d %d %llx\n", 1 << 5, m >> 5, u >> 4);   // constant counts
    printf("%d %d\n", 1 << n, m >> n);                // variable counts
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
32 -2 f00000000000000
32 -2
```

ASHIR

WARNING

In `ashc` 1.0 the helper `__ash_shr` always performs an *arithmetic* shift, even when the operand is unsigned. A variable right shift of an unsigned value whose top bit is set therefore copies 1-bits into the result. Mask the result (`(u >> n) & mask`) or shift by a constant when the operand may have its top bit set.

Copyright © 2026 Mallionic Inc. All rights reserved.

10.7 BITWISE OPERATORS

Table 10-4. Bitwise operators

r	Meaning	Translation	
	1 where both bits are 1	ASHIR `and`, x86 `and`	
	b`	1 where at least one bit is 1	ASHIR `or`, x86 `or`
	1 where the bits differ	ASHIR `xor`, x86 `xor`	
	Inverts every bit	Rewritten as `a ^ -1`	

Example 10-8. Flags with bitwise operators

ASH

```
@Entry
pub func main(): int64 {
    uint32 READ = 1;
    uint32 WRITE = 2;
    uint32 EXEC = 4;
    uint32 perms = READ | WRITE;        // 3
    uint32 can_write = perms & WRITE;   // 2
    perms |= EXEC;                      // 7
    perms &= ~WRITE;                    // 5: remove WRITE
    perms ^= READ;                      // 4: toggle READ
    printf("can_write=%u perms=%u\n", can_write, perms);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
can_write=2 perms=4
```

10.8 UNARY OPERATORS

Table 10-5. Unary operators

Operator	Meaning
`-a`	Negation. Represented as `0 - a`.
`+a`	No operation.
`~a`	Bitwise complement. Represented as `a ^ -1`.
`&x`	Address of `x`: a variable, an array (the address of its first element), a global, a structure, or a function (Section 8.7.1).
`*p`	Dereference: the value of the type pointed to by `p` at the address `p`.
`#a`	Number of elements of the array `a` (Section 13.4).
`++x`, `--x`	Increment or decrement `x` and yield the new value.
`x++`, `x--`	Increment or decrement `x` and yield the old value.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 10-9. Increment and decrement in expressions

ASH

```
@Entry
pub func main(): int64 {
    int64 i = 5;
    int64 a = i++;       // a = 5, i = 6
    int64 b = ++i;       // i = 7, b = 7
    int64 c = i--;       // c = 7, i = 6
    printf("%d %d %d %d\n", a, b, c, i);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
5 7 7 6
```

10.9 COMPARISON OPERATORS

The comparison operators `==`, `!=`, `<`, `<=`, `>` and `>=` compare two integers or two addresses. They appear in conditions, where they become a single conditional branch: `brf[a, lt, b, L]` in ASHIR ("branch if false"), and a `cmp` instruction followed by the opposite conditional jump in machine code. Whether `<` uses a signed (`jl`, `jge`) or unsigned (`jb`, `jae`) jump depends on the types of the operands.

Example 10-10. A comparison is a compare-and-branch

ASH

```
@Entry
pub func main(): int64 {
    int64 a = 3;
    int64 b = 5;
    if (a < b) {
        return 1;
    }
    return 0;
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <main>:
   0:   b8 03 00 00 00          mov    eax,0x3
   5:   49 89 c2                mov    r10,rax
   8:   b8 05 00 00 00          mov    eax,0x5
   d:   49 89 c3                mov    r11,rax
  10:   4c 89 d0                mov    rax,r10
  13:   4c 89 d9                mov    rcx,r11
  16:   48 39 c8                cmp    rax,rcx
  19:   0f 8d 06 00 00 00       jge    25 <main+0x25>
  1f:   b8 01 00 00 00          mov    eax,0x1
  24:   c3                      ret
  25:   48 31 c0                xor    rax,rax
  28:   c3                      ret
```

To obtain a comparison result as a number, write it as a condition:

ASH

```
int64 a = 3;
int64 b = 5;
int64 less = 0;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASH (CONTINUED)

```
if (a < b) { less = 1; }
```

10.10 FUNCTION CALLS

A call `f(a1, a2, ...)` evaluates its arguments from left to right, passes them according to the calling convention of the target and yields the return value of the function. A call can appear inside any expression. Calls to functions that are neither defined in ASH nor declared `extern` are treated as calls to external C functions with a variable argument list (Section 12.8.1).

10.11 CONSTANT FOLDING

The compiler computes at compile time the operators `**` and `***` on constants and every expression used as a global initializer, as a `%define` value or in a `%if` condition. Other constant sub-expressions inside functions are emitted as operations, so `int64 x = 2 + 3;` performs one addition at run time. This keeps ASHIR close to the source; write the folded value yourself where it matters, or use a `define` constant.

10.12 SUMMARY OF OPERATORS AND THEIR TRANSLATION

Table 10-6. From operator to machine code (x86-64)

Operator	ASHIR	Typical instructions
`+ -`	`add sub`	`add r, r/imm32`, `sub r, r/imm32`
`*`	`imul` / `mul`	`imul r, r/m`, `imul r, r, imm32`
`/ %`	`div mod`	`cqo; idiv r` or `xor edx, edx; div r`
`& | ^`	`and or xor`	`and`, `or`, `xor`
`<< >>` constant	`shl shr`	`shl r, imm8`, `sar r, imm8` / `shr r, imm8`
`<< >>` variable	`call[__ash_shl ...]`	call to the runtime helper
`** ***` variable	`call[__ash_pow ...]`	call to the helper
comparison in a condition	`brf` / `brt`	`cmp` + `jcc`
`&x` (local)	`addr`	`lea r, [rbp-n]` or the address held by the array variable
`&f` (function)	`faddr`	`lea r, [rip+f]`
`*p` read / write	`load` / `store`	`mov r, [r]`, `mov [r], r` (width of the pointed type)

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 11

STATEMENTS AND CONTROL FLOW

This chapter describes the statements that direct the flow of execution: blocks, conditions, `if` / `elif` / `else`, the four loops (`while`, `for`, `repeat`, `loop`), `switch`, `break`, `continue`, `return` and `terminate`. For each statement it shows the ASHIR lowering, because every structured statement is translated into labels and conditional branches.

11.1 KINDS OF STATEMENTS

Table 11-1. Statements of ASH

Statement	Described in
Declaration `T x = e;`	Chapter 9
Assignment, compound assignment, `x++`	Section 9.1.4
Expression statement (a call) `f(x);`	Section 10.10
`if` / `elif` / `else`	Section 11.4
`while`, `for`, `repeat`, `loop`	Section 11.5
`switch` / `case` / `default`	Section 11.7
`break`, `continue`	Section 11.6
`return`, `terminate`, `unreturnable`	Section 11.8, Chapter 16
`asm { }`, `asm("...")`, `emit(...)`	Chapter 20
`matrix` declaration	Section 13.7

Only calls can be used as expression statements: `f(x);` is valid, `a + b;` is not.

11.2 BLOCKS

A block is a sequence of statements enclosed in braces. The bodies of `if`, `elif`, `else`, loops and `switch` cases are blocks; **the braces are required**, even for a single statement. Empty statements (a lone `;`) are allowed and ignored.

Copyright © 2026 Mallionic Inc. All rights reserved.

11.3 CONDITIONS

The headers of `if`, `elif`, `while` and `for` contain a *condition*. A condition is built from:

- comparisons: `a == b`, `a != b`, `a < b`, `a <= b`, `a > b`, `a >= b`, where `a` and `b` are value expressions;
- a value expression alone, which is true when it is not zero: `if (flags & 4)`, `if (count)`, `if (is_ready())`;
- `!(` *condition* `)`, the negation of a parenthesized condition;
- *c1* `&&` *c2*, true when both are true; *c2* is evaluated only if *c1* is true;
- *c1* `||` *c2*, true when at least one is true; *c2* is evaluated only if *c1* is false;
- parentheses for grouping.

`&&` binds more tightly than `||`. Evaluation is *short-circuit*, as in C: the right operand is not evaluated when the left one already decides the result, so `if (p != 0 && p[0] == 'x')` is safe.

11.3.1 How conditions are compiled

A condition never produces a value. It is compiled directly into conditional branches: `brf[a, op, b, L]` jumps to label `L` when the comparison `a op b` is **false**, and `brt[a, op, b, L]` jumps when it is **true**. The comparison codes are `eq`, `ne`, `lt`, `le`, `gt` and `ge`. In machine code each branch becomes a `cmp` followed by a conditional jump chosen from the comparison and the signedness of the operands.

Example 11-1. Short-circuit lowering of a compound condition

ASH

```
func check(int64 a, int64 b): int64 {
    if ((a > 0 && b > 0) || !(a == b)) {
        return 1;
    }
    return 0;
}

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", check(1, 2), check(-1, -1), check(-1, 5));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0 1
```

ASHIR

```
func qword check(qword a, qword b) {
    %_ brf[a, gt, 0, __L3]
    %_ brt[b, gt, 0, __L2]
    %_ label[__L3]
    %_ brt[a, eq, b, __L0]
    %_ label[__L2]
    ret qword 1
    %_ label[__L0]
    ret qword 0
}
```

Read the ASHIR from top to bottom: if `a > 0` is false, the code jumps to `__L3`, which tests the right side of `||`; if `b > 0` is true, the whole condition is true and the code jumps to `__L2`, the body. The negation `!(a ==`

Copyright © 2026 Mallionic Inc. All rights reserved.

`b)` costs nothing: it simply turns into "branch to the false label if `a == b`".

> **WARNING**
> `if (x = 3)` is not a comparison: assignment is not an expression in ASH, and the compiler rejects it. Write `==`.

11.4 THE IF STATEMENT

```
if ( condition ) { ... }
[ elif ( condition ) { ... } ]...
[ else if ( condition ) { ... } ]...
[ else { ... } ]
```

`elif` and `else if` are synonyms; any number of them may follow an `if`, followed by an optional `else`. The first branch whose condition is true is executed; if none is, the `else` block (if any) is executed.

Example 11-2. if, elif and else

ASH

```
func classify(int64 x): int64 {
    if (x < 10) {
        return 1;
    } elif (x < 20) {
        return 2;
    } else if (x < 30) {
        return 3;
    } else {
        return 4;
    }
}

@Entry
pub func main(): int64 {
    printf("%d %d %d %d\n", classify(5), classify(15), classify(25), classify(99));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 2 3 4
```

11.5 LOOPS

11.5.1 while

`while ( condition ) { ... }` evaluates the condition before each iteration and executes the body while it is true. The lowering is: a label at the top, a `brf` to the exit label, the body, a jump back to the top.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 11-3. A while loop

ASH

```
@Entry
pub func main(): int64 {
    int64 n = 27;
    int64 steps = 0;
    while (n != 1) {                // the Collatz sequence of 27
        if ((n & 1) == 0) {
            n = n / 2;
        } else {
            n = 3 * n + 1;
        }
        steps++;
    }
    printf("steps=%d\n", steps);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
steps=111
```

ASHIR

```
[entry]
func qword main() {
.type n qword
.type steps qword
    %n mov[27]
    %steps mov[0]
    %_ label[__L0]
    %_ brf[n, ne, 1, __L1]
    %__tmp0 mov[n]
    %__tmp0 and[1]
    %_ brf[__tmp0, eq, 0, __L2]
    %n mov[n]
    %n div[2]
    %_ jmp[__L3]
    %_ label[__L2]
.type __tmp1 qword
    %__tmp1 mov[3]
    %__tmp1 imul[n]
    %n mov[__tmp1]
    %n add[1]
    %_ label[__L3]
    inc steps
    %_ jmp[__L0]
    %_ label[__L1]
    %__tmp2 addr[__str0]
    %__tmp3 mov[steps]
    call[printf, __tmp2, __tmp3]
    ret qword 0
}
```

NOTE

ASH has no `do ... while` loop. Write `loop { ...; if (!(condition)) { break; } }` instead.

Copyright © 2026 Mallionic Inc. All rights reserved.

11.5.2 for

```
for ( [init] ; [condition] ; [update] ) { ... }
```

init is a declaration, an assignment or an expression; *condition* is a condition; *update* is an assignment, an increment or a call. Any of the three parts may be omitted: an omitted condition is always true, so `for (;;)` loops until a `break`. A variable declared in *init* remains visible after the loop (Section 9.1.3). `continue` inside a `for` jumps to the *update* part.

Example 11-4. Forms of the for loop

ASH

```
@Entry
pub func main(): int64 {
    int64 sum = 0;
    for (int64 i = 1; i <= 10; i++) {
        sum = sum + i;
    }
    int64 j = 0;
    for (; j < 3;) {
        j++;
    }
    int64 n = 0;
    for (;;) {
        n++;
        if (n == 4) { break; }
    }
    for (int64 k = 10; k > 0; k = k - 3) {
        n = n + 1;                  // k = 10, 7, 4, 1
    }
    printf("sum=%d j=%d n=%d\n", sum, j, n);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
sum=55 j=3 n=8
```

11.5.3 repeat

`repeat ( count ) { ... }` executes the body *count* times. *count* is any value expression; it is evaluated once, before the first iteration. The loop counter is hidden (`__rp`*N*), so the body cannot change the number of iterations. `break` and `continue` work as in other loops.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 11-5. repeat and nested repeat

ASH

```
@Entry
pub func main(): int64 {
    int64 total = 0;
    repeat (5) {
        total += 2;
    }
    repeat (3) {
        repeat (2) {
            total += 1;
        }
    }
    printf("total=%d\n", total);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
total=16
```

ASHIR

```
[entry]
func qword main() {
.type total qword
.type __rp0 int64
.type __rp2 int64
.type __rp1 int64
    %total mov[0]
    %__rp0 mov[0]
    %_ label[__L0]
    %_ brf[__rp0, lt, 5, __L2]
    %total mov[total]
    %total add[2]
    %_ label[__L1]
    %__rp0 mov[__rp0]
    %__rp0 add[1]
    %_ jmp[__L0]
    %_ label[__L2]
    %__rp2 mov[0]
    %_ label[__L3]
    %_ brf[__rp2, lt, 3, __L5]
    %__rp1 mov[0]
    %_ label[__L6]
    %_ brf[__rp1, lt, 2, __L8]
    %total mov[total]
    %total add[1]
    %_ label[__L7]
    %__rp1 mov[__rp1]
    %__rp1 add[1]
    %_ jmp[__L6]
    %_ label[__L8]
    %_ label[__L4]
    %__rp2 mov[__rp2]
    %__rp2 add[1]
    %_ jmp[__L3]
    %_ label[__L5]
    %__tmp0 addr[__str0]
    %__tmp1 mov[total]
    call[printf, __tmp0, __tmp1]
    ret qword 0
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
}
```

11.5.4 loop

`loop { ... }` repeats its body forever. It is left with `break`, `return` or `terminate`. It is equivalent to `while (1 == 1) { ... }`, which is exactly how it is lowered.

Example 11-6. loop with break and continue

ASH

```
@Entry
pub func main(): int64 {
    int64 total = 0;
    int64 n = 0;
    loop {
        n++;
        if (n == 3) { continue; }      // skip 3
        if (n > 6) { break; }
        total += 10;                   // n = 1, 2, 4, 5, 6
    }
    printf("total=%d n=%d\n", total, n);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
total=50 n=7
```

11.6 BREAK AND CONTINUE

`break` leaves the innermost enclosing loop or `switch`. `continue` skips the rest of the body of the innermost enclosing loop and starts the next iteration (in a `for`, the update part runs first). Both are errors outside a loop.

> **WARNING**
> In `ashc` 1.0 a `continue` written inside a `switch` that is itself inside a loop does not reach the loop: because `switch` is lowered to an internal loop (Section 11.7.1), `continue` restarts the `switch` and the program loops forever. Inside a `switch`, set a flag and test it after the `switch`, or restructure the code with `if` / `elif`.

11.7 THE SWITCH STATEMENT

```
switch ( expression ) {
    case value [: case value]... : statements
    ...
    [ default : statements ]
}
```

The *expression* is evaluated once. Execution starts at the first `case` whose value is equal to it, or at `default` if no case matches. As in C, execution **falls through** into the next case unless a `break` (or `return`, or `terminate`) ends it. Case values are value expressions, usually constants: integer literals, character literals,

Copyright © 2026 Mallionic Inc. All rights reserved.

`define` constants or constant expressions such as `1 + 7`. Several `case` labels may share the same statements.

Example 11-7. switch with fall-through

ASH

```
func name_of(int64 n): int64 {
    switch (n) {
        case 1:
            return 10;
        case 2:
        case 3:
            return 20;        // shared by 2 and 3
        default:
            return 99;
    }
    return 0;
}

@Entry
pub func main(): int64 {
    int64 total = 0;
    for (int64 i = 0; i < 5; i++) {
        switch (i) {
            case 0:
                total += 1;   // falls through into case 1
            case 1:
                total += 10;
                break;
            case 4:
                total += 1000;
                break;
            default:
                total += 100;
        }
    }
    char c = 'b';
    switch (c) {
        case 'a': printf("a\n"); break;
        case 'b': printf("b\n");
        case 'c': printf("c (fell through)\n"); break;
    }
    printf("total=%d name_of(3)=%d\n", total, name_of(3));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
b
c (fell through)
total=1221 name_of(3)=20
```

11.7.1 How switch is compiled

The parser lowers `switch` to statements that already exist. The subject is copied to a hidden variable `__sw`*N*, and a hidden *match flag* `__swm`*N* records that a case has matched, which is what makes fall-through work. The whole construct is wrapped in a loop that runs once, so that `break` leaves it:

Copyright © 2026 Mallionic Inc. All rights reserved.

```
int64 __swN = subject;  int64 __swmN = 0;
while (1 == 1) {
    if (__swmN != 0 || __swN == A || __swN == B) { __swmN = 1; ...statements of A, B... }
    if (__swmN != 0 || __swN == C)               { __swmN = 1; ...statements of C... }
    if (__swmN != 0 || !(__swN == A || __swN == B || __swN == C)) { __swmN = 1; ...default..
↪ . }
    break;
}
```

The cost of a `switch` is therefore one comparison per case label, in order: there is no jump table. Put the most frequent cases first when it matters.

11.8 LEAVING A FUNCTION

11.8.1 return

`return expression;` ends the function and gives its value to the caller. `return;` ends a `void` function. In a non-`void` function without strict mode, `return;` returns the pending value set by `unreturnable` or 0 (Chapter 16); strict mode requires a value (Chapter 21). Reaching the closing brace of a `void` function, or of a function that uses `unreturnable`, behaves like `return;`.

> **WARNING**
> In `ashc` 1.0, a function with a result type that does not use `unreturnable` and reaches its closing brace without a `return` statement crashes when it is called: no return instruction is generated. Always end such functions with `return value;`. The VS Code linter reports a missing `return` while you type.

11.8.2 terminate

`terminate expression;` ends the function immediately and returns the value, from any depth of nested loops, `switch` statements and blocks. `terminate;` without a value returns the pending `unreturnable` value of the function, or 0 when there is none; in a `void` function it simply leaves. `terminate` behaves like `return`; its name documents the intent "stop everything now", which is particularly useful in deeply nested searches.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 11-8. terminate leaves nested loops at once

ASH

```
func find(int64 target): int64 {
    for (int64 i = 0; i < 10; i++) {
        for (int64 j = 0; j < 10; j++) {
            if (i * 10 + j == target) {
                terminate i * 100 + j;     // leaves both loops and the function
            }
        }
    }
    return -1;
}

func first_big(int64 limit): int64 {
    repeat (5) {
        loop {
            terminate limit * 2;
        }
    }
    return 0;
}

@Entry
pub func main(): int64 {
    printf("find(37)=%d first_big(21)=%d\n", find(37), first_big(21));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
find(37)=307 first_big(21)=42
```

Like other keywords, `terminate` is contextual: `terminate = 5;`, `terminate++` or `terminate(...)` refer to a variable or function named `terminate`, not to the statement.

11.9 WHAT ASH DOES NOT HAVE

For readers coming from C, the following statements do not exist in ASH 1.0: `goto` and labels (use `terminate`, `break` or flags), `do ... while` (use `loop` with `break`), the conditional operator `?:` (use `if` / `else`), and the comma operator. Inside `asm` blocks, labels and jumps are of course available (Section 20.1.2).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 12

FUNCTIONS

Functions are the units of code in ASH. This chapter describes how functions are declared, how parameters and return values work, the calling conventions used on each target, recursion, variadic functions, function pointers and callbacks, external C functions and the program entry point.

12.1 DECLARING A FUNCTION

```
[pub | priv] func name ( [parameters] ) [ : return-type ] { body }
[pub | priv] func return-type name ( [parameters] ) { body }
```

The return type may be written after the parameter list, preceded by a colon (the preferred ASH style), or before the name, as in C. When no return type is written, the function returns `void`.

Table 12-1. Parts of a function declaration

Part	Meaning
`pub`	The function is a global symbol of the object file: C code and other objects can call it. Use `pub` for the entry point and for every function exported to C.
`priv`	The function cannot be called from other ASH files that import this one (Section 15.5).
(neither)	The function is visible to the ASH files of the program but is a local symbol of the object file.
parameters	A comma-separated list of `type name`. A parameter written `type name[]` is a pointer to the first element of an array. The list may end with `...` (Section 12.6).
return-type	Any scalar type, pointer type or `void`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 12-1. Function declaration styles

ASH

```
func int64 twice(int64 x) {          // C style: return type first
    return x * 2;
}

func hello() {                       // no return type: void
    printf("hello\n");
}

func fib(int64 n): int64 {           // ASH style
    if (n < 2) { return n; }
    return fib(n - 1) + fib(n - 2);
}

@Entry
pub func main(): int64 {
    hello();
    printf("%d %d\n", twice(21), fib(20));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hello
42 6765
```

A function can be called before it is defined in the file; the order of definitions does not matter.

12.2 PARAMETERS

Parameters behave like local variables initialized by the caller. Scalar parameters are passed **by value**: the function receives a copy. Arrays, strings and structures are passed as addresses, so the function can modify the caller's data through them.

12.2.1 Assigning to a parameter

The code generator treats incoming parameters as read-only values. When a function assigns to a parameter (`n = n - 1;`, `n++`), the front end renames the parameter to `__arg_n` and declares, at the start of the function, a local variable `n` initialized from it. The function then works on the copy, exactly as C semantics require.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 12-2. A modified parameter becomes a local copy

ASH

```
func countdown(int64 n): int64 {
    int64 steps = 0;
    while (n > 0) {
        n = n - 3;
        steps++;
    }
    return steps;
}

@Entry
pub func main(): int64 {
    int64 start = 10;
    printf("%d steps, start is still %d\n", countdown(start), start);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
4 steps, start is still 10
```

ASHIR

```
func qword countdown(qword __arg_n) {
.type n qword
.type steps qword
    %n mov[__arg_n]
    %steps mov[0]
    %_ label[__L0]
    %_ brf[n, gt, 0, __L1]
    %n mov[n]
    %n sub[3]
    inc steps
    %_ jmp[__L0]
    %_ label[__L1]
    ret qword steps
}
```

12.2.2 Passing by reference

To let a function modify a variable of the caller, pass its address and dereference it:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 12-3. Modifying the caller's variables through pointers

ASH

```
func swap(int64* a, int64* b): void {
    int64 t = a[0];          // same as *a
    a[0] = b[0];
    b[0] = t;
}

func add_to(int64* target, int64 amount): void {
    *target = amount;        // a store through a pointer parameter
}

@Entry
pub func main(): int64 {
    int64 x = 1;
    int64 y = 2;
    swap(&x, &y);
    int64 z = 0;
    add_to(&z, 7);
    printf("x=%d y=%d z=%d\n", x, y, z);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
x=2 y=1 z=7
```

WARNING

In `ashc` 1.0, a function that both *reads* `*p` and then *writes* `*p` through the same pointer **parameter** fails to compile with the message "Variable does not exist: p". Use indexing (`p[0]`), as in the example above, or copy the parameter into a local pointer first (`int64* q = p;`) and work through `q`.

12.3 RETURN VALUES

A function with a non-`void` return type gives its result with `return value;` or `terminate value;`. The value is converted to the return type: a function declared `: uint8` returns only the low 8 bits. Integer and pointer results are returned in `rax` (`eax` on 32-bit targets, `ax` on 16-bit targets). Narrower results occupy the low part of the register and are extended by the caller according to the declared type.

12.4 CALLING CONVENTIONS

ASH functions use the standard calling convention of the target, which makes them directly callable from C and lets ASH call C without glue code.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 12-2. Calling conventions by target

Target	Convention
x86-64 (Linux, `--elf`)	System V AMD64: the first six integer or pointer arguments in `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`; further arguments on the stack, pushed from right to left, each in an 8-byte slot; result in `rax`. `rbx`, `rbp`, `r12`–`r15` are preserved across calls. Before a call to a variadic C function, `al` holds the number of vector registers used (always 0 for ASH).
x86-64, `--win` external calls and `@msabi`	Microsoft x64: the first four arguments in `rcx`, `rdx`, `r8`, `r9`; further arguments on the stack above 32 bytes of *shadow space* reserved by the caller; result in `rax`.
i386 (`@bits(32)`)	`cdecl`: all arguments pushed on the stack from right to left, the caller removes them; result in `eax`.
8086 (`@bits(16)`)	Arguments pushed on the stack as 16-bit words; result in `ax`.

12.4.1 More than six arguments

On x86-64, the seventh and following arguments travel on the stack. The caller pushes them before the call and removes them afterwards; the callee reads them at `[rbp+16]`, `[rbp+24]` and so on.

Example 12-4. Arguments beyond the sixth are passed on the stack

ASH

```
func many(int64 a, int64 b, int64 c, int64 d, int64 e, int64 f, int64 g, int64 h): int64 {
    return g * 10 + h;
}

@Entry
pub func main(): int64 {
    printf("%d\n", many(1, 2, 3, 4, 5, 6, 7, 8));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
78
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <many>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
  1f:   48 89 4d d0             mov    QWORD PTR [rbp-0x30],rcx
  23:   48 8b 45 10             mov    rax,QWORD PTR [rbp+0x10]
  27:   49 89 c2                mov    r10,rax
  2a:   4c 89 d0                mov    rax,r10
  2d:   48 6b c0 0a             imul   rax,rax,0xa
  31:   49 89 c2                mov    r10,rax
  34:   4c 89 d0                mov    rax,r10
  37:   49 89 c3                mov    r11,rax
  3a:   4c 89 d8                mov    rax,r11
  3d:   48 8b 4d 18             mov    rcx,QWORD PTR [rbp+0x18]
  41:   48 01 c8                add    rax,rcx
  44:   49 89 c3                mov    r11,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  47:   4c 89 d8                mov    rax,r11
  4a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  4e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  52:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  56:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  5a:   48 89 ec                mov    rsp,rbp
  5d:   5d                      pop    rbp
  5e:   c3                      ret

000000000000005f <main>:
  5f:   55                      push   rbp
  60:   48 89 e5                mov    rbp,rsp
  63:   48 81 ec 50 00 00 00    sub    rsp,0x50
  6a:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  6e:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  72:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  76:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  7a:   b8 00 00 00 00          mov    eax,0x0
  7f:   49 89 c4                mov    r12,rax
  82:   4c 8d 25 a7 00 00 00    lea    r12,[rip+0xa7]        # 130 <__str0>
  89:   b8 01 00 00 00          mov    eax,0x1
  8e:   49 89 c5                mov    r13,rax
  91:   b8 02 00 00 00          mov    eax,0x2
  96:   49 89 c6                mov    r14,rax
  99:   b8 03 00 00 00          mov    eax,0x3
  9e:   49 89 c7                mov    r15,rax
  a1:   b8 04 00 00 00          mov    eax,0x4
  a6:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
  aa:   b8 05 00 00 00          mov    eax,0x5
  af:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  b3:   b8 06 00 00 00          mov    eax,0x6
  b8:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  bc:   b8 07 00 00 00          mov    eax,0x7
  c1:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  c5:   b8 08 00 00 00          mov    eax,0x8
  ca:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  ce:   b8 00 00 00 00          mov    eax,0x0
  d3:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
  d7:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
... (26 more lines not shown)
```

In the disassembly, `main` pushes 8 and 7 (the last arguments first), loads the first six into registers, calls `many`, and then adds 16 to `rsp` to discard the pushed values. `many` reads `g` from `[rbp+0x10]` and `h` from `[rbp+0x18]`.

12.4.2 Stack frames

A function that needs stack space, saves callee-saved registers or makes calls gets a frame:

SHASM

```
push rbp                ; prologue
mov  rbp, rsp
sub  rsp, N             ; locals, saved registers, temporaries
mov  [rbp-8], r12       ; save only the r12-r15 actually used
...
mov  r12, [rbp-8]       ; restore them
leave                   ; epilogue: mov rsp, rbp / pop rbp
ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

A leaf function that keeps everything in the caller-saved registers needs no frame at all and is emitted as straight-line code ending in `ret` (see Section 17.7). The entry point `_start` additionally aligns the stack to 16 bytes (`and rsp, -16`), because the process entry is not reached through a `call`.

12.5 RECURSION

Functions may call themselves directly or indirectly. Each activation has its own parameters and locals (in registers saved by the callee, or in its frame), so recursion works as in C. The depth is limited only by the size of the stack (8 MiB by default on Linux).

Example 12-5. Recursion: greatest common divisor and Ackermann

ASH

```
func gcd(int64 a, int64 b): int64 {
    if (b == 0) { return a; }
    return gcd(b, a % b);
}

func ack(int64 m, int64 n): int64 {
    if (m == 0) { return n + 1; }
    if (n == 0) { return ack(m - 1, 1); }
    return ack(m - 1, ack(m, n - 1));
}

@Entry
pub func main(): int64 {
    printf("gcd(1071, 462)=%d ack(2, 3)=%d\n", gcd(1071, 462), ack(2, 3));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
gcd(1071, 462)=21 ack(2, 3)=9
```

12.6 VARIADIC FUNCTIONS

A function whose parameter list ends with `...` accepts up to **8** extra arguments after its fixed parameters. Inside the function:

- `va_count()` is the number of extra arguments actually passed;
- `va_arg(i)` is the extra argument number *i* (0-based), as a 64-bit integer or address.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 12-6. A variadic sum

ASH

```
func sum(int64 base, ...): int64 {
    int64 total = base;
    for (int64 i = 0; i < va_count(); i++) {
        total = total + va_arg(i);
    }
    return total;
}

func join(string first, ...): void {
    printf("%s", first);
    for (int64 i = 0; i < va_count(); i++) {
        printf(" %s", va_arg(i));      // extra strings arrive as addresses
    }
    printf("\n");
}

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", sum(100), sum(100, 1, 2, 3), sum(0, 1, 2, 3, 4, 5, 6, 7, 8));
    join("ASH", "is", "a", "language");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
100 106 36
ASH is a language
```

12.6.1 How variadic functions are implemented

A variadic ASH function is an ordinary function with nine hidden parameters: `__vac` (the count) and `__va0` to `__va7`. At every call site the compiler passes the fixed arguments, the number of extra arguments, the extra arguments, and zeros for the unused slots. `va_count()` is simply `__vac`, and `va_arg(i)` calls the helper `__va_get(i, __va0, ..., __va7)`, which returns the selected argument. A call with more than eight extra arguments is a compile-time error.

Example 12-7. The hidden parameters of a variadic function

ASH

```
func count_args(int64 first, ...): int64 {
    return va_count();
}

@Entry
pub func main(): int64 {
    return count_args(0, 10, 20, 30);
}
```

ASHIR

```
func qword count_args(qword first, qword __vac, qword __va0, qword __va1, qword __va2, qword
↪  __va3, qword __va4, qword __va5, qword __va6, qword __va7) {
    ret qword __vac
}

[entry]
func qword main() {
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
    %__tmp0 mov[0]
    %__tmp1 mov[3]
    %__tmp2 mov[10]
    %__tmp3 mov[20]
    %__tmp4 mov[30]
    %__tmp5 mov[0]
    %__tmp6 mov[0]
    %__tmp7 mov[0]
    %__tmp8 mov[0]
    %__tmp9 mov[0]
    %__ret call[count_args, __tmp0, __tmp1, __tmp2, __tmp3, __tmp4, __tmp5, __tmp6, __tmp7,
↪ __tmp8, __tmp9]
    ret qword __ret
}
```

IMPORTANT

Variadic ASH functions are not compatible with C variadic functions: C cannot call an ASH variadic function with the C `...` convention, and an ASH variadic function cannot be declared as a C function pointer type. Calls from ASH to C variadic functions such as `printf` use the C convention and are not limited to eight arguments.

12.7 FUNCTION POINTERS AND CALLBACKS

The expression `&f` (or simply `f` where a value is expected) yields the address of the function `f`. In ASHIR this is the operation `faddr[f]`, which becomes a RIP-relative `lea` in machine code. A function address can be:

- passed to C functions that expect a callback (`qsort`, `atexit`, `pthread_create`, `signal`, GUI toolkits ...);
- stored in an integer or pointer variable and called through it: `fp(2, 3)`;
- stored in a structure field and called through a member chain: `ops->handler(2, 3)`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 12-8. Callbacks passed to the C library

ASH

```
extern int64 atexit(int64 fn);
extern void qsort(int64 base, int64 n, int64 size, int64 cmp);

func goodbye(): void {
    printf("goodbye (called by atexit)\n");
}

func compare(int64 a, int64 b): int64 {
    int64 x = *(int64*)a;
    int64 y = *(int64*)b;
    if (x < y) { return -1; }
    elif (x > y) { return 1; }
    return 0;
}

@Entry
pub func main(): int64 {
    atexit(&goodbye);
    int64 v[5];
    v[0] = 5; v[1] = 2; v[2] = 9; v[3] = 1; v[4] = 7;
    qsort(&v, 5, 8, compare);
    printf("%d %d %d %d %d\n", v[0], v[1], v[2], v[3], v[4]);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 2 5 7 9
goodbye (called by atexit)
```

Example 12-9. Calling through a variable and through a structure field

ASH

```
typedef struct {
    uint64 handler;
} Ops;

func add(int64 a, int64 b): int64 {
    return a + b;
}

@Entry
pub func main(): int64 {
    int64 fp = &add;
    int64 r1 = fp(2, 3);             // call through a variable
    Ops o;
    Ops* ops = &o;
    ops->handler = &add;
    int64 r2 = ops->handler(4, 5);   // call through a structure field
    printf("%d %d\n", r1, r2);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
5 9
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR

```
[entry]
func qword main() {
.type fp qword
.type r1 qword
.type o qword
.type ops qword
.type r2 qword
    %fp faddr[add]
.type __tmp0 qword
    %__tmp0 mov[2]
.type __tmp1 qword
    %__tmp1 mov[3]
.type __tmp2 qword
    %__tmp2 mov[fp]
    %r1 mov[0]
    %_ asm[x6d6f76207264692c20407661723a5f5f746d70300a6d6f76207273692c20407661723a5f5f746d70
↪ 310a6d6f76207261782c20407661723a5f5f746d70320a63616c6c207261780a6d6f7620407661723a72312c20
↪ 7261780a]
.type __tmp3 qword
    %__tmp3 mov[8]
    %o call[__ash_alloc, __tmp3, 1, 0]
    %ops mov[o]
    %__tmp4 mov[ops]
.type __tmp5 qword
    %__tmp5 faddr[add]
    store[__tmp4, __tmp5]
.type __tmp6 qword
    %__tmp6 mov[4]
.type __tmp7 qword
    %__tmp7 mov[5]
.type __tmp8 qword
    %__tmp9 mov[ops]
.type __tmp10 qword
    %__tmp10 load[__tmp9]
    %__tmp8 mov[__tmp10]
    %r2 mov[0]
    %_ asm[x6d6f76207264692c20407661723a5f5f746d70360a6d6f76207273692c20407661723a5f5f746d70
↪ 370a6d6f76207261782c20407661723a5f5f746d70380a63616c6c207261780a6d6f7620407661723a72322c20
↪ 7261780a]
    %__tmp11 addr[__str0]
    %__tmp12 mov[r1]
    %__tmp13 mov[r2]
    call[printf, __tmp11, __tmp12, __tmp13]
    ret qword 0
}
```

An indirect call is compiled to a small assembly sequence that loads the arguments into the argument registers, loads the target address into `rax` and executes `call rax`. With `--win`, `@msabi` or `@efiapi` the Microsoft convention is used for indirect calls, which is what UEFI protocol functions such as `ConOut->OutputString` expect.

12.8 EXTERNAL FUNCTIONS

`extern` declares a function implemented outside the ASH program: in the C library, in another object file, in a DLL. The declaration gives the parameter types (names are optional) and the return type:

Copyright © 2026 Mallionic Inc. All rights reserved.

```
extern return-type name ( [type [name], ...] [, ...] ) ;
extern func name ( ... ) ;                 // returns void
```

Example 12-10. Declaring external C functions

ASH

```
extern int64 labs(int64 v);
extern func srand(uint32 seed);
extern int64 strlen(string);
extern int64 snprintf(string buf, uint64 n, string fmt, ...);

@Entry
pub func main(): int64 {
    srand(1);
    uint8[32] buf;
    snprintf(&buf, 32, "%d-%d", 12, 34);
    printf("%d %d %s\n", labs(-5), strlen("abcd"), &buf);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
5 4 12-34
```

12.8.1 Implicit external functions

A call to a name that is neither an ASH function nor declared `extern` is compiled as a call to an external symbol with that name, with all arguments passed as integers. This is how `printf`, `puts` or `malloc` can be used without declarations. Declaring the function with `extern` is still recommended: the compiler then checks the number of arguments, uses the declared widths, and the VS Code linter can verify your calls.

> **NOTE**
> Without the C library (`--no-libc`), an undeclared external function must be supplied by another object file at link time; otherwise the linker reports an undefined reference.

12.9 THE ENTRY POINT

Exactly one function of a program carries the annotation `@Entry`. It is the function that starts the program. It must be `pub` so that its symbol is visible to the linker, and it is never replaced by override rules (Section 19.4).

The entry point may receive the command line arguments. With the C runtime (an entry named `main`, or `_start` renamed to `main`), declare two parameters: the argument count and the address of the argument vector. The library `<args>` provides helpers to read the vector (Section 27.5).

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 12-11. Command line arguments

ASH

```
import <args>;

@Entry
pub func main(int64 argc, int64 argv): int64 {
    printf("argc = %d\n", argc);
    for (int64 i = 1; i < argc; i++) {
        printf("  [%d] %s\n", i, arg_at(argv, i));
    }
    printf("has -v: %d, second as number: %d\n", arg_has(argc, argv, "-v"), arg_int(argv, 2)
↪ );
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
argc = 4
  [1] alpha
  [2] 42
  [3] -v
has -v: 1, second as number: 42
```

The example above was run as `./program alpha 42 -v`. Element 0 of the vector is the program path, which is why the loop starts at 1.

12.10 NAKED FUNCTIONS

A function annotated `@naked` contains only `asm` blocks and `emit` statements; the compiler emits no prologue, no epilogue and no `ret`. Arguments are taken directly from the registers of the calling convention. Naked functions are used for interrupt handlers, entry stubs and code whose every byte must be controlled. See Section 20.5.

12.11 METHODS

Functions declared inside a `class` are methods; they are called as `Class.method(...)`. Methods are ordinary functions named `Class_method` in the object file. See Chapter 15.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 13

ARRAYS, STRINGS AND MATRICES

Arrays in ASH are blocks of elements of one type that know their own length. This chapter describes how arrays are declared and initialized, how they are laid out in memory, how indexing and bounds checking work, how two-dimensional arrays and matrices are built, and how text is stored in arrays and buffers.

13.1 DECLARING ARRAYS

Table 13-1. Array declaration forms

Form	Meaning
`int64[] v = {1, 2, 3};`	An array whose length is the number of initializers.
`int64[8] buf;`	An array of 8 elements, all zero.
`int64 buf[8];`	The same, in C syntax.
`int64[3][4] grid;`	A two-dimensional array of 3 rows and 4 columns, all zero.
`string[] names = {"Ana", "Luis"};`	An array of string addresses.
`matrix m[] []{ ... }`	A matrix written row by row (Section 13.7).
`matrix uint64 z[3][4];`	A zero-filled matrix.

The size of an array may be any expression whose value is known when the declaration executes; it does not need to be a constant. The initializer list may end with a trailing comma.

Example 13-1. Array declarations

ASH

```
@Entry
pub func main(): int64 {
    int64[] primes = {2, 3, 5, 7, 11, 13,};
    int64 n = 5;
    int64[n] squares;                    // size computed at run time
    for (int64 i = 0; i < n; i++) {
        squares[i] = i * i;
    }
    uint8 bytes[4];
    bytes[2] = 0xAB;
    printf("primes=%d squares[4]=%d bytes[2]=%x lengths: %d %d %d\n",
        #primes, squares[4], bytes[2], #primes, #squares, #bytes);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
primes=6 squares[4]=16 bytes[2]=ab lengths: 6 5 4
```

Copyright © 2026 Mallionic Inc. All rights reserved.

13.2 MEMORY LAYOUT

An array variable holds the **address of its first element**. The elements are stored in a heap block obtained from the runtime helper `__ash_alloc`, which allocates the block with `calloc` (so the elements start at zero) and prefixes it with a 16-byte header:

```
 block address        variable value (address of element 0)
      |                      |
      v                      v
      +----------+----------+----------+----------+-----+----------+
      |  length  | capacity | elem 0   | elem 1   | ... | elem n-1 |
      +----------+----------+----------+----------+-----+----------+
       8 bytes    8 bytes    element size each
```

Because the variable points at the elements, an array can be passed directly to C functions that expect a pointer (`qsort(&v, ...)`, `write(fd, &buf, n)`), and pointer arithmetic on it addresses the elements. The length is stored 16 bytes before the first element and the capacity 8 bytes before it; `#v` reads the length.

Example 13-2. Inspecting the header of an array

ASH

```
@Entry
pub func main(): int64 {
    int64[] v = {10, 20, 30, 40};
    uint64* raw = &v;
    printf("length=%d capacity=%d first=%d\n", raw[-2], raw[-1], raw[0]);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
length=4 capacity=4 first=10
```

ASHIR

```
```

The ASHIR shows the allocation `call[__ash_alloc, 4, 8, 0]` (four elements of eight bytes, no extra bytes) followed by one `store` per initializer at offsets 0, 8, 16 and 24.

> **IMPORTANT**
> Arrays declared in a function are **not freed** when the function returns: the block remains allocated until the program ends. A function that declares an array and is called many times allocates a new block on every call. For long-running loops, declare work arrays once in the caller and pass them as pointers, or use a global array.

13.3 INDEXING

`v[i]` designates element *i* of `v`, counting from 0. The element address is computed as `v + i * size`, where *size* is the element size (1, 2, 4 or 8). Loads and stores use the width of the element type, and narrow elements are extended on load according to their signedness (Section 8.2.2).

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 13-3. Machine code for an indexed store and load

ASH

```
@beautycode(false)

@Entry
pub func main(): int64 {
    int32[4] v;
    int64 i = 2;
    v[i] = 7;
    return v[i];
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000000d1 <main>:
  d1:   55                      push   rbp
  d2:   48 89 e5                mov    rbp,rsp
  d5:   48 81 ec 40 00 00 00    sub    rsp,0x40
  dc:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  e0:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  e4:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  e8:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  ec:   b8 04 00 00 00          mov    eax,0x4
  f1:   49 89 c4                mov    r12,rax
  f4:   b8 00 00 00 00          mov    eax,0x0
  f9:   49 89 c5                mov    r13,rax
  fc:   4c 89 e7                mov    rdi,r12
  ff:   be 04 00 00 00          mov    esi,0x4
 104:   ba 00 00 00 00          mov    edx,0x0
 109:   e8 f2 fe ff ff          call   0 <__ash_alloc>
 10e:   49 89 c5                mov    r13,rax
 111:   b8 02 00 00 00          mov    eax,0x2
 116:   49 89 c6                mov    r14,rax
 119:   4c 89 f0                mov    rax,r14
 11c:   49 89 c7                mov    r15,rax
 11f:   4c 89 f8                mov    rax,r15
 122:   48 6b c0 04             imul   rax,rax,0x4
 126:   49 89 c7                mov    r15,rax
 129:   4c 89 f8                mov    rax,r15
 12c:   4c 89 e9                mov    rcx,r13
 12f:   48 01 c8                add    rax,rcx
 132:   49 89 c7                mov    r15,rax
 135:   b8 07 00 00 00          mov    eax,0x7
 13a:   89 45 d8                mov    DWORD PTR [rbp-0x28],eax
 13d:   4c 89 f8                mov    rax,r15
 140:   8b 4d d8                mov    ecx,DWORD PTR [rbp-0x28]
 143:   48 63 c9                movsxd rcx,ecx
 146:   89 08                   mov    DWORD PTR [rax],ecx
 148:   4c 89 f0                mov    rax,r14
 14b:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 14f:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 153:   48 6b c0 04             imul   rax,rax,0x4
 157:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 15b:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 15f:   4c 89 e9                mov    rcx,r13
 162:   48 01 c8                add    rax,rcx
 165:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 169:   b8 00 00 00 00          mov    eax,0x0
 16e:   89 45 c8                mov    DWORD PTR [rbp-0x38],eax
 171:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 175:   8b 08                   mov    ecx,DWORD PTR [rax]
 177:   89 4d c8                mov    DWORD PTR [rbp-0x38],ecx
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
... (10 more lines not shown)
```

13.3.1 Bounds checking

By default every access to an array element is checked: before computing the address, the code calls `__ash_bounds(v, i)`, which reads the length from the header and **terminates the program with exit status 134** if `i` is not smaller than the length. Out-of-range accesses therefore stop the program instead of silently corrupting memory.

Example 13-4. An out-of-range index stops the program

ASH

```
func fill(int64 n): int64 {
    int64[8] v;
    int64 total = 0;
    for (int64 i = 0; i < n; i++) {
        v[i] = i * 3;
        total = total + v[i];
    }
    return total;
}

@Entry
pub func main(): int64 {
    printf("n=8 -> %d\n", fill(8));
    fflush(0);
    printf("n=9 -> %d\n", fill(9));     // index 8 is out of range
    return 0;
}
```

OUTPUT — EXIT STATUS 134

```
n=8 -> 84
```

Bounds checks apply to arrays (which have a header). Accesses through raw pointers (`uint8* p; p[i]`) and through `string` values are never checked, because they have no length. Bounds checking is controlled with `@bounds(true|false)` and `@beautycode(true|false)` (Section 17.7); turning it off removes one call per access and makes the code smaller and faster.

13.4 THE LENGTH OPERATOR

`#v` is the number of elements of the array `v`. For a two-dimensional array or a matrix it is the number of rows. The built-in functions `len(v)` and `length(v)` are synonyms of `#v`; `rows(m)`, `cols(m)` and `count(m)` give the number of rows, columns and elements of a matrix. These built-ins are only used when the program does not define functions with the same names.

`#` reads the header of the array at run time, so it also works on an array received through a pointer parameter:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 13-5. Length of an array passed to a function

ASH

```
func sum(int64* v): int64 {
    int64 s = 0;
    for (int64 i = 0; i < #v; i++) {
        s = s + v[i];
    }
    return s;
}

@Entry
pub func main(): int64 {
    int64[] data = {4, 8, 15, 16, 23, 42};
    printf("count=%d sum=%d\n", len(data), sum(data));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
count=6 sum=108
```

13.5 ARRAYS AS PARAMETERS

An array is passed to a function as the address of its first element. Declare the parameter as a pointer to the element type (`int64* v`, `uint8* buf`); indexing then scales by the element size and `#v` reads the length from the header.

WARNING

A parameter written in C array syntax, `int64 v[]`, is accepted but becomes an untyped address in `ashc` 1.0: indexing it addresses *bytes*, not `int64` elements. Write `int64* v`.

13.6 TWO-DIMENSIONAL ARRAYS

`T[R][C] name;` declares an array of *R* rows of *C* elements. It is built by the helper `__ash_matrix(R, C, size)` as an array of *R* row addresses, each pointing to its own array of *C* elements. `m[i][j]` first loads the address of row *i*, then indexes into it; both steps are bounds-checked. `#m` is *R*, and `#m[i]` is *C*.

Example 13-6. A two-dimensional array

ASH

```
@Entry
pub func main(): int64 {
    int64[3][3] m;
    for (int64 i = 0; i < 3; i++) {
        for (int64 j = 0; j < 3; j++) {
            m[i][j] = i * 3 + j;
        }
    }
    printf("m[2][1]=%d rows=%d\n", m[2][1], #m);
    for (int64 i = 0; i < 3; i++) {
        printf("%d %d %d\n", m[i][0], m[i][1], m[i][2]);
    }
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
m[2][1]=7 rows=3
0 1 2
3 4 5
6 7 8
```

13.7 MATRICES

The `matrix` statement declares a two-dimensional array of integers in a tabular form. Each element is written `[type value]`; **each source line is one row**, and every row must have the same number of elements. The element type of the matrix is the widest type that appears.

```
matrix name[] []{
    [type v], [type v], ...,        // row 0
    [type v], [type v], ...,        // row 1
}
matrix type name[rows][cols];       // zero-filled
```

Example 13-7. A matrix literal and its dimensions

ASH

```
@Entry
pub func main(): int64 {
    matrix m[] []{
        [uint64 64324], [uint64 64231], [uint64 6445],
        [uint64 64324], [uint64 64234], [uint64 6454],
    }
    matrix uint32 z[2][5];
    z[1][4] = 9;
    printf("rows=%d cols=%d count=%d m[1][2]=%llu\n", rows(m), cols(m), count(m), m[1][2]);
    printf("z: %dx%d z[1][4]=%d\n", rows(z), cols(z), z[1][4]);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
rows=2 cols=3 count=6 m[1][2]=6454
z: 2x5 z[1][4]=9
```

Example 13-8. Rows of different length

ASH

```
@Entry
pub func main(): int64 {
    matrix m[] []{
        [int32 1], [int32 2],
        [int32 3],
    }
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
<input>:7:5: error: matrix 'm': row 2 (line 5) has 1 elements and the first one has 2
    7 | return
      |     ^^^^^^
1 error generated.
```

Copyright © 2026 Mallionic Inc. All rights reserved.

13.8 TEXT IN ARRAYS AND BUFFERS

ASH offers several ways to hold text. Choose according to whether the text is constant and whether its length is known:

Table 13-2. Ways to store text

Declaration	Use
`string s = "literal";`	Constant text. `s` points into read-only data; do not modify it.
`string[] list = {"a", "b"};`	An array of addresses of constant strings; `#list` is the number of strings.
`uint8[N] buf;`	A modifiable, zero-filled buffer of *N* bytes. Pass `&buf` to `strcpy`, `strcat`, `snprintf`, `read` ... The buffer is bounds-checked when indexed as `buf[i]`.
`general char[] text = """..."""`	A global, modifiable array of characters initialized with text, with a length header (Section 9.4.3).

Example 13-9. Building text in a buffer with <string>

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[64] buf;
    strcpy(&buf, "Hello");
    strcat(&buf, ", ");
    strcat(&buf, "ASH");
    str_upper(&buf);
    printf("%s (%d bytes)\n", &buf, strlen(&buf));
    buf[0] = 'J';                          // a buffer may be modified
    printf("%s\n", &buf);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
HELLO, ASH (10 bytes)
JELLO, ASH
```

Example 13-10. Arrays of strings

ASH

```
@Entry
pub func main(): int64 {
    string[] days = {"Mon", "Tue", "Wed", "Thu", "Fri"};
    for (int64 i = 0; i < #days; i++) {
        printf("%d:%s ", i, days[i]);
    }
    printf("\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0:Mon 1:Tue 2:Wed 3:Thu 4:Fri
```

Copyright © 2026 Mallionic Inc. All rights reserved.

13.8.1 char arrays

A *global* `char[]` initialized with a string literal is a modifiable copy of the text with a length header: `#text` is the number of characters and `text[i]` is bounds-checked.

> **WARNING**
> In `ashc` 1.0 a *local* `char[]` initialized with a literal (`char[] c = "abc";`) points at the literal itself, without a header: `#c` returns garbage and writing `c[i]` stops the program in the bounds check. Inside functions, use a `uint8[N]` buffer and `strcpy` instead.

13.9 GLOBAL AND RUNNABLE ARRAYS

Global arrays with initializers, zero-filled global arrays and `runnable` arrays are described in Section 9.4.3 and Section 9.4.4. They have the same header as local arrays, so everything in this chapter applies to them; the difference is that their storage is part of the object file instead of the heap.

13.10 COST MODEL

Table 13-3. What array operations cost

Operation	Cost
Declaration of an array of *n* elements	One call to `__ash_alloc` (one `calloc`), plus one store per initializer.
`v[i]` with bounds checking	One call to `__ash_bounds`, an address computation and a load or store.
`v[i]` without bounds checking	An address computation and a load or store.
`m[i][j]`	Two of the above (row address, then element).
`#v`	One load from the header.
Leaving the function	Nothing: the block is not freed.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 14

STRUCTURES AND POINTERS

A structure groups named fields of different types into one object. ASH structures use the same layout rules as C, so a structure declared in ASH can describe a C structure, a hardware register block, a file header or a firmware table byte for byte. This chapter describes declaration, layout, field access, pointers to structures and chains of pointers.

14.1 DECLARING STRUCTURES

Two forms are accepted, both taken from C:

```
typedef struct [Tag] {
    type field;
    ...
} Name;

struct Name {
    type field;
    ...
};
```

After either declaration, `Name` is a type that can be used for variables, pointers (`Name*`) and fields of other structures. In the second form, `struct Name var;` is also accepted (the word `struct` is dropped).

A field type can be any integer type, `bool`, `char`, a pointer (including a pointer to a structure, even of the same structure), or another structure already defined. A structure is embedded in another one by writing its name as the field type: `Point origin;`.

14.2 LAYOUT

Fields are laid out in declaration order. Each field is aligned to its *natural alignment* (its own size for integers and pointers, the largest alignment of its fields for an embedded structure); padding bytes are inserted where needed. The size of the structure is rounded up to a multiple of its largest alignment. These are the rules of the System V ABI, which C compilers follow on x86-64.

Table 14-1. Layout of the Person structure

Field	Offset	Size	Note
`uint16 age`	0	2 |	
`uint8 alive`	2	1	followed by 1 byte of padding
`uint32 id`	4	4	aligned to 4
`int64 balance`	8	8	aligned to 8
(total)	| 16	multiple of 8	

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 14-1. Fields at their C offsets

ASH

```
typedef struct {
    uint16 age;
    uint8  alive;
    uint32 id;
    int64  balance;
} Person;

@Entry
pub func main(): int64 {
    Person p;
    p.age = 14;
    p.alive = 1;
    p.id = 0x12345678;
    p.balance = -250;
    uint8* raw = &p;                        // look at the bytes
    printf("byte 0=%x byte 2=%x bytes 4..7=%x %x %x %x\n",
        raw[0], raw[2], raw[4], raw[5], raw[6], raw[7]);
    printf("age=%d alive=%d id=%x balance=%lld\n", p.age, p.alive, p.id, p.balance);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
byte 0=e byte 2=1 bytes 4..7=78 56 34 12
age=14 alive=1 id=12345678 balance=-250
```

The bytes of `id` appear in the order 78 56 34 12 because x86 is little-endian: the least significant byte is stored first.

14.3 STRUCTURE VARIABLES

A variable of structure type is a block of memory large enough for the structure, obtained from `__ash_alloc` and zero-filled when the declaration executes. The variable holds the address of the block: `&p` and `p` used as an address are the same value. Assigning one structure variable to another copies the address, not the contents; copy contents field by field or with `memcpy`.

Example 14-2. A structure variable is a zero-filled block

ASH

```
struct Point { int32 x; int32 y; };

@Entry
pub func main(): int64 {
    Point q;
    q.x = 3;
    q.y = 4;
    return q.x + q.y;
}
```

ASHIR

Copyright © 2026 Mallionic Inc. All rights reserved.

14.4 FIELD ACCESS

Table 14-2. Field access

Expression	Meaning
`p.field`	The field of the structure variable `p`.
`ptr->field`	The field of the structure at the address held in the pointer `ptr`.
`a->b->c`, `a->b.c`, `a.b.c`	Chains: follow pointer fields with `->`, enter embedded structures with `.`.
`p.field = v;` `ptr->field = v;`	Store into a field, with the width of the field.
`p.n++`, `q->n--`	Increment or decrement a field.
`&p`	Address of the structure.

Every access is translated into a typed load or store at *base address + offset*. In ASHIR this is visible as an `add` of the offset followed by `load` or `store`; in machine code it becomes a single instruction with a displacement, such as `mov DWORD PTR [rax+0x4], ecx`.

Example 14-3. Pointers to structures and chains

ASH

```
struct Point { int32 x; int32 y; };

typedef struct {
    Point  origin;       // embedded structure at offset 0
    Point* next;         // pointer to another structure
    uint8  tag;
} Node;

func move_right(Point* pt, int32 dx): void {
    pt->x = pt->x + dx;
}

@Entry
pub func main(): int64 {
    Point q;
    q.x = 3;
    q.y = 4;
    Node n;
    n.next = &q;
    Node* np = &n;
    np->origin.x = 7;              // embedded field through a pointer
    int64 y = np->next->y;         // follow the pointer field
    np->next->x = 99;              // store through two levels
    move_right(&q, 1);
    printf("y=%d origin.x=%d q.x=%d\n", y, n.origin.x, q.x);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
y=4 origin.x=7 q.x=100
```

Copyright © 2026 Mallionic Inc. All rights reserved.

14.5 STRUCTURES AND FUNCTIONS

Structures are passed to functions by address. Declare the parameter as a pointer (`Person* x`) and access the fields with `->`. A function can return a pointer to a structure, but not a structure by value.

Example 14-4. Passing a structure to a function

ASH

```
typedef struct {
    uint16 age;
    uint8  alive;
    uint32 id;
    int64  balance;
} Person;

func birthday(Person* x): void {
    x->age = x->age + 1;
}

func describe(Person* x): void {
    printf("id=%x age=%d balance=%lld\n", x->id, x->age, x->balance);
}

@Entry
pub func main(): int64 {
    Person p;
    p.age = 14;
    p.id = 0xCAFE;
    p.balance = -250;
    Person* ptr = &p;
    ptr->balance = ptr->balance * 2;
    birthday(ptr);
    birthday(&p);
    describe(&p);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
id=cafe age=16 balance=-500
```

14.6 SHARING STRUCTURES WITH C

Because the layout is the C layout, a pointer to an ASH structure can be passed to a C function expecting the corresponding C structure, and vice versa. Declare the fields with types of the same width and signedness as the C fields (see Table 8-2).

POINT_LIB.ASH

```
struct Vec { int32 x; int32 y; int64 weight; };

pub func vec_scale(Vec* v, int32 k): void {
    v->x = v->x * k;
    v->y = v->y * k;
    v->weight = v->weight + 1;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

POINT_MAIN.C

```c
#include <stdio.h>
struct Vec { int x; int y; long weight; };
void vec_scale(struct Vec *v, int k);

int main(void) {
    struct Vec v = { 2, 3, 10 };
    vec_scale(&v, 5);
    printf("x=%d y=%d weight=%ld\n", v.x, v.y, v.weight);
    return 0;
}
```

SHELL SESSION

```
$ ashc point_lib.ash --elf -s
$ gcc -no-pie point_main.c out/point_lib.o -o demo
$ ./demo
x=10 y=15 weight=11
```

14.7 STRUCTURES FOR MEMORY-MAPPED DATA

A pointer to a structure can be set to any address, which is how hardware registers and firmware tables are accessed. Accesses through such a pointer are never bounds-checked. The following fragment describes the 16-bit VGA text cell and writes to the screen buffer of a PC in text mode (meaningful only on bare metal):

ASH

```
typedef struct {
    uint8 ch;
    uint8 attr;
} Cell;

func put(int64 col, int64 c): void {
    uint64 screen = 0xB8000;
    Cell* cell = screen + col * 2;
    cell->ch = c;
    cell->attr = 0x1F;          // white on blue
}
```

14.8 LIMITATIONS

- A structure cannot be declared inside a function.
- A field cannot be an array; use consecutive fields or a pointer to an array.
- There is no `sizeof`; the size of a structure is the offset of its last field plus its size, rounded to the alignment (see Table 14-1). Write the size as a `define` constant when you need it.
- Structures cannot be returned or passed by value.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 15

CLASSES AND VISIBILITY

ASH classes group related functions under a common name and control which of them can be called from outside. A class is a *namespace with access control*: it contains methods, not data, and it is never instantiated. This chapter describes class syntax, method calls, private methods, the symbols that classes produce, and the visibility rules for functions across files.

15.1 DECLARING A CLASS

```
[pub | priv] class Name [()] {
    [pub | priv] func method ( parameters ) [: type] { ... }
    ...
}
```

The parentheses after the class name are optional. A class body contains only function declarations. A method without `pub` or `priv` is accessible to the program, like an ordinary function. A `priv` class makes all its methods private.

Example 15-1. A class with public and private methods

ASH

```
class Geometry {
    pub func area(int64 w, int64 h): int64 {
        return w * h;
    }
    pub func perimeter(int64 w, int64 h): int64 {
        return 2 * (w + h);
    }
    priv func scale(): int64 {
        return 10;
    }
    pub func scaled_area(int64 w, int64 h): int64 {
        return Geometry.area(w, h) * Geometry.scale();     // private: allowed here
    }
}

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", Geometry.area(3, 4), Geometry.perimeter(3, 4), Geometry.scaled_area
↪ (1, 2));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
12 14 20
```

Copyright © 2026 Mallionic Inc. All rights reserved.

15.2 CALLING METHODS

A method is called with the class name, a dot and the method name: `Geometry.area(3, 4)`. Inside the class the same notation is used. There is no implicit `this` or `self`: every value a method needs is passed as a parameter.

15.3 SYMBOLS PRODUCED BY A CLASS

The parser renames each method to `Class_method`. `Geometry.area` is therefore the function `Geometry_area` in ASHIR and in the object file, and C code calls it with that name. The underscore form can also be used from ASH.

Example 15-2. Methods become functions named Class_method

ASH

```
pub class Counter {
    pub func next(int64 n): int64 {
        return n + 1;
    }
}

@Entry
pub func main(): int64 {
    return Counter.next(41) + Counter_next(0);
}
```

ASHIR

```
func qword Counter_next(qword n) {
    %__ret mov[n]
    %__ret add[1]
    ret qword __ret
}

[entry]
func qword main() {
.type __tmp0 qword
    %__tmp1 mov[41]
    %__tmp0 call[Counter_next, __tmp1]
    %__ret mov[__tmp0]
.type __tmp2 qword
    %__tmp3 mov[0]
    %__tmp2 call[Counter_next, __tmp3]
    %__ret add[__tmp2]
    ret qword __ret
}
```

15.4 PRIVATE METHODS

A `priv` method can only be called by methods of the same class. A call from anywhere else is rejected at compile time, before any code is generated:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 15-3. Calling a private method from outside its class

ASH

```
class Geometry {
    priv func scale(): int64 {
        return 10;
    }
}

@Entry
pub func main(): int64 {
    return Geometry.scale();
}
```

COMPILER DIAGNOSTICS

```
error: 'Geometry.scale' is private: only methods of class Geometry can call it (called from
↪ 'main'). Make it 'pub' to expose it.
1 error generated.
```

Private methods also cannot be exported to C with `redefine global` (Section 18.7): the compiler reports an error instead of exposing them.

15.5 VISIBILITY OF FUNCTIONS ACROSS FILES

The keywords `pub` and `priv` apply to ordinary functions as well:

Table 15-1. Visibility of functions

Keyword	Effect
`pub`	Global symbol in the object file. Callable from C, from other objects and from every ASH file of the program. Required for the entry point and for exported functions.
(none)	Callable from every ASH file of the program; a local symbol in the object file.
`priv`	Callable only from the file that defines it. A call from a file that imports it is a compile-time error.

Classes and override rights (Section 19.4) combine: a class can be given a right with `@ovr[N]` before `class`, and the right applies to all its methods.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 16

CRACKS AND PENDING RETURN VALUES

Cracks and pending return values are two features specific to ASH. A *crack* is a named entry point declared inside a function that lets other code configure that function's behavior. A *pending return value*, set with `unreturnable`, gives a function a result without stopping it. This chapter describes both, and how they are compiled.

16.1 CRACKS

16.1.1 Declaring a crack

A crack is declared inside the body of a function:

```
name -> ( [type param, ...] ) { statements };
```

The statements of a crack do not run when the function runs. They run when another part of the program *invokes* the crack with an assignment-like statement:

```
function.name = value [, value]... ;
```

The values are passed to the parameters of the crack, in order. A crack can be invoked any number of times, before or after the function itself is called.

Example 16-1. A crack that configures a logger

ASH

```
general static int64 level = 0

func logger(string msg): void {
    verbosity -> (int64 v) {
        level = v
    };
    if (level > 1) {
        printf("[log] %s\n", msg);
    }
}

@Entry
pub func main(): int64 {
    logger("this message is hidden");
    logger.verbosity = 2;              // runs the crack: level = 2
    logger("this message is shown");
    printf("level=%d\n", level);
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
[log] this message is shown
level=2
```

16.1.2 How cracks are compiled

Each crack becomes an independent function named `__crack_<function>_<crack>` with the crack's parameters, and each invocation `f.c = a, b;` becomes a call `__crack_f_c(a, b);`. The enclosing function itself is not modified.

Example 16-2. The function generated for a crack

ASH

```
general static int64 limit = 10

func worker(): int64 {
    set_limit -> (int64 n) {
        limit = n
    };
    return limit;
}

@Entry
pub func main(): int64 {
    worker.set_limit = 42;
    return worker();
}
```

ASHIR

```
func qword worker() {
    %__tmp0 addr[limit]
    %__ret load[__tmp0]
    ret qword __ret
}

[entry]
func qword main() {
    %__tmp1 mov[42]
    call[__crack_worker_set_limit, __tmp1]
    %__ret call[worker]
    ret qword __ret
}

func void __crack_worker_set_limit(qword n) {
    %__tmp2 addr[limit]
.type __tmp3 qword
    %__tmp3 mov[n]
    store[__tmp2, __tmp3]
    ret void
}

.gvar limit 8 10
```

Copyright © 2026 Mallionic Inc. All rights reserved.

> **IMPORTANT**
> Because a crack is a separate function, it **cannot see the local variables or parameters of the function in which it is written**. A crack communicates with its function through global variables (declared `static`) or through `unreturnable`. Assigning to a name that looks like a local of the enclosing function creates a local of the crack instead, and has no visible effect.

16.1.3 When to use cracks

Cracks keep the configuration of a function next to the function. Typical uses are verbosity levels, modes of operation, callbacks and test hooks: the code that configures and the code that is configured are written in one place, while callers use a short, explicit syntax (`logger.verbosity = 2;`).

16.2 PENDING RETURN VALUES

`unreturnable expression;` stores a *pending return value* for the current function **without leaving it**. The function continues to run. When it later ends without an explicit value (`return;`, `terminate;`, or reaching the closing brace), it returns the pending value. An explicit `return value;` overrides the pending value.

Table 16-1. How a function ends with a pending value

Ending	Result
`return value;`	`value` (the pending value is ignored)
`return;`	The pending value, or 0 if none was set
`terminate;`	The pending value, or 0
`terminate value;`	`value`
End of the body	The pending value

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 16-3. unreturnable sets a result and keeps running

ASH

```
general static uint64 counter = 0

func measure(): uint64 {
    unreturnable 5;            // the result is 5 ...
    counter = counter + 1;     // ... but the function keeps running
}

func pending_then_stop(): int64 {
    unreturnable 7;
    int64 k = 0;
    while (k < 100) {
        k++;
        if (k == 3) {
            terminate;         // returns the pending 7
        }
    }
    return 1;
}

func explicit_wins(): int64 {
    unreturnable 7;
    return 9;                  // an explicit value wins
}

@Entry
pub func main(): int64 {
    printf("measure=%llu counter=%llu\n", measure(), counter);
    printf("pending_then_stop=%d explicit_wins=%d\n", pending_then_stop(), explicit_wins());
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
measure=5 counter=1
pending_then_stop=7 explicit_wins=9
```

16.2.1 How pending values are compiled

For every function that uses `unreturnable`, the compiler creates a private global `__ret_<function>`. `unreturnable e;` stores `e` into it, and `return;` / `terminate;` become `return __ret_<function>;`. Because the pending value is a global, a crack of the function can also set it, which lets a crack choose the value its function will return:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 16-4. A crack that chooses its function's result

ASH

```
general static uint64 value = 10

func greeting(): string {
    name -> (uint64 num, string text) {
        value = num
        unreturnable text
    };
    return;                     // returns whatever the crack left pending
}

@Entry
pub func main(): int64 {
    greeting.name = 7, "Hello, crack";
    printf("greeting()=%s value=%llu\n", greeting(), value);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
greeting()=Hello, crack value=7
```

NOTE

`unreturnable` may also be written `unreturn`. The form `unreturnable` *type*`;` (for example `unreturnable int64;`) only declares the type of the pending value and stores nothing.

WARNING

The pending value is stored in a global, so it persists between calls: a function that sets it in one call and ends with `return;` in a later call returns the old value. Set the pending value on every path that relies on it.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 17

ANNOTATIONS (ATTRIBUTES)

An **annotation** (also called an *attribute*) is a word that starts with the character `@` and gives the compiler extra information. You write it on its own line, just above the thing it talks about: a function, a class, a global variable, or the whole file. Annotations do not compute anything by themselves; they change *how* the compiler treats your code.

This chapter explains every annotation of ASH from the beginning. For each one you will find, in this order: what it does in plain words, how to write it, where you may write it, a complete program that uses it (compiled and run while this manual was built), what changes in the machine code, and the mistakes that beginners make most often.

17.1 ANNOTATIONS IN FIVE MINUTES

You have already used one annotation in every program: `@Entry`.

Example 17-1. The annotation you always use

ASH

```
@Entry
pub func main(): int64 {
    printf("This function runs first because of @Entry.\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
This function runs first because of @Entry.
```

Read the program line by line:

Table 17-1. Line by line

Line	Code	Meaning
1	`@Entry`	An annotation. It tells the compiler: *the function written below is where the program starts*.
2	`pub func main(): int64 {`	The function that the annotation talks about.
3	`printf(...)`	Prints a message.
4	`return 0;`	Ends the program successfully.

Four things are true of every annotation:

1 It begins with `@` followed by a name, with no space: `@Entry`, not `@ Entry`.

2 Some annotations take **arguments** in parentheses: `@bits(32)`. Some take a number in brackets: `@ovr[5]`. Some take nothing: `@naked`.

Copyright © 2026 Mallionic Inc. All rights reserved.

3 It applies to the **next** function, class or global variable written after it, or (for file annotations) to the whole file.

4 Several annotations can be stacked, one per line, above the same function.

Example 17-2. Several annotations on one function

ASH

```
@Greet() {
    printf("(about to run the function)\n");
}

@Entry
@Greet()
pub func main(): int64 {
    printf("Hello!\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
(about to run the function)
Hello!
```

17.2 THE THREE KINDS OF ANNOTATIONS

Table 17-2. Kinds of annotations

Kind	Annotations	Applies to
File annotations	`@bits`, `@originoffs`, `@beautycode`, `@bounds`, `@strict`, `@nolibc`	The whole file. Write them at the top.
Function annotations	`@Entry`, `@naked`, `@msabi` / `@efiapi`, `@section`, `@attribute`, `@ovr` / `@override`, your own annotations	The function (or class) written right after them. `@section` and `@attribute` also apply to global variables.
Statement annotation	`@emit(...)`	It is itself a statement inside a function body.

Copyright © 2026 Mallionic Inc. All rights reserved.

17.3 SUMMARY TABLE

Table 17-3. All built-in annotations

Annotation	Kind	Effect
`@Entry` / `@entry`	function	Marks the entry point of the program.
`@bits(16|32|64)`	file	Selects the processor mode of the whole file.
`@originoffs(addr)`	file	Load address of a flat image or `--exe` executable.
`@beautycode(true|false)`	file	Compact frames and bounds checks (true, the default) or full frames everywhere and no checks (false).
`@bounds(true|false)`, `@boundscheck(...)`	file	Turns array bounds checking on or off.
`@strict` / `@strict(true|false)`	file	Strict mode (Chapter 21).
`@nolibc` / `@nolibc(true|false)`	file	Same as `--no-libc`.
`@naked`	function	No prologue, epilogue or `ret`: only your `asm` and `emit`.
`@msabi`, `@efiapi`, `@EFIAPI`	function	Receives arguments with the Microsoft x64 convention.
`@section(".name"[, "flags"])`	function, global	Places the function or data in its own section.
`@attribute(section(".name"))`, `@__attribute__((section(...)))`	function, global	GCC-style spelling of `@section`.
`@ovr`, `@ovr[N]`, `@override`, `@override(N)`	function, class	Sets the override right (Section 19.4).
`@emit(b, ...)`	statement	Emits raw bytes (Section 20.3).
`@Name(args)` (your own)	function	Inserts the body of your annotation at the start of the function (Section 17.16).

The names `bits`, `originoffs`, `beautycode`, `bounds`, `nolibc`, `strict`, `ovr` and `override` may be written in any mix of upper and lower case. All other names must be written exactly as shown.

17.4 @ENTRY: WHERE THE PROGRAM STARTS

What it does. A program is made of many functions. The computer needs to know which one to run first. `@Entry` marks that function. Without it, the program has no starting point.

How to write it. `@Entry` (or `@entry`) alone, on the line before `func`. It takes no arguments.

Where. Before exactly one function of the whole program (main file plus imported files). The function should be `pub`, so that the operating system's start-up code can find it.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 17-3. @Entry with the conventional name main

ASH

```
func helper(): int64 {
    return 20;
}

@Entry
pub func main(): int64 {
    int64 a = helper();
    printf("helper gave %d\n", a);
    return a + 22;
}
```

OUTPUT — EXIT STATUS 42

```
helper gave 20
```

What changes in the machine code. In ASHIR the function is preceded by `[entry]`. In an object file the function is a global symbol. In a flat binary (`--compile`) the entry function is placed first, so that the very first byte of the file is its first instruction. When the entry function is called `_start`, the compiler also aligns the stack to 16 bytes at its beginning (`and rsp, -16`).

Example 17-4. [entry] in ASHIR

ASH

```
@Entry
pub func main(): int64 {
    return 7;
}
```

ASHIR

```
[entry]
func qword main() {
    ret qword 7
}
```

Choosing the name. Call the function `main` when your program uses the C library (`printf`, `malloc` ...): the C start-up code calls `main` for you. Call it `_start` when the program uses no C library; it must then end with `exit(...)` or `sys_exit(...)` (Section 3.5).

Common mistakes.

Example 17-5. Two entry points

ASH

```
@Entry
pub func main(): int64 {
    return 0;
}

@Entry
pub func other(): int64 {
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: ASHIR: there are multiple [entry] functions
1 error generated.
```

Example 17-6. @Entry written twice on the same function

ASH

```ash
@Entry
@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
<input>:3:1: error: the attribute @Entry appears twice
    3 | pub
      | ^^^
1 error generated.
```

17.5 @BITS: CHOOSING THE PROCESSOR MODE

What it does. x86 processors can run in three modes: 16-bit (the mode in which a PC starts), 32-bit and 64-bit. `@bits` tells the compiler which one your file is written for. It decides which instructions are generated, how big pointers are and how big the largest integer is.

How to write it. `@bits(16)`, `@bits(32)` or `@bits(64)`. Without it, the file is 64-bit.

Where. Once, at the top of the file. The command line option `--bits N` overrides it.

Table 17-4. What @bits changes

Value	Registers and pointers	Typical use and output
`64` (default)	64-bit (`rax`, 8-byte pointers)	Modern programs. `--elf` gives an ELF64 object, `--win` a COFF AMD64 object.
`32`	32-bit (`eax`, 4-byte pointers); `int64` is held in 32 bits	32-bit Linux programs (`gcc -m32`), protected-mode kernels. `--elf` gives ELF32, `--exe` a complete executable.
`16`	16-bit (`ax`, 2-byte addresses); every integer is held in 16 bits	Boot sectors, DOS programs, BIOS code. Only `--compile` (a flat binary).

Example 17-7. The same idea compiled for 32 bits

ASH

```ash
@bits(32)

@Entry
pub func main(): int32 {
    int32 total = 0;
    for (int32 i = 1; i <= 4; i++) {
        total = total + i * 10;
    }
    printf("total = %d\n", total);
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
total = 100
```

X86 DISASSEMBLY (32-BIT)

```
00000000 <main>:
   0:   55                      push   ebp
   1:   89 e5                   mov    ebp,esp
   3:   81 ec 20 00 00 00       sub    esp,0x20
   9:   b8 00 00 00 00          mov    eax,0x0
   e:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  11:   b8 01 00 00 00          mov    eax,0x1
  16:   89 45 f8                mov    DWORD PTR [ebp-0x8],eax
  19:   8b 45 f8                mov    eax,DWORD PTR [ebp-0x8]
  1c:   b9 04 00 00 00          mov    ecx,0x4
  21:   39 c8                   cmp    eax,ecx
  23:   0f 8f 31 00 00 00       jg     5a <main+0x5a>
  29:   8b 45 fc                mov    eax,DWORD PTR [ebp-0x4]
  2c:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  2f:   8b 45 f8                mov    eax,DWORD PTR [ebp-0x8]
  32:   89 45 f4                mov    DWORD PTR [ebp-0xc],eax
  35:   8b 45 f4                mov    eax,DWORD PTR [ebp-0xc]
  38:   b9 0a 00 00 00          mov    ecx,0xa
  3d:   0f af c1                imul   eax,ecx
  40:   89 45 f4                mov    DWORD PTR [ebp-0xc],eax
  43:   8b 45 fc                mov    eax,DWORD PTR [ebp-0x4]
  46:   8b 4d f4                mov    ecx,DWORD PTR [ebp-0xc]
  49:   01 c8                   add    eax,ecx
  4b:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  4e:   8b 45 f8                mov    eax,DWORD PTR [ebp-0x8]
  51:   40                      inc    eax
  52:   89 45 f8                mov    DWORD PTR [ebp-0x8],eax
  55:   e9 bf ff ff ff          jmp    19 <main+0x19>
  5a:   b8 8a 00 00 00          mov    eax,0x8a
  5f:   89 45 f0                mov    DWORD PTR [ebp-0x10],eax
  62:   8b 45 fc                mov    eax,DWORD PTR [ebp-0x4]
  65:   89 45 ec                mov    DWORD PTR [ebp-0x14],eax
  68:   8b 45 ec                mov    eax,DWORD PTR [ebp-0x14]
  6b:   50                      push   eax
  6c:   8b 45 f0                mov    eax,DWORD PTR [ebp-0x10]
  6f:   50                      push   eax
  70:   e8 fc ff ff ff          call   71 <main+0x71>
  75:   81 c4 08 00 00 00       add    esp,0x8
  7b:   b8 00 00 00 00          mov    eax,0x0
  80:   89 ec                   mov    esp,ebp
  82:   5d                      pop    ebp
  83:   c3                      ret
  84:   31 c0                   xor    eax,eax
  86:   89 ec                   mov    esp,ebp
  88:   5d                      pop    ebp
  89:   c3                      ret
```

In the disassembly you can see 32-bit registers (`eax`, `ecx`, `ebp`) and arguments pushed on the stack with `push` before `call printf`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 17-8. A 16-bit program (run in the 8086 emulator)

ASH

```
@bits(16)
@originoffs(0x100)

@Entry
pub func _start(): int16 {
    int16 a = 6;
    int16 b = 7;
    return a * b;           // AX = 42
}
```

OUTPUT — EXIT STATUS 0

```
ax = 42  bp = 0x0
```

X86 DISASSEMBLY (16-BIT)

```
 100:   55                      push   bp
 101:   89 e5                   mov    bp,sp
 103:   81 ec 10 00             sub    sp,0x10
 107:   b8 06 00                mov    ax,0x6
 10a:   89 46 fe                mov    WORD PTR [bp-0x2],ax
 10d:   b8 07 00                mov    ax,0x7
 110:   89 46 fc                mov    WORD PTR [bp-0x4],ax
 113:   8b 46 fe                mov    ax,WORD PTR [bp-0x2]
 116:   89 46 fa                mov    WORD PTR [bp-0x6],ax
 119:   8b 46 fa                mov    ax,WORD PTR [bp-0x6]
 11c:   8b 4e fc                mov    cx,WORD PTR [bp-0x4]
 11f:   0f af c1                imul   ax,cx
 122:   89 46 fa                mov    WORD PTR [bp-0x6],ax
 125:   8b 46 fa                mov    ax,WORD PTR [bp-0x6]
 128:   89 ec                   mov    sp,bp
 12a:   5d                      pop    bp
 12b:   c3                      ret
 12c:   31 c0                   xor    ax,ax
 12e:   89 ec                   mov    sp,bp
 130:   5d                      pop    bp
```

Common mistakes.

Example 17-9. @bits written twice

ASH

```
@bits(64)
@bits(32)
@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
<input>:2:6: error: @bits(...) may appear only once
    2 | (
      |      ^
1 error generated.
```

- Using `printf` in a 16-bit file: a flat binary has no C library, so the build fails. Use `asm` with BIOS calls instead.

Copyright © 2026 Mallionic Inc. All rights reserved.

- Expecting `int64` to hold big values in 16-bit or 32-bit mode: it is limited by the processor mode (Section 8.3).

17.6 @ORIGINOFFS: WHERE THE PROGRAM WILL BE LOADED

What it does. A flat binary is copied into memory at an address chosen by whoever loads it (the BIOS, DOS, a boot loader). The code contains absolute addresses (for example, the address of a text string), so the compiler must know that load address in advance. `@originoffs` tells it.

How to write it. `@originoffs(number)`, for example `@originoffs(0x7C00)`. The special words `offset` or `org` use the origin given by the build environment.

Where. Once, at the top of the file. It matters for `--compile` and `--exe`; it is ignored for `--elf` and `--win`, because there the linker chooses the addresses.

Table 17-5. Common origins

Value	Who loads the program there
`0x7C00`	The BIOS loads the first sector of a disk (a *boot sector*) here.
`0x100`	DOS loads `.COM` programs here.
`0x08048000`	The traditional address of 32-bit Linux executables, used with `--exe`.
`0x100000`	Many boot loaders load kernels at 1 MiB.

Example 17-10. @originoffs for a 32-bit Linux executable

ASH

```
@bits(32)
@originoffs(0x08048000)

@Entry
pub func _start(): int32 {
    exit(3);
    return 0;
}
```

HEX DUMP (XXD)

```
00000000: 7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00  .ELF............
00000010: 02 00 03 00 01 00 00 00 54 80 04 08 34 00 00 00  ........T...4...
00000020: 00 00 00 00 00 00 00 00 34 00 20 00 01 00 00 00  ........4. .....
00000030: 00 00 00 00 01 00 00 00 00 00 00 00 00 80 04 08  ................
... (6 more lines not shown)
```

OUTPUT — EXIT STATUS 3

```
(no output)
```

The entry address stored in the header (bytes 24 to 27 of the file: `54 80 04 08`, that is `0x08048054`) is the origin plus the 84 bytes of the header.

Common mistake. Writing a wrong origin produces a program whose strings and globals are read from the wrong addresses; nothing warns you at compile time. Always use the address at which your loader really places the image.

Copyright © 2026 Mallionic Inc. All rights reserved.

17.7 @BEAUTYCODE: CLEAN CODE OR FULL FRAMES

What it does. It chooses between two styles of generated code for the whole file.

- `@beautycode(true)` (the default): the code is as clean as possible. A small function that calls nothing and needs no memory gets no *stack frame* (no `push rbp` / `leave`), and every array access is checked against the array length.
- `@beautycode(false)`: every function gets a full stack frame, and array accesses are **not** checked. The code of each function looks the same, which is convenient with debuggers, and arrays are faster but unprotected.

How to write it. `@beautycode(true)` or `@beautycode(false)`. `false`, `0` and `off` mean off; any other word means on.

Where. Once, at the top of the file.

Table 17-6. Effect of @beautycode

Aspect	(default)	
Stack frames	Only where needed.	Always (like the `-e` option).
Saved registers	Only the ones actually used.	Only the ones actually used.
Array bounds checks	On (unless `@bounds(false)`).	Off (unless `@bounds(true)`).

Example 17-11. A small function with @beautycode(true): no frame

ASH

```
@beautycode(true)

func add(int64 a, int64 b): int64 {
    return a + b;
}

@Entry
pub func main(): int64 {
    return add(40, 2);
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <add>:
   0:   48 89 f8                mov    rax,rdi
   3:   49 89 c2                mov    r10,rax
   6:   4c 89 d0                mov    rax,r10
   9:   48 89 f1                mov    rcx,rsi
   c:   48 01 c8                add    rax,rcx
   f:   49 89 c2                mov    r10,rax
  12:   4c 89 d0                mov    rax,r10
  15:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 17-12. The same function with @beautycode(false): full frame

ASH

```
@beautycode(false)

func add(int64 a, int64 b): int64 {
    return a + b;
}

@Entry
pub func main(): int64 {
    return add(40, 2);
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <add>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   48 89 f8                mov    rax,rdi
   e:   49 89 c2                mov    r10,rax
  11:   4c 89 d0                mov    rax,r10
  14:   48 89 f1                mov    rcx,rsi
  17:   48 01 c8                add    rax,rcx
  1a:   49 89 c2                mov    r10,rax
  1d:   4c 89 d0                mov    rax,r10
  20:   c9                      leave
  21:   c3                      ret
```

Compare the two listings: the second one starts with `push rbp`, `mov rbp, rsp`, `sub rsp, ...` and ends with `leave`.

17.8 @BOUNDS: ARRAY CHECKING ON OR OFF

What it does. Every array in ASH knows its length. With checking on, each `v[i]` first verifies that `i` is smaller than the length; if it is not, the program stops with exit status 134 instead of reading or writing the wrong memory. `@bounds` turns this checking on or off, independently of `@beautycode`.

How to write it. `@bounds(true)` or `@bounds(false)` (also spelled `@boundscheck`).

Where. Once, at the top of the file. Once `@bounds` is written, `@beautycode` no longer changes the checking.

Example 17-13. With checking on, a wrong index stops the program

ASH

```
@bounds(true)

@Entry
pub func main(): int64 {
    int64[4] v;
    int64 i = 4;              // valid indexes are 0, 1, 2, 3
    printf("before\n");
    fflush(0);
    v[i] = 1;                 // stops here with exit status 134
    printf("never printed\n");
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 134

```
before
```

Example 17-14. With checking off, valid code runs without the checks

ASH

```
@bounds(false)

@Entry
pub func main(): int64 {
    int64[4] v;
    v[3] = 7;                  // no check is generated for this access
    printf("%d\n", v[3]);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
7
```

When to turn it off. Only in code that has been tested and must be as small or as fast as possible. A wrong index with checking off silently damages other data.

17.9 @STRICT: EXTRA SAFETY CHECKS

What it does. It turns on *strict mode*: the compiler also rejects numbers that do not fit their variable, silent loss of bits when a big value is stored in a small variable, and `return` statements that do not match the function. Chapter 21 lists every rule.

How to write it. `@strict`, `@strict(true)` or `@strict(false)`. The line `"use strict";` at the very top of a file does the same as `@strict`.

Example 17-15. Strict mode accepts correct code

ASH

```
@strict

@Entry
pub func main(): int64 {
    uint8 small = 200;
    uint64 big = 70000;
    uint16 medium = (uint16)big;       // the cast says: I really want to keep only 16 bits
    printf("%d %d\n", small, medium);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
200 4464
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 17-16. ...and rejects a silent loss of bits

ASH

```
@strict

@Entry
pub func main(): int64 {
    uint64 big = 70000;
    uint16 medium = big;
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: strict: cannot assign a 64-bit value to 'medium' (uint16, 16 bits); use an explicit c
↪ ast
1 error generated.
```

17.10 @NOLIBC: PROGRAMS WITHOUT THE C LIBRARY

What it does. Normally a program may use the C library: `printf`, `malloc`, files... `@nolibc` promises that this program will run without it, for example as a tiny static program or on a computer with no operating system. The compiler then provides its own memory allocation and `exit`, and the program is linked with `ld` alone.

How to write it. `@nolibc`, `@nolibc(true)` or `@nolibc(false)`. It is the same as the command line option `--no-libc`.

Where. Anywhere at top level; it is detected even before the `%if` directives are evaluated, so `%if ASH_NO_LIBC` sees it.

Example 17-17. A program that declares @nolibc

ASH

```
@nolibc
import <syscalls>;

@Entry
pub func _start(): int64 {
    print("2 + 3 = ");
    print_int(2 + 3);
    println("");
    sys_exit(0);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
2 + 3 = 5
```

Common mistakes. Calling `printf` (or any C function) in a `@nolibc` program: the linker reports "undefined reference". Use `print`, `println` and `print_int` from `<syscalls>`.

Copyright © 2026 Mallionic Inc. All rights reserved.

17.11 @NAKED: FUNCTIONS WITHOUT COMPILER CODE

What it does. Normally the compiler adds instructions at the beginning and end of each function (to make room for variables, save registers and return). A `@naked` function gets **none** of that: its machine code is exactly the instructions you write in `asm` blocks and `emit` statements, nothing more. You must write the final `ret` yourself.

How to write it. `@naked` on the line before `func`.

Rules inside a naked function.

- Only `asm { }`, `asm("...")` and `emit(...)` statements are allowed.
- No variables, no ASH function calls, no `return`. Assembly placeholders such as `{x}` are not available.
- The arguments are found where the calling convention puts them: on x86-64 in `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`; in 32-bit code on the stack at `[esp+4]`, `[esp+8]`...
- The result must be left in `rax` (`eax` in 32-bit code).

Example 17-18. Three naked functions

ASH

```
@naked
func add3(int64 a, int64 b, int64 c): int64 {
    asm {
        lea rax, [rdi + rsi]     ; rax = a + b
        add rax, rdx             ; rax = a + b + c
        ret
    }
}

@naked
func answer(): int64 {
    emit(0xB8, 0x2A, 0x00, 0x00, 0x00);    // mov eax, 42
    emit(0xC3);                            // ret
}

@naked
func twice(int64 x): int64 {
    asm volatile("lea rax, [rdi + rdi]; ret");
}

@Entry
pub func main(): int64 {
    printf("add3=%d answer=%d twice=%d\n", add3(1, 2, 3), answer(), twice(21));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
add3=6 answer=42 twice=42
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <add3>:
   0:   48 8d 04 37             lea    rax,[rdi+rsi*1]
   4:   48 01 d0                add    rax,rdx
   7:   c3                      ret

0000000000000008 <answer>:
   8:   b8 2a 00 00 00          mov    eax,0x2a
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
   d:   c3                      ret

000000000000000e <twice>:
   e:   48 8d 04 7d 00 00 00    lea    rax,[rdi*2+0x0]
  15:   00
  16:   c3                      ret
```

Look at the disassembly: `add3` is exactly three instructions, with no `push rbp`.

When to use it. Interrupt handlers (which end with `iretq`), the first instructions of a kernel or boot loader, and functions whose bytes must be exact.

17.12 @MSABI AND @EFIAPI: THE MICROSOFT CALLING CONVENTION

What it does. When one function calls another, both must agree where the arguments are placed. Linux uses one agreement (the *System V* convention: `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`); Windows and UEFI firmware use another (the *Microsoft x64* convention: `rcx`, `rdx`, `r8`, `r9`). `@msabi` (also written `@efiapi` or `@EFIAPI`) makes a function receive its arguments the Microsoft way.

Where. Before a function that will be called by Windows or by UEFI firmware: the entry point of a UEFI application, or a callback given to a Windows API. With `--win` all external calls already use the Microsoft convention.

Example 17-19. A function that receives Microsoft-convention arguments

ASH

```
@msabi
pub func from_windows(int64 a, int64 b): int64 {
    return a - b;
}

@Entry
pub func main(): int64 {
    int64 r = 0;
    int64 target = &from_windows;   // the address of the function
    asm {
        mov rax, target
        mov rcx, 50                 ; first argument, Microsoft style
        mov rdx, 8                  ; second argument
        sub rsp, 32                 ; shadow space required by the convention
        call rax
        add rsp, 32
        mov r, rax
    }
    printf("from_windows(50, 8) = %d\n", r);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
from_windows(50, 8) = 42
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000089 <from_windows>:
  89:   55                      push   rbp
  8a:   48 89 e5                mov    rbp,rsp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  8d:   48 81 ec 40 00 00 00    sub    rsp,0x40
  94:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  98:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  9c:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  a0:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  a4:   48 89 75 d8             mov    QWORD PTR [rbp-0x28],rsi
  a8:   48 89 7d d0             mov    QWORD PTR [rbp-0x30],rdi
  ac:   48 89 4d c8             mov    QWORD PTR [rbp-0x38],rcx
  b0:   48 89 55 c0             mov    QWORD PTR [rbp-0x40],rdx
  b4:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
  b8:   49 89 c2                mov    r10,rax
  bb:   4c 89 d0                mov    rax,r10
  be:   48 8b 4d c0             mov    rcx,QWORD PTR [rbp-0x40]
  c2:   48 29 c8                sub    rax,rcx
  c5:   49 89 c2                mov    r10,rax
  c8:   4c 89 d0                mov    rax,r10
  cb:   48 8b 75 d8             mov    rsi,QWORD PTR [rbp-0x28]
  cf:   48 8b 7d d0             mov    rdi,QWORD PTR [rbp-0x30]
  d3:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  d7:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  db:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  df:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  e3:   48 89 ec                mov    rsp,rbp
  e6:   5d                      pop    rbp
  e7:   c3                      ret
        ...
```

The disassembly of `from_windows` shows the arguments being read from `rcx` and `rdx`.

ASH

```
// The shape of a UEFI application entry point (build with --win, link as an EFI image)
@efiapi
pub func efi_main(uint64 image, uint64 systab): uint64 {
    return 0;          // EFI_SUCCESS
}
```

17.13 @SECTION AND @ATTRIBUTE: CHOOSING WHERE CODE AND DATA GO

What it does. An object file is divided into *sections*. Normally ASH puts all code and data into one section called `.ashtext`. `@section(".name")` puts the next function or global variable into a section of its own. A *linker script* can then place that section at a precise address, which kernels and firmware need (for example a multiboot header that must be in the first 8 KiB of a kernel image).

How to write it.

```
@section(".name")
@section(".name", "flags")
@attribute(section(".name"))            // the same, in GCC style
@__attribute__((section(".name")))      // the same, exactly as GCC writes it
```

The flags say what the section is allowed to do: `a` (allocated in memory), `w` (writable), `x` (executable). Without flags a function's section is allocated and executable and a variable's section is allocated.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 17-20. Functions and data in their own sections

SECTIONS.ASH

```
@section(".fast_text")
func hot(int64 x): int64 {
    return x * 2;
}

@attribute(section(".more_text"))
func warm(int64 x): int64 {
    return x * 3;
}

@section(".tables", "a")
general uint32[] lut = {0xCAFEBABE, 7, 8}

@section(".mydata", "aw")
general static int64 counter = 5

@Entry
pub func main(): int64 {
    counter = counter + 1;
    printf("hot=%d warm=%d lut[0]=%x counter=%d\n", hot(21), warm(5), lut[0], counter);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hot=42 warm=15 lut[0]=cafebabe counter=6
```

The tool `readelf` shows the new sections and their flags (`A` allocated, `W` writable, `X` executable):

SHELL SESSION

```
$ ashc sections.ash --elf -s && readelf -S -W out/sections.o | grep -E "Name|fast|more|table
↪ s|mydata"
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
  [ 4] .fast_text        PROGBITS        0000000000000000 0008c0 000043 00  AX  0   0 16
  [ 5] .more_text        PROGBITS        0000000000000000 000910 000046 00  AX  0   0 16
  [ 6] .tables           PROGBITS        0000000000000000 000960 000020 00   A  0   0 16
  [ 7] .mydata           PROGBITS        0000000000000000 000980 000010 00  WA  0   0 16
```

Common mistake. `@attribute` supports only `section`; other GCC attributes such as `aligned` or `noinline` are rejected.

17.14 @OVR AND @OVERRIDE: WHICH DEFINITION WINS

What it does. When a program and its libraries define two functions with the same name, ASH keeps the one with the higher *override right*, a number. Normal functions have right 0 and library functions -1, so your own function wins over a library function automatically. `@ovr[N]` gives the next function (or class) the right *N*.

How to write it. `@ovr` (right 1), `@ovr[N]`, `@ovr(N)`, `@override`, `@override[N]` or `@override(N)`. *N* may be negative.

Copyright © 2026 Mallionic Inc. All rights reserved.

LIB_QUIET.ASH

```
pub func greet(): void {
    printf("hello from the library\n");
}
```

Example 17-21. @ovr chooses between two definitions

MAIN_OVR.ASH

```
import "lib_quiet.ash";

@ovr[10]
pub func greet(): void {
    printf("hello from the main file (right 10)\n");
}

@Entry
pub func main(): int64 {
    greet();
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hello from the main file (right 10)
```

Section 19.4 explains every rule, including ties and the assignment form `greet.override = 10;`.

17.15 @EMIT: RAW BYTES AS A STATEMENT

`@emit(b1, b2, ...)` is written inside a function, like a statement. It puts the listed bytes into the machine code at that point, exactly like `emit(...)`. It is useful for instructions that you know by their bytes.

Example 17-22. @emit inside a function

ASH

```
@Entry
pub func main(): int64 {
    int64 r = 0;
    @emit(0x90, 0x90);                    // two nop instructions
    @emit(0xB8, 0x05, 0x00, 0x00, 0x00);  // mov eax, 5
    asm volatile("mov {r}, rax");
    printf("r=%d\n", r);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
r=5
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <main>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
1b:   b8 00 00 00 00          mov    eax,0x0
20:   49 89 c4                mov    r12,rax
23:   90                      nop
24:   90                      nop
25:   b8 05 00 00 00          mov    eax,0x5
2a:   49 89 c4                mov    r12,rax
2d:   b8 00 00 00 00          mov    eax,0x0
32:   49 89 c5                mov    r13,rax
35:   4c 8d 2d 34 00 00 00    lea    r13,[rip+0x34]        # 70 <__str0>
3c:   4c 89 e0                mov    rax,r12
3f:   49 89 c6                mov    r14,rax
42:   4c 89 ef                mov    rdi,r13
45:   4c 89 f6                mov    rsi,r14
48:   b8 00 00 00 00          mov    eax,0x0
4d:   e8 00 00 00 00          call   52 <main+0x52>
52:   48 31 c0                xor    rax,rax
55:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
59:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
5d:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
61:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
65:   48 89 ec                mov    rsp,rbp
68:   5d                      pop    rbp
69:   c3                      ret
6a:   00 00                   add    BYTE PTR [rax],al
6c:   00 00                   add    BYTE PTR [rax],al
      ...
```

17.16 YOUR OWN ANNOTATIONS

What it does. You can invent annotations. An annotation that you define contains a block of code. Every function marked with your annotation gets that code inserted at its beginning. This is a simple way to add the same logging, counting or checking to many functions without copying the code.

How to define one.

```
@Name(parameters) {
    statements
}
```

The parentheses are required in the definition, even when there are no parameters: `@Log() { ... }`. Parameters have a type and a name, like function parameters. If you write only a type, `@Name(uint32)`, the parameter is called `value`.

How to use one. Write `@Name(arguments)` before a function. Without parameters you may write `@Name()` or just `@Name`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 17-23. Defining and using annotations

ASH

```
@Log() {
    printf("-> entering a function\n");
}

@Trace(uint32 level) {
    printf("   trace level %d\n", level);
}

@Count(uint32) {
    printf("   count=%d\n", value);
}

@Log
@Trace(3)
@Count(9)
func work(int64 x): int64 {
    return x + 1;
}

@Entry
@Trace(7)
pub func main(): int64 {
    return work(41);
}
```

OUTPUT — EXIT STATUS 42

```
   trace level 7
-> entering a function
   trace level 3
   count=9
```

ASHIR

```
func qword work(qword x) {
.type level dword
.type value dword
    %__tmp0 addr[__str0]
    call[printf, __tmp0]
    %level mov[3]
    %__tmp1 addr[__str1]
    %__tmp2 mov[level]
    call[printf, __tmp1, __tmp2]
    %value mov[9]
    %__tmp3 addr[__str2]
    %__tmp4 mov[value]
    call[printf, __tmp3, __tmp4]
    %__ret mov[x]
    %__ret add[1]
    ret qword __ret
}
```

The ASHIR of `work` shows what happened: the parameters `level` and `value` became local variables with the values 3 and 9, and the `printf` calls of the three annotations were placed before the body of the function, in the order in which the annotations were written.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 17-24. Annotations with text arguments, used twice

ASH

```
@Tag(string label) {
    printf("[%s] ", label);
}

@Tag("math")
@Tag("pure")
func square(int64 x): int64 {
    return x * x;
}

@Entry
pub func main(): int64 {
    printf("%d\n", square(12));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
[math] [pure] 144
```

How it works. User annotations are expanded on the tokens of the program before parsing (Section 6.2.4). The inserted code is compiled exactly as if you had typed it at the beginning of the function, so it can use the function's parameters and global variables.

Common mistakes.

Example 17-25. A definition without parentheses

ASH

```
@Log {
    printf("called\n");
}

@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
<input>:1:6: error: unknown attribute @Log
    1 | {
      |      ^
1 error generated.
```

- Using an annotation that was never defined: the compiler reports "unknown attribute".
- Annotations defined in an imported file can be used in the file that imports it. If two files define the same annotation, override rights decide which one is used (`@Name.override = N;`).

17.17 PLACEMENT RULES

- File annotations (`@bits`, `@originoffs`, `@beautycode`, `@bounds`, `@strict`, `@nolibc`) may appear anywhere at top level, but write them at the top of the file so that readers see them first.
- Function annotations go immediately before `func`, and before `pub` or `priv`: write `@naked pub func ...`, not `pub @naked func ...`.

Copyright © 2026 Mallionic Inc. All rights reserved.

- `@section` and `@attribute` may precede a function or a global variable.
- `@emit` appears inside a function body.
- Several annotations may be stacked; their order matters only for user annotations, whose code is inserted in the order written.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 18

COMPILE-TIME DIRECTIVES AND MACROS

ASH has two independent compile-time mechanisms. **Directives** are lines that start with `%`; they are evaluated on the raw text before the lexer runs and decide which lines of the file are compiled. **Macros** (`define`, `macro`) and **aliases** (`typedef`, `redefine`) work on tokens after the lexer and replace names by other tokens. This chapter describes both, and how they differ.

Table 18-1. Directives versus macros

Aspect	directives	/
When	Before the lexer, on text lines	After the lexer, on tokens
Purpose	Conditional compilation, configuration, build errors	Named constants, code snippets
Visible in code	No: `%define X 1` only exists inside directives	Yes: every use of the name is replaced
Values	Evaluated as integers or strings	Arbitrary token sequences
Scope	The file being compiled	The file and every file that imports it

18.1 DIRECTIVE SYNTAX

A directive is a line whose first non-blank character is `%`. The rest of the line is the directive name and its arguments; a `//` comment may follow. Directive lines are replaced by empty lines in the text passed to the lexer, so line numbers in later diagnostics remain correct.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 18-2. Directives

Directive	Meaning
`%define NAME [value]`	Defines a directive symbol. Without a value it is 1. A value made of numbers, symbols and operators is evaluated immediately; any other value is kept as text. A symbol can be redefined at any time.
`%undef NAME`	Removes a symbol.
`%if expr`	Starts a conditional block, kept if *expr* is true (not empty and not 0).
`%ifdef NAME` / `%ifndef NAME`	Kept if the symbol is (is not) defined.
`%elif expr`, `%elifdef NAME`, `%elifndef NAME`	Alternative branches.
`%else`	The branch kept when no previous branch was.
`%endif`	Ends the conditional block.
`%error text`	Stops the compilation with *text* as the error message (only in an active branch).
`%warning text`	Prints a warning and continues.
`%tip text`	Prints a tip and continues.
`%ash_start` ... `%ash_end`	A configuration block that may contain only directives and comments.

Conditional blocks can be nested to any depth. Every `%if` needs its `%endif`; an `%elif` after `%else` and a second `%else` are errors.

18.2 PREDEFINED SYMBOLS

Before the first directive is evaluated, the compiler defines symbols that describe the target of the current build:

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 18-3. Predefined directive symbols

Symbol	Values	Meaning
`ASH_BITS`	16, 32, 64	Processor mode (`@bits` or `--bits`).
`ASH_ARCH`	`i8086`, `i386`, `x86_64`	Architecture name.
`ASH_PTR_SIZE`	2, 4, 8	Size of a pointer in bytes.
`ASH_FORMAT`	`elf`, `coff`, `bin`, `exe`, `ashir`	Output format of this build.
`ASH_TARGET`	`elf`, `windows`, `raw`, `exe`, `ashir`	The output format in one word.
`ASH_OS`	`linux`, `windows`, `none`	Operating system implied by the format.
`ASH_NO_LIBC` / `ASH_LIBC`	1 / 0	Whether the C library is excluded (`--no-libc`, `@nolibc`).
`ASH_VERSION`	`1.0.0`	Version of the compiler.
`ASH_ENDIAN`	`little`	Byte order.
`ASH_X86_64`, `ASH_I386`, `ASH_I8086`	1	Defined only for the corresponding mode.
`ASH_ELF`, `ASH_COFF`, `ASH_BIN`, `ASH_EXE`	1	Defined only for the corresponding format.
`ASH_OBJECT`	1	Defined for relocatable objects (`elf`, `coff`).
`ASH_BAREMETAL`, `ASH_RAW`	1	Defined for flat binaries.
`ASH_LINUX`, `ASH_WINDOWS`	1	Defined for Linux (`elf`, `exe`) and Windows (`coff`) builds.

18.3 DIRECTIVE EXPRESSIONS

Expressions in `%if`, `%elif` and `%define` may contain:

- decimal, hexadecimal (`0x`) and binary (`0b`) numbers, and strings in double quotes;
- symbol names; a name that is not defined evaluates to **its own text**, which is what makes comparisons such as `ASH_FORMAT == elf` work without quotes;
- `defined(NAME)`;
- the operators `!`, `**` (right-associative), `* / %`, `+ -`, `<< >>`, `== != < <= > >=`, `&`, `^`, `|`, `&&`, `||`, and parentheses, with the usual precedence.

Comparisons are numeric when both sides are numbers and textual otherwise. Arithmetic is performed on 64-bit integers.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 18-1. Conditional compilation by target

ASH

```
%ash_start
%define MODE release
%define LEVEL 3
%ash_end

%define BASE 2
%define BITS (BASE ** 5) - 1                // 31
%define MASK (1 << 4) | 3                   // 19

%if ASH_BITS == 64 && ASH_FORMAT == elf
%define TARGET_OK 1
%endif

%ifndef TARGET_OK
%error "this example needs a 64-bit ELF build"
%endif

@Entry
pub func main(): int64 {
    int64 r = 0;
%if ASH_BITS == 64
    r += 64;
%elif ASH_BITS == 32
    r += 32;
%else
    r += 16;
%endif
%if MODE == release && LEVEL >= 3
    r += 10;
%endif
%if BITS == 31 && (MASK & 16) != 0
    r += 1;
%endif
%if ASH_NO_LIBC
    printf("no libc\n");
%else
    printf("with libc, r=%d\n", r);
%endif
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
with libc, r=75
```

The same file compiled for different targets keeps different lines. The following session compiles a small file in three ways and shows which branch reached the ASHIR:

TARGET.ASH

```
@Entry
pub func main(): int64 {
%if ASH_FORMAT == elf
    return 1;
%elif ASH_FORMAT == coff
    return 2;
%else
    return 3;
%endif
```

Copyright © 2026 Mallionic Inc. All rights reserved.

TARGET.ASH (CONTINUED)

```
}
```

SHELL SESSION

```
$ ashc target.ash --stdout -s | grep ret
    ret qword 3
$ ashc target.ash --elf --stdout -s | grep ret
    ret qword 1
$ ashc target.ash --win --stdout -s | grep ret
    ret qword 2
```

Example 18-2. %error stops the build

ASH

```
%if ASH_BITS != 32
%error "this program requires a 32-bit target"
%endif

@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: line 2: this program requires a 32-bit target
1 error generated.
```

18.4 THE DEFINE MACRO

`define NAME tokens;` defines a token macro: from then on every occurrence of the identifier `NAME` in the file (and in files that import it) is replaced by the tokens. The terminating `;` may be omitted when the definition is the only thing on its line. Macros may refer to other macros, in any order of definition; expansion is repeated up to a depth of 128.

Example 18-3. define constants

ASH

```
define WIDTH 80;
define HEIGHT 25;
define CELLS WIDTH * HEIGHT;
define GREETING "hello";
define VERBOSE 1

@Entry
pub func main(): int64 {
    printf("%s, %d cells\n", GREETING, CELLS);
    if (VERBOSE == 1) {
        printf("width=%d height=%d\n", WIDTH, HEIGHT);
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hello, 2000 cells
width=80 height=25
```

Copyright © 2026 Mallionic Inc. All rights reserved.

IMPORTANT

A `define` replaces tokens, not values: `define CELLS WIDTH * HEIGHT;` followed by `CELLS / 2` gives `80 * 25 / 2`, which is evaluated left to right. Put parentheses in the definition when the macro is an expression: `define CELLS (WIDTH * HEIGHT);`.

The libraries `<stdbool>`, `<null>`, `<math>` and `<syscalls>` use `define` to provide constants such as `TRUE`, `NULL`, `FX_ONE` and `SYS_WRITE`. Defining a name twice in the same file is an error.

18.5 BLOCK MACROS

`macro NAME { statements }` defines a macro whose body is a block of statements. Writing `NAME` where a statement is expected inserts the statements. Block macros have no parameters; use a function, or a user annotation (Section 17.16), when the code needs arguments.

Example 18-4. A block macro

ASH

```
macro SAY_HELLO {
    printf("hello from a macro\n");
    count = count + 1;
}

@Entry
pub func main(): int64 {
    int64 count = 0;
    SAY_HELLO
    SAY_HELLO
    printf("count=%d\n", count);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hello from a macro
hello from a macro
count=2
```

18.6 TYPEDEF

`typedef` *type Name*`;` is also a token-level substitution: every later `Name` is replaced by the type. Section 8.11 describes it.

18.7 REDEFINE

`redefine` has two forms. The **alias form** gives a new name to an existing function (an ASH function, an `extern` or a C function):

```
redefine ret-type old-name(param-types) ret-type new-name(param-types)
redefine global ret-type old-name(...) ret-type new-name(...)
```

After it, calls to the new name (and `&new-name`) are compiled as calls to the old one. Without `global` the alias is valid in the current file; with `global` it is valid in every ASH file compiled after it in the same program.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 18-5. An alias for printf

ASH

```
redefine uint64 printf(char[], ...) uint64 out(char[]...)

@Entry
pub func main(): int64 {
    out("printed through an alias: %d\n", 42);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
printed through an alias: 42
```

The **definition form** defines a function with a body under a name that must be exported, replacing any other function of the same name:

```
redefine global ret-type name(parameters) { body }
```

The function becomes a public symbol with override right 100 (Section 19.4), which makes this the simplest way to export an implementation to C, for example a wrapper around a class method. Exposing a `priv` method this way is an error.

ASH

```
class Calculator {
    pub func add(uint64 a, uint64 b): uint64 { return a + b; }
}

general static uint64 calls = 0

redefine global uint64 compute(uint64 a, uint64 b) {
    calls = calls + 1;
    return Calculator.add(a, b);
}
```

C code can then call `compute` with the prototype `unsigned long compute(unsigned long, unsigned long);`.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 19

MODULES, IMPORTS AND OVERRIDE RIGHTS

A program can be split into several ASH files. One file, the *main file*, is given to the compiler; the others are brought in with `import`. All files are merged into a single program and compiled into one object file. This chapter describes the import forms, how files and libraries are found, what an import brings into a program, and the *override rights* that decide which definition wins when several files define the same name.

19.1 THE IMPORT STATEMENT

```
import <name>;            // a system library from nlib
import "path/file.ash";   // a file of your project
import "name";            // name.ash in the project
```

Imports are written at top level, usually at the beginning of the file. An import may itself contain imports; each file is merged only once, however many times it is imported. Import statements that appear inside comments (for example in usage examples at the top of a library) are ignored.

19.2 WHERE FILES ARE FOUND

19.2.1 System libraries: import <name>

For `import <name>;` the compiler looks for `name.ash` in the following places and uses the first one found:

1. a directory named `nlib` next to the main source file;
2. a directory named `nlib` in the parent directory of the main source file;
3. `nlib` and `../nlib` relative to the current directory;
4. the copy of `nlib` embedded in the compiler itself.

A library on disk therefore always takes precedence over the embedded one, which lets a project extend or patch a system library without rebuilding the compiler. `ashc --list-nlib` lists the embedded libraries; Chapter 27 documents them.

19.2.2 Project files: import "file.ash"

For `import "file.ash";` the path is tried relative to the directory of the main source file, then relative to its parent directory, then relative to the current directory. When the import appears inside an imported file, the directory of *that* file is tried first, as with `#include "..."` in C. `import "name";` without extension looks for `name.ash`.

Copyright © 2026 Mallionic Inc. All rights reserved.

19.3 WHAT AN IMPORT BRINGS IN

Everything declared at top level in the imported file becomes part of the program:

Table 19-1. Effect of importing a file

Item	Effect
Functions and classes	Callable from the importing program (except `priv` ones, Section 15.5).
Global variables	Accessible by name; `general` globals stay public symbols.
`extern` declarations	Available to the program.
`define` and `macro`	Visible in the importing file.
`typedef` and structures	Visible in the importing file.
User annotations	Usable in the importing file.
`@Entry`	Ignored unless it is the program's only entry point; the main file's entry wins.

19.3.1 Tree shaking

After merging, the compiler removes every function that came from an imported file and is not reached, directly or indirectly, from a function of the main file. Functions of the main file are always kept because C code may call them. Importing a large library therefore costs only the functions you use, and an imported function that refers to a C symbol you never use does not create an unresolved reference.

GEOMETRY.ASH

```
// geometry.ash - a small library
pub func square_area(int64 side): int64 {
    return side * side;
}

pub func rect_area(int64 w, int64 h): int64 {
    return w * h;
}

priv func helper(): int64 {
    return 1;
}

define UNIT 10;
```

Example 19-1. Importing a project file

MAIN.ASH

```
import "geometry.ash";

@Entry
pub func main(): int64 {
    printf("%d\n", square_area(UNIT));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
100
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR

```
[entry]
func qword main() {
    %__tmp0 addr[__str0]
    %__tmp2 mov[10]
    %__tmp1 call[square_area, __tmp2]
    call[printf, __tmp0, __tmp1]
    ret qword 0
}

func qword square_area(qword side) {
    %__ret mov[side]
    %__ret imul[side]
    ret qword __ret
}

.string __str0 "%d\n"
```

The ASHIR contains `main` and `square_area` only: `rect_area` and `helper` were removed because nothing in the main file uses them.

19.4 OVERRIDE RIGHTS

When the merged program contains several functions (or classes, or annotations) with the same name, ASH does not report a duplicate immediately. Each definition has an **override right**, a signed integer, and the definition with the **highest right** is kept. The idea is similar to the `z-index` of CSS.

Table 19-2. Default and explicit override rights

Definition	Right
A function or class without annotation	0
Functions and classes of `nlib` libraries	-1
`@ovr` / `@override` (no value)	1
`@ovr[N]`, `@ovr(N)`, `@override[N]`, `@override(N)`	N (any signed 64-bit value)
`redefine` with a body	1
`redefine global` with a body	100
`name.override = N;` at top level	N, for the function `name`
`Class.override = N;` / `Class.method.override = N;`	N for the class, or for one method (the method's value wins over the class's)
`@Annotation.override = N;`	N for a user-defined annotation

Ties are resolved as follows:

- between the main file and an imported file: the **main file wins**;
- between two imported files: **error** — the compiler cannot know which one you meant;
- the function marked `@Entry` is never replaced.

The predefined constants `true` and `false` also have right -1, so a program may define its own.

Copyright © 2026 Mallionic Inc. All rights reserved.

19.4.1 Your function beats the library

Because library functions have right -1 and your functions right 0, a function you write with the name of a library function replaces it, and every caller (including other library functions) uses yours.

Example 19-2. A program function replaces the nlib version

ASH

```
import <delay>;

pub func delay(int64 ms): void {
    printf("my own delay(%d) instead of sleeping\n", ms);
}

@Entry
pub func main(): int64 {
    delay(7);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
my own delay(7) instead of sleeping
```

19.4.2 Choosing between imported files

LIB_A.ASH

```
pub func greet(): void {
    printf("greet from A\n");
}
```

LIB_B.ASH

```
@ovr[5]
pub func greet(): void {
    printf("greet from B (right 5)\n");
}
```

LIB_C.ASH

```
pub func greet(): void {
    printf("greet from C (right 0)\n");
}
```

Example 19-3. The higher right wins

MAIN_AB.ASH

```
import "lib_a.ash";
import "lib_b.ash";

@Entry
pub func main(): int64 {
    greet();
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
greet from B (right 5)
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 19-4. Two imports with the same right are an error

MAIN_AC.ASH

```
import "lib_a.ash";
import "lib_c.ash";

@Entry
pub func main(): int64 {
    greet();
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: 'greet' is defined in lib_a.ash and in lib_c.ash with the same rights (override=0): t
↪ he compiler does not know which one to use. Use @ovr[N] or greet.override = N;
1 error generated.
```

19.4.3 Raising a right from the main file

An assignment `name.override = N;` at top level gives the function `name` the right *N*. Written in the main file, it lets the program select or replace a definition without editing the libraries:

Example 19-5. Raising the right of the program's own function

MAIN_RAISE.ASH

```
import "lib_b.ash";

pub func greet(): void {
    printf("greet from the main file\n");
}
greet.override = 1000;

@Entry
pub func main(): int64 {
    greet();
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
greet from the main file
```

TIP

Use override rights deliberately: they are a powerful way to provide platform-specific implementations (a library defines a portable function with right 0, a platform file overrides it with `@ovr[10]`) and test doubles (a test file overrides a function that talks to hardware). Keep the rights in one place so that readers can see which definition wins.

19.5 MODULES AND C OBJECTS

Each `ashc` invocation produces one object file from the main file and everything it imports. To build a program from several independent ASH modules, compile each module separately and link the objects together, declaring the functions of one module as `extern` in the other, exactly as with C. ASH objects and C objects can be freely mixed in one link.

Copyright © 2026 Mallionic Inc. All rights reserved.

MATHLIB.ASH

```
pub func cube(int64 x): int64 {
    return x * x * x;
}
```

APP.ASH

```
extern int64 cube(int64 x);

@Entry
pub func main(): int64 {
    printf("cube(7)=%d\n", cube(7));
    return 0;
}
```

SHELL SESSION

```
$ ashc mathlib.ash --elf -s
$ ashc app.ash --elf -s
$ gcc out/app.o out/mathlib.o -o app
$ ./app
cube(7)=343
```

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 20

INLINE ASSEMBLY AND MACHINE CODE

ASH lets you leave the language at any point and write processor instructions directly. Four mechanisms are available, from the most comfortable to the most raw: `asm { }` blocks, GCC-style `asm("...")` statements, `emit` statements that place literal bytes in the code, and `runnable` arrays that hold executable code as data. `@naked` functions combine them to produce functions without any compiler-generated code. This chapter describes each mechanism and the rules for mixing assembly with compiled code.

20.1 ASM BLOCKS

```
asm {
    instruction
    ...
}
volatile asm { ... }
```

The body of an `asm` block is assembly in Intel syntax, one instruction per line, exactly as accepted by the `shasm` assembler (Chapter 26): mnemonics, registers, memory operands with size keywords (`qword [rax+8]`), immediate values, labels, comments starting with `;`, and assembler directives such as `db`. The block is extracted from the source before the lexer runs (Section 6.2.1), so ASH lexical rules do not apply inside it.

`volatile asm` marks the block as having side effects; the compiler never removes or moves an `asm` block in ASH 1.0, so both forms behave the same, and `volatile` documents intent.

20.1.1 Using ASH variables

Inside an `asm` block, the name of an ASH local variable or parameter can be used wherever an operand is allowed. Before the block is assembled, the code generator replaces each name with the location that holds the variable at that point of the function: a register such as `r12`, or a memory operand such as `qword [rbp-0x18]`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 20-1. An asm block that reads and writes variables

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 40;
    int64 doubled = 0;
    asm {
        mov rax, x          ; x is replaced by its register or stack slot
        add rax, 2
        shl rax, 1
        mov doubled, rax
    }
    printf("x=%d doubled=%d\n", x, doubled);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
x=40 doubled=84
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <main>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 28 00 00 00          mov    eax,0x28
  20:   49 89 c4                mov    r12,rax
  23:   b8 00 00 00 00          mov    eax,0x0
  28:   49 89 c5                mov    r13,rax
  2b:   4c 89 e0                mov    rax,r12
  2e:   48 83 c0 02             add    rax,0x2
  32:   48 d1 e0                shl    rax,1
  35:   49 89 c5                mov    r13,rax
  38:   b8 00 00 00 00          mov    eax,0x0
  3d:   49 89 c6                mov    r14,rax
  40:   4c 8d 35 39 00 00 00    lea    r14,[rip+0x39]        # 80 <__str0>
  47:   4c 89 e0                mov    rax,r12
  4a:   49 89 c7                mov    r15,rax
  4d:   4c 89 e8                mov    rax,r13
  50:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
  54:   4c 89 f7                mov    rdi,r14
  57:   4c 89 fe                mov    rsi,r15
  5a:   48 8b 55 d8             mov    rdx,QWORD PTR [rbp-0x28]
  5e:   b8 00 00 00 00          mov    eax,0x0
  63:   e8 00 00 00 00          call   68 <main+0x68>
  68:   48 31 c0                xor    rax,rax
  6b:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  6f:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  73:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  77:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  7b:   48 89 ec                mov    rsp,rbp
  7e:   5d                      pop    rbp
  7f:   c3                      ret
```

Names of registers (`rax`, `eax`, `al`, `r8d`, `xmm0`, `cr0` ...) always denote registers inside an `asm` block, never variables. Choose variable names that do not collide with register names.

Copyright © 2026 Mallionic Inc. All rights reserved.

20.1.2 Labels and control flow

Labels can be used for loops and branches inside a block. Labels starting with a dot (`.loop`) are local to the block. Each block is assembled separately, so a jump cannot target a label in another block or in compiled code.

Example 20-2. A loop written in assembly

ASH

```
func popcount(uint64 v): int64 {
    int64 n = 0;
    asm {
        mov rax, v
        xor rcx, rcx
    .loop:
        test rax, rax
        jz .done
        mov rdx, rax
        and rdx, 1
        add rcx, rdx
        shr rax, 1
        jmp .loop
    .done:
        mov n, rcx
    }
    return n;
}

@Entry
pub func main(): int64 {
    printf("popcount(0xFF00FF)=%d popcount(7)=%d\n", popcount(0xFF00FF), popcount(7));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
popcount(0xFF00FF)=16 popcount(7)=3
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <popcount>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  1f:   b8 00 00 00 00          mov    eax,0x0
  24:   49 89 c4                mov    r12,rax
  27:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  2b:   48 31 c9                xor    rcx,rcx
  2e:   48 85 c0                test   rax,rax
  31:   74 0f                   je     42 <popcount+0x42>
  33:   48 89 c2                mov    rdx,rax
  36:   48 83 e2 01             and    rdx,0x1
  3a:   48 01 d1                add    rcx,rdx
  3d:   48 d1 e8                shr    rax,1
  40:   eb ec                   jmp    2e <popcount+0x2e>
  42:   49 89 cc                mov    r12,rcx
  45:   4c 89 e0                mov    rax,r12
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  48:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  4c:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  50:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  54:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  58:   48 89 ec                mov    rsp,rbp
  5b:   5d                      pop    rbp
  5c:   c3                      ret
```

20.1.3 Registers: what an asm block may change

A function that contains assembly is compiled conservatively: it receives a full stack frame, saves all of `r12`–`r15`, keeps its parameters in stack slots and its locals in `r12`–`r15` or stack slots. Within that model the following rules apply on x86-64:

Table 20-1. Register rules for asm blocks (x86-64)

Registers	Rule
`rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`–`r11`	Free scratch registers. Their values are not preserved across the block and the compiler does not expect them to survive.
`r12`–`r15`	May hold ASH variables. Change them only through the variable names, or save and restore them yourself.
`rbx`	Callee-saved in the System V ABI, and not saved by ASH: if you modify it, restore it before the end of the block.
`rbp`, `rsp`	Frame and stack pointers: do not modify them, except for balanced `push` / `pop` sequences.
Flags	May be changed freely.

On the 32-bit target all variables live in the frame (`[ebp-n]`), and `eax`, `ecx` and `edx` are free scratch registers; preserve `ebx`, `esi`, `edi`, `ebp` and `esp`.

20.2 THE ASM("...") FORM

For compatibility with code written for GCC and Clang, ASH accepts the statement form:

```
asm [volatile] ( "template" ["template"...] [ : outputs [ : inputs [ : clobbers ]]] );
```

The template is one or more string literals (concatenated); instructions are separated by `;` or `\n`. Inside the template:

Table 20-2. Placeholders in asm templates

Placeholder	Replaced by
`{name}`	The location of the ASH variable `name`.
`{name[i]}`	The element `i` of the array `name` (memory operand).
`%0`, `%1`, ...	The operands listed after the colons, numbered from 0: outputs first, then inputs.
`%[name]`	The operand declared with `[name] "constraint"(variable)`.
`%%`	A literal `%`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Constraint strings (`"=r"`, `"+r"`, `"r"`, `"=m"` ...) and the clobber list are accepted for compatibility but do not change code generation: every operand is simply the location of the named variable. `asm volatile("");` is accepted as a compiler barrier and generates nothing.

Example 20-3. GCC-style asm statements

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 40;
    int64 y = 5;
    int64 r = 0;
    int64[] v = {1, 2, 3};
    asm volatile("");                                          // barrier only
    asm volatile("add {x}, 2");                                // x = 42
    asm("mov rax, %0\n" "add rax, %1\n" "mov %0, rax" : "+r"(x) : "r"(y));   // x = 47
    asm volatile("mov rax, {v[2]}; add rax, %[a]; mov %[d], rax" : [d] "=m"(r) : [a] "r"(x))
↪ ;
    uint64 lo = 0;
    uint64 hi = 0;
    asm volatile("rdtsc; mov {lo}, rax; mov {hi}, rdx");      // time stamp counter
    int64 ticking = 0;
    if (hi + lo != 0) { ticking = 1; }
    printf("x=%d r=%d ticking=%d\n", x, r, ticking);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
x=47 r=50 ticking=1
```

The library `<syscalls>` is written with this form: its `syscall6` function loads six variables into the system call registers with `{name}` placeholders and executes `syscall` (Section 27.4).

20.3 EMIT: RAW BYTES

`emit(b1, b2, ...)` places the given bytes at the current position of the instruction stream, without any interpretation. The variants `emit16`, `emit32` and `emit64` place 16, 32 and 64-bit values in little-endian order. `@emit(...)` is the same as `emit(...)`. The intrinsics `ashley.InjectByte`, `ashley.InjectBytes`, `ashley.InjectWord`, `ashley.InjectDword` and `ashley.InjectQword` are long names for the same operations.

Table 20-3. Byte-emitting statements

Statement	Bytes produced
`emit(0x90);`	`90`
`emit(0xB8, 0x2A, 0, 0, 0);`	`B8 2A 00 00 00` (`mov eax, 42`)
`emit16(0xC031);`	`31 C0` (`xor eax, eax`)
`emit32(0x90909090);`	`90 90 90 90`
`emit64(0x0102030405060708);`	`08 07 06 05 04 03 02 01`
`@emit(0x0F, 0x01, 0xD9);`	`0F 01 D9` (`vmmcall`)
`ashley.InjectByte(0x90);`	`90`

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 20-4. Machine code written as bytes

ASH

```
@Entry
pub func main(): int64 {
    int64 r = 0;
    emit16(0xC031);                    // xor eax, eax   (bytes 31 C0)
    emit32(0x90909090);                // four nops
    ashley.InjectByte(0x90);           // one more nop
    emit(0x83, 0xC0, 0x07);            // add eax, 7
    asm volatile("mov {r}, rax");      // collect the result
    return r;                          // exit status 7
}
```

OUTPUT — EXIT STATUS 7

```
(no output)
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <main>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 00 00 00 00          mov    eax,0x0
  20:   49 89 c4                mov    r12,rax
  23:   31 c0                   xor    eax,eax
  25:   90                      nop
  26:   90                      nop
  27:   90                      nop
  28:   90                      nop
  29:   90                      nop
  2a:   83 c0 07                add    eax,0x7
  2d:   49 89 c4                mov    r12,rax
  30:   4c 89 e0                mov    rax,r12
  33:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  37:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  3b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  3f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  43:   c9                      leave
  44:   c3                      ret
```

`emit` is the tool of last resort: it bypasses the assembler, so a wrong byte produces a wrong instruction silently. It is useful for instructions that an assembler does not know yet, for exact padding, and for byte sequences required by firmware (signatures, magic numbers inside code).

20.4 RUNNABLE ARRAYS

A `runnable` global array is data that can be executed (Section 9.4.4). Combined with indexing it gives a simple way to generate or patch code at run time: write the bytes of a function into the array, then call the array like a function.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 20-5. Generating a function at run time

ASH

```
runnable byte[] code = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}

// write "mov eax, imm32 ; add eax, edi ; ret" with the chosen constant
func make_adder(int64 k): void {
    code[0] = 0xB8;
    code[1] = k & 255;
    code[2] = (k >> 8) & 255;
    code[3] = (k >> 16) & 255;
    code[4] = (k >> 24) & 255;
    code[5] = 0x01;              // add eax, edi
    code[6] = 0xF8;
    code[7] = 0xC3;              // ret
}

@Entry
pub func main(): int64 {
    make_adder(1000);
    printf("%d ", code(1));
    make_adder(5);
    printf("%d\n", code(37));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1001 42
```

20.5 NAKED FUNCTIONS

`@naked` functions (Section 17.11) contain only assembly and `emit` statements. Because the compiler generates no frame, the code sees the registers and the stack exactly as the caller left them: on x86-64 the arguments are in `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` and the return address is at `[rsp]`. They are the right tool for:

- interrupt and exception handlers (which end with `iretq` instead of `ret`);
- the first instructions of a kernel or boot loader (setting up a stack before any compiled code runs);
- trampolines and thunks that must not touch the stack;
- functions whose exact bytes are specified externally.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 20-6. A naked function with an asm string

ASH

```
@naked
func twice(int64 x): int64 {
    asm volatile("lea rax, [rdi + rdi]; ret");
}

@naked
func max2(int64 a, int64 b): int64 {
    asm {
        mov rax, rdi
        cmp rdi, rsi
        cmovl rax, rsi
        ret
    }
}

@Entry
pub func main(): int64 {
    printf("twice(21)=%d max2(3, 9)=%d\n", twice(21), max2(3, 9));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
twice(21)=42 max2(3, 9)=9
```

20.6 ASSEMBLY IN 32-BIT AND 16-BIT PROGRAMS

In a file with `@bits(32)` or `@bits(16)` the assembler works in the corresponding mode, so `asm` blocks must use 32-bit or 16-bit instructions and registers. The following program is a complete 32-bit Linux executable that writes a message with the `int 0x80` system call interface; it is built with `--exe` and needs no linker.

Example 20-7. A 32-bit Linux executable with int 0x80

ASH

```
@bits(32)
@originoffs(0x08048000)

@Entry
pub func _start(): int32 {
    string message = "Hello from 32-bit assembly\n";
    asm {
        mov eax, 4          ; sys_write
        mov ebx, 1          ; stdout
        mov ecx, message    ; address of the text
        mov edx, 27         ; length
        int 0x80

        mov eax, 1          ; sys_exit
        xor ebx, ebx        ; status 0
        int 0x80
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Hello from 32-bit assembly
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
x:     file format binary
Disassembly of section .data:
00000054 <.data+0x54>:
  54:   55                      push   ebp
  55:   89 e5                   mov    ebp,esp
  57:   81 ec 10 00 00 00       sub    esp,0x10
  5d:   b8 91 80 04 08          mov    eax,0x8048091
  62:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  65:   b8 04 00 00 00          mov    eax,0x4
  6a:   bb 01 00 00 00          mov    ebx,0x1
  6f:   8b 4d fc                mov    ecx,DWORD PTR [ebp-0x4]
  72:   ba 1b 00 00 00          mov    edx,0x1b
  77:   cd 80                   int    0x80
  79:   b8 01 00 00 00          mov    eax,0x1
  7e:   31 db                   xor    ebx,ebx
  80:   cd 80                   int    0x80
  82:   b8 00 00 00 00          mov    eax,0x0
  87:   89 ec                   mov    esp,ebp
  89:   5d                      pop    ebp
  8a:   c3                      ret
  8b:   31 c0                   xor    eax,eax
  8d:   89 ec                   mov    esp,ebp
  8f:   5d                      pop    ebp
  90:   c3                      ret
  91:   48                      dec    eax
  92:   65 6c                   gs ins BYTE PTR es:[edi],dx
  94:   6c                      ins    BYTE PTR es:[edi],dx
  95:   6f                      outs   dx,DWORD PTR ds:[esi]
  96:   20 66 72                and    BYTE PTR [esi+0x72],ah
  99:   6f                      outs   dx,DWORD PTR ds:[esi]
  9a:   6d                      ins    DWORD PTR es:[edi],dx
  9b:   20 33                   and    BYTE PTR [ebx],dh
  9d:   32 2d 62 69 74 20       xor    ch,BYTE PTR ds:0x20746962
  a3:   61                      popa
  a4:   73 73                   jae    0x119
  a6:   65 6d                   gs ins DWORD PTR es:[edi],dx
  a8:   62 6c 79 0a             bound  ebp,QWORD PTR [ecx+edi*2+0xa]
  ac:   00 00                   add    BYTE PTR [eax],al
  ae:   00 00                   add    BYTE PTR [eax],al
  b0:   b4 80                   mov    ah,0x80
```

20.7 HOW AN ASM BLOCK REACHES THE OBJECT FILE

1 `AsmBlocks` removes the block from the source and stores its text; the statement becomes `__asm(N)`.

2 The generator writes an ASHIR operation `asm[x...]` whose payload is the hexadecimal encoding of the text, with each variable reference rewritten as `@var:name`.

3 The code generator decodes the payload, replaces every `@var:name` with the current location of the variable, and calls `AsmBridge.assemble(text, bits, origin)`.

4 `shasm` assembles the text in flat mode and returns the bytes, which are copied into the function.

Copyright © 2026 Mallionic Inc. All rights reserved.

WARNING

In `ashc` 1.0, when `shasm` cannot assemble an `asm` block (an unknown mnemonic, an invalid operand combination), the block is **silently dropped**: no bytes are emitted for it and no error is printed. A typing mistake such as `mvo rax, 1` therefore produces a program that compiles and simply does not execute that instruction. Until this is fixed, check non-trivial assembly with the stand-alone assembler (Section 26.1), which does report errors, and inspect the result with `objdump`.

CHECK.ASM

```
bits 64
    mov rax, 1
    mvo rax, 1          ; typing mistake
```

SHELL SESSION

```
$ shasm --lang en -f bin -o check.bin check.asm
check.asm:3: error: invalid syntax: 'rax, 1'
```

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 21

STRICT MODE

Strict mode adds compile-time checks that catch mistakes the normal rules allow: literals that do not fit in their variable, silent narrowing of wide values, and `return` statements that do not match the function's return type. It changes no code generation; a program that compiles in strict mode behaves exactly as it would without it.

21.1 ENABLING STRICT MODE

Strict mode is enabled for a whole file by one of:

```
"use strict";          // first item of the file
@strict                // anywhere at top level
@strict(true)
```

`@strict(false)` disables it again. The declaration applies to the file being compiled.

21.2 THE RULES

Table 21-1. Checks performed in strict mode

Rule	Description
Literal range	A literal assigned or used to initialize a variable must fit in the variable's type: 0 to 2^N-1 for unsigned types, -2^(N-1) to 2^(N-1)-1 for signed types. Outside strict mode only the combined range -2^(N-1) to 2^N-1 is enforced, so `uint8 x = -1` is accepted there.
No implicit narrowing	Assigning an expression whose widest non-literal operand has more bits than the destination is rejected. Use an explicit cast to show that the narrowing is intended.
`void` functions	`return value;` in a function declared `void` is an error.
Non-`void` functions	`return;` without a value in a function with a return type is an error (outside strict mode it returns the pending `unreturnable` value or 0).

The width of an expression is computed from its operands: the width of the widest variable that appears in it, ignoring literals; comparisons have no width; a shift has the width of its left operand; casts and function calls are considered explicit and are accepted.

Copyright © 2026 Mallionic Inc. All rights reserved.

21.3 EXAMPLES

Example 21-1. A program that satisfies strict mode

ASH

```
"use strict";

func sum(int64 a, int64 b): int64 {
    return a + b;
}

func greet(): void {
    printf("hello from void\n");
    return;
}

@Entry
pub func main(): int64 {
    uint8 small = 255;                              // fits in uint8
    int8 negative = -128;                           // fits in int8
    uint64 big = 18446744073709551615ULL;
    int64 total = sum(40, 2);
    uint32 low = (uint32)(big & 0xFFFFFFFF);        // explicit cast: allowed
    greet();
    printf("small=%u total=%lld low=%u negative=%d\n", small, total, low, negative);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hello from void
small=255 total=42 low=4294967295 negative=-128
```

Example 21-2. An unsigned variable cannot hold a negative literal

ASH

```
@strict
@Entry
pub func main(): int64 {
    uint32 a = -1;
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: strict: the value -1 does not fit in 'a' (uint32, range 0 .. 4294967295)
1 error generated.
```

Example 21-3. A literal larger than the type

ASH

```
@strict(true)
@Entry
pub func main(): int64 {
    uint8 a = 256;
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: strict: the value 256 does not fit in 'a' (uint8, range 0 .. 255)
1 error generated.
```

Example 21-4. Implicit narrowing is rejected

ASH

```
"use strict";
@Entry
pub func main(): int64 {
    uint64 big = 5;
    uint32 small = big;
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: strict: cannot assign a 64-bit value to 'small' (uint32, 32 bits); use an explicit ca
↪ st
1 error generated.
```

Example 21-5. A void function cannot return a value

ASH

```
"use strict";
func f(): void {
    return 5;
}

@Entry
pub func main(): int64 {
    f();
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: strict: the function 'f' is void and cannot return a value
1 error generated.
```

Example 21-6. A function with a result needs a value in return

ASH

```
"use strict";
func g(): int64 {
    return;
}

@Entry
pub func main(): int64 {
    g();
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: strict: the function 'g' returns qword and 'return;' needs a value
1 error generated.
```

Copyright © 2026 Mallionic Inc. All rights reserved.

21.4 WHEN TO USE STRICT MODE

Strict mode is recommended for new code, for libraries and for code that manipulates data of several widths (protocol headers, pixel formats, hardware registers), where an unnoticed truncation is a real bug. It is less useful for small experiments and for code that intentionally relies on wrap-around; there, the explicit casts it requires may add noise.

> **TIP**
> The VS Code linter performs most of these checks while you type, in strict and non-strict files alike, and reports them as warnings (Section 29.1).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 22

TARGETS, RUNTIMES AND INTEROPERABILITY

This chapter describes each target that `ashc` can produce: what the output contains, which calling convention and runtime it uses, how it is linked and loaded, and how it interoperates with C. It closes with a table of corresponding C and ASH types.

22.1 OVERVIEW

Table 22-1. Targets of ashc 1.0

Target	Options	Output	Typical use
Linux x86-64 object	`--elf`	ELF64 `.o`	Programs and libraries for 64-bit Linux, with or without libc.
Linux i386 object	`--elf` + `@bits(32)`	ELF32 `.o`	32-bit programs and libraries, protected-mode kernels.
Linux i386 executable	`--exe` + `@bits(32)`	ELF32 executable	Small static programs without a linker.
Windows x64 object	`--win`	COFF `.obj`	Windows programs and DLL code, UEFI applications.
Windows i386 object	`--win` + `@bits(32)`	COFF `.obj`	32-bit Windows code.
Flat binary	`--compile`	raw `.bin`	Boot sectors, real-mode code, firmware, images for custom loaders.

22.2 LINUX X86-64

The default target produces an ELF64 relocatable object for the System V AMD64 ABI. Functions declared `pub` and globals declared `general` are global symbols; everything else is local. Code, string literals and global data are in the `.ashtext` section; `@section` creates additional sections (Section 6.3.4).

22.2.1 Linking with the C library

When the entry function is called `main`, link with `gcc out/prog.o -o prog`. Both position-independent (default) and non-PIE (`-no-pie`) executables work. The C start-up code initializes the C library, calls `main` with `argc`, `argv` and `envp`, and passes its result to `exit`.

Because code and writable data share a section, GNU `ld` warns that the executable has a segment with RWX permissions; pass `-Wl,--no-warn-rwx-segments` to silence it. Older linkers may also warn about the absence of `.note.GNU-stack`; `ashc` writes that section, so the stack is not made executable.

Copyright © 2026 Mallionic Inc. All rights reserved.

22.2.2 Without the C library

With `--no-libc` (or `@nolibc`) the compiler replaces the three C functions that compiled code may need with its own ASHIR implementations, built on Linux system calls:

Table 22-2. The built-in --no-libc runtime (x86-64)

Function	Implementation
`calloc(n, size)`	`mmap(NULL, n*size, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)` (system call 9). Anonymous memory is zero-filled by the kernel.
`free(p)`	Does nothing. Memory returns to the system when the process exits.
`exit(code)`	`exit_group(code)` (system call 231).

The program must define `_start` as its entry point and must end with `exit(...)` or `sys_exit(...)`; it is linked with `ld -e _start out/prog.o -o prog` and produces a static executable with no dependencies. Other C functions are not available unless you link an object that provides them.

> **WARNING**
> In `ashc` 1.0 the replacement runtime is inserted only when the program also uses arrays, strings or another feature that needs the runtime block (Section 6.2.7). A `--no-libc` program that calls `exit(...)` but uses no array fails to link with "undefined reference to `exit`". Use `sys_exit(...)` from `<syscalls>`, which never depends on the runtime.

The `<syscalls>` library (Section 27.4) provides output, files, memory and process functions written directly on system calls.

Example 22-1. The runtime functions inserted by --no-libc

ASH

```
@Entry
pub func _start(): int64 {
    int64[] v = {5, 6, 7};
    int64[2][2] m;
    m[1][1] = 30;
    v[1] = v[0] + v[2];
    exit(v[1] + m[1][1]);       // 12 + 30 = 42
    return 0;
}
```

OUTPUT — EXIT STATUS 42

```
(no output)
```

ASHIR

```
func qword calloc(qword n, qword size) {
    .type r qword
    %r mov[0]
    %_ asm[x6d6f76207273692c20407661723a6e0a696d756c207273692c20407661723a73697a650a786f7220
↪ 6564692c206564690a6d6f76206564782c20330a6d6f7620723130642c20307832320a6d6f762072382c202d31
↪ 0a786f72207239642c207239640a6d6f76206561782c20390a73797363616c6c0a6d6f7620407661723a722c20
↪ 7261780a]
    ret qword r
}

func qword free(qword p) {
    ret qword 0
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
}

func qword exit(qword code) {
    %_ asm[x6d6f76207264692c20407661723a636f64650a6d6f76206561782c203233310a73797363616c6c0a
↪ ]
    ret qword 0
}
```

SHELL SESSION

```
$ ashc tiny_static.ash --elf --no-libc -s
$ ld -e _start out/tiny_static.o -o tiny_static
$ ./tiny_static; echo "exit status $?"
static and tiny
exit status 42
$ ldd tiny_static
    not a dynamic executable
$ ls -l tiny_static
-rwxrwxr-x 1 user user 8560 Oct 24 22:38 tiny_static
```

22.3 LINUX I386

With `@bits(32)` (or `--bits 32`) and `--elf`, the output is an ELF32 object for the i386 architecture. Code is generated for 32-bit protected mode; all integers wider than 32 bits are held in 32 bits (Section 8.3). Functions use the `cdecl` convention: arguments are pushed right to left, the caller removes them, the result is in `eax`, and `ebx`, `esi`, `edi`, `ebp` are preserved.

32-bit objects link with the 32-bit C library (`gcc -m32 -no-pie -z noexecstack`), or without any library using `ld -m elf_i386`. The following program is a 32-bit ASH library called from a freestanding C program that has no C library at all:

LIB32.ASH

```
@bits(32)

pub func add3(int32 a, int32 b, int32 c): int32 {
    int32[] t = {a, b, c};
    return t[0] + t[1] + t[2];
}

priv func hidden(): int32 {
    return 1;
}

@Entry
pub func ash_main(): int32 {
    return add3(1, 2, 3) + hidden();
}
```

START32.C

```
/* freestanding C: no libc, its own _start */
int add3(int, int, int);
int ash_main(void);

void _start(void) {
    int r = add3(10, 20, 30) + ash_main();          /* 60 + 7 = 67 */
    __asm__ volatile("int $0x80" :: "a"(1), "b"(r));  /* sys_exit(r) */
```

Copyright © 2026 Mallionic Inc. All rights reserved.

START32.C (CONTINUED)

```
}
```

SHELL SESSION

```
$ ashc lib32.ash --elf --no-libc -s
$ gcc -m32 -ffreestanding -fno-pic -fno-stack-protector -c start32.c -o start32.o
$ ld -m elf_i386 -z noexecstack -e _start start32.o out/lib32.o -o app32
$ ./app32; echo "exit status $?"
exit status 67
$ file app32
app32: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), statically linked, not stri
↪ pped
```

22.4 LINUX I386 EXECUTABLES (--EXE)

`--exe` produces a complete 32-bit Linux executable without a linker. The compiler generates a flat 32-bit image and prepends an 84-byte header: the ELF32 file header (52 bytes) and one program header (32 bytes) describing a single loadable segment with read, write and execute permissions. The segment is loaded at the address given by `@originoffs` (normally `0x08048000`), its memory size includes one extra mebibyte of zero-filled space used by the built-in heap, and the entry point is the first byte after the header, where the `@Entry` function is placed.

There is no C library in such a program; it communicates with the kernel through `int 0x80` (Example 20-7) and ends with `exit`, which the built-in runtime implements with system call 1.

Example 22-2. The smallest --exe program and its header

ASH

```
@bits(32)
@originoffs(0x08048000)

@Entry
pub func _start(): int32 {
    exit(7);
    return 0;
}
```

HEX DUMP (XXD)

```
00000000: 7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00  .ELF............
00000010: 02 00 03 00 01 00 00 00 54 80 04 08 34 00 00 00  ........T...4...
00000020: 00 00 00 00 00 00 00 00 34 00 20 00 01 00 00 00  ........4. .....
00000030: 00 00 00 00 01 00 00 00 00 00 00 00 00 80 04 08  ................
00000040: 00 80 04 08 94 00 00 00 94 00 10 00 07 00 00 00  ................
00000050: 00 10 00 00 55 89 e5 81 ec 10 00 00 00 b8 07 00  ....U...........
... (4 more lines not shown)
```

OUTPUT — EXIT STATUS 7

```
(no output)
```

In the dump, the first line is the ELF identification (`7F 45 4C 46` = "\x7FELF", class 1 = 32-bit, data 1 = little-endian), followed by the type 2 (executable), the machine 3 (i386) and the entry address `0x08048054` (base + 84).

Copyright © 2026 Mallionic Inc. All rights reserved.

22.5 FLAT BINARIES

`--compile` writes the machine code of the program followed by its data, with no header and no symbol table. All addresses are resolved by the compiler using the origin declared with `@originoffs`, and the `@Entry` function is placed first so that execution can begin at the first byte. Flat images cannot call external functions; the built-in `calloc`, `free` and `exit` of the runtime are available.

Table 22-3. Typical origins of flat images

Origin	Use
`0x7C00`	BIOS boot sector (512 bytes, ending with the signature `55 AA`).
`0x100`	DOS `.COM` programs.
`0x0`	Position at the start of a segment, firmware images, code copied by a loader.
`0x100000`	Kernels loaded at 1 MiB by a boot loader.

The 16-bit image of Example 3-4 and the 32-bit image below were executed by the emulators of the test suite (`tests/bits16/emu16.py` and `tests/bits32/emu32.py`, built on the Unicorn engine), which load the image, call it, and print the value of `AX` / `EAX` when the entry function returns. The variant `tests/bits32/emu_vga.py`, used below, also prints the first characters of the VGA text buffer at `0xB8000` and the set of attribute bytes found there.

Example 22-3. A 32-bit flat image that writes to VGA text memory

ASH

```
@bits(32)
@originoffs(0)

@Entry
pub func _start(): int32 {
    uint8* vga = 0xB8000;
    string msg = "ASH!";
    for (int32 i = 0; i < 4; i++) {
        vga[i * 2] = msg[i];
        vga[i * 2 + 1] = 0x1F;          // white on blue
    }
    return 42;
}
```

OUTPUT — EXIT STATUS 0

```
eax = 42 text: ASH!□□□□□□□□□□□□□□□□□□□□ attrs: {0, 31}
```

X86 DISASSEMBLY (32-BIT)

```
   0:   55                      push   ebp
   1:   89 e5                   mov    ebp,esp
   3:   81 ec 30 00 00 00       sub    esp,0x30
   9:   b8 00 80 0b 00          mov    eax,0xb8000
   e:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  11:   b8 ea 00 00 00          mov    eax,0xea
  16:   89 45 f8                mov    DWORD PTR [ebp-0x8],eax
  19:   b8 00 00 00 00          mov    eax,0x0
  1e:   89 45 f4                mov    DWORD PTR [ebp-0xc],eax
  21:   8b 45 f4                mov    eax,DWORD PTR [ebp-0xc]
  24:   b9 04 00 00 00          mov    ecx,0x4
  29:   39 c8                   cmp    eax,ecx
  2b:   0f 8d aa 00 00 00       jge    0xdb
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86 DISASSEMBLY (32-BIT) (CONTINUED)

```
  31:   8b 45 f4                mov    eax,DWORD PTR [ebp-0xc]
  34:   89 45 f0                mov    DWORD PTR [ebp-0x10],eax
  37:   8b 45 f0                mov    eax,DWORD PTR [ebp-0x10]
  3a:   b9 02 00 00 00          mov    ecx,0x2
  3f:   0f af c1                imul   eax,ecx
  42:   89 45 f0                mov    DWORD PTR [ebp-0x10],eax
  45:   8b 45 f0                mov    eax,DWORD PTR [ebp-0x10]
  48:   89 45 ec                mov    DWORD PTR [ebp-0x14],eax
  4b:   8b 45 ec                mov    eax,DWORD PTR [ebp-0x14]
  4e:   8b 4d fc                mov    ecx,DWORD PTR [ebp-0x4]
  51:   01 c8                   add    eax,ecx
  53:   89 45 ec                mov    DWORD PTR [ebp-0x14],eax
  56:   8b 45 f4                mov    eax,DWORD PTR [ebp-0xc]
  59:   89 45 e4                mov    DWORD PTR [ebp-0x1c],eax
  5c:   8b 45 e4                mov    eax,DWORD PTR [ebp-0x1c]
  5f:   8b 4d f8                mov    ecx,DWORD PTR [ebp-0x8]
  62:   01 c8                   add    eax,ecx
```

A complete boot sector is easier to write with the stand-alone assembler, which can pad the image to 510 bytes and append the signature; Section 26.10 shows one.

22.6 WINDOWS X64

`--win` writes a COFF object for AMD64. Calls to external functions use the Microsoft x64 calling convention:

- the first four integer or pointer arguments in `rcx`, `rdx`, `r8`, `r9`;
- 32 bytes of *shadow space* reserved by the caller just above the return address (`sub rsp, 0x20` before the call, `add rsp, 0x20` after);
- further arguments on the stack above the shadow space;
- the result in `rax`.

References to external functions use `IMAGE_REL_AMD64_REL32` relocations, resolved by the linker against import libraries such as `kernel32.lib`, `user32.lib` or `ntdll.lib`.

Example 22-4. A call to MessageBoxA with the Microsoft convention

ASH

```
extern int32 MessageBoxA(uint64 hwnd, uint64 text, uint64 caption, uint32 type);
extern void ExitProcess(uint32 code);

@Entry
pub func main(): void {
    int32 r = MessageBoxA(0, "Hello from ASH", "ASH", 64);   // MB_ICONINFORMATION
    ExitProcess(r);
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <main>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  1b:   b8 00 00 00 00          mov    eax,0x0
  20:   49 89 c4                mov    r12,rax
  23:   b8 00 00 00 00          mov    eax,0x0
  28:   49 89 c5                mov    r13,rax
  2b:   4c 8d 2d 6a 00 00 00    lea    r13,[rip+0x6a]        # 9c <__str0>
  32:   b8 00 00 00 00          mov    eax,0x0
  37:   49 89 c6                mov    r14,rax
  3a:   4c 8d 35 6b 00 00 00    lea    r14,[rip+0x6b]        # ac <__str1>
  41:   b8 40 00 00 00          mov    eax,0x40
  46:   49 89 c7                mov    r15,rax
  49:   b8 00 00 00 00          mov    eax,0x0
  4e:   89 45 d8                mov    DWORD PTR [rbp-0x28],eax
  51:   48 83 ec 20             sub    rsp,0x20
  55:   4c 89 e1                mov    rcx,r12
  58:   4c 89 ea                mov    rdx,r13
  5b:   4d 89 f0                mov    r8,r14
  5e:   45 89 f9                mov    r9d,r15d
  61:   e8 00 00 00 00          call   66 <main+0x66>
  66:   48 83 c4 20             add    rsp,0x20
  6a:   89 45 d8                mov    DWORD PTR [rbp-0x28],eax
  6d:   8b 45 d8                mov    eax,DWORD PTR [rbp-0x28]
  70:   89 45 d0                mov    DWORD PTR [rbp-0x30],eax
  73:   48 83 ec 20             sub    rsp,0x20
  77:   8b 4d d0                mov    ecx,DWORD PTR [rbp-0x30]
  7a:   e8 00 00 00 00          call   7f <main+0x7f>
  7f:   48 83 c4 20             add    rsp,0x20
  83:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  87:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  8b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  8f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  93:   48 89 ec                mov    rsp,rbp
  96:   5d                      pop    rbp
  97:   c3                      ret
  98:   00 00                   add    BYTE PTR [rax],al
        ...
```

The disassembly shows the shadow space and the four argument registers. The program is linked with Microsoft's linker or with MinGW-w64:

SHELL

```
link /subsystem:windows /entry:main msgbox.obj kernel32.lib user32.lib
x86_64-w64-mingw32-gcc msgbox.obj -o msgbox.exe -nostartfiles -Wl,-e,main -lkernel32 -luser3
↪ 2
```

A program linked with `-nostartfiles` has no C runtime: its entry function must end the process with `ExitProcess`. The example in Example 3-5 was linked this way and executed under Wine.

WARNING

Limitations of the Windows target in `ashc` 1.0: floating-point values passed to variadic functions are not copied to the integer registers as the Microsoft convention requires; and functions compiled by ASH use `rdi` and `rsi` (which are callee-saved on Windows) without preserving them, which is harmless when the program ends with `ExitProcess` but must be considered when ASH code is called from C code compiled for Windows.

Copyright © 2026 Mallionic Inc. All rights reserved.

22.7 UEFI APPLICATIONS

UEFI firmware uses the Microsoft x64 calling convention and UCS-2 strings. ASH provides the pieces needed to write UEFI applications: `--win` produces COFF objects that UEFI linkers accept, `@efiapi` makes the entry function receive `ImageHandle` and `SystemTable` in `rcx` and `rdx`, `L"..."` produces UCS-2 strings, structures describe the firmware tables, and pointer chains with calls such as `st->ConOut->OutputString(st->ConOut, msg)` use the Microsoft convention for indirect calls under `--win`.

ASH

```
// Simplified EFI tables: only the fields used here are described.
typedef struct {
    uint64 Reset;
    uint64 OutputString;          // EFI_TEXT_STRING
} SimpleTextOutput;

typedef struct {
    uint64 Signature;             // EFI_TABLE_HEADER
    uint32 Revision;
    uint32 HeaderSize;
    uint32 CRC32;
    uint32 Reserved;
    uint64 FirmwareVendor;
    uint32 FirmwareRevision;
    uint64 ConsoleInHandle;
    uint64 ConIn;
    uint64 ConsoleOutHandle;
    SimpleTextOutput* ConOut;
} SystemTable;
```

NOTE

The layout of every EFI table must be taken from the UEFI specification. The resulting `.obj` file is linked into a PE image of subsystem `EFI_APPLICATION`, for example with `lld-link /subsystem:efi_application /entry:efi_main`.

Copyright © 2026 Mallionic Inc. All rights reserved.

22.8 CORRESPONDING C AND ASH TYPES

Table 22-4. C types and their ASH equivalents (x86-64 Linux)

C type	ASH type	Size
`char`, `signed char`	`int8`	1
`unsigned char`, `uint8_t`, `bool` (`_Bool`)	`uint8`, `bool`	1
`short`, `int16_t`	`int16`	2
`unsigned short`, `uint16_t`	`uint16`	2
`int`, `int32_t`	`int32`	4
`unsigned int`, `uint32_t`	`uint32`	4
`long`, `long long`, `int64_t`, `ssize_t`, `intptr_t`	`int64`	8
`unsigned long`, `unsigned long long`, `size_t`, `uint64_t`, `uintptr_t`	`uint64`	8
`T*`, `void*`	`T*`, `void*`, `ptr`, `uint64`	8
`const char*`	`string`	8
`struct S`	`S` (same fields)	same
function pointer	`ptr`, `uint64`, `int64`	8

On 64-bit Windows `long` is 32 bits (`int32` in ASH); on the i386 targets `long` and pointers are 32 bits.

22.9 CALLING DIRECTIONS

Table 22-5. ASH and C calling each other

Direction	How
ASH calls C	Declare the function with `extern` (recommended) or call it directly; link the C object or library.
C calls ASH	Declare the ASH function `pub`; in C, write the prototype with corresponding types (Table 22-4). Methods are called `Class_method`.
C reads/writes ASH data	Declare the global `general` (and `static` if C writes it); in C, `extern T name;`.
ASH reads/writes C data	Take the address of the C variable with an `extern` function that returns it, or receive it as a pointer argument.
Callbacks	Pass `&f` where C expects a function pointer (Section 12.7).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 23

ASHIR: THE INTERMEDIATE REPRESENTATION

ASHIR (*ASH Intermediate Representation*) is the textual language that connects the front end and the back end of `ashc`. It is produced by `ashc file.ash` (and `--stdout`), and it is accepted as input by `ashc file.ashir`. This chapter specifies its syntax, every operation and directive, and the runtime helper functions that the front end adds to programs.

23.1 OVERVIEW

An ASHIR file is a sequence of top-level items separated by blank lines:

- `extern` declarations of functions defined elsewhere;
- function definitions, optionally preceded by attributes such as `[entry]`;
- global data directives: `.string`, `.gvar`, `.gdata`, `.section`.

Inside a function, each line is a declaration (`.type`), an operation in *destination form* (`%dest op[args]`), an operation in *statement form* (`store[...]`, `call[...]`, `inc x`, `ret ...`), or raw bytes (`.raw8`). Every operation is a small, explicit step; there are no nested expressions. The design is close to *three-address code*: an operation reads at most two values and writes one.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 23-1. A complete hand-written ASHIR program

ASHIR SOURCE

```
extern qword printf(qword fmt, qword v)

[entry]
func qword main() {
.type i qword
.type acc qword
.type neg qword
    %i mov[0]
    %acc mov[0]
    %_ label[top]
    %_ brf[i, lt, 10, done]
    %acc add[i]
    inc i
    %_ jmp[top]
    %_ label[done]
    %neg mov[-64]
    %neg sar[3]
    %__t addr[fmt]
    call[printf, __t, acc]
    %__t addr[fmt]
    call[printf, __t, neg]
    ret qword 0
}

.string fmt "%ld\n"
```

OUTPUT — EXIT STATUS 0

```
45
-8
```

23.2 LEXICAL RULES

- Identifiers follow the rules of ASH; names beginning with `__` are used by the compiler.
- Numbers are decimal (optionally negative) or hexadecimal with `0x`. Binary literals are not accepted (see Section 7.6.1).
- Strings use double quotes and the escape sequences of ASH.
- ASHIR has **no comments**: a line beginning with `//` or `;` is a syntax error.
- Indentation is free; the compiler indents operations by four spaces.

Copyright © 2026 Mallionic Inc. All rights reserved.

23.3 TYPES

Table 23-1. ASHIR types

Type	Meaning
`byte`, `word`, `dword`, `qword`	Integers of 8, 16, 32 and 64 bits; zero-extended when widened, unsigned comparisons when the source type was unsigned.
`int8`, `int16`, `int32`, `int64`	Signed integers; sign-extended when widened (`movsx`, `movsxd`).
`uint8`, `uint16`, `uint32`, `uint64`	Unsigned integers; division, right shift and comparisons use the unsigned instructions.
`ptr`, `string`, `cstring`	Addresses (64 bits on x86-64).
`float32`, `float64`	IEEE-754 values (limited support, Section 8.8).
`void`	No value (return type only).

23.4 FUNCTIONS AND EXTERNS

```
extern type name(type param, ...)
[entry] [naked] [efiapi]
func type name(type param, ...) {
    body
}
```

`[entry]` marks the entry point, `[naked]` suppresses prologue, epilogue and `ret`, and `[efiapi]` selects the Microsoft x64 convention for the parameters. The alternative header `func name(params): type` is also accepted. Parameters are local variables of the function, initialized from the argument registers or stack slots.

A function body declares its variables with `.type name type` before (or at) their first use. Variables that are not declared get their type from the first value assigned to them.

23.5 OPERATIONS

23.5.1 Destination form

```
%dest op[arg, arg, ...]
```

The operation reads its arguments (and, for arithmetic, the current value of `dest`) and writes `dest`. An argument is a variable name, an integer constant, a label or a symbol. `%_` is used as destination when the result is not needed (labels, jumps, branches, assembly).

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 23-2. Data operations

Operation	Form	Effect
`mov`	`%d mov[x]`	`d = x`. *x* is a variable or constant; widening follows the types.
`add`	`%d add[x]`	`d = d + x`
`sub`	`%d sub[x]`	`d = d - x`
`imul` / `mul`	`%d imul[x]`	`d = d * x` (low bits; both spellings are accepted)
`div`	`%d div[x]`	`d = d / x` (signed or unsigned according to the type of `d`)
`mod`	`%d mod[x]`	`d = d % x`
`and` `or` `xor`	`%d and[x]`	Bitwise operations
`shl`	`%d shl[n]`	Left shift by a constant
`shr` / `sar`	`%d shr[n]`	Right shift by a constant: logical for unsigned `d`, arithmetic for signed `d` (`sar` always arithmetic)
`load`	`%d load[a]`	`d` = the value at address `a`, with the width of `d`
`addr`	`%d addr[sym]`	`d` = address of a local array/structure block, a global or a string
`faddr`	`%d faddr[f]`	`d` = address of function `f`
`call`	`%d call[f, a1, a2, ...]`	Call `f` with the arguments; `d` receives the result
`cmp`	`%a cmp[b]`	Compare `a` with `b` (sets the flags for a following conditional jump)

Table 23-3. Control operations

Operation	Form	Effect
`label`	`%_ label[L]`	Defines label `L` at this point.
`jmp`	`%_ jmp[L]`	Unconditional jump.
`brf`	`%_ brf[a, cc, b, L]`	Jump to `L` if `a cc b` is **false**.
`brt`	`%_ brt[a, cc, b, L]`	Jump to `L` if `a cc b` is **true**.
`je` `jne` `jl` `jle` `jg` `jge` `jb` `jbe` `ja` `jae` (`jz`, `jnz`)	`jge[L]`	Conditional jump after a `cmp`.
`asm`	`%_ asm[xHEX]`	Assembly text, encoded in hexadecimal; `@var:name` inside it refers to a variable.

The comparison codes of `brf` and `brt` are `eq`, `ne`, `lt`, `le`, `gt` and `ge`; the signedness of the comparison follows the types of the operands.

Copyright © 2026 Mallionic Inc. All rights reserved.

23.5.2 Statement form

Table 23-4. Statement-form operations

Statement	Effect
`store[a, v]`	Store `v` at address `a`, with the width of `v`.
`call[f, a1, ...]`	Call without using the result.
`inc x` / `dec x`	`x = x + 1` / `x = x - 1`.
`ret type value`	Return `value` (a variable or constant) converted to `type`.
`ret void`	Return without a value.
`volatile_barrier`	Marks a volatile write; generates no code.
`.raw8 v` (also `.raw16`, `.raw32`, `.raw64`)	Emit raw bytes (used by `emit` and naked functions).

23.6 GLOBAL DATA DIRECTIVES

Table 23-5. Data directives

Directive	Meaning
`.string sym "text"`	A NUL-terminated string literal named `sym` (read-only by convention).
`.gvar sym size value`	A global scalar of *size* bytes (1, 2, 4 or 8) with an initial value. `sym` is the symbol: the variable name for `general`, `__g_name` for `ungeneral`.
`.gdata sym xHEX align`	A global array: the bytes (header and elements) in hexadecimal, aligned to *align* bytes.
`.section sym ".name" "flags"`	Places the function or data `sym` in section `.name` with the given flags.

Example 23-2. Global data in ASHIR

ASH

```
general static int64 counter = 7
ungeneral uint16 MAGIC = 0xBEEF
general uint32[] table = {1, 2, 3}

@section(".fast")
func get(): int64 {
    return counter + MAGIC + table[2];
}

@Entry
pub func main(): int64 {
    printf("%d\n", get());
    return 0;
}
```

ASHIR

```
extern qword calloc(qword n, qword size)
extern qword free(qword p)
extern qword exit(qword code)

func qword __ash_alloc(qword n, qword esz, qword extra) {
    .type n_ qword
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
    .type e_ qword
    .type x_ qword
    .type total qword
    .type raw qword
    .type hdr qword
    %n_ mov[n]
    %e_ mov[esz]
    %x_ mov[extra]
    %total mov[n_]
    %total mul[e_]
    %total add[x_]
    %total add[16]
    %raw call[calloc, 1, total]
    store[raw, n_]
    %hdr mov[raw]
    %hdr add[8]
    store[hdr, n_]
    %raw add[16]
    ret qword raw
}

func qword __ash_free(qword p) {
    .type p_ qword
    .type blk qword
    .type junk qword
    %p_ mov[p]
    %blk mov[p_]
    %blk sub[16]
    %junk call[free, blk]
    ret qword 0
}

func qword __ash_strlen(qword p) {
    .type p_ qword
    .type n qword
    .type a qword
    .type b qword
    %p_ mov[p]
    %n mov[0]
    %_ label[__ash_sl_top]
    %a mov[p_]
    %a add[n]
    %b load[a]
    %b mod[256]
    %_ brf[b, ne, 0, __ash_sl_end]
    inc n
    %_ jmp[__ash_sl_top]
    %_ label[__ash_sl_end]
    ret qword n
}

func qword __ash_memcpy(qword d, qword s, qword n) {
    .type d_ qword
    .type s_ qword
    .type n_ qword
    .type i qword
    .type a qword
    .type b qword
    .type v qword
    .type vb uint8
    %d_ mov[d]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
    %s_ mov[s]
    %n_ mov[n]
    %i mov[0]
    %_ label[__ash_mc_top]
    %_ brf[i, lt, n_, __ash_mc_end]
    %a mov[s_]
    %a add[i]
    %v load[a]
    %v mod[256]
    %vb mov[v]
    %b mov[d_]
    %b add[i]
    store[b, vb]
    inc i
    %_ jmp[__ash_mc_top]
    %_ label[__ash_mc_end]
    ret qword d_
}

func qword __ash_str_new(qword src) {
    .type src_ qword
    .type n qword
    .type p qword
... (161 more lines not shown)
```

23.7 HOW ASH CONSTRUCTS APPEAR IN ASHIR

Table 23-6. Lowering of ASH constructs

ASH	ASHIR
`int64 x = a + b * 2;`	`.type x qword`, a temporary for `b * 2`, `mov`, `add`
`if (c) { } else { }`	`brf` to the else label, body, `jmp` to the end label
`while (c) { }`	label, `brf` to the exit, body, `jmp` back
`for (i; c; u) { }`	init, label, `brf`, body, continue label, update, `jmp`
`repeat (n) { }`	hidden counter `__rpN`, as a `for` loop
`switch`	hidden subject `__swN` and match flag (Section 11.7.1)
`v[i]`	optional `call[__ash_bounds, v, i]`, address arithmetic, `load` / `store`
`p->f`	`add` of the field offset, then `load` / `store`
global `g`	`addr[g]` and a typed `load` / `store`
`f(a, b)`	temporaries for the arguments, `call[f, ...]`
`asm { }`	`%_ asm[xHEX]`
`emit(...)`	`.raw8` lines

Copyright © 2026 Mallionic Inc. All rights reserved.

23.8 RUNTIME HELPER FUNCTIONS

When a program uses arrays, strings, matrices or shifts by a variable amount, the front end appends a block of helper functions to the ASHIR. They are ordinary ASHIR functions, so they appear in every listing produced with `ir=full`, and in the object file as local symbols.

Table 23-7. Runtime helpers

Helper	Purpose
`__ash_alloc(n, esz, extra)`	Allocates a block for *n* elements of *esz* bytes plus *extra* bytes and a 16-byte header with `calloc`; stores the length and capacity in the header; returns the address of element 0.
`__ash_free(p)`	Frees a block given the address of element 0.
`__ash_bounds(base, idx)`	Returns `idx` if it is smaller than the length stored in the header of `base`; otherwise ends the program with `exit(134)`.
`__ash_strlen(p)`	Length of a NUL-terminated string.
`__ash_memcpy(d, s, n)`	Copies *n* bytes.
`__ash_str_new(src)`	Creates a heap string (with header) holding a copy of `src`.
`__ash_str_set(dst, src)`	Copies `src` into the heap string `dst`, reallocating it if its capacity is too small.
`__ash_matrix(rows, cols, esz)`	Allocates a two-dimensional array: an array of row addresses, each row an array of *cols* elements.
`__ash_shl(v, n)`, `__ash_shr(v, n)`	Shifts by a variable amount.
`__va_get(i, a0, ..., a7)`	Selects a variadic argument (Section 12.6.1).
`__ash_pow(b, e)`, `__ash_sqpow(b, n)`	Run-time `**` and `***` (Section 10.5).

The following listing shows the ASHIR of the four helpers that every array access relies on, exactly as the compiler emits them:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 23-3. The array helpers in ASHIR

ASHIR

```
func qword __ash_alloc(qword n, qword esz, qword extra) {
    .type n_ qword
    .type e_ qword
    .type x_ qword
    .type total qword
    .type raw qword
    .type hdr qword
    %n_ mov[n]
    %e_ mov[esz]
    %x_ mov[extra]
    %total mov[n_]
    %total mul[e_]
    %total add[x_]
    %total add[16]
    %raw call[calloc, 1, total]
    store[raw, n_]
    %hdr mov[raw]
    %hdr add[8]
    store[hdr, n_]
    %raw add[16]
    ret qword raw
}

func qword __ash_free(qword p) {
    .type p_ qword
    .type blk qword
    .type junk qword
    %p_ mov[p]
    %blk mov[p_]
    %blk sub[16]
    %junk call[free, blk]
    ret qword 0
}

func qword __ash_matrix(qword rows, qword cols, qword esz) {
    .type r_ qword
    .type c_ qword
    .type e_ qword
    .type m qword
    .type i qword
    .type slot qword
    .type row qword
    %r_ mov[rows]
    %c_ mov[cols]
    %e_ mov[esz]
    %m call[__ash_alloc, r_, 8, 0]
    %i mov[0]
    %_ label[__ash_mx_top]
    %_ brf[i, lt, r_, __ash_mx_end]
    %row call[__ash_alloc, c_, e_, 0]
    %slot mov[i]
    %slot mul[8]
    %slot add[m]
    store[slot, row]
    inc i
    %_ jmp[__ash_mx_top]
    %_ label[__ash_mx_end]
    ret qword m
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
func qword __ash_bounds(qword base, qword idx) {
    .type b_ qword
    .type i_ qword
    .type hdr qword
    .type len qword
    .type junk qword
    %b_ mov[base]
    %i_ mov[idx]
    %hdr mov[b_]
    %hdr sub[16]
    %len load[hdr]
    %_ brt[i_, lt, len, __ash_ok]
    %junk call[exit, 134]
    %_ label[__ash_ok]
    ret qword i_
}
[entry]
func qword main() {
.type m qword
.type __tmp0 qword
    %__tmp0 mov[2]
.type __tmp1 qword
    %__tmp1 mov[2]
    %m call[__ash_matrix, __tmp0, __tmp1, 8]
.type __tmp3 qword
    %__tmp3 mov[1]
.type __tmp4 qword
    %__tmp4 call[__ash_bounds, m, __tmp3]
    %__tmp3 imul[8]
    %__tmp3 add[m]
... (32 more lines not shown)
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 23-4. The string helpers in ASHIR

ASHIR

```
func qword __ash_strlen(qword p) {
    .type p_ qword
    .type n qword
    .type a qword
    .type b qword
    %p_ mov[p]
    %n mov[0]
    %_ label[__ash_sl_top]
    %a mov[p_]
    %a add[n]
    %b load[a]
    %b mod[256]
    %_ brf[b, ne, 0, __ash_sl_end]
    inc n
    %_ jmp[__ash_sl_top]
    %_ label[__ash_sl_end]
    ret qword n
}

func qword __ash_memcpy(qword d, qword s, qword n) {
    .type d_ qword
    .type s_ qword
    .type n_ qword
    .type i qword
    .type a qword
    .type b qword
    .type v qword
    .type vb uint8
    %d_ mov[d]
    %s_ mov[s]
    %n_ mov[n]
    %i mov[0]
    %_ label[__ash_mc_top]
    %_ brf[i, lt, n_, __ash_mc_end]
    %a mov[s_]
    %a add[i]
    %v load[a]
    %v mod[256]
    %vb mov[v]
    %b mov[d_]
    %b add[i]
    store[b, vb]
    inc i
    %_ jmp[__ash_mc_top]
    %_ label[__ash_mc_end]
    ret qword d_
}

func qword __ash_str_new(qword src) {
    .type src_ qword
    .type n qword
    .type p qword
    .type junk qword
    %src_ mov[src]
    %n call[__ash_strlen, src_]
    %p call[__ash_alloc, n, 1, 1]
    %junk call[__ash_memcpy, p, src_, n]
    ret qword p
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
func qword __ash_str_set(qword dst, qword src) {
    .type dst_ qword
    .type src_ qword
    .type n qword
    .type cap qword
    .type hdr qword
    .type res qword
    .type junk qword
    .type endp qword
    .type lenp qword
    .type z uint8
    %dst_ mov[dst]
    %src_ mov[src]
    %n call[__ash_strlen, src_]
    %res mov[dst_]
    %hdr mov[dst_]
    %hdr sub[8]
    %cap load[hdr]
    %_ brt[cap, ge, n, __ash_fit]
    %res call[__ash_alloc, n, 1, 1]
    %junk call[__ash_free, dst_]
    %_ label[__ash_fit]
    %junk call[__ash_memcpy, res, src_, n]
    %endp mov[res]
    %endp add[n]
    %z mov[0]
    store[endp, z]
    %lenp mov[res]
    %lenp sub[16]
    store[lenp, n]
... (2 more lines not shown)
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 23-5. The shift helpers in ASHIR

ASHIR

```
func qword __ash_shl(qword v, qword n) {
    .type v_ qword
    .type n_ qword
    %v_ mov[v]
    %n_ mov[n]
    %_ label[__ash_shl_top]
    %_ brf[n_, gt, 0, __ash_shl_end]
    %v_ mul[2]
    dec n_
    %_ jmp[__ash_shl_top]
    %_ label[__ash_shl_end]
    ret qword v_
}

func qword __ash_shr(qword v, qword n) {
    .type v_ qword
    .type n_ qword
    .type r qword
    %v_ mov[v]
    %n_ mov[n]
    %_ label[__ash_shr_top]
    %_ brf[n_, gt, 0, __ash_shr_end]
    %r mov[v_]
    %r mod[2]
    %v_ div[2]
    %_ brt[r, ge, 0, __ash_shr_nx]
    dec v_
    %_ label[__ash_shr_nx]
    dec n_
    %_ jmp[__ash_shr_top]
    %_ label[__ash_shr_end]
    ret qword v_
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 23-6. Power and variadic helpers in ASHIR

ASHIR

```
func qword __va_get(qword i, qword a0, qword a1, qword a2, qword a3, qword a4, qword a5, qwo
↪ rd a6, qword a7) {
    %_ brf[i, eq, 0, __L0]
    ret qword a0
    %_ label[__L0]
    %_ brf[i, eq, 1, __L2]
    ret qword a1
    %_ label[__L2]
    %_ brf[i, eq, 2, __L4]
    ret qword a2
    %_ label[__L4]
    %_ brf[i, eq, 3, __L6]
    ret qword a3
    %_ label[__L6]
    %_ brf[i, eq, 4, __L8]
    ret qword a4
    %_ label[__L8]
    %_ brf[i, eq, 5, __L10]
    ret qword a5
    %_ label[__L10]
    %_ brf[i, eq, 6, __L12]
    ret qword a6
    %_ label[__L12]
    %_ brf[i, eq, 7, __L14]
    ret qword a7
    %_ label[__L14]
    ret qword 0
}

func qword __ash_pow(qword base, qword ex) {
.type r qword
.type b qword
.type e qword
    %r mov[1]
    %b mov[base]
    %e mov[ex]
    %_ label[__L16]
    %_ brf[e, gt, 0, __L17]
    %__tmp27 mov[e]
    %__tmp27 and[1]
    %_ brf[__tmp27, eq, 1, __L18]
    %r mov[r]
    %r imul[b]
    %_ label[__L18]
    %b mov[b]
    %b imul[b]
    %e mov[e]
    %e shr[1]
    %_ jmp[__L16]
    %_ label[__L17]
    ret qword r
}

func qword __ash_sqpow(qword base, qword n) {
.type r qword
.type i qword
    %r mov[base]
    %i mov[0]
    %_ label[__L20]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASHIR (CONTINUED)

```
    %_ brf[i, lt, n, __L22]
    %r mov[r]
    %r imul[r]
    %_ label[__L21]
    inc i
    %_ jmp[__L20]
    %_ label[__L22]
    ret qword r
}
```

23.9 WORKING WITH ASHIR FILES

Because the back end reads ASHIR text, you can save the intermediate representation, modify it and compile the modified file:

W.ASH

```
@Entry
pub func main(): int64 {
    int64 x = 6;
    return x * 7;
}
```

SHELL SESSION

```
$ ashc w.ash -s
$ cat w.ashir
[entry]
func qword main() {
.type x qword
    %x mov[6]
    %__ret mov[x]
    %__ret imul[7]
    ret qword __ret
}
$ sed -i 's/imul\[7\]/imul[8]/' w.ashir
$ ashc w.ashir --elf -s && gcc out/w.o -o w && ./w; echo "exit status $?"
exit status 48
```

This makes ASHIR a convenient target for experiments: other tools can generate it, and individual operations can be changed to test the code generator.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 24

CODE GENERATION FOR X86

This chapter explains how the back end turns ASHIR into x86 machine code. It describes the model used by the 64-bit generator (register usage, frames, calls, memory access), the simpler model of the 32-bit and 16-bit generator, and shows the instruction sequences produced for each kind of operation. The goal is to let you read any disassembly of an ASH program and relate every instruction to a line of source.

24.1 A DIRECT TRANSLATOR

The code generators of `ashc` 1.0 translate each ASHIR operation independently into a short, fixed instruction pattern. They do not reorder operations, do not eliminate redundant moves and do not keep values in registers across operations beyond the variables' home locations. The resulting code is predictable and easy to map back to the source, at the price of extra `mov` instructions compared with an optimizing C compiler. When a routine must be fast, write it in an `asm` block or a `@naked` function (Chapter 20).

A typical operation is translated in three steps: load the current value of the destination into `rax`, combine it with the argument (loaded into `rcx` if it is not a constant), and store `rax` back into the destination's home:

Example 24-1. One ASHIR operation, one short pattern

ASH

```
func step(int64 a, int64 b): int64 {
    int64 x = a;
    x = x + b;          // %x add[b]
    x = x * 3;          // %x imul[3]
    return x;
}

@Entry
pub func main(): int64 {
    return step(4, 10);
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <step>:
   0:   48 89 f8                mov    rax,rdi
   3:   49 89 c2                mov    r10,rax
   6:   4c 89 d0                mov    rax,r10
   9:   48 89 f1                mov    rcx,rsi
   c:   48 01 c8                add    rax,rcx
   f:   49 89 c2                mov    r10,rax
  12:   4c 89 d0                mov    rax,r10
  15:   48 6b c0 03             imul   rax,rax,0x3
  19:   49 89 c2                mov    r10,rax
  1c:   4c 89 d0                mov    rax,r10
  1f:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

24.2 REGISTER USAGE ON X86-64

Table 24-1. Registers used by the 64-bit code generator

Register	Use
`rax`	Accumulator of every operation; return value.
`rcx`, `rdx`	Second operand; `rdx` also holds the high half for `div` / `idiv` and the remainder.
`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`	Arguments of calls (System V); copied to their home locations at function entry.
`r8`–`r11`	Homes of locals and temporaries in leaf functions (no calls).
`r12`–`r15`	Homes of locals and temporaries in functions that make calls or contain assembly; saved in the prologue and restored before returning.
`rbp`	Frame pointer, when the function has a frame.
`rsp`	Stack pointer; aligned to 16 bytes at each call.
`rbx`	Not used.
`xmm0`–`xmm7`, `xmm8`–`xmm15`	Reserved for floating-point arguments and variables.

Values that do not fit in the available registers get 8-byte slots in the frame, addressed as `[rbp-n]` (Example 9-3).

24.3 PROLOGUE AND EPILOGUE

Table 24-2. Frame shapes

Function	Code
Leaf, no stack use (`@beautycode(true)`)	No prologue; the body ends with `ret`.
Leaf with stack use	`push rbp; mov rbp, rsp; sub rsp, N` ... `leave; ret`
Function that calls or uses `asm`	`push rbp; mov rbp, rsp; sub rsp, N`, then `mov [rbp-8], r12` ... for each of `r12`–`r15`; before each `ret` the registers are reloaded and `mov rsp, rbp; pop rbp` (or `leave`) restores the frame.
`_start` entry	As above, plus `and rsp, -16` right after `mov rbp, rsp`, because the process entry is not reached by a `call`.
`@naked`	Nothing.
`-e` / `@beautycode(false)`	Every function has the full frame.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 24-2. Leaf function versus calling function

ASH

```
func leaf(int64 a): int64 {
    return a + 1;
}

func caller(int64 a): int64 {
    return leaf(a) + leaf(a + 1);
}

@Entry
pub func main(): int64 {
    return caller(5);
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <leaf>:
   0:   48 89 f8                mov    rax,rdi
   3:   49 89 c2                mov    r10,rax
   6:   4c 89 d0                mov    rax,r10
   9:   48 83 c0 01             add    rax,0x1
   d:   49 89 c2                mov    r10,rax
  10:   4c 89 d0                mov    rax,r10
  13:   c3                      ret

0000000000000014 <caller>:
  14:   55                      push   rbp
  15:   48 89 e5                mov    rbp,rsp
  18:   48 81 ec 30 00 00 00    sub    rsp,0x30
  1f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  23:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  27:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  2b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  2f:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  33:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  37:   49 89 c4                mov    r12,rax
  3a:   b8 00 00 00 00          mov    eax,0x0
  3f:   49 89 c5                mov    r13,rax
  42:   4c 89 e7                mov    rdi,r12
  45:   e8 b6 ff ff ff          call   0 <leaf>
  4a:   49 89 c5                mov    r13,rax
  4d:   4c 89 e8                mov    rax,r13
  50:   49 89 c6                mov    r14,rax
  53:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  57:   49 89 c7                mov    r15,rax
  5a:   4c 89 f8                mov    rax,r15
  5d:   48 83 c0 01             add    rax,0x1
  61:   49 89 c7                mov    r15,rax
  64:   b8 00 00 00 00          mov    eax,0x0
  69:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  6d:   4c 89 ff                mov    rdi,r15
  70:   e8 8b ff ff ff          call   0 <leaf>
  75:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  79:   4c 89 f0                mov    rax,r14
  7c:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
  80:   48 01 c8                add    rax,rcx
  83:   49 89 c6                mov    r14,rax
  86:   4c 89 f0                mov    rax,r14
  89:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  8d:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  91:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  95:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  99:   c9                      leave
  9a:   c3                      ret
```

24.4 CONSTANTS AND MOVES

Small constants are loaded with `mov eax, imm32` (which also clears the upper half of `rax`), negative or large constants with `mov rax, imm64`. Arithmetic with a constant uses the immediate forms `add rax, imm8/imm32`, `imul rax, rax, imm`. Moves between homes go through `rax`: `mov rax, r12` then `mov r13, rax`.

24.5 ARITHMETIC, DIVISION AND SHIFTS

Example 24-3. Instruction patterns for arithmetic

ASH

```
func ops(int64 a, int64 b): int64 {
    int64 s = a + b;
    int64 d = a - b;
    int64 m = a * b;
    int64 q = a / b;
    int64 r = a % b;
    int64 l = a << 3;
    int64 h = a >> 2;
    int64 x = (a & b) | (a ^ b);
    return s + d + m + q + r + l + h + x;
}

@Entry
pub func main(): int64 {
    return ops(100, 7);
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <ops>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 70 00 00 00    sub    rsp,0x70
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   49 89 c2                mov    r10,rax
  21:   4c 89 d0                mov    rax,r10
  24:   48 89 f1                mov    rcx,rsi
  27:   48 01 c8                add    rax,rcx
  2a:   49 89 c2                mov    r10,rax
  2d:   48 89 f8                mov    rax,rdi
  30:   49 89 c3                mov    r11,rax
  33:   4c 89 d8                mov    rax,r11
  36:   48 89 f1                mov    rcx,rsi
  39:   48 29 c8                sub    rax,rcx
  3c:   49 89 c3                mov    r11,rax
  3f:   48 89 f8                mov    rax,rdi
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  42:   49 89 c0                mov    r8,rax
  45:   4c 89 c0                mov    rax,r8
  48:   48 89 f1                mov    rcx,rsi
  4b:   48 0f af c1             imul   rax,rcx
  4f:   49 89 c0                mov    r8,rax
  52:   48 89 f8                mov    rax,rdi
  55:   49 89 c1                mov    r9,rax
  58:   48 89 f1                mov    rcx,rsi
  5b:   4c 89 c8                mov    rax,r9
  5e:   48 99                   cqo
  60:   48 f7 f9                idiv   rcx
  63:   49 89 c1                mov    r9,rax
  66:   48 89 f8                mov    rax,rdi
  69:   49 89 c4                mov    r12,rax
  6c:   48 89 f1                mov    rcx,rsi
  6f:   4c 89 e0                mov    rax,r12
  72:   48 99                   cqo
  74:   48 f7 f9                idiv   rcx
  77:   49 89 d4                mov    r12,rdx
  7a:   48 89 f8                mov    rax,rdi
  7d:   49 89 c5                mov    r13,rax
  80:   4c 89 e8                mov    rax,r13
  83:   48 c1 e0 03             shl    rax,0x3
  87:   49 89 c5                mov    r13,rax
  8a:   48 89 f8                mov    rax,rdi
  8d:   49 89 c6                mov    r14,rax
  90:   4c 89 f0                mov    rax,r14
  93:   48 c1 f8 02             sar    rax,0x2
  97:   49 89 c6                mov    r14,rax
  9a:   48 89 f8                mov    rax,rdi
  9d:   49 89 c7                mov    r15,rax
  a0:   4c 89 f8                mov    rax,r15
  a3:   48 89 f1                mov    rcx,rsi
  a6:   48 21 c8                and    rax,rcx
  a9:   49 89 c7                mov    r15,rax
  ac:   4c 89 f8                mov    rax,r15
  af:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
  b3:   48 89 f8                mov    rax,rdi
  b6:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  ba:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
  be:   48 89 f1                mov    rcx,rsi
  c1:   48 31 c8                xor    rax,rcx
  c4:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  c8:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  cc:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
  d0:   48 09 c8                or     rax,rcx
  d3:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
  d7:   4c 89 d0                mov    rax,r10
  da:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  de:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
  e2:   4c 89 d9                mov    rcx,r11
  e5:   48 01 c8                add    rax,rcx
  e8:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  ec:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
  f0:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  f4:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
  f8:   4c 89 c1                mov    rcx,r8
  fb:   48 01 c8                add    rax,rcx
  fe:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
... (37 more lines not shown)
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Division sign-extends `rax` into `rdx` with `cqo` and uses `idiv`; for unsigned operands it clears `edx` and uses `div`. The quotient is taken from `rax` and the remainder from `rdx`.

24.6 CONDITIONS AND LOOPS

A `brf` / `brt` operation becomes `cmp` between the two operands followed by a conditional jump. The jump tests the *opposite* condition for `brf` (jump to the false label) and the condition itself for `brt`.

Table 24-3. Conditional jumps chosen for comparisons

Code	Signed	Unsigned	Jump taken by brf (condition false)
`eq`	`je`	`je`	`jne`
`ne`	`jne`	`jne`	`je`
`lt`	`jl`	`jb`	`jge` / `jae`
`le`	`jle`	`jbe`	`jg` / `ja`
`gt`	`jg`	`ja`	`jle` / `jbe`
`ge`	`jge`	`jae`	`jl` / `jb`

Example 24-4. A for loop in machine code

ASH

```
func count(int64 n): int64 {
    int64 total = 0;
    for (int64 i = 0; i < n; i++) {
        total = total + i;
    }
    return total;
}

@Entry
pub func main(): int64 {
    return count(10);
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <count>:
   0:   b8 00 00 00 00          mov    eax,0x0
   5:   49 89 c2                mov    r10,rax
   8:   b8 00 00 00 00          mov    eax,0x0
   d:   49 89 c3                mov    r11,rax
  10:   4c 89 d8                mov    rax,r11
  13:   48 89 f9                mov    rcx,rdi
  16:   48 39 c8                cmp    rax,rcx
  19:   0f 8d 1a 00 00 00       jge    39 <count+0x39>
  1f:   4c 89 d0                mov    rax,r10
  22:   4c 89 d9                mov    rcx,r11
  25:   48 01 c8                add    rax,rcx
  28:   49 89 c2                mov    r10,rax
  2b:   4c 89 d8                mov    rax,r11
  2e:   48 ff c0                inc    rax
  31:   49 89 c3                mov    r11,rax
  34:   e9 d7 ff ff ff          jmp    10 <count+0x10>
  39:   4c 89 d0                mov    rax,r10
  3c:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

The loop has the shape of its ASHIR: the condition at the top (`cmp` and `jge` to the exit), the body, the increment, and a `jmp` back to the top.

24.7 CALLS

For a call the generator evaluates the arguments into their temporaries, pushes the arguments beyond the sixth (last first), loads the first six into `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, clears `eax` (the number of vector registers, required by variadic C functions), executes `call`, removes the pushed arguments with `add rsp, 8*n`, and moves `rax` into the destination. Calls to functions in the same object use a `call rel32` resolved by the object writer; calls to external functions use a `R_X86_64_PLT32` relocation that the linker resolves to the function or to its PLT entry.

Example 24-5. Calling a C function

ASH

```
@Entry
pub func main(): int64 {
    printf("%d %d %d\n", 1, 2, 3);
    return 0;
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <main>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 00 00 00 00          mov    eax,0x0
  20:   49 89 c4                mov    r12,rax
  23:   4c 8d 25 46 00 00 00    lea    r12,[rip+0x46]        # 70 <__str0>
  2a:   b8 01 00 00 00          mov    eax,0x1
  2f:   49 89 c5                mov    r13,rax
  32:   b8 02 00 00 00          mov    eax,0x2
  37:   49 89 c6                mov    r14,rax
  3a:   b8 03 00 00 00          mov    eax,0x3
  3f:   49 89 c7                mov    r15,rax
  42:   4c 89 e7                mov    rdi,r12
  45:   4c 89 ee                mov    rsi,r13
  48:   4c 89 f2                mov    rdx,r14
  4b:   4c 89 f9                mov    rcx,r15
  4e:   b8 00 00 00 00          mov    eax,0x0
  53:   e8 00 00 00 00          call   58 <main+0x58>
  58:   48 31 c0                xor    rax,rax
  5b:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5f:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  63:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  67:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  6b:   48 89 ec                mov    rsp,rbp
  6e:   5d                      pop    rbp
  6f:   c3                      ret
```

The `call` instruction shows `e8 00 00 00 00`: the four zero bytes are the relocation target, filled in by the linker.

Copyright © 2026 Mallionic Inc. All rights reserved.

24.8 MEMORY ACCESS

Table 24-4. Addressing used for each kind of data

Data	Addressing
Locals and parameters	Registers, or `[rbp-n]` slots.
Arguments 7 and above	`[rbp+16]`, `[rbp+24]` ...
Globals, strings, functions	RIP-relative: `lea reg, [rip+disp32]` to obtain the address, then `mov` through it.
Array and structure elements	The address is computed into `rax` (base + index * size or base + offset), then `mov` with the width of the element: `BYTE`, `WORD`, `DWORD` or `QWORD PTR [rax]`.

Example 24-6. RIP-relative access to a global variable

ASH

```
general static int64 counter = 5

func bump(): int64 {
    counter = counter + 1;
    return counter;
}

@Entry
pub func main(): int64 {
    return bump();
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <bump>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   b8 00 00 00 00          mov    eax,0x0
  1c:   49 89 c2                mov    r10,rax
  1f:   4c 8d 15 9a 00 00 00    lea    r10,[rip+0x9a]        # c0 <counter>
  26:   b8 00 00 00 00          mov    eax,0x0
  2b:   49 89 c3                mov    r11,rax
  2e:   4c 8d 1d 8b 00 00 00    lea    r11,[rip+0x8b]        # c0 <counter>
  35:   b8 00 00 00 00          mov    eax,0x0
  3a:   49 89 c0                mov    r8,rax
  3d:   4c 89 d8                mov    rax,r11
  40:   48 8b 08                mov    rcx,QWORD PTR [rax]
  43:   49 89 c8                mov    r8,rcx
  46:   4c 89 c0                mov    rax,r8
  49:   49 89 c1                mov    r9,rax
  4c:   4c 89 c8                mov    rax,r9
  4f:   49 89 c4                mov    r12,rax
  52:   4c 89 e0                mov    rax,r12
  55:   48 83 c0 01             add    rax,0x1
  59:   49 89 c4                mov    r12,rax
  5c:   4c 89 d0                mov    rax,r10
  5f:   4c 89 e1                mov    rcx,r12
  62:   48 89 08                mov    QWORD PTR [rax],rcx
  65:   b8 00 00 00 00          mov    eax,0x0
  6a:   49 89 c5                mov    r13,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  6d:   4c 8d 2d 4c 00 00 00    lea    r13,[rip+0x4c]        # c0 <counter>
  74:   b8 00 00 00 00          mov    eax,0x0
  79:   49 89 c6                mov    r14,rax
  7c:   4c 89 e8                mov    rax,r13
  7f:   48 8b 08                mov    rcx,QWORD PTR [rax]
  82:   49 89 ce                mov    r14,rcx
  85:   4c 89 f0                mov    rax,r14
  88:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  8c:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  90:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  94:   c9                      leave
  95:   c3                      ret
```

Example 24-7. Indexed access with bounds checking

ASH

```
func get(int64 i): int64 {
    int64[] data = {10, 20, 30};
    return data[i];              // bounds-checked: data is an array
}

func raw(int64* p, int64 i): int64 {
    return p[i];                 // not checked: p is a raw pointer
}

@Entry
pub func main(): int64 {
    int64[] v = {1, 2};
    return get(2) + raw(v, 1);
}
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000166 <get>:
 166:   55                      push   rbp
 167:   48 89 e5                mov    rbp,rsp
 16a:   48 81 ec 60 00 00 00    sub    rsp,0x60
 171:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 175:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 179:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 17d:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 181:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 185:   b8 00 00 00 00          mov    eax,0x0
 18a:   49 89 c4                mov    r12,rax
 18d:   bf 03 00 00 00          mov    edi,0x3
 192:   be 08 00 00 00          mov    esi,0x8
 197:   ba 00 00 00 00          mov    edx,0x0
 19c:   e8 5f fe ff ff          call   0 <__ash_alloc>
 1a1:   49 89 c4                mov    r12,rax
 1a4:   4c 89 e0                mov    rax,r12
 1a7:   49 89 c5                mov    r13,rax
 1aa:   b8 0a 00 00 00          mov    eax,0xa
 1af:   49 89 c6                mov    r14,rax
 1b2:   4c 89 e8                mov    rax,r13
 1b5:   4c 89 f1                mov    rcx,r14
 1b8:   48 89 08                mov    QWORD PTR [rax],rcx
 1bb:   4c 89 e0                mov    rax,r12
 1be:   49 89 c7                mov    r15,rax
 1c1:   4c 89 f8                mov    rax,r15
 1c4:   48 83 c0 08             add    rax,0x8
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 1c8:   49 89 c7                mov    r15,rax
 1cb:   b8 14 00 00 00          mov    eax,0x14
 1d0:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 1d4:   4c 89 f8                mov    rax,r15
 1d7:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
 1db:   48 89 08                mov    QWORD PTR [rax],rcx
 1de:   4c 89 e0                mov    rax,r12
 1e1:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 1e5:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 1e9:   48 83 c0 10             add    rax,0x10
 1ed:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 1f1:   b8 1e 00 00 00          mov    eax,0x1e
 1f6:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 1fa:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 1fe:   48 8b 4d c0             mov    rcx,QWORD PTR [rbp-0x40]
 202:   48 89 08                mov    QWORD PTR [rax],rcx
 205:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 209:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 20d:   b8 00 00 00 00          mov    eax,0x0
 212:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 216:   4c 89 e7                mov    rdi,r12
 219:   48 8b 75 b8             mov    rsi,QWORD PTR [rbp-0x48]
 21d:   e8 af fe ff ff          call   d1 <__ash_bounds>
 222:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 226:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 22a:   48 6b c0 08             imul   rax,rax,0x8
 22e:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 232:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 236:   4c 89 e1                mov    rcx,r12
 239:   48 01 c8                add    rax,rcx
 23c:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 240:   b8 00 00 00 00          mov    eax,0x0
 245:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 249:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 24d:   48 8b 08                mov    rcx,QWORD PTR [rax]
 250:   48 89 4d a8             mov    QWORD PTR [rbp-0x58],rcx
 254:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
 258:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
 25c:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
 260:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 264:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 268:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 26c:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 270:   48 89 ec                mov    rsp,rbp
 273:   5d                      pop    rbp
 274:   c3                      ret

0000000000000275 <raw>:
 275:   55                      push   rbp
 276:   48 89 e5                mov    rbp,rsp
 279:   48 81 ec 20 00 00 00    sub    rsp,0x20
 280:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 284:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 288:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 28c:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 290:   48 89 f0                mov    rax,rsi
 293:   49 89 c2                mov    r10,rax
 296:   4c 89 d0                mov    rax,r10
 299:   48 6b c0 08             imul   rax,rax,0x8
 29d:   49 89 c2                mov    r10,rax
 2a0:   4c 89 d0                mov    rax,r10
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 2a3:   48 89 f9                mov    rcx,rdi
 2a6:   48 01 c8                add    rax,rcx
... (16 more lines not shown)
```

24.9 THE 32-BIT GENERATOR

`Backend32` generates code for `@bits(32)` (and, with 16-bit encodings, for `@bits(16)`). Its model is simpler than the 64-bit one:

- every function has a frame: `push ebp; mov ebp, esp; sub esp, N`;
- every local and temporary lives in a slot `[ebp-n]`; parameters are read from `[ebp+8]`, `[ebp+12]` ...;
- `eax` is the accumulator and `ecx` the second operand; results are returned in `eax`;
- calls push the arguments from right to left and remove them with `add esp, 4*n` after the call;
- a function ends with `mov esp, ebp; pop ebp; ret`.

Example 24-8. The same function compiled for i386

ASH

```
@bits(32)

func add(int32 a, int32 b): int32 {
    int32 c = a + b;
    return c;
}

@Entry
pub func main(): int32 {
    return add(40, 2);
}
```

X86 DISASSEMBLY (32-BIT)

```
0000003c <add>:
  3c:   55                      push   ebp
  3d:   89 e5                   mov    ebp,esp
  3f:   81 ec 10 00 00 00       sub    esp,0x10
  45:   8b 45 08                mov    eax,DWORD PTR [ebp+0x8]
  48:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  4b:   8b 45 fc                mov    eax,DWORD PTR [ebp-0x4]
  4e:   8b 4d 0c                mov    ecx,DWORD PTR [ebp+0xc]
  51:   01 c8                   add    eax,ecx
  53:   89 45 fc                mov    DWORD PTR [ebp-0x4],eax
  56:   8b 45 fc                mov    eax,DWORD PTR [ebp-0x4]
  59:   89 ec                   mov    esp,ebp
  5b:   5d                      pop    ebp
  5c:   c3                      ret
  5d:   31 c0                   xor    eax,eax
  5f:   89 ec                   mov    esp,ebp
  61:   5d                      pop    ebp
  62:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

24.10 THE 16-BIT GENERATOR

In 16-bit mode the same model is applied with 16-bit registers and operand sizes: `bp`-based frames, `ax` as accumulator, arguments pushed as words, results in `ax`, and near `call` / `ret`. Addresses are 16-bit offsets within the segment in which the image is loaded; the generated code does not change segment registers, so the image must be loaded with `CS`, `DS`, `ES` and `SS` set appropriately (as DOS does for `.COM` programs and as a boot sector must arrange itself).

Example 24-9. A 16-bit function and its caller

ASH

```
@bits(16)
@originoffs(0x100)

func add(int16 a, int16 b): int16 {
    return a + b;
}

@Entry
pub func _start(): int16 {
    return add(40, 2);
}
```

OUTPUT — EXIT STATUS 0

```
ax = 42  bp = 0x0
```

X86 DISASSEMBLY (16-BIT)

```
 100:   55                      push   bp
 101:   89 e5                   mov    bp,sp
 103:   81 ec 10 00             sub    sp,0x10
 107:   b8 28 00                mov    ax,0x28
 10a:   89 46 fe                mov    WORD PTR [bp-0x2],ax
 10d:   b8 02 00                mov    ax,0x2
 110:   89 46 fc                mov    WORD PTR [bp-0x4],ax
 113:   8b 46 fc                mov    ax,WORD PTR [bp-0x4]
 116:   50                      push   ax
 117:   8b 46 fe                mov    ax,WORD PTR [bp-0x2]
 11a:   50                      push   ax
 11b:   e8 14 00                call   0x132
 11e:   81 c4 04 00             add    sp,0x4
 122:   89 46 fa                mov    WORD PTR [bp-0x6],ax
 125:   8b 46 fa                mov    ax,WORD PTR [bp-0x6]
 128:   89 ec                   mov    sp,bp
 12a:   5d                      pop    bp
 12b:   c3                      ret
 12c:   31 c0                   xor    ax,ax
 12e:   89 ec                   mov    sp,bp
 130:   5d                      pop    bp
 131:   c3                      ret
 132:   55                      push   bp
 133:   89 e5                   mov    bp,sp
 135:   81 ec 10 00             sub    sp,0x10
 139:   8b 46 04                mov    ax,WORD PTR [bp+0x4]
 13c:   89 46 fe                mov    WORD PTR [bp-0x2],ax
 13f:   8b 46 fe                mov    ax,WORD PTR [bp-0x2]
 142:   8b 4e 06                mov    cx,WORD PTR [bp+0x6]
 145:   01 c8                   add    ax,cx
 147:   89 46 fe                mov    WORD PTR [bp-0x2],ax
 14a:   8b 46 fe                mov    ax,WORD PTR [bp-0x2]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86 DISASSEMBLY (16-BIT) (CONTINUED)

```
14d:   89 ec                   mov    sp,bp
14f:   5d                      pop    bp
150:   c3                      ret
151:   31 c0                   xor    ax,ax
153:   89 ec                   mov    sp,bp
155:   5d                      pop    bp
156:   c3                      ret
157:   00 5a 01                add    BYTE PTR [bp+si+0x1],bl
```

24.11 READING GENERATED CODE

When you read a disassembly of an ASH function, the following landmarks help:

1. The prologue tells you the frame size and which callee-saved registers the function uses.
2. Each source statement starts by loading values into `rax` (and `rcx`) and ends by storing `rax` into a register in `r8`–`r15` or a frame slot.
3. A `cmp` followed by a conditional jump is a condition; a `jmp` backwards is the end of a loop body.
4. A `call` preceded by loads into `rdi`, `rsi` ... is a function call; `call` to `__ash_bounds` is a bounds check.
5. The epilogue restores `r12`–`r15` and returns.

The `examples/` directory and Appendix D contain programs for every construct; compiling them with `--elf` and running `objdump -d -M intel` on the result is the fastest way to learn the patterns.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 25

AN X86 PRIMER FOR ASH PROGRAMMERS

This chapter introduces the parts of the x86 architecture that appear throughout this manual: operating modes, registers, flags, memory addressing, the stack and the encoding of instructions. It is not a replacement for the processor manuals, but it gives enough background to read the disassembly of ASH programs and to write `asm` blocks and `shasm` sources. Every encoding shown here was produced by `shasm`.

25.1 OPERATING MODES

Table 25-1. Processor modes targeted by ASH

Mode	ASH	Characteristics
Real mode	`@bits(16)`	The mode of the processor after reset and of DOS. 16-bit registers and offsets, 1 MiB of address space through segment:offset pairs, BIOS services through `int`.
Protected mode	`@bits(32)`	32-bit registers and flat 4 GiB address space, paging, privilege levels. Used by 32-bit operating systems and programs.
Long mode (64-bit)	`@bits(64)`	64-bit registers, sixteen general-purpose registers, RIP-relative addressing, canonical 48-bit (or 57-bit) virtual addresses. Used by 64-bit operating systems.

The same instruction bytes may mean different things in different modes; the assembler must know the mode (`bits 16`, `bits 32`, `bits 64` in `shasm`, `@bits(...)` in ASH) to encode instructions correctly.

25.2 GENERAL-PURPOSE REGISTERS

In 64-bit mode there are sixteen 64-bit general-purpose registers. Each one can also be accessed as a 32-bit, 16-bit and 8-bit register. Writing a 32-bit register clears the upper 32 bits of the 64-bit register; writing an 8-bit or 16-bit register leaves the other bits unchanged.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 25-2. General-purpose registers

64-bit	32-bit	16-bit	8-bit	Conventional role (System V)
`rax`	`eax`	`ax`	`al` (`ah`)	Return value, accumulator
`rbx`	`ebx`	`bx`	`bl` (`bh`)	Callee-saved
`rcx`	`ecx`	`cx`	`cl` (`ch`)	4th argument; shift count
`rdx`	`edx`	`dx`	`dl` (`dh`)	3rd argument; high half of mul/div
`rsi`	`esi`	`si`	`sil`	2nd argument; string source
`rdi`	`edi`	`di`	`dil`	1st argument; string destination
`rbp`	`ebp`	`bp`	`bpl`	Frame pointer, callee-saved
`rsp`	`esp`	`sp`	`spl`	Stack pointer
`r8`	`r8d`	`r8w`	`r8b`	5th argument
`r9`	`r9d`	`r9w`	`r9b`	6th argument
`r10`, `r11`	`r10d`, `r11d`	`r10w`, `r11w`	`r10b`, `r11b`	Scratch (caller-saved)
`r12`–`r15`	`r12d`–`r15d`	`r12w`–`r15w`	`r12b`–`r15b`	Callee-saved

In 32-bit mode only the eight registers `eax` to `edi` exist; in 16-bit mode the eight 16-bit registers `ax` to `di`. The *segment registers* `cs`, `ds`, `es`, `fs`, `gs` and `ss` select memory segments in real and protected mode; in 64-bit mode only `fs` and `gs` keep a base address (used for thread-local storage). Appendix Appendix B lists every register known to `shasm`.

25.2.1 Other registers

- `rip` / `eip` / `ip`: the instruction pointer, the address of the next instruction.
- `rflags`: the flags register.
- `cr0`, `cr2`, `cr3`, `cr4`, `cr8`: control registers (paging, protection, interrupt priority). Accessible only at privilege level 0.
- `dr0`–`dr7`: debug registers.
- `xmm0`–`xmm15` (and `ymm`, `zmm` with AVX): vector registers, also used for floating point.
- `st0`–`st7`, `mm0`–`mm7`: the x87 floating-point stack and the MMX registers.

25.3 FLAGS

Arithmetic and logical instructions set *status flags* in `rflags`, which conditional jumps (`jcc`), conditional moves (`cmovcc`) and `setcc` test.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 25-3. Status flags

Flag	Name	Set when
`CF`	Carry	An unsigned operation produced a carry out of, or a borrow into, the most significant bit.
`ZF`	Zero	The result is zero.
`SF`	Sign	The most significant bit of the result is 1.
`OF`	Overflow	A signed operation produced a result that does not fit.
`PF`	Parity	The low byte of the result has an even number of 1 bits.
`AF`	Adjust	A carry out of bit 3 (used by BCD arithmetic).

`cmp a, b` computes `a - b`, discards the result and sets the flags. Signed comparisons then use `jl`, `jle`, `jg`, `jge` (which test `SF`, `OF` and `ZF`); unsigned comparisons use `jb`, `jbe`, `ja`, `jae` (which test `CF` and `ZF`). This is why ASH must know the signedness of the operands of a comparison (Table 8-4).

25.4 MEMORY OPERANDS

An instruction operand in memory is written between brackets. Its *effective address* is computed as:

```
[ base + index * scale + displacement ]
```

where *base* and *index* are registers, *scale* is 1, 2, 4 or 8, and *displacement* is a signed 8-bit or 32-bit constant. Any part may be omitted. In 64-bit mode an address may also be relative to the instruction pointer, `[rip + displacement]`, which is how position-independent code reaches its data. The size of the memory operand is given by the other operand or by a size keyword: `byte`, `word`, `dword`, `qword`.

Example 25-1. Addressing forms and their encodings

SHASM

```
bits 64
    mov rax, [rbx]                ; base
    mov rax, [rbx + 8]            ; base + disp8
    mov rax, [rbx + 0x1000]       ; base + disp32
    mov rax, [rbx + rcx*8]        ; base + index*scale
    mov rax, [rbx + rcx*8 + 16]   ; everything
    mov eax, [rsp + 4]            ; rsp as base needs a SIB byte
    mov byte [rdi], 0x41          ; size from the keyword
    mov qword [rbp - 8], 0        ; frame slot
    lea rax, [rel data]           ; RIP-relative address
    lea rcx, [rax + rax*4]        ; arithmetic with lea: rcx = rax * 5
data:
    dq 0
```

SHASM LISTING

```
     2 00000000 488B03                  mov rax, [rbx]
     3 00000003 488B4308                mov rax, [rbx + 8]
     4 00000007 488B8300100000          mov rax, [rbx + 0x1000]
     5 0000000E 488B04CB                mov rax, [rbx + rcx*8]
     6 00000012 488B44CB10              mov rax, [rbx + rcx*8 + 16]
     7 00000017 8B442404                mov eax, [rsp + 4]
     8 0000001B C60741                  mov byte [rdi], 0x41
     9 0000001E 48C745F800000000        mov qword [rbp - 8], 0
    10 00000026 488D0504000000          lea rax, [rel data]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

SHASM LISTING (CONTINUED)

```
11 0000002D 488D0C80                lea rcx, [rax + rax*4]
13 00000031 0000000000000000        dq 0
```

25.5 THE STACK

The stack grows towards lower addresses. `push x` decrements `rsp` by 8 and stores `x` at `[rsp]`; `pop x` loads from `[rsp]` and increments `rsp`. `call f` pushes the address of the next instruction and jumps to `f`; `ret` pops that address and jumps to it. The System V ABI requires `rsp` to be a multiple of 16 just before a `call`, which is why ASH functions reserve frame sizes in multiples of 16 and why `_start` aligns the stack explicitly (Section 24.3).

```
      higher addresses
  +------------------------+
  | argument 8             |  [rbp+24]
  | argument 7             |  [rbp+16]
  | return address         |  [rbp+8]   pushed by call
  | saved rbp              |  [rbp]     pushed by the prologue
  | saved r12              |  [rbp-8]
  | ...                    |
  | locals, temporaries    |
  +------------------------+  <- rsp (16-byte aligned before calls)
      lower addresses
```

25.6 INSTRUCTION ENCODING

An x86 instruction is a sequence of 1 to 15 bytes with the following structure; every part except the opcode is optional:

Table 25-4. Parts of an x86 instruction

Part	Size	Purpose
Legacy prefixes	0–4	`66` operand size, `67` address size, `F0` lock, `F2`/`F3` repeat or mandatory prefix, segment overrides.
REX prefix	0–1	64-bit mode only: `0100WRXB`. `W` selects 64-bit operand size, `R`, `X`, `B` extend the register fields to reach `r8`–`r15`.
Opcode	1–3	The operation: one byte, or `0F xx`, `0F 38 xx`, `0F 3A xx`.
ModRM	0–1	`mod` (2 bits): addressing mode; `reg` (3 bits): a register or an opcode extension (`/digit`); `r/m` (3 bits): a register or a memory form.
SIB	0–1	`scale` (2), `index` (3), `base` (3), for `[base + index*scale]`.
Displacement	0, 1, 2, 4	Signed offset of a memory operand.
Immediate	0, 1, 2, 4, 8	Constant operand.

The following listing shows how the same operation is encoded with different operand sizes and registers. Observe the `66` prefix for 16-bit operands, the `REX.W` prefix (`48`) for 64-bit operands, and the `REX.B` / `REX.R` bits (`41`, `44`, `4C`, `49`) needed to reach `r8`–`r15`:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 25-2. Operand size and register extension in the encoding

SHASM

```
bits 64
    add al, bl          ; 8-bit
    add ax, bx          ; 16-bit: 66 prefix
    add eax, ebx        ; 32-bit: no prefix
    add rax, rbx        ; 64-bit: REX.W
    add r8, rbx         ; REX.W + REX.B
    add rax, r9         ; REX.W + REX.R
    add r12, r13        ; REX.W + REX.R + REX.B
    add eax, 1          ; imm8, sign-extended (83 /0 ib)
    add eax, 1000       ; imm32 (05 id: short form for eax)
    add ecx, 1000       ; imm32 (81 /0 id)
```

SHASM LISTING

```
     2 00000000 00D8                    add al, bl
     3 00000002 6601D8                  add ax, bx
     4 00000005 01D8                    add eax, ebx
     5 00000007 4801D8                  add rax, rbx
     6 0000000A 4901D8                  add r8, rbx
     7 0000000D 4C01C8                  add rax, r9
     8 00000010 4D01EC                  add r12, r13
     9 00000013 83C001                  add eax, 1
    10 00000016 05E8030000              add eax, 1000
    11 0000001B 81C1E8030000            add ecx, 1000
```

25.6.1 Reading a ModRM byte

Take `add rax, rbx`, encoded `48 01 D8`. `48` is REX with `W=1` (64-bit operands). `01` is the opcode *ADD r/m64, r64*. `D8` = `11 011 000` in binary: `mod = 11` (register operand), `reg = 011` (`rbx`, the source), `r/m = 000` (`rax`, the destination). The same opcode with a memory operand, `add [rax], rbx`, has `mod = 00` and encodes as `48 01 18`.

25.7 INSTRUCTIONS USED BY ASH

The code generator of ASH uses a small subset of the instruction set. Knowing these instructions is enough to read any compiled ASH function:

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 25-5. Instructions found in compiled ASH code

Instruction	Use in compiled code
`mov`, `movzx`, `movsx`, `movsxd`	Moves between homes, loads, stores, widening.
`lea`	Addresses of globals, strings and frame slots.
`add`, `sub`, `imul`, `and`, `or`, `xor`	Arithmetic and bitwise operators.
`cqo` + `idiv`, `xor edx, edx` + `div`	Division and remainder.
`shl`, `sar`, `shr`	Shifts by constants.
`cmp`, `jcc`, `jmp`	Conditions and loops.
`call`, `ret`, `push`, `pop`, `leave`	Calls, prologues and epilogues.
`and rsp, -16`	Stack alignment at `_start`.
`syscall`, `int 0x80`	System calls (in the runtime and libraries).

Appendix Appendix A describes these and many more instructions with all the forms that `shasm` accepts.

25.8 CALLING CONVENTIONS AT A GLANCE

Table 25-6. Calling conventions

Aspect	System V x86-64	Microsoft x64	cdecl (i386)
Integer arguments	`rdi rsi rdx rcx r8 r9`, then stack	`rcx rdx r8 r9`, then stack	stack, right to left
Return value	`rax` (`rdx:rax`)	`rax`	`eax` (`edx:eax`)
Callee-saved	`rbx rbp r12-r15`	`rbx rbp rdi rsi r12-r15`	`ebx esi edi ebp`
Stack cleanup	caller	caller	caller
Shadow space	none	32 bytes	none
Stack alignment	16 bytes at call	16 bytes at call	4 (16 on Linux)

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 26

THE SHASM ASSEMBLER

`shasm` is the x86 assembler of The Ashley Project. It is embedded in `ashc`, where it encodes every `asm` block, and it is also available as a stand-alone tool that assembles complete source files into flat binaries, Intel HEX files and ELF or COFF objects. Its syntax follows the conventions of NASM, so most NASM sources assemble unchanged. This chapter is a complete guide to the stand-alone assembler: command line, source syntax, expressions, directives, the preprocessor, output formats and listings.

26.1 RUNNING SHASM

`shasm` 1.2.0 is part of the compiler's jar. It is started with the Java runtime:

```shell
java -cp out/ashc-1.0.0.jar shasm.Main [options] file.asm
```

A two-line wrapper script makes it available as a command:

```shell
#!/bin/sh
# save as ~/.local/bin/shasm and make it executable
exec java -cp "$HOME/.local/lib/ashc-1.0.0.jar" shasm.Main "$@"
```

All the sessions of this chapter use such a wrapper.

```shell session
$ shasm --lang en --help
shasm 1.2.0 THE ASHLEY PROJECT VERSION
This file is included by default in the ADK (Ashley Development Kit).

Usage: shasm [options] file.asm
  -o <file>     output file (default: <input>.bin)
  -f <fmt>         format: bin, hex, elf32, elf64, win32, win64
  -l <file>     generate a listing (address, bytes, source)
  -I <dir>         search directory for %include / incbin (repeatable)
  -D <sym>[=<val>] define a preprocessor macro
  -b <16|32|64>    initial BITS mode (default 16; 32/64 in object formats)
  -w               treat warnings as errors
  --lang <xx>      message language (de, en, es, it, pt)
  -v, --version    show version
  -h, --help       show this help
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 26-1. Command line options of shasm

Option	Meaning
`-o` *file*	Output file. Default: the input name with `.bin`, `.hex`, `.o` or `.obj`.
`-f` *fmt*	Output format: `bin` (default), `hex`, `elf32`, `elf64`, `win32`, `win64`.
`-l` *file*	Write a listing: line number, address, encoded bytes and source of every line.
`-I` *dir*	Add a directory to the search path of `%include` and `incbin` (repeatable).
`-D` *sym*[=*val*]	Define a preprocessor symbol, as `%define sym val` (the value defaults to 1).
`-b` *16|32|64*	Initial `bits` mode. Default: 16 for `bin` and `hex`, 32 or 64 according to the object format.
`-w`	Treat warnings as errors.
`--lang` *xx*	Language of messages: `en`, `es`, `de`, `it`, `pt`. The default is taken from the environment.
`--po` *file*	Load additional message translations from a gettext `.po` file.
`-v`, `--version`	Print the version.
`-h`, `--help`	Print the help text.

26.2 SOURCE SYNTAX

A source line has the general form:

```
[label:]  [prefix]  mnemonic  [operand [, operand ...]]   [; comment]
```

- **Labels** end with a colon. A label whose name begins with a dot (`.loop`) is *local*: it belongs to the most recent non-local label, so the same local name can be reused under different functions. A label without a colon is accepted when it is followed by a data directive (`msg db "x"`).
- **Mnemonics** and **register names** are case-insensitive. Labels and symbols are case-sensitive.
- **Operands** are registers, immediate expressions, or memory references in brackets. A size keyword (`byte`, `word`, `dword`, `qword`, `tword`, `oword`) gives the size of a memory operand when the other operand does not; `rel` and `abs` select RIP-relative or absolute addressing.
- **Comments** begin with `;`. Lines beginning with `;@` and blocks between `;*` and `*;` are documentation comments (*AshDoc*), recorded by the assembler and otherwise ignored.
- **Prefixes** `lock`, `rep`, `repe`, `repne`, `repz`, `repnz`, `xacquire`, `xrelease`, `a16`/`a32`/`a64`, `o16`/`o32`/`o64` may precede an instruction.
- A line ending with `\` continues on the next line.

Copyright © 2026 Mallionic Inc. All rights reserved.

26.3 NUMBERS, CHARACTERS AND STRINGS

Table 26-2. Constant forms

Form	Examples
Decimal	`42`, `0d42`, `42d`, `1_000_000` (underscores are ignored)
Hexadecimal	`0x2A`, `0h2A`, `2Ah` (must start with a digit: `0FFh`)
Binary	`0b101010`, `0y101010`, `101010b`, `101010y`
Octal	`0o52`, `0q52`, `52o`, `52q`
Floating point	`1.5`, `3.0e10` (in `dd`, `dq`, `dt` data)
Character constant	`'A'` (65), `'ab'` (multi-character, little-endian)
String	`"text"`, `'text'`, `` `text with \n escapes` ``

26.4 EXPRESSIONS

Expressions are evaluated with 64-bit integers. Two special symbols are available: `$` is the address of the beginning of the current line, and `$$` the address of the beginning of the current section, so `$ - $$` is the current offset in the section.

Table 26-3. Expression operators, from lowest to highest precedence

Operators	Meaning
`||`, `^^`, `&&`	Logical or, xor, and (result 0 or 1)
`== != <> < <= > >=`	Comparisons (result 0 or 1)
`|`	Bitwise or
`^`	Bitwise exclusive or
`&`	Bitwise and
`<< >> <<< >>>`	Shifts: `>>` is logical, `>>>` arithmetic
`+ -`	Addition, subtraction
`* / // % %%`	Multiplication; unsigned and signed division; unsigned and signed remainder
unary `- + ~ !`	Negation, identity, complement, logical not

Example 26-1. Expressions, $ and $$

SHASM

```
bits 32
start:
    mov eax, (1 << 4) | 3          ; 19
    mov ebx, -7 // 2               ; signed division: -3
    mov ecx, 0x10 >> 1             ; 8
    mov edx, 101b + 17q + 0Ah      ; 5 + 15 + 10
    mov esi, end - start           ; size of the code (forward reference)
    mov edi, $ - $$                ; offset of this instruction
end:
```

Copyright © 2026 Mallionic Inc. All rights reserved.

SHASM LISTING

```
     3 00000000 B813000000              mov eax, (1 << 4) | 3
     4 00000005 BBFDFFFFFF              mov ebx, -7 // 2
     5 0000000A B908000000              mov ecx, 0x10 >> 1
     6 0000000F BA1E000000              mov edx, 101b + 17q + 0Ah
     7 00000014 BE1E000000              mov esi, end - start
     8 00000019 BF19000000              mov edi, $ - $$
```

26.5 DATA DIRECTIVES

Table 26-4. Data definition and reservation

Directive	Meaning
`db`, `dw`, `dd`, `dq`	Define bytes, words (2), doublewords (4), quadwords (8). Operands are expressions, strings (in `db`, one byte per character) or floating-point constants (`dd` single, `dq` double).
`dt`, `do`, `dy`, `dz`	Define 10, 16, 32 and 64-byte items.
`resb`, `resw`, `resd`, `resq`, `rest`, `reso`, `resy`, `resz`	Reserve uninitialized space (zeros in flat outputs, no file space in `.bss`).
`times` *n line*	Repeat a line *n* times; *n* may depend on `$` and `$$`.
`incbin "file"[, skip[, length]]`	Include the bytes of a file.
name `equ` *expr*	Define a constant symbol.
`?`	In a data list, an uninitialized item (zero).

Example 26-2. Data definitions

SHASM

```
bits 32
table:  dw 1, 2, 3
        align 4
text:   db "ASH", 0
pi:     dd 3.14159
big:    dq 0x1122334455667788
pad:    times 4 db 0xCC
count   equ ($ - table)
size:   dd count
```

SHASM LISTING

```
     2 00000000 010002000300            table:  dw 1, 2, 3
     3 00000006 9090                    align 4
     4 00000008 41534800                text:   db "ASH", 0
     5 0000000C D00F4940                pi:     dd 3.14159
     6 00000010 8877665544332211        big:    dq 0x1122334455667788
     7 00000018 CCCCCCCC                pad:    times 4 db 0xCC
     9 0000001C 1C000000                size:   dd count
```

Copyright © 2026 Mallionic Inc. All rights reserved.

26.6 ASSEMBLER DIRECTIVES

Table 26-5. Assembler directives

Directive	Meaning
`bits 16|32|64`, `use16`, `use32`, `use64`	Select the processor mode for the following lines. May be written `[bits 32]`.
`org` *address*	Origin of a flat binary: the address at which the first byte will be loaded. Must appear before any code.
`section` *name* [`align=`*n*] [`progbits`|`nobits`], `segment`	Switch to (or create) a section. In flat output, sections follow one another in order of appearance.
`global` *sym*	Export a symbol from an object file.
`extern` *sym*	Declare a symbol defined in another object; references become relocations.
`default rel` / `default abs`	In 64-bit mode, make memory references to symbols RIP-relative (recommended) or absolute.
`align` *n* [, *fill*]	Pad to a multiple of *n* (a power of two) with `nop` in code, or with the fill value.
`alignb` *n*	Pad with zeros (or reserve space in `nobits` sections).
`fill` *size* [, *value*] / `fill` *value* `-->` *size*	Pad the current section with *value* until it is *size* bytes long.
`bootsig`	Emit the boot signature bytes `55 AA`.
`cpu`, `float`, `common`, `static`, `warning`, `map`, `list`, `required`, `sectalign`, `group`, `export`, `import`	Accepted for compatibility; no effect.

26.7 THE PREPROCESSOR

The preprocessor runs before assembly and works line by line. Its directives start with `%`:

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 26-6. Preprocessor directives

Directive	Meaning
`%define` *name*[`(`*params*`)`] *body*	Single-line macro, optionally with parameters. `%idefine` is case-insensitive; `%xdefine` / `%ixdefine` expand the body at definition time.
`%undef` *name*	Remove a definition.
`%assign` *name expr* / `%iassign`	Define *name* as the numeric value of *expr*.
`%defstr` *name text*	Define *name* as a string.
`%strlen` *name string*	Define *name* as the length of the string.
`%macro` *name n*[`-`*m*][`+`] ... `%endmacro`	Multi-line macro with *n* to *m* parameters (`+`: the last one takes the rest of the line). Inside, `%1`, `%2` ... are the arguments, `%0` their number, and `%%label` a label unique to each expansion. `%imacro` is case-insensitive.
`%rep` *n* ... `%endrep`	Repeat a block *n* times.
`%if` *expr*, `%ifn`, `%elif`, `%elifn`, `%else`, `%endif`	Conditional assembly on an expression.
`%ifdef`, `%ifndef`, `%elifdef`, `%elifndef`	On whether a single-line macro is defined.
`%ifmacro`, `%ifnmacro`	On whether a multi-line macro is defined.
`%ifidn`, `%ifidni`, `%ifnidn`, `%elifidn`	On whether two texts are identical (`i`: case-insensitive).
`%ifid`, `%ifnum`, `%ifstr`, `%ifempty`, `%elifnum`, `%elifstr`	On the kind of a token.
`%include "file"`	Insert another source file (searched in the directory of the current file and the `-I` directories).
`%error`, `%fatal`, `%warning` *text*	Report a message.
`%line`, `%pragma`, `%clear`, `%null`	Accepted for compatibility.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 26-3. Macros, repetition and conditional assembly

SHASM

```
bits 16
org 0x100
%define COLOR 0x1F
%assign COUNT 3

%macro PUTC 1               ; print one character through BIOS
    mov al, %1
    mov ah, 0x0E
    int 0x10
%endmacro

%macro DELAY 0
    mov cx, 0xFFFF
%%wait:                     ; %% makes the label unique per expansion
    loop %%wait
%endmacro

start:
    PUTC 'O'
    PUTC 'K'
    DELAY
    DELAY
%rep COUNT
    nop
%endrep
%if COUNT > 2
    mov bl, COLOR
%endif
    ret
```

SHASM LISTING

```
    19 00000100 B04F                    mov al, 'O'
    20 00000102 B40E                    mov ah, 0x0E
    21 00000104 CD10                    int 0x10
    20 00000106 B04B                    mov al, 'K'
    21 00000108 B40E                    mov ah, 0x0E
    22 0000010A CD10                    int 0x10
    21 0000010C B9FFFF                  mov cx, 0xFFFF
    23 0000010F E2FE                    loop ..@3.wait
    22 00000111 B9FFFF                  mov cx, 0xFFFF
    24 00000114 E2FE                    loop ..@4.wait
    24 00000116 90                      nop
    24 00000117 90                      nop
    24 00000118 90                      nop
    27 00000119 B31F                    mov bl, 0x1F
    29 0000011B C3                      ret
```

26.7.1 Metadata directives

The directives `%definesdk`, `%author`, `%license`, `%docs`, `%mallionic`, `%comment`, `%link`, `%dev` and `%team` attach descriptive metadata to a source file. Their value may be a quoted string or a triple-quoted multi-line string. They generate no bytes; the assembler records them together with the AshDoc comments, for tools that document assembly sources.

Copyright © 2026 Mallionic Inc. All rights reserved.

SHASM

```
%author  "Mallionic Inc."
%license "GPL-3.0-or-later"
%comment """Boot sector of the demo image.
Prints a banner and halts."""
;@brief  entry point, called by the BIOS at 0000:7C00
```

26.8 OUTPUT FORMATS

Table 26-7. Output formats of shasm

Format	Output
`bin`	A flat binary. Sections are concatenated in order; addresses start at `org`. Default mode: 16 bits.
`hex`	The same image in Intel HEX format, for EPROM programmers and microcontroller loaders.
`elf32`	ELF32 relocatable object for i386, with `.text`, `.data`, `.bss` and user sections, a symbol table with `global` and `extern` symbols, and `R_386_*` relocations.
`elf64`	ELF64 relocatable object for x86-64 with `R_X86_64_*` relocations.
`win32`	COFF object for i386 Windows.
`win64`	COFF object for x64 Windows.

26.9 A LINUX PROGRAM IN ASSEMBLY

The following program uses the 64-bit Linux system call interface directly. It is assembled as an ELF64 object, linked with `ld` and executed:

Example 26-4. Hello world for Linux x86-64

SHASM

```
; hello.asm - Linux x86-64, no C library
bits 64
default rel

section .data
msg:    db "Hello from shasm!", 10
len     equ $ - msg

section .text
global _start
_start:
    mov eax, 1          ; sys_write
    mov edi, 1          ; fd 1 = stdout
    lea rsi, [msg]
    mov edx, len
    syscall
    mov eax, 60         ; sys_exit
    xor edi, edi        ; status 0
    syscall
```

SHASM LISTING

```
     6 00000024 48656C6C6F2066726F6D2073- msg:    db "Hello from shasm!", 10
    12 00000000 B801000000               mov eax, 1
    13 00000005 BF01000000               mov edi, 1
```

Copyright © 2026 Mallionic Inc. All rights reserved.

SHASM LISTING (CONTINUED)

```
    14 0000000A 488D3500000000          lea rsi, [msg]
    15 00000011 BA12000000              mov edx, len
    16 00000016 0F05                    syscall
    17 00000018 B83C000000              mov eax, 60
    18 0000001D 31FF                    xor edi, edi
    19 0000001F 0F05                    syscall
```

OUTPUT — EXIT STATUS 0

```
Hello from shasm!
```

Example 26-5. The same program for i386 Linux

SHASM

```
bits 32

section .data
msg:    db "Hello from 32-bit shasm!", 10
len     equ $ - msg

section .text
global _start
_start:
    mov eax, 4          ; sys_write
    mov ebx, 1
    mov ecx, msg
    mov edx, len
    int 0x80
    mov eax, 1          ; sys_exit
    mov ebx, 3
    int 0x80
```

SHASM LISTING

```
     4 00000024 48656C6C6F2066726F6D2033- msg:    db "Hello from 32-bit shasm!", 10
    10 00000000 B804000000              mov eax, 4
    11 00000005 BB01000000              mov ebx, 1
    12 0000000A B900000000              mov ecx, msg
    13 0000000F BA19000000              mov edx, len
    14 00000014 CD80                    int 0x80
    15 00000016 B801000000              mov eax, 1
    16 0000001B BB03000000              mov ebx, 3
    17 00000020 CD80                    int 0x80
```

OUTPUT — EXIT STATUS 3

```
Hello from 32-bit shasm!
```

26.10 A BOOT SECTOR

A BIOS boot sector is 512 bytes long, is loaded at address `0x7C00` and must end with the signature `55 AA`. The `times` directive pads the code to 510 bytes using `$` and `$$`, and `dw 0xAA55` (or `bootsig`) adds the signature:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 26-6. A complete boot sector

SHASM

```
; boot.asm - prints "ASH" with BIOS teletype output and halts
bits 16
org 0x7C00

start:
    cli
    xor ax, ax
    mov ds, ax
    mov ss, ax
    mov sp, 0x7C00
    sti
    mov si, banner
.next:
    lodsb                   ; al = [si], si++
    test al, al
    jz .halt
    mov ah, 0x0E            ; BIOS teletype
    mov bx, 0x0007
    int 0x10
    jmp .next
.halt:
    hlt
    jmp .halt

banner: db "ASH", 13, 10, 0

times 510 - ($ - $$) db 0   ; pad to 510 bytes
dw 0xAA55                   ; boot signature
```

SHASM LISTING

```
     6 00007C00 FA                      cli
     7 00007C01 31C0                    xor ax, ax
     8 00007C03 8ED8                    mov ds, ax
     9 00007C05 8ED0                    mov ss, ax
    10 00007C07 BC007C                  mov sp, 0x7C00
    11 00007C0A FB                      sti
    12 00007C0B BE1F7C                  mov si, banner
    14 00007C0E AC                      lodsb
    15 00007C0F 84C0                    test al, al
    16 00007C11 7409                    jz .halt
    17 00007C13 B40E                    mov ah, 0x0E
    18 00007C15 BB0700                  mov bx, 0x0007
    19 00007C18 CD10                    int 0x10
    20 00007C1A EBF2                    jmp .next
    22 00007C1C F4                      hlt
    23 00007C1D EBFD                    jmp .halt
    25 00007C1F 4153480D0A00            banner: db "ASH", 13, 10, 0
    27 00007C25 000000000000000000000000- times 510 - ($ - $$) db 0
    28 00007DFE 55AA                    dw 0xAA55
```

HEX DUMP (XXD)

```
000001e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
000001f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 55 aa  ..............U.
```

The image can be written to the first sector of a disk image (`dd if=boot.bin of=disk.img conv=notrunc`) and started in an emulator such as QEMU (`qemu-system-x86_64 -drive format=raw,file=disk.img`).

Copyright © 2026 Mallionic Inc. All rights reserved.

26.11 LISTINGS

The listing written with `-l` has one line per source line that produces bytes: the source line number, the address (relative to `org` or to the section), the encoded bytes in hexadecimal (long sequences are cut and marked with `-`), and the source text after macro expansion. Lines produced by a macro or a `%rep` carry the line number of the macro body. Listings are the most reliable way to check what the assembler produced, and the source of all the encodings shown in this manual.

26.12 DIAGNOSTICS

Errors are reported as `file:line: error: message`; the assembler stops with exit status 1 and writes no output file. Warnings are printed as `file:line: warning: message` and do not stop the assembly unless `-w` is given.

Example 26-7. An error message

SHASM

```
bits 64
    mov rax, 1
    mov rax, [rbx + rcx*3]       ; scale must be 1, 2, 4 or 8
```

SHASM DIAGNOSTICS

```
example.asm:3: error: invalid scale factor
```

26.13 SHASM INSIDE ASHC

When `ashc` assembles an `asm` block it runs `shasm` in flat mode with the current `bits` and origin, after replacing ASH variable names with their locations (Section 20.7). Everything described in this chapter is therefore available in `asm` blocks, except that each block is assembled separately: labels cannot be shared between blocks, and directives that change sections or the origin have no useful meaning inside a function.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 27

THE NLIB SYSTEM LIBRARY

`nlib` is the set of libraries that ships with ASH. The libraries are ordinary ASH source files, embedded in the compiler and imported with `import <name>;`. This chapter documents every library and every function: what it does, its parameters and result, its implementation, an example executed while this manual was built, and the machine code that the compiler generates for it.

27.1 OVERVIEW

Table 27-1. Libraries of nlib

Library	Needs libc	Contents
`<string>`	no	Memory and string functions, character classification, number/text conversion.
`<math>`	no	Integer mathematics and Q16.16 fixed-point arithmetic and trigonometry, without an FPU.
`<syscalls>`	no	Linux x86-64 system calls and simple output, without the C library.
`<args>`	yes	Command line arguments.
`<delay>`	yes	Sleeping for milliseconds or microseconds.
`<stdio>`	yes	Typed declarations of the C standard I/O functions.
`<stdlib>`	see Section 27.8	Wrappers of C standard library functions.
`<types>`	no	IEEE-754 infinity and NaN bit patterns.
`<stdbool>`	no	`true`, `false`, `TRUE`, `FALSE`.
`<null>`	no	`NULL`, `null`, `nil`, `NIL`.

The libraries marked "no" in the second column use no C function at all: they work with `--no-libc`, in flat binaries and on bare metal. Thanks to tree shaking (Section 19.3.1), only the functions a program uses end up in its object file.

27.1.1 Conventions

- Strings and memory are passed as raw byte pointers (`uint8*`). A string literal, the address of a buffer (`&buf`), memory returned by C and memory-mapped hardware can all be passed. No function of `<string>` checks bounds or allocates memory: the caller provides buffers of sufficient size.
- Search functions return an **index** (or -1), not a pointer.
- Predicates return `1` or `0`.
- A function of your program with the same name as an nlib function replaces it, because nlib functions have override right -1 (Section 19.4).
- Function names that begin with __ are internal helpers of a library and are not documented here.

Copyright © 2026 Mallionic Inc. All rights reserved.

27.2 <STRING>: STRINGS AND MEMORY

`import <string>;` provides 38 functions in six groups: memory (`memset`, `memcpy`, `memmove`, `memcmp`, `memchr`), length (`strlen`, `strnlen`), copying (`strcpy`, `strncpy`, `strcat`, `strncat`), comparison (`strcmp`, `strncmp`, `strcasecmp`), searching (`strchr`, `strrchr`, `strstr`, `strspn`, `strcspn`), characters (`isdigit` ... `tolower`), conversion (`str_to_int`, `atoi`, `uint_to_str`, `int_to_str`) and utilities (`str_reverse`, `str_upper`, `str_lower`, `str_equals`, `str_starts`, `str_ends`).

Several names coincide with C library functions. When `<string>` is imported, the ASH versions are used, which means a program can do text processing without the C library. Note the two differences from C: `memchr`, `strchr`, `strrchr` and `strstr` return indices, and none of the functions allocates memory.

27.2.1 memset

ASH

```
pub func memset(uint8* dst, int64 value, uint64 n): uint8*
```

Fills the first *n* bytes of the memory at *dst* with the low 8 bits of *value*. Works on any memory: buffers, arrays, memory returned by C or by `sys_mmap`, memory-mapped hardware.

Parameter	Description
`dst`	Address of the first byte to fill.
`value`	Byte value; only the low 8 bits are used.
`n`	Number of bytes to fill.

Returns. *dst*.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func memset(uint8* dst, int64 value, uint64 n): uint8* {
    uint64 i = 0;
    while (i < n) {
        dst[i] = value;
        i++;
    }
    return dst;
}
```

Example.

Example 27-1. Using memset

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[8] b;
    memset(&b, 0x41, 7);
    printf("%s\n", &b);
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
AAAAAAA
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000000d1 <memset>:
  d1:   55                      push   rbp
  d2:   48 89 e5                mov    rbp,rsp
  d5:   48 81 ec 30 00 00 00    sub    rsp,0x30
  dc:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  e0:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  e4:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  e8:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  ec:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
  f0:   b8 00 00 00 00          mov    eax,0x0
  f5:   49 89 c2                mov    r10,rax
  f8:   4c 89 d0                mov    rax,r10
  fb:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
  ff:   48 39 c8                cmp    rax,rcx
 102:   0f 83 2f 00 00 00       jae    137 <memset+0x66>
 108:   4c 89 d0                mov    rax,r10
 10b:   49 89 c3                mov    r11,rax
 10e:   4c 89 d8                mov    rax,r11
 111:   48 89 f9                mov    rcx,rdi
 114:   48 01 c8                add    rax,rcx
 117:   49 89 c3                mov    r11,rax
 11a:   48 89 f0                mov    rax,rsi
 11d:   41 88 c0                mov    r8b,al
 120:   4c 89 d8                mov    rax,r11
 123:   49 0f b6 c8             movzx  rcx,r8b
 127:   88 08                   mov    BYTE PTR [rax],cl
 129:   4c 89 d0                mov    rax,r10
 12c:   48 ff c0                inc    rax
 12f:   49 89 c2                mov    r10,rax
 132:   e9 c1 ff ff ff          jmp    f8 <memset+0x27>
 137:   48 89 f8                mov    rax,rdi
 13a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 13e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 142:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 146:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 14a:   48 89 ec                mov    rsp,rbp
 14d:   5d                      pop    rbp
 14e:   c3                      ret
```

27.2.2 memcpy

ASH

```
pub func memcpy(uint8* dst, uint8* src, uint64 n): uint8*
```

Copies *n* bytes from *src* to *dst*, from the first byte to the last. The regions must not overlap; use `memmove` when they may.

Parameter	Description
`dst`	Destination address.
`src`	Source address.
`n`	Number of bytes to copy.

Copyright © 2026 Mallionic Inc. All rights reserved.

Returns. *dst*.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func memcpy(uint8* dst, uint8* src, uint64 n): uint8* {
    uint64 i = 0;
    while (i < n) {
        dst[i] = src[i];
        i++;
    }
    return dst;
}
```

Example.

Example 27-2. Using memcpy

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[16] b;
    memcpy(&b, "copied text", 12);
    printf("%s\n", &b);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
copied text
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000000d1 <memcpy>:
  d1:   55                      push   rbp
  d2:   48 89 e5                mov    rbp,rsp
  d5:   48 81 ec 30 00 00 00    sub    rsp,0x30
  dc:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  e0:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  e4:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  e8:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  ec:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
  f0:   b8 00 00 00 00          mov    eax,0x0
  f5:   49 89 c2                mov    r10,rax
  f8:   4c 89 d0                mov    rax,r10
  fb:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
  ff:   48 39 c8                cmp    rax,rcx
 102:   0f 83 50 00 00 00       jae    158 <memcpy+0x87>
 108:   4c 89 d0                mov    rax,r10
 10b:   49 89 c3                mov    r11,rax
 10e:   4c 89 d8                mov    rax,r11
 111:   48 89 f9                mov    rcx,rdi
 114:   48 01 c8                add    rax,rcx
 117:   49 89 c3                mov    r11,rax
 11a:   4c 89 d0                mov    rax,r10
 11d:   49 89 c0                mov    r8,rax
 120:   4c 89 c0                mov    rax,r8
 123:   48 89 f1                mov    rcx,rsi
 126:   48 01 c8                add    rax,rcx
 129:   49 89 c0                mov    r8,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 12c:   b0 00                   mov    al,0x0
 12e:   41 88 c1                mov    r9b,al
 131:   4c 89 c0                mov    rax,r8
 134:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 138:   41 88 c9                mov    r9b,cl
 13b:   44 88 c8                mov    al,r9b
 13e:   41 88 c4                mov    r12b,al
 141:   4c 89 d8                mov    rax,r11
 144:   49 0f b6 cc             movzx  rcx,r12b
 148:   88 08                   mov    BYTE PTR [rax],cl
 14a:   4c 89 d0                mov    rax,r10
 14d:   48 ff c0                inc    rax
 150:   49 89 c2                mov    r10,rax
 153:   e9 a0 ff ff ff          jmp    f8 <memcpy+0x27>
 158:   48 89 f8                mov    rax,rdi
 15b:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 15f:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 163:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 167:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 16b:   48 89 ec                mov    rsp,rbp
 16e:   5d                      pop    rbp
 16f:   c3                      ret
```

27.2.3 memmove

ASH

```
pub func memmove(uint8* dst, uint8* src, uint64 n): uint8*
```

Copies *n* bytes from *src* to *dst* correctly even when the two regions overlap: it copies forwards when *dst* is below *src* and backwards otherwise.

Parameter	Description
`dst`	Destination address.
`src`	Source address.
`n`	Number of bytes to copy.

Returns. *dst*.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func memmove(uint8* dst, uint8* src, uint64 n): uint8* {
    if (dst < src) {
        uint64 i = 0;
        while (i < n) {
            dst[i] = src[i];
            i++;
        }
        return dst;
    }
    uint64 j = n;
    while (j > 0) {
        j--;
        dst[j] = src[j];
    }
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/STRING.ASH (CONTINUED)

```
    return dst;
}
```

Example.

Example 27-3. Using memmove

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[16] b;
    strcpy(&b, "abcdef");
    uint8* p = &b;
    memmove(p + 2, p, 4);      // overlapping copy
    printf("%s\n", &b);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
ababcd
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000001b9 <memmove>:
 1b9:   55                      push   rbp
 1ba:   48 89 e5                mov    rbp,rsp
 1bd:   48 81 ec 40 00 00 00    sub    rsp,0x40
 1c4:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 1c8:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 1cc:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 1d0:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 1d4:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
 1d8:   48 89 f8                mov    rax,rdi
 1db:   48 89 f1                mov    rcx,rsi
 1de:   48 39 c8                cmp    rax,rcx
 1e1:   0f 8d 80 00 00 00       jge    267 <memmove+0xae>
 1e7:   b8 00 00 00 00          mov    eax,0x0
 1ec:   49 89 c2                mov    r10,rax
 1ef:   4c 89 d0                mov    rax,r10
 1f2:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 1f6:   48 39 c8                cmp    rax,rcx
 1f9:   0f 83 50 00 00 00       jae    24f <memmove+0x96>
 1ff:   4c 89 d0                mov    rax,r10
 202:   49 89 c3                mov    r11,rax
 205:   4c 89 d8                mov    rax,r11
 208:   48 89 f9                mov    rcx,rdi
 20b:   48 01 c8                add    rax,rcx
 20e:   49 89 c3                mov    r11,rax
 211:   4c 89 d0                mov    rax,r10
 214:   49 89 c0                mov    r8,rax
 217:   4c 89 c0                mov    rax,r8
 21a:   48 89 f1                mov    rcx,rsi
 21d:   48 01 c8                add    rax,rcx
 220:   49 89 c0                mov    r8,rax
 223:   b0 00                   mov    al,0x0
 225:   41 88 c1                mov    r9b,al
 228:   4c 89 c0                mov    rax,r8
 22b:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
22f:   41 88 c9                mov    r9b,cl
232:   44 88 c8                mov    al,r9b
235:   41 88 c4                mov    r12b,al
238:   4c 89 d8                mov    rax,r11
23b:   49 0f b6 cc             movzx  rcx,r12b
23f:   88 08                   mov    BYTE PTR [rax],cl
241:   4c 89 d0                mov    rax,r10
244:   48 ff c0                inc    rax
247:   49 89 c2                mov    r10,rax
24a:   e9 a0 ff ff ff          jmp    1ef <memmove+0x36>
24f:   48 89 f8                mov    rax,rdi
252:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
256:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
25a:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
25e:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
262:   48 89 ec                mov    rsp,rbp
265:   5d                      pop    rbp
266:   c3                      ret
267:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
26b:   49 89 c5                mov    r13,rax
26e:   4c 89 e8                mov    rax,r13
271:   b9 00 00 00 00          mov    ecx,0x0
276:   48 39 c8                cmp    rax,rcx
279:   0f 86 51 00 00 00       jbe    2d0 <memmove+0x117>
27f:   4c 89 e8                mov    rax,r13
282:   48 ff c8                dec    rax
285:   49 89 c5                mov    r13,rax
288:   4c 89 e8                mov    rax,r13
28b:   49 89 c6                mov    r14,rax
28e:   4c 89 f0                mov    rax,r14
291:   48 89 f9                mov    rcx,rdi
294:   48 01 c8                add    rax,rcx
297:   49 89 c6                mov    r14,rax
29a:   4c 89 e8                mov    rax,r13
29d:   49 89 c7                mov    r15,rax
2a0:   4c 89 f8                mov    rax,r15
2a3:   48 89 f1                mov    rcx,rsi
2a6:   48 01 c8                add    rax,rcx
2a9:   49 89 c7                mov    r15,rax
2ac:   b0 00                   mov    al,0x0
2ae:   88 45 d0                mov    BYTE PTR [rbp-0x30],al
2b1:   4c 89 f8                mov    rax,r15
2b4:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
2b8:   88 4d d0                mov    BYTE PTR [rbp-0x30],cl
2bb:   8a 45 d0                mov    al,BYTE PTR [rbp-0x30]
2be:   88 45 c8                mov    BYTE PTR [rbp-0x38],al
2c1:   4c 89 f0                mov    rax,r14
2c4:   48 0f b6 4d c8          movzx  rcx,BYTE PTR [rbp-0x38]
2c9:   88 08                   mov    BYTE PTR [rax],cl
2cb:   e9 9e ff ff ff          jmp    26e <memmove+0xb5>
2d0:   48 89 f8                mov    rax,rdi
2d3:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
2d7:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
2db:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
2df:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
2e3:   48 89 ec                mov    rsp,rbp
2e6:   5d                      pop    rbp
2e7:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.2.4 memcmp

```ash
pub func memcmp(uint8* a, uint8* b, uint64 n): int64
```

Compares the first *n* bytes of two memory regions, byte by byte, as unsigned values.

Parameter	Description
a	First region.
b	Second region.
n	Number of bytes to compare.

Returns. 0 if the regions are equal; otherwise the difference `a[i] - b[i]` of the first differing bytes (negative if *a* is smaller).

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```ash
pub func memcmp(uint8* a, uint8* b, uint64 n): int64 {
    uint64 i = 0;
    while (i < n) {
        int64 x = a[i];
        int64 y = b[i];
        if (x != y) {
            return x - y;
        }
        i++;
    }
    return 0;
}
```

Example.

Example 27-4. Using memcmp

```ash
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", memcmp("abc", "abc", 3), memcmp("abc", "abd", 3), memcmp("b", "a",
↪ 1));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0 -1 1
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <memcmp>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
1b:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
1f:   b8 00 00 00 00          mov    eax,0x0
24:   49 89 c2                mov    r10,rax
27:   4c 89 d0                mov    rax,r10
2a:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
2e:   48 39 c8                cmp    rax,rcx
31:   0f 83 9b 00 00 00       jae    d2 <memcmp+0xd2>
37:   4c 89 d0                mov    rax,r10
3a:   49 89 c3                mov    r11,rax
3d:   4c 89 d8                mov    rax,r11
40:   48 89 f9                mov    rcx,rdi
43:   48 01 c8                add    rax,rcx
46:   49 89 c3                mov    r11,rax
49:   b0 00                   mov    al,0x0
4b:   41 88 c0                mov    r8b,al
4e:   4c 89 d8                mov    rax,r11
51:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
55:   41 88 c8                mov    r8b,cl
58:   44 88 c0                mov    al,r8b
5b:   0f b6 c0                movzx  eax,al
5e:   49 89 c1                mov    r9,rax
61:   4c 89 d0                mov    rax,r10
64:   49 89 c4                mov    r12,rax
67:   4c 89 e0                mov    rax,r12
6a:   48 89 f1                mov    rcx,rsi
6d:   48 01 c8                add    rax,rcx
70:   49 89 c4                mov    r12,rax
73:   b0 00                   mov    al,0x0
75:   41 88 c5                mov    r13b,al
78:   4c 89 e0                mov    rax,r12
7b:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
7f:   41 88 cd                mov    r13b,cl
82:   44 88 e8                mov    al,r13b
85:   0f b6 c0                movzx  eax,al
88:   49 89 c6                mov    r14,rax
8b:   4c 89 c8                mov    rax,r9
8e:   4c 89 f1                mov    rcx,r14
91:   48 39 c8                cmp    rax,rcx
94:   0f 84 2a 00 00 00       je     c4 <memcmp+0xc4>
9a:   4c 89 c8                mov    rax,r9
9d:   49 89 c7                mov    r15,rax
a0:   4c 89 f8                mov    rax,r15
a3:   4c 89 f1                mov    rcx,r14
a6:   48 29 c8                sub    rax,rcx
a9:   49 89 c7                mov    r15,rax
ac:   4c 89 f8                mov    rax,r15
af:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
b3:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
b7:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
bb:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
bf:   48 89 ec                mov    rsp,rbp
c2:   5d                      pop    rbp
c3:   c3                      ret
c4:   4c 89 d0                mov    rax,r10
c7:   48 ff c0                inc    rax
ca:   49 89 c2                mov    r10,rax
cd:   e9 55 ff ff ff          jmp    27 <memcmp+0x27>
d2:   48 31 c0                xor    rax,rax
d5:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
d9:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  dd:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  e1:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  e5:   48 89 ec                mov    rsp,rbp
  e8:   5d                      pop    rbp
  e9:   c3                      ret
```

27.2.5 memchr

ASH

```
pub func memchr(uint8* p, int64 c, uint64 n): int64
```

Searches the first *n* bytes of *p* for the byte *c*.

Parameter	Description
p	Memory to search.
c	Byte to find (low 8 bits).
n	Number of bytes to examine.

Returns. The **index** of the first occurrence, or -1 if the byte does not occur. (C's `memchr` returns a pointer; the ASH version returns an index, which is easier to use without pointer arithmetic.)

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func memchr(uint8* p, int64 c, uint64 n): int64 {
    uint64 i = 0;
    int64 want = c & 255;
    while (i < n) {
        if (p[i] == want) {
            return i;
        }
        i++;
    }
    return -1;
}
```

Example.

Example 27-5. Using memchr

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", memchr("hello", 'l', 5), memchr("hello", 'z', 5));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
2 -1
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <memchr>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
  1f:   b8 00 00 00 00          mov    eax,0x0
  24:   49 89 c2                mov    r10,rax
  27:   48 89 f0                mov    rax,rsi
  2a:   49 89 c3                mov    r11,rax
  2d:   4c 89 d8                mov    rax,r11
  30:   48 81 e0 ff 00 00 00    and    rax,0xff
  37:   49 89 c3                mov    r11,rax
  3a:   4c 89 d0                mov    rax,r10
  3d:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
  41:   48 39 c8                cmp    rax,rcx
  44:   0f 83 5c 00 00 00       jae    a6 <memchr+0xa6>
  4a:   4c 89 d0                mov    rax,r10
  4d:   49 89 c0                mov    r8,rax
  50:   4c 89 c0                mov    rax,r8
  53:   48 89 f9                mov    rcx,rdi
  56:   48 01 c8                add    rax,rcx
  59:   49 89 c0                mov    r8,rax
  5c:   b0 00                   mov    al,0x0
  5e:   41 88 c1                mov    r9b,al
  61:   4c 89 c0                mov    rax,r8
  64:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
  68:   41 88 c9                mov    r9b,cl
  6b:   44 88 c8                mov    al,r9b
  6e:   41 88 c4                mov    r12b,al
  71:   41 0f b6 c4             movzx  eax,r12b
  75:   44 89 d9                mov    ecx,r11d
  78:   39 c8                   cmp    eax,ecx
  7a:   0f 85 18 00 00 00       jne    98 <memchr+0x98>
  80:   4c 89 d0                mov    rax,r10
  83:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  87:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  8b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  8f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  93:   48 89 ec                mov    rsp,rbp
  96:   5d                      pop    rbp
  97:   c3                      ret
  98:   4c 89 d0                mov    rax,r10
  9b:   48 ff c0                inc    rax
  9e:   49 89 c2                mov    r10,rax
  a1:   e9 94 ff ff ff          jmp    3a <memchr+0x3a>
  a6:   b8 00 00 00 00          mov    eax,0x0
  ab:   49 89 c5                mov    r13,rax
  ae:   4c 89 e8                mov    rax,r13
  b1:   48 83 e8 01             sub    rax,0x1
  b5:   49 89 c5                mov    r13,rax
  b8:   4c 89 e8                mov    rax,r13
  bb:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  bf:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  c3:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  c7:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  cb:   48 89 ec                mov    rsp,rbp
  ce:   5d                      pop    rbp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  cf:   c3                      ret
```

27.2.6 strlen

ASH

```
pub func strlen(uint8* s): uint64
```

Counts the bytes of a NUL-terminated string, not including the terminating NUL.

Parameter	Description
s	Address of the string.

Returns. The length in bytes.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strlen(uint8* s): uint64 {
    uint64 n = 0;
    while (s[n] != 0) {
        n++;
    }
    return n;
}
```

Example.

Example 27-6. Using strlen

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", strlen("ASH"), strlen(""));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <strlen>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 00 00 00 00          mov    eax,0x0
  20:   49 89 c2                mov    r10,rax
  23:   4c 89 d0                mov    rax,r10
  26:   49 89 c3                mov    r11,rax
  29:   4c 89 d8                mov    rax,r11
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  2c:   48 89 f9                mov    rcx,rdi
  2f:   48 01 c8                add    rax,rcx
  32:   49 89 c3                mov    r11,rax
  35:   b0 00                   mov    al,0x0
  37:   41 88 c0                mov    r8b,al
  3a:   4c 89 d8                mov    rax,r11
  3d:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
  41:   41 88 c8                mov    r8b,cl
  44:   44 88 c0                mov    al,r8b
  47:   41 88 c1                mov    r9b,al
  4a:   41 0f b6 c1             movzx  eax,r9b
  4e:   b9 00 00 00 00          mov    ecx,0x0
  53:   39 c8                   cmp    eax,ecx
  55:   0f 84 0e 00 00 00       je     69 <strlen+0x69>
  5b:   4c 89 d0                mov    rax,r10
  5e:   48 ff c0                inc    rax
  61:   49 89 c2                mov    r10,rax
  64:   e9 ba ff ff ff          jmp    23 <strlen+0x23>
  69:   4c 89 d0                mov    rax,r10
  6c:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  70:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  74:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  78:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  7c:   48 89 ec                mov    rsp,rbp
  7f:   5d                      pop    rbp
  80:   c3                      ret
```

27.2.7 strnlen

ASH

```
pub func strnlen(uint8* s, uint64 max): uint64
```

Like `strlen`, but examines at most *max* bytes. Safe for buffers that may not contain a NUL.

Parameter	Description
`s`	Address of the string.
`max`	Maximum number of bytes to examine.

Returns. The length, or *max* if no NUL was found among the first *max* bytes.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strnlen(uint8* s, uint64 max): uint64 {
    uint64 n = 0;
    while (n < max) {
        if (s[n] == 0) {
            return n;
        }
        n++;
    }
    return n;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-7. Using strnlen

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", strnlen("language", 4), strnlen("ab", 10));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
4 2
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <strnlen>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 00 00 00 00          mov    eax,0x0
  20:   49 89 c2                mov    r10,rax
  23:   4c 89 d0                mov    rax,r10
  26:   48 89 f1                mov    rcx,rsi
  29:   48 39 c8                cmp    rax,rcx
  2c:   0f 83 5e 00 00 00       jae    90 <strnlen+0x90>
  32:   4c 89 d0                mov    rax,r10
  35:   49 89 c3                mov    r11,rax
  38:   4c 89 d8                mov    rax,r11
  3b:   48 89 f9                mov    rcx,rdi
  3e:   48 01 c8                add    rax,rcx
  41:   49 89 c3                mov    r11,rax
  44:   b0 00                   mov    al,0x0
  46:   41 88 c0                mov    r8b,al
  49:   4c 89 d8                mov    rax,r11
  4c:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
  50:   41 88 c8                mov    r8b,cl
  53:   44 88 c0                mov    al,r8b
  56:   41 88 c1                mov    r9b,al
  59:   41 0f b6 c1             movzx  eax,r9b
  5d:   b9 00 00 00 00          mov    ecx,0x0
  62:   39 c8                   cmp    eax,ecx
  64:   0f 85 18 00 00 00       jne    82 <strnlen+0x82>
  6a:   4c 89 d0                mov    rax,r10
  6d:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  71:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  75:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  79:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  7d:   48 89 ec                mov    rsp,rbp
  80:   5d                      pop    rbp
  81:   c3                      ret
  82:   4c 89 d0                mov    rax,r10
  85:   48 ff c0                inc    rax
  88:   49 89 c2                mov    r10,rax
  8b:   e9 93 ff ff ff          jmp    23 <strnlen+0x23>
  90:   4c 89 d0                mov    rax,r10
  93:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  97:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  9b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  9f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  a3:   48 89 ec                mov    rsp,rbp
  a6:   5d                      pop    rbp
  a7:   c3                      ret
```

27.2.8 strcpy

ASH

```
pub func strcpy(uint8* dst, uint8* src): uint8*
```

Copies the string *src*, including its terminating NUL, to *dst*. *dst* must be large enough.

Parameter	Description
`dst`	Destination buffer.
`src`	Source string.

Returns. *dst*.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strcpy(uint8* dst, uint8* src): uint8* {
    uint64 i = 0;
    while (src[i] != 0) {
        dst[i] = src[i];
        i++;
    }
    dst[i] = 0;
    return dst;
}
```

Example.

Example 27-8. Using strcpy

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    strcpy(&b, "copy me");
    printf("%s\n", &b);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
copy me
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000000d1 <strcpy>:
  d1:   55                      push   rbp
  d2:   48 89 e5                mov    rbp,rsp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 d5:   48 81 ec 30 00 00 00    sub    rsp,0x30
 dc:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 e0:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 e4:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 e8:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 ec:   b8 00 00 00 00          mov    eax,0x0
 f1:   49 89 c2                mov    r10,rax
 f4:   4c 89 d0                mov    rax,r10
 f7:   49 89 c3                mov    r11,rax
 fa:   4c 89 d8                mov    rax,r11
 fd:   48 89 f1                mov    rcx,rsi
100:   48 01 c8                add    rax,rcx
103:   49 89 c3                mov    r11,rax
106:   b0 00                   mov    al,0x0
108:   41 88 c0                mov    r8b,al
10b:   4c 89 d8                mov    rax,r11
10e:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
112:   41 88 c8                mov    r8b,cl
115:   44 88 c0                mov    al,r8b
118:   41 88 c1                mov    r9b,al
11b:   41 0f b6 c1             movzx  eax,r9b
11f:   b9 00 00 00 00          mov    ecx,0x0
124:   39 c8                   cmp    eax,ecx
126:   0f 84 50 00 00 00       je     17c <strcpy+0xab>
12c:   4c 89 d0                mov    rax,r10
12f:   49 89 c4                mov    r12,rax
132:   4c 89 e0                mov    rax,r12
135:   48 89 f9                mov    rcx,rdi
138:   48 01 c8                add    rax,rcx
13b:   49 89 c4                mov    r12,rax
13e:   4c 89 d0                mov    rax,r10
141:   49 89 c5                mov    r13,rax
144:   4c 89 e8                mov    rax,r13
147:   48 89 f1                mov    rcx,rsi
14a:   48 01 c8                add    rax,rcx
14d:   49 89 c5                mov    r13,rax
150:   b0 00                   mov    al,0x0
152:   41 88 c6                mov    r14b,al
155:   4c 89 e8                mov    rax,r13
158:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
15c:   41 88 ce                mov    r14b,cl
15f:   44 88 f0                mov    al,r14b
162:   41 88 c7                mov    r15b,al
165:   4c 89 e0                mov    rax,r12
168:   49 0f b6 cf             movzx  rcx,r15b
16c:   88 08                   mov    BYTE PTR [rax],cl
16e:   4c 89 d0                mov    rax,r10
171:   48 ff c0                inc    rax
174:   49 89 c2                mov    r10,rax
177:   e9 78 ff ff ff          jmp    f4 <strcpy+0x23>
17c:   4c 89 d0                mov    rax,r10
17f:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
183:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
187:   48 89 f9                mov    rcx,rdi
18a:   48 01 c8                add    rax,rcx
18d:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
191:   b0 00                   mov    al,0x0
193:   88 45 d0                mov    BYTE PTR [rbp-0x30],al
196:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
19a:   48 0f b6 4d d0          movzx  rcx,BYTE PTR [rbp-0x30]
19f:   88 08                   mov    BYTE PTR [rax],cl
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 1a1:   48 89 f8                mov    rax,rdi
 1a4:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 1a8:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 1ac:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 1b0:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 1b4:   48 89 ec                mov    rsp,rbp
 1b7:   5d                      pop    rbp
 1b8:   c3                      ret
```

27.2.9 strncpy

ASH

```
pub func strncpy(uint8* dst, uint8* src, uint64 n): uint8*
```

Copies at most *n* bytes of *src* to *dst*. If *src* is shorter than *n*, the rest of the *n* bytes is filled with NUL, as in C. If *src* is *n* bytes or longer, *dst* is **not** NUL-terminated.

Parameter	Description
`dst`	Destination buffer of at least *n* bytes.
`src`	Source string.
`n`	Number of bytes to write.

Returns. *dst*.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strncpy(uint8* dst, uint8* src, uint64 n): uint8* {
    uint64 i = 0;
    while (i < n) {
        if (src[i] == 0) {
            break;
        }
        dst[i] = src[i];
        i++;
    }
    while (i < n) {
        dst[i] = 0;
        i++;
    }
    return dst;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-9. Using strncpy

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[16] b;
    strncpy(&b, "truncate", 5);
    b[5] = 0;
    printf("%s\n", &b);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
trunc
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000000d1 <strncpy>:
  d1:   55                      push   rbp
  d2:   48 89 e5                mov    rbp,rsp
  d5:   48 81 ec 40 00 00 00    sub    rsp,0x40
  dc:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  e0:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  e4:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  e8:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  ec:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
  f0:   b8 00 00 00 00          mov    eax,0x0
  f5:   49 89 c2                mov    r10,rax
  f8:   4c 89 d0                mov    rax,r10
  fb:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
  ff:   48 39 c8                cmp    rax,rcx
 102:   0f 83 8d 00 00 00       jae    195 <strncpy+0xc4>
 108:   4c 89 d0                mov    rax,r10
 10b:   49 89 c3                mov    r11,rax
 10e:   4c 89 d8                mov    rax,r11
 111:   48 89 f1                mov    rcx,rsi
 114:   48 01 c8                add    rax,rcx
 117:   49 89 c3                mov    r11,rax
 11a:   b0 00                   mov    al,0x0
 11c:   41 88 c0                mov    r8b,al
 11f:   4c 89 d8                mov    rax,r11
 122:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 126:   41 88 c8                mov    r8b,cl
 129:   44 88 c0                mov    al,r8b
 12c:   41 88 c1                mov    r9b,al
 12f:   41 0f b6 c1             movzx  eax,r9b
 133:   b9 00 00 00 00          mov    ecx,0x0
 138:   39 c8                   cmp    eax,ecx
 13a:   0f 85 05 00 00 00       jne    145 <strncpy+0x74>
 140:   e9 50 00 00 00          jmp    195 <strncpy+0xc4>
 145:   4c 89 d0                mov    rax,r10
 148:   49 89 c4                mov    r12,rax
 14b:   4c 89 e0                mov    rax,r12
 14e:   48 89 f9                mov    rcx,rdi
 151:   48 01 c8                add    rax,rcx
 154:   49 89 c4                mov    r12,rax
 157:   4c 89 d0                mov    rax,r10
 15a:   49 89 c5                mov    r13,rax
 15d:   4c 89 e8                mov    rax,r13
 160:   48 89 f1                mov    rcx,rsi
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
163:   48 01 c8                add    rax,rcx
166:   49 89 c5                mov    r13,rax
169:   b0 00                   mov    al,0x0
16b:   41 88 c6                mov    r14b,al
16e:   4c 89 e8                mov    rax,r13
171:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
175:   41 88 ce                mov    r14b,cl
178:   44 88 f0                mov    al,r14b
17b:   41 88 c7                mov    r15b,al
17e:   4c 89 e0                mov    rax,r12
181:   49 0f b6 cf             movzx  rcx,r15b
185:   88 08                   mov    BYTE PTR [rax],cl
187:   4c 89 d0                mov    rax,r10
18a:   48 ff c0                inc    rax
18d:   49 89 c2                mov    r10,rax
190:   e9 63 ff ff ff          jmp    f8 <strncpy+0x27>
195:   4c 89 d0                mov    rax,r10
198:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
19c:   48 39 c8                cmp    rax,rcx
19f:   0f 83 33 00 00 00       jae    1d8 <strncpy+0x107>
1a5:   4c 89 d0                mov    rax,r10
1a8:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
1ac:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
1b0:   48 89 f9                mov    rcx,rdi
1b3:   48 01 c8                add    rax,rcx
1b6:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
1ba:   b0 00                   mov    al,0x0
1bc:   88 45 c8                mov    BYTE PTR [rbp-0x38],al
1bf:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
1c3:   48 0f b6 4d c8          movzx  rcx,BYTE PTR [rbp-0x38]
1c8:   88 08                   mov    BYTE PTR [rax],cl
1ca:   4c 89 d0                mov    rax,r10
1cd:   48 ff c0                inc    rax
1d0:   49 89 c2                mov    r10,rax
1d3:   e9 bd ff ff ff          jmp    195 <strncpy+0xc4>
1d8:   48 89 f8                mov    rax,rdi
1db:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
1df:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
1e3:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
1e7:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
1eb:   48 89 ec                mov    rsp,rbp
1ee:   5d                      pop    rbp
1ef:   c3                      ret
```

27.2.10 strcat

ASH

```
pub func strcat(uint8* dst, uint8* src): uint8*
```

Appends the string *src* to the end of the string in *dst*, including the terminating NUL.

Parameter	Description
`dst`	Buffer holding a string, with room for the result.
`src`	String to append.

Returns. *dst*.

Copyright © 2026 Mallionic Inc. All rights reserved.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strcat(uint8* dst, uint8* src): uint8* {
    uint64 d = strlen(dst);
    uint64 i = 0;
    while (src[i] != 0) {
        dst[d + i] = src[i];
        i++;
    }
    dst[d + i] = 0;
    return dst;
}
```

Example.

Example 27-10. Using strcat

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    strcpy(&b, "Hello, ");
    strcat(&b, "world");
    printf("%s\n", &b);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Hello, world
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000023a <strcat>:
 23a:   55                      push   rbp
 23b:   48 89 e5                mov    rbp,rsp
 23e:   48 81 ec 80 00 00 00    sub    rsp,0x80
 245:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 249:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 24d:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 251:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 255:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 259:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 25d:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 261:   49 89 c4                mov    r12,rax
 264:   b8 00 00 00 00          mov    eax,0x0
 269:   49 89 c5                mov    r13,rax
 26c:   4c 89 e7                mov    rdi,r12
 26f:   e8 45 ff ff ff          call   1b9 <strlen>
 274:   49 89 c5                mov    r13,rax
 277:   b8 00 00 00 00          mov    eax,0x0
 27c:   49 89 c6                mov    r14,rax
 27f:   4c 89 f0                mov    rax,r14
 282:   49 89 c7                mov    r15,rax
 285:   4c 89 f8                mov    rax,r15
 288:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
 28c:   48 01 c8                add    rax,rcx
 28f:   49 89 c7                mov    r15,rax
 292:   b0 00                   mov    al,0x0
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
294:   88 45 c8                mov    BYTE PTR [rbp-0x38],al
297:   4c 89 f8                mov    rax,r15
29a:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
29e:   88 4d c8                mov    BYTE PTR [rbp-0x38],cl
2a1:   8a 45 c8                mov    al,BYTE PTR [rbp-0x38]
2a4:   88 45 c0                mov    BYTE PTR [rbp-0x40],al
2a7:   0f b6 45 c0             movzx  eax,BYTE PTR [rbp-0x40]
2ab:   b9 00 00 00 00          mov    ecx,0x0
2b0:   39 c8                   cmp    eax,ecx
2b2:   0f 84 71 00 00 00       je     329 <strcat+0xef>
2b8:   4c 89 e8                mov    rax,r13
2bb:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
2bf:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
2c3:   4c 89 f1                mov    rcx,r14
2c6:   48 01 c8                add    rax,rcx
2c9:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
2cd:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
2d1:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
2d5:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
2d9:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
2dd:   48 01 c8                add    rax,rcx
2e0:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
2e4:   4c 89 f0                mov    rax,r14
2e7:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
2eb:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
2ef:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
2f3:   48 01 c8                add    rax,rcx
2f6:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
2fa:   b0 00                   mov    al,0x0
2fc:   88 45 a0                mov    BYTE PTR [rbp-0x60],al
2ff:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
303:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
307:   88 4d a0                mov    BYTE PTR [rbp-0x60],cl
30a:   8a 45 a0                mov    al,BYTE PTR [rbp-0x60]
30d:   88 45 98                mov    BYTE PTR [rbp-0x68],al
310:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
314:   48 0f b6 4d 98          movzx  rcx,BYTE PTR [rbp-0x68]
319:   88 08                   mov    BYTE PTR [rax],cl
31b:   4c 89 f0                mov    rax,r14
31e:   48 ff c0                inc    rax
321:   49 89 c6                mov    r14,rax
324:   e9 56 ff ff ff          jmp    27f <strcat+0x45>
329:   4c 89 e8                mov    rax,r13
32c:   48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
330:   48 8b 45 90             mov    rax,QWORD PTR [rbp-0x70]
334:   4c 89 f1                mov    rcx,r14
337:   48 01 c8                add    rax,rcx
33a:   48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
33e:   48 8b 45 90             mov    rax,QWORD PTR [rbp-0x70]
342:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
346:   48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
34a:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
34e:   48 01 c8                add    rax,rcx
351:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
355:   b0 00                   mov    al,0x0
357:   88 45 80                mov    BYTE PTR [rbp-0x80],al
35a:   48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
35e:   48 0f b6 4d 80          movzx  rcx,BYTE PTR [rbp-0x80]
363:   88 08                   mov    BYTE PTR [rax],cl
365:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
369:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 36d:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 371:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 375:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 379:   48 89 ec                mov    rsp,rbp
 37c:   5d                      pop    rbp
 37d:   c3                      ret
```

27.2.11 strncat

ASH

```
pub func strncat(uint8* dst, uint8* src, uint64 n): uint8*
```

Appends at most *n* bytes of *src* to *dst* and always terminates the result with a NUL.

Parameter	Description
`dst`	Buffer holding a string.
`src`	String to append.
`n`	Maximum number of bytes to append.

Returns. *dst*.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strncat(uint8* dst, uint8* src, uint64 n): uint8* {
    uint64 d = strlen(dst);
    uint64 i = 0;
    while (i < n) {
        if (src[i] == 0) {
            break;
        }
        dst[d + i] = src[i];
        i++;
    }
    dst[d + i] = 0;
    return dst;
}
```

Example.

Example 27-11. Using strncat

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    strcpy(&b, "ASH ");
    strncat(&b, "language", 4);
    printf("%s\n", &b);
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
ASH lang
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000023a <strncat>:
 23a:   55                      push   rbp
 23b:   48 89 e5                mov    rbp,rsp
 23e:   48 81 ec 90 00 00 00    sub    rsp,0x90
 245:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 249:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 24d:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 251:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 255:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 259:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 25d:   48 89 55 c8             mov    QWORD PTR [rbp-0x38],rdx
 261:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 265:   49 89 c4                mov    r12,rax
 268:   b8 00 00 00 00          mov    eax,0x0
 26d:   49 89 c5                mov    r13,rax
 270:   4c 89 e7                mov    rdi,r12
 273:   e8 41 ff ff ff          call   1b9 <strlen>
 278:   49 89 c5                mov    r13,rax
 27b:   b8 00 00 00 00          mov    eax,0x0
 280:   49 89 c6                mov    r14,rax
 283:   4c 89 f0                mov    rax,r14
 286:   48 8b 4d c8             mov    rcx,QWORD PTR [rbp-0x38]
 28a:   48 39 c8                cmp    rax,rcx
 28d:   0f 83 af 00 00 00       jae    342 <strncat+0x108>
 293:   4c 89 f0                mov    rax,r14
 296:   49 89 c7                mov    r15,rax
 299:   4c 89 f8                mov    rax,r15
 29c:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
 2a0:   48 01 c8                add    rax,rcx
 2a3:   49 89 c7                mov    r15,rax
 2a6:   b0 00                   mov    al,0x0
 2a8:   88 45 c0                mov    BYTE PTR [rbp-0x40],al
 2ab:   4c 89 f8                mov    rax,r15
 2ae:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 2b2:   88 4d c0                mov    BYTE PTR [rbp-0x40],cl
 2b5:   8a 45 c0                mov    al,BYTE PTR [rbp-0x40]
 2b8:   88 45 b8                mov    BYTE PTR [rbp-0x48],al
 2bb:   0f b6 45 b8             movzx  eax,BYTE PTR [rbp-0x48]
 2bf:   b9 00 00 00 00          mov    ecx,0x0
 2c4:   39 c8                   cmp    eax,ecx
 2c6:   0f 85 05 00 00 00       jne    2d1 <strncat+0x97>
 2cc:   e9 71 00 00 00          jmp    342 <strncat+0x108>
 2d1:   4c 89 e8                mov    rax,r13
 2d4:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 2d8:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
 2dc:   4c 89 f1                mov    rcx,r14
 2df:   48 01 c8                add    rax,rcx
 2e2:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 2e6:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
 2ea:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 2ee:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
 2f2:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 2f6:   48 01 c8                add    rax,rcx
 2f9:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 2fd:   4c 89 f0                mov    rax,r14
 300:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
 304:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
308:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
30c:   48 01 c8                add    rax,rcx
30f:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
313:   b0 00                   mov    al,0x0
315:   88 45 98                mov    BYTE PTR [rbp-0x68],al
318:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
31c:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
320:   88 4d 98                mov    BYTE PTR [rbp-0x68],cl
323:   8a 45 98                mov    al,BYTE PTR [rbp-0x68]
326:   88 45 90                mov    BYTE PTR [rbp-0x70],al
329:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
32d:   48 0f b6 4d 90          movzx  rcx,BYTE PTR [rbp-0x70]
332:   88 08                   mov    BYTE PTR [rax],cl
334:   4c 89 f0                mov    rax,r14
337:   48 ff c0                inc    rax
33a:   49 89 c6                mov    r14,rax
33d:   e9 41 ff ff ff          jmp    283 <strncat+0x49>
342:   4c 89 e8                mov    rax,r13
345:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
349:   48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
34d:   4c 89 f1                mov    rcx,r14
350:   48 01 c8                add    rax,rcx
353:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
357:   48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
35b:   48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
35f:   48 8b 45 80             mov    rax,QWORD PTR [rbp-0x80]
363:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
367:   48 01 c8                add    rax,rcx
36a:   48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
36e:   b0 00                   mov    al,0x0
370:   88 85 78 ff ff ff       mov    BYTE PTR [rbp-0x88],al
376:   48 8b 45 80             mov    rax,QWORD PTR [rbp-0x80]
37a:   48 0f b6 8d 78 ff ff    movzx  rcx,BYTE PTR [rbp-0x88]
381:   ff
382:   88 08                   mov    BYTE PTR [rax],cl
384:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
388:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
38c:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
390:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
394:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
398:   48 89 ec                mov    rsp,rbp
39b:   5d                      pop    rbp
39c:   c3                      ret
```

27.2.12 strcmp

ASH

```
pub func strcmp(uint8* a, uint8* b): int64
```

Compares two strings lexicographically, byte by byte, as unsigned values.

Parameter	Description
a	First string.
b	Second string.

Returns. 0 if equal, a negative value if *a* sorts before *b*, a positive value otherwise.

Copyright © 2026 Mallionic Inc. All rights reserved.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strcmp(uint8* a, uint8* b): int64 {
    uint64 i = 0;
    while (true) {
        int64 x = a[i];
        int64 y = b[i];
        if (x != y) {
            return x - y;
        }
        if (x == 0) {
            return 0;
        }
        i++;
    }
    return 0;
}
```

Example.

Example 27-12. Using strcmp

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", strcmp("ash", "ash"), strcmp("ash", "asm"), strcmp("b", "a"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0 -5 1
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <strcmp>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 00 00 00 00          mov    eax,0x0
  20:   49 89 c2                mov    r10,rax
  23:   b8 01 00 00 00          mov    eax,0x1
  28:   b9 00 00 00 00          mov    ecx,0x0
  2d:   48 39 c8                cmp    rax,rcx
  30:   0f 84 c4 00 00 00       je     fa <strcmp+0xfa>
  36:   4c 89 d0                mov    rax,r10
  39:   49 89 c3                mov    r11,rax
  3c:   4c 89 d8                mov    rax,r11
  3f:   48 89 f9                mov    rcx,rdi
  42:   48 01 c8                add    rax,rcx
  45:   49 89 c3                mov    r11,rax
  48:   b0 00                   mov    al,0x0
  4a:   41 88 c0                mov    r8b,al
  4d:   4c 89 d8                mov    rax,r11
  50:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  54:   41 88 c8                mov    r8b,cl
  57:   44 88 c0                mov    al,r8b
  5a:   0f b6 c0                movzx  eax,al
  5d:   49 89 c1                mov    r9,rax
  60:   4c 89 d0                mov    rax,r10
  63:   49 89 c4                mov    r12,rax
  66:   4c 89 e0                mov    rax,r12
  69:   48 89 f1                mov    rcx,rsi
  6c:   48 01 c8                add    rax,rcx
  6f:   49 89 c4                mov    r12,rax
  72:   b0 00                   mov    al,0x0
  74:   41 88 c5                mov    r13b,al
  77:   4c 89 e0                mov    rax,r12
  7a:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
  7e:   41 88 cd                mov    r13b,cl
  81:   44 88 e8                mov    al,r13b
  84:   0f b6 c0                movzx  eax,al
  87:   49 89 c6                mov    r14,rax
  8a:   4c 89 c8                mov    rax,r9
  8d:   4c 89 f1                mov    rcx,r14
  90:   48 39 c8                cmp    rax,rcx
  93:   0f 84 2a 00 00 00       je     c3 <strcmp+0xc3>
  99:   4c 89 c8                mov    rax,r9
  9c:   49 89 c7                mov    r15,rax
  9f:   4c 89 f8                mov    rax,r15
  a2:   4c 89 f1                mov    rcx,r14
  a5:   48 29 c8                sub    rax,rcx
  a8:   49 89 c7                mov    r15,rax
  ab:   4c 89 f8                mov    rax,r15
  ae:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  b2:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  b6:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  ba:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  be:   48 89 ec                mov    rsp,rbp
  c1:   5d                      pop    rbp
  c2:   c3                      ret
  c3:   4c 89 c8                mov    rax,r9
  c6:   b9 00 00 00 00          mov    ecx,0x0
  cb:   48 39 c8                cmp    rax,rcx
  ce:   0f 85 18 00 00 00       jne    ec <strcmp+0xec>
  d4:   48 31 c0                xor    rax,rax
  d7:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  db:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  df:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  e3:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  e7:   48 89 ec                mov    rsp,rbp
  ea:   5d                      pop    rbp
  eb:   c3                      ret
  ec:   4c 89 d0                mov    rax,r10
  ef:   48 ff c0                inc    rax
  f2:   49 89 c2                mov    r10,rax
  f5:   e9 29 ff ff ff          jmp    23 <strcmp+0x23>
  fa:   48 31 c0                xor    rax,rax
  fd:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 101:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 105:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 109:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 10d:   48 89 ec                mov    rsp,rbp
 110:   5d                      pop    rbp
 111:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.2.13 strncmp

```ash
pub func strncmp(uint8* a, uint8* b, uint64 n): int64
```

Compares at most the first *n* bytes of two strings.

Parameter	Description
a	First string.
b	Second string.
n	Maximum number of bytes to compare.

Returns. 0 if the first *n* bytes are equal (or both strings end before), otherwise the difference of the first differing bytes.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```ash
pub func strncmp(uint8* a, uint8* b, uint64 n): int64 {
    uint64 i = 0;
    while (i < n) {
        int64 x = a[i];
        int64 y = b[i];
        if (x != y) {
            return x - y;
        }
        if (x == 0) {
            return 0;
        }
        i++;
    }
    return 0;
}
```

Example.

Example 27-13. Using strncmp

```ash
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", strncmp("ashley", "ashc", 3), strncmp("ashley", "ashc", 4));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0 9
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <strncmp>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
1b:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
1f:   b8 00 00 00 00          mov    eax,0x0
24:   49 89 c2                mov    r10,rax
27:   4c 89 d0                mov    rax,r10
2a:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
2e:   48 39 c8                cmp    rax,rcx
31:   0f 83 c4 00 00 00       jae    fb <strncmp+0xfb>
37:   4c 89 d0                mov    rax,r10
3a:   49 89 c3                mov    r11,rax
3d:   4c 89 d8                mov    rax,r11
40:   48 89 f9                mov    rcx,rdi
43:   48 01 c8                add    rax,rcx
46:   49 89 c3                mov    r11,rax
49:   b0 00                   mov    al,0x0
4b:   41 88 c0                mov    r8b,al
4e:   4c 89 d8                mov    rax,r11
51:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
55:   41 88 c8                mov    r8b,cl
58:   44 88 c0                mov    al,r8b
5b:   0f b6 c0                movzx  eax,al
5e:   49 89 c1                mov    r9,rax
61:   4c 89 d0                mov    rax,r10
64:   49 89 c4                mov    r12,rax
67:   4c 89 e0                mov    rax,r12
6a:   48 89 f1                mov    rcx,rsi
6d:   48 01 c8                add    rax,rcx
70:   49 89 c4                mov    r12,rax
73:   b0 00                   mov    al,0x0
75:   41 88 c5                mov    r13b,al
78:   4c 89 e0                mov    rax,r12
7b:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
7f:   41 88 cd                mov    r13b,cl
82:   44 88 e8                mov    al,r13b
85:   0f b6 c0                movzx  eax,al
88:   49 89 c6                mov    r14,rax
8b:   4c 89 c8                mov    rax,r9
8e:   4c 89 f1                mov    rcx,r14
91:   48 39 c8                cmp    rax,rcx
94:   0f 84 2a 00 00 00       je     c4 <strncmp+0xc4>
9a:   4c 89 c8                mov    rax,r9
9d:   49 89 c7                mov    r15,rax
a0:   4c 89 f8                mov    rax,r15
a3:   4c 89 f1                mov    rcx,r14
a6:   48 29 c8                sub    rax,rcx
a9:   49 89 c7                mov    r15,rax
ac:   4c 89 f8                mov    rax,r15
af:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
b3:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
b7:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
bb:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
bf:   48 89 ec                mov    rsp,rbp
c2:   5d                      pop    rbp
c3:   c3                      ret
c4:   4c 89 c8                mov    rax,r9
c7:   b9 00 00 00 00          mov    ecx,0x0
cc:   48 39 c8                cmp    rax,rcx
cf:   0f 85 18 00 00 00       jne    ed <strncmp+0xed>
d5:   48 31 c0                xor    rax,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  d8:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  dc:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  e0:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  e4:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  e8:   48 89 ec                mov    rsp,rbp
  eb:   5d                      pop    rbp
  ec:   c3                      ret
  ed:   4c 89 d0                mov    rax,r10
  f0:   48 ff c0                inc    rax
  f3:   49 89 c2                mov    r10,rax
  f6:   e9 2c ff ff ff          jmp    27 <strncmp+0x27>
  fb:   48 31 c0                xor    rax,rax
  fe:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 102:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 106:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 10a:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 10e:   48 89 ec                mov    rsp,rbp
 111:   5d                      pop    rbp
 112:   c3                      ret
```

27.2.14 strcasecmp

ASH

```
pub func strcasecmp(uint8* a, uint8* b): int64
```

Compares two strings ignoring the case of ASCII letters.

Parameter	Description
a	First string.
b	Second string.

Returns. 0 if equal ignoring case; otherwise the difference of the first differing lowercase bytes.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strcasecmp(uint8* a, uint8* b): int64 {
    uint64 i = 0;
    while (true) {
        int64 x = tolower(a[i]);
        int64 y = tolower(b[i]);
        if (x != y) {
            return x - y;
        }
        if (x == 0) {
            return 0;
        }
        i++;
    }
    return 0;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-14. Using strcasecmp

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", strcasecmp("ASH", "ash"), strcasecmp("Ash", "Asm"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0 -5
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000000fb <strcasecmp>:
  fb:   55                      push   rbp
  fc:   48 89 e5                mov    rbp,rsp
  ff:   48 81 ec 60 00 00 00    sub    rsp,0x60
 106:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 10a:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 10e:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 112:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 116:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 11a:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 11e:   b8 00 00 00 00          mov    eax,0x0
 123:   49 89 c4                mov    r12,rax
 126:   b8 01 00 00 00          mov    eax,0x1
 12b:   b9 00 00 00 00          mov    ecx,0x0
 130:   48 39 c8                cmp    rax,rcx
 133:   0f 84 fa 00 00 00       je     233 <strcasecmp+0x138>
 139:   4c 89 e0                mov    rax,r12
 13c:   49 89 c5                mov    r13,rax
 13f:   4c 89 e8                mov    rax,r13
 142:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 146:   48 01 c8                add    rax,rcx
 149:   49 89 c5                mov    r13,rax
 14c:   b0 00                   mov    al,0x0
 14e:   41 88 c6                mov    r14b,al
 151:   4c 89 e8                mov    rax,r13
 154:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 158:   41 88 ce                mov    r14b,cl
 15b:   44 88 f0                mov    al,r14b
 15e:   41 88 c7                mov    r15b,al
 161:   b8 00 00 00 00          mov    eax,0x0
 166:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 16a:   49 0f b6 ff             movzx  rdi,r15b
 16e:   e8 fc fe ff ff          call   6f <tolower>
 173:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 177:   4c 89 e0                mov    rax,r12
 17a:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 17e:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
 182:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
 186:   48 01 c8                add    rax,rcx
 189:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 18d:   b0 00                   mov    al,0x0
 18f:   88 45 b8                mov    BYTE PTR [rbp-0x48],al
 192:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
 196:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 19a:   88 4d b8                mov    BYTE PTR [rbp-0x48],cl
 19d:   8a 45 b8                mov    al,BYTE PTR [rbp-0x48]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
1a0:   88 45 b0                mov    BYTE PTR [rbp-0x50],al
1a3:   b8 00 00 00 00          mov    eax,0x0
1a8:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
1ac:   48 0f b6 7d b0          movzx  rdi,BYTE PTR [rbp-0x50]
1b1:   e8 b9 fe ff ff          call   6f <tolower>
1b6:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
1ba:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
1be:   48 8b 4d a8             mov    rcx,QWORD PTR [rbp-0x58]
1c2:   48 39 c8                cmp    rax,rcx
1c5:   0f 84 30 00 00 00       je     1fb <strcasecmp+0x100>
1cb:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
1cf:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
1d3:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
1d7:   48 8b 4d a8             mov    rcx,QWORD PTR [rbp-0x58]
1db:   48 29 c8                sub    rax,rcx
1de:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
1e2:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
1e6:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
1ea:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
1ee:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
1f2:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
1f6:   48 89 ec                mov    rsp,rbp
1f9:   5d                      pop    rbp
1fa:   c3                      ret
1fb:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
1ff:   b9 00 00 00 00          mov    ecx,0x0
204:   48 39 c8                cmp    rax,rcx
207:   0f 85 18 00 00 00       jne    225 <strcasecmp+0x12a>
20d:   48 31 c0                xor    rax,rax
210:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
214:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
218:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
21c:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
220:   48 89 ec                mov    rsp,rbp
223:   5d                      pop    rbp
224:   c3                      ret
225:   4c 89 e0                mov    rax,r12
228:   48 ff c0                inc    rax
22b:   49 89 c4                mov    r12,rax
22e:   e9 f3 fe ff ff          jmp    126 <strcasecmp+0x2b>
233:   48 31 c0                xor    rax,rax
236:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
23a:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
23e:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
242:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
246:   48 89 ec                mov    rsp,rbp
249:   5d                      pop    rbp
24a:   c3                      ret
```

27.2.15 strchr

ASH

```
pub func strchr(uint8* s, int64 c): int64
```

Finds the first occurrence of the byte *c* in the string *s*. Searching for 0 finds the terminating NUL.

Parameter	Description
s	String to search.

Copyright © 2026 Mallionic Inc. All rights reserved.

Parameter	Description
c	Byte to find.

Returns. The index of the first occurrence, or -1.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strchr(uint8* s, int64 c): int64 {
    uint64 i = 0;
    int64 want = c & 255;
    while (true) {
        int64 x = s[i];
        if (x == want) {
            return i;
        }
        if (x == 0) {
            return -1;
        }
        i++;
    }
    return -1;
}
```

Example.

Example 27-15. Using strchr

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", strchr("banana", 'n'), strchr("banana", 'x'), strchr("abc", 0));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
2 -1 3
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <strchr>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 00 00 00 00          mov    eax,0x0
  20:   49 89 c2                mov    r10,rax
  23:   48 89 f0                mov    rax,rsi
  26:   49 89 c3                mov    r11,rax
  29:   4c 89 d8                mov    rax,r11
  2c:   48 81 e0 ff 00 00 00    and    rax,0xff
  33:   49 89 c3                mov    r11,rax
  36:   b8 01 00 00 00          mov    eax,0x1
  3b:   b9 00 00 00 00          mov    ecx,0x0
  40:   48 39 c8                cmp    rax,rcx
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  43:   0f 84 9a 00 00 00       je     e3 <strchr+0xe3>
  49:   4c 89 d0                mov    rax,r10
  4c:   49 89 c0                mov    r8,rax
  4f:   4c 89 c0                mov    rax,r8
  52:   48 89 f9                mov    rcx,rdi
  55:   48 01 c8                add    rax,rcx
  58:   49 89 c0                mov    r8,rax
  5b:   b0 00                   mov    al,0x0
  5d:   41 88 c1                mov    r9b,al
  60:   4c 89 c0                mov    rax,r8
  63:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
  67:   41 88 c9                mov    r9b,cl
  6a:   44 88 c8                mov    al,r9b
  6d:   0f b6 c0                movzx  eax,al
  70:   49 89 c4                mov    r12,rax
  73:   4c 89 e0                mov    rax,r12
  76:   4c 89 d9                mov    rcx,r11
  79:   48 39 c8                cmp    rax,rcx
  7c:   0f 85 18 00 00 00       jne    9a <strchr+0x9a>
  82:   4c 89 d0                mov    rax,r10
  85:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  89:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  8d:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  91:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  95:   48 89 ec                mov    rsp,rbp
  98:   5d                      pop    rbp
  99:   c3                      ret
  9a:   4c 89 e0                mov    rax,r12
  9d:   b9 00 00 00 00          mov    ecx,0x0
  a2:   48 39 c8                cmp    rax,rcx
  a5:   0f 85 2a 00 00 00       jne    d5 <strchr+0xd5>
  ab:   b8 00 00 00 00          mov    eax,0x0
  b0:   49 89 c5                mov    r13,rax
  b3:   4c 89 e8                mov    rax,r13
  b6:   48 83 e8 01             sub    rax,0x1
  ba:   49 89 c5                mov    r13,rax
  bd:   4c 89 e8                mov    rax,r13
  c0:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  c4:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  c8:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  cc:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  d0:   48 89 ec                mov    rsp,rbp
  d3:   5d                      pop    rbp
  d4:   c3                      ret
  d5:   4c 89 d0                mov    rax,r10
  d8:   48 ff c0                inc    rax
  db:   49 89 c2                mov    r10,rax
  de:   e9 53 ff ff ff          jmp    36 <strchr+0x36>
  e3:   b8 00 00 00 00          mov    eax,0x0
  e8:   49 89 c5                mov    r13,rax
  eb:   4c 89 e8                mov    rax,r13
  ee:   48 83 e8 01             sub    rax,0x1
  f2:   49 89 c5                mov    r13,rax
  f5:   4c 89 e8                mov    rax,r13
  f8:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  fc:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 100:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 104:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 108:   48 89 ec                mov    rsp,rbp
 10b:   5d                      pop    rbp
 10c:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.2.16 strrchr

```ash
pub func strrchr(uint8* s, int64 c): int64
```

Finds the last occurrence of the byte *c* in the string *s*.

Parameter	Description
s	String to search.
c	Byte to find.

Returns. The index of the last occurrence, or -1.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```ash
pub func strrchr(uint8* s, int64 c): int64 {
    uint64 i = 0;
    int64 last = -1;
    int64 want = c & 255;
    while (true) {
        int64 x = s[i];
        if (x == want) {
            last = i;
        }
        if (x == 0) {
            return last;
        }
        i++;
    }
    return last;
}
```

Example.

Example 27-16. Using strrchr

```ash
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", strrchr("banana", 'a'), strrchr("path/to/file", '/'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
5 7
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <strrchr>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
1b:   b8 00 00 00 00          mov    eax,0x0
20:   49 89 c2                mov    r10,rax
23:   b8 00 00 00 00          mov    eax,0x0
28:   49 89 c3                mov    r11,rax
2b:   4c 89 d8                mov    rax,r11
2e:   48 83 e8 01             sub    rax,0x1
32:   49 89 c3                mov    r11,rax
35:   48 89 f0                mov    rax,rsi
38:   49 89 c0                mov    r8,rax
3b:   4c 89 c0                mov    rax,r8
3e:   48 81 e0 ff 00 00 00    and    rax,0xff
45:   49 89 c0                mov    r8,rax
48:   b8 01 00 00 00          mov    eax,0x1
4d:   b9 00 00 00 00          mov    ecx,0x0
52:   48 39 c8                cmp    rax,rcx
55:   0f 84 76 00 00 00       je     d1 <strrchr+0xd1>
5b:   4c 89 d0                mov    rax,r10
5e:   49 89 c1                mov    r9,rax
61:   4c 89 c8                mov    rax,r9
64:   48 89 f9                mov    rcx,rdi
67:   48 01 c8                add    rax,rcx
6a:   49 89 c1                mov    r9,rax
6d:   b0 00                   mov    al,0x0
6f:   41 88 c4                mov    r12b,al
72:   4c 89 c8                mov    rax,r9
75:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
79:   41 88 cc                mov    r12b,cl
7c:   44 88 e0                mov    al,r12b
7f:   0f b6 c0                movzx  eax,al
82:   49 89 c5                mov    r13,rax
85:   4c 89 e8                mov    rax,r13
88:   4c 89 c1                mov    rcx,r8
8b:   48 39 c8                cmp    rax,rcx
8e:   0f 85 06 00 00 00       jne    9a <strrchr+0x9a>
94:   4c 89 d0                mov    rax,r10
97:   49 89 c3                mov    r11,rax
9a:   4c 89 e8                mov    rax,r13
9d:   b9 00 00 00 00          mov    ecx,0x0
a2:   48 39 c8                cmp    rax,rcx
a5:   0f 85 18 00 00 00       jne    c3 <strrchr+0xc3>
ab:   4c 89 d8                mov    rax,r11
ae:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
b2:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
b6:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
ba:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
be:   48 89 ec                mov    rsp,rbp
c1:   5d                      pop    rbp
c2:   c3                      ret
c3:   4c 89 d0                mov    rax,r10
c6:   48 ff c0                inc    rax
c9:   49 89 c2                mov    r10,rax
cc:   e9 77 ff ff ff          jmp    48 <strrchr+0x48>
d1:   4c 89 d8                mov    rax,r11
d4:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
d8:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
dc:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
e0:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
e4:   48 89 ec                mov    rsp,rbp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  e7:   5d                      pop    rbp
  e8:   c3                      ret
```

27.2.17 strstr

ASH

```
pub func strstr(uint8* pajar, uint8* aguja): int64
```

Finds the first occurrence of the string *needle* inside *haystack*.

Parameter	Description
`haystack`	String to search in (parameter `pajar` in the source).
`needle`	String to find (parameter `aguja` in the source).

Returns. The index at which *needle* starts, 0 if *needle* is empty, or -1 if it does not occur.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strstr(uint8* pajar, uint8* aguja): int64 {
    uint64 n = strlen(aguja);
    if (n == 0) {
        return 0;
    }
    uint64 i = 0;
    while (pajar[i] != 0) {
        if (strncmp(pajar + i, aguja, n) == 0) {
            return i;
        }
        i++;
    }
    return -1;
}
```

Example.

Example 27-17. Using strstr

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", strstr("The Ashley Project", "Ashley"), strstr("abc", ""), strstr("
↪ abc", "zz"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
4 0 -1
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000194 <strstr>:
 194:   55                      push   rbp
 195:   48 89 e5                mov    rbp,rsp
 198:   48 81 ec 70 00 00 00    sub    rsp,0x70
 19f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 1a3:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 1a7:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 1ab:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 1af:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 1b3:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 1b7:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 1bb:   49 89 c4                mov    r12,rax
 1be:   b8 00 00 00 00          mov    eax,0x0
 1c3:   49 89 c5                mov    r13,rax
 1c6:   4c 89 e7                mov    rdi,r12
 1c9:   e8 32 fe ff ff          call   0 <strlen>
 1ce:   49 89 c5                mov    r13,rax
 1d1:   4c 89 e8                mov    rax,r13
 1d4:   b9 00 00 00 00          mov    ecx,0x0
 1d9:   48 39 c8                cmp    rax,rcx
 1dc:   0f 85 18 00 00 00       jne    1fa <strstr+0x66>
 1e2:   48 31 c0                xor    rax,rax
 1e5:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 1e9:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 1ed:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 1f1:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 1f5:   48 89 ec                mov    rsp,rbp
 1f8:   5d                      pop    rbp
 1f9:   c3                      ret
 1fa:   b8 00 00 00 00          mov    eax,0x0
 1ff:   49 89 c6                mov    r14,rax
 202:   4c 89 f0                mov    rax,r14
 205:   49 89 c7                mov    r15,rax
 208:   4c 89 f8                mov    rax,r15
 20b:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 20f:   48 01 c8                add    rax,rcx
 212:   49 89 c7                mov    r15,rax
 215:   b0 00                   mov    al,0x0
 217:   88 45 c8                mov    BYTE PTR [rbp-0x38],al
 21a:   4c 89 f8                mov    rax,r15
 21d:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 221:   88 4d c8                mov    BYTE PTR [rbp-0x38],cl
 224:   8a 45 c8                mov    al,BYTE PTR [rbp-0x38]
 227:   88 45 c0                mov    BYTE PTR [rbp-0x40],al
 22a:   0f b6 45 c0             movzx  eax,BYTE PTR [rbp-0x40]
 22e:   b9 00 00 00 00          mov    ecx,0x0
 233:   39 c8                   cmp    eax,ecx
 235:   0f 84 7b 00 00 00       je     2b6 <strstr+0x122>
 23b:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 23f:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 243:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 247:   4c 89 f1                mov    rcx,r14
 24a:   48 01 c8                add    rax,rcx
 24d:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 251:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 255:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 259:   4c 89 e8                mov    rax,r13
 25c:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 260:   b8 00 00 00 00          mov    eax,0x0
 265:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
 269:   48 8b 7d b8             mov    rdi,QWORD PTR [rbp-0x48]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 26d:   48 8b 75 b0             mov    rsi,QWORD PTR [rbp-0x50]
 271:   48 8b 55 a8             mov    rdx,QWORD PTR [rbp-0x58]
 275:   e8 07 fe ff ff          call   81 <strncmp>
 27a:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
 27e:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
 282:   b9 00 00 00 00          mov    ecx,0x0
 287:   48 39 c8                cmp    rax,rcx
 28a:   0f 85 18 00 00 00       jne    2a8 <strstr+0x114>
 290:   4c 89 f0                mov    rax,r14
 293:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 297:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 29b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 29f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 2a3:   48 89 ec                mov    rsp,rbp
 2a6:   5d                      pop    rbp
 2a7:   c3                      ret
 2a8:   4c 89 f0                mov    rax,r14
 2ab:   48 ff c0                inc    rax
 2ae:   49 89 c6                mov    r14,rax
 2b1:   e9 4c ff ff ff          jmp    202 <strstr+0x6e>
 2b6:   b8 00 00 00 00          mov    eax,0x0
 2bb:   48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
 2bf:   48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
 2c3:   48 83 e8 01             sub    rax,0x1
 2c7:   48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
 2cb:   48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
 2cf:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 2d3:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 2d7:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 2db:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 2df:   48 89 ec                mov    rsp,rbp
 2e2:   5d                      pop    rbp
 2e3:   c3                      ret
```

27.2.18 strspn

ASH

```
pub func strspn(uint8* s, uint8* acepta): uint64
```

Measures the initial segment of *s* made only of bytes that appear in *accept*.

Parameter	Description
`s`	String to examine.
`accept`	Set of accepted bytes, as a string (parameter `acepta`).

Returns. The length of that prefix.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func strspn(uint8* s, uint8* acepta): uint64 {
    uint64 i = 0;
    while (s[i] != 0) {
        if (strchr(acepta, s[i]) < 0) {
            return i;
        }
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/STRING.ASH (CONTINUED)

```
        i++;
    }
    return i;
}
```

Example.

Example 27-18. Using strspn

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d\n", strspn("2026-10-24", "0123456789"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
4
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000010d <strspn>:
 10d:   55                      push   rbp
 10e:   48 89 e5                mov    rbp,rsp
 111:   48 81 ec 60 00 00 00    sub    rsp,0x60
 118:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 11c:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 120:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 124:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 128:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 12c:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 130:   b8 00 00 00 00          mov    eax,0x0
 135:   49 89 c4                mov    r12,rax
 138:   4c 89 e0                mov    rax,r12
 13b:   49 89 c5                mov    r13,rax
 13e:   4c 89 e8                mov    rax,r13
 141:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 145:   48 01 c8                add    rax,rcx
 148:   49 89 c5                mov    r13,rax
 14b:   b0 00                   mov    al,0x0
 14d:   41 88 c6                mov    r14b,al
 150:   4c 89 e8                mov    rax,r13
 153:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 157:   41 88 ce                mov    r14b,cl
 15a:   44 88 f0                mov    al,r14b
 15d:   41 88 c7                mov    r15b,al
 160:   41 0f b6 c7             movzx  eax,r15b
 164:   b9 00 00 00 00          mov    ecx,0x0
 169:   39 c8                   cmp    eax,ecx
 16b:   0f 84 87 00 00 00       je     1f8 <strspn+0xeb>
 171:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 175:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 179:   4c 89 e0                mov    rax,r12
 17c:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 180:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
 184:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 188:   48 01 c8                add    rax,rcx
 18b:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
18f:   b0 00                   mov    al,0x0
191:   88 45 b8                mov    BYTE PTR [rbp-0x48],al
194:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
198:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
19c:   88 4d b8                mov    BYTE PTR [rbp-0x48],cl
19f:   8a 45 b8                mov    al,BYTE PTR [rbp-0x48]
1a2:   88 45 b0                mov    BYTE PTR [rbp-0x50],al
1a5:   b8 00 00 00 00          mov    eax,0x0
1aa:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
1ae:   48 8b 7d c8             mov    rdi,QWORD PTR [rbp-0x38]
1b2:   48 0f b6 75 b0          movzx  rsi,BYTE PTR [rbp-0x50]
1b7:   e8 44 fe ff ff          call   0 <strchr>
1bc:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
1c0:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
1c4:   b9 00 00 00 00          mov    ecx,0x0
1c9:   48 39 c8                cmp    rax,rcx
1cc:   0f 8d 18 00 00 00       jge    1ea <strspn+0xdd>
1d2:   4c 89 e0                mov    rax,r12
1d5:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
1d9:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
1dd:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
1e1:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
1e5:   48 89 ec                mov    rsp,rbp
1e8:   5d                      pop    rbp
1e9:   c3                      ret
1ea:   4c 89 e0                mov    rax,r12
1ed:   48 ff c0                inc    rax
1f0:   49 89 c4                mov    r12,rax
1f3:   e9 40 ff ff ff          jmp    138 <strspn+0x2b>
1f8:   4c 89 e0                mov    rax,r12
1fb:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
1ff:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
203:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
207:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
20b:   48 89 ec                mov    rsp,rbp
20e:   5d                      pop    rbp
20f:   c3                      ret
```

27.2.19 strcspn

ASH

```
pub func strcspn(uint8* s, uint8* rechaza): uint64
```

Measures the initial segment of *s* that contains none of the bytes of *reject*.

Parameter	Description
`s`	String to examine.
`reject`	Set of rejected bytes (parameter `rechaza`).

Returns. The length of that prefix.

Implementation. The source of the function in `nlib/string.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/STRING.ASH

```
pub func strcspn(uint8* s, uint8* rechaza): uint64 {
    uint64 i = 0;
    while (s[i] != 0) {
        if (strchr(rechaza, s[i]) >= 0) {
            return i;
        }
        i++;
    }
    return i;
}
```

Example.

Example 27-19. Using strcspn

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d\n", strcspn("key=value", "=:"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000010d <strcspn>:
 10d:   55                      push   rbp
 10e:   48 89 e5                mov    rbp,rsp
 111:   48 81 ec 60 00 00 00    sub    rsp,0x60
 118:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 11c:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 120:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 124:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 128:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 12c:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 130:   b8 00 00 00 00          mov    eax,0x0
 135:   49 89 c4                mov    r12,rax
 138:   4c 89 e0                mov    rax,r12
 13b:   49 89 c5                mov    r13,rax
 13e:   4c 89 e8                mov    rax,r13
 141:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 145:   48 01 c8                add    rax,rcx
 148:   49 89 c5                mov    r13,rax
 14b:   b0 00                   mov    al,0x0
 14d:   41 88 c6                mov    r14b,al
 150:   4c 89 e8                mov    rax,r13
 153:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 157:   41 88 ce                mov    r14b,cl
 15a:   44 88 f0                mov    al,r14b
 15d:   41 88 c7                mov    r15b,al
 160:   41 0f b6 c7             movzx  eax,r15b
 164:   b9 00 00 00 00          mov    ecx,0x0
 169:   39 c8                   cmp    eax,ecx
 16b:   0f 84 87 00 00 00       je     1f8 <strcspn+0xeb>
 171:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 175:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
179:   4c 89 e0                mov    rax,r12
17c:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
180:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
184:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
188:   48 01 c8                add    rax,rcx
18b:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
18f:   b0 00                   mov    al,0x0
191:   88 45 b8                mov    BYTE PTR [rbp-0x48],al
194:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
198:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
19c:   88 4d b8                mov    BYTE PTR [rbp-0x48],cl
19f:   8a 45 b8                mov    al,BYTE PTR [rbp-0x48]
1a2:   88 45 b0                mov    BYTE PTR [rbp-0x50],al
1a5:   b8 00 00 00 00          mov    eax,0x0
1aa:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
1ae:   48 8b 7d c8             mov    rdi,QWORD PTR [rbp-0x38]
1b2:   48 0f b6 75 b0          movzx  rsi,BYTE PTR [rbp-0x50]
1b7:   e8 44 fe ff ff          call   0 <strchr>
1bc:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
1c0:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
1c4:   b9 00 00 00 00          mov    ecx,0x0
1c9:   48 39 c8                cmp    rax,rcx
1cc:   0f 8c 18 00 00 00       jl     1ea <strcspn+0xdd>
1d2:   4c 89 e0                mov    rax,r12
1d5:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
1d9:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
1dd:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
1e1:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
1e5:   48 89 ec                mov    rsp,rbp
1e8:   5d                      pop    rbp
1e9:   c3                      ret
1ea:   4c 89 e0                mov    rax,r12
1ed:   48 ff c0                inc    rax
1f0:   49 89 c4                mov    r12,rax
1f3:   e9 40 ff ff ff          jmp    138 <strcspn+0x2b>
1f8:   4c 89 e0                mov    rax,r12
1fb:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
1ff:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
203:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
207:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
20b:   48 89 ec                mov    rsp,rbp
20e:   5d                      pop    rbp
20f:   c3                      ret
```

27.2.20 isdigit

ASH

```
pub func isdigit(int64 c): int64
```

Tests whether *c* is an ASCII decimal digit (`0`-`9`).

Parameter	Description
`c`	Character code.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/string.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/STRING.ASH

```
pub func isdigit(int64 c): int64 {
    if (c >= 48 && c <= 57) { return 1; }
    return 0;
}
```

Example.

Example 27-20. Using isdigit

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", isdigit('7'), isdigit('x'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <isdigit>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 30 00 00 00          mov    ecx,0x30
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 8c 2b 00 00 00       jl     57 <isdigit+0x57>
  2c:   48 89 f8                mov    rax,rdi
  2f:   b9 39 00 00 00          mov    ecx,0x39
  34:   48 39 c8                cmp    rax,rcx
  37:   0f 8f 1a 00 00 00       jg     57 <isdigit+0x57>
  3d:   b8 01 00 00 00          mov    eax,0x1
  42:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  46:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  4a:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  4e:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  52:   48 89 ec                mov    rsp,rbp
  55:   5d                      pop    rbp
  56:   c3                      ret
  57:   48 31 c0                xor    rax,rax
  5a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  62:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  66:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  6a:   48 89 ec                mov    rsp,rbp
  6d:   5d                      pop    rbp
  6e:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.2.21 isupper

```ash
pub func isupper(int64 c): int64
```

Tests whether *c* is an ASCII uppercase letter (A-Z).

Parameter	Description
c	Character code.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```ash
pub func isupper(int64 c): int64 {
    if (c >= 65 && c <= 90) { return 1; }
    return 0;
}
```

Example.

Example 27-21. Using isupper

```ash
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", isupper('Q'), isupper('q'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <isupper>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 41 00 00 00          mov    ecx,0x41
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 8c 2b 00 00 00       jl     57 <isupper+0x57>
  2c:   48 89 f8                mov    rax,rdi
  2f:   b9 5a 00 00 00          mov    ecx,0x5a
  34:   48 39 c8                cmp    rax,rcx
  37:   0f 8f 1a 00 00 00       jg     57 <isupper+0x57>
  3d:   b8 01 00 00 00          mov    eax,0x1
  42:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  46:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  4a:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  4e:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  52:   48 89 ec                mov    rsp,rbp
  55:   5d                      pop    rbp
  56:   c3                      ret
  57:   48 31 c0                xor    rax,rax
  5a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  62:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  66:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  6a:   48 89 ec                mov    rsp,rbp
  6d:   5d                      pop    rbp
  6e:   c3                      ret
```

27.2.22 islower

ASH

```
pub func islower(int64 c): int64
```

Tests whether *c* is an ASCII lowercase letter (a-z).

Parameter	Description
c	Character code.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func islower(int64 c): int64 {
    if (c >= 97 && c <= 122) { return 1; }
    return 0;
}
```

Example.

Example 27-22. Using islower

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", islower('q'), islower('Q'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <islower>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 61 00 00 00          mov    ecx,0x61
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 8c 2b 00 00 00       jl     57 <islower+0x57>
  2c:   48 89 f8                mov    rax,rdi
  2f:   b9 7a 00 00 00          mov    ecx,0x7a
  34:   48 39 c8                cmp    rax,rcx
  37:   0f 8f 1a 00 00 00       jg     57 <islower+0x57>
  3d:   b8 01 00 00 00          mov    eax,0x1
  42:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  46:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  4a:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  4e:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  52:   48 89 ec                mov    rsp,rbp
  55:   5d                      pop    rbp
  56:   c3                      ret
  57:   48 31 c0                xor    rax,rax
  5a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  62:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  66:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  6a:   48 89 ec                mov    rsp,rbp
  6d:   5d                      pop    rbp
  6e:   c3                      ret
```

27.2.23 isalpha

ASH

```
pub func isalpha(int64 c): int64
```

Tests whether *c* is an ASCII letter.

Parameter	Description
c	Character code.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func isalpha(int64 c): int64 {
    if (isupper(c) == 1 || islower(c) == 1) { return 1; }
    return 0;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-23. Using isalpha

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", isalpha('g'), isalpha('5'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000000de <isalpha>:
  de:   55                      push   rbp
  df:   48 89 e5                mov    rbp,rsp
  e2:   48 81 ec 30 00 00 00    sub    rsp,0x30
  e9:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  ed:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  f1:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  f5:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  f9:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  fd:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 101:   49 89 c4                mov    r12,rax
 104:   b8 00 00 00 00          mov    eax,0x0
 109:   49 89 c5                mov    r13,rax
 10c:   4c 89 e7                mov    rdi,r12
 10f:   e8 ec fe ff ff          call   0 <isupper>
 114:   49 89 c5                mov    r13,rax
 117:   4c 89 e8                mov    rax,r13
 11a:   b9 01 00 00 00          mov    ecx,0x1
 11f:   48 39 c8                cmp    rax,rcx
 122:   0f 84 2b 00 00 00       je     153 <isalpha+0x75>
 128:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 12c:   49 89 c6                mov    r14,rax
 12f:   b8 00 00 00 00          mov    eax,0x0
 134:   49 89 c7                mov    r15,rax
 137:   4c 89 f7                mov    rdi,r14
 13a:   e8 30 ff ff ff          call   6f <islower>
 13f:   49 89 c7                mov    r15,rax
 142:   4c 89 f8                mov    rax,r15
 145:   b9 01 00 00 00          mov    ecx,0x1
 14a:   48 39 c8                cmp    rax,rcx
 14d:   0f 85 1a 00 00 00       jne    16d <isalpha+0x8f>
 153:   b8 01 00 00 00          mov    eax,0x1
 158:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 15c:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 160:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 164:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 168:   48 89 ec                mov    rsp,rbp
 16b:   5d                      pop    rbp
 16c:   c3                      ret
 16d:   48 31 c0                xor    rax,rax
 170:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 174:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 178:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 17c:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 180:   48 89 ec                mov    rsp,rbp
 183:   5d                      pop    rbp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 184:   c3                      ret
```

27.2.24 isalnum

ASH

```
pub func isalnum(int64 c): int64
```

Tests whether *c* is an ASCII letter or digit.

Parameter	Description
c	Character code.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func isalnum(int64 c): int64 {
    if (isalpha(c) == 1 || isdigit(c) == 1) { return 1; }
    return 0;
}
```

Example.

Example 27-24. Using isalnum

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", isalnum('g'), isalnum('5'), isalnum('-'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000001f4 <isalnum>:
 1f4:   55                      push   rbp
 1f5:   48 89 e5                mov    rbp,rsp
 1f8:   48 81 ec 30 00 00 00    sub    rsp,0x30
 1ff:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 203:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 207:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 20b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 20f:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 213:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 217:   49 89 c4                mov    r12,rax
 21a:   b8 00 00 00 00          mov    eax,0x0
 21f:   49 89 c5                mov    r13,rax
 222:   4c 89 e7                mov    rdi,r12
 225:   e8 b4 fe ff ff          call   de <isalpha>
 22a:   49 89 c5                mov    r13,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 22d:   4c 89 e8                mov    rax,r13
 230:   b9 01 00 00 00          mov    ecx,0x1
 235:   48 39 c8                cmp    rax,rcx
 238:   0f 84 2b 00 00 00       je     269 <isalnum+0x75>
 23e:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 242:   49 89 c6                mov    r14,rax
 245:   b8 00 00 00 00          mov    eax,0x0
 24a:   49 89 c7                mov    r15,rax
 24d:   4c 89 f7                mov    rdi,r14
 250:   e8 30 ff ff ff          call   185 <isdigit>
 255:   49 89 c7                mov    r15,rax
 258:   4c 89 f8                mov    rax,r15
 25b:   b9 01 00 00 00          mov    ecx,0x1
 260:   48 39 c8                cmp    rax,rcx
 263:   0f 85 1a 00 00 00       jne    283 <isalnum+0x8f>
 269:   b8 01 00 00 00          mov    eax,0x1
 26e:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 272:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 276:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 27a:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 27e:   48 89 ec                mov    rsp,rbp
 281:   5d                      pop    rbp
 282:   c3                      ret
 283:   48 31 c0                xor    rax,rax
 286:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 28a:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 28e:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 292:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 296:   48 89 ec                mov    rsp,rbp
 299:   5d                      pop    rbp
 29a:   c3                      ret
```

27.2.25 isspace

ASH

```
pub func isspace(int64 c): int64
```

Tests whether *c* is white space: space, tab, line feed, vertical tab, form feed or carriage return.

Parameter	Description
c	Character code.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func isspace(int64 c): int64 {
    if (c == 32 || (c >= 9 && c <= 13)) { return 1; }
    return 0;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-25. Using isspace

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", isspace(' '), isspace('\n'), isspace('a'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <isspace>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 20 00 00 00          mov    ecx,0x20
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 84 22 00 00 00       je     4e <isspace+0x4e>
  2c:   48 89 f8                mov    rax,rdi
  2f:   b9 09 00 00 00          mov    ecx,0x9
  34:   48 39 c8                cmp    rax,rcx
  37:   0f 8c 2b 00 00 00       jl     68 <isspace+0x68>
  3d:   48 89 f8                mov    rax,rdi
  40:   b9 0d 00 00 00          mov    ecx,0xd
  45:   48 39 c8                cmp    rax,rcx
  48:   0f 8f 1a 00 00 00       jg     68 <isspace+0x68>
  4e:   b8 01 00 00 00          mov    eax,0x1
  53:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  57:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  5b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  5f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  63:   48 89 ec                mov    rsp,rbp
  66:   5d                      pop    rbp
  67:   c3                      ret
  68:   48 31 c0                xor    rax,rax
  6b:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  6f:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  73:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  77:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  7b:   48 89 ec                mov    rsp,rbp
  7e:   5d                      pop    rbp
  7f:   c3                      ret
```

27.2.26 isxdigit

ASH

```
pub func isxdigit(int64 c): int64
```

Tests whether *c* is a hexadecimal digit (`0`-`9`, `a`-`f`, `A`-`F`).

Copyright © 2026 Mallionic Inc. All rights reserved.

Parameter	Description
c	Character code.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func isxdigit(int64 c): int64 {
    if (isdigit(c) == 1) { return 1; }
    if (c >= 97 && c <= 102) { return 1; }
    if (c >= 65 && c <= 70) { return 1; }
    return 0;
}
```

Example.

Example 27-26. Using isxdigit

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", isxdigit('F'), isxdigit('9'), isxdigit('g'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000006f <isxdigit>:
  6f:   55                      push   rbp
  70:   48 89 e5                mov    rbp,rsp
  73:   48 81 ec 30 00 00 00    sub    rsp,0x30
  7a:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  7e:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  82:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  86:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8a:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  8e:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  92:   49 89 c4                mov    r12,rax
  95:   b8 00 00 00 00          mov    eax,0x0
  9a:   49 89 c5                mov    r13,rax
  9d:   4c 89 e7                mov    rdi,r12
  a0:   e8 5b ff ff ff          call   0 <isdigit>
  a5:   49 89 c5                mov    r13,rax
  a8:   4c 89 e8                mov    rax,r13
  ab:   b9 01 00 00 00          mov    ecx,0x1
  b0:   48 39 c8                cmp    rax,rcx
  b3:   0f 85 1a 00 00 00       jne    d3 <isxdigit+0x64>
  b9:   b8 01 00 00 00          mov    eax,0x1
  be:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  c2:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  c6:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  ca:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  ce:   48 89 ec                mov    rsp,rbp
  d1:   5d                      pop    rbp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 d2:   c3                      ret
 d3:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 d7:   b9 61 00 00 00          mov    ecx,0x61
 dc:   48 39 c8                cmp    rax,rcx
 df:   0f 8c 2c 00 00 00       jl     111 <isxdigit+0xa2>
 e5:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 e9:   b9 66 00 00 00          mov    ecx,0x66
 ee:   48 39 c8                cmp    rax,rcx
 f1:   0f 8f 1a 00 00 00       jg     111 <isxdigit+0xa2>
 f7:   b8 01 00 00 00          mov    eax,0x1
 fc:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
100:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
104:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
108:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
10c:   48 89 ec                mov    rsp,rbp
10f:   5d                      pop    rbp
110:   c3                      ret
111:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
115:   b9 41 00 00 00          mov    ecx,0x41
11a:   48 39 c8                cmp    rax,rcx
11d:   0f 8c 2c 00 00 00       jl     14f <isxdigit+0xe0>
123:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
127:   b9 46 00 00 00          mov    ecx,0x46
12c:   48 39 c8                cmp    rax,rcx
12f:   0f 8f 1a 00 00 00       jg     14f <isxdigit+0xe0>
135:   b8 01 00 00 00          mov    eax,0x1
13a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
13e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
142:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
146:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
14a:   48 89 ec                mov    rsp,rbp
14d:   5d                      pop    rbp
14e:   c3                      ret
14f:   48 31 c0                xor    rax,rax
152:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
156:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
15a:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
15e:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
162:   48 89 ec                mov    rsp,rbp
165:   5d                      pop    rbp
166:   c3                      ret
```

27.2.27 toupper

ASH

```
pub func toupper(int64 c): int64
```

Converts an ASCII lowercase letter to uppercase; other values are returned unchanged.

Parameter	Description
c	Character code.

Returns. The converted code.

Implementation. The source of the function in `nlib/string.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/STRING.ASH

```
pub func toupper(int64 c): int64 {
    if (islower(c) == 1) { return c - 32; }
    return c;
}
```

Example.

Example 27-27. Using toupper

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%c%c%c\n", toupper('a'), toupper('s'), toupper('!'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
AS!
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000006f <toupper>:
  6f:   55                      push   rbp
  70:   48 89 e5                mov    rbp,rsp
  73:   48 81 ec 30 00 00 00    sub    rsp,0x30
  7a:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  7e:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  82:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  86:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8a:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  8e:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  92:   49 89 c4                mov    r12,rax
  95:   b8 00 00 00 00          mov    eax,0x0
  9a:   49 89 c5                mov    r13,rax
  9d:   4c 89 e7                mov    rdi,r12
  a0:   e8 5b ff ff ff          call   0 <islower>
  a5:   49 89 c5                mov    r13,rax
  a8:   4c 89 e8                mov    rax,r13
  ab:   b9 01 00 00 00          mov    ecx,0x1
  b0:   48 39 c8                cmp    rax,rcx
  b3:   0f 85 29 00 00 00       jne    e2 <toupper+0x73>
  b9:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  bd:   49 89 c6                mov    r14,rax
  c0:   4c 89 f0                mov    rax,r14
  c3:   48 83 e8 20             sub    rax,0x20
  c7:   49 89 c6                mov    r14,rax
  ca:   4c 89 f0                mov    rax,r14
  cd:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  d1:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  d5:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  d9:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  dd:   48 89 ec                mov    rsp,rbp
  e0:   5d                      pop    rbp
  e1:   c3                      ret
  e2:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  e6:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  ea:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  ee:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  f2:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  f6:   48 89 ec                mov    rsp,rbp
  f9:   5d                      pop    rbp
  fa:   c3                      ret
```

27.2.28 tolower

ASH

```
pub func tolower(int64 c): int64
```

Converts an ASCII uppercase letter to lowercase; other values are returned unchanged.

Parameter	Description
c	Character code.

Returns. The converted code.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func tolower(int64 c): int64 {
    if (isupper(c) == 1) { return c + 32; }
    return c;
}
```

Example.

Example 27-28. Using tolower

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%c%c%c\n", tolower('A'), tolower('S'), tolower('H'));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
ash
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000006f <tolower>:
  6f:   55                      push   rbp
  70:   48 89 e5                mov    rbp,rsp
  73:   48 81 ec 30 00 00 00    sub    rsp,0x30
  7a:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  7e:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  82:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  86:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8a:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  8e:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  92:   49 89 c4                mov    r12,rax
  95:   b8 00 00 00 00          mov    eax,0x0
  9a:   49 89 c5                mov    r13,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  9d:   4c 89 e7                mov    rdi,r12
  a0:   e8 5b ff ff ff          call   0 <isupper>
  a5:   49 89 c5                mov    r13,rax
  a8:   4c 89 e8                mov    rax,r13
  ab:   b9 01 00 00 00          mov    ecx,0x1
  b0:   48 39 c8                cmp    rax,rcx
  b3:   0f 85 29 00 00 00       jne    e2 <tolower+0x73>
  b9:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  bd:   49 89 c6                mov    r14,rax
  c0:   4c 89 f0                mov    rax,r14
  c3:   48 83 c0 20             add    rax,0x20
  c7:   49 89 c6                mov    r14,rax
  ca:   4c 89 f0                mov    rax,r14
  cd:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  d1:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  d5:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  d9:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  dd:   48 89 ec                mov    rsp,rbp
  e0:   5d                      pop    rbp
  e1:   c3                      ret
  e2:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  e6:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  ea:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  ee:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  f2:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  f6:   48 89 ec                mov    rsp,rbp
  f9:   5d                      pop    rbp
  fa:   c3                      ret
```

27.2.29 str_to_int

ASH

```
pub func str_to_int(uint8* s, int64 base): int64
```

Converts text to a signed integer. Leading white space is skipped and an optional `+` or `-` sign is accepted. With *base* 0 the base is detected from the prefix: `0x` hexadecimal, `0b` binary, otherwise decimal. Digits beyond 9 are the letters `a-z` (either case), so bases up to 36 are supported. Conversion stops at the first character that is not a digit of the base.

Parameter	Description
`s`	Text to convert.
`base`	Base 2 to 36, or 0 for automatic detection.

Returns. The value (0 if no digit was found).

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func str_to_int(uint8* s, int64 base): int64 {
    uint64 i = 0;
    while (isspace(s[i]) == 1) {
        i++;
    }
    int64 neg = 0;
    if (s[i] == 45) {
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/STRING.ASH (CONTINUED)

```
        neg = 1;
        i++;
    } elif (s[i] == 43) {
        i++;
    }
    int64 b = base;
    if (s[i] == 48) {
        int64 x = tolower(s[i + 1]);
        if (x == 120) {
            if (b == 0 || b == 16) {
                b = 16;
                i = i + 2;
            }
        } elif (x == 98) {
            if (b == 0 || b == 2) {
                b = 2;
                i = i + 2;
            }
        }
    }
    if (b == 0) {
        b = 10;
    }
    int64 v = 0;
    while (true) {
        int64 d = __str_digit(s[i]);
        if (d >= b) {
            break;
        }
        v = v * b + d;
        i++;
    }
    if (neg == 1) {
        return 0 - v;
    }
    return v;
}
```

Example.

Example 27-29. Using str_to_int

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d %d %d\n", str_to_int("  -42", 10), str_to_int("0xFF", 0), str_to_int("0b10
↪ 1", 0), str_to_int("zz", 36));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-42 255 5 1295
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000028e <str_to_int>:
 28e:   55                      push   rbp
 28f:   48 89 e5                mov    rbp,rsp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
292:   48 81 ec f0 00 00 00    sub    rsp,0xf0
299:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
29d:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
2a1:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
2a5:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
2a9:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
2ad:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
2b1:   b8 00 00 00 00          mov    eax,0x0
2b6:   49 89 c4                mov    r12,rax
2b9:   4c 89 e0                mov    rax,r12
2bc:   49 89 c5                mov    r13,rax
2bf:   4c 89 e8                mov    rax,r13
2c2:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
2c6:   48 01 c8                add    rax,rcx
2c9:   49 89 c5                mov    r13,rax
2cc:   b0 00                   mov    al,0x0
2ce:   41 88 c6                mov    r14b,al
2d1:   4c 89 e8                mov    rax,r13
2d4:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
2d8:   41 88 ce                mov    r14b,cl
2db:   44 88 f0                mov    al,r14b
2de:   41 88 c7                mov    r15b,al
2e1:   b8 00 00 00 00          mov    eax,0x0
2e6:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
2ea:   49 0f b6 ff             movzx  rdi,r15b
2ee:   e8 0d fd ff ff          call   0 <isspace>
2f3:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
2f7:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
2fb:   b9 01 00 00 00          mov    ecx,0x1
300:   48 39 c8                cmp    rax,rcx
303:   0f 85 0e 00 00 00       jne    317 <str_to_int+0x89>
309:   4c 89 e0                mov    rax,r12
30c:   48 ff c0                inc    rax
30f:   49 89 c4                mov    r12,rax
312:   e9 a2 ff ff ff          jmp    2b9 <str_to_int+0x2b>
317:   b8 00 00 00 00          mov    eax,0x0
31c:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
320:   4c 89 e0                mov    rax,r12
323:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
327:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
32b:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
32f:   48 01 c8                add    rax,rcx
332:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
336:   b0 00                   mov    al,0x0
338:   88 45 b0                mov    BYTE PTR [rbp-0x50],al
33b:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
33f:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
343:   88 4d b0                mov    BYTE PTR [rbp-0x50],cl
346:   8a 45 b0                mov    al,BYTE PTR [rbp-0x50]
349:   88 45 a8                mov    BYTE PTR [rbp-0x58],al
34c:   0f b6 45 a8             movzx  eax,BYTE PTR [rbp-0x58]
350:   b9 2d 00 00 00          mov    ecx,0x2d
355:   39 c8                   cmp    eax,ecx
357:   0f 85 17 00 00 00       jne    374 <str_to_int+0xe6>
35d:   b8 01 00 00 00          mov    eax,0x1
362:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
366:   4c 89 e0                mov    rax,r12
369:   48 ff c0                inc    rax
36c:   49 89 c4                mov    r12,rax
36f:   e9 46 00 00 00          jmp    3ba <str_to_int+0x12c>
374:   4c 89 e0                mov    rax,r12
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
377:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
37b:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
37f:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
383:   48 01 c8                add    rax,rcx
386:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
38a:   b0 00                   mov    al,0x0
38c:   88 45 98                mov    BYTE PTR [rbp-0x68],al
38f:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
393:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
397:   88 4d 98                mov    BYTE PTR [rbp-0x68],cl
39a:   8a 45 98                mov    al,BYTE PTR [rbp-0x68]
39d:   88 45 90                mov    BYTE PTR [rbp-0x70],al
3a0:   0f b6 45 90             movzx  eax,BYTE PTR [rbp-0x70]
3a4:   b9 2b 00 00 00          mov    ecx,0x2b
3a9:   39 c8                   cmp    eax,ecx
3ab:   0f 85 09 00 00 00       jne    3ba <str_to_int+0x12c>
3b1:   4c 89 e0                mov    rax,r12
3b4:   48 ff c0                inc    rax
3b7:   49 89 c4                mov    r12,rax
3ba:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
3be:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
3c2:   4c 89 e0                mov    rax,r12
3c5:   48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
3c9:   48 8b 45 80             mov    rax,QWORD PTR [rbp-0x80]
3cd:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
3d1:   48 01 c8                add    rax,rcx
3d4:   48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
3d8:   b0 00                   mov    al,0x0
3da:   88 85 78 ff ff ff       mov    BYTE PTR [rbp-0x88],al
3e0:   48 8b 45 80             mov    rax,QWORD PTR [rbp-0x80]
3e4:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
3e8:   88 8d 78 ff ff ff       mov    BYTE PTR [rbp-0x88],cl
3ee:   8a 85 78 ff ff ff       mov    al,BYTE PTR [rbp-0x88]
3f4:   88 85 70 ff ff ff       mov    BYTE PTR [rbp-0x90],al
3fa:   0f b6 85 70 ff ff ff    movzx  eax,BYTE PTR [rbp-0x90]
401:   b9 30 00 00 00          mov    ecx,0x30
406:   39 c8                   cmp    eax,ecx
408:   0f 85 21 01 00 00       jne    52f <str_to_int+0x2a1>
40e:   4c 89 e0                mov    rax,r12
411:   48 89 85 68 ff ff ff    mov    QWORD PTR [rbp-0x98],rax
418:   48 8b 85 68 ff ff ff    mov    rax,QWORD PTR [rbp-0x98]
41f:   48 83 c0 01             add    rax,0x1
423:   48 89 85 68 ff ff ff    mov    QWORD PTR [rbp-0x98],rax
42a:   48 8b 85 68 ff ff ff    mov    rax,QWORD PTR [rbp-0x98]
431:   48 89 85 60 ff ff ff    mov    QWORD PTR [rbp-0xa0],rax
438:   48 8b 85 60 ff ff ff    mov    rax,QWORD PTR [rbp-0xa0]
43f:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
443:   48 01 c8                add    rax,rcx
446:   48 89 85 60 ff ff ff    mov    QWORD PTR [rbp-0xa0],rax
44d:   b0 00                   mov    al,0x0
44f:   88 85 58 ff ff ff       mov    BYTE PTR [rbp-0xa8],al
455:   48 8b 85 60 ff ff ff    mov    rax,QWORD PTR [rbp-0xa0]
45c:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
460:   88 8d 58 ff ff ff       mov    BYTE PTR [rbp-0xa8],cl
466:   8a 85 58 ff ff ff       mov    al,BYTE PTR [rbp-0xa8]
46c:   88 85 50 ff ff ff       mov    BYTE PTR [rbp-0xb0],al
472:   b8 00 00 00 00          mov    eax,0x0
477:   48 89 85 48 ff ff ff    mov    QWORD PTR [rbp-0xb8],rax
47e:   48 0f b6 bd 50 ff ff    movzx  rdi,BYTE PTR [rbp-0xb0]
485:   ff
486:   e8 64 fc ff ff          call   ef <tolower>
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 48b:   48 89 85 48 ff ff ff    mov    QWORD PTR [rbp-0xb8],rax
 492:   48 8b 85 48 ff ff ff    mov    rax,QWORD PTR [rbp-0xb8]
 499:   b9 78 00 00 00          mov    ecx,0x78
 49e:   48 39 c8                cmp    rax,rcx
 4a1:   0f 85 3c 00 00 00       jne    4e3 <str_to_int+0x255>
 4a7:   48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
 4ab:   b9 00 00 00 00          mov    ecx,0x0
 4b0:   48 39 c8                cmp    rax,rcx
 4b3:   0f 84 12 00 00 00       je     4cb <str_to_int+0x23d>
 4b9:   48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
 4bd:   b9 10 00 00 00          mov    ecx,0x10
 4c2:   48 39 c8                cmp    rax,rcx
 4c5:   0f 85 13 00 00 00       jne    4de <str_to_int+0x250>
 4cb:   b8 10 00 00 00          mov    eax,0x10
 4d0:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
... (99 more lines not shown)
```

27.2.30 atoi

ASH

```
pub func atoi(uint8* s): int64
```

Converts decimal text to an integer; equivalent to `str_to_int(s, 10)`.

Parameter	Description
`s`	Text to convert.

Returns. The value.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func atoi(uint8* s): int64 {
    return str_to_int(s, 10);
}
```

Example.

Example 27-30. Using atoi

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", atoi("1234"), atoi("-250 apples"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1234 -250
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000006ac <atoi>:
 6ac:   55                      push   rbp
 6ad:   48 89 e5                mov    rbp,rsp
 6b0:   48 81 ec 30 00 00 00    sub    rsp,0x30
 6b7:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 6bb:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 6bf:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 6c3:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 6c7:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 6cb:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 6cf:   49 89 c4                mov    r12,rax
 6d2:   b8 0a 00 00 00          mov    eax,0xa
 6d7:   49 89 c5                mov    r13,rax
 6da:   b8 00 00 00 00          mov    eax,0x0
 6df:   49 89 c6                mov    r14,rax
 6e2:   4c 89 e7                mov    rdi,r12
 6e5:   4c 89 ee                mov    rsi,r13
 6e8:   e8 a1 fb ff ff          call   28e <str_to_int>
 6ed:   49 89 c6                mov    r14,rax
 6f0:   4c 89 f0                mov    rax,r14
 6f3:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 6f7:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 6fb:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 6ff:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 703:   48 89 ec                mov    rsp,rbp
 706:   5d                      pop    rbp
 707:   c3                      ret
```

27.2.31 uint_to_str

ASH

```
pub func uint_to_str(uint64 value, uint8* buf, int64 base): uint8*
```

Writes the unsigned value *value* in base *base* (2 to 36, lowercase digits) into *buf*, followed by a NUL.

Parameter	Description
`value`	Value to convert.
`buf`	Destination buffer; 65 bytes are enough for any value in base 2.
`base`	Base 2 to 36.

Returns. *buf*.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func uint_to_str(uint64 value, uint8* buf, int64 base): uint8* {
    uint8* digits = "0123456789abcdefghijklmnopqrstuvwxyz";
    uint64 v = value;
    uint64 b = base;
    uint64 n = 0;
    if (v == 0) {
        buf[0] = 48;
        buf[1] = 0;
        return buf;
    }
    while (v != 0) {
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/STRING.ASH (CONTINUED)

```
        buf[n] = digits[v % b];
        v = v / b;
        n++;
    }
    buf[n] = 0;
    str_reverse(buf);
    return buf;
}
```

Example.

Example 27-31. Using uint_to_str

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[65] b;
    printf("%s ", uint_to_str(255, &b, 16));
    printf("%s ", uint_to_str(255, &b, 2));
    printf("%s\n", uint_to_str(0, &b, 10));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
ff 11111111 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000002b1 <uint_to_str>:
 2b1:   55                      push   rbp
 2b2:   48 89 e5                mov    rbp,rsp
 2b5:   48 81 ec a0 00 00 00    sub    rsp,0xa0
 2bc:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 2c0:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 2c4:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 2c8:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 2cc:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 2d0:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 2d4:   48 89 55 c8             mov    QWORD PTR [rbp-0x38],rdx
 2d8:   b8 00 00 00 00          mov    eax,0x0
 2dd:   49 89 c4                mov    r12,rax
 2e0:   4c 8d 25 f9 08 00 00    lea    r12,[rip+0x8f9]        # be0 <__str2>
 2e7:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 2eb:   49 89 c5                mov    r13,rax
 2ee:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 2f2:   49 89 c6                mov    r14,rax
 2f5:   b8 00 00 00 00          mov    eax,0x0
 2fa:   49 89 c7                mov    r15,rax
 2fd:   4c 89 e8                mov    rax,r13
 300:   b9 00 00 00 00          mov    ecx,0x0
 305:   48 39 c8                cmp    rax,rcx
 308:   0f 85 69 00 00 00       jne    377 <uint_to_str+0xc6>
 30e:   b8 00 00 00 00          mov    eax,0x0
 313:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 317:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
 31b:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
 31f:   48 01 c8                add    rax,rcx
 322:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
326:   b0 30                   mov    al,0x30
328:   88 45 b8                mov    BYTE PTR [rbp-0x48],al
32b:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
32f:   48 0f b6 4d b8          movzx  rcx,BYTE PTR [rbp-0x48]
334:   88 08                   mov    BYTE PTR [rax],cl
336:   b8 01 00 00 00          mov    eax,0x1
33b:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
33f:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
343:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
347:   48 01 c8                add    rax,rcx
34a:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
34e:   b0 00                   mov    al,0x0
350:   88 45 a8                mov    BYTE PTR [rbp-0x58],al
353:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
357:   48 0f b6 4d a8          movzx  rcx,BYTE PTR [rbp-0x58]
35c:   88 08                   mov    BYTE PTR [rax],cl
35e:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
362:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
366:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
36a:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
36e:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
372:   48 89 ec                mov    rsp,rbp
375:   5d                      pop    rbp
376:   c3                      ret
377:   4c 89 e8                mov    rax,r13
37a:   b9 00 00 00 00          mov    ecx,0x0
37f:   48 39 c8                cmp    rax,rcx
382:   0f 84 83 00 00 00       je     40b <uint_to_str+0x15a>
388:   4c 89 f8                mov    rax,r15
38b:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
38f:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
393:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
397:   48 01 c8                add    rax,rcx
39a:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
39e:   4c 89 e8                mov    rax,r13
3a1:   48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
3a5:   4c 89 f1                mov    rcx,r14
3a8:   48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
3ac:   31 d2                   xor    edx,edx
3ae:   48 f7 f1                div    rcx
3b1:   48 89 55 98             mov    QWORD PTR [rbp-0x68],rdx
3b5:   48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
3b9:   48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
3bd:   48 8b 45 90             mov    rax,QWORD PTR [rbp-0x70]
3c1:   4c 89 e1                mov    rcx,r12
3c4:   48 01 c8                add    rax,rcx
3c7:   48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
3cb:   b0 00                   mov    al,0x0
3cd:   88 45 88                mov    BYTE PTR [rbp-0x78],al
3d0:   48 8b 45 90             mov    rax,QWORD PTR [rbp-0x70]
3d4:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
3d8:   88 4d 88                mov    BYTE PTR [rbp-0x78],cl
3db:   8a 45 88                mov    al,BYTE PTR [rbp-0x78]
3de:   88 45 80                mov    BYTE PTR [rbp-0x80],al
3e1:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
3e5:   48 0f b6 4d 80          movzx  rcx,BYTE PTR [rbp-0x80]
3ea:   88 08                   mov    BYTE PTR [rax],cl
3ec:   4c 89 e8                mov    rax,r13
3ef:   4c 89 f1                mov    rcx,r14
3f2:   4c 89 e8                mov    rax,r13
3f5:   31 d2                   xor    edx,edx
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
3f7:   48 f7 f1                div    rcx
3fa:   49 89 c5                mov    r13,rax
3fd:   4c 89 f8                mov    rax,r15
400:   48 ff c0                inc    rax
403:   49 89 c7                mov    r15,rax
406:   e9 6c ff ff ff          jmp    377 <uint_to_str+0xc6>
40b:   4c 89 f8                mov    rax,r15
40e:   48 89 85 78 ff ff ff    mov    QWORD PTR [rbp-0x88],rax
415:   48 8b 85 78 ff ff ff    mov    rax,QWORD PTR [rbp-0x88]
41c:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
420:   48 01 c8                add    rax,rcx
423:   48 89 85 78 ff ff ff    mov    QWORD PTR [rbp-0x88],rax
42a:   b0 00                   mov    al,0x0
42c:   88 85 70 ff ff ff       mov    BYTE PTR [rbp-0x90],al
432:   48 8b 85 78 ff ff ff    mov    rax,QWORD PTR [rbp-0x88]
439:   48 0f b6 8d 70 ff ff    movzx  rcx,BYTE PTR [rbp-0x90]
440:   ff
441:   88 08                   mov    BYTE PTR [rax],cl
443:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
447:   48 89 85 68 ff ff ff    mov    QWORD PTR [rbp-0x98],rax
44e:   b8 00 00 00 00          mov    eax,0x0
453:   48 89 85 60 ff ff ff    mov    QWORD PTR [rbp-0xa0],rax
45a:   48 8b bd 68 ff ff ff    mov    rdi,QWORD PTR [rbp-0x98]
461:   e8 ec fc ff ff          call   152 <str_reverse>
466:   48 89 85 60 ff ff ff    mov    QWORD PTR [rbp-0xa0],rax
46d:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
471:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
475:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
479:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
47d:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
481:   48 89 ec                mov    rsp,rbp
484:   5d                      pop    rbp
485:   c3                      ret
```

27.2.32 int_to_str

ASH

```
pub func int_to_str(int64 value, uint8* buf, int64 base): uint8*
```

Writes the signed value *value* into *buf*. In base 10 negative values get a leading `-`; in other bases the value is written as unsigned (its two's complement bit pattern).

Parameter	Description
`value`	Value to convert.
`buf`	Destination buffer (at least 66 bytes).
`base`	Base 2 to 36.

Returns. *buf*.

Notes. In `ashc` 1.0 negative values in base 10 are written as unsigned numbers instead of with a `-` sign: the parameter `value` is declared unsigned in `uint_to_str` and signed here, and the compiler applies the first declaration's signedness to the comparison `value < 0` (see Appendix E). Write `buf[0] = '-'` and convert `0 - value` yourself when you need signed output.

Implementation. The source of the function in `nlib/string.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/STRING.ASH

```
pub func int_to_str(int64 value, uint8* buf, int64 base): uint8* {
    if (value < 0 && base == 10) {
        buf[0] = 45;
        uint_to_str(0 - value, buf + 1, 10);
        return buf;
    }
    return uint_to_str(value, buf, base);
}
```

Example.

Example 27-32. Using int_to_str

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[66] b;
    printf("%s ", int_to_str(1234, &b, 10));
    printf("%s\n", int_to_str(-1, &b, 16));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1234 ffffffffffffffff
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000486 <int_to_str>:
 486:   55                      push   rbp
 487:   48 89 e5                mov    rbp,rsp
 48a:   48 81 ec 60 00 00 00    sub    rsp,0x60
 491:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 495:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 499:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 49d:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 4a1:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 4a5:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 4a9:   48 89 55 c8             mov    QWORD PTR [rbp-0x38],rdx
 4ad:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 4b1:   b9 00 00 00 00          mov    ecx,0x0
 4b6:   48 39 c8                cmp    rax,rcx
 4b9:   0f 83 99 00 00 00       jae    558 <int_to_str+0xd2>
 4bf:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 4c3:   b9 0a 00 00 00          mov    ecx,0xa
 4c8:   48 39 c8                cmp    rax,rcx
 4cb:   0f 85 87 00 00 00       jne    558 <int_to_str+0xd2>
 4d1:   b8 00 00 00 00          mov    eax,0x0
 4d6:   49 89 c4                mov    r12,rax
 4d9:   4c 89 e0                mov    rax,r12
 4dc:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
 4e0:   48 01 c8                add    rax,rcx
 4e3:   49 89 c4                mov    r12,rax
 4e6:   b0 2d                   mov    al,0x2d
 4e8:   41 88 c5                mov    r13b,al
 4eb:   4c 89 e0                mov    rax,r12
 4ee:   49 0f b6 cd             movzx  rcx,r13b
 4f2:   88 08                   mov    BYTE PTR [rax],cl
 4f4:   b8 00 00 00 00          mov    eax,0x0
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
4f9:   49 89 c6                mov    r14,rax
4fc:   4c 89 f0                mov    rax,r14
4ff:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
503:   48 29 c8                sub    rax,rcx
506:   49 89 c6                mov    r14,rax
509:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
50d:   49 89 c7                mov    r15,rax
510:   4c 89 f8                mov    rax,r15
513:   48 83 c0 01             add    rax,0x1
517:   49 89 c7                mov    r15,rax
51a:   b8 0a 00 00 00          mov    eax,0xa
51f:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
523:   b8 00 00 00 00          mov    eax,0x0
528:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
52c:   4c 89 f7                mov    rdi,r14
52f:   4c 89 fe                mov    rsi,r15
532:   48 8b 55 c0             mov    rdx,QWORD PTR [rbp-0x40]
536:   e8 76 fd ff ff          call   2b1 <uint_to_str>
53b:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
53f:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
543:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
547:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
54b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
54f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
553:   48 89 ec                mov    rsp,rbp
556:   5d                      pop    rbp
557:   c3                      ret
558:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
55c:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
560:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
564:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
568:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
56c:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
570:   48 8b 7d b0             mov    rdi,QWORD PTR [rbp-0x50]
574:   48 8b 75 a8             mov    rsi,QWORD PTR [rbp-0x58]
578:   48 8b 55 a0             mov    rdx,QWORD PTR [rbp-0x60]
57c:   e8 30 fd ff ff          call   2b1 <uint_to_str>
581:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
585:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
589:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
58d:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
591:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
595:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
599:   48 89 ec                mov    rsp,rbp
59c:   5d                      pop    rbp
59d:   c3                      ret
```

27.2.33 str_reverse

ASH

```
pub func str_reverse(uint8* s): uint8*
```

Reverses a string in place.

Parameter	Description
s	String to reverse (must be writable).

Returns. *s*.

Copyright © 2026 Mallionic Inc. All rights reserved.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func str_reverse(uint8* s): uint8* {
    uint64 n = strlen(s);
    if (n < 2) {
        return s;
    }
    uint64 i = 0;
    uint64 j = n - 1;
    while (i < j) {
        int64 t = s[i];
        s[i] = s[j];
        s[j] = t;
        i++;
        j--;
    }
    return s;
}
```

Example.

Example 27-33. Using str_reverse

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[16] b;
    strcpy(&b, "stressed");
    printf("%s\n", str_reverse(&b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
desserts
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000023a <str_reverse>:
 23a:   55                      push   rbp
 23b:   48 89 e5                mov    rbp,rsp
 23e:   48 81 ec 70 00 00 00    sub    rsp,0x70
 245:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 249:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 24d:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 251:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 255:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 259:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 25d:   49 89 c4                mov    r12,rax
 260:   b8 00 00 00 00          mov    eax,0x0
 265:   49 89 c5                mov    r13,rax
 268:   4c 89 e7                mov    rdi,r12
 26b:   e8 49 ff ff ff          call   1b9 <strlen>
 270:   49 89 c5                mov    r13,rax
 273:   4c 89 e8                mov    rax,r13
 276:   b9 02 00 00 00          mov    ecx,0x2
 27b:   48 39 c8                cmp    rax,rcx
 27e:   0f 83 19 00 00 00       jae    29d <str_reverse+0x63>
 284:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
288:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
28c:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
290:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
294:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
298:   48 89 ec                mov    rsp,rbp
29b:   5d                      pop    rbp
29c:   c3                      ret
29d:   b8 00 00 00 00          mov    eax,0x0
2a2:   49 89 c6                mov    r14,rax
2a5:   4c 89 e8                mov    rax,r13
2a8:   49 89 c7                mov    r15,rax
2ab:   4c 89 f8                mov    rax,r15
2ae:   48 83 e8 01             sub    rax,0x1
2b2:   49 89 c7                mov    r15,rax
2b5:   4c 89 f0                mov    rax,r14
2b8:   4c 89 f9                mov    rcx,r15
2bb:   48 39 c8                cmp    rax,rcx
2be:   0f 83 bc 00 00 00       jae    380 <str_reverse+0x146>
2c4:   4c 89 f0                mov    rax,r14
2c7:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
2cb:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
2cf:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
2d3:   48 01 c8                add    rax,rcx
2d6:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
2da:   b0 00                   mov    al,0x0
2dc:   88 45 c8                mov    BYTE PTR [rbp-0x38],al
2df:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
2e3:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
2e7:   88 4d c8                mov    BYTE PTR [rbp-0x38],cl
2ea:   8a 45 c8                mov    al,BYTE PTR [rbp-0x38]
2ed:   0f b6 c0                movzx  eax,al
2f0:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
2f4:   4c 89 f0                mov    rax,r14
2f7:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
2fb:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
2ff:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
303:   48 01 c8                add    rax,rcx
306:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
30a:   4c 89 f8                mov    rax,r15
30d:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
311:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
315:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
319:   48 01 c8                add    rax,rcx
31c:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
320:   b0 00                   mov    al,0x0
322:   88 45 a8                mov    BYTE PTR [rbp-0x58],al
325:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
329:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
32d:   88 4d a8                mov    BYTE PTR [rbp-0x58],cl
330:   8a 45 a8                mov    al,BYTE PTR [rbp-0x58]
333:   88 45 a0                mov    BYTE PTR [rbp-0x60],al
336:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
33a:   48 0f b6 4d a0          movzx  rcx,BYTE PTR [rbp-0x60]
33f:   88 08                   mov    BYTE PTR [rax],cl
341:   4c 89 f8                mov    rax,r15
344:   48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
348:   48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
34c:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
350:   48 01 c8                add    rax,rcx
353:   48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
357:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 35b:   88 45 90                mov    BYTE PTR [rbp-0x70],al
 35e:   48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
 362:   48 0f b6 4d 90          movzx  rcx,BYTE PTR [rbp-0x70]
 367:   88 08                   mov    BYTE PTR [rax],cl
 369:   4c 89 f0                mov    rax,r14
 36c:   48 ff c0                inc    rax
 36f:   49 89 c6                mov    r14,rax
 372:   4c 89 f8                mov    rax,r15
 375:   48 ff c8                dec    rax
 378:   49 89 c7                mov    r15,rax
 37b:   e9 35 ff ff ff          jmp    2b5 <str_reverse+0x7b>
 380:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 384:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 388:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 38c:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 390:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 394:   48 89 ec                mov    rsp,rbp
 397:   5d                      pop    rbp
 398:   c3                      ret
```

27.2.34 str_upper

ASH

```
pub func str_upper(uint8* s): uint8*
```

Converts every ASCII letter of a string to uppercase, in place.

Parameter	Description
s	Writable string.

Returns. *s*.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func str_upper(uint8* s): uint8* {
    uint64 i = 0;
    while (s[i] != 0) {
        s[i] = toupper(s[i]);
        i++;
    }
    return s;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-34. Using str_upper

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[16] b;
    strcpy(&b, "quiet");
    printf("%s\n", str_upper(&b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
QUIET
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000002b4 <str_upper>:
 2b4:   55                      push   rbp
 2b5:   48 89 e5                mov    rbp,rsp
 2b8:   48 81 ec 50 00 00 00    sub    rsp,0x50
 2bf:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 2c3:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 2c7:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 2cb:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 2cf:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 2d3:   b8 00 00 00 00          mov    eax,0x0
 2d8:   49 89 c4                mov    r12,rax
 2db:   4c 89 e0                mov    rax,r12
 2de:   49 89 c5                mov    r13,rax
 2e1:   4c 89 e8                mov    rax,r13
 2e4:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 2e8:   48 01 c8                add    rax,rcx
 2eb:   49 89 c5                mov    r13,rax
 2ee:   b0 00                   mov    al,0x0
 2f0:   41 88 c6                mov    r14b,al
 2f3:   4c 89 e8                mov    rax,r13
 2f6:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 2fa:   41 88 ce                mov    r14b,cl
 2fd:   44 88 f0                mov    al,r14b
 300:   41 88 c7                mov    r15b,al
 303:   41 0f b6 c7             movzx  eax,r15b
 307:   b9 00 00 00 00          mov    ecx,0x0
 30c:   39 c8                   cmp    eax,ecx
 30e:   0f 84 6d 00 00 00       je     381 <str_upper+0xcd>
 314:   4c 89 e0                mov    rax,r12
 317:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 31b:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 31f:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 323:   48 01 c8                add    rax,rcx
 326:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 32a:   4c 89 e0                mov    rax,r12
 32d:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 331:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 335:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 339:   48 01 c8                add    rax,rcx
 33c:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 340:   b0 00                   mov    al,0x0
 342:   88 45 c0                mov    BYTE PTR [rbp-0x40],al
 345:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 349:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 34d:   88 4d c0                mov    BYTE PTR [rbp-0x40],cl
 350:   8a 45 c0                mov    al,BYTE PTR [rbp-0x40]
 353:   88 45 b8                mov    BYTE PTR [rbp-0x48],al
 356:   b0 00                   mov    al,0x0
 358:   88 45 b0                mov    BYTE PTR [rbp-0x50],al
 35b:   48 0f b6 7d b8          movzx  rdi,BYTE PTR [rbp-0x48]
 360:   e8 c3 fe ff ff          call   228 <toupper>
 365:   88 45 b0                mov    BYTE PTR [rbp-0x50],al
 368:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 36c:   48 0f b6 4d b0          movzx  rcx,BYTE PTR [rbp-0x50]
 371:   88 08                   mov    BYTE PTR [rax],cl
 373:   4c 89 e0                mov    rax,r12
 376:   48 ff c0                inc    rax
 379:   49 89 c4                mov    r12,rax
 37c:   e9 5a ff ff ff          jmp    2db <str_upper+0x27>
 381:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 385:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 389:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 38d:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 391:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 395:   48 89 ec                mov    rsp,rbp
 398:   5d                      pop    rbp
 399:   c3                      ret
```

27.2.35 str_lower

ASH

```
pub func str_lower(uint8* s): uint8*
```

Converts every ASCII letter of a string to lowercase, in place.

Parameter	Description
s	Writable string.

Returns. *s*.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func str_lower(uint8* s): uint8* {
    uint64 i = 0;
    while (s[i] != 0) {
        s[i] = tolower(s[i]);
        i++;
    }
    return s;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-35. Using str_lower

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[16] b;
    strcpy(&b, "LOUD");
    printf("%s\n", str_lower(&b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
loud
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000002b4 <str_lower>:
 2b4:   55                      push   rbp
 2b5:   48 89 e5                mov    rbp,rsp
 2b8:   48 81 ec 50 00 00 00    sub    rsp,0x50
 2bf:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 2c3:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 2c7:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 2cb:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 2cf:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 2d3:   b8 00 00 00 00          mov    eax,0x0
 2d8:   49 89 c4                mov    r12,rax
 2db:   4c 89 e0                mov    rax,r12
 2de:   49 89 c5                mov    r13,rax
 2e1:   4c 89 e8                mov    rax,r13
 2e4:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 2e8:   48 01 c8                add    rax,rcx
 2eb:   49 89 c5                mov    r13,rax
 2ee:   b0 00                   mov    al,0x0
 2f0:   41 88 c6                mov    r14b,al
 2f3:   4c 89 e8                mov    rax,r13
 2f6:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
 2fa:   41 88 ce                mov    r14b,cl
 2fd:   44 88 f0                mov    al,r14b
 300:   41 88 c7                mov    r15b,al
 303:   41 0f b6 c7             movzx  eax,r15b
 307:   b9 00 00 00 00          mov    ecx,0x0
 30c:   39 c8                   cmp    eax,ecx
 30e:   0f 84 6d 00 00 00       je     381 <str_lower+0xcd>
 314:   4c 89 e0                mov    rax,r12
 317:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 31b:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 31f:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 323:   48 01 c8                add    rax,rcx
 326:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 32a:   4c 89 e0                mov    rax,r12
 32d:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 331:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 335:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
 339:   48 01 c8                add    rax,rcx
 33c:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 340:   b0 00                   mov    al,0x0
 342:   88 45 c0                mov    BYTE PTR [rbp-0x40],al
 345:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 349:   48 0f b6 08             movzx  rcx,BYTE PTR [rax]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 34d:   88 4d c0                mov    BYTE PTR [rbp-0x40],cl
 350:   8a 45 c0                mov    al,BYTE PTR [rbp-0x40]
 353:   88 45 b8                mov    BYTE PTR [rbp-0x48],al
 356:   b0 00                   mov    al,0x0
 358:   88 45 b0                mov    BYTE PTR [rbp-0x50],al
 35b:   48 0f b6 7d b8          movzx  rdi,BYTE PTR [rbp-0x48]
 360:   e8 c3 fe ff ff          call   228 <tolower>
 365:   88 45 b0                mov    BYTE PTR [rbp-0x50],al
 368:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 36c:   48 0f b6 4d b0          movzx  rcx,BYTE PTR [rbp-0x50]
 371:   88 08                   mov    BYTE PTR [rax],cl
 373:   4c 89 e0                mov    rax,r12
 376:   48 ff c0                inc    rax
 379:   49 89 c4                mov    r12,rax
 37c:   e9 5a ff ff ff          jmp    2db <str_lower+0x27>
 381:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 385:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 389:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 38d:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 391:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 395:   48 89 ec                mov    rsp,rbp
 398:   5d                      pop    rbp
 399:   c3                      ret
```

27.2.36 str_equals

ASH

```
pub func str_equals(uint8* a, uint8* b): int64
```

Tests two strings for equality.

Parameter	Description
a	First string.
b	Second string.

Returns. 1 if equal, 0 otherwise.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func str_equals(uint8* a, uint8* b): int64 {
    if (strcmp(a, b) == 0) { return 1; }
    return 0;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-36. Using str_equals

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", str_equals("ash", "ash"), str_equals("ash", "ASH"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000112 <str_equals>:
 112:   55                      push   rbp
 113:   48 89 e5                mov    rbp,rsp
 116:   48 81 ec 30 00 00 00    sub    rsp,0x30
 11d:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 121:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 125:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 129:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 12d:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 131:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 135:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 139:   49 89 c4                mov    r12,rax
 13c:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 140:   49 89 c5                mov    r13,rax
 143:   b8 00 00 00 00          mov    eax,0x0
 148:   49 89 c6                mov    r14,rax
 14b:   4c 89 e7                mov    rdi,r12
 14e:   4c 89 ee                mov    rsi,r13
 151:   e8 aa fe ff ff          call   0 <strcmp>
 156:   49 89 c6                mov    r14,rax
 159:   4c 89 f0                mov    rax,r14
 15c:   b9 00 00 00 00          mov    ecx,0x0
 161:   48 39 c8                cmp    rax,rcx
 164:   0f 85 1a 00 00 00       jne    184 <str_equals+0x72>
 16a:   b8 01 00 00 00          mov    eax,0x1
 16f:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 173:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 177:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 17b:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 17f:   48 89 ec                mov    rsp,rbp
 182:   5d                      pop    rbp
 183:   c3                      ret
 184:   48 31 c0                xor    rax,rax
 187:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 18b:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 18f:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 193:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 197:   48 89 ec                mov    rsp,rbp
 19a:   5d                      pop    rbp
 19b:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.2.37 str_starts

```ash
pub func str_starts(uint8* s, uint8* prefijo): int64
```

Tests whether *s* begins with *prefix* (parameter `prefijo`).

Parameter	Description
`s`	String to test.
`prefix`	Prefix.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```ash
pub func str_starts(uint8* s, uint8* prefijo): int64 {
    if (strncmp(s, prefijo, strlen(prefijo)) == 0) { return 1; }
    return 0;
}
```

Example.

Example 27-37. Using str_starts

```ash
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", str_starts("--verbose", "--"), str_starts("-v", "--"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000194 <str_starts>:
 194:   55                      push   rbp
 195:   48 89 e5                mov    rbp,rsp
 198:   48 81 ec 40 00 00 00    sub    rsp,0x40
 19f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 1a3:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 1a7:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 1ab:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 1af:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 1b3:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 1b7:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 1bb:   49 89 c4                mov    r12,rax
 1be:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 1c2:   49 89 c5                mov    r13,rax
 1c5:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 1c9:   49 89 c6                mov    r14,rax
 1cc:   b8 00 00 00 00          mov    eax,0x0
 1d1:   49 89 c7                mov    r15,rax
 1d4:   4c 89 f7                mov    rdi,r14
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 1d7:   e8 24 fe ff ff          call   0 <strlen>
 1dc:   49 89 c7                mov    r15,rax
 1df:   b8 00 00 00 00          mov    eax,0x0
 1e4:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 1e8:   4c 89 e7                mov    rdi,r12
 1eb:   4c 89 ee                mov    rsi,r13
 1ee:   4c 89 fa                mov    rdx,r15
 1f1:   e8 8b fe ff ff          call   81 <strncmp>
 1f6:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 1fa:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 1fe:   b9 00 00 00 00          mov    ecx,0x0
 203:   48 39 c8                cmp    rax,rcx
 206:   0f 85 1a 00 00 00       jne    226 <str_starts+0x92>
 20c:   b8 01 00 00 00          mov    eax,0x1
 211:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 215:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 219:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 21d:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 221:   48 89 ec                mov    rsp,rbp
 224:   5d                      pop    rbp
 225:   c3                      ret
 226:   48 31 c0                xor    rax,rax
 229:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 22d:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 231:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 235:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 239:   48 89 ec                mov    rsp,rbp
 23c:   5d                      pop    rbp
 23d:   c3                      ret
```

27.2.38 str_ends

ASH

```
pub func str_ends(uint8* s, uint8* sufijo): int64
```

Tests whether *s* ends with *suffix* (parameter `sufijo`).

Parameter	Description
`s`	String to test.
`suffix`	Suffix.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/string.ash`:

NLIB/STRING.ASH

```
pub func str_ends(uint8* s, uint8* sufijo): int64 {
    uint64 n = strlen(s);
    uint64 m = strlen(sufijo);
    if (m > n) { return 0; }
    if (strcmp(s + (n - m), sufijo) == 0) { return 1; }
    return 0;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-38. Using str_ends

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", str_ends("main.ash", ".ash"), str_ends("main.c", ".ash"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000193 <str_ends>:
 193:   55                      push   rbp
 194:   48 89 e5                mov    rbp,rsp
 197:   48 81 ec 50 00 00 00    sub    rsp,0x50
 19e:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 1a2:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 1a6:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 1aa:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 1ae:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 1b2:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 1b6:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 1ba:   49 89 c4                mov    r12,rax
 1bd:   b8 00 00 00 00          mov    eax,0x0
 1c2:   49 89 c5                mov    r13,rax
 1c5:   4c 89 e7                mov    rdi,r12
 1c8:   e8 33 fe ff ff          call   0 <strlen>
 1cd:   49 89 c5                mov    r13,rax
 1d0:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 1d4:   49 89 c6                mov    r14,rax
 1d7:   b8 00 00 00 00          mov    eax,0x0
 1dc:   49 89 c7                mov    r15,rax
 1df:   4c 89 f7                mov    rdi,r14
 1e2:   e8 19 fe ff ff          call   0 <strlen>
 1e7:   49 89 c7                mov    r15,rax
 1ea:   4c 89 f8                mov    rax,r15
 1ed:   4c 89 e9                mov    rcx,r13
 1f0:   48 39 c8                cmp    rax,rcx
 1f3:   0f 86 18 00 00 00       jbe    211 <str_ends+0x7e>
 1f9:   48 31 c0                xor    rax,rax
 1fc:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 200:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 204:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 208:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 20c:   48 89 ec                mov    rsp,rbp
 20f:   5d                      pop    rbp
 210:   c3                      ret
 211:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 215:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 219:   4c 89 e8                mov    rax,r13
 21c:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 220:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
 224:   4c 89 f9                mov    rcx,r15
 227:   48 29 c8                sub    rax,rcx
 22a:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 22e:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 232:   48 8b 4d c0             mov    rcx,QWORD PTR [rbp-0x40]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
236:   48 01 c8                add    rax,rcx
239:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
23d:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
241:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
245:   b8 00 00 00 00          mov    eax,0x0
24a:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
24e:   48 8b 7d c8             mov    rdi,QWORD PTR [rbp-0x38]
252:   48 8b 75 b8             mov    rsi,QWORD PTR [rbp-0x48]
256:   e8 26 fe ff ff          call   81 <strcmp>
25b:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
25f:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
263:   b9 00 00 00 00          mov    ecx,0x0
268:   48 39 c8                cmp    rax,rcx
26b:   0f 85 1a 00 00 00       jne    28b <str_ends+0xf8>
271:   b8 01 00 00 00          mov    eax,0x1
276:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
27a:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
27e:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
282:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
286:   48 89 ec                mov    rsp,rbp
289:   5d                      pop    rbp
28a:   c3                      ret
28b:   48 31 c0                xor    rax,rax
28e:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
292:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
296:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
29a:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
29e:   48 89 ec                mov    rsp,rbp
2a1:   5d                      pop    rbp
2a2:   c3                      ret
```

27.3 <MATH>: INTEGER AND FIXED-POINT MATHEMATICS

`import <math>;` provides integer functions and Q16.16 fixed-point arithmetic. The library never uses the FPU or SSE, so it is suitable for kernels, boot loaders and firmware.

27.3.1 The Q16.16 format

A Q16.16 number is a 64-bit integer that represents the value *n* / 65536: the low 16 bits are the fraction and the rest is the integer part. 1.0 is 65536 (`FX_ONE`), 0.5 is 32768, and pi is 205887. Addition and subtraction work directly on the integers; multiplication and division need a correction by 65536, which `fx_mul` and `fx_div` apply. Angles of the trigonometric functions are in radians, also in Q16.16; `fx_deg` converts whole degrees.

Table 27-2. Constants defined by <math>

Name	Value	Meaning
`FX_ONE`	`65536`	1.0 in Q16.16
`FX_HALF`	`32768`	0.5 in Q16.16
`FX_PI`	`205887`	pi in Q16.16
`FX_TWO_PI`	`411775`	2 pi in Q16.16
`FX_HALF_PI`	`102944`	pi / 2 in Q16.16

Copyright © 2026 Mallionic Inc. All rights reserved.

27.3.2 abs

```ash
pub func abs(int64 x): int64
```

Absolute value of a signed integer.

Parameter	Description
x	Value.

Returns. |*x*|. As in C, the absolute value of the most negative 64-bit integer cannot be represented and is returned unchanged.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```ash
pub func abs(int64 x): int64 {
    if (x < 0) { return 0 - x; }
    return x;
}
```

Example.

Example 27-39. Using abs

```ash
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", abs(-17), abs(17));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
17 17
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <abs>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 00 00 00 00          mov    ecx,0x0
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 8d 2c 00 00 00       jge    58 <abs+0x58>
  2c:   b8 00 00 00 00          mov    eax,0x0
  31:   49 89 c2                mov    r10,rax
  34:   4c 89 d0                mov    rax,r10
  37:   48 89 f9                mov    rcx,rdi
  3a:   48 29 c8                sub    rax,rcx
  3d:   49 89 c2                mov    r10,rax
  40:   4c 89 d0                mov    rax,r10
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  43:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  47:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  4b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  4f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  53:   48 89 ec                mov    rsp,rbp
  56:   5d                      pop    rbp
  57:   c3                      ret
  58:   48 89 f8                mov    rax,rdi
  5b:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5f:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  63:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  67:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  6b:   48 89 ec                mov    rsp,rbp
  6e:   5d                      pop    rbp
  6f:   c3                      ret
```

27.3.3 min

ASH

```
pub func min(int64 a, int64 b): int64
```

The smaller of two signed integers.

Parameter	Description
a	First value.
b	Second value.

Returns. The minimum.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func min(int64 a, int64 b): int64 {
    if (a < b) { return a; }
    return b;
}
```

Example.

Example 27-40. Using min

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d\n", min(3, -8));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-8
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <min>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   48 89 f1                mov    rcx,rsi
  21:   48 39 c8                cmp    rax,rcx
  24:   0f 8d 18 00 00 00       jge    42 <min+0x42>
  2a:   48 89 f8                mov    rax,rdi
  2d:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  31:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  35:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  39:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  3d:   48 89 ec                mov    rsp,rbp
  40:   5d                      pop    rbp
  41:   c3                      ret
  42:   48 89 f0                mov    rax,rsi
  45:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  49:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  4d:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  51:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  55:   48 89 ec                mov    rsp,rbp
  58:   5d                      pop    rbp
  59:   c3                      ret
```

27.3.4 max

ASH

```
pub func max(int64 a, int64 b): int64
```

The larger of two signed integers.

Parameter	Description
a	First value.
b	Second value.

Returns. The maximum.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func max(int64 a, int64 b): int64 {
    if (a > b) { return a; }
    return b;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-41. Using max

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d\n", max(3, -8));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <max>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   48 89 f1                mov    rcx,rsi
  21:   48 39 c8                cmp    rax,rcx
  24:   0f 8e 18 00 00 00       jle    42 <max+0x42>
  2a:   48 89 f8                mov    rax,rdi
  2d:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  31:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  35:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  39:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  3d:   48 89 ec                mov    rsp,rbp
  40:   5d                      pop    rbp
  41:   c3                      ret
  42:   48 89 f0                mov    rax,rsi
  45:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  49:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  4d:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  51:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  55:   48 89 ec                mov    rsp,rbp
  58:   5d                      pop    rbp
  59:   c3                      ret
```

27.3.5 clamp

ASH

```
pub func clamp(int64 x, int64 lo, int64 hi): int64
```

Limits *x* to the range [*lo*, *hi*].

Parameter	Description
`x`	Value.
`lo`	Lower bound.
`hi`	Upper bound.

Copyright © 2026 Mallionic Inc. All rights reserved.

Returns. *lo* if *x* < *lo*, *hi* if *x* > *hi*, otherwise *x*.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func clamp(int64 x, int64 lo, int64 hi): int64 {
    if (x < lo) { return lo; }
    if (x > hi) { return hi; }
    return x;
}
```

Example.

Example 27-42. Using clamp

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", clamp(-5, 0, 100), clamp(50, 0, 100), clamp(500, 0, 100));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0 50 100
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <clamp>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
  1f:   48 89 f8                mov    rax,rdi
  22:   48 89 f1                mov    rcx,rsi
  25:   48 39 c8                cmp    rax,rcx
  28:   0f 8d 18 00 00 00       jge    46 <clamp+0x46>
  2e:   48 89 f0                mov    rax,rsi
  31:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  35:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  39:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  3d:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  41:   48 89 ec                mov    rsp,rbp
  44:   5d                      pop    rbp
  45:   c3                      ret
  46:   48 89 f8                mov    rax,rdi
  49:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
  4d:   48 39 c8                cmp    rax,rcx
  50:   0f 8e 19 00 00 00       jle    6f <clamp+0x6f>
  56:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  5a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  62:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  66:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  6a:   48 89 ec                mov    rsp,rbp
  6d:   5d                      pop    rbp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  6e:   c3                      ret
  6f:   48 89 f8                mov    rax,rdi
  72:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  76:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  7a:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  7e:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  82:   48 89 ec                mov    rsp,rbp
  85:   5d                      pop    rbp
  86:   c3                      ret
```

27.3.6 sign

ASH

```
pub func sign(int64 x): int64
```

The sign of a signed integer.

Parameter	Description
x	Value.

Returns. 1, 0 or -1.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func sign(int64 x): int64 {
    if (x > 0) { return 1; }
    if (x < 0) { return -1; }
    return 0;
}
```

Example.

Example 27-43. Using sign

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", sign(-9), sign(0), sign(9));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-1 0 1
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <sign>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 1b:   48 89 f8                mov    rax,rdi
 1e:   b9 00 00 00 00          mov    ecx,0x0
 23:   48 39 c8                cmp    rax,rcx
 26:   0f 8e 1a 00 00 00       jle    46 <sign+0x46>
 2c:   b8 01 00 00 00          mov    eax,0x1
 31:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 35:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 39:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 3d:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 41:   48 89 ec                mov    rsp,rbp
 44:   5d                      pop    rbp
 45:   c3                      ret
 46:   48 89 f8                mov    rax,rdi
 49:   b9 00 00 00 00          mov    ecx,0x0
 4e:   48 39 c8                cmp    rax,rcx
 51:   0f 8d 2a 00 00 00       jge    81 <sign+0x81>
 57:   b8 00 00 00 00          mov    eax,0x0
 5c:   49 89 c2                mov    r10,rax
 5f:   4c 89 d0                mov    rax,r10
 62:   48 83 e8 01             sub    rax,0x1
 66:   49 89 c2                mov    r10,rax
 69:   4c 89 d0                mov    rax,r10
 6c:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 70:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 74:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 78:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 7c:   48 89 ec                mov    rsp,rbp
 7f:   5d                      pop    rbp
 80:   c3                      ret
 81:   48 31 c0                xor    rax,rax
 84:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 88:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 8c:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 90:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 94:   48 89 ec                mov    rsp,rbp
 97:   5d                      pop    rbp
 98:   c3                      ret
```

27.3.7 gcd

ASH

```
pub func gcd(int64 a, int64 b): int64
```

Greatest common divisor, computed with Euclid's algorithm on the absolute values.

Parameter	Description
a	First value.
b	Second value.

Returns. The GCD (0 if both are 0).

Implementation. The source of the function in `nlib/math.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH

```
pub func gcd(int64 a, int64 b): int64 {
    int64 x = abs(a);
    int64 y = abs(b);
    while (y != 0) {
        int64 t = x % y;
        x = y;
        y = t;
    }
    return x;
}
```

Example.

Example 27-44. Using gcd

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", gcd(84, 36), gcd(-21, 14));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
12 7
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000070 <gcd>:
  70:   55                      push   rbp
  71:   48 89 e5                mov    rbp,rsp
  74:   48 81 ec 40 00 00 00    sub    rsp,0x40
  7b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  7f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  83:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  87:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8b:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  8f:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
  93:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  97:   49 89 c4                mov    r12,rax
  9a:   b8 00 00 00 00          mov    eax,0x0
  9f:   49 89 c5                mov    r13,rax
  a2:   4c 89 e7                mov    rdi,r12
  a5:   e8 56 ff ff ff          call   0 <abs>
  aa:   49 89 c5                mov    r13,rax
  ad:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
  b1:   49 89 c6                mov    r14,rax
  b4:   b8 00 00 00 00          mov    eax,0x0
  b9:   49 89 c7                mov    r15,rax
  bc:   4c 89 f7                mov    rdi,r14
  bf:   e8 3c ff ff ff          call   0 <abs>
  c4:   49 89 c7                mov    r15,rax
  c7:   4c 89 f8                mov    rax,r15
  ca:   b9 00 00 00 00          mov    ecx,0x0
  cf:   48 39 c8                cmp    rax,rcx
  d2:   0f 84 29 00 00 00       je     101 <gcd+0x91>
  d8:   4c 89 e8                mov    rax,r13
  db:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  df:   4c 89 f9                mov    rcx,r15
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  e2:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
  e6:   48 99                   cqo
  e8:   48 f7 f9                idiv   rcx
  eb:   48 89 55 c8             mov    QWORD PTR [rbp-0x38],rdx
  ef:   4c 89 f8                mov    rax,r15
  f2:   49 89 c5                mov    r13,rax
  f5:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
  f9:   49 89 c7                mov    r15,rax
  fc:   e9 c6 ff ff ff          jmp    c7 <gcd+0x57>
 101:   4c 89 e8                mov    rax,r13
 104:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 108:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 10c:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 110:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 114:   48 89 ec                mov    rsp,rbp
 117:   5d                      pop    rbp
 118:   c3                      ret
```

27.3.8 lcm

ASH

```
pub func lcm(int64 a, int64 b): int64
```

Least common multiple.

Parameter	Description
a	First value.
b	Second value.

Returns. The LCM, or 0 if either value is 0.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func lcm(int64 a, int64 b): int64 {
    if (a == 0 || b == 0) { return 0; }
    return abs(a / gcd(a, b) * b);
}
```

Example.

Example 27-45. Using lcm

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d\n", lcm(4, 6));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
12
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000119 <lcm>:
 119:   55                      push   rbp
 11a:   48 89 e5                mov    rbp,rsp
 11d:   48 81 ec 40 00 00 00    sub    rsp,0x40
 124:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 128:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 12c:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 130:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 134:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 138:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 13c:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 140:   b9 00 00 00 00          mov    ecx,0x0
 145:   48 39 c8                cmp    rax,rcx
 148:   0f 84 12 00 00 00       je     160 <lcm+0x47>
 14e:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 152:   b9 00 00 00 00          mov    ecx,0x0
 157:   48 39 c8                cmp    rax,rcx
 15a:   0f 85 18 00 00 00       jne    178 <lcm+0x5f>
 160:   48 31 c0                xor    rax,rax
 163:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 167:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 16b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 16f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 173:   48 89 ec                mov    rsp,rbp
 176:   5d                      pop    rbp
 177:   c3                      ret
 178:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 17c:   49 89 c4                mov    r12,rax
 17f:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 183:   49 89 c5                mov    r13,rax
 186:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 18a:   49 89 c6                mov    r14,rax
 18d:   b8 00 00 00 00          mov    eax,0x0
 192:   49 89 c7                mov    r15,rax
 195:   4c 89 ef                mov    rdi,r13
 198:   4c 89 f6                mov    rsi,r14
 19b:   e8 d0 fe ff ff          call   70 <gcd>
 1a0:   49 89 c7                mov    r15,rax
 1a3:   4c 89 e0                mov    rax,r12
 1a6:   4c 89 f9                mov    rcx,r15
 1a9:   4c 89 e0                mov    rax,r12
 1ac:   48 99                   cqo
 1ae:   48 f7 f9                idiv   rcx
 1b1:   49 89 c4                mov    r12,rax
 1b4:   4c 89 e0                mov    rax,r12
 1b7:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 1bb:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 1bf:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
 1c3:   48 0f af c1             imul   rax,rcx
 1c7:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 1cb:   b8 00 00 00 00          mov    eax,0x0
 1d0:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 1d4:   48 8b 7d c8             mov    rdi,QWORD PTR [rbp-0x38]
 1d8:   e8 23 fe ff ff          call   0 <abs>
 1dd:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 1e1:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
 1e5:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 1e9:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 1ed:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 1f1:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 1f5:   48 89 ec                mov    rsp,rbp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 1f8:   5d                      pop    rbp
 1f9:   c3                      ret
```

27.3.9 ipow

ASH

```
pub func ipow(int64 base, int64 exp): int64
```

Integer power *base* raised to *exp*, computed by repeated squaring in O(log exp) multiplications. The result wraps around modulo 2^64.

Parameter	Description
`base`	Base.
`exp`	Exponent, >= 0.

Returns. *base^exp*.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func ipow(int64 base, int64 exp): int64 {
    int64 r = 1;
    int64 b = base;
    int64 e = exp;
    while (e > 0) {
        if ((e & 1) == 1) { r = r * b; }
        b = b * b;
        e = e >> 1;
    }
    return r;
}
```

Example.

Example 27-46. Using ipow

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%lld %lld\n", ipow(3, 13), ipow(2, 62));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1594323 4611686018427387904
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <ipow>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
1b:   b8 01 00 00 00          mov    eax,0x1
20:   49 89 c2                mov    r10,rax
23:   48 89 f8                mov    rax,rdi
26:   49 89 c3                mov    r11,rax
29:   48 89 f0                mov    rax,rsi
2c:   49 89 c0                mov    r8,rax
2f:   4c 89 c0                mov    rax,r8
32:   b9 00 00 00 00          mov    ecx,0x0
37:   48 39 c8                cmp    rax,rcx
3a:   0f 8e 49 00 00 00       jle    89 <ipow+0x89>
40:   4c 89 c0                mov    rax,r8
43:   49 89 c1                mov    r9,rax
46:   4c 89 c8                mov    rax,r9
49:   48 83 e0 01             and    rax,0x1
4d:   49 89 c1                mov    r9,rax
50:   4c 89 c8                mov    rax,r9
53:   b9 01 00 00 00          mov    ecx,0x1
58:   48 39 c8                cmp    rax,rcx
5b:   0f 85 0d 00 00 00       jne    6e <ipow+0x6e>
61:   4c 89 d0                mov    rax,r10
64:   4c 89 d9                mov    rcx,r11
67:   48 0f af c1             imul   rax,rcx
6b:   49 89 c2                mov    r10,rax
6e:   4c 89 d8                mov    rax,r11
71:   4c 89 d9                mov    rcx,r11
74:   48 0f af c1             imul   rax,rcx
78:   49 89 c3                mov    r11,rax
7b:   4c 89 c0                mov    rax,r8
7e:   48 d1 f8                sar    rax,1
81:   49 89 c0                mov    r8,rax
84:   e9 a6 ff ff ff          jmp    2f <ipow+0x2f>
89:   4c 89 d0                mov    rax,r10
8c:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
90:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
94:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
98:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
9c:   48 89 ec                mov    rsp,rbp
9f:   5d                      pop    rbp
a0:   c3                      ret
```

27.3.10 isqrt

ASH

```
pub func isqrt(int64 n): int64
```

Integer square root (the floor of the real square root), by Newton's method.

Parameter	Description
n	Value >= 0.

Returns. floor(sqrt(*n*)).

Implementation. The source of the function in `nlib/math.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH

```
pub func isqrt(int64 n): int64 {
    if (n < 2) { return n; }
    int64 x = n;
    int64 y = (x + 1) / 2;
    while (y < x) {
        x = y;
        y = (x + n / x) / 2;
    }
    return x;
}
```

Example.

Example 27-47. Using isqrt

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", isqrt(99980001), isqrt(2), isqrt(1000000));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
9999 1 1000
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <isqrt>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 02 00 00 00          mov    ecx,0x2
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 8d 18 00 00 00       jge    44 <isqrt+0x44>
  2c:   48 89 f8                mov    rax,rdi
  2f:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  33:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  37:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  3b:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  3f:   48 89 ec                mov    rsp,rbp
  42:   5d                      pop    rbp
  43:   c3                      ret
  44:   48 89 f8                mov    rax,rdi
  47:   49 89 c2                mov    r10,rax
  4a:   4c 89 d0                mov    rax,r10
  4d:   49 89 c3                mov    r11,rax
  50:   4c 89 d8                mov    rax,r11
  53:   48 83 c0 01             add    rax,0x1
  57:   49 89 c3                mov    r11,rax
  5a:   4c 89 d8                mov    rax,r11
  5d:   49 89 c0                mov    r8,rax
  60:   b9 02 00 00 00          mov    ecx,0x2
  65:   4c 89 c0                mov    rax,r8
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
68:   48 99                   cqo
6a:   48 f7 f9                idiv   rcx
6d:   49 89 c0                mov    r8,rax
70:   4c 89 c0                mov    rax,r8
73:   4c 89 d1                mov    rcx,r10
76:   48 39 c8                cmp    rax,rcx
79:   0f 8d 4a 00 00 00       jge    c9 <isqrt+0xc9>
7f:   4c 89 c0                mov    rax,r8
82:   49 89 c2                mov    r10,rax
85:   4c 89 d0                mov    rax,r10
88:   49 89 c1                mov    r9,rax
8b:   48 89 f8                mov    rax,rdi
8e:   49 89 c4                mov    r12,rax
91:   4c 89 e0                mov    rax,r12
94:   4c 89 d1                mov    rcx,r10
97:   4c 89 e0                mov    rax,r12
9a:   48 99                   cqo
9c:   48 f7 f9                idiv   rcx
9f:   49 89 c4                mov    r12,rax
a2:   4c 89 c8                mov    rax,r9
a5:   4c 89 e1                mov    rcx,r12
a8:   48 01 c8                add    rax,rcx
ab:   49 89 c1                mov    r9,rax
ae:   4c 89 c8                mov    rax,r9
b1:   49 89 c0                mov    r8,rax
b4:   b9 02 00 00 00          mov    ecx,0x2
b9:   4c 89 c0                mov    rax,r8
bc:   48 99                   cqo
be:   48 f7 f9                idiv   rcx
c1:   49 89 c0                mov    r8,rax
c4:   e9 a7 ff ff ff          jmp    70 <isqrt+0x70>
c9:   4c 89 d0                mov    rax,r10
cc:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
d0:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
d4:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
d8:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
dc:   48 89 ec                mov    rsp,rbp
df:   5d                      pop    rbp
e0:   c3                      ret
```

27.3.11 ilog2

ASH

```
pub func ilog2(int64 n): int64
```

Position of the highest set bit: the floor of the base-2 logarithm.

Parameter	Description
n	Value.

Returns. floor(log2 *n*), or -1 if *n* <= 0.

Implementation. The source of the function in `nlib/math.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH

```
pub func ilog2(int64 n): int64 {
    if (n <= 0) { return -1; }
    int64 r = 0;
    int64 v = n;
    while (v > 1) {
        v = v >> 1;
        r++;
    }
    return r;
}
```

Example.

Example 27-48. Using ilog2

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", ilog2(1), ilog2(1024), ilog2(1500));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0 10 10
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <ilog2>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 00 00 00 00          mov    ecx,0x0
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 8f 2a 00 00 00       jg     56 <ilog2+0x56>
  2c:   b8 00 00 00 00          mov    eax,0x0
  31:   49 89 c2                mov    r10,rax
  34:   4c 89 d0                mov    rax,r10
  37:   48 83 e8 01             sub    rax,0x1
  3b:   49 89 c2                mov    r10,rax
  3e:   4c 89 d0                mov    rax,r10
  41:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  45:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  49:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  4d:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  51:   48 89 ec                mov    rsp,rbp
  54:   5d                      pop    rbp
  55:   c3                      ret
  56:   b8 00 00 00 00          mov    eax,0x0
  5b:   49 89 c3                mov    r11,rax
  5e:   48 89 f8                mov    rax,rdi
  61:   49 89 c0                mov    r8,rax
  64:   4c 89 c0                mov    rax,r8
  67:   b9 01 00 00 00          mov    ecx,0x1
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  6c:   48 39 c8                cmp    rax,rcx
  6f:   0f 8e 17 00 00 00       jle    8c <ilog2+0x8c>
  75:   4c 89 c0                mov    rax,r8
  78:   48 d1 f8                sar    rax,1
  7b:   49 89 c0                mov    r8,rax
  7e:   4c 89 d8                mov    rax,r11
  81:   48 ff c0                inc    rax
  84:   49 89 c3                mov    r11,rax
  87:   e9 d8 ff ff ff          jmp    64 <ilog2+0x64>
  8c:   4c 89 d8                mov    rax,r11
  8f:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  93:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  97:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  9b:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  9f:   48 89 ec                mov    rsp,rbp
  a2:   5d                      pop    rbp
  a3:   c3                      ret
```

27.3.12 ilog10

ASH

```
pub func ilog10(int64 n): int64
```

Floor of the base-10 logarithm: the number of decimal digits minus one.

Parameter	Description
n	Value.

Returns. floor(log10 *n*), or -1 if *n* <= 0.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func ilog10(int64 n): int64 {
    if (n <= 0) { return -1; }
    int64 r = 0;
    int64 v = n;
    while (v >= 10) {
        v = v / 10;
        r++;
    }
    return r;
}
```

Example.

Example 27-49. Using ilog10

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", ilog10(999), ilog10(1000));
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
2 3
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <ilog10>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 00 00 00 00          mov    ecx,0x0
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 8f 2a 00 00 00       jg     56 <ilog10+0x56>
  2c:   b8 00 00 00 00          mov    eax,0x0
  31:   49 89 c2                mov    r10,rax
  34:   4c 89 d0                mov    rax,r10
  37:   48 83 e8 01             sub    rax,0x1
  3b:   49 89 c2                mov    r10,rax
  3e:   4c 89 d0                mov    rax,r10
  41:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  45:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  49:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  4d:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  51:   48 89 ec                mov    rsp,rbp
  54:   5d                      pop    rbp
  55:   c3                      ret
  56:   b8 00 00 00 00          mov    eax,0x0
  5b:   49 89 c3                mov    r11,rax
  5e:   48 89 f8                mov    rax,rdi
  61:   49 89 c0                mov    r8,rax
  64:   4c 89 c0                mov    rax,r8
  67:   b9 0a 00 00 00          mov    ecx,0xa
  6c:   48 39 c8                cmp    rax,rcx
  6f:   0f 8c 1e 00 00 00       jl     93 <ilog10+0x93>
  75:   b9 0a 00 00 00          mov    ecx,0xa
  7a:   4c 89 c0                mov    rax,r8
  7d:   48 99                   cqo
  7f:   48 f7 f9                idiv   rcx
  82:   49 89 c0                mov    r8,rax
  85:   4c 89 d8                mov    rax,r11
  88:   48 ff c0                inc    rax
  8b:   49 89 c3                mov    r11,rax
  8e:   e9 d1 ff ff ff          jmp    64 <ilog10+0x64>
  93:   4c 89 d8                mov    rax,r11
  96:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  9a:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  9e:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  a2:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  a6:   48 89 ec                mov    rsp,rbp
  a9:   5d                      pop    rbp
  aa:   c3                      ret
```

27.3.13 factorial

ASH

```
pub func factorial(int64 n): int64
```

Copyright © 2026 Mallionic Inc. All rights reserved.

n! computed iteratively. Values above 20! overflow 64 bits.

Parameter	Description
n	Value >= 0.

Returns. *n*!.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func factorial(int64 n): int64 {
    int64 r = 1;
    for (int64 i = 2; i <= n; i++) {
        r = r * i;
    }
    return r;
}
```

Example.

Example 27-50. Using factorial

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%lld %lld\n", factorial(5), factorial(20));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
120 2432902008176640000
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <factorial>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 01 00 00 00          mov    eax,0x1
  20:   49 89 c2                mov    r10,rax
  23:   b8 02 00 00 00          mov    eax,0x2
  28:   49 89 c3                mov    r11,rax
  2b:   4c 89 d8                mov    rax,r11
  2e:   48 89 f9                mov    rcx,rdi
  31:   48 39 c8                cmp    rax,rcx
  34:   0f 8f 1b 00 00 00       jg     55 <factorial+0x55>
  3a:   4c 89 d0                mov    rax,r10
  3d:   4c 89 d9                mov    rcx,r11
  40:   48 0f af c1             imul   rax,rcx
  44:   49 89 c2                mov    r10,rax
  47:   4c 89 d8                mov    rax,r11
  4a:   48 ff c0                inc    rax
  4d:   49 89 c3                mov    r11,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  50:   e9 d6 ff ff ff          jmp    2b <factorial+0x2b>
  55:   4c 89 d0                mov    rax,r10
  58:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5c:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  60:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  64:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  68:   48 89 ec                mov    rsp,rbp
  6b:   5d                      pop    rbp
  6c:   c3                      ret
```

27.3.14 is_prime

ASH

```
pub func is_prime(int64 n): int64
```

Tests primality by trial division by odd numbers up to sqrt(*n*).

Parameter	Description
n	Value.

Returns. 1 if prime, else 0.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func is_prime(int64 n): int64 {
    if (n < 2) { return 0; }
    if (n < 4) { return 1; }
    if (n % 2 == 0) { return 0; }
    int64 i = 3;
    while (i * i <= n) {
        if (n % i == 0) { return 0; }
        i = i + 2;
    }
    return 1;
}
```

Example.

Example 27-51. Using is_prime

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", is_prime(97), is_prime(91), is_prime(2));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0 1
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <is_prime>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 02 00 00 00          mov    ecx,0x2
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 8d 18 00 00 00       jge    44 <is_prime+0x44>
  2c:   48 31 c0                xor    rax,rax
  2f:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  33:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  37:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  3b:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  3f:   48 89 ec                mov    rsp,rbp
  42:   5d                      pop    rbp
  43:   c3                      ret
  44:   48 89 f8                mov    rax,rdi
  47:   b9 04 00 00 00          mov    ecx,0x4
  4c:   48 39 c8                cmp    rax,rcx
  4f:   0f 8d 1a 00 00 00       jge    6f <is_prime+0x6f>
  55:   b8 01 00 00 00          mov    eax,0x1
  5a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  5e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  62:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  66:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  6a:   48 89 ec                mov    rsp,rbp
  6d:   5d                      pop    rbp
  6e:   c3                      ret
  6f:   48 89 f8                mov    rax,rdi
  72:   49 89 c2                mov    r10,rax
  75:   b9 02 00 00 00          mov    ecx,0x2
  7a:   4c 89 d0                mov    rax,r10
  7d:   48 99                   cqo
  7f:   48 f7 f9                idiv   rcx
  82:   49 89 d2                mov    r10,rdx
  85:   4c 89 d0                mov    rax,r10
  88:   b9 00 00 00 00          mov    ecx,0x0
  8d:   48 39 c8                cmp    rax,rcx
  90:   0f 85 18 00 00 00       jne    ae <is_prime+0xae>
  96:   48 31 c0                xor    rax,rax
  99:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  9d:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  a1:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  a5:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  a9:   48 89 ec                mov    rsp,rbp
  ac:   5d                      pop    rbp
  ad:   c3                      ret
  ae:   b8 03 00 00 00          mov    eax,0x3
  b3:   49 89 c3                mov    r11,rax
  b6:   4c 89 d8                mov    rax,r11
  b9:   49 89 c0                mov    r8,rax
  bc:   4c 89 c0                mov    rax,r8
  bf:   4c 89 d9                mov    rcx,r11
  c2:   48 0f af c1             imul   rax,rcx
  c6:   49 89 c0                mov    r8,rax
  c9:   4c 89 c0                mov    rax,r8
  cc:   48 89 f9                mov    rcx,rdi
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  cf:   48 39 c8                cmp    rax,rcx
  d2:   0f 8f 4c 00 00 00       jg     124 <is_prime+0x124>
  d8:   48 89 f8                mov    rax,rdi
  db:   49 89 c1                mov    r9,rax
  de:   4c 89 d9                mov    rcx,r11
  e1:   4c 89 c8                mov    rax,r9
  e4:   48 99                   cqo
  e6:   48 f7 f9                idiv   rcx
  e9:   49 89 d1                mov    r9,rdx
  ec:   4c 89 c8                mov    rax,r9
  ef:   b9 00 00 00 00          mov    ecx,0x0
  f4:   48 39 c8                cmp    rax,rcx
  f7:   0f 85 18 00 00 00       jne    115 <is_prime+0x115>
  fd:   48 31 c0                xor    rax,rax
 100:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 104:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 108:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 10c:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 110:   48 89 ec                mov    rsp,rbp
 113:   5d                      pop    rbp
 114:   c3                      ret
 115:   4c 89 d8                mov    rax,r11
 118:   48 83 c0 02             add    rax,0x2
 11c:   49 89 c3                mov    r11,rax
 11f:   e9 92 ff ff ff          jmp    b6 <is_prime+0xb6>
 124:   b8 01 00 00 00          mov    eax,0x1
 129:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 12d:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 131:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 135:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 139:   48 89 ec                mov    rsp,rbp
 13c:   5d                      pop    rbp
 13d:   c3                      ret
```

27.3.15 mod_pow

ASH

```
pub func mod_pow(int64 base, int64 exp, int64 m): int64
```

Modular exponentiation (*base*^*exp*) mod *m*, by repeated squaring. Intermediate products do not overflow as long as *m* < 2^31.

Parameter	Description
`base`	Base.
`exp`	Exponent >= 0.
`m`	Modulus > 0.

Returns. The result in [0, *m*).

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func mod_pow(int64 base, int64 exp, int64 m): int64 {
    int64 r = 1 % m;
    int64 b = base % m;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH (CONTINUED)

```
    int64 e = exp;
    while (e > 0) {
        if ((e & 1) == 1) { r = r * b % m; }
        b = b * b % m;
        e = e >> 1;
    }
    return r;
}
```

Example.

Example 27-52. Using mod_pow

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d\n", mod_pow(4, 13, 497));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
445
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <mod_pow>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 55 d8             mov    QWORD PTR [rbp-0x28],rdx
  1f:   b8 01 00 00 00          mov    eax,0x1
  24:   49 89 c2                mov    r10,rax
  27:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
  2b:   4c 89 d0                mov    rax,r10
  2e:   48 99                   cqo
  30:   48 f7 f9                idiv   rcx
  33:   49 89 d2                mov    r10,rdx
  36:   48 89 f8                mov    rax,rdi
  39:   49 89 c3                mov    r11,rax
  3c:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
  40:   4c 89 d8                mov    rax,r11
  43:   48 99                   cqo
  45:   48 f7 f9                idiv   rcx
  48:   49 89 d3                mov    r11,rdx
  4b:   48 89 f0                mov    rax,rsi
  4e:   49 89 c0                mov    r8,rax
  51:   4c 89 c0                mov    rax,r8
  54:   b9 00 00 00 00          mov    ecx,0x0
  59:   48 39 c8                cmp    rax,rcx
  5c:   0f 8e 7f 00 00 00       jle    e1 <mod_pow+0xe1>
  62:   4c 89 c0                mov    rax,r8
  65:   49 89 c1                mov    r9,rax
  68:   4c 89 c8                mov    rax,r9
  6b:   48 83 e0 01             and    rax,0x1
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
6f:   49 89 c1                mov    r9,rax
72:   4c 89 c8                mov    rax,r9
75:   b9 01 00 00 00          mov    ecx,0x1
7a:   48 39 c8                cmp    rax,rcx
7d:   0f 85 28 00 00 00       jne    ab <mod_pow+0xab>
83:   4c 89 d0                mov    rax,r10
86:   49 89 c4                mov    r12,rax
89:   4c 89 e0                mov    rax,r12
8c:   4c 89 d9                mov    rcx,r11
8f:   48 0f af c1             imul   rax,rcx
93:   49 89 c4                mov    r12,rax
96:   4c 89 e0                mov    rax,r12
99:   49 89 c2                mov    r10,rax
9c:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
a0:   4c 89 d0                mov    rax,r10
a3:   48 99                   cqo
a5:   48 f7 f9                idiv   rcx
a8:   49 89 d2                mov    r10,rdx
ab:   4c 89 d8                mov    rax,r11
ae:   49 89 c5                mov    r13,rax
b1:   4c 89 e8                mov    rax,r13
b4:   4c 89 d9                mov    rcx,r11
b7:   48 0f af c1             imul   rax,rcx
bb:   49 89 c5                mov    r13,rax
be:   4c 89 e8                mov    rax,r13
c1:   49 89 c3                mov    r11,rax
c4:   48 8b 4d d8             mov    rcx,QWORD PTR [rbp-0x28]
c8:   4c 89 d8                mov    rax,r11
cb:   48 99                   cqo
cd:   48 f7 f9                idiv   rcx
d0:   49 89 d3                mov    r11,rdx
d3:   4c 89 c0                mov    rax,r8
d6:   48 d1 f8                sar    rax,1
d9:   49 89 c0                mov    r8,rax
dc:   e9 70 ff ff ff          jmp    51 <mod_pow+0x51>
e1:   4c 89 d0                mov    rax,r10
e4:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
e8:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
ec:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
f0:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
f4:   48 89 ec                mov    rsp,rbp
f7:   5d                      pop    rbp
f8:   c3                      ret
```

27.3.16 div_floor

ASH

```
pub func div_floor(int64 a, int64 b): int64
```

Integer division rounded toward minus infinity (C and ASH `/` truncate toward zero).

Parameter	Description
a	Dividend.
b	Divisor, not 0.

Returns. floor(*a* / *b*).

Copyright © 2026 Mallionic Inc. All rights reserved.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func div_floor(int64 a, int64 b): int64 {
    int64 q = a / b;
    if (a % b != 0) {
        if (a < 0 && b > 0) { q = q - 1; }
        if (a > 0 && b < 0) { q = q - 1; }
    }
    return q;
}
```

Example.

Example 27-53. Using div_floor

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", -7 / 2, div_floor(-7, 2));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-3 -4
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <div_floor>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   49 89 c2                mov    r10,rax
  21:   48 89 f1                mov    rcx,rsi
  24:   4c 89 d0                mov    rax,r10
  27:   48 99                   cqo
  29:   48 f7 f9                idiv   rcx
  2c:   49 89 c2                mov    r10,rax
  2f:   48 89 f8                mov    rax,rdi
  32:   49 89 c3                mov    r11,rax
  35:   48 89 f1                mov    rcx,rsi
  38:   4c 89 d8                mov    rax,r11
  3b:   48 99                   cqo
  3d:   48 f7 f9                idiv   rcx
  40:   49 89 d3                mov    r11,rdx
  43:   4c 89 d8                mov    rax,r11
  46:   b9 00 00 00 00          mov    ecx,0x0
  4b:   48 39 c8                cmp    rax,rcx
  4e:   0f 84 58 00 00 00       je     ac <div_floor+0xac>
  54:   48 89 f8                mov    rax,rdi
  57:   b9 00 00 00 00          mov    ecx,0x0
  5c:   48 39 c8                cmp    rax,rcx
  5f:   0f 8d 1b 00 00 00       jge    80 <div_floor+0x80>
  65:   48 89 f0                mov    rax,rsi
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  68:   b9 00 00 00 00          mov    ecx,0x0
  6d:   48 39 c8                cmp    rax,rcx
  70:   0f 8e 0a 00 00 00       jle    80 <div_floor+0x80>
  76:   4c 89 d0                mov    rax,r10
  79:   48 83 e8 01             sub    rax,0x1
  7d:   49 89 c2                mov    r10,rax
  80:   48 89 f8                mov    rax,rdi
  83:   b9 00 00 00 00          mov    ecx,0x0
  88:   48 39 c8                cmp    rax,rcx
  8b:   0f 8e 1b 00 00 00       jle    ac <div_floor+0xac>
  91:   48 89 f0                mov    rax,rsi
  94:   b9 00 00 00 00          mov    ecx,0x0
  99:   48 39 c8                cmp    rax,rcx
  9c:   0f 8d 0a 00 00 00       jge    ac <div_floor+0xac>
  a2:   4c 89 d0                mov    rax,r10
  a5:   48 83 e8 01             sub    rax,0x1
  a9:   49 89 c2                mov    r10,rax
  ac:   4c 89 d0                mov    rax,r10
  af:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  b3:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  b7:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  bb:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  bf:   48 89 ec                mov    rsp,rbp
  c2:   5d                      pop    rbp
  c3:   c3                      ret
```

27.3.17 mod_floor

ASH

```
pub func mod_floor(int64 a, int64 b): int64
```

Remainder with the sign of the divisor (the companion of `div_floor`).

Parameter	Description
a	Dividend.
b	Divisor, not 0.

Returns. *a* - *b* * floor(*a* / *b*).

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func mod_floor(int64 a, int64 b): int64 {
    int64 r = a % b;
    if (r < 0 && b > 0) { r = r + b; }
    if (r > 0 && b < 0) { r = r + b; }
    return r;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-54. Using mod_floor

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", -7 % 3, mod_floor(-7, 3));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-1 2
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <mod_floor>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   49 89 c2                mov    r10,rax
  21:   48 89 f1                mov    rcx,rsi
  24:   4c 89 d0                mov    rax,r10
  27:   48 99                   cqo
  29:   48 f7 f9                idiv   rcx
  2c:   49 89 d2                mov    r10,rdx
  2f:   4c 89 d0                mov    rax,r10
  32:   b9 00 00 00 00          mov    ecx,0x0
  37:   48 39 c8                cmp    rax,rcx
  3a:   0f 8d 1d 00 00 00       jge    5d <mod_floor+0x5d>
  40:   48 89 f0                mov    rax,rsi
  43:   b9 00 00 00 00          mov    ecx,0x0
  48:   48 39 c8                cmp    rax,rcx
  4b:   0f 8e 0c 00 00 00       jle    5d <mod_floor+0x5d>
  51:   4c 89 d0                mov    rax,r10
  54:   48 89 f1                mov    rcx,rsi
  57:   48 01 c8                add    rax,rcx
  5a:   49 89 c2                mov    r10,rax
  5d:   4c 89 d0                mov    rax,r10
  60:   b9 00 00 00 00          mov    ecx,0x0
  65:   48 39 c8                cmp    rax,rcx
  68:   0f 8e 1d 00 00 00       jle    8b <mod_floor+0x8b>
  6e:   48 89 f0                mov    rax,rsi
  71:   b9 00 00 00 00          mov    ecx,0x0
  76:   48 39 c8                cmp    rax,rcx
  79:   0f 8d 0c 00 00 00       jge    8b <mod_floor+0x8b>
  7f:   4c 89 d0                mov    rax,r10
  82:   48 89 f1                mov    rcx,rsi
  85:   48 01 c8                add    rax,rcx
  88:   49 89 c2                mov    r10,rax
  8b:   4c 89 d0                mov    rax,r10
  8e:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  92:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  96:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  9a:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  9e:   48 89 ec                mov    rsp,rbp
  a1:   5d                      pop    rbp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  a2:   c3                      ret
```

27.3.18 is_pow2

ASH

```
pub func is_pow2(int64 n): int64
```

Tests whether *n* is a positive power of two (one bit set).

Parameter	Description
n	Value.

Returns. 1 or 0.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func is_pow2(int64 n): int64 {
    if (n > 0 && (n & (n - 1)) == 0) { return 1; }
    return 0;
}
```

Example.

Example 27-55. Using is_pow2

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d %d\n", is_pow2(64), is_pow2(65), is_pow2(0));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1 0 0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <is_pow2>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   b9 00 00 00 00          mov    ecx,0x0
  23:   48 39 c8                cmp    rax,rcx
  26:   0f 8e 4d 00 00 00       jle    79 <is_pow2+0x79>
  2c:   48 89 f8                mov    rax,rdi
  2f:   49 89 c2                mov    r10,rax
  32:   48 89 f8                mov    rax,rdi
  35:   49 89 c3                mov    r11,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  38:   4c 89 d8                mov    rax,r11
  3b:   48 83 e8 01             sub    rax,0x1
  3f:   49 89 c3                mov    r11,rax
  42:   4c 89 d0                mov    rax,r10
  45:   4c 89 d9                mov    rcx,r11
  48:   48 21 c8                and    rax,rcx
  4b:   49 89 c2                mov    r10,rax
  4e:   4c 89 d0                mov    rax,r10
  51:   b9 00 00 00 00          mov    ecx,0x0
  56:   48 39 c8                cmp    rax,rcx
  59:   0f 85 1a 00 00 00       jne    79 <is_pow2+0x79>
  5f:   b8 01 00 00 00          mov    eax,0x1
  64:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  68:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  6c:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  70:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  74:   48 89 ec                mov    rsp,rbp
  77:   5d                      pop    rbp
  78:   c3                      ret
  79:   48 31 c0                xor    rax,rax
  7c:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  80:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  84:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  88:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  8c:   48 89 ec                mov    rsp,rbp
  8f:   5d                      pop    rbp
  90:   c3                      ret
```

27.3.19 next_pow2

ASH

```
pub func next_pow2(int64 n): int64
```

The smallest power of two greater than or equal to *n*.

Parameter	Description
n	Value.

Returns. The power of two (1 for *n* <= 1).

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func next_pow2(int64 n): int64 {
    int64 p = 1;
    while (p < n) {
        p = p * 2;
    }
    return p;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-56. Using next_pow2

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", next_pow2(100), next_pow2(128));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
128 128
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <next_pow2>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 01 00 00 00          mov    eax,0x1
  20:   49 89 c2                mov    r10,rax
  23:   4c 89 d0                mov    rax,r10
  26:   48 89 f9                mov    rcx,rdi
  29:   48 39 c8                cmp    rax,rcx
  2c:   0f 8d 0f 00 00 00       jge    41 <next_pow2+0x41>
  32:   4c 89 d0                mov    rax,r10
  35:   48 6b c0 02             imul   rax,rax,0x2
  39:   49 89 c2                mov    r10,rax
  3c:   e9 e2 ff ff ff          jmp    23 <next_pow2+0x23>
  41:   4c 89 d0                mov    rax,r10
  44:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  48:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  4c:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  50:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  54:   48 89 ec                mov    rsp,rbp
  57:   5d                      pop    rbp
  58:   c3                      ret
```

27.3.20 popcount

ASH

```
pub func popcount(int64 n): int64
```

Counts the bits set to 1 in the 64-bit pattern of *n*.

Parameter	Description
n	Value.

Returns. The number of 1 bits (0 to 64).

Implementation. The source of the function in `nlib/math.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH

```
pub func popcount(int64 n): int64 {
    int64 c = 0;
    int64 v = n;
    for (int64 i = 0; i < 64; i++) {
        c = c + (v & 1);
        v = v >> 1;
    }
    return c;
}
```

Example.

Example 27-57. Using popcount

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", popcount(0xFF00FF), popcount(-1));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
16 64
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <popcount>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   b8 00 00 00 00          mov    eax,0x0
  20:   49 89 c2                mov    r10,rax
  23:   48 89 f8                mov    rax,rdi
  26:   49 89 c3                mov    r11,rax
  29:   b8 00 00 00 00          mov    eax,0x0
  2e:   49 89 c0                mov    r8,rax
  31:   4c 89 c0                mov    rax,r8
  34:   b9 40 00 00 00          mov    ecx,0x40
  39:   48 39 c8                cmp    rax,rcx
  3c:   0f 8d 33 00 00 00       jge    75 <popcount+0x75>
  42:   4c 89 d8                mov    rax,r11
  45:   49 89 c1                mov    r9,rax
  48:   4c 89 c8                mov    rax,r9
  4b:   48 83 e0 01             and    rax,0x1
  4f:   49 89 c1                mov    r9,rax
  52:   4c 89 d0                mov    rax,r10
  55:   4c 89 c9                mov    rcx,r9
  58:   48 01 c8                add    rax,rcx
  5b:   49 89 c2                mov    r10,rax
  5e:   4c 89 d8                mov    rax,r11
  61:   48 d1 f8                sar    rax,1
  64:   49 89 c3                mov    r11,rax
  67:   4c 89 c0                mov    rax,r8
  6a:   48 ff c0                inc    rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  6d:   49 89 c0                mov    r8,rax
  70:   e9 bc ff ff ff          jmp    31 <popcount+0x31>
  75:   4c 89 d0                mov    rax,r10
  78:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  7c:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  80:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  84:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  88:   48 89 ec                mov    rsp,rbp
  8b:   5d                      pop    rbp
  8c:   c3                      ret
```

27.3.21 align_up

ASH

```
pub func align_up(int64 x, int64 a): int64
```

Rounds *x* up to the next multiple of *a*.

Parameter	Description
x	Value.
a	Alignment > 0.

Returns. The aligned value.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func align_up(int64 x, int64 a): int64 {
    return (x + a - 1) / a * a;
}
```

Example.

Example 27-58. Using align_up

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", align_up(13, 8), align_up(16, 8));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
16 16
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <align_up>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   49 89 c2                mov    r10,rax
  21:   4c 89 d0                mov    rax,r10
  24:   48 89 f1                mov    rcx,rsi
  27:   48 01 c8                add    rax,rcx
  2a:   49 89 c2                mov    r10,rax
  2d:   4c 89 d0                mov    rax,r10
  30:   49 89 c3                mov    r11,rax
  33:   4c 89 d8                mov    rax,r11
  36:   48 83 e8 01             sub    rax,0x1
  3a:   49 89 c3                mov    r11,rax
  3d:   4c 89 d8                mov    rax,r11
  40:   49 89 c0                mov    r8,rax
  43:   48 89 f1                mov    rcx,rsi
  46:   4c 89 c0                mov    rax,r8
  49:   48 99                   cqo
  4b:   48 f7 f9                idiv   rcx
  4e:   49 89 c0                mov    r8,rax
  51:   4c 89 c0                mov    rax,r8
  54:   49 89 c1                mov    r9,rax
  57:   4c 89 c8                mov    rax,r9
  5a:   48 89 f1                mov    rcx,rsi
  5d:   48 0f af c1             imul   rax,rcx
  61:   49 89 c1                mov    r9,rax
  64:   4c 89 c8                mov    rax,r9
  67:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  6b:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  6f:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  73:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  77:   48 89 ec                mov    rsp,rbp
  7a:   5d                      pop    rbp
  7b:   c3                      ret
```

27.3.22 align_down

ASH

```
pub func align_down(int64 x, int64 a): int64
```

Rounds *x* down to a multiple of *a*.

Parameter	Description
x	Value.
a	Alignment > 0.

Returns. The aligned value.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func align_down(int64 x, int64 a): int64 {
    return x / a * a;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-59. Using align_down

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d\n", align_down(13, 8));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
8
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <align_down>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   49 89 c2                mov    r10,rax
  21:   48 89 f1                mov    rcx,rsi
  24:   4c 89 d0                mov    rax,r10
  27:   48 99                   cqo
  29:   48 f7 f9                idiv   rcx
  2c:   49 89 c2                mov    r10,rax
  2f:   4c 89 d0                mov    rax,r10
  32:   49 89 c3                mov    r11,rax
  35:   4c 89 d8                mov    rax,r11
  38:   48 89 f1                mov    rcx,rsi
  3b:   48 0f af c1             imul   rax,rcx
  3f:   49 89 c3                mov    r11,rax
  42:   4c 89 d8                mov    rax,r11
  45:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  49:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  4d:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  51:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  55:   48 89 ec                mov    rsp,rbp
  58:   5d                      pop    rbp
  59:   c3                      ret
```

27.3.23 fx

ASH

```
pub func fx(int64 n): int64
```

Converts an integer to Q16.16 fixed point (multiplies by 65536).

Parameter	Description
n	Integer value.

Returns. *n* in Q16.16.

Implementation. The source of the function in `nlib/math.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH

```
pub func fx(int64 n): int64 {
    return n * 65536;
}
```

Example.

Example 27-60. Using fx

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d\n", fx(3));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
196608
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <fx>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 20 00 00 00    sub    rsp,0x20
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 f8                mov    rax,rdi
  1e:   49 89 c2                mov    r10,rax
  21:   4c 89 d0                mov    rax,r10
  24:   48 69 c0 00 00 01 00    imul   rax,rax,0x10000
  2b:   49 89 c2                mov    r10,rax
  2e:   4c 89 d0                mov    rax,r10
  31:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  35:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  39:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  3d:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  41:   48 89 ec                mov    rsp,rbp
  44:   5d                      pop    rbp
  45:   c3                      ret
```

27.3.24 fx_int

ASH

```
pub func fx_int(int64 a): int64
```

Integer part of a Q16.16 value, truncated toward zero.

Parameter	Description
a	Q16.16 value.

Returns. The integer part.

Implementation. The source of the function in `nlib/math.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH

```
pub func fx_int(int64 a): int64 {
    return a / 65536;
}
```

Example.

Example 27-61. Using fx_int

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", fx_int(fx(7) + 40000), fx_int(0 - fx(7) - 40000));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
7 -7
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000046 <fx_int>:
  46:   55                      push   rbp
  47:   48 89 e5                mov    rbp,rsp
  4a:   48 81 ec 20 00 00 00    sub    rsp,0x20
  51:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  55:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  59:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  5d:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  61:   48 89 f8                mov    rax,rdi
  64:   49 89 c2                mov    r10,rax
  67:   b9 00 00 01 00          mov    ecx,0x10000
  6c:   4c 89 d0                mov    rax,r10
  6f:   48 99                   cqo
  71:   48 f7 f9                idiv   rcx
  74:   49 89 c2                mov    r10,rax
  77:   4c 89 d0                mov    rax,r10
  7a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  7e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  82:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  86:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  8a:   48 89 ec                mov    rsp,rbp
  8d:   5d                      pop    rbp
  8e:   c3                      ret
```

27.3.25 fx_round

ASH

```
pub func fx_round(int64 a): int64
```

Rounds a Q16.16 value to the nearest integer (halves away from zero).

Parameter	Description
a	Q16.16 value.

Returns. The rounded integer.

Copyright © 2026 Mallionic Inc. All rights reserved.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func fx_round(int64 a): int64 {
    if (a < 0) { return (a - 32768) / 65536; }
    return (a + 32768) / 65536;
}
```

Example.

Example 27-62. Using fx_round

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d %d\n", fx_round(fx(2) + 32768), fx_round(fx(2) + 32767));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3 2
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000046 <fx_round>:
  46:   55                      push   rbp
  47:   48 89 e5                mov    rbp,rsp
  4a:   48 81 ec 20 00 00 00    sub    rsp,0x20
  51:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  55:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  59:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  5d:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  61:   48 89 f8                mov    rax,rdi
  64:   b9 00 00 00 00          mov    ecx,0x0
  69:   48 39 c8                cmp    rax,rcx
  6c:   0f 8d 41 00 00 00       jge    b3 <fx_round+0x6d>
  72:   48 89 f8                mov    rax,rdi
  75:   49 89 c2                mov    r10,rax
  78:   4c 89 d0                mov    rax,r10
  7b:   48 81 e8 00 80 00 00    sub    rax,0x8000
  82:   49 89 c2                mov    r10,rax
  85:   4c 89 d0                mov    rax,r10
  88:   49 89 c3                mov    r11,rax
  8b:   b9 00 00 01 00          mov    ecx,0x10000
  90:   4c 89 d8                mov    rax,r11
  93:   48 99                   cqo
  95:   48 f7 f9                idiv   rcx
  98:   49 89 c3                mov    r11,rax
  9b:   4c 89 d8                mov    rax,r11
  9e:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  a2:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  a6:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  aa:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  ae:   48 89 ec                mov    rsp,rbp
  b1:   5d                      pop    rbp
  b2:   c3                      ret
  b3:   48 89 f8                mov    rax,rdi
  b6:   49 89 c0                mov    r8,rax
  b9:   4c 89 c0                mov    rax,r8
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  bc:   48 81 c0 00 80 00 00    add    rax,0x8000
  c3:   49 89 c0                mov    r8,rax
  c6:   4c 89 c0                mov    rax,r8
  c9:   49 89 c3                mov    r11,rax
  cc:   b9 00 00 01 00          mov    ecx,0x10000
  d1:   4c 89 d8                mov    rax,r11
  d4:   48 99                   cqo
  d6:   48 f7 f9                idiv   rcx
  d9:   49 89 c3                mov    r11,rax
  dc:   4c 89 d8                mov    rax,r11
  df:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  e3:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  e7:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  eb:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  ef:   48 89 ec                mov    rsp,rbp
  f2:   5d                      pop    rbp
  f3:   c3                      ret
```

27.3.26 fx_frac

ASH

```
pub func fx_frac(int64 a): int64
```

Fractional part of a Q16.16 value, as a number of 1/65536 units (sign of *a*).

Parameter	Description
a	Q16.16 value.

Returns. *a* mod 65536.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func fx_frac(int64 a): int64 {
    return a % 65536;
}
```

Example.

Example 27-63. Using fx_frac

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d\n", fx_frac(fx(5) + 16384));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
16384
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000046 <fx_frac>:
  46:   55                      push   rbp
  47:   48 89 e5                mov    rbp,rsp
  4a:   48 81 ec 20 00 00 00    sub    rsp,0x20
  51:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  55:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  59:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  5d:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  61:   48 89 f8                mov    rax,rdi
  64:   49 89 c2                mov    r10,rax
  67:   b9 00 00 01 00          mov    ecx,0x10000
  6c:   4c 89 d0                mov    rax,r10
  6f:   48 99                   cqo
  71:   48 f7 f9                idiv   rcx
  74:   49 89 d2                mov    r10,rdx
  77:   4c 89 d0                mov    rax,r10
  7a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  7e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  82:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  86:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  8a:   48 89 ec                mov    rsp,rbp
  8d:   5d                      pop    rbp
  8e:   c3                      ret
```

27.3.27 fx_abs

ASH

```
pub func fx_abs(int64 a): int64
```

Absolute value of a Q16.16 value.

Parameter	Description
a	Q16.16 value.

Returns. |*a*|.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func fx_abs(int64 a): int64 {
    return abs(a);
}
```

Example.

Example 27-64. Using fx_abs

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    printf("%d\n", fx_abs(0 - fx(2)));
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
131072
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000000b6 <fx_abs>:
  b6:   55                      push   rbp
  b7:   48 89 e5                mov    rbp,rsp
  ba:   48 81 ec 30 00 00 00    sub    rsp,0x30
  c1:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  c5:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  c9:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  cd:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  d1:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  d5:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  d9:   49 89 c4                mov    r12,rax
  dc:   b8 00 00 00 00          mov    eax,0x0
  e1:   49 89 c5                mov    r13,rax
  e4:   4c 89 e7                mov    rdi,r12
  e7:   e8 5a ff ff ff          call   46 <abs>
  ec:   49 89 c5                mov    r13,rax
  ef:   4c 89 e8                mov    rax,r13
  f2:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  f6:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  fa:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  fe:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 102:   48 89 ec                mov    rsp,rbp
 105:   5d                      pop    rbp
 106:   c3                      ret
```

27.3.28 fx_mul

ASH

```
pub func fx_mul(int64 a, int64 b): int64
```

Multiplies two Q16.16 values: (*a* * *b*) / 65536. The intermediate product must fit in 64 bits (|a*b| < 2^63, i.e. operands up to about 2^15 in magnitude each).

Parameter	Description
a	Q16.16 value.
b	Q16.16 value.

Returns. The product in Q16.16.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func fx_mul(int64 a, int64 b): int64 {
    return a * b / 65536;
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-65. Using fx_mul

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    printf("%s\n", fx_print_buf(fx_mul(fx(3) + 32768, fx(2)), &b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
7.0000
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000117 <fx_mul>:
 117:   55                      push   rbp
 118:   48 89 e5                mov    rbp,rsp
 11b:   48 81 ec 20 00 00 00    sub    rsp,0x20
 122:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 126:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 12a:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 12e:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 132:   48 89 f8                mov    rax,rdi
 135:   49 89 c2                mov    r10,rax
 138:   4c 89 d0                mov    rax,r10
 13b:   48 89 f1                mov    rcx,rsi
 13e:   48 0f af c1             imul   rax,rcx
 142:   49 89 c2                mov    r10,rax
 145:   4c 89 d0                mov    rax,r10
 148:   49 89 c3                mov    r11,rax
 14b:   b9 00 00 01 00          mov    ecx,0x10000
 150:   4c 89 d8                mov    rax,r11
 153:   48 99                   cqo
 155:   48 f7 f9                idiv   rcx
 158:   49 89 c3                mov    r11,rax
 15b:   4c 89 d8                mov    rax,r11
 15e:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 162:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 166:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 16a:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 16e:   48 89 ec                mov    rsp,rbp
 171:   5d                      pop    rbp
 172:   c3                      ret
```

27.3.29 fx_div

ASH

```
pub func fx_div(int64 a, int64 b): int64
```

Divides two Q16.16 values: (*a* * 65536) / *b*.

Parameter	Description
a	Dividend, Q16.16.
b	Divisor, Q16.16, not 0.

Copyright © 2026 Mallionic Inc. All rights reserved.

Returns. The quotient in Q16.16.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func fx_div(int64 a, int64 b): int64 {
    return a * 65536 / b;
}
```

Example.

Example 27-66. Using fx_div

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    printf("%s\n", fx_print_buf(fx_div(fx(1), fx(3)), &b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0.3333
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000117 <fx_div>:
 117:   55                      push   rbp
 118:   48 89 e5                mov    rbp,rsp
 11b:   48 81 ec 20 00 00 00    sub    rsp,0x20
 122:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 126:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 12a:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 12e:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 132:   48 89 f8                mov    rax,rdi
 135:   49 89 c2                mov    r10,rax
 138:   4c 89 d0                mov    rax,r10
 13b:   48 69 c0 00 00 01 00    imul   rax,rax,0x10000
 142:   49 89 c2                mov    r10,rax
 145:   4c 89 d0                mov    rax,r10
 148:   49 89 c3                mov    r11,rax
 14b:   48 89 f1                mov    rcx,rsi
 14e:   4c 89 d8                mov    rax,r11
 151:   48 99                   cqo
 153:   48 f7 f9                idiv   rcx
 156:   49 89 c3                mov    r11,rax
 159:   4c 89 d8                mov    rax,r11
 15c:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 160:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 164:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 168:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 16c:   48 89 ec                mov    rsp,rbp
 16f:   5d                      pop    rbp
 170:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.3.30 fx_sqrt

```ash
pub func fx_sqrt(int64 a): int64
```

Square root of a Q16.16 value, computed as `isqrt(a * 65536)`.

Parameter	Description
a	Q16.16 value >= 0.

Returns. sqrt(*a*) in Q16.16, or 0 for *a* <= 0.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```ash
pub func fx_sqrt(int64 a): int64 {
    if (a <= 0) { return 0; }
    return isqrt(a * 65536);
}
```

Example.

Example 27-67. Using fx_sqrt

```ash
import <math>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    printf("%s\n", fx_print_buf(fx_sqrt(fx(2)), &b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1.4141
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000001f8 <fx_sqrt>:
 1f8:   55                      push   rbp
 1f9:   48 89 e5                mov    rbp,rsp
 1fc:   48 81 ec 30 00 00 00    sub    rsp,0x30
 203:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 207:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 20b:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 20f:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 213:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 217:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 21b:   b9 00 00 00 00          mov    ecx,0x0
 220:   48 39 c8                cmp    rax,rcx
 223:   0f 8f 18 00 00 00       jg     241 <fx_sqrt+0x49>
 229:   48 31 c0                xor    rax,rax
 22c:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 230:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 234:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 238:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 23c:   48 89 ec                mov    rsp,rbp
 23f:   5d                      pop    rbp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
240:   c3                      ret
241:   48 8b 45 d8            mov    rax,QWORD PTR [rbp-0x28]
245:   49 89 c4                mov    r12,rax
248:   4c 89 e0                mov    rax,r12
24b:   48 69 c0 00 00 01 00    imul   rax,rax,0x10000
252:   49 89 c4                mov    r12,rax
255:   b8 00 00 00 00          mov    eax,0x0
25a:   49 89 c5                mov    r13,rax
25d:   4c 89 e7                mov    rdi,r12
260:   e8 b2 fe ff ff          call   117 <isqrt>
265:   49 89 c5                mov    r13,rax
268:   4c 89 e8                mov    rax,r13
26b:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
26f:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
273:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
277:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
27b:   48 89 ec                mov    rsp,rbp
27e:   5d                      pop    rbp
27f:   c3                      ret
```

27.3.31 fx_deg

ASH

```
pub func fx_deg(int64 grados): int64
```

Converts an integer number of degrees to radians in Q16.16 (multiplies by pi/180).

Parameter	Description
`degrees`	Angle in whole degrees (parameter `grados`).

Returns. The angle in radians, Q16.16.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func fx_deg(int64 grados): int64 {
    return grados * 205887 / 180;
}
```

Example.

Example 27-68. Using fx_deg

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    printf("%s\n", fx_print_buf(fx_deg(180), &b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3.1415
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000000d1 <fx_deg>:
  d1:   55                      push   rbp
  d2:   48 89 e5                mov    rbp,rsp
  d5:   48 81 ec 20 00 00 00    sub    rsp,0x20
  dc:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  e0:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  e4:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  e8:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  ec:   48 89 f8                mov    rax,rdi
  ef:   49 89 c2                mov    r10,rax
  f2:   4c 89 d0                mov    rax,r10
  f5:   48 69 c0 3f 24 03 00    imul   rax,rax,0x3243f
  fc:   49 89 c2                mov    r10,rax
  ff:   4c 89 d0                mov    rax,r10
 102:   49 89 c3                mov    r11,rax
 105:   b9 b4 00 00 00          mov    ecx,0xb4
 10a:   4c 89 d8                mov    rax,r11
 10d:   48 99                   cqo
 10f:   48 f7 f9                idiv   rcx
 112:   49 89 c3                mov    r11,rax
 115:   4c 89 d8                mov    rax,r11
 118:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 11c:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 120:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 124:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 128:   48 89 ec                mov    rsp,rbp
 12b:   5d                      pop    rbp
 12c:   c3                      ret
```

27.3.32 fx_sin

ASH

```
pub func fx_sin(int64 a): int64
```

Sine of an angle in radians (Q16.16). The argument is reduced to [-pi, pi), folded into [-pi/2, pi/2] by symmetry, and evaluated with the Taylor series up to x^9. The error is below 0.0001 over the whole range. No FPU instruction is used, so it works on bare metal.

Parameter	Description
a	Angle in radians, Q16.16.

Returns. sin(*a*) in Q16.16.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func fx_sin(int64 a): int64 {
    int64 x = mod_floor(a + 205887, 411775) - 205887;
    if (x > 102944) { x = 205887 - x; }
    if (x < -102944) { x = -205887 - x; }
    int64 x2 = x * x / 65536;
    int64 t = x;
    int64 s = x;
    t = 0 - t * x2 / 65536 / 6;    s = s + t;
    t = 0 - t * x2 / 65536 / 20;   s = s + t;
    t = 0 - t * x2 / 65536 / 42;   s = s + t;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH (CONTINUED)

```
    t = 0 - t * x2 / 65536 / 72;   s = s + t;
    return s;
}
```

Example.

Example 27-69. Using fx_sin

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    printf("%s ", fx_print_buf(fx_sin(fx_deg(30)), &b));
    printf("%s\n", fx_print_buf(fx_sin(fx_deg(90)), &b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0.5000 1.0000
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000001d0 <fx_sin>:
     1d0:       55                      push   rbp
     1d1:       48 89 e5                mov    rbp,rsp
     1d4:       48 81 ec f0 00 00 00    sub    rsp,0xf0
     1db:       4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
     1df:       4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
     1e3:       4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
     1e7:       4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
     1eb:       48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
     1ef:       48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
     1f3:       49 89 c4                mov    r12,rax
     1f6:       4c 89 e0                mov    rax,r12
     1f9:       48 81 c0 3f 24 03 00    add    rax,0x3243f
     200:       49 89 c4                mov    r12,rax
     203:       b8 7f 48 06 00          mov    eax,0x6487f
     208:       49 89 c5                mov    r13,rax
     20b:       b8 00 00 00 00          mov    eax,0x0
     210:       49 89 c6                mov    r14,rax
     213:       4c 89 e7                mov    rdi,r12
     216:       4c 89 ee                mov    rsi,r13
     219:       e8 0f ff ff ff          call   12d <mod_floor>
     21e:       49 89 c6                mov    r14,rax
     221:       4c 89 f0                mov    rax,r14
     224:       49 89 c7                mov    r15,rax
     227:       4c 89 f8                mov    rax,r15
     22a:       48 81 e8 3f 24 03 00    sub    rax,0x3243f
     231:       49 89 c7                mov    r15,rax
     234:       4c 89 f8                mov    rax,r15
     237:       b9 20 92 01 00          mov    ecx,0x19220
     23c:       48 39 c8                cmp    rax,rcx
     23f:       0f 8e 26 00 00 00       jle    26b <fx_sin+0x9b>
     245:       b8 3f 24 03 00          mov    eax,0x3243f
     24a:       48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
     24e:       4c 89 f8                mov    rax,r15
     251:       48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
     255:       48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
259:       48 8b 4d c8             mov    rcx,QWORD PTR [rbp-0x38]
25d:       48 29 c8                sub    rax,rcx
260:       48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
264:       48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
268:       49 89 c7                mov    r15,rax
26b:       b8 00 00 00 00          mov    eax,0x0
270:       48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
274:       48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
278:       48 81 e8 20 92 01 00    sub    rax,0x19220
27f:       48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
283:       4c 89 f8                mov    rax,r15
286:       48 8b 4d c0             mov    rcx,QWORD PTR [rbp-0x40]
28a:       48 39 c8                cmp    rax,rcx
28d:       0f 8d 35 00 00 00       jge    2c8 <fx_sin+0xf8>
293:       b8 00 00 00 00          mov    eax,0x0
298:       48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
29c:       48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
2a0:       48 81 e8 3f 24 03 00    sub    rax,0x3243f
2a7:       48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
2ab:       4c 89 f8                mov    rax,r15
2ae:       48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
2b2:       48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
2b6:       48 8b 4d b0             mov    rcx,QWORD PTR [rbp-0x50]
2ba:       48 29 c8                sub    rax,rcx
2bd:       48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
2c1:       48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
2c5:       49 89 c7                mov    r15,rax
2c8:       4c 89 f8                mov    rax,r15
2cb:       48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
2cf:       48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
2d3:       4c 89 f9                mov    rcx,r15
2d6:       48 0f af c1             imul   rax,rcx
2da:       48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
2de:       48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
2e2:       48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
2e6:       b9 00 00 01 00          mov    ecx,0x10000
2eb:       48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
2ef:       48 99                   cqo
2f1:       48 f7 f9                idiv   rcx
2f4:       48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
2f8:       4c 89 f8                mov    rax,r15
2fb:       48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
2ff:       4c 89 f8                mov    rax,r15
302:       48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
306:       b8 00 00 00 00          mov    eax,0x0
30b:       48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
30f:       48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
313:       48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
317:       48 8b 45 80             mov    rax,QWORD PTR [rbp-0x80]
31b:       48 8b 4d a0             mov    rcx,QWORD PTR [rbp-0x60]
31f:       48 0f af c1             imul   rax,rcx
323:       48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
327:       48 8b 45 80             mov    rax,QWORD PTR [rbp-0x80]
32b:       48 89 85 78 ff ff ff    mov    QWORD PTR [rbp-0x88],rax
332:       b9 00 00 01 00          mov    ecx,0x10000
337:       48 8b 85 78 ff ff ff    mov    rax,QWORD PTR [rbp-0x88]
33e:       48 99                   cqo
340:       48 f7 f9                idiv   rcx
343:       48 89 85 78 ff ff ff    mov    QWORD PTR [rbp-0x88],rax
34a:       48 8b 85 78 ff ff ff    mov    rax,QWORD PTR [rbp-0x88]
351:       48 89 85 70 ff ff ff    mov    QWORD PTR [rbp-0x90],rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
     358:       b9 06 00 00 00          mov    ecx,0x6
     35d:       48 8b 85 70 ff ff ff    mov    rax,QWORD PTR [rbp-0x90]
     364:       48 99                   cqo
     366:       48 f7 f9                idiv   rcx
     369:       48 89 85 70 ff ff ff    mov    QWORD PTR [rbp-0x90],rax
     370:       48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
     374:       48 8b 8d 70 ff ff ff    mov    rcx,QWORD PTR [rbp-0x90]
     37b:       48 29 c8                sub    rax,rcx
     37e:       48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
     382:       48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
     386:       48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
     38a:       48 8b 45 90             mov    rax,QWORD PTR [rbp-0x70]
     38e:       48 8b 4d 98             mov    rcx,QWORD PTR [rbp-0x68]
     392:       48 01 c8                add    rax,rcx
     395:       48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
     399:       b8 00 00 00 00          mov    eax,0x0
     39e:       48 89 85 68 ff ff ff    mov    QWORD PTR [rbp-0x98],rax
     3a5:       48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
     3a9:       48 89 85 60 ff ff ff    mov    QWORD PTR [rbp-0xa0],rax
     3b0:       48 8b 85 60 ff ff ff    mov    rax,QWORD PTR [rbp-0xa0]
     3b7:       48 8b 4d a0             mov    rcx,QWORD PTR [rbp-0x60]
     3bb:       48 0f af c1             imul   rax,rcx
     3bf:       48 89 85 60 ff ff ff    mov    QWORD PTR [rbp-0xa0],rax
     3c6:       48 8b 85 60 ff ff ff    mov    rax,QWORD PTR [rbp-0xa0]
     3cd:       48 89 85 58 ff ff ff    mov    QWORD PTR [rbp-0xa8],rax
     3d4:       b9 00 00 01 00          mov    ecx,0x10000
     3d9:       48 8b 85 58 ff ff ff    mov    rax,QWORD PTR [rbp-0xa8]
     3e0:       48 99                   cqo
     3e2:       48 f7 f9                idiv   rcx
     3e5:       48 89 85 58 ff ff ff    mov    QWORD PTR [rbp-0xa8],rax
     3ec:       48 8b 85 58 ff ff ff    mov    rax,QWORD PTR [rbp-0xa8]
     3f3:       48 89 85 50 ff ff ff    mov    QWORD PTR [rbp-0xb0],rax
     3fa:       b9 14 00 00 00          mov    ecx,0x14
     3ff:       48 8b 85 50 ff ff ff    mov    rax,QWORD PTR [rbp-0xb0]
     406:       48 99                   cqo
     408:       48 f7 f9                idiv   rcx
     40b:       48 89 85 50 ff ff ff    mov    QWORD PTR [rbp-0xb0],rax
     412:       48 8b 85 68 ff ff ff    mov    rax,QWORD PTR [rbp-0x98]
     419:       48 8b 8d 50 ff ff ff    mov    rcx,QWORD PTR [rbp-0xb0]
     420:       48 29 c8                sub    rax,rcx
     423:       48 89 85 68 ff ff ff    mov    QWORD PTR [rbp-0x98],rax
     42a:       48 8b 85 68 ff ff ff    mov    rax,QWORD PTR [rbp-0x98]
     431:       48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
... (76 more lines not shown)
```

27.3.33 fx_cos

ASH

```
pub func fx_cos(int64 a): int64
```

Cosine of an angle in radians (Q16.16), computed as sin(*a* + pi/2).

Parameter	Description
a	Angle in radians, Q16.16.

Returns. cos(*a*) in Q16.16.

Implementation. The source of the function in `nlib/math.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH

```
pub func fx_cos(int64 a): int64 {
    return fx_sin(a + 102944);
}
```

Example.

Example 27-70. Using fx_cos

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    printf("%s\n", fx_print_buf(fx_cos(fx_deg(60)), &b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0.5000
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000005b3 <fx_cos>:
     5b3:       55                      push   rbp
     5b4:       48 89 e5                mov    rbp,rsp
     5b7:       48 81 ec 30 00 00 00    sub    rsp,0x30
     5be:       4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
     5c2:       4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
     5c6:       4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
     5ca:       4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
     5ce:       48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
     5d2:       48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
     5d6:       49 89 c4                mov    r12,rax
     5d9:       4c 89 e0                mov    rax,r12
     5dc:       48 81 c0 20 92 01 00    add    rax,0x19220
     5e3:       49 89 c4                mov    r12,rax
     5e6:       b8 00 00 00 00          mov    eax,0x0
     5eb:       49 89 c5                mov    r13,rax
     5ee:       4c 89 e7                mov    rdi,r12
     5f1:       e8 da fb ff ff          call   1d0 <fx_sin>
     5f6:       49 89 c5                mov    r13,rax
     5f9:       4c 89 e8                mov    rax,r13
     5fc:       4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
     600:       4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
     604:       4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
     608:       4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
     60c:       48 89 ec                mov    rsp,rbp
     60f:       5d                      pop    rbp
     610:       c3                      ret
```

27.3.34 fx_tan

ASH

```
pub func fx_tan(int64 a): int64
```

Tangent of an angle in radians (Q16.16), computed as sin/cos.

Copyright © 2026 Mallionic Inc. All rights reserved.

Parameter	Description
a	Angle in radians, Q16.16.

Returns. tan(*a*) in Q16.16, or the largest 64-bit value when the cosine is 0.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func fx_tan(int64 a): int64 {
    int64 c = fx_cos(a);
    if (c == 0) { return 9223372036854775807; }
    return fx_div(fx_sin(a), c);
}
```

Example.

Example 27-71. Using fx_tan

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    printf("%s\n", fx_print_buf(fx_tan(fx_deg(45)), &b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0.9999
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000066b <fx_tan>:
     66b:       55                      push   rbp
     66c:       48 89 e5                mov    rbp,rsp
     66f:       48 81 ec 40 00 00 00    sub    rsp,0x40
     676:       4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
     67a:       4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
     67e:       4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
     682:       4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
     686:       48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
     68a:       48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
     68e:       49 89 c4                mov    r12,rax
     691:       b8 00 00 00 00          mov    eax,0x0
     696:       49 89 c5                mov    r13,rax
     699:       4c 89 e7                mov    rdi,r12
     69c:       e8 12 ff ff ff          call   5b3 <fx_cos>
     6a1:       49 89 c5                mov    r13,rax
     6a4:       4c 89 e8                mov    rax,r13
     6a7:       b9 00 00 00 00          mov    ecx,0x0
     6ac:       48 39 c8                cmp    rax,rcx
     6af:       0f 85 1f 00 00 00       jne    6d4 <fx_tan+0x69>
     6b5:       48 b8 ff ff ff ff ff    movabs rax,0x7fffffffffffffff
     6bc:       ff ff 7f
     6bf:       4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
     6c3:       4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
     6c7:       4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
     6cb:       4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
     6cf:       48 89 ec                mov    rsp,rbp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
    6d2:       5d                      pop    rbp
    6d3:       c3                      ret
    6d4:       48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
    6d8:       49 89 c6                mov    r14,rax
    6db:       b8 00 00 00 00          mov    eax,0x0
    6e0:       49 89 c7                mov    r15,rax
    6e3:       4c 89 f7                mov    rdi,r14
    6e6:       e8 e5 fa ff ff          call   1d0 <fx_sin>
    6eb:       49 89 c7                mov    r15,rax
    6ee:       4c 89 e8                mov    rax,r13
    6f1:       48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
    6f5:       b8 00 00 00 00          mov    eax,0x0
    6fa:       48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
    6fe:       4c 89 ff                mov    rdi,r15
    701:       48 8b 75 d0             mov    rsi,QWORD PTR [rbp-0x30]
    705:       e8 07 ff ff ff          call   611 <fx_div>
    70a:       48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
    70e:       48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
    712:       4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
    716:       4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
    71a:       4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
    71e:       4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
    722:       48 89 ec                mov    rsp,rbp
    725:       5d                      pop    rbp
    726:       c3                      ret
```

27.3.35 fx_print_buf

ASH

```
pub func fx_print_buf(int64 a, uint8* buf): uint8*
```

Formats a Q16.16 value as decimal text with four decimals (`[-]integer.dddd`) into *buf*.

Parameter	Description
a	Q16.16 value.
buf	Destination buffer, at least 32 bytes.

Returns. *buf*.

Implementation. The source of the function in `nlib/math.ash`:

NLIB/MATH.ASH

```
pub func fx_print_buf(int64 a, uint8* buf): uint8* {
    int64 v = a;
    int64 n = 0;
    if (v < 0) {
        buf[0] = 45;
        n = 1;
        v = 0 - v;
    }
    int64 ent = v / 65536;
    int64 dec = (v % 65536) * 10000 / 65536;
    uint8[24] tmp;
    int64 k = 0;
    if (ent == 0) {
        tmp[0] = 48;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/MATH.ASH (CONTINUED)

```
        k = 1;
    }
    while (ent > 0) {
        tmp[k] = 48 + ent % 10;
        ent = ent / 10;
        k++;
    }
    while (k > 0) {
        k--;
        buf[n] = tmp[k];
        n++;
    }
    buf[n] = 46;
    buf[n + 1] = 48 + dec / 1000;
    buf[n + 2] = 48 + dec / 100 % 10;
    buf[n + 3] = 48 + dec / 10 % 10;
    buf[n + 4] = 48 + dec % 10;
    buf[n + 5] = 0;
    return buf;
}
```

Example.

Example 27-72. Using fx_print_buf

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    uint8[32] b;
    printf("%s\n", fx_print_buf(fx(1) / 8, &b));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0.1250
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000001ac <fx_print_buf>:
 1ac:   55                      push   rbp
 1ad:   48 89 e5                mov    rbp,rsp
 1b0:   48 81 ec c0 01 00 00    sub    rsp,0x1c0
 1b7:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 1bb:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 1bf:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 1c3:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 1c7:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 1cb:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 1cf:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 1d3:   49 89 c4                mov    r12,rax
 1d6:   b8 00 00 00 00          mov    eax,0x0
 1db:   49 89 c5                mov    r13,rax
 1de:   4c 89 e0                mov    rax,r12
 1e1:   b9 00 00 00 00          mov    ecx,0x0
 1e6:   48 39 c8                cmp    rax,rcx
 1e9:   0f 8d 51 00 00 00       jge    240 <fx_print_buf+0x94>
 1ef:   b8 00 00 00 00          mov    eax,0x0
 1f4:   49 89 c6                mov    r14,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
1f7:   4c 89 f0                mov    rax,r14
1fa:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
1fe:   48 01 c8                add    rax,rcx
201:   49 89 c6                mov    r14,rax
204:   b0 2d                   mov    al,0x2d
206:   41 88 c7                mov    r15b,al
209:   4c 89 f0                mov    rax,r14
20c:   49 0f b6 cf             movzx  rcx,r15b
210:   88 08                   mov    BYTE PTR [rax],cl
212:   b8 01 00 00 00          mov    eax,0x1
217:   49 89 c5                mov    r13,rax
21a:   b8 00 00 00 00          mov    eax,0x0
21f:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
223:   4c 89 e0                mov    rax,r12
226:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
22a:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
22e:   48 8b 4d c0             mov    rcx,QWORD PTR [rbp-0x40]
232:   48 29 c8                sub    rax,rcx
235:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
239:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
23d:   49 89 c4                mov    r12,rax
240:   4c 89 e0                mov    rax,r12
243:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
247:   b9 00 00 01 00          mov    ecx,0x10000
24c:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
250:   48 99                   cqo
252:   48 f7 f9                idiv   rcx
255:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
259:   4c 89 e0                mov    rax,r12
25c:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
260:   b9 00 00 01 00          mov    ecx,0x10000
265:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
269:   48 99                   cqo
26b:   48 f7 f9                idiv   rcx
26e:   48 89 55 b0             mov    QWORD PTR [rbp-0x50],rdx
272:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
276:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
27a:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
27e:   48 69 c0 10 27 00 00    imul   rax,rax,0x2710
285:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
289:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
28d:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
291:   b9 00 00 01 00          mov    ecx,0x10000
296:   48 8b 45 a0             mov    rax,QWORD PTR [rbp-0x60]
29a:   48 99                   cqo
29c:   48 f7 f9                idiv   rcx
29f:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
2a3:   b8 18 00 00 00          mov    eax,0x18
2a8:   48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
2ac:   b8 00 00 00 00          mov    eax,0x0
2b1:   48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
2b5:   48 8b 7d 98             mov    rdi,QWORD PTR [rbp-0x68]
2b9:   be 01 00 00 00          mov    esi,0x1
2be:   ba 00 00 00 00          mov    edx,0x0
2c3:   e8 38 fd ff ff          call   0 <__ash_alloc>
2c8:   48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
2cc:   b8 00 00 00 00          mov    eax,0x0
2d1:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
2d5:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
2d9:   b9 00 00 00 00          mov    ecx,0x0
2de:   48 39 c8                cmp    rax,rcx
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 2e1:   0f 85 57 00 00 00       jne    33e <fx_print_buf+0x192>
 2e7:   b8 00 00 00 00          mov    eax,0x0
 2ec:   48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
 2f0:   b8 00 00 00 00          mov    eax,0x0
 2f5:   48 89 85 78 ff ff ff    mov    QWORD PTR [rbp-0x88],rax
 2fc:   48 8b 7d 90             mov    rdi,QWORD PTR [rbp-0x70]
 300:   48 8b 75 80             mov    rsi,QWORD PTR [rbp-0x80]
 304:   e8 0e fe ff ff          call   117 <__ash_bounds>
 309:   48 89 85 78 ff ff ff    mov    QWORD PTR [rbp-0x88],rax
 310:   48 8b 45 80             mov    rax,QWORD PTR [rbp-0x80]
 314:   48 8b 4d 90             mov    rcx,QWORD PTR [rbp-0x70]
 318:   48 01 c8                add    rax,rcx
 31b:   48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
 31f:   b0 30                   mov    al,0x30
 321:   88 85 70 ff ff ff       mov    BYTE PTR [rbp-0x90],al
 327:   48 8b 45 80             mov    rax,QWORD PTR [rbp-0x80]
 32b:   48 0f b6 8d 70 ff ff    movzx  rcx,BYTE PTR [rbp-0x90]
 332:   ff
 333:   88 08                   mov    BYTE PTR [rax],cl
 335:   b8 01 00 00 00          mov    eax,0x1
 33a:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
 33e:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 342:   b9 00 00 00 00          mov    ecx,0x0
 347:   48 39 c8                cmp    rax,rcx
 34a:   0f 8e ca 00 00 00       jle    41a <fx_print_buf+0x26e>
 350:   48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
 354:   48 89 85 68 ff ff ff    mov    QWORD PTR [rbp-0x98],rax
 35b:   b8 00 00 00 00          mov    eax,0x0
 360:   48 89 85 60 ff ff ff    mov    QWORD PTR [rbp-0xa0],rax
 367:   48 8b 7d 90             mov    rdi,QWORD PTR [rbp-0x70]
 36b:   48 8b b5 68 ff ff ff    mov    rsi,QWORD PTR [rbp-0x98]
 372:   e8 a0 fd ff ff          call   117 <__ash_bounds>
 377:   48 89 85 60 ff ff ff    mov    QWORD PTR [rbp-0xa0],rax
 37e:   48 8b 85 68 ff ff ff    mov    rax,QWORD PTR [rbp-0x98]
 385:   48 8b 4d 90             mov    rcx,QWORD PTR [rbp-0x70]
 389:   48 01 c8                add    rax,rcx
 38c:   48 89 85 68 ff ff ff    mov    QWORD PTR [rbp-0x98],rax
 393:   b8 30 00 00 00          mov    eax,0x30
 398:   48 89 85 58 ff ff ff    mov    QWORD PTR [rbp-0xa8],rax
 39f:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 3a3:   48 89 85 50 ff ff ff    mov    QWORD PTR [rbp-0xb0],rax
 3aa:   b9 0a 00 00 00          mov    ecx,0xa
 3af:   48 8b 85 50 ff ff ff    mov    rax,QWORD PTR [rbp-0xb0]
 3b6:   48 99                   cqo
 3b8:   48 f7 f9                idiv   rcx
 3bb:   48 89 95 50 ff ff ff    mov    QWORD PTR [rbp-0xb0],rdx
 3c2:   48 8b 85 58 ff ff ff    mov    rax,QWORD PTR [rbp-0xa8]
 3c9:   48 8b 8d 50 ff ff ff    mov    rcx,QWORD PTR [rbp-0xb0]
 3d0:   48 01 c8                add    rax,rcx
 3d3:   48 89 85 58 ff ff ff    mov    QWORD PTR [rbp-0xa8],rax
 3da:   48 8b 85 58 ff ff ff    mov    rax,QWORD PTR [rbp-0xa8]
 3e1:   88 85 48 ff ff ff       mov    BYTE PTR [rbp-0xb8],al
 3e7:   48 8b 85 68 ff ff ff    mov    rax,QWORD PTR [rbp-0x98]
 3ee:   48 0f b6 8d 48 ff ff    movzx  rcx,BYTE PTR [rbp-0xb8]
 3f5:   ff
 3f6:   88 08                   mov    BYTE PTR [rax],cl
 3f8:   b9 0a 00 00 00          mov    ecx,0xa
 3fd:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 401:   48 99                   cqo
... (217 more lines not shown)
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.4 <SYSCALLS>: LINUX SYSTEM CALLS WITHOUT LIBC

`import <syscalls>;` gives direct access to the Linux x86-64 kernel through the `syscall` instruction. It needs no C library: programs that use only `<syscalls>`, `<string>` and `<math>` can be built with `--no-libc` and linked with `ld` alone (Section 22.2.2). Every function returns what the kernel returns: a non-negative value on success, or a negative error number (-errno) on failure.

Table 27-3. Constants defined by <syscalls>

Name	Value	Meaning
SYS_READ	0	Linux system call number
SYS_WRITE	1	Linux system call number
SYS_OPEN	2	Linux system call number
SYS_CLOSE	3	Linux system call number
SYS_LSEEK	8	Linux system call number
SYS_MMAP	9	Linux system call number
SYS_MUNMAP	11	Linux system call number
SYS_BRK	12	Linux system call number
SYS_NANOSLEEP	35	Linux system call number
SYS_GETPID	39	Linux system call number
SYS_EXIT	60	Linux system call number
SYS_KILL	62	Linux system call number
SYS_MKDIR	83	Linux system call number
SYS_UNLINK	87	Linux system call number
O_RDONLY	0	open for reading
O_WRONLY	1	open for writing
O_RDWR	2	open for reading and writing
O_CREAT	64	create the file if needed
O_TRUNC	512	truncate to length 0
STDIN	0	standard input descriptor
STDOUT	1	standard output descriptor
STDERR	2	standard error descriptor

27.4.1 syscall6

```ASH
pub func syscall6(int64 n, int64 a1, int64 a2, int64 a3, int64 a4, int64 a5, int64 a6): int6
↪ 4
```

Performs any Linux x86-64 system call with up to six arguments. The number goes in `rax` and the arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`, as the kernel expects; unused arguments should be 0. All the other functions of `<syscalls>` are built on it.

Copyright © 2026 Mallionic Inc. All rights reserved.

Parameter	Description
`n`	System call number (see the `SYS_*` constants).
`a1 ... a6`	Arguments.

Returns. The value returned by the kernel in `rax`: a non-negative result, or -errno on failure.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func syscall6(int64 n, int64 a1, int64 a2, int64 a3, int64 a4, int64 a5, int64 a6): int6
↪ 4 {
    int64 r = 0;
    asm volatile(
        "mov rax, {n}\n"
        "mov rdi, {a1}\n"
        "mov rsi, {a2}\n"
        "mov rdx, {a3}\n"
        "mov r10, {a4}\n"
        "mov r8, {a5}\n"
        "mov r9, {a6}\n"
        "syscall\n"
        "mov {r}, rax"
    );
    return r;
}
```

Example.

Example 27-73. Using syscall6

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    int64 pid = syscall6(SYS_GETPID, 0, 0, 0, 0, 0, 0);
    if (pid > 0) { println("got a pid"); }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
got a pid
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <syscall6>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 50 00 00 00    sub    rsp,0x50
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  1f:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
  23:   48 89 55 c8             mov    QWORD PTR [rbp-0x38],rdx
  27:   48 89 4d c0             mov    QWORD PTR [rbp-0x40],rcx
  2b:   4c 89 45 b8             mov    QWORD PTR [rbp-0x48],r8
  2f:   4c 89 4d b0             mov    QWORD PTR [rbp-0x50],r9
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  33:   b8 00 00 00 00          mov    eax,0x0
  38:   49 89 c4                mov    r12,rax
  3b:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  3f:   48 8b 7d d0             mov    rdi,QWORD PTR [rbp-0x30]
  43:   48 8b 75 c8             mov    rsi,QWORD PTR [rbp-0x38]
  47:   48 8b 55 c0             mov    rdx,QWORD PTR [rbp-0x40]
  4b:   4c 8b 55 b8             mov    r10,QWORD PTR [rbp-0x48]
  4f:   4c 8b 45 b0             mov    r8,QWORD PTR [rbp-0x50]
  53:   4c 8b 4d 10             mov    r9,QWORD PTR [rbp+0x10]
  57:   0f 05                   syscall
  59:   49 89 c4                mov    r12,rax
  5c:   4c 89 e0                mov    rax,r12
  5f:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  63:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  67:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  6b:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  6f:   48 89 ec                mov    rsp,rbp
  72:   5d                      pop    rbp
  73:   c3                      ret
```

27.4.2 sys_read

ASH

```
pub func sys_read(int64 fd, uint8* buf, int64 n): int64
```

Reads up to *n* bytes from file descriptor *fd* into *buf* (system call 0).

Parameter	Description
`fd`	File descriptor.
`buf`	Destination buffer.
`n`	Maximum number of bytes.

Returns. The number of bytes read (0 at end of file) or -errno.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_read(int64 fd, uint8* buf, int64 n): int64 {
    return syscall6(0, fd, buf, n, 0, 0, 0);
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-74. Using sys_read

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    uint8[64] b;
    int64 fd = sys_open("/proc/sys/kernel/ostype", O_RDONLY, 0);
    int64 n = sys_read(fd, &b, 63);
    sys_close(fd);
    print("read: ");
    sys_write(STDOUT, &b, n);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
read: Linux
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000001f0 <sys_read>:
 1f0:   55                      push   rbp
 1f1:   48 89 e5                mov    rbp,rsp
 1f4:   48 81 ec 60 00 00 00    sub    rsp,0x60
 1fb:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 1ff:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 203:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 207:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 20b:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 20f:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 213:   48 89 55 c8             mov    QWORD PTR [rbp-0x38],rdx
 217:   b8 00 00 00 00          mov    eax,0x0
 21c:   49 89 c4                mov    r12,rax
 21f:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 223:   49 89 c5                mov    r13,rax
 226:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 22a:   49 89 c6                mov    r14,rax
 22d:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 231:   49 89 c7                mov    r15,rax
 234:   b8 00 00 00 00          mov    eax,0x0
 239:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 23d:   b8 00 00 00 00          mov    eax,0x0
 242:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 246:   b8 00 00 00 00          mov    eax,0x0
 24b:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 24f:   b8 00 00 00 00          mov    eax,0x0
 254:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 258:   48 83 ec 08             sub    rsp,0x8
 25c:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
 260:   50                      push   rax
 261:   4c 89 e7                mov    rdi,r12
 264:   4c 89 ee                mov    rsi,r13
 267:   4c 89 f2                mov    rdx,r14
 26a:   4c 89 f9                mov    rcx,r15
 26d:   4c 8b 45 c0             mov    r8,QWORD PTR [rbp-0x40]
 271:   4c 8b 4d b8             mov    r9,QWORD PTR [rbp-0x48]
 275:   e8 57 fe ff ff          call   d1 <syscall6>
 27a:   48 83 c4 10             add    rsp,0x10
 27e:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 282:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
 286:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 28a:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 28e:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 292:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 296:   48 89 ec                mov    rsp,rbp
 299:   5d                      pop    rbp
 29a:   c3                      ret
```

27.4.3 sys_write

ASH

```
pub func sys_write(int64 fd, uint8* buf, int64 n): int64
```

Writes *n* bytes from *buf* to file descriptor *fd* (system call 1).

Parameter	Description
`fd`	File descriptor (`STDOUT` is 1, `STDERR` is 2).
`buf`	Data.
`n`	Number of bytes.

Returns. The number of bytes written or -errno.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_write(int64 fd, uint8* buf, int64 n): int64 {
    return syscall6(1, fd, buf, n, 0, 0, 0);
}
```

Example.

Example 27-75. Using sys_write

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    sys_write(STDOUT, "raw write\n", 10);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
raw write
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000074 <sys_write>:
  74:   55                       push   rbp
  75:   48 89 e5                 mov    rbp,rsp
  78:   48 81 ec 60 00 00 00     sub    rsp,0x60
  7f:   4c 89 65 f8              mov    QWORD PTR [rbp-0x8],r12
  83:   4c 89 6d f0              mov    QWORD PTR [rbp-0x10],r13
  87:   4c 89 75 e8              mov    QWORD PTR [rbp-0x18],r14
  8b:   4c 89 7d e0              mov    QWORD PTR [rbp-0x20],r15
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  8f:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  93:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
  97:   48 89 55 c8             mov    QWORD PTR [rbp-0x38],rdx
  9b:   b8 01 00 00 00          mov    eax,0x1
  a0:   49 89 c4                mov    r12,rax
  a3:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  a7:   49 89 c5                mov    r13,rax
  aa:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
  ae:   49 89 c6                mov    r14,rax
  b1:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
  b5:   49 89 c7                mov    r15,rax
  b8:   b8 00 00 00 00          mov    eax,0x0
  bd:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  c1:   b8 00 00 00 00          mov    eax,0x0
  c6:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  ca:   b8 00 00 00 00          mov    eax,0x0
  cf:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
  d3:   b8 00 00 00 00          mov    eax,0x0
  d8:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
  dc:   48 83 ec 08             sub    rsp,0x8
  e0:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
  e4:   50                      push   rax
  e5:   4c 89 e7                mov    rdi,r12
  e8:   4c 89 ee                mov    rsi,r13
  eb:   4c 89 f2                mov    rdx,r14
  ee:   4c 89 f9                mov    rcx,r15
  f1:   4c 8b 45 c0             mov    r8,QWORD PTR [rbp-0x40]
  f5:   4c 8b 4d b8             mov    r9,QWORD PTR [rbp-0x48]
  f9:   e8 02 ff ff ff          call   0 <syscall6>
  fe:   48 83 c4 10             add    rsp,0x10
 102:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 106:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
 10a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 10e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 112:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 116:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 11a:   48 89 ec                mov    rsp,rbp
 11d:   5d                      pop    rbp
 11e:   c3                      ret
```

27.4.4 sys_open

ASH

```
pub func sys_open(uint8* path, int64 flags, int64 mode): int64
```

Opens a file (system call 2).

Parameter	Description
`path`	NUL-terminated path.
`flags`	`O_RDONLY`, `O_WRONLY`, `O_RDWR`, combined with `O_CREAT`, `O_TRUNC`.
`mode`	Permissions for a created file, e.g. 0x1A4 (octal 644).

Returns. A file descriptor >= 0, or -errno.

Implementation. The source of the function in `nlib/syscalls.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/SYSCALLS.ASH

```
pub func sys_open(uint8* path, int64 flags, int64 mode): int64 {
    return syscall6(2, path, flags, mode, 0, 0, 0);
}
```

Example.

Example 27-76. Using sys_open

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    int64 fd = sys_open("/tmp/ash_guide_demo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0x1A4);
    sys_write(fd, "data\n", 5);
    sys_close(fd);
    if (fd >= 0) { println("file created"); }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
file created
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000074 <sys_open>:
  74:   55                      push   rbp
  75:   48 89 e5                mov    rbp,rsp
  78:   48 81 ec 60 00 00 00    sub    rsp,0x60
  7f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  83:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  87:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  8b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8f:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  93:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
  97:   48 89 55 c8             mov    QWORD PTR [rbp-0x38],rdx
  9b:   b8 02 00 00 00          mov    eax,0x2
  a0:   49 89 c4                mov    r12,rax
  a3:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  a7:   49 89 c5                mov    r13,rax
  aa:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
  ae:   49 89 c6                mov    r14,rax
  b1:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
  b5:   49 89 c7                mov    r15,rax
  b8:   b8 00 00 00 00          mov    eax,0x0
  bd:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  c1:   b8 00 00 00 00          mov    eax,0x0
  c6:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  ca:   b8 00 00 00 00          mov    eax,0x0
  cf:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
  d3:   b8 00 00 00 00          mov    eax,0x0
  d8:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
  dc:   48 83 ec 08             sub    rsp,0x8
  e0:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
  e4:   50                      push   rax
  e5:   4c 89 e7                mov    rdi,r12
  e8:   4c 89 ee                mov    rsi,r13
  eb:   4c 89 f2                mov    rdx,r14
  ee:   4c 89 f9                mov    rcx,r15
  f1:   4c 8b 45 c0             mov    r8,QWORD PTR [rbp-0x40]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  f5:   4c 8b 4d b8             mov    r9,QWORD PTR [rbp-0x48]
  f9:   e8 02 ff ff ff          call   0 <syscall6>
  fe:   48 83 c4 10             add    rsp,0x10
 102:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 106:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
 10a:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 10e:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 112:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 116:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 11a:   48 89 ec                mov    rsp,rbp
 11d:   5d                      pop    rbp
 11e:   c3                      ret
```

27.4.5 sys_close

ASH

```
pub func sys_close(int64 fd): int64
```

Closes a file descriptor (system call 3).

Parameter	Description
fd	Descriptor to close.

Returns. 0 or -errno.

Notes. Closing an invalid descriptor returns -9 (EBADF), as the example shows.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_close(int64 fd): int64 {
    return syscall6(3, fd, 0, 0, 0, 0, 0);
}
```

Example.

Example 27-77. Using sys_close

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    print_int(sys_close(9999));
    println("");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-9
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000074 <sys_close>:
  74:   55                      push   rbp
  75:   48 89 e5                mov    rbp,rsp
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  78:   48 81 ec 50 00 00 00    sub    rsp,0x50
  7f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  83:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  87:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  8b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8f:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  93:   b8 03 00 00 00          mov    eax,0x3
  98:   49 89 c4                mov    r12,rax
  9b:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  9f:   49 89 c5                mov    r13,rax
  a2:   b8 00 00 00 00          mov    eax,0x0
  a7:   49 89 c6                mov    r14,rax
  aa:   b8 00 00 00 00          mov    eax,0x0
  af:   49 89 c7                mov    r15,rax
  b2:   b8 00 00 00 00          mov    eax,0x0
  b7:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  bb:   b8 00 00 00 00          mov    eax,0x0
  c0:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  c4:   b8 00 00 00 00          mov    eax,0x0
  c9:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  cd:   b8 00 00 00 00          mov    eax,0x0
  d2:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  d6:   48 83 ec 08             sub    rsp,0x8
  da:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
  de:   50                      push   rax
  df:   4c 89 e7                mov    rdi,r12
  e2:   4c 89 ee                mov    rsi,r13
  e5:   4c 89 f2                mov    rdx,r14
  e8:   4c 89 f9                mov    rcx,r15
  eb:   4c 8b 45 d0             mov    r8,QWORD PTR [rbp-0x30]
  ef:   4c 8b 4d c8             mov    r9,QWORD PTR [rbp-0x38]
  f3:   e8 08 ff ff ff          call   0 <syscall6>
  f8:   48 83 c4 10             add    rsp,0x10
  fc:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 100:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 104:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 108:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 10c:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 110:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 114:   48 89 ec                mov    rsp,rbp
 117:   5d                      pop    rbp
 118:   c3                      ret
```

27.4.6 sys_lseek

ASH

```
pub func sys_lseek(int64 fd, int64 off, int64 whence): int64
```

Moves the file offset of *fd* (system call 8).

Parameter	Description
`fd`	Descriptor.
`off`	Offset.
`whence`	0 = from start, 1 = from current position, 2 = from end.

Returns. The new offset or -errno.

Copyright © 2026 Mallionic Inc. All rights reserved.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_lseek(int64 fd, int64 off, int64 whence): int64 {
    return syscall6(8, fd, off, whence, 0, 0, 0);
}
```

Example.

Example 27-78. Using sys_lseek

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    int64 fd = sys_open("/proc/self/exe", O_RDONLY, 0);
    int64 size = sys_lseek(fd, 0, 2);
    sys_close(fd);
    if (size > 0) { println("the executable has a size"); }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
the executable has a size
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000011f <sys_lseek>:
 11f:   55                      push   rbp
 120:   48 89 e5                mov    rbp,rsp
 123:   48 81 ec 60 00 00 00    sub    rsp,0x60
 12a:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 12e:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 132:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 136:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 13a:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 13e:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 142:   48 89 55 c8             mov    QWORD PTR [rbp-0x38],rdx
 146:   b8 08 00 00 00          mov    eax,0x8
 14b:   49 89 c4                mov    r12,rax
 14e:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 152:   49 89 c5                mov    r13,rax
 155:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 159:   49 89 c6                mov    r14,rax
 15c:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 160:   49 89 c7                mov    r15,rax
 163:   b8 00 00 00 00          mov    eax,0x0
 168:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 16c:   b8 00 00 00 00          mov    eax,0x0
 171:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 175:   b8 00 00 00 00          mov    eax,0x0
 17a:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 17e:   b8 00 00 00 00          mov    eax,0x0
 183:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 187:   48 83 ec 08             sub    rsp,0x8
 18b:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
 18f:   50                      push   rax
 190:   4c 89 e7                mov    rdi,r12
 193:   4c 89 ee                mov    rsi,r13
 196:   4c 89 f2                mov    rdx,r14
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 199:   4c 89 f9                mov    rcx,r15
 19c:   4c 8b 45 c0             mov    r8,QWORD PTR [rbp-0x40]
 1a0:   4c 8b 4d b8             mov    r9,QWORD PTR [rbp-0x48]
 1a4:   e8 57 fe ff ff          call   0 <syscall6>
 1a9:   48 83 c4 10             add    rsp,0x10
 1ad:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
 1b1:   48 8b 45 a8             mov    rax,QWORD PTR [rbp-0x58]
 1b5:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 1b9:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 1bd:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 1c1:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 1c5:   48 89 ec                mov    rsp,rbp
 1c8:   5d                      pop    rbp
 1c9:   c3                      ret
```

27.4.7 sys_mmap

ASH

```
pub func sys_mmap(int64 addr, int64 n): int64
```

Maps *n* bytes of anonymous, zero-filled, readable and writable memory (system call 9 with PROT_READ|PROT_WRITE and MAP_PRIVATE|MAP_ANONYMOUS).

Parameter	Description
`addr`	Requested address, normally 0 (let the kernel choose).
`n`	Size in bytes.

Returns. The address of the memory, or -errno.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_mmap(int64 addr, int64 n): int64 {
    return syscall6(9, addr, n, 3, 34, -1, 0);
}
```

Example.

Example 27-79. Using sys_mmap

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    uint8* p = sys_mmap(0, 4096);
    p[0] = 72; p[1] = 105; p[2] = 0;
    println(p);
    sys_munmap(p, 4096);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Hi
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000074 <sys_mmap>:
  74:   55                      push   rbp
  75:   48 89 e5                mov    rbp,rsp
  78:   48 81 ec 50 00 00 00    sub    rsp,0x50
  7f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  83:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  87:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  8b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8f:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  93:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
  97:   b8 09 00 00 00          mov    eax,0x9
  9c:   49 89 c4                mov    r12,rax
  9f:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  a3:   49 89 c5                mov    r13,rax
  a6:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
  aa:   49 89 c6                mov    r14,rax
  ad:   b8 03 00 00 00          mov    eax,0x3
  b2:   49 89 c7                mov    r15,rax
  b5:   b8 22 00 00 00          mov    eax,0x22
  ba:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  be:   b8 00 00 00 00          mov    eax,0x0
  c3:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  c7:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
  cb:   48 83 e8 01             sub    rax,0x1
  cf:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  d3:   b8 00 00 00 00          mov    eax,0x0
  d8:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  dc:   b8 00 00 00 00          mov    eax,0x0
  e1:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
  e5:   48 83 ec 08             sub    rsp,0x8
  e9:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
  ed:   50                      push   rax
  ee:   4c 89 e7                mov    rdi,r12
  f1:   4c 89 ee                mov    rsi,r13
  f4:   4c 89 f2                mov    rdx,r14
  f7:   4c 89 f9                mov    rcx,r15
  fa:   4c 8b 45 c8             mov    r8,QWORD PTR [rbp-0x38]
  fe:   4c 8b 4d c0             mov    r9,QWORD PTR [rbp-0x40]
 102:   e8 f9 fe ff ff          call   0 <syscall6>
 107:   48 83 c4 10             add    rsp,0x10
 10b:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 10f:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
 113:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 117:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 11b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 11f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 123:   48 89 ec                mov    rsp,rbp
 126:   5d                      pop    rbp
 127:   c3                      ret
```

27.4.8 sys_munmap

ASH

```
pub func sys_munmap(int64 addr, int64 n): int64
```

Unmaps memory obtained with `sys_mmap` (system call 11).

Copyright © 2026 Mallionic Inc. All rights reserved.

Parameter	Description
addr	Address.
n	Size.

Returns. 0 or -errno.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_munmap(int64 addr, int64 n): int64 {
    return syscall6(11, addr, n, 0, 0, 0, 0);
}
```

Example.

Example 27-80. Using sys_munmap

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    uint8* p = sys_mmap(0, 8192);
    print_int(sys_munmap(p, 8192));
    println("");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000128 <sys_munmap>:
 128:   55                      push   rbp
 129:   48 89 e5                mov    rbp,rsp
 12c:   48 81 ec 50 00 00 00    sub    rsp,0x50
 133:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 137:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 13b:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 13f:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 143:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 147:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
 14b:   b8 0b 00 00 00          mov    eax,0xb
 150:   49 89 c4                mov    r12,rax
 153:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 157:   49 89 c5                mov    r13,rax
 15a:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 15e:   49 89 c6                mov    r14,rax
 161:   b8 00 00 00 00          mov    eax,0x0
 166:   49 89 c7                mov    r15,rax
 169:   b8 00 00 00 00          mov    eax,0x0
 16e:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 172:   b8 00 00 00 00          mov    eax,0x0
 177:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 17b:   b8 00 00 00 00          mov    eax,0x0
 180:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 184:   b8 00 00 00 00          mov    eax,0x0
 189:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 18d:   48 83 ec 08             sub    rsp,0x8
 191:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 195:   50                      push   rax
 196:   4c 89 e7                mov    rdi,r12
 199:   4c 89 ee                mov    rsi,r13
 19c:   4c 89 f2                mov    rdx,r14
 19f:   4c 89 f9                mov    rcx,r15
 1a2:   4c 8b 45 c8             mov    r8,QWORD PTR [rbp-0x38]
 1a6:   4c 8b 4d c0             mov    r9,QWORD PTR [rbp-0x40]
 1aa:   e8 51 fe ff ff          call   0 <syscall6>
 1af:   48 83 c4 10             add    rsp,0x10
 1b3:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 1b7:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
 1bb:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 1bf:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 1c3:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 1c7:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 1cb:   48 89 ec                mov    rsp,rbp
 1ce:   5d                      pop    rbp
 1cf:   c3                      ret
```

27.4.9 sys_brk

ASH

```
pub func sys_brk(int64 addr): int64
```

Sets the end of the data segment (system call 12). `sys_brk(0)` returns the current end.

Parameter	Description
`addr`	New end, or 0 to query.

Returns. The current program break.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_brk(int64 addr): int64 {
    return syscall6(12, addr, 0, 0, 0, 0, 0);
}
```

Example.

Example 27-81. Using sys_brk

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    int64 b = sys_brk(0);
    if (b != 0) { println("program break found"); }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
program break found
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000074 <sys_brk>:
  74:   55                      push   rbp
  75:   48 89 e5                mov    rbp,rsp
  78:   48 81 ec 50 00 00 00    sub    rsp,0x50
  7f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  83:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  87:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  8b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8f:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  93:   b8 0c 00 00 00          mov    eax,0xc
  98:   49 89 c4                mov    r12,rax
  9b:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  9f:   49 89 c5                mov    r13,rax
  a2:   b8 00 00 00 00          mov    eax,0x0
  a7:   49 89 c6                mov    r14,rax
  aa:   b8 00 00 00 00          mov    eax,0x0
  af:   49 89 c7                mov    r15,rax
  b2:   b8 00 00 00 00          mov    eax,0x0
  b7:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  bb:   b8 00 00 00 00          mov    eax,0x0
  c0:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  c4:   b8 00 00 00 00          mov    eax,0x0
  c9:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  cd:   b8 00 00 00 00          mov    eax,0x0
  d2:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  d6:   48 83 ec 08             sub    rsp,0x8
  da:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
  de:   50                      push   rax
  df:   4c 89 e7                mov    rdi,r12
  e2:   4c 89 ee                mov    rsi,r13
  e5:   4c 89 f2                mov    rdx,r14
  e8:   4c 89 f9                mov    rcx,r15
  eb:   4c 8b 45 d0             mov    r8,QWORD PTR [rbp-0x30]
  ef:   4c 8b 4d c8             mov    r9,QWORD PTR [rbp-0x38]
  f3:   e8 08 ff ff ff          call   0 <syscall6>
  f8:   48 83 c4 10             add    rsp,0x10
  fc:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 100:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 104:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 108:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 10c:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 110:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 114:   48 89 ec                mov    rsp,rbp
 117:   5d                      pop    rbp
 118:   c3                      ret
```

27.4.10 sys_getpid

ASH

```
pub func sys_getpid(): int64
```

Returns the process identifier (system call 39).

Returns. The PID.

Implementation. The source of the function in `nlib/syscalls.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/SYSCALLS.ASH

```
pub func sys_getpid(): int64 {
    return syscall6(39, 0, 0, 0, 0, 0, 0);
}
```

Example.

Example 27-82. Using sys_getpid

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    int64 pid = sys_getpid();
    if (pid > 0) { println("we have a pid"); }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
we have a pid
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000074 <sys_getpid>:
  74:   55                      push   rbp
  75:   48 89 e5                mov    rbp,rsp
  78:   48 81 ec 40 00 00 00    sub    rsp,0x40
  7f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  83:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  87:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  8b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8f:   b8 27 00 00 00          mov    eax,0x27
  94:   49 89 c4                mov    r12,rax
  97:   b8 00 00 00 00          mov    eax,0x0
  9c:   49 89 c5                mov    r13,rax
  9f:   b8 00 00 00 00          mov    eax,0x0
  a4:   49 89 c6                mov    r14,rax
  a7:   b8 00 00 00 00          mov    eax,0x0
  ac:   49 89 c7                mov    r15,rax
  af:   b8 00 00 00 00          mov    eax,0x0
  b4:   48 89 45 d8             mov    QWORD PTR [rbp-0x28],rax
  b8:   b8 00 00 00 00          mov    eax,0x0
  bd:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  c1:   b8 00 00 00 00          mov    eax,0x0
  c6:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  ca:   b8 00 00 00 00          mov    eax,0x0
  cf:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  d3:   48 83 ec 08             sub    rsp,0x8
  d7:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
  db:   50                      push   rax
  dc:   4c 89 e7                mov    rdi,r12
  df:   4c 89 ee                mov    rsi,r13
  e2:   4c 89 f2                mov    rdx,r14
  e5:   4c 89 f9                mov    rcx,r15
  e8:   4c 8b 45 d8             mov    r8,QWORD PTR [rbp-0x28]
  ec:   4c 8b 4d d0             mov    r9,QWORD PTR [rbp-0x30]
  f0:   e8 0b ff ff ff          call   0 <syscall6>
  f5:   48 83 c4 10             add    rsp,0x10
  f9:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  fd:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 101:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 105:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 109:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 10d:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 111:   48 89 ec                mov    rsp,rbp
 114:   5d                      pop    rbp
 115:   c3                      ret
```

27.4.11 sys_kill

ASH

```
pub func sys_kill(int64 pid, int64 sig): int64
```

Sends signal *sig* to process *pid* (system call 62). Signal 0 only checks that the process exists.

Parameter	Description
`pid`	Process identifier.
`sig`	Signal number.

Returns. 0 or -errno.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_kill(int64 pid, int64 sig): int64 {
    return syscall6(62, pid, sig, 0, 0, 0, 0);
}
```

Example.

Example 27-83. Using sys_kill

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    print_int(sys_kill(sys_getpid(), 0));
    println("");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000116 <sys_kill>:
 116:   55                      push   rbp
 117:   48 89 e5                mov    rbp,rsp
 11a:   48 81 ec 50 00 00 00    sub    rsp,0x50
 121:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 125:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 129:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 12d:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
131:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
135:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
139:   b8 3e 00 00 00          mov    eax,0x3e
13e:   49 89 c4                mov    r12,rax
141:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
145:   49 89 c5                mov    r13,rax
148:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
14c:   49 89 c6                mov    r14,rax
14f:   b8 00 00 00 00          mov    eax,0x0
154:   49 89 c7                mov    r15,rax
157:   b8 00 00 00 00          mov    eax,0x0
15c:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
160:   b8 00 00 00 00          mov    eax,0x0
165:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
169:   b8 00 00 00 00          mov    eax,0x0
16e:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
172:   b8 00 00 00 00          mov    eax,0x0
177:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
17b:   48 83 ec 08             sub    rsp,0x8
17f:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
183:   50                      push   rax
184:   4c 89 e7                mov    rdi,r12
187:   4c 89 ee                mov    rsi,r13
18a:   4c 89 f2                mov    rdx,r14
18d:   4c 89 f9                mov    rcx,r15
190:   4c 8b 45 c8             mov    r8,QWORD PTR [rbp-0x38]
194:   4c 8b 4d c0             mov    r9,QWORD PTR [rbp-0x40]
198:   e8 63 fe ff ff          call   0 <syscall6>
19d:   48 83 c4 10             add    rsp,0x10
1a1:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
1a5:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
1a9:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
1ad:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
1b1:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
1b5:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
1b9:   48 89 ec                mov    rsp,rbp
1bc:   5d                      pop    rbp
1bd:   c3                      ret
```

27.4.12 sys_mkdir

ASH

```
pub func sys_mkdir(uint8* path, int64 mode): int64
```

Creates a directory (system call 83).

Parameter	Description
`path`	Path.
`mode`	Permissions, e.g. 0x1ED (octal 755).

Returns. 0 or -errno (-17 if it already exists).

Implementation. The source of the function in `nlib/syscalls.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/SYSCALLS.ASH

```
pub func sys_mkdir(uint8* path, int64 mode): int64 {
    return syscall6(83, path, mode, 0, 0, 0, 0);
}
```

Example.

Example 27-84. Using sys_mkdir

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    sys_mkdir("/tmp/ash_guide_dir", 0x1ED);
    print_int(sys_mkdir("/tmp/ash_guide_dir", 0x1ED));
    println("");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-17
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000074 <sys_mkdir>:
  74:   55                      push   rbp
  75:   48 89 e5                mov    rbp,rsp
  78:   48 81 ec 50 00 00 00    sub    rsp,0x50
  7f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  83:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  87:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  8b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8f:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  93:   48 89 75 d0             mov    QWORD PTR [rbp-0x30],rsi
  97:   b8 53 00 00 00          mov    eax,0x53
  9c:   49 89 c4                mov    r12,rax
  9f:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  a3:   49 89 c5                mov    r13,rax
  a6:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
  aa:   49 89 c6                mov    r14,rax
  ad:   b8 00 00 00 00          mov    eax,0x0
  b2:   49 89 c7                mov    r15,rax
  b5:   b8 00 00 00 00          mov    eax,0x0
  ba:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  be:   b8 00 00 00 00          mov    eax,0x0
  c3:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  c7:   b8 00 00 00 00          mov    eax,0x0
  cc:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  d0:   b8 00 00 00 00          mov    eax,0x0
  d5:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
  d9:   48 83 ec 08             sub    rsp,0x8
  dd:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
  e1:   50                      push   rax
  e2:   4c 89 e7                mov    rdi,r12
  e5:   4c 89 ee                mov    rsi,r13
  e8:   4c 89 f2                mov    rdx,r14
  eb:   4c 89 f9                mov    rcx,r15
  ee:   4c 8b 45 c8             mov    r8,QWORD PTR [rbp-0x38]
  f2:   4c 8b 4d c0             mov    r9,QWORD PTR [rbp-0x40]
  f6:   e8 05 ff ff ff          call   0 <syscall6>
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  fb:   48 83 c4 10             add    rsp,0x10
  ff:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
 103:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
 107:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 10b:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 10f:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 113:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 117:   48 89 ec                mov    rsp,rbp
 11a:   5d                      pop    rbp
 11b:   c3                      ret
```

27.4.13 sys_unlink

ASH

```
pub func sys_unlink(uint8* path): int64
```

Deletes a file (system call 87).

Parameter	Description
path	Path.

Returns. 0 or -errno (-2 if it does not exist).

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_unlink(uint8* path): int64 {
    return syscall6(87, path, 0, 0, 0, 0, 0);
}
```

Example.

Example 27-85. Using sys_unlink

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    print_int(sys_unlink("/tmp/ash_guide_missing_file"));
    println("");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-2
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000074 <sys_unlink>:
  74:   55                      push   rbp
  75:   48 89 e5                mov    rbp,rsp
  78:   48 81 ec 50 00 00 00    sub    rsp,0x50
  7f:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
  83:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  87:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  8b:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  8f:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  93:   b8 57 00 00 00          mov    eax,0x57
  98:   49 89 c4                mov    r12,rax
  9b:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  9f:   49 89 c5                mov    r13,rax
  a2:   b8 00 00 00 00          mov    eax,0x0
  a7:   49 89 c6                mov    r14,rax
  aa:   b8 00 00 00 00          mov    eax,0x0
  af:   49 89 c7                mov    r15,rax
  b2:   b8 00 00 00 00          mov    eax,0x0
  b7:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
  bb:   b8 00 00 00 00          mov    eax,0x0
  c0:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
  c4:   b8 00 00 00 00          mov    eax,0x0
  c9:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
  cd:   b8 00 00 00 00          mov    eax,0x0
  d2:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
  d6:   48 83 ec 08             sub    rsp,0x8
  da:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
  de:   50                      push   rax
  df:   4c 89 e7                mov    rdi,r12
  e2:   4c 89 ee                mov    rsi,r13
  e5:   4c 89 f2                mov    rdx,r14
  e8:   4c 89 f9                mov    rcx,r15
  eb:   4c 8b 45 d0             mov    r8,QWORD PTR [rbp-0x30]
  ef:   4c 8b 4d c8             mov    r9,QWORD PTR [rbp-0x38]
  f3:   e8 08 ff ff ff          call   0 <syscall6>
  f8:   48 83 c4 10             add    rsp,0x10
  fc:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 100:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
 104:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 108:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 10c:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 110:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 114:   48 89 ec                mov    rsp,rbp
 117:   5d                      pop    rbp
 118:   c3                      ret
```

27.4.14 sys_nanosleep

ASH

```
pub func sys_nanosleep(int64 ms): int64
```

Sleeps for *ms* milliseconds (system call 35). A `timespec` is built on the stack from *ms*.

Parameter	Description
`ms`	Milliseconds.

Returns. 0, or -errno if interrupted.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_nanosleep(int64 ms): int64 {
    int64[2] ts;
    ts[0] = ms / 1000;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/SYSCALLS.ASH (CONTINUED)

```
    ts[1] = ms % 1000 * 1000000;
    return syscall6(35, ts, 0, 0, 0, 0, 0);
}
```

Example.

Example 27-86. Using sys_nanosleep

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    sys_nanosleep(20);
    println("slept 20 ms");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
slept 20 ms
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000001da <sys_nanosleep>:
 1da:   55                      push   rbp
 1db:   48 89 e5                mov    rbp,rsp
 1de:   48 81 ec 90 00 00 00    sub    rsp,0x90
 1e5:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 1e9:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 1ed:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 1f1:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 1f5:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 1f9:   b8 02 00 00 00          mov    eax,0x2
 1fe:   49 89 c4                mov    r12,rax
 201:   b8 00 00 00 00          mov    eax,0x0
 206:   49 89 c5                mov    r13,rax
 209:   4c 89 e7                mov    rdi,r12
 20c:   be 08 00 00 00          mov    esi,0x8
 211:   ba 00 00 00 00          mov    edx,0x0
 216:   e8 e5 fd ff ff          call   0 <__ash_alloc>
 21b:   49 89 c5                mov    r13,rax
 21e:   b8 00 00 00 00          mov    eax,0x0
 223:   49 89 c6                mov    r14,rax
 226:   b8 00 00 00 00          mov    eax,0x0
 22b:   49 89 c7                mov    r15,rax
 22e:   4c 89 ef                mov    rdi,r13
 231:   4c 89 f6                mov    rsi,r14
 234:   e8 98 fe ff ff          call   d1 <__ash_bounds>
 239:   49 89 c7                mov    r15,rax
 23c:   4c 89 f0                mov    rax,r14
 23f:   48 6b c0 08             imul   rax,rax,0x8
 243:   49 89 c6                mov    r14,rax
 246:   4c 89 f0                mov    rax,r14
 249:   4c 89 e9                mov    rcx,r13
 24c:   48 01 c8                add    rax,rcx
 24f:   49 89 c6                mov    r14,rax
 252:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 256:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 25a:   b9 e8 03 00 00          mov    ecx,0x3e8
 25f:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
263:   48 99                   cqo
265:   48 f7 f9                idiv   rcx
268:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
26c:   4c 89 f0                mov    rax,r14
26f:   48 8b 4d d0             mov    rcx,QWORD PTR [rbp-0x30]
273:   48 89 08                mov    QWORD PTR [rax],rcx
276:   b8 01 00 00 00          mov    eax,0x1
27b:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
27f:   b8 00 00 00 00          mov    eax,0x0
284:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
288:   4c 89 ef                mov    rdi,r13
28b:   48 8b 75 c8             mov    rsi,QWORD PTR [rbp-0x38]
28f:   e8 3d fe ff ff          call   d1 <__ash_bounds>
294:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
298:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
29c:   48 6b c0 08             imul   rax,rax,0x8
2a0:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
2a4:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
2a8:   4c 89 e9                mov    rcx,r13
2ab:   48 01 c8                add    rax,rcx
2ae:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
2b2:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
2b6:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
2ba:   b9 e8 03 00 00          mov    ecx,0x3e8
2bf:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
2c3:   48 99                   cqo
2c5:   48 f7 f9                idiv   rcx
2c8:   48 89 55 b8             mov    QWORD PTR [rbp-0x48],rdx
2cc:   48 8b 45 b8             mov    rax,QWORD PTR [rbp-0x48]
2d0:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
2d4:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
2d8:   48 69 c0 40 42 0f 00    imul   rax,rax,0xf4240
2df:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
2e3:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
2e7:   48 8b 4d b0             mov    rcx,QWORD PTR [rbp-0x50]
2eb:   48 89 08                mov    QWORD PTR [rax],rcx
2ee:   b8 23 00 00 00          mov    eax,0x23
2f3:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
2f7:   4c 89 e8                mov    rax,r13
2fa:   48 89 45 a0             mov    QWORD PTR [rbp-0x60],rax
2fe:   b8 00 00 00 00          mov    eax,0x0
303:   48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
307:   b8 00 00 00 00          mov    eax,0x0
30c:   48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
310:   b8 00 00 00 00          mov    eax,0x0
315:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
319:   b8 00 00 00 00          mov    eax,0x0
31e:   48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
322:   b8 00 00 00 00          mov    eax,0x0
327:   48 89 85 78 ff ff ff    mov    QWORD PTR [rbp-0x88],rax
32e:   b8 00 00 00 00          mov    eax,0x0
333:   48 89 85 70 ff ff ff    mov    QWORD PTR [rbp-0x90],rax
33a:   48 83 ec 08             sub    rsp,0x8
33e:   48 8b 85 78 ff ff ff    mov    rax,QWORD PTR [rbp-0x88]
345:   50                      push   rax
346:   48 8b 7d a8             mov    rdi,QWORD PTR [rbp-0x58]
34a:   48 8b 75 a0             mov    rsi,QWORD PTR [rbp-0x60]
34e:   48 8b 55 98             mov    rdx,QWORD PTR [rbp-0x68]
352:   48 8b 4d 90             mov    rcx,QWORD PTR [rbp-0x70]
356:   4c 8b 45 88             mov    r8,QWORD PTR [rbp-0x78]
35a:   4c 8b 4d 80             mov    r9,QWORD PTR [rbp-0x80]
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 35e:   e8 03 fe ff ff          call   166 <syscall6>
 363:   48 83 c4 10             add    rsp,0x10
 367:   48 89 85 70 ff ff ff    mov    QWORD PTR [rbp-0x90],rax
 36e:   48 8b 85 70 ff ff ff    mov    rax,QWORD PTR [rbp-0x90]
 375:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 379:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 37d:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 381:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 385:   48 89 ec                mov    rsp,rbp
 388:   5d                      pop    rbp
 389:   c3                      ret
```

27.4.15 sys_exit

ASH

```
pub func sys_exit(int64 code): void
```

Ends the process immediately with status *code* (system call 60). Buffered C output is not flushed.

Parameter	Description
`code`	Exit status (0 to 255).

Returns. Does not return.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func sys_exit(int64 code): void {
    syscall6(60, code, 0, 0, 0, 0, 0);
}
```

Example.

Example 27-87. Using sys_exit

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    println("bye");
    sys_exit(5);
    return 0;
}
```

OUTPUT — EXIT STATUS 5

```
bye
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000002a9 <sys_exit>:
 2a9:   55                      push   rbp
 2aa:   48 89 e5                mov    rbp,rsp
 2ad:   48 81 ec 50 00 00 00    sub    rsp,0x50
 2b4:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 2b8:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 2bc:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 2c0:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 2c4:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 2c8:   b8 3c 00 00 00          mov    eax,0x3c
 2cd:   49 89 c4                mov    r12,rax
 2d0:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 2d4:   49 89 c5                mov    r13,rax
 2d7:   b8 00 00 00 00          mov    eax,0x0
 2dc:   49 89 c6                mov    r14,rax
 2df:   b8 00 00 00 00          mov    eax,0x0
 2e4:   49 89 c7                mov    r15,rax
 2e7:   b8 00 00 00 00          mov    eax,0x0
 2ec:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 2f0:   b8 00 00 00 00          mov    eax,0x0
 2f5:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 2f9:   b8 00 00 00 00          mov    eax,0x0
 2fe:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 302:   b8 00 00 00 00          mov    eax,0x0
 307:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 30b:   48 83 ec 08             sub    rsp,0x8
 30f:   48 8b 45 c0             mov    rax,QWORD PTR [rbp-0x40]
 313:   50                      push   rax
 314:   4c 89 e7                mov    rdi,r12
 317:   4c 89 ee                mov    rsi,r13
 31a:   4c 89 f2                mov    rdx,r14
 31d:   4c 89 f9                mov    rcx,r15
 320:   4c 8b 45 d0             mov    r8,QWORD PTR [rbp-0x30]
 324:   4c 8b 4d c8             mov    r9,QWORD PTR [rbp-0x38]
 328:   e8 54 fd ff ff          call   81 <syscall6>
 32d:   48 83 c4 10             add    rsp,0x10
 331:   48 89 45 b8             mov    QWORD PTR [rbp-0x48],rax
 335:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 339:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 33d:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 341:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 345:   48 89 ec                mov    rsp,rbp
 348:   5d                      pop    rbp
 349:   c3                      ret
```

27.4.16 print

ASH

```
pub func print(uint8* s): int64
```

Writes a NUL-terminated string to standard output with `sys_write`.

Parameter	Description
`s`	String.

Returns. The result of `sys_write`.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func print(uint8* s): int64 {
    return sys_write(1, s, __sys_len(s));
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Example.

Example 27-88. Using print

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    print("no newline, ");
    print("then a newline\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
no newline, then a newline
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
00000000000001a0 <print>:
 1a0:   55                      push   rbp
 1a1:   48 89 e5                mov    rbp,rsp
 1a4:   48 81 ec 30 00 00 00    sub    rsp,0x30
 1ab:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 1af:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 1b3:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 1b7:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 1bb:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 1bf:   b8 01 00 00 00          mov    eax,0x1
 1c4:   49 89 c4                mov    r12,rax
 1c7:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 1cb:   49 89 c5                mov    r13,rax
 1ce:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 1d2:   49 89 c6                mov    r14,rax
 1d5:   b8 00 00 00 00          mov    eax,0x0
 1da:   49 89 c7                mov    r15,rax
 1dd:   4c 89 f7                mov    rdi,r14
 1e0:   e8 1b fe ff ff          call   0 <__sys_len>
 1e5:   49 89 c7                mov    r15,rax
 1e8:   b8 00 00 00 00          mov    eax,0x0
 1ed:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 1f1:   4c 89 e7                mov    rdi,r12
 1f4:   4c 89 ee                mov    rsi,r13
 1f7:   4c 89 fa                mov    rdx,r15
 1fa:   e8 f6 fe ff ff          call   f5 <sys_write>
 1ff:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 203:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 207:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
 20b:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
 20f:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
 213:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
 217:   48 89 ec                mov    rsp,rbp
 21a:   5d                      pop    rbp
 21b:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.4.17 println

```ash
pub func println(uint8* s): int64
```

Writes a string followed by a line feed to standard output.

Parameter	Description
s	String.

Returns. The result of writing the string.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```ash
pub func println(uint8* s): int64 {
    int64 r = print(s);
    sys_write(1, "\n", 1);
    return r;
}
```

Example.

Example 27-89. Using println

```ash
import <syscalls>;

@Entry
pub func main(): int64 {
    println("one line");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
one line
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
000000000000021c <println>:
 21c:   55                      push   rbp
 21d:   48 89 e5                mov    rbp,rsp
 220:   48 81 ec 40 00 00 00    sub    rsp,0x40
 227:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 22b:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 22f:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 233:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 237:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 23b:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 23f:   49 89 c4                mov    r12,rax
 242:   b8 00 00 00 00          mov    eax,0x0
 247:   49 89 c5                mov    r13,rax
 24a:   4c 89 e7                mov    rdi,r12
 24d:   e8 4e ff ff ff          call   1a0 <print>
 252:   49 89 c5                mov    r13,rax
 255:   b8 01 00 00 00          mov    eax,0x1
 25a:   49 89 c6                mov    r14,rax
 25d:   b8 00 00 00 00          mov    eax,0x0
 262:   49 89 c7                mov    r15,rax
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
265:   4c 8d 3d 4c 07 00 00    lea    r15,[rip+0x74c]        # 9b8 <__str1>
26c:   b8 01 00 00 00          mov    eax,0x1
271:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
275:   b8 00 00 00 00          mov    eax,0x0
27a:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
27e:   4c 89 f7                mov    rdi,r14
281:   4c 89 fe                mov    rsi,r15
284:   48 8b 55 d0             mov    rdx,QWORD PTR [rbp-0x30]
288:   e8 68 fe ff ff          call   f5 <sys_write>
28d:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
291:   4c 89 e8                mov    rax,r13
294:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
298:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
29c:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
2a0:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
2a4:   48 89 ec                mov    rsp,rbp
2a7:   5d                      pop    rbp
2a8:   c3                      ret
```

27.4.18 print_int

ASH

```
pub func print_int(int64 n): int64
```

Writes a signed integer in decimal to standard output, without a line feed.

Parameter	Description
n	Value.

Returns. The result of `sys_write`.

Implementation. The source of the function in `nlib/syscalls.ash`:

NLIB/SYSCALLS.ASH

```
pub func print_int(int64 n): int64 {
    uint8[24] buf;
    int64 v = n;
    int64 k = 23;
    int64 neg = 0;
    buf[23] = 0;
    if (v == 0) {
        k--;
        buf[k] = 48;
    }
    if (v < 0) {
        neg = 1;
        v = 0 - v;
    }
    while (v > 0) {
        k--;
        buf[k] = 48 + v % 10;
        v = v / 10;
    }
    if (neg == 1) {
        k--;
        buf[k] = 45;
    }
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/SYSCALLS.ASH (CONTINUED)

```
    return sys_write(1, &buf + k, 23 - k);
}
```

Example.

Example 27-90. Using print_int

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    print_int(-1234);
    println("");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-1234
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000285 <print_int>:
 285:   55                      push   rbp
 286:   48 89 e5                mov    rbp,rsp
 289:   48 81 ec e0 00 00 00    sub    rsp,0xe0
 290:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
 294:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
 298:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
 29c:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
 2a0:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
 2a4:   b8 18 00 00 00          mov    eax,0x18
 2a9:   49 89 c4                mov    r12,rax
 2ac:   b8 00 00 00 00          mov    eax,0x0
 2b1:   49 89 c5                mov    r13,rax
 2b4:   4c 89 e7                mov    rdi,r12
 2b7:   be 01 00 00 00          mov    esi,0x1
 2bc:   ba 00 00 00 00          mov    edx,0x0
 2c1:   e8 3a fd ff ff          call   0 <__ash_alloc>
 2c6:   49 89 c5                mov    r13,rax
 2c9:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
 2cd:   49 89 c6                mov    r14,rax
 2d0:   b8 17 00 00 00          mov    eax,0x17
 2d5:   49 89 c7                mov    r15,rax
 2d8:   b8 00 00 00 00          mov    eax,0x0
 2dd:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
 2e1:   b8 17 00 00 00          mov    eax,0x17
 2e6:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 2ea:   b8 00 00 00 00          mov    eax,0x0
 2ef:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 2f3:   4c 89 ef                mov    rdi,r13
 2f6:   48 8b 75 c8             mov    rsi,QWORD PTR [rbp-0x38]
 2fa:   e8 d2 fd ff ff          call   d1 <__ash_bounds>
 2ff:   48 89 45 c0             mov    QWORD PTR [rbp-0x40],rax
 303:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
 307:   4c 89 e9                mov    rcx,r13
 30a:   48 01 c8                add    rax,rcx
 30d:   48 89 45 c8             mov    QWORD PTR [rbp-0x38],rax
 311:   b0 00                   mov    al,0x0
 313:   88 45 b8                mov    BYTE PTR [rbp-0x48],al
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
316:   48 8b 45 c8             mov    rax,QWORD PTR [rbp-0x38]
31a:   48 0f b6 4d b8          movzx  rcx,BYTE PTR [rbp-0x48]
31f:   88 08                   mov    BYTE PTR [rax],cl
321:   4c 89 f0                mov    rax,r14
324:   b9 00 00 00 00          mov    ecx,0x0
329:   48 39 c8                cmp    rax,rcx
32c:   0f 85 47 00 00 00       jne    379 <print_int+0xf4>
332:   4c 89 f8                mov    rax,r15
335:   48 ff c8                dec    rax
338:   49 89 c7                mov    r15,rax
33b:   4c 89 f8                mov    rax,r15
33e:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
342:   b8 00 00 00 00          mov    eax,0x0
347:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
34b:   4c 89 ef                mov    rdi,r13
34e:   48 8b 75 b0             mov    rsi,QWORD PTR [rbp-0x50]
352:   e8 7a fd ff ff          call   d1 <__ash_bounds>
357:   48 89 45 a8             mov    QWORD PTR [rbp-0x58],rax
35b:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
35f:   4c 89 e9                mov    rcx,r13
362:   48 01 c8                add    rax,rcx
365:   48 89 45 b0             mov    QWORD PTR [rbp-0x50],rax
369:   b0 30                   mov    al,0x30
36b:   88 45 a0                mov    BYTE PTR [rbp-0x60],al
36e:   48 8b 45 b0             mov    rax,QWORD PTR [rbp-0x50]
372:   48 0f b6 4d a0          movzx  rcx,BYTE PTR [rbp-0x60]
377:   88 08                   mov    BYTE PTR [rax],cl
379:   4c 89 f0                mov    rax,r14
37c:   b9 00 00 00 00          mov    ecx,0x0
381:   48 39 c8                cmp    rax,rcx
384:   0f 8d 2f 00 00 00       jge    3b9 <print_int+0x134>
38a:   b8 01 00 00 00          mov    eax,0x1
38f:   48 89 45 d0             mov    QWORD PTR [rbp-0x30],rax
393:   b8 00 00 00 00          mov    eax,0x0
398:   48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
39c:   4c 89 f0                mov    rax,r14
39f:   48 89 45 90             mov    QWORD PTR [rbp-0x70],rax
3a3:   48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
3a7:   48 8b 4d 90             mov    rcx,QWORD PTR [rbp-0x70]
3ab:   48 29 c8                sub    rax,rcx
3ae:   48 89 45 98             mov    QWORD PTR [rbp-0x68],rax
3b2:   48 8b 45 98             mov    rax,QWORD PTR [rbp-0x68]
3b6:   49 89 c6                mov    r14,rax
3b9:   4c 89 f0                mov    rax,r14
3bc:   b9 00 00 00 00          mov    ecx,0x0
3c1:   48 39 c8                cmp    rax,rcx
3c4:   0f 8e ad 00 00 00       jle    477 <print_int+0x1f2>
3ca:   4c 89 f8                mov    rax,r15
3cd:   48 ff c8                dec    rax
3d0:   49 89 c7                mov    r15,rax
3d3:   4c 89 f8                mov    rax,r15
3d6:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
3da:   b8 00 00 00 00          mov    eax,0x0
3df:   48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
3e3:   4c 89 ef                mov    rdi,r13
3e6:   48 8b 75 88             mov    rsi,QWORD PTR [rbp-0x78]
3ea:   e8 e2 fc ff ff          call   d1 <__ash_bounds>
3ef:   48 89 45 80             mov    QWORD PTR [rbp-0x80],rax
3f3:   48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
3f7:   4c 89 e9                mov    rcx,r13
3fa:   48 01 c8                add    rax,rcx
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
 3fd:   48 89 45 88             mov    QWORD PTR [rbp-0x78],rax
 401:   b8 30 00 00 00          mov    eax,0x30
 406:   48 89 85 78 ff ff ff    mov    QWORD PTR [rbp-0x88],rax
 40d:   4c 89 f0                mov    rax,r14
 410:   48 89 85 70 ff ff ff    mov    QWORD PTR [rbp-0x90],rax
 417:   b9 0a 00 00 00          mov    ecx,0xa
 41c:   48 8b 85 70 ff ff ff    mov    rax,QWORD PTR [rbp-0x90]
 423:   48 99                   cqo
 425:   48 f7 f9                idiv   rcx
 428:   48 89 95 70 ff ff ff    mov    QWORD PTR [rbp-0x90],rdx
 42f:   48 8b 85 78 ff ff ff    mov    rax,QWORD PTR [rbp-0x88]
 436:   48 8b 8d 70 ff ff ff    mov    rcx,QWORD PTR [rbp-0x90]
 43d:   48 01 c8                add    rax,rcx
 440:   48 89 85 78 ff ff ff    mov    QWORD PTR [rbp-0x88],rax
 447:   48 8b 85 78 ff ff ff    mov    rax,QWORD PTR [rbp-0x88]
 44e:   88 85 68 ff ff ff       mov    BYTE PTR [rbp-0x98],al
 454:   48 8b 45 88             mov    rax,QWORD PTR [rbp-0x78]
 458:   48 0f b6 8d 68 ff ff    movzx  rcx,BYTE PTR [rbp-0x98]
 45f:   ff
 460:   88 08                   mov    BYTE PTR [rax],cl
 462:   b9 0a 00 00 00          mov    ecx,0xa
 467:   4c 89 f0                mov    rax,r14
 46a:   48 99                   cqo
 46c:   48 f7 f9                idiv   rcx
 46f:   49 89 c6                mov    r14,rax
 472:   e9 42 ff ff ff          jmp    3b9 <print_int+0x134>
 477:   48 8b 45 d0             mov    rax,QWORD PTR [rbp-0x30]
 47b:   b9 01 00 00 00          mov    ecx,0x1
 480:   48 39 c8                cmp    rax,rcx
 483:   0f 85 62 00 00 00       jne    4eb <print_int+0x266>
 489:   4c 89 f8                mov    rax,r15
 48c:   48 ff c8                dec    rax
 48f:   49 89 c7                mov    r15,rax
 492:   4c 89 f8                mov    rax,r15
 495:   48 89 85 60 ff ff ff    mov    QWORD PTR [rbp-0xa0],rax
 49c:   b8 00 00 00 00          mov    eax,0x0
 4a1:   48 89 85 58 ff ff ff    mov    QWORD PTR [rbp-0xa8],rax
 4a8:   4c 89 ef                mov    rdi,r13
 4ab:   48 8b b5 60 ff ff ff    mov    rsi,QWORD PTR [rbp-0xa0]
 4b2:   e8 1a fc ff ff          call   d1 <__ash_bounds>
 4b7:   48 89 85 58 ff ff ff    mov    QWORD PTR [rbp-0xa8],rax
... (41 more lines not shown)
```

27.5 <ARGS>: COMMAND LINE ARGUMENTS

`import <args>;` helps to read the arguments of the program. The entry function must receive them: `pub func main(int64 argc, int64 argv): int64` (Section 12.9). `arg_equals` and `arg_int` call the C functions `strcmp` and `atoi`, so this library requires the C library.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-91. Every function of <args>

ASH

```
import <args>;

@Entry
pub func main(int64 argc, int64 argv): int64 {
    for (int64 i = 1; i < argc; i++) {
        printf("arg_at(argv, %d) = %s\n", i, arg_at(argv, i));
    }
    printf("arg_equals(argv, 3, \"hello\") = %d\n", arg_equals(argv, 3, "hello"));
    printf("arg_int(argv, 2) = %d\n", arg_int(argv, 2));
    printf("arg_has(argc, argv, \"-v\") = %d\n", arg_has(argc, argv, "-v"));
    printf("arg_has(argc, argv, \"-q\") = %d\n", arg_has(argc, argv, "-q"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
arg_at(argv, 1) = -v
arg_at(argv, 2) = 42
arg_at(argv, 3) = hello
arg_equals(argv, 3, "hello") = 1
arg_int(argv, 2) = 42
arg_has(argc, argv, "-v") = 1
arg_has(argc, argv, "-q") = 0
```

The example was run as `./program -v 42 hello`.

27.5.1 arg_at

ASH

```
pub func arg_at(int64 argv, int64 i): int64
```

Returns the address of the text of command line argument *i*. It reads `argv[i]` with inline assembly (`mov rax, [rax + rcx*8]`).

Parameter	Description
`argv`	The second parameter of the entry function.
`i`	Index, 0 to argc-1.

Returns. The address of the argument, usable with `%s`.

Implementation. The source of the function in `nlib/args.ash`:

NLIB/ARGS.ASH

```
pub func arg_at(int64 argv, int64 i): int64 {
    int64 r = 0;
    asm {
        mov rax, argv
        mov rcx, i
        mov rax, [rax + rcx*8]
        mov r, rax
    }
    return r;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.5.2 arg_equals

```ash
pub func arg_equals(int64 argv, int64 i, string s): int64
```

Compares argument *i* with the string *s* using the C library's `strcmp`.

Parameter	Description
`argv`	Argument vector.
`i`	Index.
`s`	String to compare with.

Returns. 1 if equal, else 0.

Implementation. The source of the function in `nlib/args.ash`:

```ash
pub func arg_equals(int64 argv, int64 i, string s): int64 {
    if (strcmp(arg_at(argv, i), s) == 0) {
        return 1;
    }
    return 0;
}
```

27.5.3 arg_int

```ash
pub func arg_int(int64 argv, int64 i): int64
```

Converts argument *i* to an integer with the C library's `atoi`.

Parameter	Description
`argv`	Argument vector.
`i`	Index.

Returns. The numeric value (0 if not a number).

Implementation. The source of the function in `nlib/args.ash`:

```ash
pub func arg_int(int64 argv, int64 i): int64 {
    return atoi(arg_at(argv, i));
}
```

27.5.4 arg_has

```ash
pub func arg_has(int64 argc, int64 argv, string s): int64
```

Tests whether any argument is equal to *s*, typically a flag such as `-v`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Parameter	Description
`argc`	Argument count.
`argv`	Argument vector.
`s`	String to look for.

Returns. 1 if found, else 0.

Implementation. The source of the function in `nlib/args.ash`:

NLIB/ARGS.ASH

```
pub func arg_has(int64 argc, int64 argv, string s): int64 {
    for (int64 i = 0; i < argc; i++) {
        if (arg_equals(argv, i, s) == 1) {
            return 1;
        }
    }
    return 0;
}
```

27.6 <DELAY>: PAUSES

`import <delay>;` provides two functions built on the C library's `usleep`. In programs without the C library use `sys_nanosleep` from `<syscalls>`.

27.6.1 delay

ASH

```
pub func delay(int64 ms): void
```

Pauses the program for *ms* milliseconds, through the C library's `usleep`.

Parameter	Description
`ms`	Milliseconds.

Returns. Nothing.

Implementation. The source of the function in `nlib/delay.ash`:

NLIB/DELAY.ASH

```
pub func delay(int64 ms): void {
    usleep(ms * 1000);
}
```

Example.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-92. Using delay

ASH

```
import <delay>;

@Entry
pub func main(): int64 {
    delay(25);
    printf("waited 25 ms\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
waited 25 ms
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <delay>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  1f:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  23:   49 89 c4                mov    r12,rax
  26:   4c 89 e0                mov    rax,r12
  29:   48 69 c0 e8 03 00 00    imul   rax,rax,0x3e8
  30:   49 89 c4                mov    r12,rax
  33:   4c 89 e7                mov    rdi,r12
  36:   b8 00 00 00 00          mov    eax,0x0
  3b:   e8 00 00 00 00          call   40 <delay+0x40>
  40:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  44:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  48:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  4c:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  50:   48 89 ec                mov    rsp,rbp
  53:   5d                      pop    rbp
  54:   c3                      ret
```

27.6.2 delayus

ASH

```
pub func delayus(int64 us): void
```

Pauses the program for *us* microseconds (`usleep`).

Parameter	Description
`us`	Microseconds.

Returns. Nothing.

Implementation. The source of the function in `nlib/delay.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/DELAY.ASH

```
pub func delayus(int64 us): void {
    usleep(us);
}
```

Example.

Example 27-93. Using delayus

ASH

```
import <delay>;

@Entry
pub func main(): int64 {
    delayus(500);
    printf("waited 500 us\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
waited 500 us
```

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <delayus>:
   0:   55                      push   rbp
   1:   48 89 e5                mov    rbp,rsp
   4:   48 81 ec 30 00 00 00    sub    rsp,0x30
   b:   4c 89 65 f8             mov    QWORD PTR [rbp-0x8],r12
   f:   4c 89 6d f0             mov    QWORD PTR [rbp-0x10],r13
  13:   4c 89 75 e8             mov    QWORD PTR [rbp-0x18],r14
  17:   4c 89 7d e0             mov    QWORD PTR [rbp-0x20],r15
  1b:   48 89 7d d8             mov    QWORD PTR [rbp-0x28],rdi
  1f:   48 8b 45 d8             mov    rax,QWORD PTR [rbp-0x28]
  23:   49 89 c4                mov    r12,rax
  26:   4c 89 e7                mov    rdi,r12
  29:   b8 00 00 00 00          mov    eax,0x0
  2e:   e8 00 00 00 00          call   33 <delayus+0x33>
  33:   4c 8b 65 f8             mov    r12,QWORD PTR [rbp-0x8]
  37:   4c 8b 6d f0             mov    r13,QWORD PTR [rbp-0x10]
  3b:   4c 8b 75 e8             mov    r14,QWORD PTR [rbp-0x18]
  3f:   4c 8b 7d e0             mov    r15,QWORD PTR [rbp-0x20]
  43:   48 89 ec                mov    rsp,rbp
  46:   5d                      pop    rbp
  47:   c3                      ret
```

27.7 <STDIO>: C STANDARD INPUT AND OUTPUT

`import <stdio>;` declares the functions of the C header `<stdio.h>` with ASH types, so that calls are checked and arguments are passed with the declared widths. It contains no ASH code: the functions are those of the C library linked with the program. Streams (`FILE*`) are handled as `void*` values.

Copyright © 2026 Mallionic Inc. All rights reserved.

NOTE

The C functions can also be called without importing `<stdio>`, as implicit external functions (Section 12.8.1). Importing the library documents the program and enables argument checking. Because the library declares `tmpnam` and `tempnam`, the GNU linker prints a warning about them when `<stdio>` is imported; the warning is harmless.

Example 27-94. Writing and reading a file with <stdio>

ASH

```
import <stdio>;

@Entry
pub func main(): int64 {
    void* f = fopen("/tmp/ash_guide_stdio.txt", "w");
    fprintf(f, "line %d\n", 1);
    fputs("line 2\n", f);
    fclose(f);

    uint8[64] line;
    f = fopen("/tmp/ash_guide_stdio.txt", "r");
    while (fgets(&line, 64, f) != 0) {
        printf("read: %s", &line);
    }
    fseek(f, 0, 2);
    printf("size: %d bytes\n", ftell(f));
    fclose(f);
    remove("/tmp/ash_guide_stdio.txt");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
read: line 1
read: line 2
size: 14 bytes
```

Example 27-95. Formatting into buffers: snprintf and sscanf

ASH

```
import <stdio>;

@Entry
pub func main(): int64 {
    uint8[64] buf;
    snprintf(&buf, 64, "%s-%04d-%x", "ASH", 42, 255);
    puts(&buf);
    int64 a = 0;
    int64 b = 0;
    int64 n = sscanf("12 34", "%ld %ld", &a, &b);
    printf("sscanf matched %d values: %d + %d = %d\n", n, a, b, a + b);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
ASH-0042-ff
sscanf matched 2 values: 12 + 34 = 46
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 27-96. Reading standard input

ASH

```
import <stdio>;

@Entry
pub func main(): int64 {
    uint8[32] name;
    printf("Your name? ");
    scanf("%31s", &name);
    printf("Hello, %s!\n", &name);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Your name? Hello, Ada\n!
```

Example 27-97. Running a command with popen

ASH

```
import <stdio>;

@Entry
pub func main(): int64 {
    void* p = popen("echo 'from a shell command'", "r");
    uint8[128] line;
    while (fgets(&line, 128, p) != 0) {
        printf("got: %s", &line);
    }
    printf("status: %d\n", pclose(p));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
got: from a shell command
status: 0
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 27-4. Declarations of <stdio>

Declaration	Description
`int64 remove(string filename)`	Deletes a file.
`int64 rename(string old, string new)`	Renames or moves a file.
`void* tmpfile()`	Creates a temporary file that is deleted when closed.
`string tmpnam(void* buffer)`	Generates a unique temporary file name (unsafe; prefer mkstemp).
`string tmpnam_r(void* buffer)`	Reentrant tmpnam.
`string tempnam(string dir, string prefix)`	Generates a temporary name in a directory with a prefix.
`int64 fflush(void* stream)`	Writes the buffered output of a stream (all streams if the argument is 0).
`int64 fclose(void* stream)`	Flushes and closes a stream.
`int64 fcloseall()`	Closes all open streams.
`void* fopen(string filename, string modes)`	Opens a file and returns a stream (`FILE*`), or 0 on failure. Modes: "r", "w", "a", "r+", "w+", "a+", with "b" accepted.
`void* freopen(string filename, string modes, void* stream)`	Reopens a stream on another file.
`void* fopen64(string filename, string modes)`	fopen with 64-bit file offsets.
`void* freopen64(string filename, string modes, void* stream)`	freopen with 64-bit file offsets.
`void setbuf(void* stream, string buffer)`	Sets the buffer of a stream.
`int64 setvbuf(void* stream, string buffer, int64 mode, uint64 size)`	Sets the buffer and buffering mode of a stream (0 full, 1 line, 2 none).
`void setbuffer(void* stream, string buffer, uint64 size)`	Sets the buffer of a stream with an explicit size.
`void setlinebuf(void* stream)`	Makes a stream line-buffered.
`int64 printf(string format, ...)`	Formatted output to standard output.
`int64 fprintf(void* stream, string format, ...)`	Formatted output to a stream.
`int64 sprintf(string buffer, string format, ...)`	Formatted output into a buffer (no size check; prefer snprintf).
`int64 vprintf(string format, void* args)`	printf with a va_list.
`int64 vfprintf(void* stream, string format, void* args)`	fprintf with a va_list.
`int64 vsprintf(string buffer, string format, void* args)`	sprintf with a va_list.
`int64 snprintf(string buffer, uint64 size, string format, ...)`	Formatted output into a buffer of a given size, always NUL-terminated.
`int64 vsnprintf(string buffer, uint64 size, string format, void* args)`	snprintf with a va_list.

Copyright © 2026 Mallionic Inc. All rights reserved.

Declaration	Description
`int64 asprintf(void* buffer, string format, ...)`	Formatted output into a newly allocated string.
`int64 vasprintf(void* buffer, string format, void* args)`	asprintf with a va_list.
`int64 dprintf(int64 fd, string format, ...)`	Formatted output to a file descriptor.
`int64 vdprintf(int64 fd, string format, void* args)`	dprintf with a va_list.
`int64 scanf(string format, ...)`	Formatted input from standard input.
`int64 fscanf(void* stream, string format, ...)`	Formatted input from a stream.
`int64 sscanf(string buffer, string format, ...)`	Formatted input from a string.
`int64 vscanf(string format, void* args)`	scanf with a va_list.
`int64 vfscanf(void* stream, string format, void* args)`	fscanf with a va_list.
`int64 vsscanf(string buffer, string format, void* args)`	sscanf with a va_list.
`int64 fgetc(void* stream)`	Reads one byte from a stream; -1 (EOF) at end of file.
`int64 getc(void* stream)`	Same as fgetc.
`int64 getchar()`	Reads one byte from standard input.
`int64 getc_unlocked(void* stream)`	getc without locking.
`int64 getchar_unlocked()`	getchar without locking.
`int64 fgetc_unlocked(void* stream)`	fgetc without locking.
`int64 fputc(int64 c, void* stream)`	Writes one byte to a stream.
`int64 putc(int64 c, void* stream)`	Same as fputc.
`int64 putchar(int64 c)`	Writes one byte to standard output.
`int64 fputc_unlocked(int64 c, void* stream)`	fputc without locking.
`int64 putc_unlocked(int64 c, void* stream)`	putc without locking.
`int64 putchar_unlocked(int64 c)`	putchar without locking.
`int64 getw(void* stream)`	Reads an int from a stream (obsolete).
`int64 putw(int64 value, void* stream)`	Writes an int to a stream (obsolete).
`string fgets(string buffer, int64 size, void* stream)`	Reads a line (at most size-1 bytes) into a buffer, keeping the newline; returns 0 at end of file.
`string fgets_unlocked(string buffer, int64 size, void* stream)`	fgets without locking.
`int64 fputs(string string, void* stream)`	Writes a string to a stream (no newline added).
`int64 fputs_unlocked(string string, void* stream)`	fputs without locking.

Copyright © 2026 Mallionic Inc. All rights reserved.

Declaration	Description
`int64 puts(string string)`	Writes a string and a newline to standard output.
`int64 ungetc(int64 c, void* stream)`	Pushes a byte back onto a stream.
`uint64 fread(void* ptr, uint64 size, uint64 count, void* stream)`	Reads count items of size bytes from a stream.
`uint64 fwrite(void* ptr, uint64 size, uint64 count, void* stream)`	Writes count items of size bytes to a stream.
`uint64 fread_unlocked(void* ptr, uint64 size, uint64 count, void* stream)`	fread without locking.
`uint64 fwrite_unlocked(void* ptr, uint64 size, uint64 count, void* stream)`	fwrite without locking.
`int64 fseek(void* stream, int64 offset, int64 whence)`	Moves the position of a stream (0 SEEK_SET, 1 SEEK_CUR, 2 SEEK_END).
`int64 ftell(void* stream)`	Returns the position of a stream.
`void rewind(void* stream)`	Moves to the start of a stream and clears its error flag.
`int64 fseeko(void* stream, int64 offset, int64 whence)`	fseek with an off_t offset.
`int64 ftello(void* stream)`	ftell returning off_t.
`int64 fgetpos(void* stream, void* position)`	Stores the position of a stream in an fpos_t.
`int64 fsetpos(void* stream, void* position)`	Restores a position saved by fgetpos.
`int64 fseeko64(void* stream, int64 offset, int64 whence)`	64-bit fseeko.
`int64 ftello64(void* stream)`	64-bit ftello.
`int64 fgetpos64(void* stream, void* position)`	64-bit fgetpos.
`int64 fsetpos64(void* stream, void* position)`	64-bit fsetpos.
`void clearerr(void* stream)`	Clears the end-of-file and error flags of a stream.
`int64 feof(void* stream)`	Non-zero if the end of file was reached.
`int64 ferror(void* stream)`	Non-zero if an error occurred on the stream.
`void clearerr_unlocked(void* stream)`	clearerr without locking.
`int64 feof_unlocked(void* stream)`	feof without locking.
`int64 ferror_unlocked(void* stream)`	ferror without locking.
`void perror(string string)`	Prints a message followed by the description of errno on standard error.
`int64 fileno(void* stream)`	Returns the file descriptor of a stream.
`int64 fileno_unlocked(void* stream)`	fileno without locking.
`int64 pclose(void* stream)`	Closes a stream opened by popen and waits for the command.

Copyright © 2026 Mallionic Inc. All rights reserved.

Declaration	Description
`void* popen(string command, string modes)`	Runs a shell command and returns a stream connected to its input or output ("r" or "w").
`string ctermid(string buffer)`	Returns the name of the controlling terminal.
`string cuserid(string buffer)`	Returns the name of the user (obsolete).
`int64 flockfile(void* stream)`	Locks a stream for the calling thread.
`int64 ftrylockfile(void* stream)`	Tries to lock a stream.
`void funlockfile(void* stream)`	Unlocks a stream.
`int64 __uflow(void* stream)`	Internal glibc helper (refill a read buffer).
`int64 __overflow(void* stream, int64 c)`	Internal glibc helper (flush a write buffer).

27.8 <STDLIB>: C STANDARD LIBRARY WRAPPERS

`<stdlib>` defines `NULL`, `RAND_MAX`, `EXIT_SUCCESS` and `EXIT_FAILURE`, and wraps the functions of `<stdlib.h>`: each wrapper `f` calls an external function named `__stlib_f`.

WARNING

In ASH 1.0 the `__stlib_*` functions are not provided by any library, so a program that imports `<stdlib>` cannot be linked ("undefined reference to `__stlib_malloc`" and so on). Until a shim library is supplied, **do not import** `<stdlib>`: call the C functions directly, as implicit externs or with your own `extern` declarations, as in the examples below. The constants can be defined with `define` in your own program.

Example 27-98. Using C standard library functions directly

ASH

```
extern void* malloc(uint64 size);
extern void free(void* p);
extern int64 strtol(string s, void* end, int64 base);
extern string getenv(string name);
extern void srand(uint32 seed);
extern int32 rand();

@Entry
pub func main(): int64 {
    int64* p = malloc(4 * 8);
    p[0] = 10; p[1] = 20; p[2] = 30; p[3] = 40;
    printf("sum=%d\n", p[0] + p[1] + p[2] + p[3]);
    free(p);
    printf("strtol: %d %d\n", strtol("ff", 0, 16), strtol("-1010", 0, 2));
    string home = getenv("HOME");
    if (home != 0) {
        printf("HOME is set\n");
    }
    srand(42);
    int32 r = rand();
    if (r >= 0) {
        printf("rand returned a non-negative number\n");
    }
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
sum=100
strtol: 255 -10
HOME is set
rand returned a non-negative number
```

Table 27-5. Functions wrapped by <stdlib>

Group	Functions	Purpose
Number conversion	`atof`, `atoi`, `atol`, `atoll`, `strtod`, `strtof`, `strtold`, `strtol`, `strtoul`, `strtoll`, `strtoull`	Convert text to numbers; the `strto*` functions report where conversion stopped and accept a base.
Random numbers	`rand`, `srand`, `random`, `srandom`, `initstate`, `setstate`, `random_r`, `srandom_r`, `initstate_r`, `setstate_r`, `drand48`, `erand48`, `lrand48`, `nrand48`, `mrand48`, `jrand48`, `srand48`, `seed48`, `lcong48`, `drand48_r`, `erand48_r`, `lrand48_r`, `nrand48_r`, `mrand48_r`, `jrand48_r`, `srand48_r`, `seed48_r`, `lcong48_r`	Pseudo-random generators of the C library and of the System V `*rand48` family.
Memory	`malloc`, `calloc`, `realloc`, `reallocarray`, `free`, `posix_memalign`, `aligned_alloc`, `valloc`	Heap allocation.
Process control	`abort`, `exit`, `quick_exit`, `_Exit`, `atexit`, `at_quick_exit`, `on_exit`, `system`	Ending the process, registering exit handlers, running shell commands.
Environment	`getenv`, `secure_getenv`, `putenv`, `setenv`, `unsetenv`, `clearenv`	Environment variables.
Temporary files and paths	`mktemp`, `mkstemp`, `mkostemp`, `mkstemps`, `mkostemps`, `mkdtemp`, `realpath`, `canonicalize_file_name`	Safe temporary names and canonical paths.
Searching and sorting	`bsearch`, `qsort`	Binary search and quicksort with a comparison callback.
Integer arithmetic	`abs`, `labs`, `llabs`, `div`, `ldiv`, `lldiv`	Absolute values and quotient/remainder pairs.
Multibyte characters	`mblen`, `mbtowc`, `wctomb`, `mbstowcs`, `wcstombs`	Conversion between multibyte and wide characters.
Terminals and miscellany	`getsubopt`, `getloadavg`, `rpmatch`, `posix_openpt`, `grantpt`, `unlockpt`, `ptsname`, `ptsname_r`, `getpt`, `ttyslot`	Pseudo-terminals, option parsing, load average.
Floating-point formatting	`ecvt`, `fcvt`, `gcvt`, `qecvt`, `qfcvt`, `qgcvt`, `ecvt_r`, `fcvt_r`, `qecvt_r`, `qfcvt_r`, `strfromd`, `strfromf`, `strfroml`	Conversion of floating-point values to text.

Copyright © 2026 Mallionic Inc. All rights reserved.

27.9 <TYPES>: FLOATING-POINT SPECIAL VALUES

`import <types>;` provides functions that return IEEE-754 double-precision special values as bit patterns. Because floating-point support is limited in ASH 1.0 (Section 8.8), these values can be stored in `double` variables and passed to C, but not converted to integers or used in arithmetic.

27.9.1 inf

ASH

```
pub func inf(): double
```

Returns positive infinity as a `double` (bit pattern 0x7FF0000000000000).

Returns. +inf.

Implementation. The source of the function in `nlib/types.ash`:

NLIB/TYPES.ASH

```
pub func inf(): double {
    double x = 0x7ff0000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <inf>:
   0:   48 b8 00 00 00 00 00    movabs rax,0x7ff0000000000000
   7:   00 f0 7f
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0x7ff0000000000000
  1b:   00 f0 7f
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.9.2 ninf

ASH

```
pub func ninf(): double
```

Returns negative infinity (0xFFF0000000000000).

Returns. -inf.

Implementation. The source of the function in `nlib/types.ash`:

NLIB/TYPES.ASH

```
pub func ninf(): double {
    double x = 0xfff0000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <ninf>:
   0:   48 b8 00 00 00 00 00    movabs rax,0xfff0000000000000
   7:   00 f0 ff
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0xfff0000000000000
  1b:   00 f0 ff
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.9.3 pos_inf

ASH

```
pub func pos_inf(): double
```

Synonym of `inf`.

Returns. +inf.

Implementation. The source of the function in `nlib/types.ash`:

NLIB/TYPES.ASH

```
pub func pos_inf(): double {
    double x = 0x7ff0000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <pos_inf>:
   0:   48 b8 00 00 00 00 00    movabs rax,0x7ff0000000000000
   7:   00 f0 7f
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0x7ff0000000000000
  1b:   00 f0 7f
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.9.4 neg_inf

ASH

```
pub func neg_inf(): double
```

Synonym of `ninf`.

Returns. -inf.

Implementation. The source of the function in `nlib/types.ash`:

NLIB/TYPES.ASH

```
pub func neg_inf(): double {
    double x = 0xfff0000000000000;
    return x;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/TYPES.ASH (CONTINUED)

```
}
```

Machine code. The function as compiled in a program that calls it:

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <neg_inf>:
   0:   48 b8 00 00 00 00 00    movabs rax,0xfff0000000000000
   7:   00 f0 ff
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0xfff0000000000000
  1b:   00 f0 ff
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.9.5 float_nan

ASH

```
pub func float_nan(): double
```

Returns a quiet NaN (0x7FF8000000000000).

Returns. NaN.

Implementation. The source of the function in `nlib/types.ash`:

NLIB/TYPES.ASH

```
pub func float_nan(): double {
    double x = 0x7ff8000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <float_nan>:
   0:   48 b8 00 00 00 00 00    movabs rax,0x7ff8000000000000
   7:   00 f8 7f
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0x7ff8000000000000
  1b:   00 f8 7f
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.9.6 float_inf

ASH

```
pub func float_inf(): double
```

Synonym of `inf`.

Returns. +inf.

Implementation. The source of the function in `nlib/types.ash`:

Copyright © 2026 Mallionic Inc. All rights reserved.

NLIB/TYPES.ASH

```
pub func float_inf(): double {
    double x = 0x7ff0000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <float_inf>:
   0:   48 b8 00 00 00 00 00    movabs rax,0x7ff0000000000000
   7:   00 f0 7f
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0x7ff0000000000000
  1b:   00 f0 7f
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.9.7 float_ninf

ASH

```
pub func float_ninf(): double
```

Synonym of `ninf`.

Returns. -inf.

Implementation. The source of the function in `nlib/types.ash`:

NLIB/TYPES.ASH

```
pub func float_ninf(): double {
    double x = 0xfff0000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <float_ninf>:
   0:   48 b8 00 00 00 00 00    movabs rax,0xfff0000000000000
   7:   00 f0 ff
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0xfff0000000000000
  1b:   00 f0 ff
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.9.8 double_nan

ASH

```
pub func double_nan(): double
```

Synonym of `float_nan`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Returns. NaN.

Implementation. The source of the function in `nlib/types.ash`:

NLIB/TYPES.ASH

```
pub func double_nan(): double {
    double x = 0x7ff8000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <double_nan>:
   0:   48 b8 00 00 00 00 00    movabs rax,0x7ff8000000000000
   7:   00 f8 7f
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0x7ff8000000000000
  1b:   00 f8 7f
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.9.9 double_inf

ASH

```
pub func double_inf(): double
```

Synonym of `inf`.

Returns. +inf.

Implementation. The source of the function in `nlib/types.ash`:

NLIB/TYPES.ASH

```
pub func double_inf(): double {
    double x = 0x7ff0000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX)

```
0000000000000000 <double_inf>:
   0:   48 b8 00 00 00 00 00    movabs rax,0x7ff0000000000000
   7:   00 f0 7f
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0x7ff0000000000000
  1b:   00 f0 7f
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

Copyright © 2026 Mallionic Inc. All rights reserved.

27.9.10 double_ninf

```ash
pub func double_ninf(): double
```

Synonym of `ninf`.

Returns. -inf.

Implementation. The source of the function in `nlib/types.ash`:

```ash
pub func double_ninf(): double {
    double x = 0xfff0000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

```
0000000000000000 <double_ninf>:
   0:   48 b8 00 00 00 00 00    movabs rax,0xfff0000000000000
   7:   00 f0 ff
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0xfff0000000000000
  1b:   00 f0 ff
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.9.11 nan

```ash
pub func nan(): double
```

Returns a quiet NaN (0x7FF8000000000000).

Returns. NaN.

Implementation. The source of the function in `nlib/types.ash`:

```ash
pub func nan(): double {
    double x = 0x7FF8000000000000;
    return x;
}
```

Machine code. The function as compiled in a program that calls it:

```
0000000000000000 <nan>:
   0:   48 b8 00 00 00 00 00    movabs rax,0x7ff8000000000000
   7:   00 f8 7f
   a:   66 48 0f 6e c0          movq   xmm0,rax
   f:   f2 44 0f 10 c0          movsd  xmm8,xmm0
  14:   48 b8 00 00 00 00 00    movabs rax,0x7ff8000000000000
  1b:   00 f8 7f
```

Copyright © 2026 Mallionic Inc. All rights reserved.

X86-64 DISASSEMBLY (OBJDUMP, INTEL SYNTAX) (CONTINUED)

```
  1e:   66 48 0f 6e c0          movq   xmm0,rax
  23:   c3                      ret
```

27.10 <STDBOOL> AND <NULL>: CONSTANTS

<stdbool> defines the boolean constants with define. true and false are also predefined by the compiler with right -1, so importing <stdbool> is only needed for the uppercase forms.

Table 27-6. Constants defined by <stdbool>

Name	Value	Meaning
true	1	boolean true
false	0	boolean false
TRUE	1	boolean true
FALSE	0	boolean false

Example 27-99. Boolean constants

ASH

```
import <stdbool>;

@Entry
pub func main(): int64 {
    bool done = FALSE;
    int64 n = 0;
    while (done == FALSE) {
        n++;
        if (n == 3) { done = TRUE; }
    }
    printf("n=%d TRUE=%d FALSE=%d\n", n, TRUE, FALSE);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
n=3 TRUE=1 FALSE=0
```

<null> defines the null pointer constant under five names:

Table 27-7. Constants defined by <null>

Name	Value	Meaning
unsafePtr	(void*)0	the null pointer constant used by the others
NULL	unsafePtr	null pointer
null	unsafePtr	null pointer
nil	unsafePtr	null pointer
NIL	unsafePtr	null pointer

Copyright © 2026 Mallionic Inc. All rights reserved.

WARNING

In ASH 1.0, `import <null>;` fails, because the macro `null` defined by the library rewrites the name of the import itself. Define the constant yourself (`define NULL 0;`) or compare pointers with 0.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 28

DIAGNOSTICS REFERENCE

This chapter lists the diagnostics that `ashc` prints most often, grouped by the stage that detects them. For each one it shows a program that triggers it, the message, the cause and the fix. Messages that `ashc` 1.0 prints in Spanish are shown translated, and the box says so.

28.1 READING A DIAGNOSTIC

A diagnostic consists of a class (`error`, `warning`, `tip`, `internal compiler error`), an optional position (`<input>:line:column:` for errors found by the parser), the message, and for parser errors the offending line with a marker. When the compiler knows a likely fix it prints a `tip` line after the error. The build ends with a summary such as `1 error generated.` See Section 5.5 for the colors and classes.

28.2 LEXICAL ERRORS

28.2.1 Unexpected character

ASH

```ash
@Entry
pub func main(): int64 {
    int64 $price = 3;
    return 0;
}
```

COMPILER DIAGNOSTICS

```
error: ASH lexer error at 3:11: unexpected character '$'
1 error generated.
```

Cause. A character that cannot start any token, here `$`. **Fix.** Identifiers contain only letters, digits and underscores (Section 7.4).

28.2.2 Unterminated string

ASH

```ash
@Entry
pub func main(): int64 {
    printf("missing quote\n);
    return 0;
}
```

COMPILER DIAGNOSTICS

```
error: ASH lexer error at 3:30: String sin cerrar
1 error generated.
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Cause. A string literal that reaches the end of the line. **Fix.** Close the string, or use a triple-quoted string for text that spans lines.

28.3 SYNTAX ERRORS

28.3.1 Unexpected statement

ASH

```
@Entry
pub func main(): int64 {
    int64 i = 0;
    do {
        i++;
    } while (i < 3);
    return i;
}
```

COMPILER DIAGNOSTICS

```
<input>:4:5: error: unexpected statement: IDENT(do)@4:5
    4 | do
      |     ^^
1 error generated.
```

Cause. A word that does not start any ASH statement; here `do`, which ASH does not have. **Fix.** Use `loop { ... if (...) { break; } }` (Section 11.5.1).

28.3.2 Missing brace

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 5;
    if (x > 3) printf("big\n");
    return 0;
}
```

COMPILER DIAGNOSTICS

```
<input>:4:16: error: expected { pero se encontro IDENT(printf)@4:16
    4 | printf
      |                ^^^^^^
1 error generated.
```

Cause. The body of `if`, `while` and other statements must be a block in braces. **Fix.** Write `if (x > 3) { printf("big\n"); }`.

28.3.3 Expected func, class, extern or import

ASH

```
@Entry
pub func main(): int64 {
    return 0;
}
x = 3;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

COMPILER DIAGNOSTICS

```
<input>:5:1: error: expected 'func', 'class', 'extern' or 'import'
    5 | x
      | ^
1 error generated.
```

Cause. A statement at top level, outside any function. **Fix.** Move it into a function, or declare a global variable with `general` / `ungeneral` (Section 9.4).

28.3.4 Annotation errors

ASH

```ash
@Entry
@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS

```
<input>:3:1: error: El atributo @Entry aparece dos veces
    3 | pub
      | ^^^
1 error generated.
```

Cause. Two `@Entry` annotations. **Fix.** A program has exactly one entry point.

ASH

```ash
@Inline
func f(): int64 {
    return 1;
}

@Entry
pub func main(): int64 {
    return f();
}
```

COMPILER DIAGNOSTICS

```
<input>:2:1: error: Atributo unknown @Inline
    2 | func
      | ^^^^
1 error generated.
```

Cause. An annotation that is neither built in nor defined by the program. **Fix.** Check the spelling against Table 17-3 or define the annotation (Section 17.16).

28.4 SEMANTIC ERRORS

Copyright © 2026 Mallionic Inc. All rights reserved.

28.4.1 Undeclared variable

ASH

```
@Entry
pub func main(): int64 {
    int64 total = 0;
    totl = 5;
    return total;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: undeclared variable: totl
1 error generated.
```

Cause. A name used without declaration, often a typing mistake. **Fix.** Declare the variable or correct the name.

28.4.2 Global without visibility

ASH

```
uint64 total = 0

@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS

```
<input>:1:1: error: variable 'total' declared outside a function: at top level only general,
↪  ungeneral, extern, import, define, func, class and annotations are allowed
hint: a variable declared outside a function must say who can see it. Write 'general uint64
↪ total = ...;' (public: visible to C and to other files under the name 'total') or 'ungener
↪ al uint64 total = ...;' (private to this script). Add 'static' when the value must change
↪ at run time ('general static uint64 total = ...;'); without 'static' the variable is a con
↪ stant.
    1 | uint64
      | ^^^^^^
1 error generated.
```

Cause. A global must say who can see it. **Fix.** Write `general` or `ungeneral`, plus `static` if it changes (Section 9.4).

28.4.3 Assignment to a constant global

ASH

```
general uint32 LIMIT = 1000

@Entry
pub func main(): int64 {
    LIMIT = 5;
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: the global 'LIMIT' is not static (it is a constant): declare it 'static uint32 LIMIT'
↪  to be able to change it
1 error generated.
```

Cause. Globals without `static` are constants. **Fix.** Declare it `general static uint32 LIMIT = 1000`.

28.4.4 Literal out of range

ASH

```ash
@Entry
pub func main(): int64 {
    uint8 small = 300;
    return small;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: the value 300 does not fit in 8 bits (allowed range: -128..255)
1 error generated.
```

Cause. The literal does not fit in the type. **Fix.** Use a wider type, or a value in range.

28.4.5 Private method

ASH

```ash
class Vault {
    priv func secret(): int64 { return 7; }
}

@Entry
pub func main(): int64 {
    return Vault.secret();
}
```

COMPILER DIAGNOSTICS

```
error: 'Vault.secret' is private: only methods of class Vault can call it (called from 'main
↪ '). Make it 'pub' to expose it.
1 error generated.
```

Cause. A `priv` method called from outside its class. **Fix.** Make it `pub`, or call it through a public method of the class (Section 15.4).

28.4.6 Too many variadic arguments

ASH

```ash
func sum(int64 base, ...): int64 {
    return base;
}

@Entry
pub func main(): int64 {
    return sum(0, 1, 2, 3, 4, 5, 6, 7, 8, 9);
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

COMPILER DIAGNOSTICS

```
error: 'sum' admite como maximo 8 arguments variables, recibidos 9
1 error generated.
```

Cause. ASH variadic functions accept at most eight extra arguments. **Fix.** Pass an array instead.

28.4.7 Comparison used as a value

ASH

```ash
@Entry
pub func main(): int64 {
    int64 a = 3;
    int64 b = 5;
    int64 less = a < b;
    return less;
}
```

COMPILER DIAGNOSTICS

```
error: operator '<' is not supported
1 error generated.
```

Cause. Comparisons belong to conditions (Section 10.9). **Fix.** `int64 less = 0; if (a < b) { less = 1; }`.

28.4.8 Constant expression too large

ASH

```ash
@Entry
pub func main(): int64 {
    int64 x = 2 ** 64;
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
<input>:3:22: error: the result of '**' does not fit in 64 bits: 18446744073709551616
    3 | ;
      |                      ^
1 error generated.
```

Cause. A constant power whose value needs more than 64 bits. **Fix.** Reduce the exponent.

28.4.9 Division by a constant zero

ASH

```ash
@Entry
pub func main(): int64 {
    int64 x = 7 / 0;
    return x;
}
```

COMPILER DIAGNOSTICS

```
error: division by zero
1 error generated.
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Cause. Division by the literal 0. **Fix.** Correct the divisor; division by a variable that is zero at run time raises `SIGFPE` instead.

28.4.10 Matrix rows of different length

```ash
@Entry
pub func main(): int64 {
    matrix m[] []{
        [int32 1], [int32 2],
        [int32 3],
    }
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
<input>:7:5: error: matrix 'm': row 2 (line 5) has 1 elements and the first one has 2
    7 | return
      |     ^^^^^^
1 error generated.
```

Cause. Every row of a `matrix` literal needs the same number of elements. **Fix.** Complete the short row.

28.5 MODULE ERRORS

28.5.1 Unknown library

```ash
import <graphics>;

@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
error: library <graphics> was found neither in nlib (nlib) nor embedded. Available: [args.as
↪ h, delay.ash, math.ash, null.ash, stdbool.ash, stdio.ash, stdlib.ash, string.ash, syscalls
↪ .ash, types.ash]
1 error generated.
```

Cause. No `graphics.ash` in an `nlib` directory or in the embedded libraries. **Fix.** Run `ashc --list-nlib` to see the available names, or use `import "file.ash";` for project files.

28.5.2 Ambiguous definitions

Two imported files that define the same function with the same right produce an error (Section 19.4); give one of them a higher right with `@ovr[N]` or `name.override = N;`.

Copyright © 2026 Mallionic Inc. All rights reserved.

28.6 DIRECTIVE ERRORS

ASH

```
%if ASH_BITS == 64
@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS

```
error: line 1: %if without matching %endif
1 error generated.
```

Cause. An `%if` without `%endif`. **Fix.** Close every conditional block.

ASH

```
%iff ASH_BITS == 64
%endif
@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS

```
error: line 1: unknown directive '%iff'
1 error generated.
```

Cause. An unknown directive. **Fix.** See Table 18-2.

28.7 STRICT MODE ERRORS

The five errors of strict mode are shown with their causes in Section 21.3.

28.8 COMMAND LINE ERRORS

SHELL SESSION

```
$ ashc missing.ash --elf
error: The file doesn't exit or is not a file: missing.ash
1 error generated.
$ ashc hello.ash --elf --win
error: --win and --elf are mutually exclusive
1 error generated.
$ ashc hello.ash --bits 48
error: unsupported --bits 48 (use 16, 32 or 64)
1 error generated.
```

28.9 ERRORS REPORTED BY THE LINKER

Some problems are found only when the object file is linked. They are reported by `ld` or `gcc`, not by `ashc`:

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 28-1. Common link errors

Message	Cause and fix
`undefined reference to 'main'`	The entry function is called `_start` but the program was linked with the C start-up files. Rename it `main`, or rename the symbol with `objcopy --redefine-sym _start=main` (Section 3.5).
`multiple definition of '_start'`	The same situation seen from the other side: both the C start-up files and the object define `_start`.
`undefined reference to 'f'`	A function that is called but not defined: a missing `extern` library (`-lasound`, `-lm` ...), or a C function used with `--no-libc`.
`undefined reference to 'exit'` with `--no-libc`	A program without arrays calls `exit`; use `sys_exit` (Section 22.2.2).
`undefined reference to '__stlib_malloc'`	`<stdlib>` was imported (Section 27.8).
`warning: ... has a LOAD segment with RWX permissions`	Harmless for ASH programs; silence it with `-Wl,--no-warn-rwx-segments`.
`relocation R_X86_64_32 against ... can not be used when making a PIE object`	Not produced by ASH objects, which use PC-relative relocations; it can appear when mixing with C objects compiled without `-fPIC`. Link with `-no-pie`.

28.10 ERRORS AT RUN TIME

Table 28-2. Run-time failures

Symptom	Cause
Exit status 134 without a message	An array index out of range, stopped by `__ash_bounds` (Section 13.3.1).
`Segmentation fault`	Writing to a string literal, dereferencing an invalid pointer, a `_start` function that returns, an `asm` block that corrupts `rsp` or a callee-saved register.
`Floating point exception`	Division or remainder by zero (the processor's divide error).
The program loops forever	A `continue` inside a `switch` inside a loop (Section 11.6).
Output lost before a crash	`printf` output is buffered; call `fflush(0)` before code that may crash.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 29

TOOLING

This chapter describes the tools that accompany the compiler: the Visual Studio Code extension, the build scripts, the test suite with its emulators, and the utilities used to embed the system libraries.

29.1 THE VS CODE EXTENSION

The directory `vscode-ash/` contains the extension *ASH Language* (version 0.5.0) for Visual Studio Code 1.64 or later, VSCodium and VS Code Insiders. It provides:

- syntax highlighting for `.ash` and `.ashir` files, with a scope for every family of tokens (annotations, control keywords, types by signedness, cracks, nlib and libc functions, numbers with prefixes and suffixes, escapes and format specifiers, assembly mnemonics and registers, directives);
- two color themes, *ASH Dark* and *ASH Light*, in which every family has its own color;
- pixel-art file icons for `.ash` and `.ashir` files;
- 39 snippets;
- a linter that checks the file while you type, and runs the real compiler when you save.

29.1.1 Installation

```shell
cd vscode-ash
./install_extension.sh            # Linux and macOS
./install_extension.sh --remove   # uninstall
install_extension.bat             # Windows
```

The script installs the extension in every VS Code variant it finds. After restarting the editor, choose a theme with **Ctrl+K Ctrl+T**.

29.1.2 The linter

The linter (`vscode-ash/src/linter.js`) runs inside the editor and needs no compiler. It reads the files you import (`import "x.ash"`, `import <string>`) to know their functions. Problems are underlined in red (errors) or yellow (warnings), and every message explains in English what is wrong and how to fix it.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 29-1. Checks performed by the linter

Check	Examples of what it reports
Brackets	Unclosed or crossed `{`, `(` and `[`.
Literals and comments	Unterminated strings, comments and character literals.
Top-level items	Statements outside a function; a global without `general` / `ungeneral`.
Constants	Assignment to a global without `static`, or to a `define`.
Returns	A value returned from a `void` function, `return;` in a function with a result, a missing return, text returned from an integer function.
Types	Text assigned to an integer, literals out of range, unknown types with a suggestion (`uint46` -> `uint16`).
Typing mistakes	Unknown words with suggestions: `retrun`, `fucn`, `prinft`, `@Entyr`, `<strng>`, misspelled variables.
Declarations	Undefined variables, variables declared twice.
Calls	Wrong number of arguments, more `%d` than values in `printf`, private methods called from outside, unknown cracks.
`@naked`	Variables, calls, `return` or `{x}` placeholders inside a naked function.
Control flow	`break` outside a loop, `else` without `if`, `case` outside `switch`, `va_arg` outside a variadic function, code after `return`, `if (x = 3)`.

On save, the extension also runs `ashc file --elf` (or `--compile` for `@bits(16)` files) with a temporary output file and adds the compiler's errors with the prefix `ashc:`. The command **ASH: Check file (linter + compiler)** (Ctrl+Shift+P) runs both checks on demand.

Table 29-2. Settings of the extension

Setting	Type	Default	Meaning
`ash.lint.enable`	boolean	true	Check `.ash` files while typing.
`ash.lint.delay`	number	250	Milliseconds to wait after typing before checking.
`ash.lint.showExternalHints`	boolean	true	Show a hint when an undeclared function will be linked as an external C function.
`ash.compiler.checkOnSave`	boolean	true	Run `ashc` when the file is saved.
`ash.compiler.path`	string	`ashc`	Path of the compiler used on save.
`ash.nlibPath`	string	(empty)	Folder of the nlib libraries; empty means `<workspace>/nlib`.

29.1.3 Snippets

Type the prefix and press Tab.

Copyright © 2026 Mallionic Inc. All rights reserved.

Table 29-3. Snippets of the extension

Prefix	Inserts
`main`	An `@Entry` function.
`mainargs`	An entry function with `argc` / `argv` and a loop over the arguments (`import <args>`).
`func`, `pubfunc`	A function; a public function.
`funcvar`	A variadic function with `va_count()` / `va_arg(i)`.
`class`, `struct`	A class with a public method; a structure.
`crack`	A crack inside a function.
`if`, `ifelse`, `ifelif`	Conditional statements.
`for`, `forarr`, `while`, `repeat`, `loop`	Loops; `forarr` iterates over an array with `#`.
`switch`	`switch` with `case` and `default`.
`terminate`, `terminatefor`	`terminate` statements; a search loop that leaves the function when it finds a value.
`printf`	A `printf` call.
`global`, `garray`	A writable global visible to C; a global array with data.
`array`, `mlstr`	A local array; a triple-quoted string.
`asm`, `asmv`, `asmop`	An `asm` block; `asm volatile("...")` with placeholders; the GCC operand form.
`emit`, `runnable`	Raw bytes; a runnable array.
`@section`, `@beautycode`, `@bits`, `@naked`	Annotations.
`import`, `extern`, `define`, `%if`	Imports, external declarations, constants, directives.

29.1.4 Changing colors

The theme colors are generated by `vscode-ash/themes/gen_themes.py` (one line per rule, with a dark and a light color); edit it and run `python3 themes/gen_themes.py`. Without touching the extension, a rule can be overridden in `settings.json`:

```
"editor.tokenColorCustomizations": {
  "textMateRules": [
    { "scope": "storage.type.annotation.entry.ash",
      "settings": { "foreground": "#00FF88", "fontStyle": "bold" } }
  ]
}
```

The command *Developer: Inspect Editor Tokens and Scopes* shows the scope of any word. The icons are drawn pixel by pixel in `vscode-ash/icons/make_icons.py`.

Copyright © 2026 Mallionic Inc. All rights reserved.

29.2 BUILD SCRIPTS

`build.sh` and `build.bat` build the compiler; their commands are listed in Table 3-3. Before compiling, both run `java tools/EmbedNlib.java nlib Builder/ash/NlibEmbedded.java`, which turns every file of `nlib/` into a Java string constant, so that the native executable needs no resource files and no reflection.

29.3 THE TEST SUITE

The directory `tests/` contains small programs, one per feature, and helper scripts:

Table 29-4. Contents of tests/

Path	Purpose
`tests/*.ash`	Programs for individual features (arrays, cracks, globals, shifts, strict mode, variadics ...). `tests/run.sh file.ash` compiles, links and runs one of them.
`tests/strict_bad_*.ash`	Programs that must fail in strict mode with a `strict:` error; `strict_ok_use.ash` must compile.
`tests/class_priv_error.ash`, `tests/global_without_general_error.ash`	Programs that must be rejected.
`tests/bits16/`	16-bit programs and `emu16.py`, an 8086 emulator built on the Unicorn engine: it loads a flat image at the origin, calls it and prints `AX`.
`tests/bits32/`	32-bit flat programs, `emu32.py` (prints `EAX`) and `emu_vga.py` (also prints the VGA text buffer at `0xB8000`).
`tests/asm/`	Assembly in 32 and 64-bit programs (Linux system calls).
`tests/objects/`	Object files linked with C: ASH called from C, C called from ASH, programs without libc.

The emulators need the Python package `unicorn` (`pip install unicorn`):

```shell
ashc tests/bits16/fib_loops_arrays.ash --compile
python3 tests/bits16/emu16.py tests/bits16/out/fib_loops_arrays.bin 0x100
```

29.4 BUILDING THIS MANUAL

This manual is generated from the sources in `docs/ash-guide-src/` by `build_guide.py`, which compiles and runs every example with the real compiler. To rebuild it:

```shell
pip install reportlab pygments unicorn       # once
python3 docs/ash-guide-src/build_guide.py    # writes docs/ash-guide.pdf
python3 docs/ash-guide-src/build_guide.py --only 25   # preview one chapter
python3 docs/ash-guide-src/set_logo.py my_logo.png    # replace the logo and rebuild
```

Results of compilations are cached in `docs/ash-guide-src/.cache/`, keyed by the source of each example and the compiler binary, so a rebuild after editing text takes seconds; a new compiler binary re-runs every example.

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 30

COOKBOOK: COMPLETE PROGRAMS

This chapter collects complete programs that combine the features described in this manual. Each program solves a small, realistic task, is explained step by step, and was compiled, linked and executed while the manual was built; the output shown is the real output. Use them as starting points for your own programs.

30.1 PRIME NUMBERS WITH THE SIEVE OF ERATOSTHENES

The sieve marks the multiples of every prime in an array of flags. It shows array declaration with a computed size, nested loops with a variable step, and `continue`.

Example 30-1. Sieve of Eratosthenes

ASH

```
func sieve(int64 limit): int64 {
    uint8[limit + 1] composite;           // all zero: every number is a candidate
    int64 count = 0;
    for (int64 n = 2; n <= limit; n++) {
        if (composite[n] != 0) {
            continue;
        }
        count++;
        if (count <= 20) {
            printf("%d ", n);
        }
        for (int64 m = n * n; m <= limit; m = m + n) {
            composite[m] = 1;
        }
    }
    printf("\n");
    return count;
}

@Entry
pub func main(): int64 {
    int64 total = sieve(100000);
    printf("there are %d primes below 100000\n", total);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71
there are 9592 primes below 100000
```

Copyright © 2026 Mallionic Inc. All rights reserved.

30.2 SORTING: INSERTION SORT AND QSORT

The same data is sorted twice: with an insertion sort written in ASH and with the C library's `qsort`, which receives an ASH comparison function as a callback.

Example 30-2. Two ways to sort an array

ASH

```
extern void qsort(int64 base, int64 n, int64 size, int64 cmp);

func insertion_sort(int64* v, int64 n): void {
    for (int64 i = 1; i < n; i++) {
        int64 key = v[i];
        int64 j = i - 1;
        while (j >= 0 && v[j] > key) {
            v[j + 1] = v[j];
            j--;
        }
        v[j + 1] = key;
    }
}

func by_value(int64 a, int64 b): int64 {
    int64 x = *(int64*)a;
    int64 y = *(int64*)b;
    if (x < y) { return -1; }
    if (x > y) { return 1; }
    return 0;
}

func show(string label, int64* v, int64 n): void {
    printf("%s:", label);
    for (int64 i = 0; i < n; i++) {
        printf(" %d", v[i]);
    }
    printf("\n");
}

@Entry
pub func main(): int64 {
    int64[] a = {42, 7, 19, 3, 88, 1, 56, 23};
    int64[] b = {42, 7, 19, 3, 88, 1, 56, 23};
    insertion_sort(a, #a);
    qsort(b, #b, 8, by_value);
    show("insertion", a, #a);
    show("qsort    ", b, #b);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
insertion: 1 3 7 19 23 42 56 88
qsort    : 1 3 7 19 23 42 56 88
```

Copyright © 2026 Mallionic Inc. All rights reserved.

30.3 A LINKED LIST WITH STRUCTURES

A singly linked list stores its nodes in memory obtained from `malloc`. The program shows structures with pointer fields, pointer chains and a function that walks the list.

Example 30-3. A singly linked list

ASH

```
extern void* malloc(uint64 size);
extern void free(void* p);

typedef struct {
    int64 value;
    uint64 next;          // address of the next Node, or 0
} Node;

func push(uint64 head, int64 value): uint64 {
    Node* n = malloc(16);
    n->value = value;
    n->next = head;
    return n;
}

func sum(uint64 head): int64 {
    int64 total = 0;
    uint64 cur = head;
    while (cur != 0) {
        Node* n = cur;
        total = total + n->value;
        cur = n->next;
    }
    return total;
}

func print_list(uint64 head): void {
    uint64 cur = head;
    while (cur != 0) {
        Node* n = cur;
        printf("%d -> ", n->value);
        cur = n->next;
    }
    printf("end\n");
}

@Entry
pub func main(): int64 {
    uint64 list = 0;
    for (int64 i = 1; i <= 5; i++) {
        list = push(list, i * 10);
    }
    print_list(list);
    printf("sum=%d\n", sum(list));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
50 -> 40 -> 30 -> 20 -> 10 -> end
sum=150
```

Copyright © 2026 Mallionic Inc. All rights reserved.

30.4 A STACK CALCULATOR IN REVERSE POLISH NOTATION

A tiny RPN calculator evaluates an expression given as a string. It uses a fixed-size array as a stack, `<string>` character tests, and `switch` on characters.

Example 30-4. Reverse Polish calculator

ASH

```
import <string>;

func rpn(uint8* text): int64 {
    int64[32] stack;
    int64 sp = 0;
    int64 i = 0;
    while (text[i] != 0) {
        int64 c = text[i];
        if (isdigit(c) == 1) {
            int64 value = 0;
            while (isdigit(text[i]) == 1) {
                value = value * 10 + text[i] - '0';
                i++;
            }
            stack[sp] = value;
            sp++;
        } else {
            if (c != ' ') {
                int64 b = stack[sp - 1];
                int64 a = stack[sp - 2];
                sp = sp - 2;
                int64 r = 0;
                switch (c) {
                    case '+': r = a + b; break;
                    case '-': r = a - b; break;
                    case '*': r = a * b; break;
                    case '/': r = a / b; break;
                }
                stack[sp] = r;
                sp++;
            }
            i++;
        }
    }
    return stack[0];
}

@Entry
pub func main(): int64 {
    printf("3 4 + 2 *       = %d\n", rpn("3 4 + 2 *"));
    printf("100 7 / 5 -     = %d\n", rpn("100 7 / 5 -"));
    printf("2 3 4 * + 10 -  = %d\n", rpn("2 3 4 * + 10 -"));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3 4 + 2 *       = 14
100 7 / 5 -     = 9
2 3 4 * + 10 -  = 4
```

Copyright © 2026 Mallionic Inc. All rights reserved.

30.5 COUNTING LINES, WORDS AND CHARACTERS

A `wc`-like counter that reads its own source text from a global `char[]` array. It uses a state variable to detect the beginning of words.

Example 30-5. A word counter

ASH

```
import <string>;

general char[] text = """The Ashley Project
makes systems programming
close to the machine
and easy to read.""";

@Entry
pub func main(): int64 {
    int64 lines = 1;
    int64 words = 0;
    int64 in_word = 0;
    for (int64 i = 0; i < #text; i++) {
        int64 c = text[i];
        if (c == '\n') { lines++; }
        if (isspace(c) == 1) {
            in_word = 0;
        } elif (in_word == 0) {
            in_word = 1;
            words++;
        }
    }
    printf("lines=%d words=%d chars=%d\n", lines, words, #text);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
lines=4 words=14 chars=84
```

30.6 NUMBER BASE CONVERSION

Converting numbers between bases with `<string>`: `str_to_int` parses text in any base, `uint_to_str` formats it.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 30-6. Printing numbers in several bases

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[65] buf;
    string[] inputs = {"255", "0x1F", "0b1010", "777"};
    for (int64 i = 0; i < #inputs; i++) {
        int64 v = str_to_int(inputs[i], 0);
        printf("%-7s = %4d", inputs[i], v);
        printf("  bin %s", uint_to_str(v, &buf, 2));
        printf("  oct %s", uint_to_str(v, &buf, 8));
        printf("  hex %s", uint_to_str(v, &buf, 16));
        printf("  b36 %s\n", uint_to_str(v, &buf, 36));
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
255     =  255  bin 11111111  oct 377  hex ff  b36 73
0x1F    =   31  bin 11111  oct 37  hex 1f  b36 v
0b1010  =   10  bin 1010  oct 12  hex a  b36 a
777     =  777  bin 1100001001  oct 1411  hex 309  b36 ll
```

30.7 HASHING WITH FNV-1A

FNV-1a is a simple, fast hash for strings. The program uses 64-bit unsigned arithmetic, ^ and multiplication modulo 2^64.

Example 30-7. FNV-1a 64-bit hash

ASH

```
func fnv1a(uint8* s): uint64 {
    uint64 h = 0xcbf29ce484222325;          // offset basis
    uint64 prime = 0x100000001b3;           // FNV prime
    int64 i = 0;
    while (s[i] != 0) {
        uint64 c = s[i];
        h = h ^ c;
        h = h * prime;
        i++;
    }
    return h;
}

@Entry
pub func main(): int64 {
    string[] keys = {"", "a", "hello", "The Ashley Project"};
    for (int64 i = 0; i < #keys; i++) {
        printf("fnv1a(\"%s\") = %016llx\n", keys[i], fnv1a(keys[i]));
    }
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
fnv1a("") = cbf29ce484222325
fnv1a("a") = af63dc4c8601ec8c
fnv1a("hello") = a430d84680aabd0b
fnv1a("The Ashley Project") = 1d4beabb7545dbff
```

30.8 MATRIX MULTIPLICATION

Two-dimensional arrays, triple loops and a helper that prints a matrix.

Example 30-8. Multiplying two 3x3 matrices

ASH

```
func fill(int64 base, int64 step): int64 {
    return base + step;
}

@Entry
pub func main(): int64 {
    int64[3][3] a;
    int64[3][3] b;
    int64[3][3] c;
    for (int64 i = 0; i < 3; i++) {
        for (int64 j = 0; j < 3; j++) {
            a[i][j] = i * 3 + j + 1;          // 1..9
            if (i == j) { b[i][j] = 2; }      // 2 * identity
        }
    }
    b[0][2] = 1;
    for (int64 i = 0; i < 3; i++) {
        for (int64 j = 0; j < 3; j++) {
            int64 s = 0;
            for (int64 k = 0; k < 3; k++) {
                s = s + a[i][k] * b[k][j];
            }
            c[i][j] = s;
        }
    }
    for (int64 i = 0; i < 3; i++) {
        printf("%4d %4d %4d\n", c[i][0], c[i][1], c[i][2]);
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
   2    4    7
   8   10   16
  14   16   25
```

30.9 ASCII MANDELBROT SET IN FIXED POINT

The Mandelbrot set drawn with characters, computed entirely with Q16.16 fixed-point arithmetic from `<math>`: no floating-point instruction is used, so the same code runs on bare metal.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 30-9. Fixed-point Mandelbrot

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    string shades = " .:-=+*#%@";
    for (int64 row = 0; row < 22; row++) {
        for (int64 col = 0; col < 64; col++) {
            int64 cx = fx(col - 44) / 20;          // real part, about -2.2 .. 1.0
            int64 cy = fx(row - 11) / 10;          // imaginary part, about -1.1 .. 1.1
            int64 x = 0;
            int64 y = 0;
            int64 n = 0;
            while (n < 9) {
                int64 x2 = fx_mul(x, x);
                int64 y2 = fx_mul(y, y);
                if (x2 + y2 > fx(4)) { break; }
                int64 t = x2 - y2 + cx;
                y = 2 * fx_mul(x, y) + cy;
                x = t;
                n++;
            }
            printf("%c", shades[n]);
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
...........:::::::----------------=====+@%++===----::::::::::::::
..........:::::----------------======++*#@@@@+====----::::::::::::
.........:::----------------=======+++*%@@@@%*++====-----:::::::::
.........::----------------=======++**#%@@@@@@%*++++==------:::::
.......::--------------======++*#%%%%@@@@@@@@@@%***@+==-----::::
.......::------------====++++**#@@@@@@@@@@@@@@@@@@@@@@+=------:::
......:----------==+++++++***%@@@@@@@@@@@@@@@@@@@@@@@#*+=------::
.....:-------===++#@#########%@@@@@@@@@@@@@@@@@@@@@@@@@@+==------:
.....---======+++*#@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@%*==-------
.....-======++++*#%@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@#+===------
.....=====****#%@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@%*+===------
....@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@%#*+===------
.....=====****#%@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@%*+===------
.....-======++++*#%@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@#+===------
.....---======+++*#@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@%*==-------
.....:-------===++#@#########%@@@@@@@@@@@@@@@@@@@@@@@@@@+==------:
......:----------==+++++++***%@@@@@@@@@@@@@@@@@@@@@@@#*+=------::
......::----------====++++**#@@@@@@@@@@@@@@@@@@@@@@+=------:::
.......::--------------======++*#%%%%@@@@@@@@@@%***@+==-----::::
.........::----------------=======++**#%@@@@@@%*++++==------:::::
..........:::----------------=======+++*%@@@@@%*++====-----:::::::
...........:::::----------------======++*#@@@@+====----::::::::::
```

Copyright © 2026 Mallionic Inc. All rights reserved.

30.10 CONWAY'S GAME OF LIFE

A cellular automaton on a small toroidal grid, printed for three generations. It uses two-dimensional arrays, modular arithmetic for wrap-around and a function that counts neighbours.

Example 30-10. Game of Life: a glider

ASH

```
define W 12;
define H 8;

func neighbours(int64* g, int64 r, int64 c): int64 {
    int64 n = 0;
    for (int64 dr = -1; dr <= 1; dr++) {
        for (int64 dc = -1; dc <= 1; dc++) {
            if (dr != 0 || dc != 0) {
                int64 rr = (r + dr + H) % H;
                int64 cc = (c + dc + W) % W;
                n = n + g[rr * W + cc];
            }
        }
    }
    return n;
}

func show(int64* g, int64 gen): void {
    printf("generation %d\n", gen);
    for (int64 r = 0; r < H; r++) {
        for (int64 c = 0; c < W; c++) {
            if (g[r * W + c] == 1) { printf("#"); } else { printf("."); }
        }
        printf("\n");
    }
}

@Entry
pub func main(): int64 {
    int64[W * H] grid;
    int64[W * H] next;
    grid[0 * W + 1] = 1;                // a glider
    grid[1 * W + 2] = 1;
    grid[2 * W + 0] = 1;
    grid[2 * W + 1] = 1;
    grid[2 * W + 2] = 1;
    for (int64 gen = 0; gen < 3; gen++) {
        show(grid, gen);
        for (int64 r = 0; r < H; r++) {
            for (int64 c = 0; c < W; c++) {
                int64 n = neighbours(grid, r, c);
                int64 alive = grid[r * W + c];
                int64 v = 0;
                if (n == 3) { v = 1; }
                if (n == 2 && alive == 1) { v = 1; }
                next[r * W + c] = v;
            }
        }
        for (int64 k = 0; k < W * H; k++) {
            grid[k] = next[k];
        }
    }
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
generation 0
.#..........
..#.........
###.........
............
............
............
............
............
generation 1
............
#.#.........
.##.........
.#..........
............
............
............
............
generation 2
............
..#.........
#.#.........
.##.........
............
............
............
............
```

30.11 A HEX DUMP TOOL

A program that reads a file with the C standard I/O functions and prints it in the classic hex dump format: offset, sixteen bytes in hexadecimal and the printable characters.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 30-11. Hex dump of a file

ASH

```
import <stdio>;

@Entry
pub func main(): int64 {
    void* w = fopen("/tmp/ash_hexdump_demo.bin", "wb");
    fputs("The Ashley Project\n\x01\x02\x7F binary bytes too\n", w);
    fclose(w);

    void* f = fopen("/tmp/ash_hexdump_demo.bin", "rb");
    if (f == 0) {
        puts("cannot open file");
        return 1;
    }
    uint8[16] row;
    int64 offset = 0;
    loop {
        int64 n = fread(&row, 1, 16, f);
        if (n == 0) { break; }
        printf("%08x  ", offset);
        for (int64 i = 0; i < 16; i++) {
            if (i < n) { printf("%02x ", row[i]); } else { printf("   "); }
        }
        printf(" |");
        for (int64 i = 0; i < n; i++) {
            int64 c = row[i];
            if (c >= 32 && c < 127) { printf("%c", c); } else { printf("."); }
        }
        printf("|\n");
        offset = offset + n;
    }
    fclose(f);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
00000000  54 68 65 20 41 73 68 6c 65 79 20 50 72 6f 6a 65  |The Ashley Proje|
00000010  63 74 0a 01 02 7f 20 62 69 6e 61 72 79 20 62 79  |ct.... binary by|
00000020  74 65 73 20 74 6f 6f 0a                          |tes too.|
```

30.12 WRITING A WAV FILE: A BYTEBEAT GENERATOR

Bytebeat is music produced by a one-line formula of a time counter `t`. The program writes one second of 8 kHz, 8-bit sound to a WAV file. It shows writing binary data with `fwrite`, building a file header field by field in a byte buffer, and shifts and bitwise operators.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 30-12. Bytebeat to a WAV file

ASH

```
import <stdio>;

func put32(uint8* p, int64 at, int64 v): void {
    p[at] = v & 255;
    p[at + 1] = (v >> 8) & 255;
    p[at + 2] = (v >> 16) & 255;
    p[at + 3] = (v >> 24) & 255;
}

func put16(uint8* p, int64 at, int64 v): void {
    p[at] = v & 255;
    p[at + 1] = (v >> 8) & 255;
}

@Entry
pub func main(): int64 {
    int64 rate = 8000;
    int64 samples = 8000;
    uint8[44] h;
    memcpy(&h, "RIFF", 4);
    put32(&h, 4, 36 + samples);
    memcpy(&h + 8, "WAVEfmt ", 8);
    put32(&h, 16, 16);              // size of the fmt chunk
    put16(&h, 20, 1);               // PCM
    put16(&h, 22, 1);               // mono
    put32(&h, 24, rate);
    put32(&h, 28, rate);            // bytes per second
    put16(&h, 32, 1);               // block align
    put16(&h, 34, 8);               // bits per sample
    memcpy(&h + 36, "data", 4);
    put32(&h, 40, samples);

    uint8[samples] pcm;
    for (int64 t = 0; t < samples; t++) {
        pcm[t] = (t * ((t >> 12 | t >> 8) & 63 & t >> 4)) & 255;
    }
    void* f = fopen("/tmp/ash_bytebeat.wav", "wb");
    fwrite(&h, 1, 44, f);
    fwrite(&pcm, 1, samples, f);
    fclose(f);
    printf("wrote /tmp/ash_bytebeat.wav: %d bytes of header, %d samples\n", #h, samples);
    printf("first samples: %d %d %d %d\n", pcm[4097], pcm[4098], pcm[5000], pcm[7000]);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
wrote /tmp/ash_bytebeat.wav: 44 bytes of header, 8000 samples
first samples: 0 0 128 216
```

NOTE

`memcpy` is called here without importing `<string>`: the C library's `memcpy` is used as an implicit external function. Importing `<string>` would use the ASH implementation instead; both work.

Copyright © 2026 Mallionic Inc. All rights reserved.

30.13 CAT WITHOUT THE C LIBRARY

A version of `cat` that uses only system calls. It is built with `--no-libc` and linked with `ld`, producing a static executable with no dependencies. It reads from standard input and copies to standard output in blocks.

Example 30-13. cat with system calls only

ASH

```
import <syscalls>;

@Entry
pub func _start(): int64 {
    uint8[4096] buf;
    int64 total = 0;
    loop {
        int64 n = sys_read(STDIN, &buf, 4096);
        if (n <= 0) { break; }
        sys_write(STDOUT, &buf, n);
        total = total + n;
    }
    print("-- copied ");
    print_int(total);
    println(" bytes");
    sys_exit(0);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
first line
second line
-- copied 23 bytes
```

30.14 A COMMAND LINE TOOL WITH OPTIONS

A program that parses options in the usual style: flags (`-v`), an option with a value (`-n 3`) and positional arguments. It uses `<args>` and `<string>`.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 30-14. Parsing command line options

ASH

```
import <args>;
import <string>;

@Entry
pub func main(int64 argc, int64 argv): int64 {
    int64 verbose = 0;
    int64 repeat_count = 1;
    int64 first_positional = 0;
    int64 i = 1;
    while (i < argc) {
        uint8* a = arg_at(argv, i);
        if (str_equals(a, "-v") == 1) {
            verbose = 1;
        } elif (str_equals(a, "-n") == 1) {
            i++;
            repeat_count = atoi(arg_at(argv, i));
        } else {
            if (first_positional == 0) { first_positional = i; }
        }
        i++;
    }
    if (verbose == 1) {
        printf("verbose mode, repeat=%d, first positional at %d\n", repeat_count, first_posi
↪ tional);
    }
    for (int64 k = first_positional; k < argc; k++) {
        repeat (repeat_count) {
            printf("%s ", arg_at(argv, k));
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
verbose mode, repeat=3, first positional at 4
alpha alpha alpha
beta beta beta
```

30.15 GENERATING MACHINE CODE AT RUN TIME

A `runnable` array receives machine code produced by the program itself: a function that computes `x * k + c` for constants chosen at run time. The example combines code generation, little-endian encoding of immediates and calling the generated code.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 30-15. A tiny JIT compiler

ASH

```
runnable byte[] code = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
↪  0}

general static int64 pos = 0

func b(int64 v): void {
    code[pos] = v & 255;
    pos = pos + 1;
}

func imm32(int64 v): void {
    b(v);
    b(v >> 8);
    b(v >> 16);
    b(v >> 24);
}

// generate: mov rax, rdi ; imul rax, rax, k ; add rax, c ; ret
func compile(int64 k, int64 c): void {
    pos = 0;
    b(0x48); b(0x89); b(0xF8);                  // mov rax, rdi
    b(0x48); b(0x69); b(0xC0); imm32(k);        // imul rax, rax, imm32
    b(0x48); b(0x05); imm32(c);                 // add rax, imm32
    b(0xC3);                                    // ret
}

@Entry
pub func main(): int64 {
    compile(3, 1);
    printf("3x+1 at x=5: %d\n", code(5));
    compile(10, -7);
    printf("10x-7 at x=5: %d\n", code(5));
    printf("generated %d bytes\n", pos);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3x+1 at x=5: 16
10x-7 at x=5: 43
generated 17 bytes
```

30.16 CALLBACKS AND FUNCTION TABLES

A table of operations stored as function addresses in an array, selected at run time and called through a variable. This is how interpreters and state machines dispatch on an opcode.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 30-16. A dispatch table of functions

ASH

```
func op_add(int64 a, int64 b): int64 { return a + b; }
func op_sub(int64 a, int64 b): int64 { return a - b; }
func op_mul(int64 a, int64 b): int64 { return a * b; }
func op_max(int64 a, int64 b): int64 { if (a > b) { return a; } return b; }

@Entry
pub func main(): int64 {
    int64[4] table;
    table[0] = &op_add;
    table[1] = &op_sub;
    table[2] = &op_mul;
    table[3] = &op_max;
    string[] names = {"add", "sub", "mul", "max"};
    for (int64 i = 0; i < 4; i++) {
        int64 f = table[i];
        printf("%s(12, 5) = %d\n", names[i], f(12, 5));
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
add(12, 5) = 17
sub(12, 5) = 7
mul(12, 5) = 60
max(12, 5) = 12
```

30.17 STATISTICS WITH VARIADIC FUNCTIONS

Variadic ASH functions that compute the maximum, minimum and average of their arguments, used on the lengths of Collatz sequences.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 30-17. Collatz lengths and variadic statistics

ASH

```
func collatz(int64 n): int64 {
    int64 steps = 0;
    while (n != 1) {
        if ((n & 1) == 0) { n = n / 2; } else { n = 3 * n + 1; }
        steps++;
    }
    return steps;
}

func maximum(int64 first, ...): int64 {
    int64 m = first;
    for (int64 i = 0; i < va_count(); i++) {
        if (va_arg(i) > m) { m = va_arg(i); }
    }
    return m;
}

func average(int64 first, ...): int64 {
    int64 total = first;
    for (int64 i = 0; i < va_count(); i++) {
        total = total + va_arg(i);
    }
    return total / (va_count() + 1);
}

@Entry
pub func main(): int64 {
    int64 a = collatz(7);
    int64 b = collatz(27);
    int64 c = collatz(97);
    int64 d = collatz(871);
    printf("lengths: %d %d %d %d\n", a, b, c, d);
    printf("max=%d average=%d\n", maximum(a, b, c, d), average(a, b, c, d));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
lengths: 16 111 118 178
max=178 average=105
```

30.18 A REAL-MODE ROUTINE

A 16-bit flat binary that computes a checksum of a table in real mode. It shows `@bits(16)`, `@originoffs`, arrays in a flat image and running the result in the 8086 emulator of the test suite.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 30-18. A 16-bit checksum routine

ASH

```
@bits(16)
@originoffs(0x100)

@Entry
pub func _start(): int16 {
    uint16[] table = {0x1234, 0x0F0F, 0x00FF, 0x7000};
    int16 sum = 0;
    for (int16 i = 0; i < 4; i++) {
        sum = sum ^ table[i];
    }
    return sum;               // AX = 0x1234 ^ 0x0F0F ^ 0x00FF ^ 0x7000
}
```

OUTPUT — EXIT STATUS 0

```
ax = 28100  bp = 0x0
```

30.19 A WINDOWS CONSOLE PROGRAM

A complete Windows program that writes several lines to the console with `WriteFile` and ends with `ExitProcess`. It was built with `--win`, linked with MinGW-w64 and executed under Wine.

Example 30-19. Windows console output

ASH

```
extern uint64 GetStdHandle(int32 nStdHandle);
extern int32  WriteFile(uint64 h, uint64 buf, uint32 n, uint64 written, uint64 overlapped);
extern void   ExitProcess(uint32 code);

func say(uint64 h, uint8* text): void {
    int64 n = 0;
    while (text[n] != 0) { n++; }
    uint32 written = 0;
    WriteFile(h, text, n, &written, 0);
}

@Entry
pub func main(): void {
    uint64 out = GetStdHandle(-11);
    say(out, "ASH on Windows\r\n");
    for (int64 i = 0; i < 3; i++) {
        say(out, "  a line from a loop\r\n");
    }
    ExitProcess(0);
}
```

OUTPUT — EXIT STATUS 0

```
ASH on Windows
  a line from a loop
  a line from a loop
  a line from a loop
```

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 31

BARE-METAL PROGRAMMING: THE VGA TEXT SCREEN

A program that runs *on bare metal* runs directly on the processor, with no operating system underneath: it is the first code of a kernel, a boot loader, a firmware or an embedded system. There is no `printf`, no files and no C library. To show something on the screen the program writes directly into the memory of the video card. This chapter explains how a PC screen works in text mode, builds a minimal kernel in ASH that writes to the screen at address `0xB8000`, and grows it into a small VGA driver. Every screen shown here is a real screenshot of the program running in the QEMU emulator.

31.1 THE MEMORY OF A PC

When a PC starts, the first megabyte of the address space has a fixed layout inherited from the original IBM PC. Some of these regions are not ordinary memory but windows into hardware:

Table 31-1. The first megabyte of a PC

Address	Size	Content
`0x00000`	1 KiB	Interrupt vector table (real mode).
`0x00400`	256 bytes	BIOS data area (keyboard buffer, timer ticks, screen size...).
`0x07C00`	512 bytes	Where the BIOS loads a boot sector.
`0xA0000`	64 KiB	VGA graphics memory (mode 13h: 320 x 200 pixels, one byte each).
`0xB8000`	32 KiB	VGA **text** memory: the characters you see on the screen.
`0xF0000`	64 KiB	The BIOS itself.
`0x100000`	...	Extended memory, above 1 MiB: where kernels are usually loaded.

31.2 HOW THE TEXT SCREEN WORKS

In text mode the screen has 80 columns and 25 rows. Each position, a *cell*, is stored as **two bytes** at address `0xB8000 + (row * 80 + column) * 2`:

Table 31-2. One cell of the text screen

Byte	Meaning
first	The character code, in code page 437: the ASCII characters plus line-drawing and block characters such as `0xDB` (a full block) and `0xC9` (a double-line corner).
second	The *attribute*: the foreground color in bits 0–3 and the background color in bits 4–6. Bit 7 makes the character blink (or selects a bright background).

Copyright © 2026 Mallionic Inc. All rights reserved.

Writing two bytes at the right address makes a character appear immediately: the video card reads this memory about seventy times per second.

Table 31-3. The 16 text colors

Value	Color	Value	Color	Value	Color
0	black	6	brown	12	light red
1	blue	7	light gray	13	light magenta
2	green	8	dark gray	14	yellow
3	cyan	9	light blue	15	white
4	red	10	light green	|	
5	magenta	11	light cyan	|	

An attribute is computed as `background * 16 + foreground`: `0x1F` is white (15) on blue (1), `0x4E` is yellow on red, `0x07` is the usual light gray on black.

31.3 THE SKELETON OF A KERNEL

The kernels of this chapter are started by QEMU's built-in *multiboot* loader (`qemu-system-i386 -kernel kernel.elf`), which loads them at 1 MiB in 32-bit protected mode. Two small files are shared by all of them; ch-multiboot explains them line by line.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 31-1. boot.asm: the entry point, written for shasm

BOOT.ASM

```
; boot.asm - multiboot header, stack and entry point of an ASH kernel
bits 32

MB_MAGIC equ 0x1BADB002
MB_FLAGS equ 0                    ; no special requests to the boot loader

section .multiboot
align 4
    dd MB_MAGIC
    dd MB_FLAGS
    dd -(MB_MAGIC + MB_FLAGS)     ; checksum: the three fields add up to 0

section .bss
align 16
stack_bottom:
    resb 16384                    ; 16 KiB of stack
stack_top:

section .text
global _start
extern kmain
_start:
    mov esp, stack_top            ; the loader does not give us a stack
    push ebx                      ; second argument: multiboot information
    push eax                      ; first argument: magic number 0x2BADB002
    call kmain                    ; kmain(magic, info), written in ASH
.hang:
    cli
    hlt
    jmp .hang
```

Example 31-2. linker.ld: where each part of the kernel goes

LINKER.LD

```
ENTRY(_start)
SECTIONS
{
    . = 1M;                                 /* the kernel is loaded at 1 MiB */
    .text   : { *(.multiboot) *(.text) *(.ashtext) }
    .rodata : { *(.rodata) }
    .data   : { *(.data) }
    .bss    : { *(.bss) }
}
```

The ASH part of every kernel is a function `kmain(magic, info)` marked `@Entry`, compiled with `@bits(32)` and `--no-libc`. The build always has the same steps:

SHELL

```
shasm --lang en -f elf32 -o boot.o boot.asm      # assemble the entry point
ashc kernel.ash --elf --no-libc -s -o kernel.o    # compile the ASH code
ld -m elf_i386 -T linker.ld -o kernel.elf boot.o kernel.o
qemu-system-i386 -kernel kernel.elf               # boot it
```

Copyright © 2026 Mallionic Inc. All rights reserved.

31.4 HELLO, SCREEN

The smallest useful kernel writes one line into video memory. It does not clear the screen, so the messages of the BIOS remain visible above it.

Example 31-3. hello.ash: writing characters into video memory

HELLO.ASH

```
@bits(32)

@Entry
pub func kmain(uint32 magic, uint32 info): void {
    uint8* video = 0xB8000;                  // the first cell of the screen
    string message = " Hello from an ASH kernel! ";
    int32 row = 9;                           // below the BIOS messages
    int32 i = 0;
    while (message[i] != 0) {
        int32 cell = (row * 80 + i) * 2;
        video[cell] = message[i];            // the character
        video[cell + 1] = 0x1F;              // white on blue
        i++;
    }
}
```

Table 31-4. Line by line: hello.ash

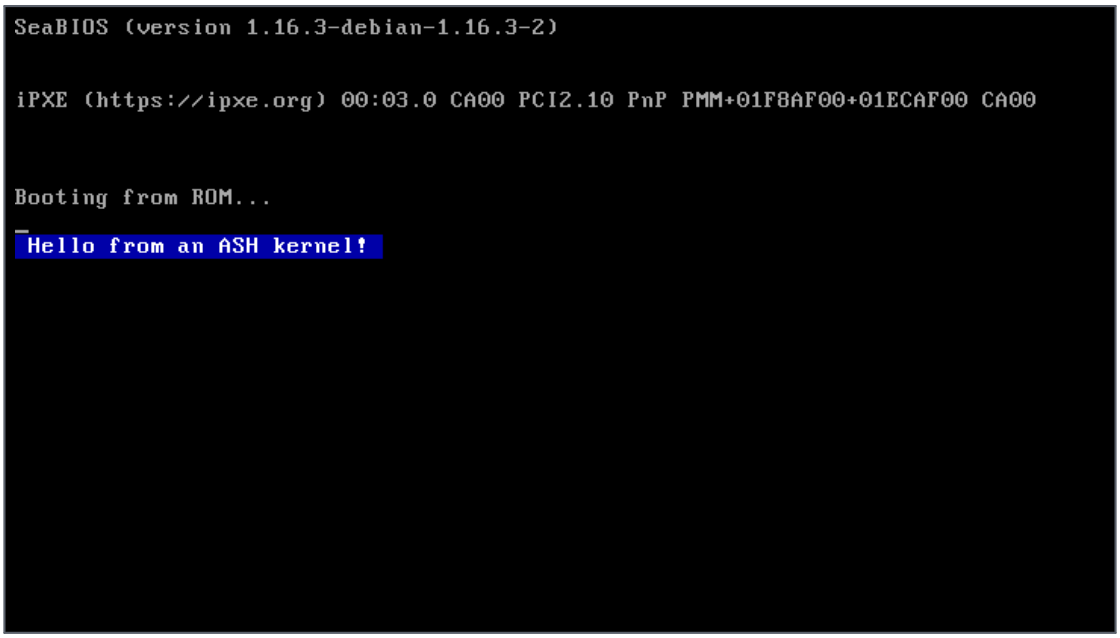
Code	What it does
`@bits(32)`	The kernel runs in 32-bit protected mode.
`pub func kmain(uint32 magic, uint32 info)`	Called by `boot.asm` with the two values that the loader leaves in `eax` and `ebx`.
`uint8* video = 0xB8000;`	A pointer to the first byte of text memory.
`int32 cell = (row * 80 + i) * 2;`	The byte offset of column `i` of row `row`.
`video[cell] = message[i];`	Stores the character code.
`video[cell + 1] = 0x1F;`	Stores its attribute: white (15) on blue (1).
(end of `kmain`)	Returns to `boot.asm`, which halts the processor with `cli` and `hlt`.

SHELL SESSION

```
$ shasm --lang en -f elf32 -o boot.o boot.asm
$ ashc hello.ash --elf --no-libc -s -o hello.o
$ ld -m elf_i386 -z noexecstack --no-warn-rwx-segments -T linker.ld -o hello.elf boot.o hell
↪ o.o
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Figure 31-1. hello.ash running in QEMU



31.5 ALL THE COLORS

The next kernel shows every color the text screen offers. Below the BIOS messages ("Booting from ROM...") it draws sixteen squares of space characters, each with a different background, and a row of block characters in each foreground color.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 31-4. colors.ash: squares in the sixteen VGA colors

COLORS.ASH

```
@bits(32)

define VGA 0xB8000;

func put_at(int32 col, int32 row, int32 ch, int32 attr): void {
    uint8* cell = VGA + (row * 80 + col) * 2;
    cell[0] = ch;
    cell[1] = attr;
}

func text_at(int32 col, int32 row, string s, int32 attr): void {
    int32 i = 0;
    while (s[i] != 0) {
        put_at(col + i, row, s[i], attr);
        i++;
    }
}

// a block of 8 x 3 cells whose background is the given color
func square(int32 col, int32 row, int32 color): void {
    for (int32 y = 0; y < 3; y++) {
        for (int32 x = 0; x < 8; x++) {
            put_at(col + x, row + y, ' ', color << 4);
        }
    }
}

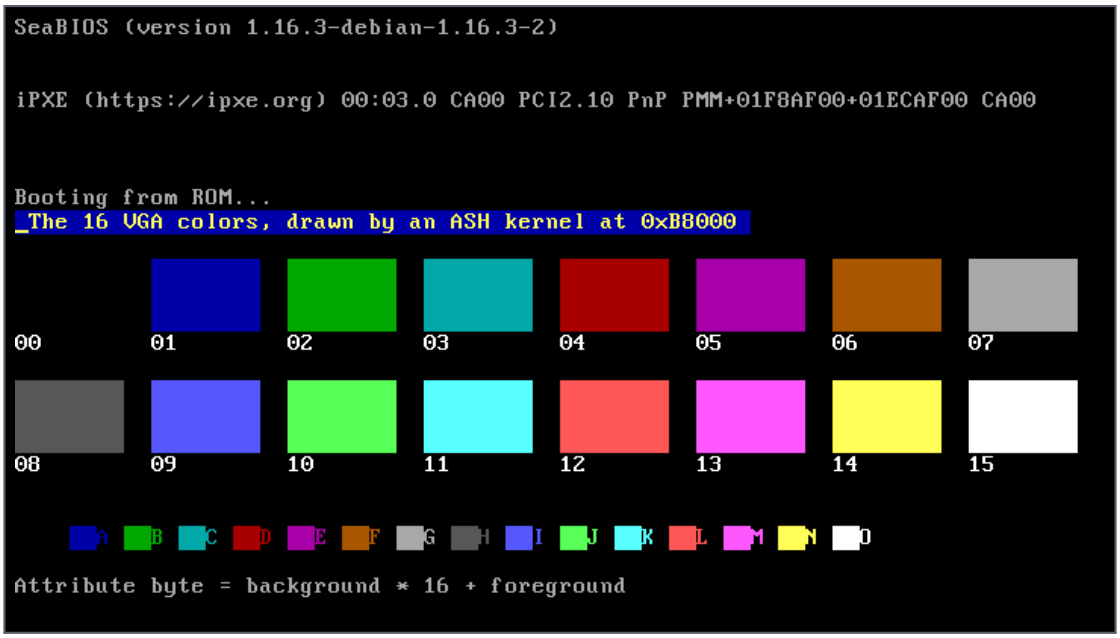
@Entry
pub func kmain(uint32 magic, uint32 info): void {
    text_at(0, 8, " The 16 VGA colors, drawn by an ASH kernel at 0xB8000 ", 0x1E);
    for (int32 c = 0; c < 16; c++) {
        int32 col = (c % 8) * 10;
        int32 row = 10 + (c / 8) * 5;
        square(col, row, c);
        put_at(col, row + 3, '0' + c / 10, 0x0F);
        put_at(col + 1, row + 3, '0' + c % 10, 0x0F);
    }
    for (int32 c = 1; c < 16; c++) {
        put_at(c * 4, 21, 0xDB, c);              // a full block in color c
        put_at(c * 4 + 1, 21, 0xDB, c);
        put_at(c * 4 + 2, 21, 'A' + c - 1, c);   // a letter in color c
    }
    text_at(0, 23, "Attribute byte = background * 16 + foreground", 0x07);
}
```

SHELL SESSION

```
$ shasm --lang en -f elf32 -o boot.o boot.asm
$ ashc colors.ash --elf --no-libc -s -o colors.o
$ ld -m elf_i386 -z noexecstack --no-warn-rwx-segments -T linker.ld -o colors.elf boot.o col
↪ ors.o
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Figure 31-2. colors.ash: the sixteen colors below the BIOS messages



Square 0 is black on black, so it is invisible; squares 8 to 15 are the bright colors, which QEMU's VGA shows as backgrounds because blinking is disabled.

31.6 A SMALL VGA DRIVER

Writing cells one by one quickly becomes tedious. The file `vga.ash` collects the operations every kernel needs: clearing the screen, writing text that wraps and scrolls, printing numbers in decimal and hexadecimal, drawing frames and bars, and moving the blinking hardware cursor. The cursor is not in video memory: it is a register of the video card's CRT controller, reached through the I/O ports `0x3D4` (register index) and `0x3D5` (value) with the `out` instruction.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 31-5. vga.ash: a reusable text-mode driver

VGA.ASH

```
// vga.ash - a small driver for the VGA text screen (80 x 25 cells at 0xB8000)
define VGA_TEXT 0xB8000;
define VGA_COLS 80;
define VGA_ROWS 25;

general static int32 vga_row = 0
general static int32 vga_col = 0
general static int32 vga_color = 0x07

// attribute byte: background color in bits 4-6, foreground color in bits 0-3
func vga_attr(int32 fg, int32 bg): int32 {
    return (bg << 4) | fg;
}

func vga_put(int32 col, int32 row, int32 ch, int32 attr): void {
    uint8* cell = VGA_TEXT + (row * VGA_COLS + col) * 2;
    cell[0] = ch;
    cell[1] = attr;
}

func vga_clear(int32 attr): void {
    for (int32 i = 0; i < VGA_COLS * VGA_ROWS; i++) {
        vga_put(i % VGA_COLS, i / VGA_COLS, ' ', attr);
    }
    vga_row = 0;
    vga_col = 0;
}

func vga_goto(int32 col, int32 row): void {
    vga_col = col;
    vga_row = row;
}

// move the blinking hardware cursor: CRT controller registers 14 and 15
func vga_cursor(int32 col, int32 row): void {
    int32 pos = row * VGA_COLS + col;
    int32 high = (pos >> 8) & 255;
    int32 low = pos & 255;
    asm {
        mov dx, 0x3D4
        mov al, 14
        out dx, al
        mov dx, 0x3D5
        mov eax, high
        out dx, al
        mov dx, 0x3D4
        mov al, 15
        out dx, al
        mov dx, 0x3D5
        mov eax, low
        out dx, al
    }
}

// move every line up by one and clear the last line
func vga_scroll(): void {
    uint8* screen = VGA_TEXT;
    for (int32 i = 0; i < (VGA_ROWS - 1) * VGA_COLS * 2; i++) {
        screen[i] = screen[i + VGA_COLS * 2];
```

Copyright © 2026 Mallionic Inc. All rights reserved.

VGA.ASH (CONTINUED)

```
    }
    for (int32 c = 0; c < VGA_COLS; c++) {
        vga_put(c, VGA_ROWS - 1, ' ', vga_color);
    }
    vga_row = VGA_ROWS - 1;
}

func vga_putc(int32 ch): void {
    if (ch == '\n') {
        vga_col = 0;
        vga_row = vga_row + 1;
    } else {
        vga_put(vga_col, vga_row, ch, vga_color);
        vga_col = vga_col + 1;
        if (vga_col == VGA_COLS) {
            vga_col = 0;
            vga_row = vga_row + 1;
        }
    }
    if (vga_row == VGA_ROWS) {
        vga_scroll();
    }
}

func vga_puts(uint8* s): void {
    int32 i = 0;
    while (s[i] != 0) {
        vga_putc(s[i]);
        i++;
    }
}

func vga_dec(uint32 v): void {
    uint8[12] digits;
    int32 n = 0;
    if (v == 0) {
        vga_putc('0');
        return;
    }
    while (v > 0) {
        digits[n] = '0' + v % 10;
        v = v / 10;
        n++;
    }
    while (n > 0) {
        n--;
        vga_putc(digits[n]);
    }
}

func vga_hex(uint32 v): void {
    vga_puts("0x");
    for (int32 shift = 28; shift >= 0; shift = shift - 4) {
        int32 d = (v >> shift) & 15;
        if (d < 10) { vga_putc('0' + d); } else { vga_putc('A' + d - 10); }
    }
}

// a frame drawn with the double-line characters of code page 437
func vga_frame(int32 col, int32 row, int32 w, int32 h, int32 attr): void {
    vga_put(col, row, 0xC9, attr);
```

Copyright © 2026 Mallionic Inc. All rights reserved.

VGA.ASH (CONTINUED)

```
    vga_put(col + w - 1, row, 0xBB, attr);
    vga_put(col, row + h - 1, 0xC8, attr);
    vga_put(col + w - 1, row + h - 1, 0xBC, attr);
    for (int32 x = 1; x < w - 1; x++) {
        vga_put(col + x, row, 0xCD, attr);
        vga_put(col + x, row + h - 1, 0xCD, attr);
    }
    for (int32 y = 1; y < h - 1; y++) {
        vga_put(col, row + y, 0xBA, attr);
        vga_put(col + w - 1, row + y, 0xBA, attr);
    }
}

// a bar of 'full' cells out of 'size', in the given color
func vga_bar(int32 col, int32 row, int32 size, int32 full, int32 attr): void {
    for (int32 i = 0; i < size; i++) {
        if (i < full) {
            vga_put(col + i, row, 0xDB, attr);
        } else {
            vga_put(col + i, row, 0xB0, 0x08);
        }
    }
}
```

Table 31-5. Functions of vga.ash

Function	What it does
`vga_attr(fg, bg)`	Builds an attribute byte from two colors of Table 31-3.
`vga_put(col, row, ch, attr)`	Writes one character with one attribute at a position.
`vga_clear(attr)`	Fills the screen with spaces of the given attribute and moves the text position to the top left.
`vga_goto(col, row)`	Sets the position where the next text goes.
`vga_putc(ch)`	Writes one character at the text position in `vga_color`; handles `\n`, line wrapping and scrolling.
`vga_puts(s)`	Writes a string with `vga_putc`.
`vga_dec(v)`, `vga_hex(v)`	Write a number in decimal, or as `0x` followed by eight hexadecimal digits.
`vga_frame(col, row, w, h, attr)`	Draws a double-line frame with the characters `0xC9`, `0xBB`, `0xC8`, `0xBC`, `0xCD`, `0xBA`.
`vga_bar(col, row, size, full, attr)`	Draws a bar of `size` cells, the first `full` of them solid (`0xDB`), the rest shaded (`0xB0`).
`vga_cursor(col, row)`	Moves the hardware cursor: writes the high and low byte of `row * 80 + col` to CRT registers 14 and 15.
`vga_scroll()`	Moves every line up by one and clears the last line.

The following kernel uses the driver to draw a control panel:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 31-6. dash.ash: a control panel drawn with vga.ash

DASH.ASH

```
@bits(32)
import "vga.ash";

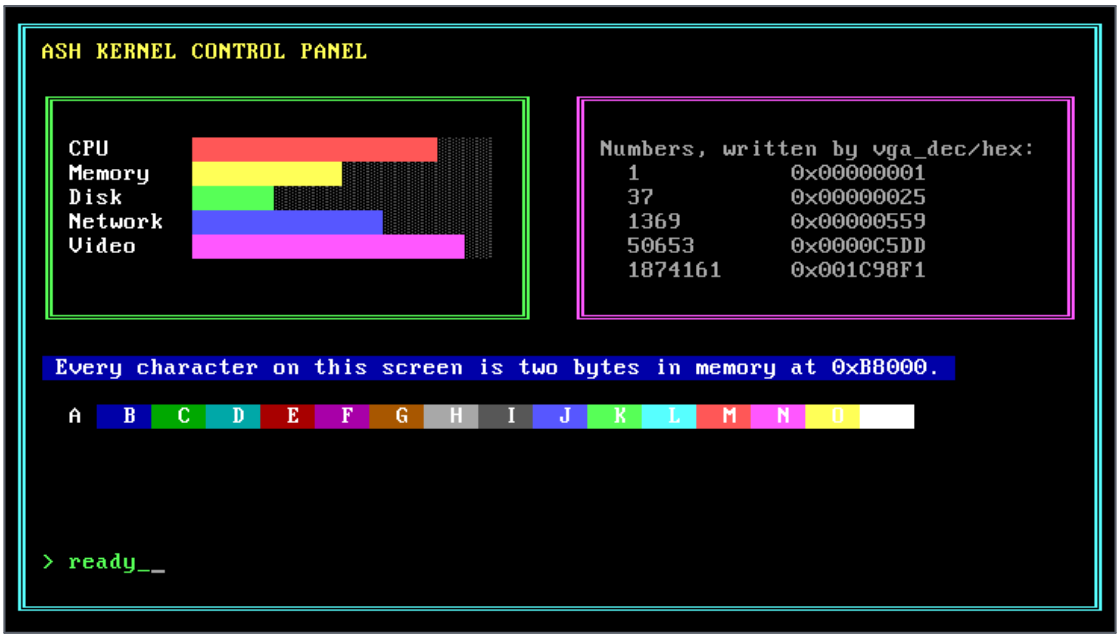
@Entry
pub func kmain(uint32 magic, uint32 info): void {
    vga_clear(vga_attr(7, 0));
    vga_frame(0, 0, 80, 25, vga_attr(11, 0));
    vga_goto(2, 1);
    vga_color = vga_attr(14, 0);
    vga_puts("ASH KERNEL CONTROL PANEL");
    vga_frame(2, 3, 36, 10, vga_attr(10, 0));
    vga_frame(41, 3, 37, 10, vga_attr(13, 0));
    string[] names = {"CPU", "Memory", "Disk", "Network", "Video"};
    int32[] levels = {18, 11, 6, 14, 20};
    int32[] colors = {12, 14, 10, 9, 13};
    for (int32 i = 0; i < 5; i++) {
        vga_goto(4, 5 + i);
        vga_color = vga_attr(15, 0);
        vga_puts(names[i]);
        vga_bar(13, 5 + i, 22, levels[i], vga_attr(colors[i], 0));
    }
    vga_goto(43, 5);
    vga_color = vga_attr(7, 0);
    vga_puts("Numbers, written by vga_dec/hex:");
    int32 v = 1;
    for (int32 r = 0; r < 5; r++) {
        vga_goto(45, 6 + r);
        vga_dec(v);
        vga_goto(57, 6 + r);
        vga_hex(v);
        v = v * 37;
    }
    vga_goto(2, 14);
    vga_color = vga_attr(15, 1);
    vga_puts(" Every character on this screen is two bytes in memory at 0xB8000. ");
    vga_goto(2, 16);
    for (int32 c = 0; c < 16; c++) {
        vga_color = vga_attr(c, 0);
        vga_putc(0xDB);
        vga_putc(0xDB);
        vga_color = vga_attr(15, c);
        vga_putc('A' + c);
        vga_putc(' ');
    }
    vga_goto(2, 22);
    vga_color = vga_attr(10, 0);
    vga_puts("> ready_");
    vga_cursor(10, 22);
}
```

SHELL SESSION

```
$ shasm --lang en -f elf32 -o boot.o boot.asm
$ ashc dash.ash --elf --no-libc -s -o dash.o
$ ld -m elf_i386 -z noexecstack --no-warn-rwx-segments -T linker.ld -o dash.elf boot.o dash.
↪ o
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Figure 31-3. dash.ash: frames, bars, numbers and colors



31.7 GOING FURTHER

- ch-multiboot explains the multiboot header, the linker script and the information the boot loader passes to the kernel.
- ch-dos shows real mode, where the same text memory is reached as segment `0xB800`.
- A keyboard driver reads port `0x60` when interrupt 1 arrives; an interrupt handler is a `@naked` function that ends with `iret` (Section 20.5).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 32

COMMON MISTAKES, TECHNIQUES AND QUESTIONS

This chapter collects the mistakes that beginners make most often in ASH, each with the wrong program, what happens, and the corrected program; then techniques that help you write, understand and debug ASH programs like an expert; and finally answers to frequently asked questions.

32.1 COMMON MISTAKES

Every mistake below was reproduced with the real compiler. When the compiler prints the message in Spanish, the English translation is shown.

32.1.1 Using = instead of == in a condition

= stores a value; == compares two values. A condition needs a comparison.

Example 32-1. Wrong: = inside if

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 3;
    if (x = 3) { return 1; }
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
<input>:4:11: error: expected ) but found ASSIGN(=)@4:11
    4 | =
      |           ^
1 error generated.
```

Example 32-2. Right: == compares

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 3;
    if (x == 3) {
        printf("x is three\n");
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
x is three
```

Copyright © 2026 Mallionic Inc. All rights reserved.

32.1.2 Forgetting the braces of if and loops

In ASH the body of `if`, `while`, `for`, `repeat` and `loop` must always be written between `{` and `}`, even when it has one statement. A `;` right after the condition is also wrong.

Example 32-3. Wrong: a semicolon instead of a block

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 3;
    if (x == 3); { printf("hi\n"); }
    return 0;
}
```

COMPILER DIAGNOSTICS (MESSAGE TRANSLATED TO ENGLISH)

```
<input>:4:16: error: expected { but found SEMICOLON(;)@4:16
    4 | ;
      |                ^
1 error generated.
```

Example 32-4. Right: always a block

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 3;
    if (x == 3) { printf("hi\n"); }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hi
```

32.1.3 Counting array positions from 1

The first element of an array is at position 0, and the last at position `#v - 1`. Using `<=` with the length goes one step too far, and the program stops with exit status 134.

Example 32-5. Wrong: i <= #v reads one element too many

ASH

```
@Entry
pub func main(): int64 {
    int64[] v = {5, 10, 15};
    int64 total = 0;
    for (int64 i = 0; i <= #v; i++) {      // i takes 0, 1, 2 and 3: position 3 does not exi
↪ st
        total = total + v[i];
    }
    printf("total=%d\n", total);
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 134

```
(no output)
```

Example 32-6. Right: i < #v

ASH

```
@Entry
pub func main(): int64 {
    int64[] v = {5, 10, 15};
    int64 total = 0;
    for (int64 i = 0; i < #v; i++) {
        total = total + v[i];
    }
    printf("total=%d\n", total);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
total=30
```

32.1.4 Expecting decimals from integer division

Dividing two integers gives an integer: the fraction is dropped. Multiply before dividing, or use fixed-point arithmetic from `<math>`.

Example 32-7. Integer division drops the fraction

ASH

```
import <math>;

@Entry
pub func main(): int64 {
    int64 a = 7;
    int64 b = 2;
    printf("7 / 2 = %d (not 3.5)\n", a / b);
    printf("7 * 100 / 2 = %d hundredths\n", a * 100 / b);
    uint8[32] text;
    printf("with fixed point: %s\n", fx_print_buf(fx_div(fx(a), fx(b)), &text));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
7 / 2 = 3 (not 3.5)
7 * 100 / 2 = 350 hundredths
with fixed point: 3.5000
```

32.1.5 Choosing a type that is too small

A `uint8` holds 0 to 255. Adding beyond that wraps around to 0 without any warning at run time. Choose `int64` unless you need a smaller type.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 32-8. A small type wraps around

ASH

```
@Entry
pub func main(): int64 {
    uint8 visitors = 250;
    repeat (10) {
        visitors = visitors + 1;
    }
    printf("uint8 after 260 visitors: %d\n", visitors);
    int64 better = 250;
    repeat (10) {
        better = better + 1;
    }
    printf("int64 after 260 visitors: %d\n", better);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
uint8 after 260 visitors: 4
int64 after 260 visitors: 260
```

32.1.6 Using %d for very large numbers

`%d` prints a 32-bit value. For numbers that need 64 bits use `%lld` (signed) or `%llu` (unsigned).

Example 32-9. %d versus %lld

ASH

```
@Entry
pub func main(): int64 {
    int64 big = 5000000000;
    printf("with %%d:   %d\n", big);
    printf("with %%lld: %lld\n", big);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
with %d:   705032704
with %lld: 5000000000
```

32.1.7 Forgetting return in a function with a result

A function declared with a result type (`: int64`) must give a value back with `return`. If it reaches its closing brace without a `return`, the program crashes when it is called (exit status 139, a segmentation fault).

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 32-10. Wrong: no return statement

ASH

```
func five(): int64 {
    int64 x = 5;
}

@Entry
pub func main(): int64 {
    printf("calling five()\n");
    fflush(0);
    return five();
}
```

OUTPUT — TERMINATED BY SIGNAL 11 (SIGSEGV)

```
calling five()
calling five()
calling five()
calling five()
... (95152 more lines not shown)
```

Because no return instruction was generated, the processor continued into the code that follows `five` in memory; here it ran into `main` again, printed the message several times and finally crashed with a segmentation fault (signal 11).

Example 32-11. Right: return the value

ASH

```
func five(): int64 {
    int64 x = 5;
    return x;
}

@Entry
pub func main(): int64 {
    printf("five() = %d\n", five());
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
five() = 5
```

32.1.8 Changing the letters of a string literal

A text written in quotes is stored in memory that cannot be changed. Writing into it crashes the program. Copy the text into a buffer first.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 32-12. Wrong: writing into a literal

ASH

```
@Entry
pub func main(): int64 {
    string s = "cat";
    printf("before\n");
    fflush(0);
    s[0] = 'b';               // the literal cannot be modified: crash
    printf("%s\n", s);
    return 0;
}
```

OUTPUT — TERMINATED BY SIGNAL 11 (SIGSEGV)

```
before
```

Example 32-13. Right: modify a copy in a buffer

ASH

```
import <string>;

@Entry
pub func main(): int64 {
    uint8[16] s;
    strcpy(&s, "cat");
    s[0] = 'b';
    printf("%s\n", &s);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
bat
```

32.1.9 Joining strings with +

+ adds numbers; it does not join texts. Use `strcpy` and `strcat`, or `snprintf`.

Example 32-14. Right: joining texts with snprintf

ASH

```
@Entry
pub func main(): int64 {
    string first = "Ada";
    string last = "Lovelace";
    uint8[64] full;
    snprintf(&full, 64, "%s %s", first, last);
    printf("%s\n", &full);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Ada Lovelace
```

Copyright © 2026 Mallionic Inc. All rights reserved.

32.1.10 Storing a comparison in a variable

In ASH a comparison is not a value; it can only be used as a condition.

Example 32-15. Wrong: a comparison as a value

ASH

```
@Entry
pub func main(): int64 {
    int64 a = 3;
    int64 is_small = a < 10;
    return is_small;
}
```

COMPILER DIAGNOSTICS

```
error: operator '<' is not supported
1 error generated.
```

Example 32-16. Right: set the variable inside an if

ASH

```
@Entry
pub func main(): int64 {
    int64 a = 3;
    int64 is_small = 0;
    if (a < 10) {
        is_small = 1;
    }
    printf("is_small=%d\n", is_small);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
is_small=1
```

32.1.11 Declaring a global without general or ungeneral

Example 32-17. Wrong: the global does not say who can see it

ASH

```
int64 score = 0

@Entry
pub func main(): int64 {
    return 0;
}
```

COMPILER DIAGNOSTICS

```
<input>:1:1: error: variable 'score' declared outside a function: at top level only general,
↪  ungeneral, extern, import, define, func, class and annotations are allowed
hint: a variable declared outside a function must say who can see it. Write 'general int64 s
↪ core = ...;' (public: visible to C and to other files under the name 'score') or 'ungenera
↪ l int64 score = ...;' (private to this script). Add 'static' when the value must change at
↪  run time ('general static int64 score = ...;'); without 'static' the variable is a consta
↪ nt.
    1 | int64
      | ^^^^^
```

Copyright © 2026 Mallionic Inc. All rights reserved.

COMPILER DIAGNOSTICS (CONTINUED)

```
1 error generated.
```

Example 32-18. Right: general static for a writable, public global

ASH

```
general static int64 score = 0

func add_points(int64 p): void {
    score = score + p;
}

@Entry
pub func main(): int64 {
    add_points(10);
    add_points(5);
    printf("score=%d\n", score);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
score=15
```

32.1.12 Mixing printf with direct writes

`printf` keeps its output in a buffer and writes it later; `sys_write` (and `print` from `<syscalls>`) writes immediately. Mixing both can print lines in an unexpected order. Use one kind of output, or call `fflush(0)` before writing directly.

Example 32-19. Output order with and without fflush

ASH

```
import <syscalls>;

@Entry
pub func main(): int64 {
    printf("1. printf (buffered)\n");
    fflush(0);                          // without this line, 1 could appear after 2
    println("2. println (direct)");
    printf("3. printf again\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1. printf (buffered)
2. println (direct)
3. printf again
```

32.1.13 Adding to a pointer to move one element

Adding 1 to a pointer moves it by one **byte**, not by one element. Use indexing.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 32-20. Indexing instead of pointer arithmetic

ASH

```
@Entry
pub func main(): int64 {
    int64[] v = {100, 200, 300};
    int64* p = &v;
    printf("p[1]     = %d\n", p[1]);          // the second element
    printf("*(p + 8) = %d\n", *(p + 8));      // also the second element: 8 bytes later
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
p[1]     = 200
*(p + 8) = 200
```

32.1.14 More mistakes in brief

Table 32-1. More common mistakes

Mistake	What to do
Calling `printf` in a `--no-libc` program	Use `print`, `println`, `print_int` from `<syscalls>`.
A `_start` entry function that ends with `return`	Use `main` with the C library, or end `_start` with `exit(...)` / `sys_exit(...)`.
Using `printf` in a 16-bit program	16-bit programs have no C library; write output with BIOS calls in `asm`.
Naming a variable like a register (`al`, `cx`) and using it in `asm`	Inside `asm`, register names always mean registers. Choose another name.
`import <stdlib>;`	Do not import it in ASH 1.0; call the C functions directly (Section 27.8).
`continue` inside a `switch` inside a loop	Use a flag (Section 11.6).
Writing `int64 v[]` as a parameter	Write `int64* v`.
Forgetting `static` on a global you change	Without `static`, a global is constant.
Expecting a variable declared inside `{ }` to disappear after it	In ASH 1.0 it remains visible until the end of the function; use different names.
Forgetting `#` and writing a fixed count in a loop	`for (int64 i = 0; i < #v; i++)` adapts when the array changes.

32.2 TECHNIQUES TO MASTER ASH

32.2.1 Look at what the compiler did

Two commands show you exactly what your program became. Use them whenever a result surprises you:

SHELL

```
ashc prog.ash --stdout -s                       # the ASHIR: one simple step per line
ashc prog.ash --elf -s && objdump -d -M intel out/prog.o   # the machine code
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 32-21. Reading the ASHIR of your own function

ASH

```
func average(int64 a, int64 b, int64 c): int64 {
    return (a + b + c) / 3;
}

@Entry
pub func main(): int64 {
    return average(10, 20, 30);
}
```

ASHIR

```
func qword average(qword a, qword b, qword c) {
.type __tmp0 qword
.type __tmp1 qword
    %__tmp1 mov[a]
    %__tmp1 add[b]
    %__tmp0 mov[__tmp1]
    %__tmp0 add[c]
    %__ret mov[__tmp0]
    %__ret div[3]
    ret qword __ret
}
```

Each line does one thing: copy `a`, add `b`, add `c`, divide by 3, return. When an expression gives an unexpected value, the ASHIR shows the order in which it was computed.

32.2.2 Use the exit status for quick tests

Returning a value from `main` and printing `$?` in the shell is the quickest way to check a small function without writing any output code.

SHELL

```
ashc test.ash --elf -s && gcc out/test.o -o test && ./test; echo "result: $?"
```

32.2.3 Write your own checks

A small `check` function that prints a message when a condition fails, plus a counter of failures, turns any program into a test program:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 32-22. A tiny test framework

ASH

```
import <math>;
import <string>;

general static int64 failures = 0
general static int64 checks = 0

func check(int64 ok, string what): void {
    checks = checks + 1;
    if (ok == 0) {
        failures = failures + 1;
        printf("FAILED: %s\n", what);
    }
}

func same(int64 a, int64 b): int64 {
    if (a == b) { return 1; }
    return 0;
}

@Entry
pub func main(): int64 {
    check(same(gcd(84, 36), 12), "gcd(84, 36) == 12");
    check(same(isqrt(1000000), 1000), "isqrt(1000000) == 1000");
    check(same(strlen("ASH"), 3), "strlen(\"ASH\") == 3");
    check(same(ipow(2, 10), 1000), "ipow(2, 10) == 1000 (this check is wrong on purpose)");
    printf("%d checks, %d failed\n", checks, failures);
    return failures;
}
```

OUTPUT — EXIT STATUS 1

```
FAILED: ipow(2, 10) == 1000 (this check is wrong on purpose)
4 checks, 1 failed
```

32.2.4 Trace functions with an annotation

A user annotation adds the same tracing code to as many functions as you like, and can be removed in one place (Section 17.16):

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 32-23. Tracing with an annotation

ASH

```
@Trace(string name) {
    printf("-> %s\n", name);
}

@Trace("parse")
func parse(): int64 {
    return 1;
}

@Trace("evaluate")
func evaluate(int64 x): int64 {
    return x * 10;
}

@Entry
pub func main(): int64 {
    int64 r = evaluate(parse());
    printf("result %d\n", r);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
-> parse
-> evaluate
result 10
```

32.2.5 Let the compiler help you

Start every new file with `"use strict";` (Chapter 21), keep the VS Code linter enabled (Section 29.1) and declare external C functions with `extern`, so that the number of arguments is checked.

32.2.6 Give numbers a name

`define` and `typedef` make programs easier to read and to change:

Example 32-24. Named constants and type names

ASH

```
define WIDTH 40;
define HEIGHT 10;
define CHAR_WALL '#';
typedef int64 pixels;

func area(pixels w, pixels h): pixels {
    return w * h;
}

@Entry
pub func main(): int64 {
    for (int64 x = 0; x < WIDTH; x++) { printf("%c", CHAR_WALL); }
    printf("\narea = %d pixels\n", area(WIDTH, HEIGHT));
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
#####################################
area = 400 pixels
```

32.2.7 Measure how long code takes

The processor's time-stamp counter, read with the `rdtsc` instruction, counts clock cycles. Reading it before and after a piece of code measures its cost:

Example 32-25. Measuring cycles with rdtsc

ASH

```
func cycles(): uint64 {
    uint64 lo = 0;
    uint64 hi = 0;
    asm volatile("rdtsc; mov {lo}, rax; mov {hi}, rdx");
    return (hi << 32) | lo;
}

@Entry
pub func main(): int64 {
    uint64 t0 = cycles();
    int64 sum = 0;
    for (int64 i = 0; i < 1000000; i++) {
        sum = sum + i;
    }
    uint64 t1 = cycles();
    int64 spent = t1 - t0;
    if (spent > 0) {
        printf("sum=%lld, measured a positive number of cycles\n", sum);
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
sum=499999500000, measured a positive number of cycles
```

(The exact number of cycles changes from run to run and from machine to machine, so the example prints only that the measurement worked.)

32.2.8 Bit tricks

Many small problems have fast answers with bitwise operators:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 32-26. Useful bit tricks

ASH

```
@Entry
pub func main(): int64 {
    int64 n = 40;
    if ((n & (n - 1)) == 0) { printf("%d is a power of two\n", n); } else { printf("%d is no
↪ t a power of two\n", n); }
    printf("lowest bit of %d: %d\n", n, n & 1);
    printf("%d rounded up to a multiple of 16: %d\n", n, (n + 15) & ~15);
    printf("bit 3 of %d is %d\n", n, (n >> 3) & 1);
    printf("%d with bit 0 set: %d\n", n, n | 1);
    printf("%d with bit 3 cleared: %d\n", n, n & ~8);
    printf("swap 6 and 9 with xor: ");
    int64 a = 6;
    int64 b = 9;
    a = a ^ b;
    b = a ^ b;
    a = a ^ b;
    printf("a=%d b=%d\n", a, b);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
40 is not a power of two
lowest bit of 40: 0
40 rounded up to a multiple of 16: 48
bit 3 of 40 is 1
40 with bit 0 set: 41
40 with bit 3 cleared: 32
swap 6 and 9 with xor: a=9 b=6
```

32.2.9 Debug with gdb

ASH programs are ordinary executables, so the GNU debugger works with them: you can stop at a function, step instruction by instruction and inspect registers. Build with `-e` (a full frame in every function) to get clearer backtraces.

SHELL

```
ashc prog.ash --elf -e -s && gcc -no-pie out/prog.o -o prog
gdb ./prog
(gdb) break square              # stop when square is called
(gdb) run
(gdb) disassemble               # the machine code of the current function
(gdb) info registers rdi rax    # argument and result registers
(gdb) stepi                     # execute one instruction
(gdb) finish                    # run until the function returns
```

32.2.10 Write code for several targets

Use `%if` with the predefined symbols to keep 16, 32 and 64-bit variants, or Linux and Windows variants, in one file (Chapter 18). Keep the target-specific parts small and put them in separate functions.

Copyright © 2026 Mallionic Inc. All rights reserved.

32.3 FREQUENTLY ASKED QUESTIONS

32.3.1 Is ASH a replacement for C?

ASH can do most of what C does in systems programming, and adds direct assembly, multi-mode code generation and safety checks. It is not source-compatible with C, has no floating-point arithmetic in version 1.0 and has a smaller library. It links freely with C, so the two can be combined.

32.3.2 Do ASH programs need Java to run?

No. Only the compiler is written in Java, and even the compiler is distributed as a native executable. The programs it produces are ordinary machine code.

32.3.3 Which name should the entry function have?

`main` if the program uses the C library (the usual case), `_start` if it does not. See Section 3.5.

32.3.4 Why does my program print nothing and return a strange number?

The exit status is shown with `echo $?` after running the program. A value of 134 means an array index was out of range; 139 means a segmentation fault (a crash); 136 a division by zero. See Table 28-2.

32.3.5 How do I read a number typed by the user?

Use `scanf` from the C library, or read a line with `fgets` and convert it with `atoi`:

Example 32-27. Reading a number

ASH

```
@Entry
pub func main(): int64 {
    int64 n = 0;
    printf("Enter a number: ");
    scanf("%lld", &n);
    printf("twice %lld is %lld\n", n, n * 2);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
Enter a number: twice 21 is 42
```

32.3.6 How do I generate random numbers?

Call the C functions `srand` and `rand` directly:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 32-28. Random numbers

ASH

```
extern void srand(uint32 seed);
extern int32 rand();

@Entry
pub func main(): int64 {
    srand(2026);
    int64 dice_total = 0;
    repeat (10) {
        int64 die = rand() % 6 + 1;
        dice_total = dice_total + die;
    }
    if (dice_total >= 10) {
        if (dice_total <= 60) {
            printf("ten dice thrown, the total is between 10 and 60\n");
        }
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
ten dice thrown, the total is between 10 and 60
```

32.3.7 How do I allocate memory whose size is known only at run time?

Declare an array with a computed size (`int64[n] v;`), or call `malloc` and `free` from the C library (Section 30.3). Without the C library use `sys_mmap` (Section 27.4).

32.3.8 How do I return several values from a function?

Pass pointers to variables of the caller, or fill a structure that the caller provides:

Example 32-29. Returning two values through pointers

ASH

```
func divide(int64 a, int64 b, int64* quotient, int64* remainder): void {
    quotient[0] = a / b;
    remainder[0] = a % b;
}

@Entry
pub func main(): int64 {
    int64 q = 0;
    int64 r = 0;
    divide(17, 5, &q, &r);
    printf("17 / 5 = %d remainder %d\n", q, r);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
17 / 5 = 3 remainder 2
```

Copyright © 2026 Mallionic Inc. All rights reserved.

32.3.9 Can I write a whole program in assembly?

Yes: either as `shasm` source files (Chapter 26) or as ASH functions marked `@naked` (Section 20.5). Mixing both styles is the usual approach: ASH for structure, assembly for the critical parts.

32.3.10 Why are local arrays not freed?

ASH 1.0 allocates local arrays on the heap and never frees them, to keep the generated code simple (Section 13.2). For programs that run long loops, allocate arrays once outside the loop.

32.3.11 How big are ASH programs?

A program that uses the C library is a few kilobytes, like an equivalent C program. A static `--no-libc` program is typically under 10 kilobytes, and a `--exe` program can be under 200 bytes.

32.3.12 Can ASH build an operating system or a boot loader?

Yes. Use `@bits(16)` for real-mode code, `@bits(32)` or `@bits(64)` with `--elf` and a linker script for kernels, `@section` to place headers, `@naked` for entry and interrupt code, and `<string>`, `<math>` and structures, which need no C library (Section 22.5).

32.3.13 Where can I see more examples?

In the `examples/` directory of the source tree (Appendix D), in the cookbook (Chapter 30) and in the library reference, where every function has an example (Chapter 27).

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 33

LANGUAGE DICTIONARY

This chapter lists every word and symbol of the ASH language in one place, in the order a beginner is likely to look them up: first the symbols, then the words in alphabetical order. Each entry says what kind of element it is, shows how to write it, explains what it means in plain words, and gives a small complete program with its real output. Follow the *See also* links for the full rules.

33.1 SYMBOLS AND OPERATORS

33.1.1

Kind: operator.

```
#array
```

Gives the number of elements of an array (for a two-dimensional array, the number of rows). The length is read from the array itself at run time, so it is always correct.

ASH

```ash
@Entry
pub func main(): int64 {
    int64[] v = {4, 8, 15};
    printf("the array has %d elements\n", #v);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
the array has 3 elements
```

See also: Section 13.4.

33.1.2 &

Kind: operator.

```
&variable  |  a & b
```

Before a name, & gives the **address** of the variable, array, global or function: the place in memory where it is stored. Between two values, & is the bitwise AND: each bit of the result is 1 only if both bits are 1.

ASH

```ash
@Entry
pub func main(): int64 {
    int64 x = 5;
    int64* p = &x;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASH (CONTINUED)

```
    *p = 9;
    printf("x is now %d; 12 & 10 = %d\n", x, 12 & 10);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
x is now 9; 12 & 10 = 8
```

See also: Section 8.7, Section 10.7.

33.1.3 *

Kind: operator.

```
a * b  |  *pointer  |  type*
```

Between two values, * multiplies. Before a pointer, * reads (or, on the left of =, writes) the value the pointer points to. After a type name, * makes a pointer type: `int64*` is "address of an int64".

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 6;
    int64* p = &x;
    printf("%d %d\n", x * 7, *p);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
42 6
```

See also: Section 8.7.

33.1.4 **

Kind: operator.

```
base ** exponent
```

Raises a number to a power: `2 ** 10` is 1024. With constants the result is computed by the compiler.

ASH

```
@Entry
pub func main(): int64 {
    int64 n = 3;
    printf("%d %d\n", 2 ** 10, n ** 4);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1024 81
```

Copyright © 2026 Mallionic Inc. All rights reserved.

See also: Section 10.5.

33.1.5 ***

Kind: operator.

```
base *** n
```

Squares the base *n* times: `2 *** 3` is ((2 squared) squared) squared = 256.

ASH

```
@Entry
pub func main(): int64 {
    printf("%d\n", 2 *** 3);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
256
```

See also: Section 10.5.

33.1.6 ++ and --

Kind: operator.

```
x++  ++x  x--  --x
```

Add 1 to or subtract 1 from a variable. `x++` gives the old value and then increments; `++x` increments first.

ASH

```
@Entry
pub func main(): int64 {
    int64 i = 5;
    int64 a = i++;
    int64 b = ++i;
    i--;
    printf("a=%d b=%d i=%d\n", a, b, i);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
a=5 b=7 i=6
```

See also: Section 10.8.

33.1.7 += -= *= /= %=

Kind: operator.

```
x += value
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Shorthand for changing a variable with its own value: `x += 3` means `x = x + 3`. The same exists for `-`, `*`, `/`, `%`, `<<`, `>>`, `&`, `|` and `^`.

ASH

```
@Entry
pub func main(): int64 {
    int64 x = 10;
    x += 5;
    x *= 2;
    x -= 1;
    printf("%d\n", x);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
29
```

See also: Section 7.8.2.

33.1.8 -> and .

Kind: operator.

```
pointer->field  |  variable.field  |  Class.method()
```

Reach a field of a structure: `.` on a structure variable, `->` through a pointer. `.` also calls class methods and invokes cracks.

ASH

```
typedef struct {
    int64 w;
    int64 h;
} Box;

@Entry
pub func main(): int64 {
    Box b;
    b.w = 3;
    Box* p = &b;
    p->h = 4;
    printf("%d x %d\n", b.w, p->h);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3 x 4
```

See also: Section 14.4.

33.1.9 ...

Kind: syntax.

```
func name(type first, ...)
```

Copyright © 2026 Mallionic Inc. All rights reserved.

At the end of a parameter list, `...` lets a function receive up to eight extra arguments, read with `va_count()` and `va_arg(i)`.

```ash
func total(int64 first, ...): int64 {
    int64 t = first;
    for (int64 i = 0; i < va_count(); i++) {
        t = t + va_arg(i);
    }
    return t;
}

@Entry
pub func main(): int64 {
    printf("%d\n", total(1, 2, 3, 4));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
10
```

See also: Section 12.6.

33.1.10 << and >>

Kind: operator.

```
value << n  |  value >> n
```

Shift the bits of a value left or right by *n* positions. Shifting left by one doubles the value; shifting right by one halves it (dropping the remainder).

```ash
@Entry
pub func main(): int64 {
    printf("%d %d\n", 3 << 4, 100 >> 2);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
48 25
```

See also: Section 10.6.

33.1.11 == != < <= > >=

Kind: operator.

```
if (a == b) ...
```

Compare two values: equal, not equal, less, less or equal, greater, greater or equal. They are used in conditions (`if`, `while`, `for`).

Copyright © 2026 Mallionic Inc. All rights reserved.

ASH

```
@Entry
pub func main(): int64 {
    int64 a = 3;
    if (a != 4) { printf("a is not 4\n"); }
    if (a <= 3) { printf("a is at most 3\n"); }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
a is not 4
a is at most 3
```

See also: Section 10.9.

33.1.12 && || !

Kind: operator.

```
if (a && b) ...  if (a || b) ...  if (!(a)) ...
```

Combine conditions: && (and) is true when both are true, || (or) when at least one is true, ! (not) inverts a condition in parentheses.

ASH

```
@Entry
pub func main(): int64 {
    int64 age = 30;
    if (age >= 18 && age < 65) { printf("working age\n"); }
    if (!(age == 0)) { printf("not zero\n"); }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
working age
not zero
```

See also: Section 11.3.

33.1.13 @

Kind: syntax.

```
@Name  |  @Name(arguments)
```

Starts an annotation: extra information for the compiler written before a function, a global or a file.

See also: Chapter 17.

Copyright © 2026 Mallionic Inc. All rights reserved.

33.2 WORDS, FROM A TO Z

33.2.1 asm

Kind: statement.

```
asm { instructions }  |  asm volatile("instructions")
```

Writes processor instructions directly. ASH variables can be named inside the block.

ASH

```ash
@Entry
pub func main(): int64 {
    int64 x = 20;
    asm {
        add x, 22
    }
    printf("%d\n", x);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
42
```

See also: Chapter 20.

33.2.2 bool, true, false

Kind: type and constants.

```
bool flag = true;
```

A type for yes/no values. `true` is 1 and `false` is 0.

ASH

```ash
@Entry
pub func main(): int64 {
    bool ready = true;
    if (ready == true) { printf("ready=%d\n", ready); }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
ready=1
```

See also: Section 8.4.

33.2.3 break

Kind: statement.

```
break;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Leaves the loop (or `switch`) it is in, immediately.

ASH

```
@Entry
pub func main(): int64 {
    for (int64 i = 0; i < 100; i++) {
        if (i == 3) { break; }
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0 1 2
```

See also: Section 11.6.

33.2.4 case and default

Kind: statement.

```
switch (x) { case 1: ... default: ... }
```

Labels inside a `switch`. `case value:` marks where to start when the value matches; `default:` is used when no case matches.

ASH

```
@Entry
pub func main(): int64 {
    int64 day = 6;
    switch (day) {
        case 6:
        case 7:
            printf("weekend\n");
            break;
        default:
            printf("weekday\n");
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
weekend
```

See also: Section 11.7.

33.2.5 char

Kind: type.

```
char c = 'A';
```

A type for one character, stored as its 8-bit code.

Copyright © 2026 Mallionic Inc. All rights reserved.

ASH

```ash
@Entry
pub func main(): int64 {
    char c = 'A';
    c = c + 2;
    printf("%c\n", c);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
C
```

See also: Section 8.5.

33.2.6 class

Kind: declaration.

```
class Name { pub func m(...) { ... } }
```

Groups functions under a name. Methods are called as `Name.method(...)`; `priv` methods can be called only from the class.

ASH

```ash
class Counter {
    pub func next(int64 n): int64 {
        return n + 1;
    }
}

@Entry
pub func main(): int64 {
    printf("%d\n", Counter.next(41));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
42
```

See also: Chapter 15.

33.2.7 continue

Kind: statement.

```
continue;
```

Skips the rest of the current loop iteration and goes on with the next one.

ASH

```ash
@Entry
pub func main(): int64 {
    for (int64 i = 0; i < 6; i++) {
        if (i % 2 == 1) { continue; }
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASH (CONTINUED)

```
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
0 2 4
```

See also: Section 11.6.

33.2.8 define

Kind: declaration.

```
define NAME value;
```

Gives a name to a value or a piece of code. Every later use of the name is replaced by the value.

ASH

```
define SIZE 8;

@Entry
pub func main(): int64 {
    int64[SIZE] v;
    printf("%d\n", #v);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
8
```

See also: Section 18.4.

33.2.9 elif and else

Kind: statement.

```
if (c) { } elif (c) { } else { }
```

`elif` (or `else if`) tests another condition when the previous ones were false; `else` runs when none was true.

ASH

```
@Entry
pub func main(): int64 {
    int64 n = 0;
    if (n > 0) { printf("positive\n"); }
    elif (n < 0) { printf("negative\n"); }
    else { printf("zero\n"); }
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
zero
```

See also: Section 11.4.

33.2.10 emit

Kind: statement.

```
emit(byte, byte, ...);  emit16(...)  emit32(...)  emit64(...)
```

Places raw machine-code bytes in the program.

ASH

```ash
@Entry
pub func main(): int64 {
    int64 r = 0;
    emit(0xB8, 0x07, 0x00, 0x00, 0x00);   // mov eax, 7
    asm volatile("mov {r}, rax");
    printf("%d\n", r);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
7
```

See also: Section 20.3.

33.2.11 extern

Kind: declaration.

```
extern type name(parameter types);
```

Announces a function that exists outside the ASH program, for example in the C library.

ASH

```ash
extern int64 labs(int64 v);

@Entry
pub func main(): int64 {
    printf("%d\n", labs(-9));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
9
```

See also: Section 12.8.

Copyright © 2026 Mallionic Inc. All rights reserved.

33.2.12 for

Kind: statement.

```
for (start; condition; step) { ... }
```

Repeats a block: runs *start* once, then while *condition* is true runs the block and then *step*.

ASH

```ash
@Entry
pub func main(): int64 {
    for (int64 i = 3; i > 0; i--) {
        printf("%d... ", i);
    }
    printf("go!\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3... 2... 1... go!
```

See also: Section 11.5.2.

33.2.13 func

Kind: declaration.

```
func name(type param, ...): result { ... }
```

Defines a function: a named piece of code that receives parameters and may return a result.

ASH

```ash
func triple(int64 x): int64 {
    return x * 3;
}

@Entry
pub func main(): int64 {
    printf("%d\n", triple(14));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
42
```

See also: Section 12.1.

33.2.14 general and ungeneral

Kind: declaration.

```
general static type name = value
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Declare a global variable (outside every function). `general` makes it visible to other files and to C; `ungeneral` keeps it private. Add `static` to make it changeable.

ASH

```
general static int64 calls = 0

func tick(): void {
    calls = calls + 1;
}

@Entry
pub func main(): int64 {
    tick();
    tick();
    printf("%d\n", calls);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
2
```

See also: Section 9.4.

33.2.15 if

Kind: statement.

```
if (condition) { ... }
```

Runs a block only when the condition is true.

ASH

```
@Entry
pub func main(): int64 {
    int64 t = 31;
    if (t > 30) {
        printf("hot\n");
    }
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hot
```

See also: Section 11.4.

33.2.16 import

Kind: declaration.

```
import <library>;  |  import "file.ash";
```

Brings in the functions and constants of a library or of another file of your project.

Copyright © 2026 Mallionic Inc. All rights reserved.

```ash
import <math>;

@Entry
pub func main(): int64 {
    printf("%d\n", gcd(12, 18));
    return 0;
}
```

```
6
```

See also: Chapter 19.

33.2.17 int8 ... uint64

Kind: types.

```
int64 x;  uint8 b;
```

Integer types. The number is the size in bits (8, 16, 32 or 64); `u` means unsigned (no negative values).

```ash
@Entry
pub func main(): int64 {
    int8 small = -100;
    uint16 medium = 60000;
    int64 large = 9000000000;
    printf("%d %d %lld\n", small, medium, large);
    return 0;
}
```

```
-100 60000 9000000000
```

See also: Section 8.2.

33.2.18 len, length, rows, cols, count

Kind: built-in functions.

```
len(a)  rows(m)  cols(m)  count(m)
```

`len(a)` and `length(a)` are the same as `#a`. For a matrix, `rows`, `cols` and `count` give its rows, columns and number of elements.

```ash
@Entry
pub func main(): int64 {
    matrix m[] []{
        [int32 1], [int32 2], [int32 3],
        [int32 4], [int32 5], [int32 6],
    }
    printf("%d rows, %d cols, %d cells\n", rows(m), cols(m), count(m));
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASH (CONTINUED)

```
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
2 rows, 3 cols, 6 cells
```

See also: Section 13.4.

33.2.19 loop

Kind: statement.

```
loop { ... }
```

Repeats a block forever; leave it with `break`, `return` or `terminate`.

ASH

```
@Entry
pub func main(): int64 {
    int64 n = 1;
    loop {
        n = n * 3;
        if (n > 100) { break; }
    }
    printf("%d\n", n);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
243
```

See also: Section 11.5.4.

33.2.20 macro

Kind: declaration.

```
macro NAME { statements }
```

A named block of statements, inserted wherever the name is written.

ASH

```
macro HELLO {
    printf("hello\n");
}

@Entry
pub func main(): int64 {
    HELLO
    HELLO
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
hello
hello
```

See also: Section 18.5.

33.2.21 matrix

Kind: statement.

```
matrix name[] []{ [type value], ... }
```

Declares a table of numbers written row by row (each line is a row).

ASH

```ash
@Entry
pub func main(): int64 {
    matrix t[] []{
        [int32 1], [int32 2],
        [int32 3], [int32 4],
    }
    printf("%d\n", t[1][0]);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
3
```

See also: Section 13.7.

33.2.22 pub and priv

Kind: modifier.

```
pub func ...  |  priv func ...
```

`pub` makes a function visible outside the file (to C and the linker); `priv` restricts it to its file or class.

See also: Section 15.5.

33.2.23 redefine

Kind: declaration.

```
redefine type old(...) type new(...)
```

Gives an existing function a new name, or exports a new definition.

ASH

```ash
redefine uint64 printf(char[], ...) uint64 say(char[]...)

@Entry
pub func main(): int64 {
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASH (CONTINUED)

```
    say("said with %s\n", "an alias");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
said with an alias
```

See also: Section 18.7.

33.2.24 repeat

Kind: statement.

```
repeat (count) { ... }
```

Runs a block a fixed number of times.

ASH

```
@Entry
pub func main(): int64 {
    repeat (3) {
        printf("ha");
    }
    printf("!\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hahaha!
```

See also: Section 11.5.3.

33.2.25 return

Kind: statement.

```
return value;  |  return;
```

Ends the function and gives a value back to the caller.

ASH

```
func half(int64 x): int64 {
    return x / 2;
}

@Entry
pub func main(): int64 {
    printf("%d\n", half(84));
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

```
OUTPUT — EXIT STATUS 0

42
```

See also: Section 11.8.

33.2.26 runnable

Kind: modifier.

```
runnable byte[] code = { ... }
```

Declares a global array whose bytes are machine code that can be called like a function.

```
ASH

runnable byte[] answer = {0xB8, 42, 0, 0, 0, 0xC3}

@Entry
pub func main(): int64 {
    printf("%d\n", answer());
    return 0;
}
```

```
OUTPUT — EXIT STATUS 0

42
```

See also: Section 9.4.4.

33.2.27 static

Kind: modifier.

```
general static int64 g = 0
```

Makes a global variable changeable. Without `static`, a global is a constant.

See also: Section 9.4.

33.2.28 string

Kind: type.

```
string s = "text";
```

A type for text: holds the address of characters ending with a 0 byte.

```
ASH

@Entry
pub func main(): int64 {
    string s = "ASH";
    printf("%s starts with %c\n", s, s[0]);
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
ASH starts with A
```

See also: Section 8.6.

33.2.29 struct and typedef

Kind: declaration.

```
typedef struct { fields } Name;  |  typedef type Name;
```

`struct` groups fields into one type; `typedef` gives a new name to a type.

ASH

```ash
typedef uint8 byte_t;
typedef struct {
    int64 x;
    int64 y;
} Point;

@Entry
pub func main(): int64 {
    Point p;
    p.x = 2;
    p.y = 5;
    byte_t b = 200;
    printf("(%d, %d) %d\n", p.x, p.y, b);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
(2, 5) 200
```

See also: Chapter 14.

33.2.30 switch

Kind: statement.

```
switch (value) { case ...: ... }
```

Chooses among several cases by comparing one value with each `case`.

ASH

```ash
@Entry
pub func main(): int64 {
    char op = '*';
    switch (op) {
        case '+': printf("plus\n"); break;
        case '*': printf("times\n"); break;
        default: printf("other\n");
    }
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
times
```

See also: Section 11.7.

33.2.31 terminate

Kind: statement.

```
terminate value;  |  terminate;
```

Ends the function at once, even from inside several loops, and returns the value.

ASH

```ash
func first_even(int64* v, int64 n): int64 {
    for (int64 i = 0; i < n; i++) {
        if (v[i] % 2 == 0) { terminate v[i]; }
    }
    return -1;
}

@Entry
pub func main(): int64 {
    int64[] v = {3, 7, 10, 12};
    printf("%d\n", first_even(v, #v));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
10
```

See also: Section 11.8.2.

33.2.32 unreturnable

Kind: statement.

```
unreturnable value;
```

Sets the result a function will return later, without stopping the function.

ASH

```ash
func work(): int64 {
    unreturnable 99;
    printf("still working\n");
    return;
}

@Entry
pub func main(): int64 {
    printf("%d\n", work());
    return 0;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

OUTPUT — EXIT STATUS 0

```
still working
99
```

See also: Section 16.2.

33.2.33 "use strict"

Kind: directive.

```
"use strict";
```

At the top of a file, turns on strict mode: extra checks on numbers and returns.

ASH

```
"use strict";

@Entry
pub func main(): int64 {
    uint8 small = 255;
    printf("%d\n", small);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
255
```

See also: Chapter 21.

33.2.34 va_count and va_arg

Kind: built-in functions.

```
va_count()  va_arg(i)
```

Inside a function with `...`: how many extra arguments were passed, and the extra argument number *i*.

See also: Section 12.6.

33.2.35 void

Kind: type.

```
func f(): void
```

Means "no value": the result type of a function that returns nothing.

ASH

```
func say_hi(): void {
    printf("hi\n");
}

@Entry
```

Copyright © 2026 Mallionic Inc. All rights reserved.

ASH (CONTINUED)

```ash
pub func main(): int64 {
    say_hi();
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
hi
```

See also: Section 8.9.

33.2.36 volatile

Kind: modifier.

```
volatile type name = value;  |  volatile asm { }
```

Keeps a variable even if the program never reads it, or marks an `asm` block as having effects.

ASH

```ash
@Entry
pub func main(): int64 {
    volatile int64 marker = 1234;
    printf("ok\n");
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
ok
```

See also: Section 9.3.

33.2.37 while

Kind: statement.

```
while (condition) { ... }
```

Repeats a block as long as the condition is true.

ASH

```ash
@Entry
pub func main(): int64 {
    int64 n = 1;
    while (n < 1000) {
        n = n * 2;
    }
    printf("%d\n", n);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
1024
```

Copyright © 2026 Mallionic Inc. All rights reserved.

See also: Section 11.5.1.

33.2.38 %if, %define and other directives

Kind: directives.

```
%if CONDITION ... %endif
```

Lines that start with `%` choose which parts of the file are compiled, depending on the target.

ASH

```ash
%if ASH_BITS == 64
define MODE 64;
%else
define MODE 32;
%endif

@Entry
pub func main(): int64 {
    printf("compiled for %d bits\n", MODE);
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
compiled for 64 bits
```

See also: Chapter 18.

Copyright © 2026 Mallionic Inc. All rights reserved.

APPENDIX A

X86 INSTRUCTION REFERENCE

This appendix describes the x86 instructions most relevant to ASH programmers: those that the compiler generates, those used in `asm` blocks for system programming, and the most common x87 and SSE instructions. For every instruction it lists the forms that `shasm` 1.2.0 can encode, taken directly from its instruction database, and an example of each form **assembled by** `shasm` while this manual was built, with the resulting bytes.

A.1 HOW TO READ THE TABLES

Table A-1. Columns of the instruction tables

Column	Meaning
Opcode	The encoding template. Hexadecimal bytes are literal. `REX.W` is a REX prefix with W = 1 (`48` to `4F`); `66` the operand-size prefix; `/r` a ModRM byte with a register operand in `reg`; `/0`–`/7` a ModRM byte whose `reg` field is that digit (an opcode extension); `+r` the register number added to the byte; `+cc` the condition code added to the byte; `ib`, `iw`, `id`, `iq` an immediate of 1, 2, 4 or 8 bytes; `cb`, `cd` a relative displacement of 1 or 4 bytes.
Instruction	The form in Intel notation. `r8`/`r16`/`r32`/`r64` are registers, `r/m`*N* a register or memory operand, `m`*N* memory only, `imm`*N* an immediate, `rel8`/`rel32` a jump displacement, `Sreg` a segment register, `CRn`/`DRn` control and debug registers, `xmm1` and `xmm2/m128` XMM operands, `ST(i)` an x87 register.
64-bit	`Valid` if the form can be used in 64-bit mode, `Invalid` if it exists only in 16-bit and 32-bit modes.
Since	The first processor or extension that provides the form. `priv` marks privileged forms (CPL 0).
Example and encoding	An instance of the form and the bytes produced by `shasm`. The examples use `rbx` as the address register, `rcx`/`ecx`/`cx`/`cl` as register operands and `0x12`, `0x1234`, `0x12345678` as immediates; forms marked *(32-bit)* were assembled with `bits 32`.

The condition codes used by `Jcc`, `SETcc` and `CMOVcc` are numbered: O = 0, NO = 1, B/C/NAE = 2, AE/NC/NB = 3, E/Z = 4, NE/NZ = 5, BE/NA = 6, A/NBE = 7, S = 8, NS = 9, P/PE = A, NP/PO = B, L/NGE = C, GE/NL = D, LE/NG = E, G/NLE = F. The examples use E (4).

A.2 GENERAL-PURPOSE INSTRUCTIONS

Instructions that operate on the general-purpose registers and memory: data transfer, arithmetic, logic, shifts, bit manipulation, control transfer, strings and I/O.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.1 MOV — Move

Opcode	Instruction	64-bit	Since	Example and encoding
88 /r	MOV r/m8, r8	Valid	8086	mov byte [rbx], cl = 88 0B
66 89 /r	MOV r/m16, r16	Valid	8086	mov word [rbx], cx = 66 89 0B
89 /r	MOV r/m32, r32	Valid	80386	mov dword [rbx], ecx = 89 0B
REX.W 89 /r	MOV r/m64, r64	Valid	x86-64	mov qword [rbx], rcx = 48 89 0B
8A /r	MOV r8, r/m8	Valid	8086	mov cl, byte [rbx] = 8A 0B
66 8B /r	MOV r16, r/m16	Valid	8086	mov cx, word [rbx] = 66 8B 0B
8B /r	MOV r32, r/m32	Valid	80386	mov ecx, dword [rbx] = 8B 0B
REX.W 8B /r	MOV r64, r/m64	Valid	x86-64	mov rcx, qword [rbx] = 48 8B 0B
B0+r ib	MOV r8, imm	Valid	8086	mov cl, 0x12 = B1 12
66 B8+r iw	MOV r16, imm	Valid	8086	mov cx, 0x1234 = 66 B9 34 12
B8+r id	MOV r32, imm	Valid	80386	mov ecx, 0x12345678 = B9 78 56 34 12
REX.W B8+r iq	MOV r64, imm	Valid	x86-64	mov rcx, 0x1122334455667788 = 48 B9 88 77 66 55 44 33 22 11
C6 /0 ib	MOV r/m8, imm8	Valid	8086	mov byte [rbx], 0x12 = C6 03 12
66 C7 /0 iw	MOV r/m16, imm16	Valid	8086	mov word [rbx], 0x1234 = 66 C7 03 34 12
C7 /0 id	MOV r/m32, imm32	Valid	80386	mov dword [rbx], 0x12345678 = C7 03 78 56 34 12
REX.W C7 /0 id	MOV r/m64, imm32	Valid	x86-64	mov qword [rbx], 0x12345678 = 48 C7 03 78 56 34 12
8C /r	MOV m, Sreg	Valid	8086	mov [rbx], ds = 8C 1B
66 8C /r	MOV r16, Sreg	Valid	8086	mov cx, ds = 66 8C D9
8C /r	MOV r32, Sreg	Valid	80386	mov ecx, ds = 8C D9
8E /r	MOV Sreg, m	Valid	8086	mov ds, [rbx] = 8E 1B
66 8E /r	MOV Sreg, r16	Valid	8086	mov ds, cx = 66 8E D9
8E /r	MOV Sreg, r32	Valid	80386	mov ds, ecx = 8E D9
0F 20 /r	MOV r32, CRn	Invalid	80386 priv	mov ecx, cr0 = 0F 20 C1 (32-bit)
0F 22 /r	MOV CRn, r32	Invalid	80386 priv	mov cr0, ecx = 0F 22 C1 (32-bit)
0F 20 /r	MOV r64, CRn	Valid	x86-64 priv	mov rcx, cr0 = 0F 20 C1
0F 22 /r	MOV CRn, r64	Valid	x86-64 priv	mov cr0, rcx = 0F 22 C1
0F 21 /r	MOV r32, DRn	Invalid	80386 priv	mov ecx, dr1 = 0F 21 C9 (32-bit)

Copyright © 2026 Mallionic Inc. All rights reserved.

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 23 /r`	`MOV DRn, r32`	Invalid	80386 priv	`mov dr1, ecx` = `0F 23 C9` (32-bit)
`0F 21 /r`	`MOV r64, DRn`	Valid	x86-64 priv	`mov rcx, dr1` = `0F 21 C9`
`0F 23 /r`	`MOV DRn, r64`	Valid	x86-64 priv	`mov dr1, rcx` = `0F 23 C9`

Copies the second operand (source) to the first operand (destination). The source may be an immediate, a register or memory; the destination a register or memory; both cannot be memory. Forms with segment, control and debug registers move values to and from those registers (the latter two are privileged). In 64-bit mode `mov r64, imm64` is the only instruction with a 64-bit immediate, and writing a 32-bit register clears the upper half of the 64-bit register. ASH uses `mov` for almost every transfer between variable homes.

Flags affected. None.

A.2.2 MOVZX — Move with Zero-Extend

Opcode	Instruction	64-bit	Since	Example and encoding
`66 0F B6 /r`	`MOVZX r16, r/m8`	Valid	80386	`movzx cx, byte [rbx]` = `66 0F B6 0B`
`0F B6 /r`	`MOVZX r32, r/m8`	Valid	80386	`movzx ecx, byte [rbx]` = `0F B6 0B`
`0F B7 /r`	`MOVZX r32, r/m16`	Valid	80386	`movzx ecx, word [rbx]` = `0F B7 0B`
`REX.W 0F B6 /r`	`MOVZX r64, r/m8`	Valid	x86-64	`movzx rcx, byte [rbx]` = `48 0F B6 0B`
`REX.W 0F B7 /r`	`MOVZX r64, r/m16`	Valid	x86-64	`movzx rcx, word [rbx]` = `48 0F B7 0B`

Copies a byte or word source into a larger register and fills the remaining bits with zeros. ASH uses it to widen unsigned 8-bit and 16-bit values.

Flags affected. None.

A.2.3 MOVSX — Move with Sign-Extend

Opcode	Instruction	64-bit	Since	Example and encoding
`66 0F BE /r`	`MOVSX r16, r/m8`	Valid	80386	`movsx cx, byte [rbx]` = `66 0F BE 0B`
`0F BE /r`	`MOVSX r32, r/m8`	Valid	80386	`movsx ecx, byte [rbx]` = `0F BE 0B`
`0F BF /r`	`MOVSX r32, r/m16`	Valid	80386	`movsx ecx, word [rbx]` = `0F BF 0B`
`REX.W 0F BE /r`	`MOVSX r64, r/m8`	Valid	x86-64	`movsx rcx, byte [rbx]` = `48 0F BE 0B`
`REX.W 0F BF /r`	`MOVSX r64, r/m16`	Valid	x86-64	`movsx rcx, word [rbx]` = `48 0F BF 0B`

Copyright © 2026 Mallionic Inc. All rights reserved.

Copies a byte or word source into a larger register and fills the remaining bits with copies of the sign bit. ASH uses it to widen `int8` and `int16` values.

Flags affected. None.

A.2.4 MOVSXD — Move Doubleword with Sign-Extend

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `movsxd`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Sign-extends a 32-bit source to 64 bits. Used by ASH when an `int32` value is widened.

Flags affected. None.

A.2.5 XCHG — Exchange

Opcode	Instruction	64-bit	Since	Example and encoding
`86 /r`	`XCHG r/m8, r8`	Valid	8086	`xchg byte [rbx], cl` = `86 0B`
`66 87 /r`	`XCHG r/m16, r16`	Valid	8086	`xchg word [rbx], cx` = `66 87 0B`
`87 /r`	`XCHG r/m32, r32`	Valid	80386	`xchg dword [rbx], ecx` = `87 0B`
`66 90+r`	`XCHG AX, r16`	Valid	8086	`xchg ax, cx` = `66 87 C8`
`66 90+r`	`XCHG r16, AX`	Valid	8086	`xchg cx, ax` = `66 87 C1`

Exchanges the contents of the two operands. With a memory operand the exchange is atomic (the processor asserts a lock implicitly), which makes `xchg` a building block of spin locks. `xchg eax, eax` with the one-byte encoding `90` is `nop`.

Flags affected. None.

A.2.6 XADD — Exchange and Add

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `xadd`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Stores the sum of the operands in the destination and the original destination in the source. With a `lock` prefix it is an atomic fetch-and-add.

Flags affected. OF, SF, ZF, AF, CF and PF are set according to the result.

A.2.7 CMPXCHG — Compare and Exchange

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `cmpxchg`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Compares the accumulator (`al`, `ax`, `eax`, `rax`) with the destination. If they are equal, the source is stored in the destination and ZF is set; otherwise the destination is loaded into the accumulator and ZF is cleared. With `lock`, the basis of lock-free algorithms.

Flags affected. OF, SF, ZF, AF, CF and PF are set according to the result.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.8 CMPXCHG8B — Compare and Exchange 8 Bytes

Opcode	Instruction	64-bit	Since	Example and encoding
`0F C7 /1`	`CMPXCHG8B m64`	Valid	Pentium	`cmpxchg8b qword [rbx]` = `0F C7 0B`

Compares `edx:eax` with a 64-bit memory operand; if equal stores `ecx:ebx` there, otherwise loads the operand into `edx:eax`. `cmpxchg16b` is the 128-bit form in 64-bit mode.

Flags affected. ZF is set if the values were equal; other flags unaffected.

A.2.9 BSWAP — Byte Swap

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `bswap`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Reverses the order of the bytes of a 32-bit or 64-bit register, converting between little-endian and big-endian representations.

Flags affected. None.

A.2.10 MOVBE — Move Data After Swapping Bytes

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `movbe`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Loads from or stores to memory while reversing the byte order, combining `mov` and `bswap`.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.11 PUSH — Push onto the Stack

Opcode	Instruction	64-bit	Since	Example and encoding
`66 50+r`	`PUSH r16`	Valid	8086	`push cx` = `66 51`
`66 FF /6`	`PUSH r/m16`	Valid	8086	`push word [rbx]` = `66 FF 33`
`50+r`	`PUSH r32`	Invalid	80386	`push ecx` = `51` (32-bit)
`FF /6`	`PUSH r/m32`	Invalid	80386	`push dword [ebx]` = `FF 33` (32-bit)
`50+r`	`PUSH r64`	Valid	x86-64	`push rcx` = `51`
`FF /6`	`PUSH r/m64`	Valid	x86-64	`push qword [rbx]` = `FF 33`
`6A ib`	`PUSH imm8`	Valid	80186	`push 0x12` = `6A 12`
`66 68 iw`	`PUSH imm16`	Valid	80186	`push 0x1234` = `66 68 34 12`
`68 id`	`PUSH imm32`	Invalid	80386	`push 0x12345678` = `68 78 56 34 12` (32-bit)
`68 id`	`PUSH imm32`	Valid	x86-64	`push 0x12345678` = `68 78 56 34 12`
`06`	`PUSH ES`	Invalid	8086	`push es` = `06` (32-bit)
`0E`	`PUSH CS`	Invalid	8086	`push cs` = `0E` (32-bit)
`16`	`PUSH SS`	Invalid	8086	`push ss` = `16` (32-bit)
`1E`	`PUSH DS`	Invalid	8086	`push ds` = `1E` (32-bit)
`0F A0`	`PUSH FS`	Valid	80386	`push fs` = `0F A0`
`0F A8`	`PUSH GS`	Valid	80386	`push gs` = `0F A8`

Decrements the stack pointer by the operand size and stores the operand at the new top of the stack. In 64-bit mode pushes are 8 bytes (immediates are sign-extended). ASH prologues start with `push rbp`.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.12 POP — Pop from the Stack

Opcode	Instruction	64-bit	Since	Example and encoding
`66 58+r`	`POP r16`	Valid	8086	`pop cx` = `66 59`
`66 8F /0`	`POP r/m16`	Valid	8086	`pop word [rbx]` = `66 8F 03`
`58+r`	`POP r32`	Invalid	80386	`pop ecx` = `59` (32-bit)
`8F /0`	`POP r/m32`	Invalid	80386	`pop dword [ebx]` = `8F 03` (32-bit)
`58+r`	`POP r64`	Valid	x86-64	`pop rcx` = `59`
`8F /0`	`POP r/m64`	Valid	x86-64	`pop qword [rbx]` = `8F 03`
`07`	`POP ES`	Invalid	8086	`pop es` = `07` (32-bit)
`17`	`POP SS`	Invalid	8086	`pop ss` = `17` (32-bit)
`1F`	`POP DS`	Invalid	8086	`pop ds` = `1F` (32-bit)
`0F A1`	`POP FS`	Valid	80386	`pop fs` = `0F A1`
`0F A9`	`POP GS`	Valid	80386	`pop gs` = `0F A9`

Loads the value at the top of the stack into the operand and increments the stack pointer.

Flags affected. None.

A.2.13 PUSHF — Push Flags

Opcode	Instruction	64-bit	Since	Example and encoding
`9C`	`PUSHF`	Valid	8086	`pushf` = `9C`

Pushes the flags register (`pushfq` in 64-bit mode).

Flags affected. None.

A.2.14 POPF — Pop Flags

Opcode	Instruction	64-bit	Since	Example and encoding
`9D`	`POPF`	Valid	8086	`popf` = `9D`

Pops a value into the flags register; which flags may change depends on the privilege level.

Flags affected. All modifiable flags are loaded from the stack.

A.2.15 LEA — Load Effective Address

Opcode	Instruction	64-bit	Since	Example and encoding
`66 8D /r`	`LEA r16, m`	Valid	8086	`lea cx, [rbx]` = `66 8D 0B`
`8D /r`	`LEA r32, m`	Valid	80386	`lea ecx, [rbx]` = `8D 0B`
`REX.W 8D /r`	`LEA r64, m`	Valid	x86-64	`lea rcx, [rbx]` = `48 8D 0B`

Computes the address of the memory operand and stores the address (not the memory contents) in the register. Because it performs `base + index*scale + displacement` without touching the flags, it is also

Copyright © 2026 Mallionic Inc. All rights reserved.

used for arithmetic. ASH uses `lea reg, [rip+disp]` to obtain the address of strings, globals and functions.

Flags affected. None.

A.2.16 CBW — Convert Byte to Word

Opcode	Instruction	64-bit	Since	Example and encoding
`66 98`	`CBW`	Valid	8086	`cbw` = `66 98`

Sign-extends `al` into `ax`. `cwde` sign-extends `ax` into `eax` and `cdqe` `eax` into `rax`.

Flags affected. None.

A.2.17 CWD — Convert Word to Doubleword

Opcode	Instruction	64-bit	Since	Example and encoding
`66 99`	`CWD`	Valid	8086	`cwd` = `66 99`

Sign-extends `ax` into `dx:ax`. `cdq` sign-extends `eax` into `edx:eax` and `cqo` `rax` into `rdx:rax`.

Flags affected. None.

A.2.18 CQO — Convert Quadword to Octoword

Opcode	Instruction	64-bit	Since	Example and encoding
`REX.W 99`	`CQO`	Valid	x86-64	`cqo` = `48 99`

Sign-extends `rax` into `rdx:rax`. ASH emits `cqo` before every signed 64-bit division.

Flags affected. None.

A.2.19 CDQ — Convert Doubleword to Quadword

Opcode	Instruction	64-bit	Since	Example and encoding
`99`	`CDQ`	Valid	80386	`cdq` = `99`

Sign-extends `eax` into `edx:eax`, before a signed 32-bit division.

Flags affected. None.

A.2.20 CDQE — Convert Doubleword to Quadword in RAX

Opcode	Instruction	64-bit	Since	Example and encoding
`REX.W 98`	`CDQE`	Valid	x86-64	`cdqe` = `48 98`

Sign-extends `eax` into `rax`.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.21 CWDE — Convert Word to Doubleword in EAX

Opcode	Instruction	64-bit	Since	Example and encoding
98	CWDE	Valid	80386	cwde = 98

Sign-extends ax into eax.

Flags affected. None.

A.2.22 XLATB — Table Look-up Translation

Opcode	Instruction	64-bit	Since	Example and encoding
D7	XLATB	Valid	8086	xlatb = D7

Replaces al with the byte at [rbx + al] ([ebx+al], [bx+al]).

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.23 ADD — Add

Opcode	Instruction	64-bit	Since	Example and encoding
00 /r	ADD r/m8, r8	Valid	8086	add byte [rbx], cl = 00 0B
66 01 /r	ADD r/m16, r16	Valid	8086	add word [rbx], cx = 66 01 0B
01 /r	ADD r/m32, r32	Valid	80386	add dword [rbx], ecx = 01 0B
REX.W 01 /r	ADD r/m64, r64	Valid	x86-64	add qword [rbx], rcx = 48 01 0B
02 /r	ADD r8, r/m8	Valid	8086	add cl, byte [rbx] = 02 0B
66 03 /r	ADD r16, r/m16	Valid	8086	add cx, word [rbx] = 66 03 0B
03 /r	ADD r32, r/m32	Valid	80386	add ecx, dword [rbx] = 03 0B
REX.W 03 /r	ADD r64, r/m64	Valid	x86-64	add rcx, qword [rbx] = 48 03 0B
04 ib	ADD AL, imm8	Valid	8086	add al, 0x12 = 04 12
80 /0 ib	ADD r/m8, imm8	Valid	8086	add byte [rbx], 0x12 = 80 03 12
66 83 /0 ib	ADD r/m16, imm8	Valid	8086	add word [rbx], 0x12 = 66 83 03 12
66 05 iw	ADD AX, imm16	Valid	8086	add ax, 0x1234 = 66 05 34 12
66 81 /0 iw	ADD r/m16, imm16	Valid	8086	add word [rbx], 0x1234 = 66 81 03 34 12
83 /0 ib	ADD r/m32, imm8	Valid	80386	add dword [rbx], 0x12 = 83 03 12
05 id	ADD EAX, imm32	Valid	80386	add eax, 0x12345678 = 05 78 56 34 12
81 /0 id	ADD r/m32, imm32	Valid	80386	add dword [rbx], 0x12345678 = 81 03 78 56 34 12
REX.W 83 /0 ib	ADD r/m64, imm8	Valid	x86-64	add qword [rbx], 0x12 = 48 83 03 12
REX.W 05 id	ADD RAX, imm32	Valid	x86-64	add rax, 0x12345678 = 48 05 78 56 34 12
REX.W 81 /0 id	ADD r/m64, imm32	Valid	x86-64	add qword [rbx], 0x12345678 = 48 81 03 78 56 34 12

Adds the source to the destination and stores the result in the destination. The same instruction serves signed and unsigned integers: OF signals signed overflow and CF unsigned carry.

Flags affected. OF, SF, ZF, AF, CF and PF are set according to the result.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.24 ADC — Add with Carry

Opcode	Instruction	64-bit	Since	Example and encoding
10 /r	ADC r/m8, r8	Valid	8086	adc byte [rbx], cl = 10 0B
66 11 /r	ADC r/m16, r16	Valid	8086	adc word [rbx], cx = 66 11 0B
11 /r	ADC r/m32, r32	Valid	80386	adc dword [rbx], ecx = 11 0B
REX.W 11 /r	ADC r/m64, r64	Valid	x86-64	adc qword [rbx], rcx = 48 11 0B
12 /r	ADC r8, r/m8	Valid	8086	adc cl, byte [rbx] = 12 0B
66 13 /r	ADC r16, r/m16	Valid	8086	adc cx, word [rbx] = 66 13 0B
13 /r	ADC r32, r/m32	Valid	80386	adc ecx, dword [rbx] = 13 0B
REX.W 13 /r	ADC r64, r/m64	Valid	x86-64	adc rcx, qword [rbx] = 48 13 0B
14 ib	ADC AL, imm8	Valid	8086	adc al, 0x12 = 14 12
80 /2 ib	ADC r/m8, imm8	Valid	8086	adc byte [rbx], 0x12 = 80 13 12
66 83 /2 ib	ADC r/m16, imm8	Valid	8086	adc word [rbx], 0x12 = 66 83 13 12
66 15 iw	ADC AX, imm16	Valid	8086	adc ax, 0x1234 = 66 15 34 12
66 81 /2 iw	ADC r/m16, imm16	Valid	8086	adc word [rbx], 0x1234 = 66 81 13 34 12
83 /2 ib	ADC r/m32, imm8	Valid	80386	adc dword [rbx], 0x12 = 83 13 12
15 id	ADC EAX, imm32	Valid	80386	adc eax, 0x12345678 = 15 78 56 34 12
81 /2 id	ADC r/m32, imm32	Valid	80386	adc dword [rbx], 0x12345678 = 81 13 78 56 34 12
REX.W 83 /2 ib	ADC r/m64, imm8	Valid	x86-64	adc qword [rbx], 0x12 = 48 83 13 12
REX.W 15 id	ADC RAX, imm32	Valid	x86-64	adc rax, 0x12345678 = 48 15 78 56 34 12
REX.W 81 /2 id	ADC r/m64, imm32	Valid	x86-64	adc qword [rbx], 0x12345678 = 48 81 13 78 56 34 12

Adds the source and the carry flag to the destination. Used to add numbers wider than a register: `add` the low parts, then `adc` the high parts.

Flags affected. OF, SF, ZF, AF, CF and PF are set according to the result.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.25 SUB — Subtract

Opcode	Instruction	64-bit	Since	Example and encoding
28 /r	SUB r/m8, r8	Valid	8086	sub byte [rbx], cl = 28 0B
66 29 /r	SUB r/m16, r16	Valid	8086	sub word [rbx], cx = 66 29 0B
29 /r	SUB r/m32, r32	Valid	80386	sub dword [rbx], ecx = 29 0B
REX.W 29 /r	SUB r/m64, r64	Valid	x86-64	sub qword [rbx], rcx = 48 29 0B
2A /r	SUB r8, r/m8	Valid	8086	sub cl, byte [rbx] = 2A 0B
66 2B /r	SUB r16, r/m16	Valid	8086	sub cx, word [rbx] = 66 2B 0B
2B /r	SUB r32, r/m32	Valid	80386	sub ecx, dword [rbx] = 2B 0B
REX.W 2B /r	SUB r64, r/m64	Valid	x86-64	sub rcx, qword [rbx] = 48 2B 0B
2C ib	SUB AL, imm8	Valid	8086	sub al, 0x12 = 2C 12
80 /5 ib	SUB r/m8, imm8	Valid	8086	sub byte [rbx], 0x12 = 80 2B 12
66 83 /5 ib	SUB r/m16, imm8	Valid	8086	sub word [rbx], 0x12 = 66 83 2B 12
66 2D iw	SUB AX, imm16	Valid	8086	sub ax, 0x1234 = 66 2D 34 12
66 81 /5 iw	SUB r/m16, imm16	Valid	8086	sub word [rbx], 0x1234 = 66 81 2B 34 12
83 /5 ib	SUB r/m32, imm8	Valid	80386	sub dword [rbx], 0x12 = 83 2B 12
2D id	SUB EAX, imm32	Valid	80386	sub eax, 0x12345678 = 2D 78 56 34 12
81 /5 id	SUB r/m32, imm32	Valid	80386	sub dword [rbx], 0x12345678 = 81 2B 78 56 34 12
REX.W 83 /5 ib	SUB r/m64, imm8	Valid	x86-64	sub qword [rbx], 0x12 = 48 83 2B 12
REX.W 2D id	SUB RAX, imm32	Valid	x86-64	sub rax, 0x12345678 = 48 2D 78 56 34 12
REX.W 81 /5 id	SUB r/m64, imm32	Valid	x86-64	sub qword [rbx], 0x12345678 = 48 81 2B 78 56 34 12

Subtracts the source from the destination.

Flags affected. OF, SF, ZF, AF, CF and PF are set according to the result.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.26 SBB — Subtract with Borrow

Opcode	Instruction	64-bit	Since	Example and encoding
18 /r	SBB r/m8, r8	Valid	8086	sbb byte [rbx], cl = 18 0B
66 19 /r	SBB r/m16, r16	Valid	8086	sbb word [rbx], cx = 66 19 0B
19 /r	SBB r/m32, r32	Valid	80386	sbb dword [rbx], ecx = 19 0B
REX.W 19 /r	SBB r/m64, r64	Valid	x86-64	sbb qword [rbx], rcx = 48 19 0B
1A /r	SBB r8, r/m8	Valid	8086	sbb cl, byte [rbx] = 1A 0B
66 1B /r	SBB r16, r/m16	Valid	8086	sbb cx, word [rbx] = 66 1B 0B
1B /r	SBB r32, r/m32	Valid	80386	sbb ecx, dword [rbx] = 1B 0B
REX.W 1B /r	SBB r64, r/m64	Valid	x86-64	sbb rcx, qword [rbx] = 48 1B 0B
1C ib	SBB AL, imm8	Valid	8086	sbb al, 0x12 = 1C 12
80 /3 ib	SBB r/m8, imm8	Valid	8086	sbb byte [rbx], 0x12 = 80 1B 12
66 83 /3 ib	SBB r/m16, imm8	Valid	8086	sbb word [rbx], 0x12 = 66 83 1B 12
66 1D iw	SBB AX, imm16	Valid	8086	sbb ax, 0x1234 = 66 1D 34 12
66 81 /3 iw	SBB r/m16, imm16	Valid	8086	sbb word [rbx], 0x1234 = 66 81 1B 34 12
83 /3 ib	SBB r/m32, imm8	Valid	80386	sbb dword [rbx], 0x12 = 83 1B 12
1D id	SBB EAX, imm32	Valid	80386	sbb eax, 0x12345678 = 1D 78 56 34 12
81 /3 id	SBB r/m32, imm32	Valid	80386	sbb dword [rbx], 0x12345678 = 81 1B 78 56 34 12
REX.W 83 /3 ib	SBB r/m64, imm8	Valid	x86-64	sbb qword [rbx], 0x12 = 48 83 1B 12
REX.W 1D id	SBB RAX, imm32	Valid	x86-64	sbb rax, 0x12345678 = 48 1D 78 56 34 12
REX.W 81 /3 id	SBB r/m64, imm32	Valid	x86-64	sbb qword [rbx], 0x12345678 = 48 81 1B 78 56 34 12

Subtracts the source and the carry flag from the destination; the counterpart of `adc` for multi-word subtraction. `sbb reg, reg` produces 0 or -1 from CF.

Flags affected. OF, SF, ZF, AF, CF and PF are set according to the result.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.27 INC — Increment by 1

Opcode	Instruction	64-bit	Since	Example and encoding
`66 40+r`	`INC r16`	Invalid	8086	`inc cx` = `66 41` (32-bit)
`40+r`	`INC r32`	Invalid	80386	`inc ecx` = `41` (32-bit)
`FE /0`	`INC r/m8`	Valid	8086	`inc byte [rbx]` = `FE 03`
`66 FF /0`	`INC r/m16`	Valid	8086	`inc word [rbx]` = `66 FF 03`
`FF /0`	`INC r/m32`	Valid	80386	`inc dword [rbx]` = `FF 03`
`REX.W FF /0`	`INC r/m64`	Valid	x86-64	`inc qword [rbx]` = `48 FF 03`

Adds 1 to the operand without changing the carry flag. ASHIR `inc x` becomes this or an `add`.

Flags affected. CF is not affected; OF, SF, ZF, AF and PF are set according to the result.

A.2.28 DEC — Decrement by 1

Opcode	Instruction	64-bit	Since	Example and encoding
`66 48+r`	`DEC r16`	Invalid	8086	`dec cx` = `66 49` (32-bit)
`48+r`	`DEC r32`	Invalid	80386	`dec ecx` = `49` (32-bit)
`FE /1`	`DEC r/m8`	Valid	8086	`dec byte [rbx]` = `FE 0B`
`66 FF /1`	`DEC r/m16`	Valid	8086	`dec word [rbx]` = `66 FF 0B`
`FF /1`	`DEC r/m32`	Valid	80386	`dec dword [rbx]` = `FF 0B`
`REX.W FF /1`	`DEC r/m64`	Valid	x86-64	`dec qword [rbx]` = `48 FF 0B`

Subtracts 1 from the operand without changing the carry flag.

Flags affected. CF is not affected; OF, SF, ZF, AF and PF are set according to the result.

A.2.29 NEG — Two's Complement Negation

Opcode	Instruction	64-bit	Since	Example and encoding
`F6 /3`	`NEG r/m8`	Valid	8086	`neg byte [rbx]` = `F6 1B`
`66 F7 /3`	`NEG r/m16`	Valid	8086	`neg word [rbx]` = `66 F7 1B`
`F7 /3`	`NEG r/m32`	Valid	80386	`neg dword [rbx]` = `F7 1B`
`REX.W F7 /3`	`NEG r/m64`	Valid	x86-64	`neg qword [rbx]` = `48 F7 1B`

Replaces the operand with 0 minus the operand.

Flags affected. CF is cleared if the operand was 0 and set otherwise; OF, SF, ZF, AF and PF according to the result.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.30 CMP — Compare Two Operands

Opcode	Instruction	64-bit	Since	Example and encoding
38 /r	CMP r/m8, r8	Valid	8086	cmp byte [rbx], cl = 38 0B
66 39 /r	CMP r/m16, r16	Valid	8086	cmp word [rbx], cx = 66 39 0B
39 /r	CMP r/m32, r32	Valid	80386	cmp dword [rbx], ecx = 39 0B
REX.W 39 /r	CMP r/m64, r64	Valid	x86-64	cmp qword [rbx], rcx = 48 39 0B
3A /r	CMP r8, r/m8	Valid	8086	cmp cl, byte [rbx] = 3A 0B
66 3B /r	CMP r16, r/m16	Valid	8086	cmp cx, word [rbx] = 66 3B 0B
3B /r	CMP r32, r/m32	Valid	80386	cmp ecx, dword [rbx] = 3B 0B
REX.W 3B /r	CMP r64, r/m64	Valid	x86-64	cmp rcx, qword [rbx] = 48 3B 0B
3C ib	CMP AL, imm8	Valid	8086	cmp al, 0x12 = 3C 12
80 /7 ib	CMP r/m8, imm8	Valid	8086	cmp byte [rbx], 0x12 = 80 3B 12
66 83 /7 ib	CMP r/m16, imm8	Valid	8086	cmp word [rbx], 0x12 = 66 83 3B 12
66 3D iw	CMP AX, imm16	Valid	8086	cmp ax, 0x1234 = 66 3D 34 12
66 81 /7 iw	CMP r/m16, imm16	Valid	8086	cmp word [rbx], 0x1234 = 66 81 3B 34 12
83 /7 ib	CMP r/m32, imm8	Valid	80386	cmp dword [rbx], 0x12 = 83 3B 12
3D id	CMP EAX, imm32	Valid	80386	cmp eax, 0x12345678 = 3D 78 56 34 12
81 /7 id	CMP r/m32, imm32	Valid	80386	cmp dword [rbx], 0x12345678 = 81 3B 78 56 34 12
REX.W 83 /7 ib	CMP r/m64, imm8	Valid	x86-64	cmp qword [rbx], 0x12 = 48 83 3B 12
REX.W 3D id	CMP RAX, imm32	Valid	x86-64	cmp rax, 0x12345678 = 48 3D 78 56 34 12
REX.W 81 /7 id	CMP r/m64, imm32	Valid	x86-64	cmp qword [rbx], 0x12345678 = 48 81 3B 78 56 34 12

Subtracts the second operand from the first, sets the flags like `sub` and discards the result. Followed by a conditional jump, `setcc` or `cmovcc`. Every ASH condition becomes a `cmp`.

Flags affected. OF, SF, ZF, AF, CF and PF are set according to the result.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.31 MUL — Unsigned Multiply

Opcode	Instruction	64-bit	Since	Example and encoding
F6 /4	MUL r/m8	Valid	8086	mul byte [rbx] = F6 23
66 F7 /4	MUL r/m16	Valid	8086	mul word [rbx] = 66 F7 23
F7 /4	MUL r/m32	Valid	80386	mul dword [rbx] = F7 23
REX.W F7 /4	MUL r/m64	Valid	x86-64	mul qword [rbx] = 48 F7 23

Multiplies the accumulator by the operand as unsigned integers; the double-width product is stored in `ah:al`, `dx:ax`, `edx:eax` or `rdx:rax`.

Flags affected. OF and CF are set if the upper half of the product is not zero; SF, ZF, AF and PF are undefined.

A.2.32 IMUL — Signed Multiply

Opcode	Instruction	64-bit	Since	Example and encoding
F6 /5	IMUL r/m8	Valid	8086	imul byte [rbx] = F6 2B
66 F7 /5	IMUL r/m16	Valid	8086	imul word [rbx] = 66 F7 2B
F7 /5	IMUL r/m32	Valid	80386	imul dword [rbx] = F7 2B
REX.W F7 /5	IMUL r/m64	Valid	x86-64	imul qword [rbx] = 48 F7 2B
66 0F AF /r	IMUL r16, r/m16	Valid	80386	imul cx, word [rbx] = 66 0F AF 0B
0F AF /r	IMUL r32, r/m32	Valid	80386	imul ecx, dword [rbx] = 0F AF 0B
REX.W 0F AF /r	IMUL r64, r/m64	Valid	x86-64	imul rcx, qword [rbx] = 48 0F AF 0B
66 6B /r ib	IMUL r16, r/m16, imm8	Valid	80186	imul cx, word [rbx], 0x12 = 66 6B 0B 12
66 69 /r iw	IMUL r16, r/m16, imm16	Valid	80186	imul cx, word [rbx], 0x1234 = 66 69 0B 34 12
6B /r ib	IMUL r32, r/m32, imm8	Valid	80386	imul ecx, dword [rbx], 0x12 = 6B 0B 12
69 /r id	IMUL r32, r/m32, imm32	Valid	80386	imul ecx, dword [rbx], 0x12345678 = 69 0B 78 56 34 12
REX.W 6B /r ib	IMUL r64, r/m64, imm8	Valid	x86-64	imul rcx, qword [rbx], 0x12 = 48 6B 0B 12
REX.W 69 /r id	IMUL r64, r/m64, imm32	Valid	x86-64	imul rcx, qword [rbx], 0x12345678 = 48 69 0B 78 56 34 12

Multiplies signed integers. The one-operand form works like `mul`; the two-operand form `imul r, r/m` and the three-operand form `imul r, r/m, imm` keep only the low half of the product, which is the same for signed and unsigned operands. ASH uses the two and three-operand forms for `*`.

Flags affected. OF and CF are set when the result had to be truncated; SF, ZF, AF and PF are undefined.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.33 DIV — Unsigned Divide

Opcode	Instruction	64-bit	Since	Example and encoding
F6 /6	DIV r/m8	Valid	8086	div byte [rbx] = F6 33
66 F7 /6	DIV r/m16	Valid	8086	div word [rbx] = 66 F7 33
F7 /6	DIV r/m32	Valid	80386	div dword [rbx] = F7 33
REX.W F7 /6	DIV r/m64	Valid	x86-64	div qword [rbx] = 48 F7 33

Divides `ax`, `dx:ax`, `edx:eax` or `rdx:rax` by the operand as unsigned integers; the quotient goes to `al`/`ax`/`eax`/`rax` and the remainder to `ah`/`dx`/`edx`/`rdx`. Division by zero or a quotient too large raises the divide error (#DE), reported by Linux as SIGFPE.

Flags affected. All arithmetic flags are undefined.

A.2.34 IDIV — Signed Divide

Opcode	Instruction	64-bit	Since	Example and encoding
F6 /7	IDIV r/m8	Valid	8086	idiv byte [rbx] = F6 3B
66 F7 /7	IDIV r/m16	Valid	8086	idiv word [rbx] = 66 F7 3B
F7 /7	IDIV r/m32	Valid	80386	idiv dword [rbx] = F7 3B
REX.W F7 /7	IDIV r/m64	Valid	x86-64	idiv qword [rbx] = 48 F7 3B

Signed counterpart of `div`; the quotient is truncated toward zero and the remainder has the sign of the dividend. ASH emits `cqo` followed by `idiv` for `/` and `%`.

Flags affected. All arithmetic flags are undefined.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.35 AND — Logical AND

Opcode	Instruction	64-bit	Since	Example and encoding
20 /r	AND r/m8, r8	Valid	8086	and byte [rbx], cl = 20 0B
66 21 /r	AND r/m16, r16	Valid	8086	and word [rbx], cx = 66 21 0B
21 /r	AND r/m32, r32	Valid	80386	and dword [rbx], ecx = 21 0B
REX.W 21 /r	AND r/m64, r64	Valid	x86-64	and qword [rbx], rcx = 48 21 0B
22 /r	AND r8, r/m8	Valid	8086	and cl, byte [rbx] = 22 0B
66 23 /r	AND r16, r/m16	Valid	8086	and cx, word [rbx] = 66 23 0B
23 /r	AND r32, r/m32	Valid	80386	and ecx, dword [rbx] = 23 0B
REX.W 23 /r	AND r64, r/m64	Valid	x86-64	and rcx, qword [rbx] = 48 23 0B
24 ib	AND AL, imm8	Valid	8086	and al, 0x12 = 24 12
80 /4 ib	AND r/m8, imm8	Valid	8086	and byte [rbx], 0x12 = 80 23 12
66 83 /4 ib	AND r/m16, imm8	Valid	8086	and word [rbx], 0x12 = 66 83 23 12
66 25 iw	AND AX, imm16	Valid	8086	and ax, 0x1234 = 66 25 34 12
66 81 /4 iw	AND r/m16, imm16	Valid	8086	and word [rbx], 0x1234 = 66 81 23 34 12
83 /4 ib	AND r/m32, imm8	Valid	80386	and dword [rbx], 0x12 = 83 23 12
25 id	AND EAX, imm32	Valid	80386	and eax, 0x12345678 = 25 78 56 34 12
81 /4 id	AND r/m32, imm32	Valid	80386	and dword [rbx], 0x12345678 = 81 23 78 56 34 12
REX.W 83 /4 ib	AND r/m64, imm8	Valid	x86-64	and qword [rbx], 0x12 = 48 83 23 12
REX.W 25 id	AND RAX, imm32	Valid	x86-64	and rax, 0x12345678 = 48 25 78 56 34 12
REX.W 81 /4 id	AND r/m64, imm32	Valid	x86-64	and qword [rbx], 0x12345678 = 48 81 23 78 56 34 12

Bitwise AND of the operands, stored in the destination.

Flags affected. OF and CF are cleared; SF, ZF and PF are set according to the result; AF is undefined.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.36 OR — Logical Inclusive OR

Opcode	Instruction	64-bit	Since	Example and encoding
08 /r	OR r/m8, r8	Valid	8086	or byte [rbx], cl = 08 0B
66 09 /r	OR r/m16, r16	Valid	8086	or word [rbx], cx = 66 09 0B
09 /r	OR r/m32, r32	Valid	80386	or dword [rbx], ecx = 09 0B
REX.W 09 /r	OR r/m64, r64	Valid	x86-64	or qword [rbx], rcx = 48 09 0B
0A /r	OR r8, r/m8	Valid	8086	or cl, byte [rbx] = 0A 0B
66 0B /r	OR r16, r/m16	Valid	8086	or cx, word [rbx] = 66 0B 0B
0B /r	OR r32, r/m32	Valid	80386	or ecx, dword [rbx] = 0B 0B
REX.W 0B /r	OR r64, r/m64	Valid	x86-64	or rcx, qword [rbx] = 48 0B 0B
0C ib	OR AL, imm8	Valid	8086	or al, 0x12 = 0C 12
80 /1 ib	OR r/m8, imm8	Valid	8086	or byte [rbx], 0x12 = 80 0B 12
66 83 /1 ib	OR r/m16, imm8	Valid	8086	or word [rbx], 0x12 = 66 83 0B 12
66 0D iw	OR AX, imm16	Valid	8086	or ax, 0x1234 = 66 0D 34 12
66 81 /1 iw	OR r/m16, imm16	Valid	8086	or word [rbx], 0x1234 = 66 81 0B 34 12
83 /1 ib	OR r/m32, imm8	Valid	80386	or dword [rbx], 0x12 = 83 0B 12
0D id	OR EAX, imm32	Valid	80386	or eax, 0x12345678 = 0D 78 56 34 12
81 /1 id	OR r/m32, imm32	Valid	80386	or dword [rbx], 0x12345678 = 81 0B 78 56 34 12
REX.W 83 /1 ib	OR r/m64, imm8	Valid	x86-64	or qword [rbx], 0x12 = 48 83 0B 12
REX.W 0D id	OR RAX, imm32	Valid	x86-64	or rax, 0x12345678 = 48 0D 78 56 34 12
REX.W 81 /1 id	OR r/m64, imm32	Valid	x86-64	or qword [rbx], 0x12345678 = 48 81 0B 78 56 34 12

Bitwise OR of the operands.

Flags affected. OF and CF are cleared; SF, ZF and PF are set according to the result; AF is undefined.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.37 XOR — Logical Exclusive OR

Opcode	Instruction	64-bit	Since	Example and encoding
`30 /r`	`XOR r/m8, r8`	Valid	8086	`xor byte [rbx], cl` = `30 0B`
`66 31 /r`	`XOR r/m16, r16`	Valid	8086	`xor word [rbx], cx` = `66 31 0B`
`31 /r`	`XOR r/m32, r32`	Valid	80386	`xor dword [rbx], ecx` = `31 0B`
`REX.W 31 /r`	`XOR r/m64, r64`	Valid	x86-64	`xor qword [rbx], rcx` = `48 31 0B`
`32 /r`	`XOR r8, r/m8`	Valid	8086	`xor cl, byte [rbx]` = `32 0B`
`66 33 /r`	`XOR r16, r/m16`	Valid	8086	`xor cx, word [rbx]` = `66 33 0B`
`33 /r`	`XOR r32, r/m32`	Valid	80386	`xor ecx, dword [rbx]` = `33 0B`
`REX.W 33 /r`	`XOR r64, r/m64`	Valid	x86-64	`xor rcx, qword [rbx]` = `48 33 0B`
`34 ib`	`XOR AL, imm8`	Valid	8086	`xor al, 0x12` = `34 12`
`80 /6 ib`	`XOR r/m8, imm8`	Valid	8086	`xor byte [rbx], 0x12` = `80 33 12`
`66 83 /6 ib`	`XOR r/m16, imm8`	Valid	8086	`xor word [rbx], 0x12` = `66 83 33 12`
`66 35 iw`	`XOR AX, imm16`	Valid	8086	`xor ax, 0x1234` = `66 35 34 12`
`66 81 /6 iw`	`XOR r/m16, imm16`	Valid	8086	`xor word [rbx], 0x1234` = `66 81 33 34 12`
`83 /6 ib`	`XOR r/m32, imm8`	Valid	80386	`xor dword [rbx], 0x12` = `83 33 12`
`35 id`	`XOR EAX, imm32`	Valid	80386	`xor eax, 0x12345678` = `35 78 56 34 12`
`81 /6 id`	`XOR r/m32, imm32`	Valid	80386	`xor dword [rbx], 0x12345678` = `81 33 78 56 34 12`
`REX.W 83 /6 ib`	`XOR r/m64, imm8`	Valid	x86-64	`xor qword [rbx], 0x12` = `48 83 33 12`
`REX.W 35 id`	`XOR RAX, imm32`	Valid	x86-64	`xor rax, 0x12345678` = `48 35 78 56 34 12`
`REX.W 81 /6 id`	`XOR r/m64, imm32`	Valid	x86-64	`xor qword [rbx], 0x12345678` = `48 81 33 78 56 34 12`

Bitwise exclusive OR of the operands. `xor reg, reg` is the idiomatic way to set a register to zero; ASH uses `xor eax, eax` for `return 0`.

Flags affected. OF and CF are cleared; SF, ZF and PF are set according to the result; AF is undefined.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.38 NOT — One's Complement Negation

Opcode	Instruction	64-bit	Since	Example and encoding
F6 /2	NOT r/m8	Valid	8086	not byte [rbx] = F6 13
66 F7 /2	NOT r/m16	Valid	8086	not word [rbx] = 66 F7 13
F7 /2	NOT r/m32	Valid	80386	not dword [rbx] = F7 13
REX.W F7 /2	NOT r/m64	Valid	x86-64	not qword [rbx] = 48 F7 13

Inverts every bit of the operand.

Flags affected. None.

A.2.39 TEST — Logical Compare

Opcode	Instruction	64-bit	Since	Example and encoding
84 /r	TEST r/m8, r8	Valid	8086	test byte [rbx], cl = 84 0B
66 85 /r	TEST r/m16, r16	Valid	8086	test word [rbx], cx = 66 85 0B
85 /r	TEST r/m32, r32	Valid	80386	test dword [rbx], ecx = 85 0B
REX.W 85 /r	TEST r/m64, r64	Valid	x86-64	test qword [rbx], rcx = 48 85 0B
A8 ib	TEST AL, imm8	Valid	8086	test al, 0x12 = A8 12
66 A9 iw	TEST AX, imm16	Valid	8086	test ax, 0x1234 = 66 A9 34 12
A9 id	TEST EAX, imm32	Valid	80386	test eax, 0x12345678 = A9 78 56 34 12
REX.W A9 id	TEST RAX, imm32	Valid	x86-64	test rax, 0x12345678 = 48 A9 78 56 34 12
F6 /0 ib	TEST r/m8, imm8	Valid	8086	test byte [rbx], 0x12 = F6 03 12
66 F7 /0 iw	TEST r/m16, imm16	Valid	8086	test word [rbx], 0x1234 = 66 F7 03 34 12
F7 /0 id	TEST r/m32, imm32	Valid	80386	test dword [rbx], 0x12345678 = F7 03 78 56 34 12
REX.W F7 /0 id	TEST r/m64, imm32	Valid	x86-64	test qword [rbx], 0x12345678 = 48 F7 03 78 56 34 12

Computes the AND of the operands, sets the flags and discards the result. `test reg, reg` checks a register for zero or negative.

Flags affected. OF and CF are cleared; SF, ZF and PF are set according to the result; AF is undefined.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.40 SHL — Shift Left

Opcode	Instruction	64-bit	Since	Example and encoding
D0 /4	SHL r/m8, 1	Valid	8086	shl byte [rbx], 1 = D0 23
D2 /4	SHL r/m8, CL	Valid	8086	shl byte [rbx], cl = D2 23
C0 /4 ib	SHL r/m8, imm8	Valid	80186	shl byte [rbx], 0x12 = C0 23 12
66 D1 /4	SHL r/m16, 1	Valid	8086	shl word [rbx], 1 = 66 D1 23
66 D3 /4	SHL r/m16, CL	Valid	8086	shl word [rbx], cl = 66 D3 23
66 C1 /4 ib	SHL r/m16, imm8	Valid	80186	shl word [rbx], 0x12 = 66 C1 23 12
D1 /4	SHL r/m32, 1	Valid	80386	shl dword [rbx], 1 = D1 23
D3 /4	SHL r/m32, CL	Valid	80386	shl dword [rbx], cl = D3 23
C1 /4 ib	SHL r/m32, imm8	Valid	80386	shl dword [rbx], 0x12 = C1 23 12
REX.W D1 /4	SHL r/m64, 1	Valid	x86-64	shl qword [rbx], 1 = 48 D1 23
REX.W D3 /4	SHL r/m64, CL	Valid	x86-64	shl qword [rbx], cl = 48 D3 23
REX.W C1 /4 ib	SHL r/m64, imm8	Valid	x86-64	shl qword [rbx], 0x12 = 48 C1 23 12

Shifts the destination left by 1, by `cl` or by an immediate; zeros enter from the right and the last bit shifted out goes to CF. The count is masked to 5 bits (6 for 64-bit operands). `sal` is the same instruction.

Flags affected. CF receives the last bit shifted out; OF is defined only for 1-bit shifts; SF, ZF and PF according to the result.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.41 SAL — Shift Arithmetic Left

Opcode	Instruction	64-bit	Since	Example and encoding
D0 /4	SAL r/m8, 1	Valid	8086	sal byte [rbx], 1 = D0 23
D2 /4	SAL r/m8, CL	Valid	8086	sal byte [rbx], cl = D2 23
C0 /4 ib	SAL r/m8, imm8	Valid	80186	sal byte [rbx], 0x12 = C0 23 12
66 D1 /4	SAL r/m16, 1	Valid	8086	sal word [rbx], 1 = 66 D1 23
66 D3 /4	SAL r/m16, CL	Valid	8086	sal word [rbx], cl = 66 D3 23
66 C1 /4 ib	SAL r/m16, imm8	Valid	80186	sal word [rbx], 0x12 = 66 C1 23 12
D1 /4	SAL r/m32, 1	Valid	80386	sal dword [rbx], 1 = D1 23
D3 /4	SAL r/m32, CL	Valid	80386	sal dword [rbx], cl = D3 23
C1 /4 ib	SAL r/m32, imm8	Valid	80386	sal dword [rbx], 0x12 = C1 23 12
REX.W D1 /4	SAL r/m64, 1	Valid	x86-64	sal qword [rbx], 1 = 48 D1 23
REX.W D3 /4	SAL r/m64, CL	Valid	x86-64	sal qword [rbx], cl = 48 D3 23
REX.W C1 /4 ib	SAL r/m64, imm8	Valid	x86-64	sal qword [rbx], 0x12 = 48 C1 23 12

Synonym of `shl`.

Flags affected. As for SHL.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.42 SHR — Shift Logical Right

Opcode	Instruction	64-bit	Since	Example and encoding
D0 /5	SHR r/m8, 1	Valid	8086	shr byte [rbx], 1 = D0 2B
D2 /5	SHR r/m8, CL	Valid	8086	shr byte [rbx], cl = D2 2B
C0 /5 ib	SHR r/m8, imm8	Valid	80186	shr byte [rbx], 0x12 = C0 2B 12
66 D1 /5	SHR r/m16, 1	Valid	8086	shr word [rbx], 1 = 66 D1 2B
66 D3 /5	SHR r/m16, CL	Valid	8086	shr word [rbx], cl = 66 D3 2B
66 C1 /5 ib	SHR r/m16, imm8	Valid	80186	shr word [rbx], 0x12 = 66 C1 2B 12
D1 /5	SHR r/m32, 1	Valid	80386	shr dword [rbx], 1 = D1 2B
D3 /5	SHR r/m32, CL	Valid	80386	shr dword [rbx], cl = D3 2B
C1 /5 ib	SHR r/m32, imm8	Valid	80386	shr dword [rbx], 0x12 = C1 2B 12
REX.W D1 /5	SHR r/m64, 1	Valid	x86-64	shr qword [rbx], 1 = 48 D1 2B
REX.W D3 /5	SHR r/m64, CL	Valid	x86-64	shr qword [rbx], cl = 48 D3 2B
REX.W C1 /5 ib	SHR r/m64, imm8	Valid	x86-64	shr qword [rbx], 0x12 = 48 C1 2B 12

Shifts right, filling with zeros: unsigned division by a power of two. ASH uses it for >> on unsigned operands.

Flags affected. As for SHL.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.43 SAR — Shift Arithmetic Right

Opcode	Instruction	64-bit	Since	Example and encoding
D0 /7	SAR r/m8, 1	Valid	8086	sar byte [rbx], 1 = D0 3B
D2 /7	SAR r/m8, CL	Valid	8086	sar byte [rbx], cl = D2 3B
C0 /7 ib	SAR r/m8, imm8	Valid	80186	sar byte [rbx], 0x12 = C0 3B 12
66 D1 /7	SAR r/m16, 1	Valid	8086	sar word [rbx], 1 = 66 D1 3B
66 D3 /7	SAR r/m16, CL	Valid	8086	sar word [rbx], cl = 66 D3 3B
66 C1 /7 ib	SAR r/m16, imm8	Valid	80186	sar word [rbx], 0x12 = 66 C1 3B 12
D1 /7	SAR r/m32, 1	Valid	80386	sar dword [rbx], 1 = D1 3B
D3 /7	SAR r/m32, CL	Valid	80386	sar dword [rbx], cl = D3 3B
C1 /7 ib	SAR r/m32, imm8	Valid	80386	sar dword [rbx], 0x12 = C1 3B 12
REX.W D1 /7	SAR r/m64, 1	Valid	x86-64	sar qword [rbx], 1 = 48 D1 3B
REX.W D3 /7	SAR r/m64, CL	Valid	x86-64	sar qword [rbx], cl = 48 D3 3B
REX.W C1 /7 ib	SAR r/m64, imm8	Valid	x86-64	sar qword [rbx], 0x12 = 48 C1 3B 12

Shifts right, filling with copies of the sign bit: signed division by a power of two rounded toward minus infinity. ASH uses it for >> on signed operands.

Flags affected. As for SHL.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.44 ROL — Rotate Left

Opcode	Instruction	64-bit	Since	Example and encoding
D0 /0	ROL r/m8, 1	Valid	8086	rol byte [rbx], 1 = D0 03
D2 /0	ROL r/m8, CL	Valid	8086	rol byte [rbx], cl = D2 03
C0 /0 ib	ROL r/m8, imm8	Valid	80186	rol byte [rbx], 0x12 = C0 03 12
66 D1 /0	ROL r/m16, 1	Valid	8086	rol word [rbx], 1 = 66 D1 03
66 D3 /0	ROL r/m16, CL	Valid	8086	rol word [rbx], cl = 66 D3 03
66 C1 /0 ib	ROL r/m16, imm8	Valid	80186	rol word [rbx], 0x12 = 66 C1 03 12
D1 /0	ROL r/m32, 1	Valid	80386	rol dword [rbx], 1 = D1 03
D3 /0	ROL r/m32, CL	Valid	80386	rol dword [rbx], cl = D3 03
C1 /0 ib	ROL r/m32, imm8	Valid	80386	rol dword [rbx], 0x12 = C1 03 12
REX.W D1 /0	ROL r/m64, 1	Valid	x86-64	rol qword [rbx], 1 = 48 D1 03
REX.W D3 /0	ROL r/m64, CL	Valid	x86-64	rol qword [rbx], cl = 48 D3 03
REX.W C1 /0 ib	ROL r/m64, imm8	Valid	x86-64	rol qword [rbx], 0x12 = 48 C1 03 12

Rotates the bits of the destination left; bits leaving on the left enter on the right.

Flags affected. CF receives the last bit rotated; OF is defined only for 1-bit rotations; other flags unaffected.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.45 ROR — Rotate Right

Opcode	Instruction	64-bit	Since	Example and encoding
D0 /1	ROR r/m8, 1	Valid	8086	ror byte [rbx], 1 = D0 0B
D2 /1	ROR r/m8, CL	Valid	8086	ror byte [rbx], cl = D2 0B
C0 /1 ib	ROR r/m8, imm8	Valid	80186	ror byte [rbx], 0x12 = C0 0B 12
66 D1 /1	ROR r/m16, 1	Valid	8086	ror word [rbx], 1 = 66 D1 0B
66 D3 /1	ROR r/m16, CL	Valid	8086	ror word [rbx], cl = 66 D3 0B
66 C1 /1 ib	ROR r/m16, imm8	Valid	80186	ror word [rbx], 0x12 = 66 C1 0B 12
D1 /1	ROR r/m32, 1	Valid	80386	ror dword [rbx], 1 = D1 0B
D3 /1	ROR r/m32, CL	Valid	80386	ror dword [rbx], cl = D3 0B
C1 /1 ib	ROR r/m32, imm8	Valid	80386	ror dword [rbx], 0x12 = C1 0B 12
REX.W D1 /1	ROR r/m64, 1	Valid	x86-64	ror qword [rbx], 1 = 48 D1 0B
REX.W D3 /1	ROR r/m64, CL	Valid	x86-64	ror qword [rbx], cl = 48 D3 0B
REX.W C1 /1 ib	ROR r/m64, imm8	Valid	x86-64	ror qword [rbx], 0x12 = 48 C1 0B 12

Rotates the bits of the destination right.

Flags affected. As for ROL.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.46 RCL — Rotate Through Carry Left

Opcode	Instruction	64-bit	Since	Example and encoding
D0 /2	RCL r/m8, 1	Valid	8086	rcl byte [rbx], 1 = D0 13
D2 /2	RCL r/m8, CL	Valid	8086	rcl byte [rbx], cl = D2 13
C0 /2 ib	RCL r/m8, imm8	Valid	80186	rcl byte [rbx], 0x12 = C0 13 12
66 D1 /2	RCL r/m16, 1	Valid	8086	rcl word [rbx], 1 = 66 D1 13
66 D3 /2	RCL r/m16, CL	Valid	8086	rcl word [rbx], cl = 66 D3 13
66 C1 /2 ib	RCL r/m16, imm8	Valid	80186	rcl word [rbx], 0x12 = 66 C1 13 12
D1 /2	RCL r/m32, 1	Valid	80386	rcl dword [rbx], 1 = D1 13
D3 /2	RCL r/m32, CL	Valid	80386	rcl dword [rbx], cl = D3 13
C1 /2 ib	RCL r/m32, imm8	Valid	80386	rcl dword [rbx], 0x12 = C1 13 12
REX.W D1 /2	RCL r/m64, 1	Valid	x86-64	rcl qword [rbx], 1 = 48 D1 13
REX.W D3 /2	RCL r/m64, CL	Valid	x86-64	rcl qword [rbx], cl = 48 D3 13
REX.W C1 /2 ib	RCL r/m64, imm8	Valid	x86-64	rcl qword [rbx], 0x12 = 48 C1 13 12

Rotates left through the carry flag (a rotation of N+1 bits).

Flags affected. As for ROL.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.47 RCR — Rotate Through Carry Right

Opcode	Instruction	64-bit	Since	Example and encoding
D0 /3	RCR r/m8, 1	Valid	8086	rcr byte [rbx], 1 = D0 1B
D2 /3	RCR r/m8, CL	Valid	8086	rcr byte [rbx], cl = D2 1B
C0 /3 ib	RCR r/m8, imm8	Valid	80186	rcr byte [rbx], 0x12 = C0 1B 12
66 D1 /3	RCR r/m16, 1	Valid	8086	rcr word [rbx], 1 = 66 D1 1B
66 D3 /3	RCR r/m16, CL	Valid	8086	rcr word [rbx], cl = 66 D3 1B
66 C1 /3 ib	RCR r/m16, imm8	Valid	80186	rcr word [rbx], 0x12 = 66 C1 1B 12
D1 /3	RCR r/m32, 1	Valid	80386	rcr dword [rbx], 1 = D1 1B
D3 /3	RCR r/m32, CL	Valid	80386	rcr dword [rbx], cl = D3 1B
C1 /3 ib	RCR r/m32, imm8	Valid	80386	rcr dword [rbx], 0x12 = C1 1B 12
REX.W D1 /3	RCR r/m64, 1	Valid	x86-64	rcr qword [rbx], 1 = 48 D1 1B
REX.W D3 /3	RCR r/m64, CL	Valid	x86-64	rcr qword [rbx], cl = 48 D3 1B
REX.W C1 /3 ib	RCR r/m64, imm8	Valid	x86-64	rcr qword [rbx], 0x12 = 48 C1 1B 12

Rotates right through the carry flag.

Flags affected. As for ROL.

A.2.48 SHLD — Double Precision Shift Left

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `shld`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Shifts the destination left, filling the vacated bits from the most significant bits of the source register.

Flags affected. CF receives the last bit shifted out; SF, ZF, PF according to the result.

A.2.49 SHRD — Double Precision Shift Right

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `shrd`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Shifts the destination right, filling from the low bits of the source.

Flags affected. As for SHLD.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.50 BT — Bit Test

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `bt`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Copies the selected bit of the first operand into CF.

Flags affected. CF receives the bit; other flags undefined or unaffected.

A.2.51 BTS — Bit Test and Set

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `bts`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Copies the selected bit into CF and sets it to 1.

Flags affected. As for BT.

A.2.52 BTR — Bit Test and Reset

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `btr`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Copies the selected bit into CF and clears it.

Flags affected. As for BT.

A.2.53 BTC — Bit Test and Complement

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `btc`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Copies the selected bit into CF and inverts it.

Flags affected. As for BT.

A.2.54 BSF — Bit Scan Forward

Opcode	Instruction	64-bit	Since	Example and encoding
`66 0F BC /r`	`BSF r16, r/m16`	Valid	80386	`bsf cx, word [rbx]` = `66 0F BC 0B`
`0F BC /r`	`BSF r32, r/m32`	Valid	80386	`bsf ecx, dword [rbx]` = `0F BC 0B`
`REX.W 0F BC /r`	`BSF r64, r/m64`	Valid	x86-64	`bsf rcx, qword [rbx]` = `48 0F BC 0B`

Stores the index of the lowest set bit of the source. If the source is zero, ZF is set and the destination is undefined.

Flags affected. ZF is set if the source is zero; other flags undefined.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.55 BSR — Bit Scan Reverse

Opcode	Instruction	64-bit	Since	Example and encoding
`66 0F BD /r`	`BSR r16, r/m16`	Valid	80386	`bsr cx, word [rbx]` = `66 0F BD 0B`
`0F BD /r`	`BSR r32, r/m32`	Valid	80386	`bsr ecx, dword [rbx]` = `0F BD 0B`
`REX.W 0F BD /r`	`BSR r64, r/m64`	Valid	x86-64	`bsr rcx, qword [rbx]` = `48 0F BD 0B`

Stores the index of the highest set bit of the source (the integer base-2 logarithm, as computed by `ilog2` in `<math>`).

Flags affected. As for BSF.

A.2.56 POPCNT — Population Count

Opcode	Instruction	64-bit	Since	Example and encoding
`66 F3 0F B8 /r`	`POPCNT r16, r/m16`	Valid		`popcnt cx, word [rbx]` = `66 F3 0F B8 0B`
`F3 0F B8 /r`	`POPCNT r32, r/m32`	Valid		`popcnt ecx, dword [rbx]` = `F3 0F B8 0B`
`REX.W F3 0F B8 /r`	`POPCNT r64, r/m64`	Valid		`popcnt rcx, qword [rbx]` = `F3 48 0F B8 0B`

Counts the bits set to 1 in the source (the operation of `popcount` in `<math>`, in one instruction).

Flags affected. ZF is set if the source is zero; OF, SF, AF, CF, PF are cleared.

A.2.57 LZCNT — Count Leading Zero Bits

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `lzcnt`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Counts the zero bits above the highest set bit. On processors without LZCNT the encoding executes as `bsr`.

Flags affected. CF is set if the source is zero, ZF if the result is zero.

A.2.58 TZCNT — Count Trailing Zero Bits

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `tzcnt`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Counts the zero bits below the lowest set bit.

Flags affected. CF is set if the source is zero, ZF if the result is zero.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.59 CRC32 — Accumulate CRC32 Value

Opcode	Instruction	64-bit	Since	Example and encoding
`REX.W F2 0F 38 F0 /r`	`CRC32 r64, r/m8`	Valid	SSE4.2	`crc32 rcx, byte [rbx]` = `F2 48 0F 38 F0 0B`

Updates a CRC-32C (Castagnoli) checksum in the destination with the source byte, word, doubleword or quadword.

Flags affected. None.

A.2.60 JMP — Jump

Opcode	Instruction	64-bit	Since	Example and encoding
`EB cb`	`JMP rel8`	Valid	8086	`jmp short $+0x10` = `EB 0E`
`E9 cd`	`JMP rel32`	Valid	8086	`jmp near $+0x100` = `E9 FB 00 00 00`
`E9 cd`	`JMP rel (assembler chooses)`	Valid	8086	`jmp $+0x10` = `EB 0E`
`66 FF /4`	`JMP r/m16`	Valid	8086	`jmp word [rbx]` = `66 FF 23`
`FF /4`	`JMP r/m32`	Invalid	80386	`jmp dword [ebx]` = `FF 23` (32-bit)
`FF /4`	`JMP r/m64`	Valid	x86-64	`jmp qword [rbx]` = `FF 23`

Transfers control to the target: relative (`jmp rel8`, `jmp rel32`), indirect through a register or memory, or far. ASH uses `jmp` for the end of loop bodies and for `else` branches.

Flags affected. None.

A.2.61 Jcc — Jump if Condition Is Met

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 80+cc cd`	`Jcc rel32`	Valid	80386	`je near $+0x100` = `0F 84 FA 00 00 00`
`70+cc cb`	`Jcc rel8`	Valid	8086	`je short $+0x10` = `74 0E`
`0F 80+cc cd`	`Jcc rel (assembler chooses)`	Valid	80386	`je $+0x10` = `74 0E`

Jumps if the condition holds, otherwise continues with the next instruction. The conditions test the flags: `jo`/`jno` (OF), `jb`/`jc`/`jnae` and `jae`/`jnb`/`jnc` (CF), `je`/`jz` and `jne`/`jnz` (ZF), `jbe`/`jna` and `ja`/`jnbe` (CF or ZF), `js`/`jns` (SF), `jp`/`jpe` and `jnp`/`jpo` (PF), `jl`/`jnge` and `jge`/`jnl` (SF != OF), `jle`/`jng` and `jg`/`jnle` (ZF or SF != OF). The short form takes an 8-bit displacement, the near form a 32-bit one. ASH uses the signed forms for signed operands and the unsigned forms (`jb`, `ja` ...) for unsigned ones.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.62 JCXZ — Jump if CX Is Zero

Opcode	Instruction	64-bit	Since	Example and encoding
`67 E3 cb`	`JCXZ rel (assembler chooses)`	Valid	8086	

Jumps if `cx` (`jecxz`: `ecx`, `jrcxz`: `rcx`) is zero, without testing flags.

Flags affected. None.

A.2.63 LOOP — Loop According to ECX Counter

Opcode	Instruction	64-bit	Since	Example and encoding
`E2 cb`	`LOOP rel (assembler chooses)`	Valid	8086	`loop $+0x10` = `E2 0E`

Decrements `rcx` (`ecx`, `cx`) and jumps if it is not zero. `loope`/`loopz` and `loopne`/`loopnz` also test ZF.

Flags affected. None.

A.2.64 CALL — Call Procedure

Opcode	Instruction	64-bit	Since	Example and encoding
`E8 cd`	`CALL rel32`	Valid	8086	`call near $+0x100` = `E8 FB 00 00 00`
`E8 cd`	`CALL rel (assembler chooses)`	Valid	8086	`call $+0x10` = `E8 0B 00 00 00`
`66 FF /2`	`CALL r/m16`	Valid	8086	`call word [rbx]` = `66 FF 13`
`FF /2`	`CALL r/m32`	Invalid	80386	`call dword [ebx]` = `FF 13` (32-bit)
`FF /2`	`CALL r/m64`	Valid	x86-64	`call qword [rbx]` = `FF 13`

Pushes the address of the next instruction and jumps to the target. ASH calls functions with `call rel32` (resolved by the object writer or by a relocation) and calls through pointers with `call rax`.

Flags affected. None.

A.2.65 RET — Return from Procedure

Opcode	Instruction	64-bit	Since	Example and encoding
`C3`	`RET`	Valid	8086	`ret` = `C3`
`C2 iw`	`RET imm16`	Valid	8086	`ret 0x1234` = `C2 34 12`

Pops the return address and jumps to it. `ret imm16` also removes *imm16* bytes of arguments (the callee-cleanup conventions).

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.66 ENTER — Make Stack Frame

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `enter`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Creates a stack frame: `push rbp; mov rbp, rsp; sub rsp, n`, with optional nesting levels. Slower than the explicit sequence, which is what ASH emits.

Flags affected. None.

A.2.67 LEAVE — High Level Procedure Exit

Opcode	Instruction	64-bit	Since	Example and encoding
`C9`	`LEAVE`	Valid	80186	`leave` = `C9`

Releases a frame: `mov rsp, rbp; pop rbp`. Used by ASH epilogues.

Flags affected. None.

A.2.68 INT — Call to Interrupt Procedure

Opcode	Instruction	64-bit	Since	Example and encoding
`CD ib`	`INT imm8`	Valid	8086	`int 0x12` = `CD 12`

Raises software interrupt *n*: in real mode it calls the BIOS or DOS service in the interrupt vector table (`int 0x10` video, `int 0x13` disk, `int 0x21` DOS); in 32-bit Linux `int 0x80` is the system call interface. `int3` (`CC`) is the breakpoint instruction.

Flags affected. None.

A.2.69 INT3 — Breakpoint

Opcode	Instruction	64-bit	Since	Example and encoding
`CC`	`INT3`	Valid	8086	`int3` = `CC`

The one-byte breakpoint `CC`, used by debuggers.

Flags affected. None.

A.2.70 INTO — Interrupt on Overflow

Opcode	Instruction	64-bit	Since	Example and encoding
`CE`	`INTO`	Invalid	8086	`into` = `CE` (32-bit)

Raises interrupt 4 if OF is set. Not valid in 64-bit mode.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.71 IRET — Interrupt Return

Opcode	Instruction	64-bit	Since	Example and encoding
`CF`	`IRET`	Valid	8086	`iret` = `CF`

Returns from an interrupt or exception handler, restoring the instruction pointer, code segment and flags (and the stack in 64-bit mode, `iretq`). The last instruction of a `@naked` interrupt handler.

Flags affected. None.

A.2.72 SETcc — Set Byte on Condition

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 90+cc /0`	`SETcc r/m8`	Valid	80386	`sete byte [rbx]` = `0F 94 03`

Stores 1 in the byte operand if the condition holds and 0 otherwise. The conditions are those of `jcc`. A way to turn a comparison into a value without branching.

Flags affected. None.

A.2.73 CMOVcc — Conditional Move

Opcode	Instruction	64-bit	Since	Example and encoding
`66 0F 40+cc /r`	`CMOVcc r16, r/m16`	Valid	P6	`cmove cx, word [rbx]` = `66 0F 44 0B`
`0F 40+cc /r`	`CMOVcc r32, r/m32`	Valid	P6	`cmove ecx, dword [rbx]` = `0F 44 0B`
`REX.W 0F 40+cc /r`	`CMOVcc r64, r/m64`	Valid	x86-64	`cmove rcx, qword [rbx]` = `48 0F 44 0B`

Moves the source into the destination register if the condition holds. The conditions are those of `jcc`. Used to compute minimum and maximum without branches.

Flags affected. None.

A.2.74 CLC — Clear Carry Flag

Opcode	Instruction	64-bit	Since	Example and encoding
`F8`	`CLC`	Valid	8086	`clc` = `F8`

Sets CF to 0.

Flags affected. CF = 0.

A.2.75 STC — Set Carry Flag

Opcode	Instruction	64-bit	Since	Example and encoding
`F9`	`STC`	Valid	8086	`stc` = `F9`

Sets CF to 1.

Copyright © 2026 Mallionic Inc. All rights reserved.

Flags affected. CF = 1.

A.2.76 CMC — Complement Carry Flag

Opcode	Instruction	64-bit	Since	Example and encoding
F5	CMC	Valid	8086	cmc = F5

Inverts CF.

Flags affected. CF is inverted.

A.2.77 CLD — Clear Direction Flag

Opcode	Instruction	64-bit	Since	Example and encoding
FC	CLD	Valid	8086	cld = FC

Clears DF: string instructions advance (the state required by the System V ABI at function boundaries).

Flags affected. DF = 0.

A.2.78 STD — Set Direction Flag

Opcode	Instruction	64-bit	Since	Example and encoding
FD	STD	Valid	8086	std = FD

Sets DF: string instructions go backwards.

Flags affected. DF = 1.

A.2.79 CLI — Clear Interrupt Flag

Opcode	Instruction	64-bit	Since	Example and encoding
FA	CLI	Valid	8086	cli = FA

Disables maskable hardware interrupts (privileged in protected mode). Boot code typically starts with `cli` while it sets up segments and the stack.

Flags affected. IF = 0.

A.2.80 STI — Set Interrupt Flag

Opcode	Instruction	64-bit	Since	Example and encoding
FB	STI	Valid	8086	sti = FB

Enables maskable interrupts after the next instruction.

Flags affected. IF = 1.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.81 LAHF — Load Status Flags into AH

Opcode	Instruction	64-bit	Since	Example and encoding
`9F`	`LAHF`	Valid	8086	`lahf` = `9F`

Copies SF, ZF, AF, PF and CF into `ah`.

Flags affected. None.

A.2.82 SAHF — Store AH into Flags

Opcode	Instruction	64-bit	Since	Example and encoding
`9E`	`SAHF`	Valid	8086	`sahf` = `9E`

Loads SF, ZF, AF, PF and CF from `ah`.

Flags affected. SF, ZF, AF, PF and CF are loaded from AH.

A.2.83 NOP — No Operation

Opcode	Instruction	64-bit	Since	Example and encoding
`90`	`NOP`	Valid	8086	`nop` = `90`

Does nothing. Besides the one-byte `90`, multi-byte forms (`0F 1F /0`) exist for alignment padding; `shasm` pads code with `nop` in `align`.

Flags affected. None.

A.2.84 PAUSE — Spin Loop Hint

Opcode	Instruction	64-bit	Since	Example and encoding
`F3 90`	`PAUSE`	Valid	8086	`pause` = `F3 90`

Tells the processor that the code is a spin-wait loop, saving power and avoiding memory-order penalties.

Flags affected. None.

A.2.85 HLT — Halt

Opcode	Instruction	64-bit	Since	Example and encoding
`F4`	`HLT`	Valid	8086 priv	`hlt` = `F4`

Stops the processor until the next interrupt. Privileged. A boot loader ends with a `hlt` loop.

Flags affected. None.

A.2.86 UD2 — Undefined Instruction

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 0B`	`UD2`	Valid	80186	`ud2` = `0F 0B`

Copyright © 2026 Mallionic Inc. All rights reserved.

Raises the invalid-opcode exception on purpose; used to mark unreachable code.

Flags affected. None.

A.2.87 CPUID — CPU Identification

Opcode	Instruction	64-bit	Since	Example and encoding
`0F A2`	`CPUID`	Valid	Pentium	`cpuid` = `0F A2`

Returns processor identification and feature information in `eax`, `ebx`, `ecx` and `edx` for the leaf selected by `eax` (and sub-leaf in `ecx`).

Flags affected. None.

A.2.88 RDTSC — Read Time-Stamp Counter

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 31`	`RDTSC`	Valid	Pentium	`rdtsc` = `0F 31`

Loads the 64-bit time-stamp counter into `edx:eax`. Used in Example 20-3 to measure elapsed cycles.

Flags affected. None.

A.2.89 RDTSCP — Read Time-Stamp Counter and Processor ID

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 01 F9`	`RDTSCP`	Valid	x86-64	`rdtscp` = `0F 01 F9`

Like `rdtsc`, waiting for previous instructions, and also loads `IA32_TSC_AUX` into `ecx`.

Flags affected. None.

A.2.90 RDRAND — Read Random Number

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `rdrand`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Loads a hardware random number into the register.

Flags affected. CF = 1 if a value was available; other arithmetic flags cleared.

A.2.91 RDSEED — Read Random Seed

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `rdseed`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Loads a random seed from the hardware entropy source.

Flags affected. As for RDRAND.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.92 MFENCE — Memory Fence

Opcode	Instruction	64-bit	Since	Example and encoding
`0F AE F0`	`MFENCE`	Valid	x86-64	`mfence` = `0F AE F0`

Orders all loads and stores issued before it with respect to those after it.

Flags affected. None.

A.2.93 LFENCE — Load Fence

Opcode	Instruction	64-bit	Since	Example and encoding
`0F AE E8`	`LFENCE`	Valid	x86-64	`lfence` = `0F AE E8`

Orders loads, and serializes instruction execution with respect to prior instructions.

Flags affected. None.

A.2.94 SFENCE — Store Fence

Opcode	Instruction	64-bit	Since	Example and encoding
`0F AE F8`	`SFENCE`	Valid	x86-64	`sfence` = `0F AE F8`

Orders stores.

Flags affected. None.

A.2.95 CLFLUSH — Flush Cache Line

Opcode	Instruction	64-bit	Since	Example and encoding
`0F AE /7`	`CLFLUSH m`	Valid	SSE2	`clflush [rbx]` = `0F AE 3B`

Writes back and invalidates the cache line containing the address.

Flags affected. None.

A.2.96 PREFETCHT0 — Prefetch Data into Caches

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 18 /1`	`PREFETCHT0 m8`	Valid	SSE	`prefetcht0 byte [rbx]` = `0F 18 0B`

Hints the processor to load the cache line into all levels of the cache (`prefetcht1`, `prefetcht2` and `prefetchnta` select other levels).

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.97 MOVSB — Move String Byte

Opcode	Instruction	64-bit	Since	Example and encoding
A4	MOVSB	Valid	8086	movsb = A4

Copies the byte at [rsi] to [rdi] and advances both by 1 (backwards if DF is set). movsw, movsd and movsq move 2, 4 and 8 bytes. With rep, repeats rcx times: rep movsb is a complete memcpy.

Flags affected. None.

A.2.98 STOSB — Store String Byte

Opcode	Instruction	64-bit	Since	Example and encoding
AA	STOSB	Valid	8086	stosb = AA

Stores al at [rdi] and advances rdi. With rep, fills rcx bytes: a complete memset. stosw, stosd, stosq store ax, eax, rax.

Flags affected. None.

A.2.99 LODSB — Load String Byte

Opcode	Instruction	64-bit	Since	Example and encoding
AC	LODSB	Valid	8086	lodsb = AC

Loads the byte at [rsi] into al and advances rsi. Used in the boot sector of Example 26-6 to walk a string.

Flags affected. None.

A.2.100 SCASB — Scan String Byte

Opcode	Instruction	64-bit	Since	Example and encoding
AE	SCASB	Valid	8086	scasb = AE

Compares al with the byte at [rdi] and advances rdi. With repne, searches for a byte: the core of strlen and memchr.

Flags affected. OF, SF, ZF, AF, CF and PF are set according to the result.

A.2.101 CMPSB — Compare String Bytes

Opcode	Instruction	64-bit	Since	Example and encoding
A6	CMPSB	Valid	8086	cmpsb = A6

Compares [rsi] with [rdi] and advances both. With repe, compares two memory blocks: the core of memcmp.

Flags affected. OF, SF, ZF, AF, CF and PF are set according to the result.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.2.102 IN — Input from Port

Opcode	Instruction	64-bit	Since	Example and encoding
`E4 ib`	`IN AL, imm8`	Valid	8086	`in al, 0x12` = `E4 12`
`66 E5 ib`	`IN AX, imm8`	Valid	8086	`in ax, 0x12` = `66 E5 12`
`E5 ib`	`IN EAX, imm8`	Valid	80386	`in eax, 0x12` = `E5 12`
`EC`	`IN AL, DX`	Valid	8086	`in al, dx` = `EC`
`66 ED`	`IN AX, DX`	Valid	8086	`in ax, dx` = `66 ED`
`ED`	`IN EAX, DX`	Valid	80386	`in eax, dx` = `ED`

Reads a byte, word or doubleword from an I/O port, given as an immediate (0-255) or in `dx`, into the accumulator. Privileged unless the I/O permission allows it.

Flags affected. None.

A.2.103 OUT — Output to Port

Opcode	Instruction	64-bit	Since	Example and encoding
`E6 ib`	`OUT imm8, AL`	Valid	8086	`out 0x12, al` = `E6 12`
`66 E7 ib`	`OUT imm8, AX`	Valid	8086	`out 0x12, ax` = `66 E7 12`
`E7 ib`	`OUT imm8, EAX`	Valid	80386	`out 0x12, eax` = `E7 12`
`EE`	`OUT DX, AL`	Valid	8086	`out dx, al` = `EE`
`66 EF`	`OUT DX, AX`	Valid	8086	`out dx, ax` = `66 EF`
`EF`	`OUT DX, EAX`	Valid	80386	`out dx, eax` = `EF`

Writes the accumulator to an I/O port. Used by kernels to program devices such as the PIC, the PIT and the serial port.

Flags affected. None.

A.2.104 INSB — Input String from Port

Opcode	Instruction	64-bit	Since	Example and encoding
`6C`	`INSB`	Valid	80186	`insb` = `6C`

Reads from port `dx` into `[rdi]` and advances `rdi`; with `rep`, reads a block.

Flags affected. None.

A.2.105 OUTSB — Output String to Port

Opcode	Instruction	64-bit	Since	Example and encoding
`6E`	`OUTSB`	Valid	80186	`outsb` = `6E`

Writes `[rsi]` to port `dx` and advances `rsi`.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.3 SYSTEM INSTRUCTIONS

Instructions used by operating systems and boot code. Most are privileged: they fault at privilege level 3.

A.3.1 SYSCALL — Fast System Call

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 05`	`SYSCALL`	Valid	x86-64	`syscall` = `0F 05`

Enters the operating system kernel in 64-bit mode: the return address is saved in `rcx` and the flags in `r11`. Linux takes the system call number in `rax` and the arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`, and returns the result in `rax`. Every function of `<syscalls>` ends in a `syscall`.

Flags affected. None.

A.3.2 SYSRET — Return from Fast System Call

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 07`	`SYSRET`	Valid	x86-64 priv	`sysret` = `0F 07`

Returns from the kernel to user mode, restoring `rip` from `rcx` and the flags from `r11`. Privileged.

Flags affected. None.

A.3.3 SYSENTER — Fast System Call (32-bit)

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 34`	`SYSENTER`	Valid	P6	`sysenter` = `0F 34`

Fast entry to the kernel in 32-bit mode, using model-specific registers for the target.

Flags affected. None.

A.3.4 SYSEXIT — Fast Return from SYSENTER

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 35`	`SYSEXIT`	Valid	P6 priv	`sysexit` = `0F 35`

Returns to user mode after `sysenter`. Privileged.

Flags affected. None.

A.3.5 LGDT — Load Global Descriptor Table Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 01 /2`	`LGDT m`	Valid	80286 priv	`lgdt [rbx]` = `0F 01 13`

Loads the base and limit of the GDT from memory. Executed by boot loaders before entering protected or long mode. Privileged.

Copyright © 2026 Mallionic Inc. All rights reserved.

Flags affected. None.

A.3.6 LIDT — Load Interrupt Descriptor Table Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 01 /3`	`LIDT m`	Valid	80286 priv	`lidt [rbx]` = `0F 01 1B`

Loads the base and limit of the IDT. Privileged.

Flags affected. None.

A.3.7 SGDT — Store Global Descriptor Table Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 01 /0`	`SGDT m`	Valid	80286	`sgdt [rbx]` = `0F 01 03`

Stores the GDT base and limit to memory.

Flags affected. None.

A.3.8 SIDT — Store Interrupt Descriptor Table Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 01 /1`	`SIDT m`	Valid	80286	`sidt [rbx]` = `0F 01 0B`

Stores the IDT base and limit to memory.

Flags affected. None.

A.3.9 LLDT — Load Local Descriptor Table Register

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `lldt`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Loads the LDT selector. Privileged.

Flags affected. None.

A.3.10 LTR — Load Task Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 00 /3`	`LTR r/m16`	Valid	80286 priv	`ltr word [rbx]` = `0F 00 1B`

Loads the task register (TSS selector). Privileged.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.3.11 INVLPG — Invalidate TLB Entry

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 01 /7`	`INVLPG m`	Valid	80486 priv	`invlpg [rbx]` = `0F 01 3B`

Invalidates the translation lookaside buffer entry of a page after the kernel changes its page table entry. Privileged.

Flags affected. None.

A.3.12 WBINVD — Write Back and Invalidate Cache

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 09`	`WBINVD`	Valid	80486 priv	`wbinvd` = `0F 09`

Writes back all modified cache lines and invalidates the caches. Privileged.

Flags affected. None.

A.3.13 RDMSR — Read from Model Specific Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 32`	`RDMSR`	Valid	Pentium priv	`rdmsr` = `0F 32`

Reads the MSR selected by `ecx` into `edx:eax`. Privileged.

Flags affected. None.

A.3.14 WRMSR — Write to Model Specific Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 30`	`WRMSR`	Valid	Pentium priv	`wrmsr` = `0F 30`

Writes `edx:eax` to the MSR selected by `ecx`. Privileged.

Flags affected. None.

A.3.15 RDPMC — Read Performance-Monitoring Counter

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 33`	`RDPMC`	Valid	P6	`rdpmc` = `0F 33`

Reads the counter selected by `ecx` into `edx:eax`.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.3.16 CLTS — Clear Task-Switched Flag in CR0

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 06`	`CLTS`	Valid	80286 priv	`clts` = `0F 06`

Clears CR0.TS. Privileged.

Flags affected. None.

A.3.17 SWAPGS — Swap GS Base Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 01 F8`	`SWAPGS`	Valid	x86-64	`swapgs` = `0F 01 F8`

Exchanges the GS base with the `IA32_KERNEL_GS_BASE` MSR, at kernel entry and exit in 64-bit mode. Privileged.

Flags affected. None.

A.3.18 LMSW — Load Machine Status Word

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 01 /6`	`LMSW r/m16`	Valid	80286 priv	`lmsw word [rbx]` = `0F 01 33`

Loads the low four bits of CR0 (legacy way to enter protected mode).

Flags affected. None.

A.3.19 SMSW — Store Machine Status Word

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 01 /4`	`SMSW r/m16`	Valid	80286	`smsw word [rbx]` = `0F 01 23`

Stores the low bits of CR0.

Flags affected. None.

A.3.20 ARPL — Adjust RPL Field of Segment Selector

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `arpl`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Raises the requested privilege level of a selector. Not valid in 64-bit mode.

Flags affected. None.

A.3.21 LAR — Load Access Rights Byte

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `lar`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Loads the access rights of a segment descriptor.

Copyright © 2026 Mallionic Inc. All rights reserved.

Flags affected. ZF is set if the selector is valid.

A.3.22 LSL — Load Segment Limit

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `lsl`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Loads the limit of a segment descriptor.

Flags affected. ZF is set if the selector is valid.

A.3.23 VERR — Verify a Segment for Reading

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `verr`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Sets ZF if the segment can be read at the current privilege level.

Flags affected. ZF as described.

A.3.24 VERW — Verify a Segment for Writing

Availability. The instruction database of `shasm` 1.2.0 contains no encodable form of `verw`: it is rejected in `shasm` sources and (silently) in `asm` blocks. Encode it with `emit` if you need it.

Sets ZF if the segment can be written.

Flags affected. ZF as described.

A.4 X87 FLOATING POINT INSTRUCTIONS

Instructions of the x87 floating-point unit, which works on a stack of eight 80-bit registers `st(0)-st(7)`. In memory forms, `m32`, `m64` and `m80` are single, double and extended precision values.

A.4.1 FLD — Load Floating-Point Value

Opcode	Instruction	64-bit	Since	Example and encoding
`D9 /0`	`FLD m32`	Valid	8086	`fld dword [rbx]` = `D9 03`
`DD /0`	`FLD m64`	Valid	8086	`fld qword [rbx]` = `DD 03`
`DB /5`	`FLD m80`	Valid	8086	`fld tword [rbx]` = `DB 2B`
`D9 C0+r`	`FLD ST(i)`	Valid	8086	`fld st1` = `D9 C1`

Pushes a 32, 64 or 80-bit floating-point value (or `st(i)`) onto the x87 register stack.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.4.2 FST — Store Floating-Point Value

Opcode	Instruction	64-bit	Since	Example and encoding
`D9 /2`	`FST m32`	Valid	8086	`fst dword [rbx]` = `D9 13`
`DD /2`	`FST m64`	Valid	8086	`fst qword [rbx]` = `DD 13`
`DD D0+r`	`FST ST(i)`	Valid	8086	`fst st1` = `DD D1`

Stores `st(0)` to memory or to `st(i)`. `fstp` also pops the stack.

Flags affected. None.

A.4.3 FSTP — Store Floating-Point Value and Pop

Opcode	Instruction	64-bit	Since	Example and encoding
`D9 /3`	`FSTP m32`	Valid	8086	`fstp dword [rbx]` = `D9 1B`
`DD /3`	`FSTP m64`	Valid	8086	`fstp qword [rbx]` = `DD 1B`
`DB /7`	`FSTP m80`	Valid	8086	`fstp tword [rbx]` = `DB 3B`
`DD D8+r`	`FSTP ST(i)`	Valid	8086	`fstp st1` = `DD D9`

Stores `st(0)` and pops the register stack.

Flags affected. None.

A.4.4 FILD — Load Integer

Opcode	Instruction	64-bit	Since	Example and encoding
`DB /0`	`FILD m32`	Valid	8086	`fild dword [rbx]` = `DB 03`
`DF /0`	`FILD m16`	Valid	8086	`fild word [rbx]` = `DF 03`
`DF /5`	`FILD m64`	Valid	8086	`fild qword [rbx]` = `DF 2B`

Converts a 16, 32 or 64-bit integer to floating point and pushes it.

Flags affected. None.

A.4.5 FISTP — Store Integer and Pop

Opcode	Instruction	64-bit	Since	Example and encoding
`DB /3`	`FISTP m32`	Valid	8086	`fistp dword [rbx]` = `DB 1B`
`DF /3`	`FISTP m16`	Valid	8086	`fistp word [rbx]` = `DF 1B`
`DF /7`	`FISTP m64`	Valid	8086	`fistp qword [rbx]` = `DF 3B`

Converts `st(0)` to an integer with the current rounding mode, stores it and pops.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.4.6 FADD — Add

Opcode	Instruction	64-bit	Since	Example and encoding
D8 /0	FADD m32	Valid	8086	fadd dword [rbx] = D8 03
DC /0	FADD m64	Valid	8086	fadd qword [rbx] = DC 03
DC C0+r	FADD TO ST(i)	Valid	8086	fadd to st1 = DC C1
D8 C0+r	FADD ST(i)	Valid	8086	fadd st1 = D8 C1
DC C0+r	FADD ST(i), ST(0)	Valid	8086	fadd st1, st0 = DC C1
D8 C0+r	FADD ST(0), ST(i)	Valid	8086	fadd st0, st1 = D8 C1

Adds floating-point values: `st(0) += m`, `st(0) += st(i)` or `st(i) += st(0)`; `faddp` pops.

Flags affected. None.

A.4.7 FSUB — Subtract

Opcode	Instruction	64-bit	Since	Example and encoding
D8 /4	FSUB m32	Valid	8086	fsub dword [rbx] = D8 23
DC /4	FSUB m64	Valid	8086	fsub qword [rbx] = DC 23
DC E8+r	FSUB TO ST(i)	Valid	8086	fsub to st1 = DC E9
DC E8+r	FSUB ST(i), ST(0)	Valid	8086	fsub st1, st0 = DC E9
D8 E0+r	FSUB ST(i)	Valid	8086	fsub st1 = D8 E1
D8 E0+r	FSUB ST(0), ST(i)	Valid	8086	fsub st0, st1 = D8 E1

Subtracts floating-point values; `fsubr` reverses the operands.

Flags affected. None.

A.4.8 FMUL — Multiply

Opcode	Instruction	64-bit	Since	Example and encoding
D8 /1	FMUL m32	Valid	8086	fmul dword [rbx] = D8 0B
DC /1	FMUL m64	Valid	8086	fmul qword [rbx] = DC 0B
DC C8+r	FMUL TO ST(i)	Valid	8086	fmul to st1 = DC C9
DC C8+r	FMUL ST(i), ST(0)	Valid	8086	fmul st1, st0 = DC C9
D8 C8+r	FMUL ST(i)	Valid	8086	fmul st1 = D8 C9
D8 C8+r	FMUL ST(0), ST(i)	Valid	8086	fmul st0, st1 = D8 C9

Multiplies floating-point values.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.4.9 FDIV — Divide

Opcode	Instruction	64-bit	Since	Example and encoding
D8 /6	FDIV m32	Valid	8086	fdiv dword [rbx] = D8 33
DC /6	FDIV m64	Valid	8086	fdiv qword [rbx] = DC 33
DC F8+r	FDIV TO ST(i)	Valid	8086	fdiv to st1 = DC F9
D8 F0+r	FDIV ST(i)	Valid	8086	fdiv st1 = D8 F1
DC F8+r	FDIV ST(i), ST(0)	Valid	8086	fdiv st1, st0 = DC F9
D8 F0+r	FDIV ST(0), ST(i)	Valid	8086	fdiv st0, st1 = D8 F1

Divides floating-point values; `fdivr` reverses the operands.

Flags affected. None.

A.4.10 FCHS — Change Sign

Opcode	Instruction	64-bit	Since	Example and encoding
D9 E0	FCHS	Valid	8086	fchs = D9 E0

Negates `st(0)`.

Flags affected. None.

A.4.11 FABS — Absolute Value

Opcode	Instruction	64-bit	Since	Example and encoding
D9 E1	FABS	Valid	8086	fabs = D9 E1

Clears the sign of `st(0)`.

Flags affected. None.

A.4.12 FSQRT — Square Root

Opcode	Instruction	64-bit	Since	Example and encoding
D9 FA	FSQRT	Valid	8086	fsqrt = D9 FA

Replaces `st(0)` with its square root.

Flags affected. None.

A.4.13 FSIN — Sine

Opcode	Instruction	64-bit	Since	Example and encoding
D9 FE	FSIN	Valid	80386	fsin = D9 FE

Replaces `st(0)` (radians) with its sine. Compare with the FPU-free `fx_sin` of `<math>`.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.4.14 FCOS — Cosine

Opcode	Instruction	64-bit	Since	Example and encoding
D9 FF	FCOS	Valid	80386	fcos = D9 FF

Replaces st(0) with its cosine.

Flags affected. None.

A.4.15 FXCH — Exchange Register Contents

Opcode	Instruction	64-bit	Since	Example and encoding
D9 C8+r	FXCH ST(i)	Valid	8086	fxch st1 = D9 C9
D9 C8+r	FXCH ST(i), ST(0)	Valid	8086	fxch st1, st0 = D9 C9
D9 C8+r	FXCH ST(0), ST(i)	Valid	8086	fxch st0, st1 = D9 C9

Exchanges st(0) and st(i).

Flags affected. None.

A.4.16 FCOM — Compare Floating-Point Values

Opcode	Instruction	64-bit	Since	Example and encoding
D8 /2	FCOM m32	Valid	8086	fcom dword [rbx] = D8 13
DC /2	FCOM m64	Valid	8086	fcom qword [rbx] = DC 13
D8 D0+r	FCOM ST(i)	Valid	8086	fcom st1 = D8 D1
D8 D0+r	FCOM ST(0), ST(i)	Valid	8086	fcom st0, st1 = D8 D1

Compares st(0) with the operand and sets the x87 condition codes.

Flags affected. None.

A.4.17 FCOMI — Compare and Set EFLAGS

Opcode	Instruction	64-bit	Since	Example and encoding
DB F0+r	FCOMI ST(i)	Valid	P6	fcomi st1 = DB F1
DB F0+r	FCOMI ST(0), ST(i)	Valid	P6	fcomi st0, st1 = DB F1

Compares st(0) with st(i) and sets ZF, PF and CF directly, so that ordinary conditional jumps can follow.

Flags affected. ZF, PF, CF as for an unsigned comparison.

A.4.18 FINIT — Initialize FPU

Opcode	Instruction	64-bit	Since	Example and encoding
9B DB E3	FINIT	Valid	8086	finit = 9B DB E3

Resets the x87 unit (fninit without waiting for pending exceptions).

Copyright © 2026 Mallionic Inc. All rights reserved.

Flags affected. None.

A.4.19 FLDZ — Load +0.0

Opcode	Instruction	64-bit	Since	Example and encoding
`D9 EE`	`FLDZ`	Valid	8086	`fldz` = `D9 EE`

Pushes 0.0. `fld1` pushes 1.0, `fldpi` pushes pi.

Flags affected. None.

A.4.20 FLD1 — Load +1.0

Opcode	Instruction	64-bit	Since	Example and encoding
`D9 E8`	`FLD1`	Valid	8086	`fld1` = `D9 E8`

Pushes 1.0.

Flags affected. None.

A.4.21 FLDPI — Load Pi

Opcode	Instruction	64-bit	Since	Example and encoding
`D9 EB`	`FLDPI`	Valid	8086	`fldpi` = `D9 EB`

Pushes pi.

Flags affected. None.

A.4.22 FWAIT — Wait

Opcode	Instruction	64-bit	Since	Example and encoding
`9B`	`FWAIT`	Valid	8086	`fwait` = `9B`

Waits for pending x87 exceptions (`wait`).

Flags affected. None.

A.5 SSE AND SSE2 INSTRUCTIONS

Scalar and packed instructions on the XMM registers. On x86-64 the floating-point calling convention uses `xmm0-xmm7`.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.5.1 MOVAPS — Move Aligned Packed Single

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 28 /r`	`MOVAPS xmm1, xmm2/m128`	Valid	SSE	`movaps xmm1, xmm2` = `0F 28 CA`
`0F 29 /r`	`MOVAPS xmm2/m128, xmm1`	Valid	SSE	`movaps xmm2, xmm1` = `0F 28 D1`

Moves 128 bits between XMM registers or aligned memory.

Flags affected. None.

A.5.2 MOVUPS — Move Unaligned Packed Single

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 10 /r`	`MOVUPS xmm1, xmm2/m128`	Valid	SSE	`movups xmm1, xmm2` = `0F 10 CA`
`0F 11 /r`	`MOVUPS xmm2/m128, xmm1`	Valid	SSE	`movups xmm2, xmm1` = `0F 10 D1`

Moves 128 bits between XMM registers or memory of any alignment.

Flags affected. None.

A.5.3 MOVDQA — Move Aligned Double Quadword

Opcode	Instruction	64-bit	Since	Example and encoding
`66 0F 6F /r`	`MOVDQA xmm1, xmm2/m128`	Valid	SSE2	`movdqa xmm1, xmm2` = `66 0F 6F CA`
`66 0F 7F /r`	`MOVDQA xmm2/m128, xmm1`	Valid	SSE2	`movdqa xmm2, xmm1` = `66 0F 6F D1`

Moves 128 bits of integer data; memory must be 16-byte aligned.

Flags affected. None.

A.5.4 MOVDQU — Move Unaligned Double Quadword

Opcode	Instruction	64-bit	Since	Example and encoding
`F3 0F 6F /r`	`MOVDQU xmm1, xmm2/m128`	Valid	SSE2	`movdqu xmm1, xmm2` = `F3 0F 6F CA`
`F3 0F 7F /r`	`MOVDQU xmm2/m128, xmm1`	Valid	SSE2	`movdqu xmm2, xmm1` = `F3 0F 6F D1`

Moves 128 bits of integer data at any alignment.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.5.5 MOVSS — Move Scalar Single

Opcode	Instruction	64-bit	Since	Example and encoding
F3 0F 10 /r	MOVSS xmm1, xmm2/m32	Valid	SSE	movss xmm1, xmm2 = F3 0F 10 CA
F3 0F 11 /r	MOVSS xmm2/m32, xmm1	Valid	SSE	movss xmm2, xmm1 = F3 0F 10 D1

Moves one single-precision value.

Flags affected. None.

A.5.6 MOVSD — Move Scalar Double

Opcode	Instruction	64-bit	Since	Example and encoding
A5	MOVSD	Valid	80386	movsd = A5
F2 0F 10 /r	MOVSD xmm1, xmm2/m64	Valid	SSE2	movsd xmm1, xmm2 = F2 0F 10 CA
F2 0F 11 /r	MOVSD xmm2/m64, xmm1	Valid	SSE2	movsd xmm2, xmm1 = F2 0F 10 D1

Moves one double-precision value (the same mnemonic also names the string instruction `movsd` without operands).

Flags affected. None.

A.5.7 MOVD — Move Doubleword

Opcode	Instruction	64-bit	Since	Example and encoding
0F 6E /r	MOVD mm1, r/m32	Valid	Pentium	movd mm1, dword [rbx] = 0F 6E 0B
0F 7E /r	MOVD r/m32, mm1	Valid	Pentium	movd dword [rbx], mm1 = 0F 7E 0B
66 0F 7E /r	MOVD m, xmm1	Valid	SSE2	movd [rbx], xmm1 = 66 0F 7E 0B
66 0F 6E /r	MOVD xmm1, m	Valid	SSE2	movd xmm1, [rbx] = 66 0F 6E 0B
66 0F 6E /r	MOVD xmm1, r/m32	Valid	SSE2	movd xmm1, dword [rbx] = 66 0F 6E 0B
66 0F 7E /r	MOVD r/m32, xmm1	Valid	SSE2	movd dword [rbx], xmm1 = 66 0F 7E 0B

Moves 32 bits between a general register or memory and an XMM or MMX register.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.5.8 MOVQ — Move Quadword

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 6F /r`	`MOVQ mm1, mm2/m64`	Valid	Pentium	`movq mm1, mm2` = `0F 6F CA`
`0F 7F /r`	`MOVQ mm2/m64, mm1`	Valid	Pentium	`movq mm2, mm1` = `0F 6F D1`
`REX.W 0F 6E /r`	`MOVQ mm1, r/m64`	Valid	x86-64	`movq mm1, qword [rbx]` = `0F 6F 0B`
`REX.W 0F 7E /r`	`MOVQ r/m64, mm1`	Valid	x86-64	`movq qword [rbx], mm1` = `0F 7F 0B`
`F3 0F 7E /r`	`MOVQ xmm1, xmm1`	Valid	SSE2	`movq xmm1, xmm1` = `F3 0F 7E C9`
`66 0F D6 /r`	`MOVQ m, xmm1`	Valid	SSE2	`movq [rbx], xmm1` = `66 0F D6 0B`
`F3 0F 7E /r`	`MOVQ xmm1, m`	Valid	SSE2	`movq xmm1, [rbx]` = `F3 0F 7E 0B`
`66 REX.W 0F 6E /r`	`MOVQ xmm1, r/m64`	Valid	x86-64	`movq xmm1, qword [rbx]` = `F3 0F 7E 0B`
`66 REX.W 0F 7E /r`	`MOVQ r/m64, xmm1`	Valid	x86-64	`movq qword [rbx], xmm1` = `66 0F D6 0B`

Moves 64 bits between a general register or memory and an XMM or MMX register. `movq rax, xmm0` is how the bit pattern of a `double` reaches an integer register.

Flags affected. None.

A.5.9 ADDSS — Add Scalar Single

Opcode	Instruction	64-bit	Since	Example and encoding
`F3 0F 58 /r`	`ADDSS xmm1, xmm2/m32`	Valid	SSE	`addss xmm1, xmm2` = `F3 0F 58 CA`

Adds the low single-precision values.

Flags affected. None.

A.5.10 ADDSD — Add Scalar Double

Opcode	Instruction	64-bit	Since	Example and encoding
`F2 0F 58 /r`	`ADDSD xmm1, xmm2/m128`	Valid	SSE2	`addsd xmm1, xmm2` = `F2 0F 58 CA`

Adds the low double-precision values.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.5.11 SUBSD — Subtract Scalar Double

Opcode	Instruction	64-bit	Since	Example and encoding
F2 0F 5C /r	SUBSD xmm1, xmm2/m64	Valid	SSE2	subsd xmm1, xmm2 = F2 0F 5C CA

Subtracts the low double-precision values.

Flags affected. None.

A.5.12 MULSD — Multiply Scalar Double

Opcode	Instruction	64-bit	Since	Example and encoding
F2 0F 59 /r	MULSD xmm1, xmm2/m64	Valid	SSE2	mulsd xmm1, xmm2 = F2 0F 59 CA

Multiplies the low double-precision values.

Flags affected. None.

A.5.13 DIVSD — Divide Scalar Double

Opcode	Instruction	64-bit	Since	Example and encoding
F2 0F 5E /r	DIVSD xmm1, xmm2/m128	Valid	SSE2	divsd xmm1, xmm2 = F2 0F 5E CA

Divides the low double-precision values.

Flags affected. None.

A.5.14 SQRTSD — Square Root Scalar Double

Opcode	Instruction	64-bit	Since	Example and encoding
F2 0F 51 /r	SQRTSD xmm1, xmm2/m64	Valid	SSE2	sqrtsd xmm1, xmm2 = F2 0F 51 CA

Computes the square root of the low double-precision value.

Flags affected. None.

A.5.15 ADDPS — Add Packed Single

Opcode	Instruction	64-bit	Since	Example and encoding
0F 58 /r	ADDPS xmm1, xmm2/m128	Valid	SSE	addps xmm1, xmm2 = 0F 58 CA

Adds four single-precision values in parallel.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.5.16 MULPS — Multiply Packed Single

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 59 /r`	`MULPS xmm1, xmm2/m128`	Valid	SSE	`mulps xmm1, xmm2` = `0F 59 CA`

Multiplies four single-precision values in parallel.

Flags affected. None.

A.5.17 CVTSI2SD — Convert Integer to Scalar Double

Opcode	Instruction	64-bit	Since	Example and encoding
`F2 0F 2A /r`	`CVTSI2SD xmm1, r/m32`	Valid	SSE2	`cvtsi2sd xmm1, dword [rbx]` = `F2 0F 2A 0B`
`REX.W F2 0F 2A /r`	`CVTSI2SD xmm1, r/m64`	Valid	x86-64	`cvtsi2sd xmm1, qword [rbx]` = `F2 48 0F 2A 0B`

Converts a 32 or 64-bit integer to double precision.

Flags affected. None.

A.5.18 CVTTSD2SI — Convert with Truncation Scalar Double to Integer

Opcode	Instruction	64-bit	Since	Example and encoding
`F2 0F 2C /r`	`CVTTSD2SI r32, xmm2/m64`	Valid	SSE2	`cvttsd2si ecx, xmm2` = `F2 0F 2C CA`
`REX.W F2 0F 2C /r`	`CVTTSD2SI r64, xmm2/m64`	Valid	x86-64	`cvttsd2si rcx, xmm2` = `F2 48 0F 2C CA`

Converts a double to an integer, truncating toward zero (the C cast).

Flags affected. None.

A.5.19 CVTSD2SS — Convert Scalar Double to Scalar Single

Opcode	Instruction	64-bit	Since	Example and encoding
`F2 0F 5A /r`	`CVTSD2SS xmm1, xmm2/m64`	Valid	SSE2	`cvtsd2ss xmm1, xmm2` = `F2 0F 5A CA`

Narrows a double to single precision.

Flags affected. None.

A.5.20 CVTSS2SD — Convert Scalar Single to Scalar Double

Opcode	Instruction	64-bit	Since	Example and encoding
`F3 0F 5A /r`	`CVTSS2SD xmm1, xmm2/m128`	Valid	SSE2	`cvtss2sd xmm1, xmm2` = `F3 0F 5A CA`

Widens a single-precision value to double.

Copyright © 2026 Mallionic Inc. All rights reserved.

Flags affected. None.

A.5.21 COMISD — Compare Scalar Ordered Double

Opcode	Instruction	64-bit	Since	Example and encoding
`66 0F 2F /r`	`COMISD xmm1, xmm2/m64`	Valid	SSE2	`comisd xmm1, xmm2` = `66 0F 2F CA`

Compares the low doubles and sets ZF, PF and CF; raises an exception for NaN operands. `ucomisd` does not.

Flags affected. ZF, PF, CF as for an unsigned comparison; OF, SF, AF cleared.

A.5.22 UCOMISD — Unordered Compare Scalar Double

Opcode	Instruction	64-bit	Since	Example and encoding
`66 0F 2E /r`	`UCOMISD xmm1, xmm2/m64`	Valid	SSE2	`ucomisd xmm1, xmm2` = `66 0F 2E CA`

Compares the low doubles and sets ZF, PF and CF (PF = 1 for unordered results).

Flags affected. As for COMISD.

A.5.23 ANDPS — Bitwise AND of Packed Single

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 54 /r`	`ANDPS xmm1, xmm2/m128`	Valid	SSE	`andps xmm1, xmm2` = `0F 54 CA`

Bitwise AND of 128 bits; used to clear sign bits (absolute value).

Flags affected. None.

A.5.24 ORPS — Bitwise OR of Packed Single

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 56 /r`	`ORPS xmm1, xmm2/m128`	Valid	SSE	`orps xmm1, xmm2` = `0F 56 CA`

Bitwise OR of 128 bits.

Flags affected. None.

A.5.25 XORPS — Bitwise XOR of Packed Single

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 57 /r`	`XORPS xmm1, xmm2/m128`	Valid	SSE	`xorps xmm1, xmm2` = `0F 57 CA`

Bitwise XOR of 128 bits; `xorps xmm0, xmm0` zeroes a register.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.5.26 PXOR — Packed Logical XOR

Opcode	Instruction	64-bit	Since	Example and encoding
`0F EF /r`	`PXOR mm1, mm2/m64`	Valid	Pentium	`pxor mm1, mm2` = `0F EF CA`
`66 0F EF /r`	`PXOR xmm1, xmm2/m128`	Valid	SSE2	`pxor xmm1, xmm2` = `66 0F EF CA`

Bitwise XOR of 128 integer bits.

Flags affected. None.

A.5.27 PAND — Packed Logical AND

Opcode	Instruction	64-bit	Since	Example and encoding
`0F DB /r`	`PAND mm1, mm2/m64`	Valid	Pentium	`pand mm1, mm2` = `0F DB CA`
`66 0F DB /r`	`PAND xmm1, xmm2/m128`	Valid	SSE2	`pand xmm1, xmm2` = `66 0F DB CA`

Bitwise AND of 128 integer bits.

Flags affected. None.

A.5.28 POR — Packed Logical OR

Opcode	Instruction	64-bit	Since	Example and encoding
`0F EB /r`	`POR mm1, mm2/m64`	Valid	Pentium	`por mm1, mm2` = `0F EB CA`
`66 0F EB /r`	`POR xmm1, xmm2/m128`	Valid	SSE2	`por xmm1, xmm2` = `66 0F EB CA`

Bitwise OR of 128 integer bits.

Flags affected. None.

A.5.29 PADDB — Packed Add Bytes

Opcode	Instruction	64-bit	Since	Example and encoding
`0F FC /r`	`PADDB mm1, mm2/m64`	Valid	Pentium	`paddb mm1, mm2` = `0F FC CA`
`66 0F FC /r`	`PADDB xmm1, xmm2/m128`	Valid	SSE2	`paddb xmm1, xmm2` = `66 0F FC CA`

Adds sixteen bytes in parallel (wrap-around). `paddw`, `paddd`, `paddq` work on words, doublewords and quadwords.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.5.30 PADDD — Packed Add Doublewords

Opcode	Instruction	64-bit	Since	Example and encoding
`0F FE /r`	`PADDD mm1, mm2/m64`	Valid	Pentium	`paddd mm1, mm2` = `0F FE CA`
`66 0F FE /r`	`PADDD xmm1, xmm2/m128`	Valid	SSE2	`paddd xmm1, xmm2` = `66 0F FE CA`

Adds four doublewords in parallel.

Flags affected. None.

A.5.31 PADDQ — Packed Add Quadwords

Opcode	Instruction	64-bit	Since	Example and encoding
`0F D4 /r`	`PADDQ mm1, mm2/m64`	Valid	SSE2	`paddq mm1, mm2` = `0F D4 CA`
`66 0F D4 /r`	`PADDQ xmm1, xmm2/m128`	Valid	SSE2	`paddq xmm1, xmm2` = `66 0F D4 CA`

Adds two quadwords in parallel.

Flags affected. None.

A.5.32 PSUBD — Packed Subtract Doublewords

Opcode	Instruction	64-bit	Since	Example and encoding
`0F FA /r`	`PSUBD mm1, mm2/m64`	Valid	Pentium	`psubd mm1, mm2` = `0F FA CA`
`66 0F FA /r`	`PSUBD xmm1, xmm2/m128`	Valid	SSE2	`psubd xmm1, xmm2` = `66 0F FA CA`

Subtracts four doublewords in parallel.

Flags affected. None.

A.5.33 PCMPEQB — Packed Compare for Equal Bytes

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 74 /r`	`PCMPEQB mm1, mm2/m64`	Valid	Pentium	`pcmpeqb mm1, mm2` = `0F 74 CA`
`66 0F 74 /r`	`PCMPEQB xmm1, xmm2/m128`	Valid	SSE2	`pcmpeqb xmm1, xmm2` = `66 0F 74 CA`

Compares sixteen bytes and sets each result byte to all ones where equal; combined with `pmovmskb`, finds a byte in 16 at a time (vectorized `strlen`).

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

A.5.34 PMOVMSKB — Move Byte Mask

Opcode	Instruction	64-bit	Since	Example and encoding
`0F D7 /r`	`PMOVMSKB r32, mm1`	Valid	SSE	`pmovmskb ecx, mm1` = `0F D7 C9`
`66 0F D7 /r`	`PMOVMSKB r32, xmm1`	Valid	SSE2	`pmovmskb ecx, xmm1` = `66 0F D7 C9`

Collects the top bit of each of the sixteen bytes into a 16-bit integer mask.

Flags affected. None.

A.5.35 PSHUFD — Shuffle Packed Doublewords

Opcode	Instruction	64-bit	Since	Example and encoding
`66 0F 70 /r ib`	`PSHUFD xmm1, xmm1, imm`	Valid	SSE2	`pshufd xmm1, xmm1, 0x12345678` = `66 0F 70 C9 78`
`66 0F 70 /r ib`	`PSHUFD xmm1, m, imm`	Valid	SSE2	`pshufd xmm1, [rbx], 0x12345678` = `66 0F 70 0B 78`

Rearranges the four doublewords according to an immediate selector.

Flags affected. None.

A.5.36 SHUFPS — Shuffle Packed Single

Opcode	Instruction	64-bit	Since	Example and encoding
`0F C6 /r ib`	`SHUFPS xmm1, xmm2/m128, imm8`	Valid	SSE	`shufps xmm1, xmm2, 0x12` = `0F C6 CA 12`

Selects two single-precision values from each operand according to an immediate.

Flags affected. None.

A.5.37 UNPCKLPS — Unpack and Interleave Low Packed Single

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 14 /r`	`UNPCKLPS xmm1, xmm2/m128`	Valid	SSE	`unpcklps xmm1, xmm2` = `0F 14 CA`

Interleaves the low halves of two registers.

Flags affected. None.

A.5.38 LDMXCSR — Load MXCSR Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F AE /2`	`LDMXCSR m32`	Valid	SSE	`ldmxcsr dword [rbx]` = `0F AE 13`

Loads the SSE control and status register (rounding, exception masks).

Copyright © 2026 Mallionic Inc. All rights reserved.

Flags affected. None.

A.5.39 STMXCSR — Store MXCSR Register

Opcode	Instruction	64-bit	Since	Example and encoding
`0F AE /3`	`STMXCSR m32`	Valid	SSE	`stmxcsr dword [rbx]` = `0F AE 1B`

Stores the SSE control and status register.

Flags affected. None.

A.5.40 EMMS — Empty MMX State

Opcode	Instruction	64-bit	Since	Example and encoding
`0F 77`	`EMMS`	Valid	Pentium	`emms` = `0F 77`

Marks the x87 registers as empty after MMX code.

Flags affected. None.

Copyright © 2026 Mallionic Inc. All rights reserved.

APPENDIX B

REGISTER REFERENCE

This appendix lists every register name known to `shasm`, with the number used in instruction encodings (the value of the `reg`, `r/m` or `+r` field, extended by REX bits for 8 to 15 and by the APX extension for 16 to 31). Register names can be used in `asm` blocks and `shasm` sources, in any letter case. The 64-bit, 32-bit and 16-bit tables include the registers 16–31 of the APX extension, which `shasm` 1.2.0 recognizes as names but does not encode.

Table B-1. 8-bit legacy registers

Register	Number	Register	Number
`al`	0	`ah`	4
`bl`	3	`bh`	7
`cl`	1	`ch`	5
`dl`	2	`dh`	6

Table B-2. 8-bit registers that need a REX prefix

Register	Number	Register	Number
`spl`	4	`bpl`	5
`sil`	6	`dil`	7
`r8b`	8	`r9b`	9
`r10b`	10	`r11b`	11
`r12b`	12	`r13b`	13
`r14b`	14	`r15b`	15
`r16b`	16	`r17b`	17
`r18b`	18	`r19b`	19
`r20b`	20	`r21b`	21
`r22b`	22	`r23b`	23
`r24b`	24	`r25b`	25
`r26b`	26	`r27b`	27
`r28b`	28	`r29b`	29
`r30b`	30	`r31b`	31

Copyright © 2026 Mallionic Inc. All rights reserved.

Table B-3. 16-bit general-purpose registers

Register	Number	Register	Number
ax	0	bx	3
cx	1	dx	2
sp	4	bp	5
si	6	di	7
r8w	8	r9w	9
r10w	10	r11w	11
r12w	12	r13w	13
r14w	14	r15w	15
r16w	16	r17w	17
r18w	18	r19w	19
r20w	20	r21w	21
r22w	22	r23w	23
r24w	24	r25w	25
r26w	26	r27w	27
r28w	28	r29w	29
r30w	30	r31w	31

Table B-4. 32-bit general-purpose registers

Register	Number	Register	Number
eax	0	ebx	3
ecx	1	edx	2
esp	4	ebp	5
esi	6	edi	7
r8d	8	r9d	9
r10d	10	r11d	11
r12d	12	r13d	13
r14d	14	r15d	15
r16d	16	r17d	17
r18d	18	r19d	19
r20d	20	r21d	21
r22d	22	r23d	23
r24d	24	r25d	25
r26d	26	r27d	27
r28d	28	r29d	29
r30d	30	r31d	31

Copyright © 2026 Mallionic Inc. All rights reserved.

Table B-5. 64-bit general-purpose registers

Register	Number	Register	Number
rax	0	rbx	3
rcx	1	rdx	2
rsp	4	rbp	5
rsi	6	rdi	7
r8	8	r9	9
r10	10	r11	11
r12	12	r13	13
r14	14	r15	15
r16	16	r17	17
r18	18	r19	19
r20	20	r21	21
r22	22	r23	23
r24	24	r25	25
r26	26	r27	27
r28	28	r29	29
r30	30	r31	31

Table B-6. Segment registers

Register	Number	Register	Number
es	0	cs	1
ss	2	ds	3
fs	4	gs	5
segr6	6	segr7	7

Copyright © 2026 Mallionic Inc. All rights reserved.

Table B-7. Control registers

Register	Number	Register	Number
cr0	0	cr1	1
cr2	2	cr3	3
cr4	4	cr5	5
cr6	6	cr7	7
cr8	8	cr9	9
cr10	10	cr11	11
cr12	12	cr13	13
cr14	14	cr15	15
cr16	16	cr17	17
cr18	18	cr19	19
cr20	20	cr21	21
cr22	22	cr23	23
cr24	24	cr25	25
cr26	26	cr27	27
cr28	28	cr29	29
cr30	30	cr31	31

Table B-8. Debug registers

Register	Number	Register	Number
dr0	0	dr1	1
dr2	2	dr3	3
dr4	4	dr5	5
dr6	6	dr7	7
dr8	8	dr9	9
dr10	10	dr11	11
dr12	12	dr13	13
dr14	14	dr15	15
dr16	16	dr17	17
dr18	18	dr19	19
dr20	20	dr21	21
dr22	22	dr23	23
dr24	24	dr25	25
dr26	26	dr27	27
dr28	28	dr29	29
dr30	30	dr31	31

Copyright © 2026 Mallionic Inc. All rights reserved.

Table B-9. Test registers

Register	Number	Register	Number
tr0	0	tr1	1
tr2	2	tr3	3
tr4	4	tr5	5
tr6	6	tr7	7

Table B-10. x87 floating-point stack

Register	Number	Register	Number
st0	0	st1	1
st2	2	st3	3
st4	4	st5	5
st6	6	st7	7

Table B-11. MMX registers

Register	Number	Register	Number
mm0	0	mm1	1
mm2	2	mm3	3
mm4	4	mm5	5
mm6	6	mm7	7

Copyright © 2026 Mallionic Inc. All rights reserved.

Table B-12. XMM registers (SSE)

Register	Number	Register	Number
xmm0	0	xmm1	1
xmm2	2	xmm3	3
xmm4	4	xmm5	5
xmm6	6	xmm7	7
xmm8	8	xmm9	9
xmm10	10	xmm11	11
xmm12	12	xmm13	13
xmm14	14	xmm15	15
xmm16	16	xmm17	17
xmm18	18	xmm19	19
xmm20	20	xmm21	21
xmm22	22	xmm23	23
xmm24	24	xmm25	25
xmm26	26	xmm27	27
xmm28	28	xmm29	29
xmm30	30	xmm31	31

Table B-13. YMM registers (AVX)

Register	Number	Register	Number
ymm0	0	ymm1	1
ymm2	2	ymm3	3
ymm4	4	ymm5	5
ymm6	6	ymm7	7
ymm8	8	ymm9	9
ymm10	10	ymm11	11
ymm12	12	ymm13	13
ymm14	14	ymm15	15
ymm16	16	ymm17	17
ymm18	18	ymm19	19
ymm20	20	ymm21	21
ymm22	22	ymm23	23
ymm24	24	ymm25	25
ymm26	26	ymm27	27
ymm28	28	ymm29	29
ymm30	30	ymm31	31

Copyright © 2026 Mallionic Inc. All rights reserved.

Table B-14. ZMM registers (AVX-512)

Register	Number	Register	Number
zmm0	0	zmm1	1
zmm2	2	zmm3	3
zmm4	4	zmm5	5
zmm6	6	zmm7	7
zmm8	8	zmm9	9
zmm10	10	zmm11	11
zmm12	12	zmm13	13
zmm14	14	zmm15	15
zmm16	16	zmm17	17
zmm18	18	zmm19	19
zmm20	20	zmm21	21
zmm22	22	zmm23	23
zmm24	24	zmm25	25
zmm26	26	zmm27	27
zmm28	28	zmm29	29
zmm30	30	zmm31	31

Table B-15. Opmask registers (AVX-512)

Register	Number	Register	Number
k0	0	k1	1
k2	2	k3	3
k4	4	k5	5
k6	6	k7	7

Table B-16. Bound registers (MPX)

Register	Number	Register	Number
bnd0	0	bnd1	1
bnd2	2	bnd3	3

Table B-17. Tile registers (AMX)

Register	Number	Register	Number
tmm0	0	tmm1	1
tmm2	2	tmm3	3
tmm4	4	tmm5	5
tmm6	6	tmm7	7

Copyright © 2026 Mallionic Inc. All rights reserved.

APPENDIX C

GRAMMAR SUMMARY

This appendix summarizes the syntax of ASH in an extended BNF notation: `[ x ]` is optional, `{ x }` repeats zero or more times, `a | b` is an alternative, quoted text is literal, and names in italics in the rest of the manual are written here in lowercase. Directives (`%...`) and assembly bodies are processed before this grammar applies (Section 6.2).

C.1 PROGRAM STRUCTURE

```
program        = [ "\"use strict\"" [";"] ] { top-item } ;
top-item       = import | extern | global | struct-decl | typedef | define | macro
               | redefine | override-assign | { annotation } ( function | class | global ) ;

import         = "import" ( "<" name ">" | string ) ";" ;
extern         = "extern" ( type | "func" ) name "(" [ ext-params ] ")" ";" ;
ext-params     = ext-param { "," ext-param } [ "," "..." ] | "..." ;
ext-param      = type [ name ] ;

global         = ( "general" | "ungeneral" | "runnable" ) [ "static" ] [ "const" ]
                 type name [ "=" global-init ] [ ";" ] ;
global-init    = const-expr | "{" const-expr { "," const-expr } [","] "}" | string ;

struct-decl    = "typedef" "struct" [ name ] "{" { field } "}" name ";"
               | "struct" name "{" { field } "}" ";" ;
field          = type name ";" ;
typedef        = "typedef" type name [ ";" ] ;
define         = "define" name tokens [ ";" ] ;
macro          = "macro" name "{" { statement } "}" ;
redefine       = "redefine" [ "global" ] type name "(" params ")" type name "(" params ")"
               | "redefine" [ "global" ] type name "(" params ")" block ;
override-assign = [ "@" ] name { "." name } "." "override" "=" integer ";" ;
```

C.2 FUNCTIONS AND CLASSES

```
annotation     = "@" name [ "(" [ ann-args ] ")" | "[" [ integer ] "]" ] ;
ann-definition = "@" name "(" [ params ] ")" block ;

function       = [ "pub" | "priv" ] "func" [ type ] name "(" [ params ] ")" [ ":" type ] bod
↪ y ;
params         = param { "," param } [ "," "..." ] ;
param          = type name [ "[" "]" ] ;
body           = "{" { crack | statement } "}" ;
crack          = name "->" "(" [ params ] ")" block [ ";" ] ;

class          = [ "pub" | "priv" ] "class" name [ "(" ")" ] "{" { [ "pub" | "priv" ] functi
↪ on } "}" ;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

C.3 TYPES

```
type           = base-type { "*" } ;
base-type      = int-type | "bool" | "char" | "string" | "cstring" | "ptr" | "void"
               | "float" | "float32" | "double" | "float64" | struct-name | alias ;
int-type       = "int8" | "int16" | "int32" | "int64" | "uint8" | "uint16" | "uint32" | "uin
↪ t64"
               | "byte" | "word" | "dword" | "qword" | "int" | "uint" | "short" | "ushort"
               | "long" | "ulong" | "llong" | "ullong" | "uchar" | "schar" | "size_t" | "ssi
↪ ze_t"
               | "intptr_t" | "uintptr_t" | "ptrdiff_t" | "int8_t" ... "uint64_t"
               | c-type-words ;
c-type-words   = { "unsigned" | "signed" | "long" | "short" | "int" | "char" } ;
```

C.4 STATEMENTS

```
block          = "{" { statement } "}" ;
statement      = declaration | assignment | expression-stmt | if | while | for | repeat | lo
↪ op
               | switch | "break" [";"] | "continue" [";"] | return | terminate | unreturnab
↪ le
               | asm | emit | matrix | crack-call | block-macro-use | ";" ;

declaration    = [ "volatile" ] type name [ array-dims ] [ "=" init ] [ ";" ]
               | [ "volatile" ] type array-dims name [ "=" init ] [ ";" ] ;
array-dims     = "[" [ expr ] "]" [ "[" [ expr ] "]" ] ;
init           = expr | "{" [ init { "," init } [ "," ] ] "}" ;

assignment     = lvalue ( "=" | compound-op ) expr [ ";" ]
               | lvalue ( "++" | "--" ) [ ";" ] | ( "++" | "--" ) name [ ";" ] ;
lvalue         = name | name index { index } | "*" unary | member-chain ;
compound-op    = "+=" | "-=" | "*=" | "/=" | "%=" | "<<=" | ">>=" | "&=" | "|=" | "^=" ;
expression-stmt = call [ ";" ] ;
crack-call     = name "." name "=" expr { "," expr } [ ";" ] ;

if             = "if" "(" cond ")" block { ( "elif" | "else" "if" ) "(" cond ")" block } [ "
↪ else" block ] ;
while          = "while" "(" cond ")" block ;
for            = "for" "(" [ for-init ] ";" [ cond ] ";" [ for-update ] ")" block ;
repeat         = "repeat" "(" expr ")" block ;
loop           = "loop" block ;
switch         = "switch" "(" expr ")" "{" { case-label { statement } } "}" ;
case-label     = "case" expr ":" | "default" ":" ;

return         = "return" [ expr ] [ ";" ] ;
terminate      = "terminate" [ expr ] [ ";" ] ;
unreturnable   = ( "unreturnable" | "unreturn" ) ( expr | type ) [ ";" ] ;

asm            = [ "volatile" ] "asm" "{" assembly "}"
               | "asm" [ "volatile" ] "(" string { string } [ ":" operands [ ":" operands [
↪ ":" clobbers ] ] ] ")" [ ";" ] ;
emit           = ( "emit" | "emit8" | "emit16" | "emit32" | "emit64" | "@emit" | "ashley" ".
↪ " name )
                 "(" expr { "," expr } ")" [ ";" ] ;
matrix         = "matrix" name "[" "]" "[" "]" "{" { "[" type expr "]" [ "," ] } "}" [ ";" ]
               | "matrix" type name "[" expr "]" "[" expr "]" [ ";" ] ;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

C.5 EXPRESSIONS AND CONDITIONS

```
cond            = or-cond ;
or-cond         = and-cond { "||" and-cond } ;
and-cond        = not-cond { "&&" not-cond } ;
not-cond        = "!" "(" cond ")" | "(" cond ")" | comparison | expr ;
comparison      = expr ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) expr ;

expr            = bit-or ;
bit-or          = bit-xor { "|" bit-xor } ;
bit-xor         = bit-and { "^" bit-and } ;
bit-and         = shift { "&" shift } ;
shift           = additive { ( "<<" | ">>" ) additive } ;
additive        = term { ( "+" | "-" ) term } ;
term            = power { ( "*" | "/" | "%" ) power } ;
power           = unary [ ( "**" | "***" ) power ] ;
unary           = ( "-" | "+" | "~" | "&" | "*" | "#" | "++" | "--" ) unary
                | "(" type ")" unary | postfix ;
postfix         = primary { index | "++" | "--" } ;
index           = "[" expr "]" ;
primary         = literal | name | "(" expr ")" | call | member-chain | "{" [ expr { "," expr
↪  } [","] ] "}" ;
call            = name "(" [ args ] ")" | name "." name "(" [ args ] ")" | member-chain "(" [
↪  args ] ")" ;
member-chain    = name ( "." | "->" ) name { ( "." | "->" ) name } ;
args            = expr { "," expr } ;
literal         = integer | float | char | string { string } | "L" string ;
```

C.6 LEXICAL ELEMENTS

```
name            = letter { letter | digit | "_" } ;          letter = "A".."Z" | "a".."z" | "
↪ _" ;
integer         = digits [ suffix ] | "0x" hex-digits [ suffix ] | "0b" bin-digits ;
suffix          = up to three of "u" "U" "l" "L" ;
float           = digits "." digits ;
char            = "'" ( character | escape ) "'" ;
string          = "\"" { character | escape } "\"" | "\"\"\"" { any } "\"\"\"" ;
escape          = "\n" | "\r" | "\t" | "\0" | "\a" | "\b" | "\f" | "\v" | "\\" | "\'" | "\""
                | "\x" hex hex | "\" octal [ octal [ octal ] ] | "\u" hex{4} | "\U" hex{8} ;
comment         = "//" { any } end-of-line | "/*" { any } "*/" ;
```

Copyright © 2026 Mallionic Inc. All rights reserved.

APPENDIX D

INDEX OF THE EXAMPLE PROGRAMS

The directory `examples/` of the source tree contains one small program for each feature of the language. The table lists them with the command used to build them, the expected result, and the section of this manual that explains the feature. (The comments inside those files are written in Spanish; the explanations in this manual are the English reference.)

Copyright © 2026 Mallionic Inc. All rights reserved.

Table D-1. Programs in examples/

File	Demonstrates	Manual
`01_hello_asm32.ash`	32-bit Linux executable with `int 0x80`; build with `--exe`	Example 20-7
`02_asm64_volatile.ash`	`asm` and `volatile asm` with variables; exit status 84	Section 20.1
`03_switch_case.ash`	`switch` with fall-through, `break`, `default`	Section 11.7
`04_classes.ash`	Classes, `pub` / `priv` methods	Chapter 15
`05_typedef_annotations.ash`	`typedef` and user-defined annotations	Section 17.16
`06_shifts_compound.ash`	Shifts and compound assignment	Section 7.8.2
`07_arrays_matrix.ash`	Arrays, two-dimensional arrays, `string[]`	Chapter 13
`08_bits16.ash`	16-bit flat binary (`--compile`), AX = 60	Example 3-4
`09_lib32.ash` + `09_main.c`	32-bit library called from C	Section 22.3
`10_no_libc_64.ash`	64-bit program without libc	Section 22.2.2
`11_ash_calls_c.ash` + `11_c_side.c`	ASH calling a C function	Section 12.8
`12_repeat_loop.ash`	`repeat` and `loop` with `break` / `continue`	Section 11.5
`13_directives.ash`	`%if`, `%ifdef`, `%define`, `%error` by target	Chapter 18
`14_bitwise.ash`	`&`, `|`, `^`, `~` and C precedence	Section 10.7
`15_c_types.ash`	C type names and literal suffixes	Section 7.8.1
`16_compact_epilogue.ash`	Saving only the registers used	Section 24.3
`17_bit_flags.ash`	Permission flags with bitwise operators	Example 10-8
`18_hash_fnv.ash`	FNV-1a 64-bit hash	Section 30.7
`19_rgb_pack.ash`	Packing colors with shifts and masks	Section 8.2.2
`20_strict_mode.ash`	`"use strict"`	Chapter 21
`21_variadic_basic.ash`, `22_variadic_stats.ash`	Variadic functions	Section 12.6
`23_args_cli.ash`, `24_args_flags.ash`	Command line arguments with `<args>`	Section 27.5
`25_many_params.ash`	More than six arguments	Section 12.4.1
`26_beautycode_on.ash`, `27_beautycode_off.ash`	Bounds checking on and off (see `beautycode_compare.txt`)	Section 13.3.1
`28_bytebeat_alsa.ash`	Bytebeat music through ALSA (`-lasound`)	Section 30.12
`29_pow_directives.ash`	`**`, `@nolibc(false)`, arithmetic in `%define`	Section 10.5
`30_bool.ash`	`bool`, `true`, `false`	Section 8.4
`35_globals_static.ash`	Globals: `general`, `ungeneral`, `static`, `&global`	Section 9.4

Copyright © 2026 Mallionic Inc. All rights reserved.

File	Demonstrates	Manual
`36_cracks.ash`	Cracks and `unreturnable`	Chapter 16
`37_struct.ash`	Structures, `.` and `->`	Chapter 14
`38_matrix_strings.ash`	`matrix`, `rows` / `cols` / `count`, triple-quoted strings	Section 13.7
`39_callbacks_redefine.ash`	Callbacks for `qsort` / `atexit`, `redefine` alias	Section 12.7
`40_emit.ash`	`emit`, `@emit`, raw bytes	Section 20.3
`41_section.ash`	`@section`, `@attribute(section(...))`	Section 17.13
`42_asm_string.ash`	GCC-style `asm volatile("...")`	Section 20.2
`43_runnable.ash`	`runnable` arrays patched at run time	Section 9.4.4
`44_global_arrays.ash`	Global arrays and multi-line `char[]`	Section 9.4.3
`45_beautycode_frames.ash`	`@beautycode` and frames	Section 17.7
`46_nlib_string_math.ash`	`<string>` and `<math>` without libc	Chapter 27
`47_syscalls_nolibc.ash`	`<syscalls>` with `--no-libc`	Section 27.4
`48_sqpow.ash`	`***` repeated squaring	Section 10.5
`49_terminate.ash`	`terminate` from loops and `switch`	Section 11.8.2
`50_naked.ash`	`@naked` functions	Section 20.5
`ovr/`	Override rights between files	Section 19.4
`win/`	Windows objects: console, MessageBox, Native API, waveOut	Section 22.6
`c_interop/`	Functions and variables shared with C	Section 22.9

Copyright © 2026 Mallionic Inc. All rights reserved.

APPENDIX E

KNOWN LIMITATIONS OF ASHC 1.0

The following behaviors of `ashc` 1.0.0 were observed while this manual was produced. Each is described where it matters in the body of the manual, with a workaround; this appendix collects them in one place. They are expected to change in future versions.

E.1 LANGUAGE

Table E-1. Language limitations

Limitation	Workaround
Comparisons and `&&`, `||`, `!` cannot be used as values.	Use them in conditions: `int64 r = 0; if (a < b) { r = 1; }`.
No `do ... while`, `goto`, `?:`, comma operator, `sizeof`.	`loop` with `break`; `terminate`; `if` / `else`; define sizes as constants.
Locals are visible from their declaration to the end of the function, not of the block.	Use distinct names.
Redeclaring a local with the same name and type re-initializes it silently.	Use distinct names.
Pointer arithmetic is in bytes (`p + 1` is the next byte); indexing `p[i]` scales.	Prefer indexing.
A cast to a narrower type does not truncate inside an expression.	Mask (`x & 0xFF`) or assign to a narrow variable.
Floating-point literals and arithmetic are not supported; `printf("%f")` cannot receive ASH doubles.	Use `<math>` fixed point.
Strings: `+` does not concatenate, writing to a literal crashes, `#s` is meaningless.	`uint8[N]` buffers with `strcpy` / `strcat`.
Local `char[] c = "..."` has no length header.	Use a `uint8[N]` buffer, or a global `char[]`.
Parameters written `T v[]` are untyped byte pointers.	Write `T* v`.
Variadic ASH functions accept at most 8 extra arguments.	Pass an array.
Arrays and structures declared in a function are never freed.	Allocate once and reuse.
Structures cannot contain array fields or be passed by value.	Use separate fields or pointers.

Copyright © 2026 Mallionic Inc. All rights reserved.

E.2 CODE GENERATION

Table E-2. Code generation issues

Issue	Workaround
Binary literals (`0b...`) inside functions fail in the ASHIR stage.	Use hexadecimal.
Negative constants whose magnitude exceeds 2^31-1 are computed incorrectly.	Hexadecimal two's complement, or subtract a variable.
`>>` by a variable amount is always arithmetic, even for unsigned values.	Mask the result or shift by a constant.
The signedness of a parameter can be taken from a parameter with the same name in an earlier function (this affects `int_to_str` in `<string>`).	Give parameters distinct names in functions with different signedness.
`continue` inside a `switch` inside a loop loops forever.	Set a flag and test it after the `switch`.
Reading and then writing `*p` through a pointer parameter fails to compile.	Use `p[0]` or a local copy of the pointer.
Storing a `uint8` parameter into a `uint8` structure field causes an internal error.	Declare the parameter `int64`.
An `asm` block that `shasm` cannot assemble is dropped without an error.	Check assembly with stand-alone `shasm`.
With `--no-libc`, `exit()` is defined only if the program uses the runtime (arrays, strings).	Use `sys_exit` from `<syscalls>`.
Global arrays read in 16-bit flat binaries crash.	Use local arrays in 16-bit code.
A function with a result type whose body ends without any `return` crashes when called.	Always end such functions with `return value;` (or use `unreturnable` and `return;`).
Windows: variadic floating-point arguments; `rdi` / `rsi` not preserved.	End programs with `ExitProcess`.

Copyright © 2026 Mallionic Inc. All rights reserved.

E.3 LIBRARIES AND TOOLS

Table E-3. Library and tool issues

Issue	Workaround
`<stdlib>` calls `__stlib_*` functions that do not exist and cannot be linked.	Call the C functions directly (Example 27-98).
`import <null>;` fails because of the library's own `null` macro.	`define NULL 0;`
`<types>`: the values cannot be converted to integers.	Use them only as `double` values passed to C.
`shasm` 1.2.0 has no encodings for `bt*`, `xadd`, `cmpxchg`, `bswap`, `shld`/`shrd`, `lzcnt`/`tzcnt`, `enter`, `rdrand`, `lar`/`lsl` and a few others (Appendix A).	Encode them with `emit`.
ELF objects contain host information (user name, home directory, source path).	`objcopy --strip-unneeded --remove-section=.comment`.
Some diagnostics, the `--debug` output and `logs/build.log` are in Spanish.	This manual gives English translations of the messages it shows.

Copyright © 2026 Mallionic Inc. All rights reserved.

APPENDIX F

GLOSSARY

Copyright © 2026 Mallionic Inc. All rights reserved.

Table F-1. Glossary of terms

Term	Definition
ABI	Application Binary Interface: the rules for calling functions, passing arguments, returning values and laying out data, shared by compilers so that their code can be linked together. ASH follows the System V AMD64 ABI on Linux and the Microsoft x64 ABI on Windows.
annotation	A name preceded by `@` that configures the compiler or marks a function (Chapter 17).
array header	The 16 bytes before the first element of an ASH array, holding its length and capacity (Section 13.2).
ASHIR	The ASH Intermediate Representation, the textual three-address code between the front end and the back end (Chapter 23).
back end	The part of the compiler that turns ASHIR into machine code and object files.
bounds check	The test that an array index is smaller than the length of the array, performed by `__ash_bounds`.
callee-saved register	A register that a function must preserve for its caller (`rbx`, `rbp`, `r12`–`r15` in System V).
crack	A named entry point declared inside a function that can be invoked with `function.crack = values;` (Chapter 16).
directive	A line starting with `%`, evaluated before lexing (Chapter 18).
ELF	Executable and Linkable Format, the object and executable format of Linux.
COFF	Common Object File Format, the object format used by Windows (and by UEFI tools).
entry point	The function where a program starts, marked `@Entry`.
flat binary	Machine code with no header, loaded at a known address (Section 22.5).
front end	The part of the compiler that reads ASH and produces ASHIR.
leaf function	A function that calls no other function; ASH emits it without a frame when possible.
libc	The C standard library (glibc on Linux).
naked function	A function without compiler-generated prologue or epilogue (Section 20.5).
nlib	The system libraries of ASH, embedded in the compiler (Chapter 27).
override right	A signed integer that decides which of several same-named definitions survives (Section 19.4).
pending return value	A result stored with `unreturnable` and returned later (Section 16.2).
Q16.16	A fixed-point number format with 16 fraction bits (Section 27.3.1).
relocation	A record in an object file telling the linker to patch an address once it is known.
REX prefix	The x86-64 prefix byte (`40`–`4F`) that selects 64-bit operands and the registers `r8`–`r15`.
runnable array	A global array stored in executable memory, callable as a function (Section 9.4.4).
runtime helper	A function added to the program by the compiler, such as `__ash_alloc` (Section 23.8).
shadow space	The 32 bytes the caller reserves on the stack for the callee in the Microsoft x64 convention.
shasm	The assembler of The Ashley Project (Chapter 26).
strict mode	A mode with additional compile-time checks (Chapter 21).
System V AMD64	The calling convention of 64-bit Linux, BSD and macOS.
tree shaking	The removal of imported functions that the program does not use (Section 19.3.1).

Copyright © 2026 Mallionic Inc. All rights reserved.

Term	Definition
UEFI	Unified Extensible Firmware Interface, the firmware of modern PCs (Section 22.7).

Copyright © 2026 Mallionic Inc. All rights reserved.

APPENDIX G

ASH FOR C PROGRAMMERS

This appendix is a side-by-side comparison for programmers who already know C. Most C code translates to ASH line by line; the tables point out what is written differently, what ASH adds and what it does not have.

Copyright © 2026 Mallionic Inc. All rights reserved.

G.1 SYNTAX DIFFERENCES

Table G-1. C constructs and their ASH equivalents

Construct	C	ASH
Function	`long add(long a, long b) { ... }`	`func add(int64 a, int64 b): int64 { ... }` (C order also accepted: `func int64 add(...)`)
Entry point	`int main(int argc, char **argv)`	`@Entry pub func main(int64 argc, int64 argv): int64`
Exported function	(default)	`pub func ...`
File-private function	`static long f(...)`	`priv func f(...)`
Global variable	`long counter = 0;`	`general static int64 counter = 0`
File-private global	`static long counter;`	`ungeneral static int64 counter`
Constant	`const int LIMIT = 10;` or `#define LIMIT 10`	`general int32 LIMIT = 10` (no `static`) or `define LIMIT 10;`
Conditional compilation	`#if`, `#ifdef`	`%if`, `%ifdef`
Include	`#include "x.h"`	`import "x.ash";` (definitions, not only declarations)
Else-if	`else if`	`elif` or `else if`
Counted loop	`for (i = 0; i < n; i++)`	the same, or `repeat (n) { }`
Infinite loop	`for (;;)` / `while (1)`	`loop { }`
Do-while	`do { } while (c);`	`loop { ...; if (!(c)) { break; } }`
Power	`pow(a, b)` (floating point)	`a ** b` (integer)
Array with length	`int a[4]; sizeof a / sizeof a[0]`	`int64[4] a; #a`
Struct	`struct P { int x; };`	`struct P { int32 x; };` or `typedef struct { ... } P;`
Pointer to struct field	`p->x`	`p->x`
Function pointer call	`fp(1, 2)`	`fp(1, 2)` where `fp` holds `&f`
Inline assembly	`__asm__("..." : ... )`	`asm { ... }` with variable names, or the same `asm("...")` form
Comments	`//`, `/* */`	`//`, `/* */`

G.2 WHAT ASH ADDS

- `repeat`, `loop`, `elif`, `terminate` (leave a function from any depth) and `unreturnable` (pending results).
- Arrays that know their length (`#a`) and bounds-checked indexing.
- Classes as namespaces with private methods.

Copyright © 2026 Mallionic Inc. All rights reserved.

- Cracks: configuration entry points inside functions.
- `**` and `***` operators with compile-time evaluation.
- `asm { }` blocks that use variables by name; `emit` for raw bytes; `@naked` functions; `runnable` executable arrays.
- One compiler for 16, 32 and 64-bit code, flat binaries and ELF/COFF objects; a built-in assembler.
- Annotations, including user-defined ones whose code is inserted into functions.
- Override rights to resolve duplicate definitions across files.
- A readable intermediate representation (ASHIR) for every program.

G.3 WHAT ASH DOES NOT HAVE (IN 1.0)

- Floating-point arithmetic (use `<math>` fixed point).
- `sizeof`, `?:`, `goto`, the comma operator, `do`/`while`, `union`, `enum` (use `define` constants), bit fields.
- Comparisons and logical operators as values (`x = a < b`).
- Block scope for local variables.
- Scaled pointer arithmetic: `p + 1` is the next byte; use `p[i]`.
- Header files: there are no separate declarations; `import` brings in the whole file.

G.4 SEMANTIC DIFFERENCES THAT MATTER

Table G-2. Behaviors that differ from C

Topic	ASH behavior
Uninitialized locals	Always zero.
Parameters	Assigning to a parameter creates a private copy (as in C), handled by the compiler.
Signed overflow	Wraps around (C leaves it undefined).
`char`	Unsigned 8-bit.
Undeclared function	Called as an external C function with integer arguments (C89 implicit declaration).
Global constants	Globals are constant unless declared `static`; assigning to them is a compile-time error.
Arrays as values	An array variable is the address of its elements (like a decayed C array) and also carries its length.
Local arrays and structures	Allocated on the heap, never freed automatically.
Duplicate definitions	Resolved by override rights instead of always being an error.

G.5 PORTING A C FUNCTION

The following C function and its ASH translation compute the length of the longest run of equal bytes in a buffer:

Copyright © 2026 Mallionic Inc. All rights reserved.

C

```c
size_t longest_run(const unsigned char *buf, size_t n) {
    size_t best = 0, cur = 0;
    for (size_t i = 0; i < n; i++) {
        if (i > 0 && buf[i] == buf[i - 1]) cur++;
        else cur = 1;
        if (cur > best) best = cur;
    }
    return best;
}
```

Example G-1. The same function in ASH

ASH

```
func longest_run(uint8* buf, uint64 n): uint64 {
    uint64 best = 0;
    uint64 cur = 0;
    for (uint64 i = 0; i < n; i++) {
        if (i > 0 && buf[i] == buf[i - 1]) {
            cur++;
        } else {
            cur = 1;
        }
        if (cur > best) {
            best = cur;
        }
    }
    return best;
}

@Entry
pub func main(): int64 {
    printf("%llu\n", longest_run("aabbbbcdddd", 11));
    return 0;
}
```

OUTPUT — EXIT STATUS 0

```
4
```

The translation changes only the declarations (types after names are written before, the result type after the parameters), adds the braces that ASH requires, and keeps everything else.

Copyright © 2026 Mallionic Inc. All rights reserved.

APPENDIX H

QUICK REFERENCE

A compact summary of the language, the command line and the libraries, for use while programming.

H.1 PROGRAM SKELETONS

```ash
// A program that uses the C library
@Entry
pub func main(int64 argc, int64 argv): int64 {
    printf("hello\n");
    return 0;
}
```

```ash
// A program without the C library:  ashc p.ash --elf --no-libc ; ld -e _start out/p.o -o p
import <syscalls>;

@Entry
pub func _start(): int64 {
    println("hello");
    sys_exit(0);
    return 0;
}
```

```ash
// A 16-bit flat image:  ashc p.ash --compile
@bits(16)
@originoffs(0x100)

@Entry
pub func _start(): int16 {
    return 42;
}
```

Copyright © 2026 Mallionic Inc. All rights reserved.

H.2 DECLARATIONS

Table H-1. Declarations at a glance

Form	Meaning
`int64 x = 5;`	Local variable.
`uint8[16] buf;` / `int64[] v = {1, 2};`	Arrays.
`int64[3][4] m;` / `matrix m[] []{ [int32 1], [int32 2], }`	Two-dimensional arrays.
`T* p = &x;`	Pointer.
`general static int64 g = 0` / `ungeneral int32 K = 7`	Globals (public / private; `static` = writable).
`runnable byte[] code = {0xC3}`	Executable array.
`typedef uint32 u32;` / `define N 10;` / `macro M { ... }`	Aliases and macros.
`struct S { int32 a; };` / `typedef struct { ... } S;`	Structures.
`extern int64 f(int64 x);`	External function.
`import <string>;` / `import "lib.ash";`	Imports.
`[pub|priv] func f(T a, ...): R { }`	Function (variadic with `...`).
`class C { pub func m() { } }`	Class; call `C.m()`.
`name -> (T p) { ... };` + `f.name = v;`	Crack.

Copyright © 2026 Mallionic Inc. All rights reserved.

H.3 STATEMENTS

Table H-2. Statements at a glance

Form	Meaning
`if (c) { } elif (c) { } else { }`	Conditional.
`while (c) { }` / `for (init; c; step) { }`	Loops.
`repeat (n) { }` / `loop { }`	Counted / infinite loops.
`switch (e) { case 1: ... break; default: ... }`	Multi-way branch (falls through).
`break;` `continue;`	Leave / restart a loop.
`return e;` `terminate e;` `terminate;`	Leave the function.
`unreturnable e;`	Set the pending result and continue.
`asm { ... }` / `asm volatile("... {x} ...");`	Inline assembly.
`emit(0x90);` `emit16(..)` `emit32(..)` `emit64(..)`	Raw machine bytes.

H.4 OPERATORS

Table H-3. Operators from highest to lowest precedence

Operators	Notes
`() [] . -> x++ x--`	Postfix.
`++x --x - + ~ & * # (T)`	Unary, right to left.
`** ***`	Power and repeated squaring, right to left.
`* / %`	Multiplicative.
`+ -`	Additive.
`<< >>`	Shifts.
`== != < <= > >=`	Comparisons (conditions only).
&, then ^, then |	Bitwise.
`! && ||`	Logical (conditions only).

Copyright © 2026 Mallionic Inc. All rights reserved.

H.5 ANNOTATIONS

Table H-4. Annotations at a glance

Annotation	Meaning
`@Entry`	Program entry point.
`@bits(16|32|64)`, `@originoffs(a)`	Target mode, load address.
`@beautycode(b)`, `@bounds(b)`, `@strict`, `@nolibc`	Code style, bounds checks, strict mode, no libc.
`@naked`, `@msabi` / `@efiapi`	No prologue; Microsoft convention.
`@section(".x", "flags")`, `@attribute(section(".x"))`	Section placement.
`@ovr[N]` / `@override`	Override right.
`@Name(T p) { ... }`	User-defined annotation.

H.6 COMMAND LINE

Table H-5. ashc options at a glance

Command	Result
`ashc p.ash`	`p.ashir`
`ashc p.ash --elf` / `--win`	`out/p.o` / `out/p.obj`
`ashc p.ash --compile` / `--exe`	`out/p.bin` / `out/p` (32-bit)
`--bits N`, `--no-libc`, `-e`	Width, no libc, frames everywhere.
`-o file`, `-o dir/`, `--outdir dir`, `--stdout`	Output location.
`-s`, `--msg fmt`, `--tokens`, `--debug`, `--trace`	Messages and debugging.
`--list-nlib`, `--version`, `--license`, `--help`	Information.

Copyright © 2026 Mallionic Inc. All rights reserved.

H.7 LIBRARY FUNCTIONS

Table H-6. nlib functions at a glance

Library	Functions
<string>	memset memcpy memmove memcmp memchr strlen strnlen strcpy strncpy strcat strncat strcmp strncmp strcasecmp strchr strrchr strstr strspn strcspn isdigit isupper islower isalpha isalnum isspace isxdigit toupper tolower str_to_int atoi uint_to_str int_to_str str_reverse str_upper str_lower str_equals str_starts str_ends
<math>	abs min max clamp sign gcd lcm ipow isqrt ilog2 ilog10 factorial is_prime mod_pow div_floor mod_floor is_pow2 next_pow2 popcount align_up align_down fx fx_int fx_round fx_frac fx_abs fx_mul fx_div fx_sqrt fx_deg fx_sin fx_cos fx_tan fx_print_buf
<syscalls>	syscall6 sys_read sys_write sys_open sys_close sys_lseek sys_mmap sys_munmap sys_brk sys_getpid sys_kill sys_mkdir sys_unlink sys_nanosleep sys_exit print println print_int
<args>	arg_at arg_equals arg_int arg_has
<delay>	delay delayus
<types>	inf ninf pos_inf neg_inf float_nan float_inf float_ninf double_nan double_inf double_ninf nan

Copyright © 2026 Mallionic Inc. All rights reserved.

INDEX

!

! . . . 133

#

. . . 158, 519

%

%define . . . 271

%if

%define and other directives . . . 541

%macro . . . 271

%rep . . . 271

&

& . . . 519

&& . . . 133

&& || ! . . . 524

*

* . . . 520

** . . . 127, 520

*** . . . 127, 521

+

++ and -- . . . 521

+= -= *= /= %= . . . 521

-

--bits . . . 76

--compile . . . 228

--epilogue . . . 76

--exe . . . 227

--msg . . . 77

--no-libc . . . 76, 225

--outdir . . . 75

--tokens . . . 78

-> . . . 165

-> and . . . 522

Copyright © 2026 Mallionic Inc. All rights reserved.

-o . . . 75

.

... . . . 522

0

0xB8000 . . . 490

<

<< . . . 128
<< and >> . . . 523
. . . 438
. . . 441
. . . 354
. . . 457
. . . 457
. . . 443
. . . 449
. . . 279
. . . 408
. . . 451

=

== != < <= > >= . . . 523

>

>> . . . 128

@

@ . . . 176, 524
@beautycode . . . 184
@bits . . . 106, 180
@bounds . . . 185
@efiapi . . . 189
@Entry . . . 153, 178
@msabi . . . 189
@naked . . . 188
@nolibc . . . 187
@originoffs . . . 183
@override . . . 191
@ovr . . . 191, 206
@section . . . 190

Copyright © 2026 Mallionic Inc. All rights reserved.

@strict . . . 186

A

ABI . . . 145, 224
abs . . . 355
abs () . . . 355
adc . . . 552
add . . . 551
addition . . . 125
addps . . . 596
addsd . . . 595
addss . . . 595
alignment . . . 163
align_down . . . 386
align_down () . . . 386
align_up . . . 385
align_up () . . . 385
and . . . 129, 559
andps . . . 598
annotations . . . 176
architecture . . . 82
argc . . . 153, 438
argv . . . 153, 438
arg_at . . . 439
arg_at () . . . 439
arg_equals . . . 440
arg_equals () . . . 440
arg_has . . . 440
arg_has () . . . 440
arg_int . . . 440
arg_int () . . . 440
arpl . . . 586
array layout . . . 156
arrays . . . 155
ASH . . . 28
ashc . . . 73
ASHIR . . . 233
ASH_BITS . . . 198
ASH_FORMAT . . . 198
asm . . . 210, 525
asm variables . . . 210

Copyright © 2026 Mallionic Inc. All rights reserved.

asm volatile . . . 213
AsmBlocks . . . 84
AsmBridge . . . 87
assembler . . . 267
assignment . . . 115
AST . . . 85
atoi . . . 336
atoi () . . . 336
attribute byte . . . 490
attributes . . . 176

B

Backend32 . . . 258
bare metal . . . 490
basics . . . 36
beginners . . . 36
binary . . . 36
binary literal . . . 94
bit . . . 36
bitwise operators . . . 129
bool . . . 106
true
false . . . 525
boot sector . . . 228, 275
bounds checking . . . 158, 185
break . . . 138, 525
bsf . . . 571
bsr . . . 572
bswap . . . 546
bt . . . 571
btc . . . 571
btr . . . 571
bts . . . 571
buffer . . . 161
build.sh . . . 49
byte . . . 36

C

C programmers . . . 622
call . . . 574
callback . . . 109, 150

Copyright © 2026 Mallionic Inc. All rights reserved.

calling convention . . . 145
case . . . 138
case and default . . . 526
cast . . . 110
cbw . . . 549
cdq . . . 549
cdqe . . . 549
char . . . 107, 526
character literal . . . 95
char[] . . . 162
clamp . . . 358
clamp () . . . 358
class . . . 168, 527
clc . . . 576
cld . . . 577
clflush . . . 580
cli . . . 577
clts . . . 586
cmc . . . 577
cmovcc . . . 576
cmp . . . 556
cmpsb . . . 581
cmpxchg . . . 545
cmpxchg8b . . . 546
code generation . . . 86, 248
COFF . . . 87, 229
comisd . . . 598
command line . . . 73
comments . . . 92
common mistakes . . . 502
comparison . . . 130
compiler . . . 41
compiler pipeline . . . 82
compound assignment . . . 98
condition . . . 133
constant folding . . . 131
constraints . . . 213
continue . . . 138, 527
control flow . . . 132
cookbook . . . 472

Copyright © 2026 Mallionic Inc. All rights reserved.

cpuid . . . 579
cqo . . . 549
crack . . . 171
crc32 . . . 573
cvtsd2ss . . . 597
cvtsi2sd . . . 597
cvtss2sd . . . 597
cvttsd2si . . . 597
cwd . . . 549
cwde . . . 550

D

dec . . . 555
default . . . 138
define . . . 201, 528
delay . . . 441
delay () . . . 441
delayus . . . 442
delayus () . . . 442
diagnostics . . . 79, 459
dictionary . . . 519
Directives . . . 84
directives . . . 197
div . . . 558
division . . . 125
divsd . . . 596
div_floor . . . 377
div_floor () . . . 377
double . . . 110
double_inf . . . 455
double_inf () . . . 455
double_nan . . . 454
double_nan () . . . 454
double_ninf . . . 456
double_ninf () . . . 456

E

EFI . . . 231
ELF . . . 87
ELF section . . . 190
elif . . . 134

Copyright © 2026 Mallionic Inc. All rights reserved.

elif and else . . . 528
else . . . 134
emit . . . 214, 529
emms . . . 602
encoding . . . 264
enter . . . 575
entry point . . . 52, 153
epilogue . . . 147
errata . . . 616
error . . . 79
error messages . . . 459
escape sequence . . . 95
examples . . . 472
examples directory . . . 613
exit_group . . . 225
expressions . . . 123
extern . . . 152, 529

F

fabs . . . 590
factorial . . . 372
factorial () . . . 372
fadd . . . 589
faddr . . . 150
false . . . 106
FAQ . . . 502
fchs . . . 590
fcom . . . 591
fcomi . . . 591
fcos . . . 591
fdiv . . . 590
fild . . . 588
finit . . . 591
fistp . . . 588
fixed point . . . 354
flat binary . . . 228
fld . . . 587
fld1 . . . 592
fldpi . . . 592
fldz . . . 592
float . . . 110

Copyright © 2026 Mallionic Inc. All rights reserved.

floating point . . . 95
float_inf . . . 453
float_inf () . . . 453
float_nan . . . 453
float_nan () . . . 453
float_ninf . . . 454
float_ninf () . . . 454
fmul . . . 589
fopen . . . 443
for . . . 136, 530
fsin . . . 590
fsqrt . . . 590
fst . . . 588
fstp . . . 588
fsub . . . 589
func . . . 530
function pointer . . . 109, 150
functions . . . 142
fwait . . . 592
fx . . . 387
fx () . . . 387
fxch . . . 591
fx_abs . . . 392
fx_abs () . . . 392
fx_cos . . . 401
fx_cos () . . . 401
fx_deg . . . 397
fx_deg () . . . 397
fx_div . . . 394
fx_div () . . . 394
fx_frac . . . 391
fx_frac () . . . 391
fx_int . . . 388
fx_int () . . . 388
fx_mul . . . 393
fx_mul () . . . 393
fx_print_buf . . . 404
fx_print_buf () . . . 404
fx_round . . . 389
fx_round () . . . 389

Copyright © 2026 Mallionic Inc. All rights reserved.

fx_sin . . . 398
fx_sin () . . . 398
fx_sqrt . . . 396
fx_sqrt () . . . 396
fx_tan . . . 402
fx_tan () . . . 402

G

GCC syntax . . . 213
gcd . . . 361
gcd () . . . 361
general . . . 117
general and ungeneral . . . 530
Generator . . . 86
global array . . . 120
global variable . . . 117
glossary . . . 622
GraalVM . . . 31
grammar . . . 610

H

heap . . . 156
hello world . . . 50
hexadecimal . . . 36, 94
hlt . . . 578

I

i386 code . . . 258
identifier . . . 92
idiv . . . 558
IEEE-754 . . . 110
if . . . 134, 531
ilog10 . . . 370
ilog10 () . . . 370
ilog2 . . . 368
ilog2 () . . . 368
import . . . 204, 531
imul . . . 557
in . . . 582
inc . . . 555
indexing . . . 156

Copyright © 2026 Mallionic Inc. All rights reserved.

inf . . . 451
inf () . . . 451
infinity . . . 451
inline assembly . . . 210
insb . . . 582
instruction encoding . . . 261
instruction reference . . . 542
int . . . 575
int3 . . . 575
int64 . . . 100
int8 ... uint64 . . . 532
integer literal . . . 94
integer types . . . 100
intermediate representation . . . 233
internal compiler error . . . 78
interoperability . . . 224
into . . . 575
int_to_str . . . 340
int_to_str () . . . 340
invlpg . . . 585
ipow . . . 365
ipow () . . . 365
iret . . . 576
isalnum . . . 325
isalnum () . . . 325
isalpha . . . 323
isalpha () . . . 323
isdigit . . . 319
isdigit () . . . 319
islower . . . 322
islower () . . . 322
isqrt . . . 366
isqrt () . . . 366
isspace . . . 326
isspace () . . . 326
isupper . . . 321
isupper () . . . 321
isxdigit . . . 327
isxdigit () . . . 327
is_pow2 . . . 381

Copyright © 2026 Mallionic Inc. All rights reserved.

is_pow2 () . . . 381
is_prime . . . 373
is_prime () . . . 373

J

Java . . . 31
jcc . . . 573
jcxz . . . 574
jmp . . . 573

K

keywords . . . 93

L

lahf . . . 578
lar . . . 586
lcm . . . 363
lcm () . . . 363
ldmxcsr . . . 601
lea . . . 548
leaf function . . . 147, 184
leave . . . 575
len
length
rows
cols
count . . . 532
length . . . 158
lexer . . . 78, 85
lexical structure . . . 91
lfence . . . 580
lgdt . . . 583
library . . . 278
lidt . . . 584
limitations . . . 616
linker . . . 41
linter . . . 468
literal type . . . 105
lldt . . . 584
lmsw . . . 586
lodsb . . . 581

Copyright © 2026 Mallionic Inc. All rights reserved.

long long . . . 97
loop . . . 138, 533, 574
lsl . . . 587
ltr . . . 584
lzcnt . . . 572

M

machine code . . . 248
machine code bytes . . . 214
macro . . . 533
macros . . . 197
main . . . 52
matrix . . . 160, 534
max . . . 357
max () . . . 357
member access . . . 165
memchr . . . 287
memchr () . . . 287
memcmp . . . 285
memcmp () . . . 285
memcpy . . . 280
memcpy () . . . 280
memmove . . . 282
memmove () . . . 282
memset . . . 279
memset () . . . 279
mfence . . . 580
Microsoft x64 ABI . . . 229
min . . . 356
min () . . . 356
mmap . . . 225
ModRM . . . 264
modules . . . 204
modulo . . . 125
mod_floor . . . 379
mod_floor () . . . 379
mod_pow . . . 375
mod_pow () . . . 375
mov . . . 544
movaps . . . 593
movbe . . . 546

Copyright © 2026 Mallionic Inc. All rights reserved.

movd . . . 594
movdqa . . . 593
movdqu . . . 593
movq . . . 595
movsb . . . 581
movsd . . . 594
movss . . . 594
movsx . . . 102, 545
movsxd . . . 545
movups . . . 593
movzx . . . 102, 544
mul . . . 557
mulps . . . 597
mulsd . . . 596
multi-line string . . . 96
multiplication . . . 125

N

name mangling . . . 169
NaN . . . 451
nan . . . 456
nan () . . . 456
native-image . . . 31
neg . . . 555
neg_inf . . . 452
neg_inf () . . . 452
next_pow2 . . . 382
next_pow2 () . . . 382
ninf . . . 451
ninf () . . . 451
nlib . . . 278
nop . . . 578
not . . . 129, 562
NULL . . . 457

O

opcode . . . 264
operators . . . 97, 123
or . . . 129, 560
orps . . . 598
out . . . 582

Copyright © 2026 Mallionic Inc. All rights reserved.

output mode . . . 74
outsb . . . 582
overflow . . . 104
override . . . 206

P

paddb . . . 599
paddd . . . 600
padding . . . 163
paddq . . . 600
pand . . . 599
parameters . . . 143
parser . . . 85
pause . . . 578
pcmpeqb . . . 600
pending return . . . 173
pmovmskb . . . 601
pointer . . . 108
pop . . . 548
popcnt . . . 572
popcount . . . 383
popcount () . . . 383
popf . . . 548
por . . . 599
pos_inf . . . 452
pos_inf () . . . 452
power operator . . . 127
precedence . . . 123
predefined symbols . . . 198
prefetcht0 . . . 580
preprocessor . . . 197
print . . . 432
print () . . . 432
printf . . . 443
println . . . 434
println () . . . 434
print_int . . . 435
print_int () . . . 435
priv . . . 168
priv function . . . 170
prologue . . . 147

Copyright © 2026 Mallionic Inc. All rights reserved.

pshufd . . . 601
psubd . . . 600
ptr . . . 108
pub . . . 168
pub and priv . . . 534
push . . . 547
pushf . . . 548
pxor . . . 599

Q

Q16.16 . . . 354
quick reference . . . 626

R

rcl . . . 569
rcr . . . 570
rdmsr . . . 585
rdpmc . . . 585
rdrand . . . 579
rdseed . . . 579
rdtsc . . . 579
rdtscp . . . 579
redefine . . . 202, 534
register allocation . . . 115
registers . . . 261, 603
relocation . . . 87
repeat . . . 136, 535
ret . . . 574
return . . . 140, 535
REX . . . 264
rights . . . 206
rol . . . 567
ror . . . 568
runnable . . . 121, 536
runtime . . . 225
runtime helpers . . . 86, 240

S

sahf . . . 578
sal . . . 564
sar . . . 566

Copyright © 2026 Mallionic Inc. All rights reserved.

sbb . . . 554
scasb . . . 581
scope . . . 114
SDK . . . 49
setcc . . . 576
sfence . . . 580
sgdt . . . 584
shadow space . . . 229
shasm . . . 87, 267
shift . . . 128
shl . . . 563
shld . . . 570
shr . . . 565
shrd . . . 570
shufps . . . 601
SIB . . . 264
sidt . . . 584
sign . . . 360
sign () . . . 360
sign extension . . . 102
smsw . . . 586
snippets . . . 468
source code . . . 41
sqrtsd . . . 596
stack frame . . . 76
statements . . . 132
static . . . 117, 536
stc . . . 576
std . . . 577
sti . . . 577
stmxcsr . . . 602
stosb . . . 581
strcasecmp . . . 306
strcasecmp () . . . 306
strcat . . . 296
strcat () . . . 296
strchr . . . 308
strchr () . . . 308
strcmp . . . 301
strcmp () . . . 301

Copyright © 2026 Mallionic Inc. All rights reserved.

strcpy . . . 292
strcpy () . . . 292
strcspn . . . 317
strcspn () . . . 317
strict mode . . . 220
string . . . 536
string array . . . 161
string literal . . . 96
string type . . . 107
strlen . . . 289
strlen () . . . 289
strncat . . . 299
strncat () . . . 299
strncmp . . . 304
strncmp () . . . 304
strncpy . . . 294
strncpy () . . . 294
strnlen . . . 290
strnlen () . . . 290
strrchr . . . 311
strrchr () . . . 311
strspn . . . 315
strspn () . . . 315
strstr . . . 313
strstr () . . . 313
struct . . . 163
struct and typedef . . . 537
structure layout . . . 163
structures . . . 163
str_ends . . . 352
str_ends () . . . 352
str_equals . . . 349
str_equals () . . . 349
str_lower . . . 347
str_lower () . . . 347
str_reverse . . . 342
str_reverse () . . . 342
str_starts . . . 351
str_starts () . . . 351
str_to_int . . . 332

Copyright © 2026 Mallionic Inc. All rights reserved.

str_to_int () . . . 332
str_upper . . . 345
str_upper () . . . 345
sub . . . 553
subsd . . . 596
swapgs . . . 586
switch . . . 138, 537
syscall . . . 583
syscall6 . . . 408
syscall6 () . . . 408
sysenter . . . 583
sysexit . . . 583
sysret . . . 583
system call . . . 408
System V . . . 145
sys_brk . . . 421
sys_brk () . . . 421
sys_close . . . 415
sys_close () . . . 415
sys_exit . . . 431
sys_exit () . . . 431
sys_getpid . . . 422
sys_getpid () . . . 422
sys_kill . . . 424
sys_kill () . . . 424
sys_lseek . . . 416
sys_lseek () . . . 416
sys_mkdir . . . 425
sys_mkdir () . . . 425
sys_mmap . . . 418
sys_mmap () . . . 418
sys_munmap . . . 419
sys_munmap () . . . 419
sys_nanosleep . . . 428
sys_nanosleep () . . . 428
sys_open . . . 413
sys_open () . . . 413
sys_read . . . 410
sys_read () . . . 410
sys_unlink . . . 427

Copyright © 2026 Mallionic Inc. All rights reserved.

sys_unlink () . . . 427
sys_write . . . 412
sys_write () . . . 412

T

target width . . . 106
targets . . . 224
techniques . . . 502
temporaries . . . 124
terminate . . . 140, 538
test . . . 562
text mode . . . 490
The Ashley Project . . . 28
three-address code . . . 124
tip . . . 79
tokens . . . 91
tolower . . . 331
tolower () . . . 331
tooling . . . 468
toupper . . . 329
toupper () . . . 329
tour . . . 61
tree shaking . . . 205
true . . . 106
truncation . . . 104
tutorial . . . 61
two-dimensional array . . . 159
typedef . . . 111
types . . . 100
tzcnt . . . 572

U

ucomisd . . . 598
UCS-2 . . . 96
ud2 . . . 579
UEFI . . . 96, 189, 231
uint8 . . . 100
uint_to_str . . . 337
uint_to_str () . . . 337
ungeneral . . . 117
unpcklps . . . 601

Copyright © 2026 Mallionic Inc. All rights reserved.

unreturnable . . . 171, 173, 538
unsigned . . . 97
Untokenizer . . . 85
use strict . . . 220, 539
user annotation . . . 193

V

variables . . . 113
variadic . . . 148
va_arg . . . 148
va_count . . . 148
va_count and va_arg . . . 539
verr . . . 587
verw . . . 587
VGA . . . 490
visibility . . . 168
void . . . 539
void* . . . 108
volatile . . . 116, 540
VS Code . . . 60, 468

W

warning . . . 79
wbinvd . . . 585
while . . . 134, 540
wide string . . . 96
Windows . . . 229
wrmsr . . . 585

X

x86 . . . 261
xadd . . . 545
xchg . . . 545
xlatb . . . 550
xor . . . 129, 561
xorps . . . 598

Z

zero extension . . . 102

_

_start . . . 52

Copyright © 2026 Mallionic Inc. All rights reserved.

__ash_alloc 240
__ash_bounds 158, 240

|

|| 133

Copyright © 2026 Mallionic Inc. All rights reserved.